

Notes

Trevor Vaughn

September 28, 2026

*“There are more things in heaven and earth, Horatio,
Than are dreamt of in your philosophy.”*

—William Shakespeare, *Hamlet*

Contents

1	Foreword	9
2	Analysis and Asymptotics	9
2.1	Stolz–Cesàro	9
2.2	Wallis Product	10
2.3	Stirling’s Formula	11
2.4	Banach Fixed Point Theorem	13
2.5	Inverse Function Theorem	14
2.6	Implicit Function Theorem	16
2.7	Lagrange Multipliers	17
2.8	Karamata’s Inequality	18
2.9	The Isoperimetric Inequality	20
3	Linear Algebra	28
3.1	Determinant and Inverse Identities	28
3.1.1	Woodbury Identity	30
3.1.2	Matrix Determinant Lemma	31
3.2	Characteristic Polynomial	32
3.2.1	Determinant of Affine Matrix Polynomial	33
3.3	Frobenius Form	33
3.3.1	Algorithm Description	33
3.3.2	Correctness	34
3.3.3	Failure Probability	34
3.3.4	Polynomial Matrix Composition and Powering	35
3.4	Black Box Linear Algebra	36
3.4.1	Minimal Polynomial	36
3.4.2	Determinant	37
3.4.3	Wiedemann’s Algorithm	38
3.5	Perron-Frobenius Theorem	39
3.6	Continuity of Eigenvalues	40
3.7	Gershgorin’s Circle Theorem	41
4	Number Theory and Arithmetic	44
4.1	Bertrand’s Postulate	44
4.2	Product of Small Primes	46
4.3	Lucas’s Theorem	48
4.4	Quadratic Reciprocity	49
4.5	Values of the Zeta Function at Even Integers	52
4.6	Arithmetic Algorithms	54
4.6.1	Linear Sieve	54
4.6.2	Extended GCD with Preprocessing	57
4.6.3	Modular Square Roots	60
4.6.4	Generalized Floor Sum	62
4.6.5	Multiplicative Function Prefix Sums	65
4.6.6	LCM and GCD Convolution	67
4.7	Concave Min-Plus Convolution	68
4.7.1	Description	68
4.7.2	Crossings	69
4.7.3	Boundary Conditions	69
4.8	Fast Factorials and Deterministic Factorization	69

4.8.1	Fast Factorial	70
4.8.2	Deterministic Integer Factorization	72
4.9	Miller–Rabin Primality Test	75
5	Combinatorics	81
5.1	Entropy Method	81
5.1.1	Shearer’s Lemma	81
5.1.2	Entropy Bound for Binomial Sums	81
5.1.3	Irregular Moore Bound	82
5.1.4	Permanent Bound	85
5.2	Additive Combinatorics	86
5.2.1	Kneser’s Addition Theorem	86
5.3	Extremal Set Theory	89
5.3.1	Bollobás Two Families Theorem	89
5.3.2	Sunflower Lemma	90
5.4	Ramsey Theory	96
5.4.1	Hales–Jewett Theorem	96
5.5	Polynomial Method	99
5.5.1	Combinatorial Nullstellensatz	99
5.5.2	Multiplicity Schwartz–Zippel	105
5.5.3	Multilinear-Term Detection	106
5.6	Probabilistic Method	110
5.6.1	Isolation Lemma	110
5.7	Concentration Inequalities	111
5.7.1	Hoeffding	111
5.7.2	Chernoff	113
5.7.3	Bernstein	113
5.8	Martingales	114
5.8.1	Azuma and McDiarmid	115
5.8.2	Freedman’s Inequality	117
5.9	Enumerative and Algebraic Combinatorics	119
5.9.1	Lindström–Gessel–Viennot Lemma	119
5.9.2	Q-Analogs	120
5.9.3	Jacobi Triple Product	122
5.9.4	MacMahon’s Master Theorem	123
5.9.5	Hook-Length Formula	124
5.9.6	Robinson–Schensted Correspondence	129
6	Convexity and Duality	130
6.1	Sion’s Minimax Theorem	130
6.2	Linear Programming Duality	134
6.3	Majorization	136
6.3.1	Muirhead’s Inequality	137
6.4	Radon’s Theorem	138
6.5	Helly’s Theorem	139
6.6	Carathéodory’s Theorem	139
6.7	Colorful Carathéodory	140
6.8	Tverberg’s Theorem	141
6.9	Matroid Intersection	143
7	Complex Analysis	147
7.1	Cauchy’s Theorem	147

7.2	Cauchy's Integral Formula	149
7.3	Laurent Series	150
7.4	Residue Theorem	152
7.5	Cotangent Partial Fractions	153
7.6	Euler's Product for Sine	155
8	Data Structures	156
8.1	RMQ	156
8.2	DSU	157
8.2.1	Rank Facts	157
8.2.2	Ackermann Levels	158
8.2.3	Potential	158
8.2.4	Why a Find Costs $O(\alpha)$ Amortized	159
8.3	DSU with Potential	160
8.4	Persistent Queue	160
8.5	Online Suffix Max	161
8.6	Mo's Algorithm	162
8.7	A Fast Succinct Hash Table	164
8.7.1	Small Buckets	164
8.7.2	Storing the Full Keys Succinctly	165
8.7.3	Local Fingerprints	165
8.7.4	A Constant-Time Dictionary Inside a Bucket	166
8.7.5	Cost of Fingerprint Rebuilding	167
8.7.6	Deamortizing Rebuilds	167
8.7.7	The Update Buffers Are Succinct	169
8.7.8	Dynamic Resizing	169
8.7.9	Summary	169
8.8	Simulating Fully Random Hashing	169
8.8.1	Private Random Cells	170
8.8.2	Splitting a Large Set into Small Fixed Sets	171
8.8.3	Randomize After Splitting	172
9	Trees	173
9.1	LCA	173
9.2	$O(n)$ Binary Lifting	173
9.3	Virtual Tree	176
9.4	XOR-Linked Traversal	177
9.5	DFS	178
9.6	Diameter	178
9.7	Shallowest Decomposition Tree	179
9.8	Static Top Tree	181
9.8.1	The Four Operations	182
9.8.2	Heavy Paths	183
9.8.3	The Expression Tree	183
9.8.4	Weight-Balanced Merging	184
9.8.5	The DP Interface	186
9.8.6	Point Updates	186
9.8.7	Rooted-Subtree Queries	187
9.8.8	Path Queries	187
9.9	Splay Tree	190
9.9.1	Rotation	190
9.9.2	Splaying	190

9.9.3	Access Lemma	191
9.9.4	Augmentation	193
9.10	Link-Cut Tree	193
9.10.1	Preferred Paths	193
9.10.2	How a Preferred Path Is Stored	194
9.10.3	Connecting the Auxiliary Trees	195
9.10.4	Access	195
9.10.5	Why Access Is Fast	196
9.10.6	The Analysis Tree	197
9.10.7	How Many Auxiliary Trees Does Access Cross?	198
9.10.8	Rerooting	199
9.10.9	Dynamic-Forest Operations	199
9.10.10	Path Queries and Updates	199
9.11	Top Tree	200
9.11.1	Real and Fake Nodes	201
9.11.2	The Same Path/Point Interface Reappears	201
9.11.3	Access with Virtual-Child Trees	202
9.11.4	Aggregates	202
9.11.5	Two Kinds of Lazy Tags	204
9.11.6	Push Before Changing the Decomposition	204
9.11.7	Path Operations	205
9.11.8	Subtree Operations	205
9.11.9	Link, Cut, and Reroot	206
9.11.10	Amortized Analysis	206
9.12	Lazy Anti-Monopoly Tree	207
10	Graph Algorithms	210
10.1	Incremental SCCs	210
10.2	All-Pairs Connectivity	212
10.2.1	A Walk Generating Matrix	213
10.2.2	Determinants Enumerate Edge-Disjoint Walks	213
10.2.3	Connectivity is Matrix Rank	214
10.2.4	Random Evaluation	215
10.2.5	Bounded All-Pairs Connectivity	216
10.3	Flows	220
10.3.1	Highest Label Push-Relabel	220
10.3.2	Flow Rounding	221
10.4	Arboricity	222
10.5	Directed Minimum Spanning Tree	224
10.5.1	The Two Reductions	224
10.5.2	Meldable Incoming-Edge Heaps	225
10.5.3	Detecting Cycles with DSU	226
10.5.4	Maintaining Contractions	226
10.5.5	Fast Directed MST Algorithm	226
10.5.6	Correctness	227
10.5.7	Rollback Reconstruction	228
10.5.8	Running Time	228
10.6	Matching and Coloring	228
10.6.1	Hall's Marriage Theorem	228
10.6.2	Gale–Ryser Theorem	229
10.6.3	Bipartite Regular Matching	230
10.6.4	Bipartite Edge Coloring	231

10.6.5	Chromatic Number	235
10.7	Node-Weighted Triangle	237
10.7.1	One Fixed Weight	237
10.7.2	The Square Case	238
10.7.3	Rectangular Version	240
10.7.4	Minimum Node-Weighted Triangle	242
10.8	An Algebraic Criterion for Bipartite Perfect Matching	244
10.8.1	The Wronskian as a Multiplicity Budget	244
10.8.2	Designing the Polynomial System	246
10.8.3	Hall Deficiency Leaves Too Much Freedom	247
10.8.4	Private Zeros Force the Active Polynomials to be Independent	248
10.8.5	Hall Expansion Exhausts the Remaining Multiplicity Budget	249
10.8.6	The Resulting Algorithm	250
10.9	Shortest Paths	253
10.9.1	Eppstein's Algorithm	253
10.9.2	Bellman–Ford–Dijkstra	254
11	Algebraic and Spectral Graph Theory	256
11.1	The Matrix–Tree Theorem	256
11.1.1	Counting Hamiltonian Paths	257
11.1.2	Counting k -Matchings in a Bipartite Graph	258
11.2	Cheeger Inequality	260
12	Electrical Flows	263
12.1	Electrical Flows	263
12.1.1	Potentials and Ohm's Law	264
12.1.2	Effective Resistance	264
12.1.3	Maximum Principle	265
12.1.4	Dirichlet's Principle	265
12.1.5	Series and Parallel Laws	267
12.1.6	Rayleigh Monotonicity	268
12.1.7	Effective Resistance is a Metric	268
12.2	Random Walks and Electrical Networks	269
12.2.1	Voltages are Hitting Probabilities	269
12.2.2	Currents are Expected Net Traversals	270
12.2.3	Hitting a Vertex Through a Given Edge	270
12.2.4	Hitting Times	271
12.2.5	Commute Time	271
12.2.6	Tetali's Hitting-Time Formula	272
12.2.7	Cover-Time Bound	272
12.3	Kron Reduction	273
12.4	Infinite Graphs, Recurrence, and Transience	274
12.4.1	Infinite Thomson Principle	274
12.4.2	Escape Probability	275
12.4.3	Nash–Williams Recurrence Criterion	276
12.4.4	Trees	276
12.5	Random Spanning Trees	277
12.5.1	Weighted Random Spanning Trees	277
12.6	Localization of Electrical Flows	279
12.6.1	Replacing L^+ by a Continuous-Time Walk	280
12.6.2	Splitting the Walk at its Midpoint	282
12.6.3	Turning Edge Differences into a Time Derivative	283

12.6.4	Choosing the Stationary Distribution	287
12.6.5	Integrating the Dissipation	287
12.6.6	Completing the Localization Bound	289
13	Polynomial Algorithms	290
13.1	Formal Power Series Operations	290
13.1.1	Inverse	290
13.1.2	Logarithm	290
13.1.3	Square Root	290
13.1.4	Inverse Square Root	291
13.1.5	Exponential	291
13.1.6	Power	292
13.2	Polynomial Arithmetic	293
13.2.1	Division	293
13.2.2	Shift	294
13.2.3	Evaluation Shift	295
13.2.4	GCD	296
13.2.5	Modular Inversion	300
13.2.6	Root Finding	301
13.2.7	Minimum Linear Recurrence	303
13.3	Rational Functions and Transposition	307
13.3.1	Bostan-Mori	307
13.3.2	Transposition Principle	308
13.3.3	Transposed Multiplication	309
13.3.4	Transposed NTT and INTT	310
13.3.5	Transposed Bostan-Mori	311
13.4	Composition and Inversion	312
13.4.1	Power Projection	312
13.4.2	Composition	314
13.4.3	Lagrange Inversion	316
13.4.4	Compositional Inverse	318
13.5	Evaluation and Interpolation	319
13.5.1	Multipoint Evaluation	319
13.5.2	Chirp-Z	320
13.5.3	Interpolation	321
13.5.4	Inverse Chirp-Z	322
13.5.5	Hermite Interpolation	324
13.5.6	Multivariate Evaluation	329
14	Set Power Series	337
14.1	Subset Convolution	337
14.2	Transposed Subset Convolution	338
14.3	Inversion and Quotient	338
14.4	Exponential and Logarithm	340
14.5	Composition	341
14.6	Power Projection	343
14.7	Power	344
15	Matrix Analysis	345
15.1	Sampling Matrices without Replacement	345
15.2	Schur and Kronecker Product	346
15.3	Differentiating Matrix Functions	347

15.3.1	Product, Inverse, and Power Rules	347
15.3.2	Divided Differences	348
15.3.3	Traces Recover the Scalar Derivative Rule	349
15.3.4	The Matrix Exponential	351
15.4	Integral Representations and Fractional Powers	352
15.4.1	The Logarithm	352
15.4.2	Fractional Powers	353
15.4.3	Operator Convexity of Inversion	354
15.4.4	Löwner–Heinz and Operator Concavity of Powers	355
15.5	Golden–Thompson Inequality	356
15.6	Trace Rearrangement and Schatten Hölder Inequalities	358
15.7	Generalized Klein Inequality	361
15.8	Wielandt Minimax Formula	361
15.9	The Operator Jensen Inequality	363
15.10	Lieb’s Concavity Theorem	364
15.10.1	Joint Convexity of Matrix Relative Entropy	367
15.10.2	A Variational Formula for the Trace	367
15.10.3	Lieb’s Trace-Exponential Theorem	368
15.11	Lieb–Thirring Inequalities	368
15.12	Matrix Concentration	371
16	Convex Optimization	378
16.1	Linear Systems, Richardson Iteration, and Preconditioning	378
16.1.1	Richardson Iteration	378
16.1.2	Preconditioning	379
16.1.3	Chebyshev Acceleration: Optimize the Whole Iteration	380
16.2	Smoothness, Strong Convexity, and Euclidean Gradient Methods	383
16.2.1	Smooth Gradient Descent	383
16.2.2	Subgradient Descent and Averaging	384
16.2.3	Equivalent Curvature Conditions	385
16.3	Geometry Beyond Euclidean Space	386
16.3.1	Dual Norms and the Steepest Direction	386
16.3.2	Convergence from a Sublevel Radius	387
16.3.3	Gradient and Hessian Characterizations	387
16.3.4	The Squared ℓ_p Norm	388
16.4	Bregman Geometry and Relative Curvature	389
16.4.1	Comparing Curvature to a Reference Function	389
16.4.2	Proximal Steps and the Three-Point Identity	389
16.4.3	Relative Gradient Descent	390
16.5	Mirror Descent, Entropy, and Multiplicative Weights	391
16.5.1	Why Entropy Fits the Simplex	392
16.5.2	The Multiplicative Update	393
16.6	Matrix Multiplicative Weights from Entropy Geometry	393
16.6.1	Entropy Determines the Update	394
16.6.2	Potential Analysis	395
16.7	Under Construction	396

1 Foreword

The goal of these notes is primarily to explain interesting things. I do not claim to have invented anything herein, although some I have simplified or generalized. Some of the algorithms contained herein have been implemented in [my library](#).

2 Analysis and Asymptotics

2.1 Stolz–Cesàro

Stolz–Cesàro is the discrete analogue of l'Hôpital's rule. The useful idea is that a ratio of two growing sequences is controlled by the ratios of their increments.

Theorem 1 (Stolz–Cesàro). *Let (a_n) and (b_n) be real sequences.*

(i) *Suppose b_n is strictly increasing and*

$$b_n \rightarrow \infty.$$

Then

$$\liminf_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n} \leq \liminf_{n \rightarrow \infty} \frac{a_n}{b_n} \leq \limsup_{n \rightarrow \infty} \frac{a_n}{b_n} \leq \limsup_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n}.$$

(ii) *Suppose*

$$a_n \rightarrow 0, \quad b_n \rightarrow 0,$$

and b_n is strictly decreasing. Then the same inequalities hold:

$$\liminf_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n} \leq \liminf_{n \rightarrow \infty} \frac{a_n}{b_n} \leq \limsup_{n \rightarrow \infty} \frac{a_n}{b_n} \leq \limsup_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n}.$$

In either case, if

$$\frac{a_{n+1} - a_n}{b_{n+1} - b_n} \rightarrow L,$$

then

$$\frac{a_n}{b_n} \rightarrow L.$$

Proof. Write

$$\ell = \liminf_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n}, \quad u = \limsup_{n \rightarrow \infty} \frac{a_{n+1} - a_n}{b_{n+1} - b_n}.$$

Fix $\varepsilon > 0$.

For the ∞ case, for all sufficiently large k ,

$$(\ell - \varepsilon)(b_{k+1} - b_k) \leq a_{k+1} - a_k \leq (u + \varepsilon)(b_{k+1} - b_k).$$

Summing from a fixed sufficiently large N to $n - 1$ gives

$$(\ell - \varepsilon)(b_n - b_N) \leq a_n - a_N \leq (u + \varepsilon)(b_n - b_N).$$

Divide by b_n and let $n \rightarrow \infty$. Since $b_n \rightarrow \infty$, the fixed terms disappear, yielding

$$\ell - \varepsilon \leq \liminf_{n \rightarrow \infty} \frac{a_n}{b_n} \leq \limsup_{n \rightarrow \infty} \frac{a_n}{b_n} \leq u + \varepsilon.$$

For the 0/0 case, it is more convenient to reverse both differences. Since b_n is strictly decreasing,

$$b_k - b_{k+1} > 0,$$

and

$$\frac{a_k - a_{k+1}}{b_k - b_{k+1}} = \frac{a_{k+1} - a_k}{b_{k+1} - b_k}.$$

Thus, for all sufficiently large k ,

$$(\ell - \varepsilon)(b_k - b_{k+1}) \leq a_k - a_{k+1} \leq (u + \varepsilon)(b_k - b_{k+1}).$$

Sum from $k = n$ to $m - 1$:

$$(\ell - \varepsilon)(b_n - b_m) \leq a_n - a_m \leq (u + \varepsilon)(b_n - b_m).$$

Now let $m \rightarrow \infty$. Since

$$a_m \rightarrow 0, \quad b_m \rightarrow 0,$$

we obtain

$$(\ell - \varepsilon)b_n \leq a_n \leq (u + \varepsilon)b_n.$$

A strictly decreasing sequence converging to 0 is positive, so dividing by b_n gives

$$\ell - \varepsilon \leq \frac{a_n}{b_n} \leq u + \varepsilon$$

for all sufficiently large n . Hence again

$$\ell - \varepsilon \leq \liminf_{n \rightarrow \infty} \frac{a_n}{b_n} \leq \limsup_{n \rightarrow \infty} \frac{a_n}{b_n} \leq u + \varepsilon.$$

Letting $\varepsilon \rightarrow 0$ proves both statements. □

2.2 Wallis Product

Theorem 2 (Wallis Product).

$$\frac{\pi}{2} = \prod_{k=1}^{\infty} \frac{(2k)^2}{(2k-1)(2k+1)}.$$

Proof. Let

$$I_m = \int_0^{\pi/2} \sin^m x \, dx.$$

Integration by parts gives, for $m \geq 2$,

$$I_m = \frac{m-1}{m} I_{m-2}.$$

Since $I_0 = \pi/2$ and $I_1 = 1$, this gives

$$I_{2n} = \frac{\pi}{2} \prod_{k=1}^n \frac{2k-1}{2k}, \quad I_{2n+1} = \prod_{k=1}^n \frac{2k}{2k+1}.$$

Because $0 \leq \sin x \leq 1$,

$$I_{2n+1} \leq I_{2n} \leq I_{2n-1}.$$

The recurrence also gives

$$\frac{I_{2n-1}}{I_{2n+1}} = \frac{2n+1}{2n} \rightarrow 1.$$

Therefore, by squeezing,

$$\frac{I_{2n}}{I_{2n+1}} \rightarrow 1.$$

On the other hand,

$$\frac{I_{2n}}{I_{2n+1}} = \frac{\pi}{2} \frac{(2n-1)!!(2n+1)!!}{((2n)!!)^2} = \frac{\pi/2}{\prod_{k=1}^n \frac{(2k)^2}{(2k-1)(2k+1)}}.$$

Thus the product tends to $\pi/2$. □

The same argument immediately gives the asymptotic form of the central binomial coefficient, which will determine the constant in Stirling's formula.

Corollary 3 (Central Binomial Coefficient).

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}.$$

Proof. Set

$$c_n = \frac{\binom{2n}{n}}{4^n} = \frac{(2n-1)!!}{(2n)!!}.$$

Then

$$I_{2n} = \frac{\pi}{2} c_n, \quad I_{2n+1} = \frac{1}{(2n+1)c_n}.$$

Since $I_{2n}/I_{2n+1} \rightarrow 1$,

$$\frac{\pi}{2} (2n+1) c_n^2 \rightarrow 1.$$

Therefore

$$c_n \sim \frac{1}{\sqrt{\pi n}},$$

which is the claim. □

2.3 Stirling's Formula

Theorem 4 (Stirling's Formula).

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

Proof. We proceed in three steps. The integral approximation

$$\log(n!) = \sum_{k=1}^n \log k \approx \int_1^n \log x \, dx = n \log n - n + 1$$

suggests first removing the terms $n \log n - n$. Define

$$s_n = \log(n!) - n \log n + n.$$

We determine the size of the remaining error using Stolz–Cesàro with $b_n = \log n$. We have

$$s_{n+1} - s_n = 1 - n \log \left(1 + \frac{1}{n}\right),$$

and

$$\log(n+1) - \log n = \log\left(1 + \frac{1}{n}\right).$$

Thus

$$\frac{s_{n+1} - s_n}{\log(n+1) - \log n} = \frac{1}{\log(1 + 1/n)} - n.$$

and hence

$$\frac{s_{n+1} - s_n}{\log(n+1) - \log n} \rightarrow \frac{1}{2}.$$

Stolz–Cesàro therefore gives

$$\frac{s_n}{\log n} \rightarrow \frac{1}{2}.$$

So the next-order term is forced to be $\frac{1}{2} \log n$. This motivates

$$c_n = \log(n!) - \left(n + \frac{1}{2}\right) \log n + n.$$

We have

$$c_{n+1} - c_n = 1 - \left(n + \frac{1}{2}\right) \log\left(1 + \frac{1}{n}\right).$$

Set

$$m = n + \frac{1}{2}.$$

Then

$$\log\left(1 + \frac{1}{n}\right) = \log \frac{m + 1/2}{m - 1/2} = \int_{-1/2}^{1/2} \frac{dt}{m + t}.$$

Therefore

$$\begin{aligned} m \log\left(1 + \frac{1}{n}\right) - 1 &= \int_{-1/2}^{1/2} \left(\frac{m}{m+t} - 1\right) dt \\ &= \int_0^{1/2} \left(\frac{m}{m+t} + \frac{m}{m-t} - 2\right) dt \\ &= \int_0^{1/2} \frac{2t^2}{m^2 - t^2} dt. \end{aligned}$$

In particular,

$$0 \leq m \log\left(1 + \frac{1}{n}\right) - 1 = O(n^{-2}).$$

Hence

$$|c_{n+1} - c_n| = O(n^{-2}).$$

Since $\sum_n n^{-2} < \infty$, the sequence (c_n) converges. Write

$$c_n \rightarrow \log C.$$

Exponentiating gives

$$n! \sim C \sqrt{n} \left(\frac{n}{e}\right)^n$$

for some constant $C > 0$.

Apply the preceding asymptotic to

$$\binom{2n}{n} = \frac{(2n)!}{(n!)^2}.$$

We obtain

$$\begin{aligned} \binom{2n}{n} &\sim \frac{C\sqrt{2n}(2n/e)^{2n}}{C^2n(n/e)^{2n}} \\ &= \frac{\sqrt{2}}{C} \frac{4^n}{\sqrt{n}}. \end{aligned}$$

But by [Corollary 3](#),

$$\binom{2n}{n} \sim \frac{4^n}{\sqrt{\pi n}}.$$

Comparing constants gives

$$\frac{\sqrt{2}}{C} = \frac{1}{\sqrt{\pi}},$$

so

$$C = \sqrt{2\pi}.$$

Therefore

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n.$$

□

2.4 Banach Fixed Point Theorem

A map $T : X \rightarrow X$ on a metric space is a *contraction* if there is some $q < 1$ such that

$$d(Tx, Ty) \leq q d(x, y)$$

for every $x, y \in X$.

The point is that repeatedly applying a contraction makes successive errors shrink geometrically.

Theorem 5 (Banach Fixed Point Theorem). *Let X be a complete metric space and let $T : X \rightarrow X$ be a contraction with constant $q < 1$. Then T has a unique fixed point x^* .*

Moreover, starting from any $x_0 \in X$ and defining $x_{n+1} = T(x_n)$, we have

$$d(x_n, x^*) \leq \frac{q^n}{1-q} d(x_1, x_0).$$

Proof. Since T contracts distances,

$$d(x_{n+1}, x_n) \leq q^n d(x_1, x_0).$$

Thus for $m > n$, the triangle inequality gives

$$d(x_m, x_n) \leq \sum_{j=n}^{m-1} d(x_{j+1}, x_j) \leq d(x_1, x_0) \sum_{j=n}^{\infty} q^j = \frac{q^n}{1-q} d(x_1, x_0).$$

Hence (x_n) is Cauchy. Completeness gives $x_n \rightarrow x^*$ for some $x^* \in X$.

A contraction is continuous, so

$$T(x^*) = T\left(\lim_n x_n\right) = \lim_n T(x_n) = \lim_n x_{n+1} = x^*.$$

Thus x^* is a fixed point. Letting $m \rightarrow \infty$ in the preceding estimate gives the claimed error bound. Finally, if y^* is another fixed point, then

$$d(x^*, y^*) = d(Tx^*, Ty^*) \leq q d(x^*, y^*).$$

Since $q < 1$, this forces $x^* = y^*$. □

The theorem is especially useful when an equation can be rewritten as $x = T(x)$. If T is a contraction on a closed region and maps that region into itself, the equation has exactly one solution there.

2.5 Inverse Function Theorem

Suppose $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is differentiable near a , and let $A = DF(a)$. If A is invertible, then near a the map F should behave like the invertible linear map

$$F(a) + A(x - a).$$

The clean way to make this precise is to solve $F(x) = y$ by fixed-point iteration. Rearranging the equation gives

$$x = x - A^{-1}(F(x) - y).$$

At $x = a$, the derivative of the right-hand side with respect to x is

$$I - A^{-1}DF(a) = 0.$$

Thus, sufficiently close to a , this map is a strong contraction.

Theorem 6 (Inverse Function Theorem). *Let $U \subseteq \mathbb{R}^n$ be open and let $F : U \rightarrow \mathbb{R}^n$ be continuously differentiable. Suppose $a \in U$ and $DF(a)$ is invertible.*

Then there are open neighborhoods V of a and W of $F(a)$ such that

$$F : V \rightarrow W$$

is a bijection whose inverse $G : W \rightarrow V$ is continuously differentiable.

Moreover,

$$\boxed{DG(y) = DF(G(y))^{-1}.$$

In particular,

$$DG(F(a)) = DF(a)^{-1}.$$

Proof. Set $b = F(a)$ and $A = DF(a)$. For each y near b , define

$$T_y(x) = x - A^{-1}(F(x) - y).$$

A point x is fixed by T_y exactly when $F(x) = y$.

The derivative with respect to x is

$$DT_y(x) = I - A^{-1}DF(x).$$

At $x = a$ this is 0. Since DF is continuous, we may choose a closed ball $\overline{B}(a, r) \subseteq U$ and some $q < 1$ such that

$$\|I - A^{-1}DF(x)\| \leq q \quad (x \in \overline{B}(a, r)).$$

By the mean value inequality,

$$\|T_y(x) - T_y(x')\| \leq q\|x - x'\|$$

throughout this ball.

We next ensure that T_y maps the ball into itself. Since

$$T_y(a) = a + A^{-1}(y - b),$$

for $x \in \overline{B}(a, r)$,

$$|T_y(x) - a| \leq |T_y(x) - T_y(a)| + |T_y(a) - a| \leq qr + |A^{-1}(y - b)|.$$

Thus if y is close enough to b that

$$\|A^{-1}(y - b)\| < (1 - q)r,$$

then T_y maps $\overline{B}(a, r)$ into itself.

Let

$$W = \{y : \|A^{-1}(y - b)\| < (1 - q)r\}.$$

For every $y \in W$, Banach's theorem gives a unique $x \in \overline{B}(a, r)$ satisfying $F(x) = y$. In fact the strict inequality above shows $x \in B(a, r)$. Define this unique solution to be $G(y)$.

Let

$$V = B(a, r) \cap F^{-1}(W).$$

Then V and W are open, and $F : V \rightarrow W$ is bijective with inverse G .

It remains to prove that G is continuously differentiable.

First observe that G is locally Lipschitz. Define $\Psi(x) = x - A^{-1}F(x)$. The derivative bound above says that Ψ is q -Lipschitz on the ball. If $x = G(y)$ and $x' = G(y')$, then

$$x = \Psi(x) + A^{-1}y, \quad x' = \Psi(x') + A^{-1}y'.$$

Therefore

$$\|x - x'\| \leq q\|x - x'\| + \|A^{-1}\| \|y - y'\|,$$

so

$$\|G(y) - G(y')\| \leq \frac{\|A^{-1}\|}{1 - q} \|y - y'\|.$$

Now fix $y \in W$, put $x = G(y)$, and let

$$G(y + h) = x + k.$$

The Lipschitz estimate gives $k = O(\|h\|)$. Since $F(x + k) = y + h$ and $F(x) = y$,

$$h = F(x + k) - F(x) = DF(x)k + o(\|k\|) = DF(x)k + o(\|h\|).$$

The matrix $DF(x)$ is invertible: indeed $\|I - A^{-1}DF(x)\| \leq q < 1$, so $A^{-1}DF(x)$ is invertible by the geometric-series argument. Hence

$$k = DF(x)^{-1}h + o(\|h\|).$$

Thus G is differentiable and

$$DG(y) = DF(G(y))^{-1}.$$

Continuity follows from the continuity of DF and of matrix inversion on the invertible matrices. $\square$

2.6 Implicit Function Theorem

The implicit function theorem says that if a system of equations can be uniquely solved for some variables at the linearized level, then it can also be solved for those variables locally in the nonlinear system.

Theorem 7 (Implicit Function Theorem). *Let*

$$F : U \subseteq \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^m$$

be continuously differentiable, and suppose

$$F(a, b) = 0.$$

If the $m \times m$ matrix

$$D_y F(a, b)$$

is invertible, then there are neighborhoods X of a and Y of b , and a unique continuously differentiable function

$$g : X \rightarrow Y$$

such that

$$F(x, g(x)) = 0$$

for every $x \in X$.

Moreover,

$$\boxed{Dg(x) = -D_y F(x, g(x))^{-1} D_x F(x, g(x)).}$$

Proof. The theorem is an immediate application of the inverse function theorem. Define

$$H(x, y) = (x, F(x, y)).$$

Its derivative at (a, b) is the block matrix

$$DH(a, b) = \begin{pmatrix} I & 0 \\ D_x F(a, b) & D_y F(a, b) \end{pmatrix}.$$

This is invertible because $D_y F(a, b)$ is invertible.

By the inverse function theorem, H has a continuously differentiable local inverse near

$$H(a, b) = (a, 0).$$

Since the first component of $H(x, y)$ is exactly x , its inverse must have the form

$$H^{-1}(x, z) = (x, \psi(x, z)).$$

Set

$$g(x) = \psi(x, 0).$$

Then

$$H(x, g(x)) = (x, 0),$$

which means exactly

$$F(x, g(x)) = 0.$$

Local uniqueness follows from the local injectivity of H .
To compute the derivative, differentiate

$$F(x, g(x)) = 0.$$

The chain rule gives

$$D_x F(x, g(x)) + D_y F(x, g(x)) Dg(x) = 0.$$

Solving for $Dg(x)$ yields

$$Dg(x) = -D_y F(x, g(x))^{-1} D_x F(x, g(x)).$$

□

2.7 Lagrange Multipliers

Suppose we want to optimize $f : \mathbb{R}^n \rightarrow \mathbb{R}$ subject to m continuously differentiable equality constraints

$$g_1(x) = \cdots = g_m(x) = 0.$$

Write

$$g = (g_1, \dots, g_m) : \mathbb{R}^n \rightarrow \mathbb{R}^m.$$

If the constraint gradients are linearly independent, the implicit function theorem says that the feasible set is locally an $(n - m)$ -dimensional surface. At a constrained extremum, f cannot change to first order in any tangent direction to this surface. Its gradient must therefore point entirely in the normal directions, which are exactly the directions spanned by the constraint gradients.

Theorem 8 (Lagrange Multipliers). *Let $f, g_1, \dots, g_m$ be continuously differentiable near $a \in \mathbb{R}^n$, and suppose*

$$g_1(a) = \cdots = g_m(a) = 0.$$

Assume that

$$\nabla g_1(a), \dots, \nabla g_m(a)$$

are linearly independent.

If a is a local maximum or minimum of f subject to the constraints $g_1 = \cdots = g_m = 0$, then there are unique scalars $\lambda_1, \dots, \lambda_m$ such that

$$\nabla f(a) = \sum_{i=1}^m \lambda_i \nabla g_i(a).$$

Proof. Since the constraint gradients are linearly independent, the Jacobian $Dg(a)$ has rank m . After permuting the coordinates, write

$$x = (u, v), \quad u \in \mathbb{R}^{n-m}, \quad v \in \mathbb{R}^m,$$

so that the square matrix $D_v g(a)$ is invertible.

Write $a = (u_0, v_0)$. By the implicit function theorem, all nearby feasible points are uniquely of the form

$$(u, \phi(u))$$

for a continuously differentiable function ϕ , with

$$g(u, \phi(u)) = 0.$$

Differentiating this identity at u_0 gives

$$D_u g(a) + D_v g(a) D\phi(u_0) = 0,$$

and hence

$$D\phi(u_0) = -D_v g(a)^{-1} D_u g(a).$$

Now define the restriction of f to the feasible surface by

$$h(u) = f(u, \phi(u)).$$

Since a is a constrained local extremum, u_0 is an ordinary local extremum of h . Therefore $Dh(u_0) = 0$. By the chain rule,

$$D_u f(a) + D_v f(a) D\phi(u_0) = 0.$$

Substituting the formula for $D\phi(u_0)$,

$$D_u f(a) = D_v f(a) D_v g(a)^{-1} D_u g(a).$$

Define the row vector

$$\lambda^T = D_v f(a) D_v g(a)^{-1}.$$

Then

$$D_u f(a) = \lambda^T D_u g(a), \quad D_v f(a) = \lambda^T D_v g(a).$$

Together these say

$$Df(a) = \lambda^T Dg(a).$$

Writing this in terms of gradients gives

$$\nabla f(a) = \sum_{i=1}^m \lambda_i \nabla g_i(a).$$

The multipliers are unique because the constraint gradients are linearly independent. □

2.8 Karamata's Inequality

Karamata's inequality is a consequence of two facts:

- majorization says that the partial-sum differences have one sign;
- convexity says that secant slopes increase as their endpoints move to the right.

Summation by parts then combines these two.

Theorem 9 (Karamata's Inequality). *Let*

$$x, y \in \mathbb{R}^n, \quad x \prec y,$$

and let f be convex on an interval containing all coordinates of x and y . Then

$$\boxed{\sum_{i=1}^n f(x_i) \leq \sum_{i=1}^n f(y_i)}.$$

More generally, if

$$x \prec_w y$$

and f is convex and nondecreasing, then the same inequality holds.

Proof. Since both majorization and the desired inequality are invariant under permuting coordinates, assume

$$x_1 \geq \cdots \geq x_n, \quad y_1 \geq \cdots \geq y_n.$$

Define the partial sums

$$X_k = \sum_{i=1}^k x_i, \quad Y_k = \sum_{i=1}^k y_i,$$

and their difference

$$D_k = Y_k - X_k, \quad D_0 = 0.$$

Since $x \prec y$,

$$D_k \geq 0 \quad (1 \leq k < n),$$

and

$$D_n = 0.$$

If $x_i = y_i$ for some i , we may delete both coordinates. The two remaining sequences are still decreasing, and their partial-sum inequalities are unchanged after reindexing. Thus we may assume

$$x_i \neq y_i \quad \text{for every } i.$$

For each i , define the secant slope

$$c_i = \frac{f(y_i) - f(x_i)}{y_i - x_i}.$$

The basic one-dimensional fact about a convex function is that its secant slope is nondecreasing in either endpoint: if

$$a \leq a', \quad b \leq b',$$

then

$$\frac{f(b) - f(a)}{b - a} \leq \frac{f(b') - f(a')}{b' - a'}$$

whenever the quotients are defined.

Since

$$x_i \geq x_{i+1}, \quad y_i \geq y_{i+1},$$

both endpoints of the i -th secant lie no farther left than the corresponding endpoints of the $(i+1)$ -st secant. Therefore

$$\boxed{c_1 \geq c_2 \geq \cdots \geq c_n}.$$

Now

$$y_i - x_i = D_i - D_{i-1},$$

so

$$\begin{aligned}\sum_{i=1}^n (f(y_i) - f(x_i)) &= \sum_{i=1}^n c_i (y_i - x_i) \\ &= \sum_{i=1}^n c_i (D_i - D_{i-1}).\end{aligned}$$

Summation by parts gives

$$\sum_{i=1}^n c_i (D_i - D_{i-1}) = c_n D_n + \sum_{i=1}^{n-1} (c_i - c_{i+1}) D_i.$$

For ordinary majorization,

$$D_n = 0,$$

while

$$c_i - c_{i+1} \geq 0, \quad D_i \geq 0.$$

Hence

$$\sum_{i=1}^n (f(y_i) - f(x_i)) \geq 0,$$

which proves

$$\sum_i f(x_i) \leq \sum_i f(y_i).$$

For weak majorization $x \prec_w y$, we instead have

$$D_i \geq 0 \quad (1 \leq i \leq n).$$

If f is also nondecreasing, every secant slope satisfies

$$c_i \geq 0.$$

Thus the additional endpoint term

$$c_n D_n$$

is also nonnegative, and the same argument proves the weak form. □

2.9 The Isoperimetric Inequality

The isoperimetric inequality says that among all simple closed curves of a given length, the circle encloses the largest area:

$4\pi A \leq L^2.$

There are two ingredients.

First, the area enclosed by an oriented curve can be written as

$$2A = \int (x dy - y dx).$$

For polygons, this is just the shoelace formula, and the shoelace formula itself follows immediately by writing the area of each triangle as a determinant and observing that internal edges cancel.

Second, if a periodic function has mean zero, then it cannot be large without changing substantially:

$$\int_0^{2\pi} f^2 \leq \int_0^{2\pi} (f')^2.$$

This is the periodic Wirtinger inequality.

Lemma 10 (Shoelace Formula). *Let*

$$p_i = (x_i, y_i), \quad i = 1, \dots, n,$$

be the vertices of a simple polygon P , listed counterclockwise, with $p_{n+1} = p_1$. Then

$$\text{area}(P) = \frac{1}{2} \sum_{i=1}^n \det(p_i, p_{i+1}) = \frac{1}{2} \sum_{i=1}^n (x_i y_{i+1} - y_i x_{i+1}).$$

Proof. For three points a, b, c , the signed area of the oriented triangle abc is

$$[\triangle abc] = \frac{1}{2} \det(b - a, c - a).$$

Expanding the determinant gives

$$\begin{aligned} \det(b - a, c - a) &= \det(b, c) - \det(b, a) - \det(a, c) \\ &= \det(a, b) + \det(b, c) + \det(c, a). \end{aligned}$$

Thus

$$2[\triangle abc] = \det(a, b) + \det(b, c) + \det(c, a).$$

In other words, twice the signed area of a triangle is the sum of $\det(u, v)$ over its three oriented boundary edges $u \rightarrow v$.

Now triangulate P , and orient every triangle counterclockwise. Since the triangles have disjoint interiors,

$$\text{area}(P) = \sum_{\triangle \in T} \text{area}(\triangle).$$

Hence

$$2 \text{area}(P) = \sum_{\triangle \in T} \sum_{(u,v) \in \partial \triangle} \det(u, v).$$

Every internal edge uv occurs in exactly two triangles. One triangle traverses it as $u \rightarrow v$, while the other traverses it as $v \rightarrow u$, so their contributions cancel:

$$\det(u, v) + \det(v, u) = 0.$$

Thus only the boundary edges remain. These are

$$p_1 \rightarrow p_2, \quad p_2 \rightarrow p_3, \quad \dots, \quad p_n \rightarrow p_1,$$

and therefore

$$2 \text{area}(P) = \sum_{i=1}^n \det(p_i, p_{i+1}).$$

□

Lemma 11 (Signed Area Formula). *Let*

$$\gamma(t) = (x(t), y(t)), \quad a \leq t \leq b,$$

be a counterclockwise simple closed piecewise C^1 curve enclosing a region of area A . Then

$$2A = \int_a^b (x(t)y'(t) - y(t)x'(t)) dt.$$

Proof. First suppose that γ is polygonal, with consecutive vertices $p_1, \dots, p_n$. On an edge from p to q , parameterized by

$$\gamma(t) = (1-t)p + tq, \quad 0 \leq t \leq 1,$$

we have

$$\gamma'(t) = q - p,$$

and hence

$$\det(\gamma(t), \gamma'(t)) = \det(p, q).$$

Therefore

$$\int_{\text{edge}} (x dy - y dx) = \det(p, q).$$

Summing over all edges and applying [Lemma 10](#) gives

$$\int_{\gamma} (x dy - y dx) = 2A.$$

For a general piecewise C^1 simple closed curve, choose increasingly fine inscribed polygonal approximations P_k . Since γ is piecewise C^1 , the polygonal parameterizations converge uniformly to γ , while their derivatives converge to γ' in L^1 on each C^1 piece. Consequently,

$$\int_{\partial P_k} (x dy - y dx) \longrightarrow \int_{\gamma} (x dy - y dx).$$

The enclosed polygonal regions also converge to the region enclosed by γ , so

$$\text{area}(P_k) \longrightarrow A.$$

Applying the polygonal formula and taking limits gives

$$2A = \int_{\gamma} (x dy - y dx).$$

□

We next prove the analytic ingredient. We begin with the version on an interval. Minimizing

$$\lambda \int_0^\pi (u')^2 - \int_0^\pi u^2$$

subject to

$$u(0) = u(\pi) = 0,$$

we guess that $u'u'' \propto uu'$, so $u = C \sin t$.

We then factor the extremizer out:

$$u(t) = v(t) \sin t.$$

The gap in the inequality will measure exactly how far v is from being constant.

Lemma 12 (Dirichlet Wirtinger Inequality). *Let*

$$u : [0, \pi] \rightarrow \mathbb{R}$$

be continuously differentiable with

$$u(0) = u(\pi) = 0.$$

Then

$$\boxed{\int_0^\pi u(t)^2 dt \leq \int_0^\pi u'(t)^2 dt.}$$

Equality holds exactly for

$$u(t) = C \sin t.$$

Proof. For $0 < t < \pi$, write

$$u(t) = v(t) \sin t.$$

Then

$$u' = v' \sin t + v \cos t.$$

Consequently,

$$(u')^2 - u^2 = (v')^2 \sin^2 t + 2vv' \sin t \cos t + v^2(\cos^2 t - \sin^2 t).$$

The last two terms form a derivative:

$$\frac{d}{dt}(v^2 \sin t \cos t) = 2vv' \sin t \cos t + v^2(\cos^2 t - \sin^2 t).$$

Thus

$$\boxed{(u')^2 - u^2 = (v')^2 \sin^2 t + \frac{d}{dt}(v^2 \sin t \cos t).}$$

Integrating first over

$$[\varepsilon, \pi - \varepsilon]$$

gives

$$\int_\varepsilon^{\pi-\varepsilon} ((u')^2 - u^2) dt = \int_\varepsilon^{\pi-\varepsilon} (v')^2 \sin^2 t dt + [v^2 \sin t \cos t]_\varepsilon^{\pi-\varepsilon}.$$

Since u is continuously differentiable and vanishes at the endpoints,

$$u(t) = O(t) \quad (t \rightarrow 0)$$

and

$$u(t) = O(\pi - t) \quad (t \rightarrow \pi).$$

Since $\sin t$ has the same behavior, $v = u/\sin t$ remains bounded near both endpoints. Hence

$$v(t)^2 \sin t \cos t \rightarrow 0$$

at both endpoints.

Letting $\varepsilon \rightarrow 0$ therefore gives

$$\int_0^\pi ((u')^2 - u^2) dt = \int_0^\pi \sin^2 t \left(\frac{d}{dt} \frac{u(t)}{\sin t} \right)^2 dt \geq 0.$$

This proves the inequality.

Equality holds precisely when

$$v' = 0,$$

so v is constant and

$$u(t) = C \sin t.$$

□

Lemma 13 (Mean-Zero Wirtinger Inequality). *Let*

$$f : [0, \pi] \rightarrow \mathbb{R}$$

be continuously differentiable and suppose

$$\int_0^\pi f(t) dt = 0.$$

Then

$$\int_0^\pi f(t)^2 dt \leq \int_0^\pi f'(t)^2 dt.$$

Proof. Define

$$F(x) = \int_0^x f(t) dt.$$

Then

$$F(0) = 0,$$

while the mean-zero assumption gives

$$F(\pi) = 0.$$

Thus [Lemma 12](#) gives

$$\int_0^\pi F^2 \leq \int_0^\pi (F')^2 = \int_0^\pi f^2.$$

Since $F' = f$, integration by parts gives

$$\begin{aligned} \int_0^\pi f^2 &= \int_0^\pi F' f \\ &= [F f]_0^\pi - \int_0^\pi F f' \\ &= - \int_0^\pi F f'. \end{aligned}$$

Hence Cauchy–Schwarz gives

$$\int_0^\pi f^2 \leq \left(\int_0^\pi F^2 \right)^{1/2} \left(\int_0^\pi (f')^2 \right)^{1/2}.$$

Using

$$\int_0^\pi F^2 \leq \int_0^\pi f^2,$$

we obtain

$$\int_0^\pi f^2 \leq \left(\int_0^\pi f^2 \right)^{1/2} \left(\int_0^\pi (f')^2 \right)^{1/2}.$$

Canceling the first factor proves

$$\int_0^\pi f^2 \leq \int_0^\pi (f')^2.$$

□

We now pass from an interval to a circle.

The only issue is that the preceding lemma requires an interval of length π on which the integral is zero. If a function has mean zero on the whole circle, such a semicircle always exists.

Indeed, slide a semicircle around the circle and measure the integral of f over it. Rotating the semicircle by π replaces it by its complement, so its integral changes sign. Somewhere in between, it must be zero.

Lemma 14 (Periodic Wirtinger Inequality). *Let*

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

be continuously differentiable and 2π -periodic. If

$$\int_0^{2\pi} f(t) dt = 0,$$

then

$$\boxed{\int_0^{2\pi} f(t)^2 dt \leq \int_0^{2\pi} f'(t)^2 dt.}$$

Proof. For $a \in \mathbb{R}$, define

$$H(a) = \int_a^{a+\pi} f(t) dt.$$

The function H is continuous.

The intervals

$$[a, a + \pi] \quad \text{and} \quad [a + \pi, a + 2\pi]$$

together form one full period, so

$$H(a) + H(a + \pi) = \int_a^{a+2\pi} f(t) dt = 0.$$

Therefore

$$H(a + \pi) = -H(a).$$

If $H(a) = 0$ for some a , we already have the desired cut. Otherwise, $H(a)$ and $H(a + \pi)$ have opposite signs. By continuity, there is some $b \in [a, a + \pi]$ such that

$$H(b) = 0.$$

Thus

$$\int_b^{b+\pi} f(t) dt = 0.$$

Since the integral over the entire period is zero,

$$\int_{b+\pi}^{b+2\pi} f(t) dt = 0$$

as well.

Apply [Lemma 13](#) to the first semicircle:

$$\int_b^{b+\pi} f^2 \leq \int_b^{b+\pi} (f')^2.$$

Apply it again to the second:

$$\int_{b+\pi}^{b+2\pi} f^2 \leq \int_{b+\pi}^{b+2\pi} (f')^2.$$

Adding gives

$$\int_b^{b+2\pi} f^2 \leq \int_b^{b+2\pi} (f')^2.$$

By periodicity,

$$\int_0^{2\pi} f^2 \leq \int_0^{2\pi} (f')^2.$$

□

Applying [Lemma 14](#) separately to the two coordinates gives the vector form: if

$$\gamma : [0, 2\pi] \rightarrow \mathbb{R}^2$$

is periodic and

$$\int_0^{2\pi} \gamma(t) dt = 0,$$

then

$$\boxed{\int_0^{2\pi} |\gamma(t)|^2 dt \leq \int_0^{2\pi} |\gamma'(t)|^2 dt.}$$

Theorem 15 (Isoperimetric Inequality). *Let Γ be a simple closed piecewise C^1 curve of length L , enclosing area A . Then*

$$\boxed{4\pi A \leq L^2.}$$

Equality holds exactly for circles.

Proof. Parameterize Γ counterclockwise at constant speed on $[0, 2\pi]$:

$$\gamma(t) = (x(t), y(t)), \quad |\gamma'(t)| = \frac{L}{2\pi}$$

wherever γ' exists.

Translation does not change length or area, so translate the curve so that

$$\int_0^{2\pi} \gamma(t) dt = 0.$$

By [Lemma 11](#),

$$2A = \int_0^{2\pi} \det(\gamma(t), \gamma'(t)) dt.$$

Using

$$|\det(u, v)| \leq |u||v|$$

and then Cauchy–Schwarz,

$$\begin{aligned} 2A &\leq \int_0^{2\pi} |\gamma(t)| |\gamma'(t)| dt \\ &\leq \left(\int_0^{2\pi} |\gamma(t)|^2 dt \right)^{1/2} \left(\int_0^{2\pi} |\gamma'(t)|^2 dt \right)^{1/2}. \end{aligned}$$

Periodic Wirtinger gives

$$\int_0^{2\pi} |\gamma(t)|^2 dt \leq \int_0^{2\pi} |\gamma'(t)|^2 dt,$$

so

$$2A \leq \int_0^{2\pi} |\gamma'(t)|^2 dt.$$

Since the speed is constant,

$$\int_0^{2\pi} |\gamma'(t)|^2 dt = 2\pi \left(\frac{L}{2\pi} \right)^2 = \frac{L^2}{2\pi}.$$

Therefore

$$2A \leq \frac{L^2}{2\pi},$$

or

$$\boxed{4\pi A \leq L^2.}$$

Suppose equality holds. Equality in periodic Wirtinger implies

$$\gamma(t) = p \cos t + q \sin t$$

after the translation, for some fixed vectors $p, q \in \mathbb{R}^2$.
Since γ has constant speed,

$$|\gamma'(t)|^2 = |p|^2 \sin^2 t + |q|^2 \cos^2 t - 2\langle p, q \rangle \sin t \cos t$$

is independent of t . Hence

$$|p| = |q|, \quad \langle p, q \rangle = 0.$$

Thus p and q are perpendicular vectors of equal length, and

$$\gamma(t) = p \cos t + q \sin t$$

parameterizes a circle.

Conversely, a circle of radius r satisfies

$$L = 2\pi r, \quad A = \pi r^2,$$

so equality holds. □

3 Linear Algebra

3.1 Determinant and Inverse Identities

Lemma 16 (Cauchy-Binet Formula). *Let A be an $m \times n$ matrix and B be an $n \times m$ matrix (with $m \leq n$). Then*

$$\det(AB) = \sum_{S \in \binom{[n]}{m}} \det(A_{[m],S}) \det(B_{S,[m]})$$

where $\binom{[n]}{m}$ denotes the set of all m -element subsets of $\{1, \dots, n\}$.

Proof. **Fact 1:** For any $n \times n$ matrix X and any integer $1 \leq k \leq n$, the coefficient of z^{n-k} in the polynomial $\det(zI_n + X)$ is exactly the sum of all $k \times k$ principal minors of X .

Fact 2: For any $m \times n$ matrix A and $n \times m$ matrix B we have from [Theorem 21](#):

$$\det(zI_n + BA) = z^{n-m} \det(zI_m + AB)$$

Applying Fact 1 with $k = m$, the coefficient of z^{n-m} in $\det(zI_n + BA)$ is the sum of the $m \times m$ principal minors of BA . A principal minor of BA corresponding to an index subset $S \subseteq [n]$ of size m is the determinant of the submatrix $(BA)_{S,S}$, and $(BA)_{S,S} = B_{S,[m]} A_{[m],S}$.

Because $B_{S,[m]}$ and $A_{[m],S}$ are both square:

$$\det((BA)_{S,S}) = \det(B_{S,[m]} A_{[m],S}) = \det(B_{S,[m]}) \det(A_{[m],S})$$

The coefficient of z^{n-m} on the left-hand side is:

$$\sum_{S \in \binom{[n]}{m}} \det(B_{S,[m]}) \det(A_{[m],S})$$

On the right-hand side, the coefficient $[z^{n-m}]$ is $[x^0] \det(zI_m + AB) = \det(AB)$. □

Lemma 17.

$$\det'(I) = \text{Tr}$$

Proof. We have:

$$\det'(I)(T) = \nabla_T \det(I) = \lim_{\varepsilon \rightarrow 0} \frac{\det(I + \varepsilon T) - \det(I)}{\varepsilon}$$

$\det(I + \varepsilon T)$ is a polynomial in ε of order n . The constant term is 1, while the linear term is $\text{Tr}(T)$. □

Lemma 18. *For an invertible matrix A , we have:*

$$\det'(A)(T) = \det(A) \text{Tr}(A^{-1}T)$$

Proof. Consider the following function of X :

$$\det(X) = \det(AA^{-1}X) = \det(A) \det(A^{-1}X)$$

Using [Lemma 17](#):

$$\det'(A)(T) = \det(A) \det'(I)(A^{-1}T) = \det(A) \text{Tr}(A^{-1}T)$$

□

Lemma 19 (Jacobi's Formula).

$$\frac{d}{dt} \det A = \text{Tr} \left(\text{adj}(A) \frac{dA}{dt} \right)$$

Proof. Let $A(t)$ be a differentiable matrix function. We introduce a variable x and consider $A_x(t) = A(t) - xI$, $A_x(t)$ is invertible for infinitely many values of x .

For all x where $A_x(t)$ is invertible, we can apply [Lemma 18](#) with $T = \frac{dA_x}{dt} = \frac{dA}{dt}$:

$$\frac{d}{dt} \det(A_x) = \det(A_x) \text{Tr} \left(A_x^{-1} \frac{dA}{dt} \right)$$

Using $\det(A_x)A_x^{-1} = \text{adj}(A_x)$, this becomes:

$$\frac{d}{dt} \det(A(t) - xI) = \text{Tr} \left(\text{adj}(A(t) - xI) \frac{dA}{dt} \right)$$

Since they agree for infinitely many x , they agree as polynomials, substituting $x = 0$ we have:

$$\frac{d}{dt} \det A = \text{Tr} \left(\text{adj}(A) \frac{dA}{dt} \right)$$

□

Lemma 20. *Let A be an $n \times n$ matrix. For any subset of indices $S \subseteq \{1, 2, \dots, n\}$, let A_S denote the $(n - |S|) \times (n - |S|)$ principal submatrix. Then:*

$$\frac{p_A^{(k)}(x)}{k!} = \sum_{|S|=k} p_{A_S}(x)$$

Proof. We evaluate $p_A(x + y)$ in two different ways.

We can consider the Taylor expansion of $p_A(x + y)$ around x :

$$p_A(x + y) = \sum_{k=0}^n \frac{p_A^{(k)}(x)}{k!} y^k$$

Second:

$$p_A(x + y) = \det((x + y)I - A) = \det(yI + (xI - A))$$

The coefficient y here is the sum of the determinant of all $(n-k) \times (n-k)$ sized minors $\det(xI_{n-k} - A_S) = p_{A_S}(x)$.

Equating the coefficients of y^k yields:

$$\frac{p_A^{(k)}(x)}{k!} = \sum_{|S|=k} p_{A_S}(x)$$

□

3.1.1 Woodbury Identity

The Woodbury identity is most naturally viewed as an identity between the two Schur complements of the same block matrix.

Recall that for

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix},$$

if A is invertible, the Schur complement of A is

$$M/A = D - CA^{-1}B.$$

The block factorization

$$M = \begin{pmatrix} I & 0 \\ CA^{-1} & I \end{pmatrix} \begin{pmatrix} A & 0 \\ 0 & M/A \end{pmatrix} \begin{pmatrix} I & A^{-1}B \\ 0 & I \end{pmatrix}$$

immediately gives

$$\det M = \det(A) \det(M/A).$$

If M/A is also invertible, inverting the factorization shows that the upper-left block of M^{-1} is

$$\boxed{(M^{-1})_{11} = A^{-1} + A^{-1}B(M/A)^{-1}CA^{-1}.}$$

Similarly, if D is invertible, then

$$M/D = A - BD^{-1}C,$$

and eliminating the D -block first shows that

$$\boxed{(M^{-1})_{11} = (M/D)^{-1}.}$$

These two descriptions of the same block give Woodbury immediately.

Theorem 21 (Woodbury Identity). *Let A and C be invertible, and suppose*

$$C^{-1} + VA^{-1}U$$

is invertible. Then

$$\boxed{(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}.}$$

Proof. Consider

$$M = \begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}.$$

Taking the Schur complement of A gives

$$M/A = C^{-1} + VA^{-1}U.$$

Hence the upper-left block of M^{-1} is

$$(M^{-1})_{11} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}.$$

On the other hand, taking the Schur complement of C^{-1} gives

$$M/C^{-1} = A - UC(-V) = A + UCV.$$

Therefore

$$(M^{-1})_{11} = (M/C^{-1})^{-1} = (A + UCV)^{-1}.$$

Equating the two expressions for the same block of M^{-1} gives the identity. □

Thus the identity can be remembered from the block matrix

$$\boxed{\begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}},$$

whose two Schur complements are

$$\boxed{M/A = C^{-1} + VA^{-1}U, \quad M/C^{-1} = A + UCV.}$$

3.1.2 Matrix Determinant Lemma

The determinant analogue of Woodbury follows from exactly the same block matrix.

Theorem 22 (Matrix Determinant Lemma). *Let*

$$A \in \mathbb{F}^{n \times n}, \quad C \in \mathbb{F}^{k \times k}$$

be invertible, and let

$$U \in \mathbb{F}^{n \times k}, \quad V \in \mathbb{F}^{k \times n}.$$

Then

$$\boxed{\det(A + UCV) = \det(A) \det(C) \det(C^{-1} + VA^{-1}U).}$$

Proof. Again take

$$M = \begin{pmatrix} A & U \\ -V & C^{-1} \end{pmatrix}.$$

Computing its determinant using the Schur complement of A ,

$$\det M = \det(A) \det(M/A) = \det(A) \det(C^{-1} + VA^{-1}U).$$

Computing it instead using the Schur complement of C^{-1} ,

$$\det M = \det(C^{-1}) \det(M/C^{-1}) = \det(C)^{-1} \det(A + UCV).$$

Equating the two expressions and multiplying by $\det(C)$ gives

$$\det(A + UCV) = \det(A) \det(C) \det(C^{-1} + VA^{-1}U).$$

□

For $C = I$, this becomes

$$\boxed{\det(A + UV) = \det(A) \det(I + VA^{-1}U).}$$

Thus Woodbury and the matrix determinant lemma are the inverse and determinant consequences of the same observation:

$$\boxed{M/A = C^{-1} + VA^{-1}U, \quad M/C^{-1} = A + UCV.}$$

The large $n \times n$ update $A + UCV$ and the small $k \times k$ matrix $C^{-1} + VA^{-1}U$ are the two Schur complements of one block matrix.

The following is the positive-semidefiniteness version of the determinant lemma.

Lemma 23 (Schur Complement Positivity). *For $B \succ 0$, $\begin{pmatrix} A & X \\ X^* & B \end{pmatrix} \succeq 0 \iff A \succeq XB^{-1}X^*$.*

Proof. Conjugating a block matrix by an invertible matrix preserves positive semidefiniteness. We calculate:

$$\begin{pmatrix} I & -XB^{-1} \\ 0 & I \end{pmatrix} \begin{pmatrix} A & X \\ X^* & B \end{pmatrix} \begin{pmatrix} I & 0 \\ -B^{-1}X^* & I \end{pmatrix} = \begin{pmatrix} A - XB^{-1}X^* & 0 \\ 0 & B \end{pmatrix}$$

Because $B \succ 0$, the block diagonal matrix on the right is positive semidefinite if and only if $A - XB^{-1}X^* \succeq 0$. □

3.2 Characteristic Polynomial

To compute the characteristic polynomial $p(x) = \det(xI - A)$ of an $n \times n$ matrix A , the algorithm first reduces A to an upper Hessenberg matrix H (where $H_{i,j} = 0$ for all $i > j + 1$) in $O(n^3)$ time. It then evaluates the characteristic polynomial using a dynamic programming approach.

Lemma 24 (Hessenberg Characteristic Polynomial Recurrence). *Let H_k be the $k \times k$ leading principal submatrix of an upper Hessenberg matrix H , and let $p_k(x) = \det(xI_k - H_k)$ be its characteristic polynomial. With the base case $p_0(x) = 1$, the polynomials satisfy the recurrence:*

$$p_{k+1}(x) = (x - H_{k,k})p_k(x) - \sum_{i=0}^{k-1} \left(H_{i,k} \prod_{j=i}^{k-1} H_{j+1,j} \right) p_i(x)$$

Proof. We evaluate $p_{k+1}(x) = \det(xI_{k+1} - H_{k+1})$ by performing a Laplace expansion along its last column. The entries in the last column of $xI_{k+1} - H_{k+1}$ are $-H_{0,k}, -H_{1,k}, \dots, -H_{k-1,k}$, with the diagonal entry being $(x - H_{k,k})$.

The cofactor for the diagonal entry $(x - H_{k,k})$ is the determinant of the top-left $k \times k$ block, which is $p_k(x)$.

For any other entry $-H_{i,k}$ (where $i < k$) in the last column, removing its row and column leaves a block upper-triangular matrix. The top-left block is $(xI_i - H_i)$, which has determinant $p_i(x)$. Because H is upper Hessenberg, the bottom-right block is upper triangular, with its main diagonal consisting entirely of the subdiagonal elements of H : $-H_{i+1,i}, -H_{i+2,i+1}, \dots, -H_{k,k-1}$. The alternating signs from the cofactor expansion cancels with the negative signs of these subdiagonal elements. □

3.2.1 Determinant of Affine Matrix Polynomial

The algorithm computes the polynomial $P(x) = \det(A + xB)$ in $O(n^3)$ time.

The algorithm samples a random scalar $a \in \mathbb{F}$ and evaluates the matrix at this point, defining $A' = A + aB$. Assuming A' is invertible, we can factor it out of the original polynomial:

$$\begin{aligned} P(x) &= \det(A + aB + (x - a)B) \\ &= \det(A' + (x - a)B) \\ &= \det(A') \det(I + (x - a)A'^{-1}B) \end{aligned}$$

Let $M = -A'^{-1}B$. The polynomial becomes $P(x) = \det(A') \det(I - (x - a)M)$.

Notice that $\det(I - tM) = t^n \det(t^{-1}I - M) = t^n p_M(t^{-1})$, the reversed characteristic polynomial of M . Let $R(t)$ denote this reversed polynomial. The algorithm computes $p_M(t)$ using the Hessenberg DP, reverses its coefficients to obtain $R(t)$, multiplies the polynomial by the scalar $\det(A')$, and performs a Taylor shift $x \mapsto x - a$ to recover $P(x) = \det(A')R(x - a)$.

Lemma 25 (Failure Probability of the Affine Determinant). *If $P(x) = \det(A + xB)$ is not identically the zero polynomial, the probability that the randomized reduction algorithm incorrectly fails (by returning 0) is bounded by $\frac{n}{|\mathbb{F}|}$.*

Proof. The algorithm only fails if the shifted matrix $A' = A + aB$ is not invertible, if and only if $P(a) = 0$.

The algorithm samples a uniformly and out of all possible random choices for a , at most n of them can cause A' to be singular. If $P(x)$ is identically the zero polynomial, the algorithm will correctly find $\det(A') = 0$ for any a . $\square$

3.3 Frobenius Form

3.3.1 Algorithm Description

To compute the Frobenius form of a matrix $A \in \mathbb{F}^{n \times n}$ in $O(n^3)$ time, the algorithm employs a randomized Krylov subspace approach. The Frobenius form decomposes the vector space V into a direct sum of A -cyclic subspaces $V = \bigoplus_{i=1}^m W_i$. This yields a block-diagonal matrix where each block is the companion matrix of an invariant factor $P_i(z)$ of A . By the structure theorem of finitely generated modules over a PID, these invariant factors divide each other: $P_m(z) \mid P_{m-1}(z) \mid \dots \mid P_1(z)$.

1. **State:** Maintain a set of generator vectors $\{w_1, \dots, w_k\}$ for the resolved cyclic subspaces $W_1, \dots, W_k$, and a row-reduced basis $\mathcal{B}$ representing their combined span $W = \bigoplus_{i=1}^k W_i$.
2. **Random Sampling:** Draw a uniformly random vector $x \in \mathbb{F}^n$ to act as the candidate generator for the next cyclic block, W_{k+1} .
3. **Krylov Sequence:** Generate the Krylov sequence $x, Ax, A^2x, \dots$ and incrementally reduce each new vector against the existing basis $\mathcal{B}$ using Gaussian elimination. This process halts when a linear dependence is found, yielding the minimal annihilating polynomial $C(z)$ of x modulo the previously resolved space W .
4. **Orthogonalization:** The linear dependence indicates that $C(A)x \in W$. To ensure the newly found cyclic subspace forms a direct summand, x must be updated to $\tilde{x} = x - u$ (for some $u \in W$) such that $C(A)\tilde{x} = 0$. The algorithm extracts the necessary shift by performing a polynomial division over the concatenated dependency arrays, and backtracks to apply it to x . It then recomputes the Krylov sequence.
5. **Iteration:** The vectors $\tilde{x}, A\tilde{x}, \dots, A^{\deg(C)-1}\tilde{x}$ are added to $\mathcal{B}$. This repeats until $\dim(W) = n$.

3.3.2 Correctness

Theorem 26 (Divisibility of the Residuals). *Let $W = \bigoplus_{i=1}^k W_i$ be the resolved space, where each $W_i = \langle w_i \rangle_A$ is a cyclic submodule with minimal polynomial $P_i(z)$. Let x be the random candidate vector, and $C(z)$ be its minimal polynomial modulo W . The Krylov reduction yields a dependency of the form:*

$$C(A)x = \sum_{i=1}^k R^{(i)}(A)w_i$$

Then the polynomial $C(z)$ divides $R^{(i)}(z)$ in $\mathbb{F}[z]$ for all $1 \leq i \leq k$.

Proof. As shown in the next section, with good probability, the minimal polynomial of the newly sampled block must divide the minimal polynomials of all previously extracted blocks: $C(z) \mid P_i(z)$ for all $i \leq k$.

Because the true vector space decomposes into a direct sum of cyclic submodules, we can uniquely decompose our random candidate as $x = \tilde{x} + \sum_{i=1}^k v_i$, where $\tilde{x}$ belongs to the target complementary subspace (meaning it is exactly annihilated by $C(A)$) and $v_i \in W_i$.

Applying $C(A)$ to this decomposition yields $C(A)x = C(A)\tilde{x} + \sum_{i=1}^k C(A)v_i = \sum_{i=1}^k C(A)v_i$. Since each $v_i \in \langle w_i \rangle_A$, there exist polynomials $S^{(i)}(z)$ such that $v_i = S^{(i)}(A)w_i$. Thus:

$$C(A)x = \sum_{i=1}^k C(A)S^{(i)}(A)w_i$$

Equating this with the relationship obtained from our Krylov reduction, we have:

$$\sum_{i=1}^k (C(A)S^{(i)}(A) - R^{(i)}(A))w_i = 0$$

Because W is a direct sum, the components must independently evaluate to zero: $(C(A)S^{(i)}(A) - R^{(i)}(A))w_i = 0$. This implies that the minimal polynomial of w_i , which is $P_i(z)$, must divide $C(z)S^{(i)}(z) - R^{(i)}(z)$.

Since we previously established that $C(z) \mid P_i(z)$, it follows by transitivity that $C(z)$ must divide $C(z)S^{(i)}(z) - R^{(i)}(z)$. Consequently, $C(z)$ must divide the residual polynomial $R^{(i)}(z)$ in $\mathbb{F}[z]$. $\square$

3.3.3 Failure Probability

Theorem 27. *The probability that the randomized Krylov algorithm fails to extract the Frobenius form of $A \in \mathbb{F}^{n \times n}$ is bounded by $\frac{n}{|\mathbb{F}|}$.*

Proof. The algorithm fails if and only if, at some iteration, the randomly drawn vector x fails to generate the correct cyclic subspace in the quotient V/W . Let $P(z)$ be the true invariant factor of the remaining space. The vector x fails if its minimal polynomial modulo W is a strict divisor of $P(z)$.

We view V/W as a finitely generated torsion module over the principal ideal domain $\mathbb{F}[z]$, where z acts as A . The module decomposes into primary cyclic submodules:

$$V/W \cong \bigoplus_{j=1}^{\ell} \mathbb{F}[z]/\langle Q_j(z)^{e_j} \rangle$$

where $Q_j(z)$ are the distinct monic irreducible factors of $P(z)$, and e_j are their multiplicities.

A random vector $x \in V/W$ fails to have $P(z)$ as its minimal polynomial if and only if it is annihilated by $P(z)/Q_j(z)$ for at least one irreducible factor Q_j . This occurs exactly when the projection of x onto the primary component $\mathbb{F}[z]/\langle Q_j(z)^{e_j} \rangle$ is a multiple of $Q_j(z)$.

Because x is drawn uniformly at random, its projection onto any component is uniformly distributed. The submodule consisting of multiples of $Q_j(z)$ has a vector space codimension equal to $\deg(Q_j)$. Therefore, the probability that x falls into this "bad" submodule is $|\mathbb{F}|^{-\deg(Q_j)}$.

Applying the union bound over all distinct irreducible factors $Q_j \mid P(z)$, the probability that x fails in a given step is bounded by:

$$\mathbb{P}(\text{Failure at step}) \leq \sum_{Q_j \mid P} \frac{1}{|\mathbb{F}|^{\deg(Q_j)}}$$

Because $\sum \deg(P_i) = n$, the worst-case upper bound occurs when the characteristic polynomial splits entirely into n distinct linear factors. In this scenario:

$$\mathbb{P}(\text{Total Failure}) \leq \frac{n}{|\mathbb{F}|}$$

□

3.3.4 Polynomial Matrix Composition and Powering

With the Frobenius form $A = T^{-1}FT$ explicitly constructed, we can evaluate any polynomial function $f(A)$, such as matrix powers A^k .

Module Isomorphism on Cyclic Blocks Let $W_i = \langle w_i \rangle_A$ be a cyclic subspace of dimension d_i with minimal polynomial $P_i(z)$. The subspace is spanned by the Krylov basis $\mathcal{B}_i = \{w_i, Aw_i, \dots, A^{d_i-1}w_i\}$.

There is a natural $\mathbb{F}[z]$ -module isomorphism between W_i and the polynomial quotient ring $\mathbb{F}[z]/\langle P_i(z) \rangle$, given by the mapping:

$$A^j w_i \longleftrightarrow z^j \pmod{P_i(z)}$$

Under this isomorphism, applying the matrix function $f(A)$ to a basis vector $A^j w_i$ corresponds simply to polynomial multiplication in the quotient ring:

$$f(A)(A^j w_i) = A^j f(A)w_i \longleftrightarrow z^j f(z) \pmod{P_i(z)}$$

Constructing the Blocks To construct the matrix block $S_i = f(C_i)$ representing the action of $f(A)$ on the subspace W_i , we need to find the coordinates of $f(A)(A^j w_i)$ in the basis $\mathcal{B}_i$. By the isomorphism above, these coordinates are the coefficients of the polynomial:

$$R_{i,j}(z) = z^j f(z) \pmod{P_i(z)}$$

1. First, computes the base polynomial $R_{i,0}(z) = f(z) \pmod{P_i(z)}$.
2. This polynomial forms the 0-th row (or column depending on preference) of the block S_i .
3. For each subsequent basis vector $j \in \{1, \dots, d_i - 1\}$, the algorithm calculates the next polynomial by shifting the previous one and reducing modulo $P_i(z)$:

$$R_{i,j}(z) = z \cdot R_{i,j-1}(z) \pmod{P_i(z)}$$

This populates the $d_i \times d_i$ block S_i in $O(d_i^2)$ time.

Once all blocks S_i are constructed, they are assembled into the block-diagonal matrix $S = \text{diag}(S_1, \dots, S_m)$, and the final matrix is recovered via the similarity transform $f(A) = T^{-1}ST$.

Theorem 28 (Complexity of Matrix Exponentiation). *Using the Frobenius form, the matrix power A^k can be computed in $O(n^3 + n \log n \log k)$ time.*

Proof. We compute the Frobenius form $A = T^{-1}FT$ in $O(n^3)$ time. For each invariant factor $P_i(z)$, our target function is $f(z) = z^k$. We evaluate the base polynomial $R_{i,0}(z) = z^k \pmod{P_i(z)}$ using binary exponentiation in the polynomial ring $\mathbb{F}[z]/\langle P_i(z) \rangle$ or using specialized algorithms. Because $\deg(P_i) = d_i$, each polynomial multiplication takes $O(d_i \log d_i)$ with NTT. The total time to compute $z^k \pmod{P_i(z)}$ is $O(d_i^2 \log k)$.

Summing this cost over all blocks yields $\sum_i O(d_i \log d_i \log k) \leq O(n \log n \log k)$. Constructing the blocks via shifts takes $\sum_i O(d_i^2) = O(n^2)$ time. Finally, recovering the matrix via $T^{-1}ST$ requires two $O(n^3)$ matrix multiplications. □

3.4 Black Box Linear Algebra

In the following algorithms, we consider a matrix $A \in \mathbb{F}^{n \times n}$ given strictly as a black-box function $f(v) = Av$.

3.4.1 Minimal Polynomial

Algorithm Description To compute the minimal polynomial of A , the algorithm generates two random vectors $u, v \in \mathbb{F}^n$. It then computes the scalar sequence $s_k = u^T A^k v$ for $k = 0, \dots, 2n$. Finally, it applies an algorithm to find the minimal linear recurrence of the sequence s_k , which corresponds to the minimal polynomial of A .

Proof of Correctness Let $A \in \mathbb{F}^{n \times n}$. We define three polynomials:

1. $m_A(x)$: The minimal polynomial of the matrix A .
2. $m_{A,v}(x)$: The minimal polynomial of A with respect to the vector v , which is the monic polynomial of least degree such that $m_{A,v}(A)v = 0$.
3. $m_{u,v}(x)$: The minimal polynomial of the scalar sequence $s_k = u^T A^k v$, which is the monic polynomial of least degree such that $\sum_{i=0}^d c_i s_{k+i} = 0$ for all k .

Divisibility:

Because $m_A(A) = 0$, it follows that $m_A(A)v = 0$, meaning $m_{A,v}(x)$ must divide $m_A(x)$. Similarly, since $m_{A,v}(A)v = 0$, left-multiplying by $u^T A^k$ gives $u^T A^k m_{A,v}(A)v = 0$. This means the sequence s_k satisfies the recurrence defined by $m_{A,v}(x)$, so $m_{u,v}(x)$ must divide $m_{A,v}(x)$. This establishes the chain of divisibility:

$$m_{u,v}(x) \mid m_{A,v}(x) \mid m_A(x)$$

To prove $m_{u,v}(x) = m_A(x)$ with high probability, we bound the probability of a strict division occurring at either step.

Part 1: The Probability that $m_{A,v}(x) = m_A(x)$

Let $m_A(x)$ be factored into its distinct irreducible monic polynomials over $\mathbb{F}$:

$$m_A(x) = p_1(x)^{e_1} p_2(x)^{e_2} \dots p_r(x)^{e_r}$$

where $r \leq n$. For $m_{A,v}(x)$ to be a proper divisor of $m_A(x)$, it must divide the polynomial $q_i(x) = m_A(x)/p_i(x)$ for at least one index $i \in \{1, \dots, r\}$. This happens if and only if $q_i(A)v = 0$, which means v resides in the nullspace of the matrix $q_i(A)$.

Let $W_i = \ker(q_i(A))$. Because $m_A(x)$ is strictly the minimal polynomial of A , the matrix $q_i(A)$ cannot be the zero matrix. Therefore, W_i is a proper subspace of $\mathbb{F}^n$, meaning $\dim(W_i) \leq n - 1$. If v is chosen uniformly at random from $\mathbb{F}^n$, the probability that v falls into a specific proper subspace W_i is at most $1/|\mathbb{F}|$. By the union bound over all r irreducible factors, the probability that v falls into any of these kernels is:

$$P(m_{A,v}(x) \neq m_A(x)) = P\left(v \in \bigcup_{i=1}^r W_i\right) \leq \frac{r}{|\mathbb{F}|} \leq \frac{n}{|\mathbb{F}|}$$

Part 2: The Probability that $m_{u,v}(x) = m_{A,v}(x)$

Assume we have chosen a “good” vector v such that $m_{A,v}(x) = m_A(x)$. We now pick $u \in \mathbb{F}^n$ uniformly at random. By the same logic, for $m_{u,v}(x)$ to be a proper divisor of $m_A(x)$, it must divide $q_i(x) = m_A(x)/p_i(x)$ for some index i . This happens if and only if the sequence generated by $q_i(x)$ evaluates to zero for all k :

$$u^T A^k q_i(A)v = 0 \quad \text{for all } k \geq 0$$

Let $w_i = q_i(A)v$. Because we assumed $m_{A,v}(x) = m_A(x)$, we know that $w_i \neq 0$. The condition $u^T A^k w_i = 0$ for all k means that the vector u must be orthogonal to the Krylov subspace $\mathcal{K} = \text{span}\{w_i, Aw_i, A^2 w_i, \dots\}$.

Since $w_i \neq 0$, the subspace $\mathcal{K}$ has a dimension of at least 1. Therefore, its orthogonal complement $\mathcal{K}^\perp$ is a proper subspace of $\mathbb{F}^n$ with dimension at most $n - 1$. The probability that a uniformly random u falls into $\mathcal{K}^\perp$ for a specific index i is at most $1/|\mathbb{F}|$. Applying the union bound again across the r factors:

$$P(m_{u,v}(x) \neq m_A(x) \mid v \text{ is good}) \leq \frac{r}{|\mathbb{F}|} \leq \frac{n}{|\mathbb{F}|}$$

Conclusion

Combining the two failure probabilities using the union bound, the total probability that the algorithm fails to find the true minimal polynomial is bounded by:

$$P(\text{failure}) \leq \frac{2n}{|\mathbb{F}|}$$

Therefore, for a sufficiently large field (where $|\mathbb{F}| \gg 2n$), the minimal recurrence of the scalar sequence s_k is exactly $m_A(x)$ with high probability.

3.4.2 Determinant

Description To evaluate $\det(A)$ for a matrix $A \in \mathbb{F}^{n \times n}$, the algorithm generates a random diagonal matrix D (represented by its diagonal entries $c_1, \dots, c_n$) and a random vector $v \in \mathbb{F}^n$. It defines the linear map $A' = AD$ and computes its minimal polynomial $m_{AD}(x)$ using a minimal recurrence algorithm on the Krylov sequence $v, A'v, (A')^2v, \dots$. If the degree of $m_{AD}(x)$ is n , the determinant of A is extracted from the polynomial's constant term.

Proof of Correctness The minimal polynomial $m_{AD}(x)$ always divides the characteristic polynomial $p_{AD}(x) = \det(xI - AD)$. If A is singular, AD is singular, and $m_{AD}(x)$ will trivially have a constant term of 0, yielding a correct determinant of 0. Therefore, we assume A is non-singular. If we can prove that AD generically has distinct eigenvalues, then with high probability, its minimal polynomial will have degree n , allowing us to safely assume $m_{AD}(x) = p_{AD}(x)$.

Lemma 29 (Krylov Subspace Dimension). *A matrix $M \in \mathbb{F}^{n \times n}$ is non-derogatory (i.e., its minimal polynomial has degree n) if and only if there exists a vector $y \in \mathbb{F}^n$ such that the Krylov matrix*

$$K_M(y) = \begin{bmatrix} y & My & M^2y & \dots & M^{n-1}y \end{bmatrix}$$

is non-singular.

Proof. By the rational canonical form, the vector space decomposes into cyclic subspaces governed by the invariant factors of M . M is non-derogatory if and only if its largest invariant factor (the minimal polynomial) has degree n . If so, summing the generators of the coprime cyclic subspaces yields a vector y that generates an n -dimensional cyclic subspace, making $K_M(y)$ full rank. Conversely, if $K_M(y)$ is full rank, y generates an n -dimensional cyclic subspace, forcing $\deg(m_M) = n$. $\square$

Let $\mathbf{x} = (x_1, \dots, x_n)$ and $\mathbf{y} = (y_1, \dots, y_n)$ be formal variables, and let $D_{\mathbf{x}} = \text{diag}(x_1, \dots, x_n)$. We construct the formal Krylov matrix $K_{AD_{\mathbf{x}}}(\mathbf{y})$, whose determinant $F(\mathbf{x}, \mathbf{y}) = \det(K_{AD_{\mathbf{x}}}(\mathbf{y}))$ is a multivariate polynomial.

Lemma 30 (Generic Distinct Eigenvalues). *If A is non-singular, the formal polynomial $F(\mathbf{x}, \mathbf{y})$ is not identically zero.*

Proof. Let $B = A^{-1}$. Since $\det(A)$ is a non-zero constant, the roots of $\det(\lambda I - AD_{\mathbf{x}}) = 0$ are distinct if and only if the roots of $Q(\lambda, \mathbf{x}) = \det(\lambda B - D_{\mathbf{x}})$ are distinct. We proceed by induction on n .

Base Case ($n = 1$): $Q(\lambda, x_1) = \lambda B_{11} - x_1$. Since A is invertible, $B_{11} \neq 0$, yielding exactly one root. The discriminant is trivially non-zero.

Inductive Step: Assume the claim holds for $n - 1$. Because B is non-singular, we may permute the coordinates such that the upper-left $(n - 1) \times (n - 1)$ submatrix, B_{n-1} , is invertible. Expanding $Q(\lambda, \mathbf{x})$ along the last diagonal entry yields:

$$Q(\lambda, \mathbf{x}) = Q_0(\lambda, x_1, \dots, x_{n-1}) - x_n Q_1(\lambda, x_1, \dots, x_{n-1})$$

where $Q_1 = \det(\lambda B_{n-1} - D_{n-1})$. By the inductive hypothesis, there exists an assignment $\mathbf{c} \in \mathbb{F}^{n-1}$ such that $Q_1(\lambda, \mathbf{c})$ has $n - 1$ distinct, non-zero roots. Fixing these values, we study the two-variable polynomial $Q(\lambda, x_n) = Q_0(\lambda) - x_n Q_1(\lambda)$.

We construct the reverse polynomial. Let $w = 1/\lambda$, and define:

$$R(w, x_n) = w^n Q(1/w, x_n) = R_0(w) - x_n w R_1(w)$$

Substituting $z = 1/x_n$ and multiplying by z yields:

$$H(w, z) = z R(w, 1/z) = z R_0(w) - w R_1(w)$$

Evaluating at $z = 0$ leaves $H(w, 0) = -w R_1(w)$. Because $Q_1(\lambda)$ has $n - 1$ distinct non-zero roots, its reverse polynomial $R_1(w)$ has $n - 1$ distinct finite roots, none of which are zero. Therefore, $H(w, 0)$ has exactly n distinct roots.

Thus, there exists an assignment for $\mathbf{x}$ yielding n distinct roots, meaning the generic $AD_{\mathbf{x}}$ is non-derogatory and $F(\mathbf{x}, \mathbf{y}) \neq 0$. $\square$

Lemma 31 (Failure Probability). *For random assignments from a finite field $\mathbb{F}$, the probability that the algorithm fails is bounded by $\frac{n(n+1)}{2|\mathbb{F}|}$.*

Proof. To bound the probability of failure, we calculate the total degree of $F(\mathbf{x}, \mathbf{y})$. The k -th column of the Krylov matrix (for $k = 0, \dots, n - 1$) is $(AD_{\mathbf{x}})^k \mathbf{y}$. The entries in this column are homogeneous polynomials of degree k in $\mathbf{x}$ and degree 1 in $\mathbf{y}$. Because the determinant is a sum of products containing exactly one entry from each column, the total degree of F is:

$$\deg(F) = \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} 1 = \frac{n(n-1)}{2} + n = \frac{n(n+1)}{2}$$

By the Schwartz-Zippel Lemma, if we draw the entries for D and v uniformly at random from a finite field $\mathbb{F}$, the probability that they form a root of F is bounded by:

$$\mathbb{P}(F(\mathbf{c}, \mathbf{v}) = 0) \leq \frac{n(n+1)}{2|\mathbb{F}|}$$

Finally, the algorithm extracts the constant term c_0 of the minimal polynomial. Since $p_{AD}(0) = \det(-AD) = (-1)^n \det(A) \det(D)$, we correctly compute the determinant as:

$$\det(A) = \frac{(-1)^n c_0}{\prod_{i=1}^n c_i}$$

$\square$

3.4.3 Wiedemann's Algorithm

Description Wiedemann's algorithm solves the linear system $Ax = b$ for x . It generates a random vector u and computes the scalar sequence $s_k = u^T A^k b$. It finds the minimal polynomial of this sequence, $p(x) = \sum_{i=0}^d p_i x^i$. Assuming A is non-singular, $p_0 \neq 0$. The algorithm then computes the solution vector using Horner's method: $x = -\frac{1}{p_0} \sum_{i=1}^d p_i A^{i-1} b$.

Proof of Correctness Let $m_{A,b}(x)$ be the minimal polynomial of A with respect to the vector b , meaning it is the polynomial of minimal degree such that $m_{A,b}(A)b = 0$. With high probability, the minimal polynomial $p(x)$ of the sequence $u^T A^k b$ (found via linear recurrence finding) is exactly $m_{A,b}(x)$.

By definition, $p(A)b = 0$. Expanding this polynomial expression yields:

$$\sum_{i=0}^d p_i A^i b = 0$$

Because A is non-singular, the constant term of its minimal polynomial is non-zero, so $p_0 \neq 0$. We can isolate the $i = 0$ term:

$$\begin{aligned} p_0 b + \sum_{i=1}^d p_i A^i b &= 0 \\ p_0 b + A \left(\sum_{i=1}^d p_i A^{i-1} b \right) &= 0 \end{aligned}$$

Rearranging to solve for b , we get:

$$A \left[-\frac{1}{p_0} \sum_{i=1}^d p_i A^{i-1} b \right] = b$$

Since $Ax = b$, the unique solution x is the term inside the brackets. To compute x efficiently without storing $O(d)$ dense vectors, the algorithm uses Horner's method, evaluating the sum iteratively from $i = d$ down to 1 via $R \leftarrow A(R) + p_i b$, initializing with $R = p_d b$. After d iterations, R equals the summation, giving the correct solution x using only $O(d)$ matrix-vector multiplications and $O(n)$ space.

3.5 Perron-Frobenius Theorem

The Perron-Frobenius theorem establishes the spectral properties of non-negative matrices. A matrix $A \geq 0$ is irreducible if its directed graph is strongly connected; for any i, j , there exists an integer $k > 0$ such that $(A^k)_{ij} > 0$.

Theorem 32 (Perron-Frobenius for Irreducible Matrices). *Let A be an $n \times n$ non-negative, irreducible matrix. Then:*

1. *A has a real, strictly positive eigenvalue $r > 0$ (the Perron root).*
2. *There exists a strictly positive eigenvector $v > 0$ corresponding to r .*
3. *For any other eigenvalue $\lambda \in \mathbb{C}$ of A , $|\lambda| \leq r$.*
4. *The eigenvalue r is algebraically (and geometrically) simple.*

Proof. Let $\Delta = \{x \in \mathbb{R}^n : x \geq 0, \sum_{i=1}^n x_i = 1\}$ be the simplex. For any non-zero $x \geq 0$, define the function:

$$r(x) = \max\{\rho \geq 0 \mid Ax \geq \rho x\} = \min_{x_i > 0} \frac{(Ax)_i}{x_i}$$

Let $r = \sup_{x \in \Delta} r(x)$.

Because A is irreducible, the matrix $B = (I + A)^{n-1}$ is positive ($B > 0$). Furthermore, B commutes with A . For any $x \in \Delta$, if $Ax \geq \rho x$, then $A(Bx) = B(Ax) \geq \rho(Bx)$, which implies $r(Bx) \geq r(x)$. The map $x \mapsto Bx$ sends Δ into $\mathbb{R}_{>0}^n$. In this interior, $r(x)$ is continuous. Since the image $B(\Delta)$ is compact and contained in the interior, the supremum r is finite and achieved at some positive vector $v = Bx > 0$. Thus, $Av \geq rv$.

$Av = rv$. Suppose for contradiction that $Av \geq rv$ but $Av \neq rv$. This means the vector $z = Av - rv$ is non-negative and non-zero ($z \gneq 0$). Because $B > 0$, multiplying by B yields a positive vector: $Bz > 0$.

Thus, $B(Av - rv) > 0 \implies A(Bv) > r(Bv)$. Since strict inequality holds for all coordinates of Bv , we can find some small $\epsilon > 0$ such that $A(Bv) \geq (r + \epsilon)(Bv)$. This implies $r(Bv) \geq r + \epsilon$, which contradicts the maximality of r . Therefore, we must have $Av = rv$. Since $v > 0$, r must be strictly positive (as $A \neq 0$).

Strict positivity of the eigenvector. Suppose $u \geq 0$ is any non-zero eigenvector for r , meaning $Au = ru$. Then $Bu = (1 + r)^{n-1}u$. Since $u \geq 0$ and $B > 0$, we have $Bu > 0$, forcing $(1 + r)^{n-1}u > 0$, which means u must be strictly positive.

Maximality of r . Let λ be any eigenvalue of A with eigenvector $y \in \mathbb{C}^n$, so $Ay = \lambda y$. Taking absolute values component-wise, and using the triangle inequality alongside the non-negativity of A , we get:

$$|\lambda||y| = |\lambda y| = |Ay| \leq A|y|$$

This implies that the non-negative, non-zero vector $|y|$ satisfies $A|y| \geq |\lambda||y|$. By the definition of our function, $r(|y|) \geq |\lambda|$. Since r is the supremum over all such vectors, $r \geq |\lambda|$.

Simplicity of r . By applying the exact same argument to A^T (which is also irreducible), there exists a strictly positive left eigenvector $w^T > 0$ such that $w^T A = r w^T$.

To show the geometric multiplicity is 1, suppose v and y are two linearly independent eigenvectors for r . Since $v > 0$, we can choose a real scalar c such that $z = v - cy$ has at least one coordinate equal to zero, but z is not the zero vector. However, $Az = rz$. Taking absolute values, $A|z| \geq r|z|$. This forces $A|z| = r|z|$, making $|z|$ a non-negative eigenvector for r . Then $|z|$ must be strictly positive, contradicting that it has a zero coordinate. Thus, the geometric multiplicity is 1.

To prove the algebraic multiplicity is 1, we utilize the strictly positive left eigenvector w^T . If the algebraic multiplicity were strictly greater than 1, there would exist a generalized eigenvector $g \in \ker(A - rI)^2$ such that $(A - rI)g = v$. Multiplying both sides on the left by w^T yields:

$$w^T(A - rI)g = w^T v$$

Because w^T is a left eigenvector associated with r , we have $w^T A = r w^T$, meaning $w^T(A - rI) = 0$. Thus, the left side is 0. However, because both $w > 0$ and $v > 0$, their inner product is strictly positive ($w^T v > 0$). Therefore, r must be an algebraically simple root. $\square$

3.6 Continuity of Eigenvalues

The basic fact is that roots of a polynomial with continuously varying coefficients cannot suddenly appear far away, and any limit of roots of nearby polynomials must still be a root of the limiting polynomial. It is convenient to formulate this for the unordered multiset of roots.

For two unordered n -tuples $[a_1, \dots, a_n]$ and $[b_1, \dots, b_n]$, define

$$d([a_1, \dots, a_n], [b_1, \dots, b_n]) = \min_{\pi \in S_n} \max_i |a_i - b_{\pi(i)}|.$$

This is the natural metric on $\mathbb{C}^n/S_n$.

Theorem 33 (Continuity of Roots). *The unordered multiset of roots of a monic complex polynomial depends continuously on its coefficients.*

Proof. Let

$$p_k(z) = z^n + c_{n-1}^{(k)} z^{n-1} + \dots + c_0^{(k)}$$

converge coefficientwise to

$$p(z) = z^n + c_{n-1} z^{n-1} + \dots + c_0.$$

We prove that the root multisets of p_k converge to the root multiset of p .

First, all roots stay in one fixed compact disk. Indeed, if

$$M_k = \max_{0 \leq j < n} |c_j^{(k)}|,$$

then every root z of p_k satisfies the Cauchy bound

$$|z| \leq 1 + M_k.$$

Since the coefficients converge, the numbers M_k are bounded, so one disk contains every root of every sufficiently large p_k .

Write the roots of p_k , in an arbitrary order, as

$$a_1^{(k)}, \dots, a_n^{(k)}.$$

Consider any subsequence. Because all roots lie in a fixed compact disk, after passing to a further subsequence we may assume

$$a_i^{(k)} \longrightarrow a_i \quad (i = 1, \dots, n).$$

Then for every z ,

$$p_k(z) = \prod_{i=1}^n (z - a_i^{(k)}) \longrightarrow \prod_{i=1}^n (z - a_i).$$

On the other hand coefficientwise convergence gives $p_k(z) \rightarrow p(z)$. Hence

$$p(z) = \prod_{i=1}^n (z - a_i).$$

Thus every subsequential limit of the root multiset is exactly the root multiset of p , with multiplicity.

If the root multisets did not converge to the roots of p , there would be an $\varepsilon > 0$ and a subsequence remaining at distance at least ε from that multiset. The preceding compactness argument would produce a further subsequence converging to the root multiset of p , a contradiction. Therefore the roots depend continuously on the coefficients. $\square$

Corollary 34 (Continuity of Eigenvalues). *Let $A \in M_n(\mathbb{C})$. The unordered multiset of eigenvalues of A depends continuously on the entries of A .*

Proof. The coefficients of

$$p_A(z) = \det(zI - A)$$

are polynomials in the entries of A , hence depend continuously on A . The eigenvalues are precisely the roots of p_A , so the result follows from the preceding theorem. $\square$

3.7 Gershgorin's Circle Theorem

Gershgorin's theorem localizes the eigenvalues of a matrix using only its entries.

The basic idea is extremely simple. If

$$Ax = \lambda x,$$

choose a coordinate of the eigenvector which is largest in magnitude. In that coordinate, the diagonal term must almost account for the eigenvalue, because all other coordinates are no larger.

It is essentially no harder to allow a different scale for each coordinate. This gives a useful weighted version of the theorem.

Theorem 35 (Weighted Gershgorin Circle Theorem). *Let*

$$A = (a_{ij}) \in \mathbb{C}^{n \times n},$$

and let

$$w_1, \dots, w_n > 0.$$

For each i , define the weighted row radius

$$R_i(w) = \sum_{j \neq i} |a_{ij}| \frac{w_j}{w_i}$$

and the disk

$$D_i(w) = \{z \in \mathbb{C} : |z - a_{ii}| \leq R_i(w)\}.$$

Then every eigenvalue of A lies in

$$\sigma(A) \subseteq \bigcup_{i=1}^n D_i(w).$$

Moreover, suppose $I \subseteq [n]$ is such that

$$\bigcup_{i \in I} D_i(w)$$

is disjoint from

$$\bigcup_{i \notin I} D_i(w).$$

Then the first union contains exactly

$$|I|$$

eigenvalues of A , counted with algebraic multiplicity.

Proof. We first prove the localization statement.

Let

$$Ax = \lambda x, \quad x \neq 0.$$

Choose i maximizing

$$\frac{|x_i|}{w_i}.$$

In particular, $x_i \neq 0$, and for every j ,

$$|x_j| \leq \frac{w_j}{w_i} |x_i|.$$

The i -th coordinate of $Ax = \lambda x$ gives

$$(\lambda - a_{ii})x_i = \sum_{j \neq i} a_{ij}x_j.$$

Taking absolute values,

$$|\lambda - a_{ii}| |x_i| \leq \sum_{j \neq i} |a_{ij}| |x_j| \leq \sum_{j \neq i} |a_{ij}| \frac{w_j}{w_i} |x_i|.$$

Canceling $|x_i|$,

$$|\lambda - a_{ii}| \leq R_i(w).$$

Thus

$$\lambda \in D_i(w).$$

For the counting statement, write

$$A = D + E,$$

where

$$D = \text{diag}(a_{11}, \dots, a_{nn})$$

and E has zero diagonal. Consider the (continuous) family

$$A(t) = D + tE, \quad 0 \leq t \leq 1.$$

Its weighted Gershgorin disks are

$$D_i(t) = \{z : |z - a_{ii}| \leq tR_i(w)\}.$$

Since

$$D_i(t) \subseteq D_i(1),$$

the two unions

$$\mathcal{D}_I(t) = \bigcup_{i \in I} D_i(t), \quad \mathcal{D}_{\bar{I}}(t) = \bigcup_{i \notin I} D_i(t)$$

remain disjoint for every $t \in [0, 1]$.

By the first part, every eigenvalue of $A(t)$ belongs to one of these two sets.

Because the two original unions are compact and disjoint, they are separated by some positive distance. The eigenvalues depend continuously on the entries of the matrix by [Corollary 34](#), so as t changes, no eigenvalue can pass from one union to the other without crossing the gap between them. Consequently, the number of eigenvalues in $\mathcal{D}_I(t)$, counted with algebraic multiplicity, is constant in t .

At $t = 0$,

$$A(0) = D,$$

whose eigenvalues are exactly

$$a_{11}, \dots, a_{nn}.$$

The union $\mathcal{D}_I(0)$ contains precisely the diagonal entries

$$a_{ii}, \quad i \in I,$$

and hence exactly $|I|$ eigenvalues, with multiplicity.

The same is therefore true at $t = 1$, where $A(1) = A$. □

The usual Gershgorin theorem is the special case

$$w_1 = \dots = w_n = 1.$$

Thus, writing

$$R_i = \sum_{j \neq i} |a_{ij}|,$$

every eigenvalue lies in one of the ordinary Gershgorin disks

$$\boxed{D_i = \{z : |z - a_{ii}| \leq R_i\}}.$$

The weighted form can also be viewed as ordinary Gershgorin after a diagonal change of coordinates. Indeed, if

$$W = \text{diag}(w_1, \dots, w_n),$$

then

$$W^{-1}AW$$

is similar to A , while its ij -entry is

$$a_{ij} \frac{w_j}{w_i}.$$

Thus the weights simply choose the coordinate scaling in which we apply Gershgorin.

Corollary 36 (Second Gershgorin Theorem). *Every connected component of*

$$\bigcup_{i=1}^n D_i$$

which consists of k Gershgorin disks contains exactly k eigenvalues, counted with algebraic multiplicity.

Proof. Each disk is connected, so every disk lies entirely within one connected component of the union. A connected component is therefore the union of some subset I of the disks.

Since there are finitely many disks, distinct components are compact and disjoint. Apply the counting part of [Theorem 35](#). $\square$

There is an analogous column version. Define

$$C_i = \sum_{j \neq i} |a_{ji}|.$$

Since A and A^T have the same characteristic polynomial, applying Gershgorin to A^T gives

$$\sigma(A) \subseteq \bigcup_i \{z : |z - a_{ii}| \leq C_i\}.$$

Hence, using both row and column information,

$$\sigma(A) \subseteq \left(\bigcup_i D_i^{\text{row}} \right) \cap \left(\bigcup_i D_i^{\text{col}} \right).$$

For Hermitian matrices all eigenvalues are real, so Gershgorin becomes the interval bound

$$\sigma(A) \subseteq \bigcup_i [a_{ii} - R_i, a_{ii} + R_i].$$

In particular, if A is Hermitian and

$$a_{ii} > R_i \quad \text{for every } i,$$

then every eigenvalue is positive, and hence

$$A \succ 0.$$

More generally, if

$$|a_{ii}| > R_i \quad \text{for every } i,$$

then 0 belongs to no Gershgorin disk, so A is nonsingular.

4 Number Theory and Arithmetic

4.1 Bertrand's Postulate

For $x \geq 1$, write

$$P(x) := \prod_{p \leq x} p$$

for the product of the primes at most x .

We first need a crude upper bound on $P(x)$.

Lemma 37. For every integer $n \geq 1$,

$$P(n) \leq 4^n.$$

Proof. We induct on n .

If $n \geq 4$ is even, then n is composite, so

$$P(n) = P(n-1) \leq 4^{n-1} < 4^n.$$

Now let $n = 2m + 1$ be odd. Every prime

$$m + 1 < p \leq 2m + 1$$

divides

$$\binom{2m+1}{m},$$

since it appears in the numerator

$$(m+2)(m+3) \cdots (2m+1)$$

and not in $m!$. Hence

$$P(2m+1) \leq P(m+1) \binom{2m+1}{m}.$$

By induction and the binomial theorem,

$$P(2m+1) \leq 4^{m+1} 2^{2m} = 4^{2m+1}.$$

The cases $n \leq 2$ are immediate. □

Lemma 38. If

$$p^e \mid \binom{2n}{n},$$

then

$$p^e \leq 2n.$$

Moreover, if $p > \sqrt{2n}$, then $e \leq 1$.

Proof. By Legendre's formula,

$$e = \sum_{j \geq 1} \left(\left\lfloor \frac{2n}{p^j} \right\rfloor - 2 \left\lfloor \frac{n}{p^j} \right\rfloor \right).$$

Every summand is either 0 or 1, and it vanishes once $p^j > 2n$. Thus

$$e \leq \lfloor \log_p(2n) \rfloor,$$

so $p^e \leq 2n$.

If $p > \sqrt{2n}$, then $p^2 > 2n$, so only the $j = 1$ term can be nonzero. □

Theorem 39 (Bertrand's Postulate). For every integer $n \geq 2$, there is a prime p satisfying

$$n < p < 2n.$$

Proof. First suppose $n \geq 468$, and assume for contradiction that there is no prime in $(n, 2n)$.

Set

$$B = \binom{2n}{n}.$$

Since B is the largest coefficient of $(1 + 1)^{2n}$,

$$B \geq \frac{4^n}{2n + 1}.$$

A prime p with

$$\frac{2n}{3} < p \leq n$$

does not divide B : since $n \geq 468$, we have $p > \sqrt{2n}$, and

$$v_p(B) = \left\lfloor \frac{2n}{p} \right\rfloor - 2 \left\lfloor \frac{n}{p} \right\rfloor = 2 - 2 = 0.$$

By assumption there are also no prime factors in $(n, 2n)$.

Thus every prime factor of B is at most $2n/3$.

For primes $p \leq \sqrt{2n}$, [Lemma 38](#) says that their entire prime-power contribution to B is at most $2n$. There are at most $\sqrt{2n}$ such primes, so their total contribution is at most

$$(2n)^{\sqrt{2n}}.$$

For larger primes, the exponent is at most one. Hence

$$B \leq (2n)^{\sqrt{2n}} \prod_{p \leq 2n/3} p.$$

By [Lemma 37](#),

$$B \leq (2n)^{\sqrt{2n}} 4^{2n/3}.$$

Combining the upper and lower bounds gives

$$\frac{4^n}{2n + 1} \leq (2n)^{\sqrt{2n}} 4^{2n/3},$$

or

$$\frac{n}{3} \log 4 \leq \log(2n + 1) + \sqrt{2n} \log(2n).$$

A direct check shows that the reverse strict inequality holds at $n = 468$ and its gap is increasing thereafter. This is a contradiction.

It remains only to handle $2 \leq n < 468$. The prime sequence

$$2, 3, 5, 7, 13, 23, 43, 83, 163, 317, 631$$

has the property that every term after the first is less than twice the preceding term. Therefore, if q is the first term greater than n , then

$$n < q < 2n.$$

□

4.2 Product of Small Primes

The following crude bound is useful when a collection of small primes is needed whose product exceeds a given integer.

Theorem 40. *For every integer $N \geq 2$,*

$$\prod_{p \leq 16 \ln N} p > N.$$

Consequently the same statement holds if $\log$ denotes logarithm base 2.

Proof. Write

$$\vartheta(x) = \sum_{p \leq x} \ln p = \ln \prod_{p \leq x} p.$$

We first prove that

$$\vartheta(x) > \frac{x}{16} \quad (x \geq 100).$$

Let m be the largest even integer at most x . Thus

$$m > x - 2 \quad \text{and} \quad m \geq 100.$$

Set

$$B = \binom{m}{m/2}.$$

Again,

$$B \geq \frac{2^m}{m+1}.$$

Factor $B = \prod p^{e_p}$. For every $p \leq \sqrt{m}$, [Lemma 38](#) gives

$$p^{e_p} \leq m,$$

so all such prime powers contribute at most

$$m^{\sqrt{m}}.$$

If $p > \sqrt{m}$, then $e_p \leq 1$, so the remaining contribution is at most

$$\prod_{p \leq m} p = e^{\vartheta(m)}.$$

Therefore

$$\frac{2^m}{m+1} \leq B \leq m^{\sqrt{m}} e^{\vartheta(m)},$$

and hence

$$\vartheta(m) \geq m \ln 2 - \ln(m+1) - \sqrt{m} \ln m.$$

For $m \geq 100$, the elementary inequalities

$$\ln m \leq \frac{\sqrt{m}}{2}, \quad \ln(m+1) \leq \frac{m}{16}, \quad \ln 2 > \frac{2}{3}$$

give

$$\vartheta(m) > \frac{2m}{3} - \frac{m}{16} - \frac{m}{2} = \frac{5m}{48}.$$

Since $m > x - 2$ and $x \geq 100$,

$$\vartheta(x) \geq \vartheta(m) > \frac{5(x-2)}{48} > \frac{x}{16}.$$

Now set

$$x = 16 \ln N.$$

If $x \geq 100$, then

$$\ln \prod_{p \leq x} p = \vartheta(x) > \frac{x}{16} = \ln N,$$

so the desired product is greater than N .

It remains only to handle $x < 100$.

If $N < 210$, then

$$x = 16 \ln N \geq 16 \ln 2 > 8 > 7,$$

so

$$\prod_{p \leq x} p \geq 2 \cdot 3 \cdot 5 \cdot 7 = 210 > N.$$

Otherwise $N \geq 210$. Then

$$x \geq 16 \ln 210 > 16 > 11,$$

so

$$\prod_{p \leq x} p \geq 2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 = 2310.$$

Since we are still in the case $x < 100$,

$$N = e^{x/16} < e^{25/4} < 3^7 = 2187 < 2310.$$

Thus the result holds in all cases. □

4.3 Lucas's Theorem

Theorem 41 (Lucas's Theorem). *Let p be prime, and write the base- p expansions*

$$n = n_0 + n_1p + \cdots + n_rp^r, \quad k = k_0 + k_1p + \cdots + k_rp^r,$$

where $0 \leq n_i, k_i < p$. Then

$$\binom{n}{k} \equiv \prod_{i=0}^r \binom{n_i}{k_i} \pmod{p}.$$

Proof. Over $\mathbb{F}_p$, the binomial theorem gives

$$(1+x)^p = \sum_{j=0}^p \binom{p}{j} x^j \equiv 1 + x^p \pmod{p},$$

since $p \mid \binom{p}{j}$ for $0 < j < p$. Iterating,

$$(1+x)^{p^i} \equiv 1 + x^{p^i} \pmod{p}.$$

Therefore

$$\begin{aligned} (1+x)^n &= \prod_i (1+x)^{n_i p^i} \\ &\equiv \prod_i (1+x^{p^i})^{n_i} \pmod{p}. \end{aligned}$$

Expanding the right-hand side,

$$\prod_i \left(\sum_{j=0}^{n_i} \binom{n_i}{j} x^{j p^i} \right).$$

Since every $j < p$, the exponent

$$k = \sum_i k_i p^i$$

has a unique base- p representation. Thus the only way to obtain x^k is to choose $x^{k_i p^i}$ from the i -th factor. Hence

$$[x^k](1+x)^n \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

Since $[x^k](1+x)^n = \binom{n}{k}$, the result follows. □

Corollary 42. *With the same notation,*

$$\binom{n}{k} \not\equiv 0 \pmod{p} \iff k_i \leq n_i \text{ for every } i.$$

Consequently, the number of entries in the n -th row of Pascal's triangle that are non-zero modulo p is

$$\prod_i (n_i + 1).$$

Proof. Because $0 \leq n_i, k_i < p$,

$$\binom{n_i}{k_i} \not\equiv 0 \pmod{p}$$

exactly when $k_i \leq n_i$. Lucas's theorem gives the first claim. For the second, each digit k_i may independently be chosen from

$$0, 1, \dots, n_i.$$

□

4.4 Quadratic Reciprocity

For an odd prime p , define the Legendre symbol by

$$\left(\frac{a}{p}\right) = \begin{cases} 0, & p \mid a, \\ 1, & a \not\equiv 0 \pmod{p} \text{ is a square,} \\ -1, & a \text{ is not a square.} \end{cases}$$

We first prove the fact that will give both quadratic reciprocity and its supplementary laws.

Lemma 43 (Euler's Criterion). *For p an odd prime,*

$$a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}.$$

Proof. Assume $a \not\equiv 0 \pmod{p}$. There are exactly $(p-1)/2$ non-zero squares modulo p , since x and $-x$ have the same square.

Every square $a = x^2$ satisfies

$$a^{(p-1)/2} = x^{p-1} \equiv 1 \pmod{p}.$$

Thus the $(p-1)/2$ non-zero squares are exactly the roots of

$$x^{(p-1)/2} - 1.$$

If a is not a square, then $a^{(p-1)/2} \not\equiv 1$, while Fermat's little theorem gives

$$\left(a^{(p-1)/2}\right)^2 = a^{p-1} \equiv 1.$$

Hence $a^{(p-1)/2} \equiv -1$.

□

Lemma 44 (Gauss's Lemma). *Let p be an odd prime, $p \nmid a$, and set $m = (p-1)/2$. Consider*

$$a, 2a, \dots, ma$$

modulo p , represented by integers in $\{1, \dots, p-1\}$. Let N be the number of these residues that lie in $(p/2, p)$. Then

$$\left(\frac{a}{p}\right) = (-1)^N.$$

Proof. For each $1 \leq j \leq m$, replace the residue of aj by its representative up to sign in

$$\{1, \dots, m\}.$$

These absolute representatives are all distinct: if

$$aj \equiv \pm ak \pmod{p},$$

then

$$j \equiv \pm k \pmod{p},$$

and for $1 \leq j, k \leq m$ this forces $j = k$.

Thus they are exactly a permutation of $1, \dots, m$. A residue contributes a minus sign precisely when it lies in $(p/2, p)$, so

$$a^m m! \equiv (-1)^N m! \pmod{p}.$$

Canceling $m!$ gives

$$a^{(p-1)/2} \equiv (-1)^N \pmod{p}.$$

Now apply [Lemma 43](#). □

Corollary 45 (First Supplement). *For every odd prime p ,*

$$\left(\frac{-1}{p}\right) = (-1)^{(p-1)/2}.$$

Equivalently, -1 is a quadratic residue modulo p if and only if

$$p \equiv 1 \pmod{4}.$$

Proof. Multiplication by -1 sends every

$$1, \dots, \frac{p-1}{2}$$

to a residue greater than $p/2$. Thus $N = (p-1)/2$ in Gauss's lemma. □

Corollary 46 (Second Supplement). *For every odd prime p ,*

$$\left(\frac{2}{p}\right) = (-1)^{(p^2-1)/8}.$$

Equivalently, 2 is a quadratic residue modulo p exactly when

$$p \equiv \pm 1 \pmod{8}.$$

Proof. For

$$1 \leq j \leq \frac{p-1}{2},$$

the numbers $2j$ already lie in $\{1, \dots, p-1\}$. They lie above $p/2$ exactly when

$$j > \frac{p}{4}.$$

Hence

$$N = \frac{p-1}{2} - \left\lfloor \frac{p}{4} \right\rfloor.$$

Checking the four possibilities $p \equiv 1, 3, 5, 7 \pmod{8}$, this number is even for $p \equiv 1, 7 \pmod{8}$ and odd for $p \equiv 3, 5 \pmod{8}$. Thus

$$(-1)^N = (-1)^{(p^2-1)/8}.$$

Now apply Gauss's lemma. □

For quadratic reciprocity, it is useful to rewrite Gauss's lemma as a lattice-point count.

Lemma 47 (Eisenstein's Form of Gauss's Lemma). *Let p be an odd prime and let a be odd with $p \nmid a$. Then*

$$\left(\frac{a}{p}\right) = (-1)^{\sum_{j=1}^{(p-1)/2} \left\lfloor \frac{aj}{p} \right\rfloor}.$$

Proof. Set $m = (p-1)/2$ and write

$$aj = pq_j + r_j, \quad 1 \leq r_j \leq p-1.$$

Let

$$s_j = \min(r_j, p - r_j) \in \{1, \dots, m\}.$$

As in the proof of Gauss's lemma, the s_j are a permutation of $1, \dots, m$.

Let $\epsilon_j = 0$ if $r_j \leq m$ and $\epsilon_j = 1$ otherwise. Since a and p are odd, reducing

$$aj = pq_j + r_j$$

modulo 2 gives

$$j \equiv q_j + s_j + \epsilon_j \pmod{2}.$$

Summing over j , the sums of the j 's and the s_j 's cancel modulo 2, since (s_j) is a permutation of (j) . Therefore

$$\sum_j q_j \equiv \sum_j \epsilon_j = N \pmod{2}.$$

Since $q_j = \lfloor aj/p \rfloor$, Gauss's lemma gives the result. □

Theorem 48 (Quadratic Reciprocity). *For distinct odd primes p and q ,*

$$\left(\frac{p}{q}\right) \left(\frac{q}{p}\right) = (-1)^{\frac{(p-1)(q-1)}{4}}.$$

Equivalently,

$$\left(\frac{p}{q}\right) = \left(\frac{q}{p}\right)$$

unless

$$p \equiv q \equiv 3 \pmod{4},$$

in which case the two symbols have opposite signs.

Proof. Set

$$m = \frac{p-1}{2}, \quad n = \frac{q-1}{2}.$$

By [Lemma 47](#),

$$\left(\frac{q}{p}\right) = (-1)^A, \quad A = \sum_{x=1}^m \left\lfloor \frac{qx}{p} \right\rfloor,$$

and

$$\left(\frac{p}{q}\right) = (-1)^B, \quad B = \sum_{y=1}^n \left\lfloor \frac{py}{q} \right\rfloor.$$

Consider the mn lattice points

$$(x, y), \quad 1 \leq x \leq m, \quad 1 \leq y \leq n.$$

The number below the line

$$py = qx$$

is exactly

$$A,$$

because for a fixed x there are

$$\left\lfloor \frac{qx}{p} \right\rfloor$$

possible positive y .

Similarly, the number above the line is exactly B .

No lattice point lies on the line: $py = qx$ would imply $p \mid x$, impossible since $1 \leq x < p$. Thus every point is either above or below, and

$$A + B = mn = \frac{(p-1)(q-1)}{4}.$$

Consequently,

$$\left(\frac{q}{p}\right) \left(\frac{p}{q}\right) = (-1)^{A+B} = (-1)^{\frac{(p-1)(q-1)}{4}}.$$

□

4.5 Values of the Zeta Function at Even Integers

The formula for

$$\zeta(2k) = \sum_{n=1}^{\infty} \frac{1}{n^{2k}}$$

comes from expanding the same function in two different ways.

Expanding the partial fractions of $\cot(\pi z)$ produces the sums $\zeta(2k)$.

On the other hand, $\cot z$ can be written using the exponential function, whose expansion is naturally encoded by the Bernoulli numbers.

Define the Bernoulli numbers by

$$\frac{t}{e^t - 1} = \sum_{m=0}^{\infty} B_m \frac{t^m}{m!}.$$

Thus

$$B_0 = 1, \quad B_1 = -\frac{1}{2}.$$

Theorem 49 (Euler). *For every integer $k \geq 1$,*

$$\zeta(2k) = (-1)^{k+1} \frac{B_{2k}(2\pi)^{2k}}{2(2k)!}.$$

Proof. We compute the power series of

$$\pi z \cot(\pi z)$$

in two ways.

From the poles of cotangent. The partial-fraction expansion

$$\pi \cot(\pi z) = \frac{1}{z} + \sum_{n=1}^{\infty} \frac{2z}{z^2 - n^2}$$

gives

$$\pi z \cot(\pi z) = 1 - 2 \sum_{n=1}^{\infty} \frac{z^2}{n^2 - z^2}.$$

For $|z| < 1$,

$$\frac{z^2}{n^2 - z^2} = \frac{z^2/n^2}{1 - z^2/n^2} = \sum_{k=1}^{\infty} \frac{z^{2k}}{n^{2k}}.$$

Hence

$$\begin{aligned} \pi z \cot(\pi z) &= 1 - 2 \sum_{n=1}^{\infty} \sum_{k=1}^{\infty} \frac{z^{2k}}{n^{2k}} \\ &= 1 - 2 \sum_{k=1}^{\infty} \left(\sum_{n=1}^{\infty} \frac{1}{n^{2k}} \right) z^{2k}. \end{aligned}$$

Therefore

$$\boxed{\pi z \cot(\pi z) = 1 - 2 \sum_{k=1}^{\infty} \zeta(2k) z^{2k}.} \quad (1)$$

The interchange of the sums is justified for $|z| < 1$, where the series is absolutely convergent.

From the Bernoulli generating function. Using

$$\cot x = i \frac{e^{2ix} + 1}{e^{2ix} - 1} = i + \frac{2i}{e^{2ix} - 1},$$

we obtain

$$x \cot x = ix + \frac{2ix}{e^{2ix} - 1}.$$

Set

$$t = 2ix.$$

By the Bernoulli generating function,

$$\frac{2ix}{e^{2ix} - 1} = \sum_{m=0}^{\infty} B_m \frac{(2ix)^m}{m!}.$$

Thus

$$x \cot x = ix + \sum_{m=0}^{\infty} B_m \frac{(2ix)^m}{m!}.$$

The $m = 1$ term is

$$B_1(2ix) = -\frac{1}{2}(2ix) = -ix,$$

so it cancels the initial ix .

Moreover, $x \cot x$ is even, so all remaining odd coefficients vanish. Thus

$$x \cot x = \sum_{k=0}^{\infty} B_{2k} \frac{(2ix)^{2k}}{(2k)!}.$$

Since

$$i^{2k} = (-1)^k,$$

this becomes

$$x \cot x = \sum_{k=0}^{\infty} (-1)^k \frac{2^{2k} B_{2k}}{(2k)!} x^{2k}.$$

Substituting $x = \pi z$,

$$\pi z \cot(\pi z) = \sum_{k=0}^{\infty} (-1)^k \frac{B_{2k}(2\pi)^{2k}}{(2k)!} z^{2k}. \quad (2)$$

Finally compare the coefficient of z^{2k} in (1) and (2). For $k \geq 1$,

$$-2\zeta(2k) = (-1)^k \frac{B_{2k}(2\pi)^{2k}}{(2k)!}.$$

Therefore

$$\zeta(2k) = (-1)^{k+1} \frac{B_{2k}(2\pi)^{2k}}{2(2k)!}.$$

□

4.6 Arithmetic Algorithms

4.6.1 Linear Sieve

The linear sieve is distinguished from the ordinary Eratosthenes sieve by the fact that every composite integer is generated exactly once.

This also makes it convenient for evaluating arbitrary multiplicative functions.

Suppose f is multiplicative. We assume access to two operations:

- PrimePower(p, e) computes $f(p^e)$ at cost G ;
- Combine($f(a), f(b)$) computes $f(ab)$ when $\gcd(a, b) = 1$, at cost F .

For each integer i , let

$$\text{lp}[i]$$

be its least prime factor. We also maintain

$$e[i] = v_{\text{lp}[i]}(i), \quad q[i] = \text{lp}[i]^{e[i]}.$$

Thus

$$i = q[i] \cdot r \quad \text{with} \quad \gcd(q[i], r) = 1.$$

Algorithm 1 Linear Sieve for Multiplicative Functions

Input: Integer n

Output: Primes below n , least prime factors, and $f(1), \dots, f(n-1)$

```
1:  $f[1] \leftarrow f(1)$ 
2:  $primes \leftarrow \emptyset$ 
3: Initialize  $lp, e, q$  to zero
4: for  $i = 2, \dots, n-1$  do
5:   if  $lp[i] = 0$  then
6:      $lp[i] \leftarrow i$ 
7:      $e[i] \leftarrow 1$ 
8:      $q[i] \leftarrow i$ 
9:     append  $i$  to  $primes$ 
10:     $f[i] \leftarrow \text{PrimePower}(i, 1)$ 
11:    for  $p$  in  $primes$  do
12:      if  $ip \geq n$  then
13:        break
14:       $lp[ip] \leftarrow p$ 
15:      if  $p = lp[i]$  then
16:         $e[ip] \leftarrow e[i] + 1$ 
17:         $q[ip] \leftarrow q[i]p$ 
18:        if  $q[ip] = ip$  then
19:           $f[ip] \leftarrow \text{PrimePower}(p, e[ip])$ 
20:        else
21:           $f[ip] \leftarrow \text{Combine}(f[ip/q[ip]], f[q[ip]])$ 
22:        break
23:      else
24:         $e[ip] \leftarrow 1$ 
25:         $q[ip] \leftarrow p$ 
26:         $f[ip] \leftarrow \text{Combine}(f[i], f[p])$ 
```

Lemma 50. Every composite integer $x < n$ is generated exactly once by *Algorithm 1*.

Proof. Let

$$p = lp[x], \quad i = x/p.$$

Then every prime factor of i is at least p , so

$$p \leq lp[i].$$

Therefore the inner loop for i reaches p before it terminates, and generates $x = ip$.

Conversely, the sieve only generates ip while

$$p \leq lp[i].$$

Thus p is the least prime factor of ip . Hence if x is generated as ip , necessarily

$$p = lp[x], \quad i = x/p,$$

which is unique. □

Theorem 51. *Algorithm 1* correctly computes any multiplicative function f on $[1, n)$ in

$$O\left(n + nF + \frac{n}{\log n}G\right)$$

time.

Proof. By the preceding lemma, the total number of inner-loop iterations is $O(n)$.

Whenever $p < \text{lp}[i]$, we have $\gcd(i, p) = 1$, so

$$f(ip) = \text{Combine}(f(i), f(p)).$$

When $p = \text{lp}[i]$, write

$$i = p^e r, \quad \gcd(p, r) = 1.$$

Then

$$ip = p^{e+1} r,$$

and the algorithm computes

$$f(ip) = \text{Combine}(f(r), f(p^{e+1})).$$

The value $f(p^{e+1})$ has already been computed unless $ip = p^{e+1}$ itself is a pure prime power; in that case it is computed directly by PrimePower.

Thus PrimePower is called exactly once for each prime power below n .

There are

$$\pi(n) + O(\sqrt{n} \log n) = O\left(\frac{n}{\log n}\right)$$

prime powers below n . Indeed, the powers with exponent at least two contribute $O(\sqrt{n} \log n)$, while

$$\pi(n) = O\left(\frac{n}{\log n}\right)$$

follows already from [Lemma 37](#): since

$$\sum_{p \leq n} \log p \leq n \log 4,$$

the primes larger than $\sqrt{n}$ number at most

$$\frac{2n \log 4}{\log n},$$

and there are at most $\sqrt{n}$ smaller primes.

Hence there are $O(n)$ calls to Combine and $O(n/\log n)$ calls to PrimePower, giving

$$O\left(n + nF + \frac{n}{\log n} G\right).$$

□

For ordinary multiplicative functions, Combine is simply multiplication. The only function-specific information needed is therefore the prime-power formula. For example,

$$\varphi(p^e) = p^e - p^{e-1}, \quad \mu(p^e) = \begin{cases} -1, & e = 1, \\ 0, & e \geq 2, \end{cases}$$

and

$$\sigma_k(p^e) = 1 + p^k + p^{2k} + \cdots + p^{ek}.$$

4.6.2 Extended GCD with Preprocessing

Given integers a, b , we want integers x, y satisfying

$$ax + by = \gcd(a, b).$$

Ordinary extended Euclid takes $O(\log N)$ arithmetic operations for $|a|, |b| \leq N$. With preprocessing, we can batch many Euclidean steps.

The basic observation is that after scaling a pair (u, v) to a fixed box, we only have to consider $O(T)$ reductions: one coordinate lies on the boundary of the box.

For an integer $T \geq 2$, define

$$\mathcal{B}_T = \{(p, q) \in \mathbb{Z}^2 : \max(|p|, |q|) = T\}.$$

Then

$$|\mathcal{B}_T| = O(T).$$

Lemma 52 (Short boundary reduction). *For every $(p, q) \in \mathcal{B}_T$, there is a unimodular matrix $R \in \text{SL}_2(\mathbb{Z})$ whose entries have magnitude $O(\sqrt{T})$ and such that the first coordinate of*

$$R \begin{pmatrix} p \\ q \end{pmatrix}$$

has magnitude $O(\sqrt{T})$.

Proof. After swapping coordinates and changing signs, assume $|q| = T$.

Set

$$Q = \lceil \sqrt{T} \rceil.$$

Consider the $Q + 1$ fractional parts

$$0, \frac{p}{q}, \frac{2p}{q}, \dots, \frac{Qp}{q}.$$

Partition $[0, 1)$ into Q intervals of length $1/Q$. Two fractional parts lie in the same interval, so for some $1 \leq r \leq Q$ and some integer s ,

$$|rp - sq| \leq \frac{|q|}{Q} = O(\sqrt{T}).$$

Also,

$$|s| \leq \left\lceil \frac{rp}{q} \right\rceil + 1 = O(\sqrt{T}).$$

Dividing (r, s) by $\gcd(r, s)$ only makes these bounds smaller, so assume $\gcd(r, s) = 1$. Then the row vector $(r, -s)$ can be completed to a unimodular matrix

$$R = \begin{pmatrix} r & -s \\ u & v \end{pmatrix}.$$

The Bézout coefficients u, v may be chosen with magnitude $O(\max(|r|, |s|)) = O(\sqrt{T})$.

Its first output coordinate is

$$rp - sq = O(\sqrt{T}),$$

as required. □

Precompute one such matrix $R_{p,q}$ for every $(p, q) \in \mathcal{B}_T$. The table contains only $O(T)$ matrices.

Lemma 53 (One lookup shrinks the pair). *Let*

$$z = \begin{pmatrix} u \\ v \end{pmatrix} \neq 0, \quad B = \max(|u|, |v|).$$

Using one lookup in the boundary table followed by one Euclidean division, we can apply a unimodular transformation sending z to z' with

$$\|z'\|_\infty = O\left(\frac{B}{\sqrt{T}}\right).$$

Proof. Round the normalized pair to the boundary:

$$p = \left\lfloor \frac{Tu}{B} \right\rfloor, \quad q = \left\lfloor \frac{Tv}{B} \right\rfloor.$$

Because one of $|u|, |v|$ equals B , one of $|p|, |q|$ equals T , so

$$(p, q) \in \mathcal{B}_T.$$

Moreover,

$$z = \frac{B}{T} \begin{pmatrix} p \\ q \end{pmatrix} + e, \quad \|e\|_\infty = O(B/T).$$

Let $R = R_{p,q}$. By [Lemma 52](#), the first coordinate of $R(p, q)^T$ is $O(\sqrt{T})$ and every entry of R is $O(\sqrt{T})$. Thus the first coordinate of Rz is

$$O\left(\frac{B}{T}\sqrt{T}\right) + O\left(\sqrt{T}\frac{B}{T}\right) = O\left(\frac{B}{\sqrt{T}}\right).$$

Call this coordinate w_1 and the other coordinate w_2 . If $w_1 = 0$, the gcd has already been computed. Otherwise choose

$$Q = \left\lfloor \frac{w_2}{w_1} \right\rfloor$$

and apply

$$\begin{pmatrix} 1 & 0 \\ -Q & 1 \end{pmatrix}.$$

The second coordinate becomes

$$w_2 - Qw_1$$

and has magnitude at most $|w_1|/2$. Hence both coordinates are

$$O(B/\sqrt{T}).$$

□

The extended coefficients are maintained simply by accumulating the unimodular transformations.

Algorithm 2 Preprocessed Extended GCD

Input: Integers a, b , boundary table for parameter T

Output: (x, y) satisfying $ax + by = \gcd(a, b)$

```
1:  $z \leftarrow (a, b)^T$ 
2:  $P \leftarrow I$ 
3: while  $z_1 \neq 0$  and  $z_2 \neq 0$  do
4:    $B \leftarrow \max(|z_1|, |z_2|)$ 
5:    $p \leftarrow \lfloor Tz_1/B \rfloor$ 
6:    $q \leftarrow \lfloor Tz_2/B \rfloor$ 
7:    $R \leftarrow R_{p,q}$ 
8:    $z \leftarrow Rz$ 
9:    $P \leftarrow RP$ 
10:  if  $z_1 = 0$  or  $z_2 = 0$  then
11:    break
12:   $Q \leftarrow \lfloor z_2/z_1 \rfloor$ 
13:   $E \leftarrow \begin{pmatrix} 1 & 0 \\ -Q & 1 \end{pmatrix}$ 
14:   $z \leftarrow Ez$ 
15:   $P \leftarrow EP$ 
16:  if  $z_1 \neq 0$  then
17:    return  $\text{sgn}(z_1)$  times the first row of  $P$ 
18:  else
19:    return  $\text{sgn}(z_2)$  times the second row of  $P$ 
```

Theorem 54. Suppose

$$0 < \max(|a|, |b|) \leq N.$$

After storing an $O(T)$ -entry boundary table, *Algorithm 2* uses

$$O\left(\frac{\log N}{\log T}\right)$$

operations.

Proof. At all times,

$$z = P(a, b)^T$$

with P unimodular, so

$$\gcd(z_1, z_2) = \gcd(a, b).$$

By *Lemma 53*, every nonterminal iteration decreases

$$B = \|z\|_\infty$$

by a factor $\Theta(\sqrt{T})$. Hence after s iterations,

$$B \leq N \cdot O(T^{-s/2}).$$

Since the coordinates are integers, after

$$s = O(\log N / \log T)$$

iterations one coordinate must become zero.

If

$$z = (g, 0)^T,$$

then the first row (x, y) of P satisfies

$$ax + by = g,$$

and $|g| = \gcd(a, b)$. The other case is identical. □

Thus, for example,

$$T = N^\varepsilon$$

gives $O(N^\varepsilon)$ stored table entries and

$$O(1/\varepsilon)$$

query iterations, while

$$T = (\log N)^c$$

gives polylogarithmic space and

$$O\left(\frac{\log N}{\log \log N}\right)$$

query iterations.

4.6.3 Modular Square Roots

Let p be an odd prime. Given $a \in \mathbb{F}_p$, we want to find

$$x^2 = a,$$

or determine that no solution exists.

If $p \equiv 3 \pmod{4}$, Euler's criterion immediately gives the particularly simple formula

$$x = a^{(p+1)/4},$$

since

$$x^2 = a^{(p+1)/2} = a \cdot a^{(p-1)/2} = a.$$

For general odd primes, Cipolla's algorithm gives an equally simple $O(\log p)$ expected-time algorithm.

Lemma 55. *Let d be a quadratic non-residue modulo p , and let*

$$K = \mathbb{F}_p[\omega]/(\omega^2 - d).$$

Then

$$\omega^p = -\omega.$$

Consequently,

$$(u + v\omega)^p = u - v\omega.$$

Proof. By Euler's criterion,

$$d^{(p-1)/2} = -1.$$

Thus

$$\omega^p = \omega(\omega^2)^{(p-1)/2} = \omega d^{(p-1)/2} = -\omega.$$

Also, in characteristic p ,

$$(u + v\omega)^p = u^p + v^p \omega^p = u - v\omega.$$

□

Theorem 56 (Cipolla's Algorithm). Suppose $a \neq 0$ is a quadratic residue modulo the odd prime p . Choose $t \in \mathbb{F}_p$ such that

$$d = t^2 - a$$

is a quadratic non-residue. In

$$K = \mathbb{F}_p[\omega]/(\omega^2 - d),$$

define

$$x = (t + \omega)^{(p+1)/2}.$$

Then $x \in \mathbb{F}_p$ and

$$x^2 = a.$$

Proof. By [Lemma 55](#),

$$(t + \omega)^p = t - \omega.$$

Therefore

$$x^2 = (t + \omega)^{p+1} = (t + \omega)(t - \omega) = t^2 - \omega^2 = t^2 - d = a.$$

Furthermore,

$$x^{p-1} = (x^2)^{(p-1)/2} = a^{(p-1)/2} = 1$$

by Euler's criterion. Hence

$$x^p = x,$$

so x is fixed by Frobenius and therefore lies in $\mathbb{F}_p$. □

Arithmetic in the quadratic extension is easily computable:

$$(u + v\omega)(r + s\omega) = (ur + dvs) + (us + vr)\omega.$$

Thus the exponentiation in [Theorem 56](#) is ordinary binary exponentiation, using only $O(\log p)$ operations in $\mathbb{F}_p$.

A random choice of t succeeds with constant probability. In fact, exactly $(p-1)/2$ values of t make $t^2 - a$ a non-residue.

Proof. Let χ denote the Legendre symbol. The number of pairs (t, y) satisfying

$$y^2 = t^2 - a$$

is $p-1$: indeed,

$$(t - y)(t + y) = a,$$

and every choice of nonzero $u = t - y$ uniquely determines

$$t + y = a/u.$$

On the other hand, for fixed t , the number of choices of y is

$$1 + \chi(t^2 - a).$$

Hence

$$\sum_{t \in \mathbb{F}_p} \chi(t^2 - a) = -1.$$

Since a is a nonzero square, exactly two choices of t give $t^2 - a = 0$. If R and S denote the numbers giving a nonzero residue and a non-residue, then

$$R + S = p - 2, \quad R - S = -1.$$

Thus

$$S = \frac{p-1}{2}.$$

□

Algorithm 3 Modular Square Root

Input: Odd prime p , $a \in \mathbb{F}_p$

Output: x with $x^2 = a$, or None if no root exists

```
1: if  $a = 0$  then
2:    $\perp$  return 0
3: if  $a^{(p-1)/2} \neq 1$  then
4:    $\perp$  return None
5: if  $p \equiv 3 \pmod{4}$  then
6:    $\perp$  return  $a^{(p+1)/4}$ 
7: repeat
8:   Choose  $t \in \mathbb{F}_p$  uniformly at random
9:    $d \leftarrow t^2 - a$ 
10:  if  $d = 0$  then
11:     $\perp$  return  $t$ 
12: until  $d^{(p-1)/2} = -1$ 
13: Work in  $\mathbb{F}_p[\omega]/(\omega^2 - d)$ 
14:  $x \leftarrow (t + \omega)^{(p+1)/2}$ 
15: return  $x$ 
```

The expected number of choices of t is $O(1)$, and every test and the final exponentiation take $O(\log p)$ field operations. Thus the expected running time is $O(\log p)$.

4.6.4 Generalized Floor Sum

For nonnegative integers a, b, n, r, s and $m > 0$, define

$$F_{r,s}(a, b; m, n) = \sum_{x=0}^{n-1} x^r \left\lfloor \frac{ax + b}{m} \right\rfloor^s.$$

The ordinary floor sum is

$$F_{0,1}(a, b; m, n).$$

We use

$$P_r(t) = \sum_{x=0}^{t-1} x^r,$$

viewed as the usual degree- $(r+1)$ power-sum polynomial.

The algorithm follows the same idea as Euclid's algorithm. First remove integer parts of a/m and b/m . Once

$$0 \leq a, b < m,$$

transpose the lattice points underneath the line

$$y = \frac{ax + b}{m}.$$

The useful point is that after the transposition, the new slope is m/a , so the pair (a, m) undergoes an ordinary Euclidean step.

Removing Integer Parts Write

$$a = qm + a_0, \quad b = tm + b_0,$$

with

$$0 \leq a_0, b_0 < m.$$

Then

$$\left\lfloor \frac{ax + b}{m} \right\rfloor = qx + t + \left\lfloor \frac{a_0x + b_0}{m} \right\rfloor.$$

Expanding the power gives

$$F_{r,s}(a, b; m, n) = \sum_{j=0}^s \binom{s}{j} \sum_{\ell=0}^{s-j} \binom{s-j}{\ell} q^\ell t^{s-j-\ell} F_{r+\ell,j}(a_0, b_0; m, n).$$

Thus we may reduce immediately to

$$0 \leq a, b < m.$$

The Reciprocal Step Assume now

$$0 < a < m, \quad 0 \leq b < m.$$

Set

$$h = \left\lfloor \frac{an + b}{m} \right\rfloor, \quad c = (an + b) \bmod m.$$

Thus

$$an + b = mh + c.$$

The geometric motivation is to regard

$$q_x = \left\lfloor \frac{ax + b}{m} \right\rfloor$$

as the height of the x -th column and then transpose the resulting lattice diagram. Algebraically, write

$$q_x^s = \sum_{t=1}^{q_x} (t^s - (t-1)^s).$$

For $t = h - y$, the condition $q_x \geq t$ is

$$ax + b \geq m(h - y).$$

Using $an + b = mh + c$,

$$\begin{aligned} x &\geq \left\lceil \frac{m(h - y) - b}{a} \right\rceil \\ &= n - \left\lfloor \frac{my + c}{a} \right\rfloor. \end{aligned}$$

Define

$$u_y = \left\lfloor \frac{my + c}{a} \right\rfloor.$$

Then the contributing x 's are exactly

$$n - u_y, \dots, n - 1.$$

Now define the two polynomials

$$D_s(Y) = (h - Y)^s - (h - Y - 1)^s = \sum_{i=0}^{s-1} d_i Y^i,$$

and

$$R_r(U) = \sum_{j=1}^U (n-j)^r = P_r(n) - P_r(n-U) = \sum_{j=1}^{r+1} e_j U^j.$$

Theorem 57 (Reciprocal Floor-Sum Recurrence). *For $s > 0$,*

$$F_{r,s}(a, b; m, n) = \sum_{i=0}^{s-1} \sum_{j=1}^{r+1} d_i e_j F_{i,j}(m, c; a, h).$$

Proof. Using the finite-difference expansion of q_x^s and swapping the order of summation,

$$\begin{aligned} F_{r,s}(a, b; m, n) &= \sum_{x=0}^{n-1} x^r \sum_{t=1}^{q_x} (t^s - (t-1)^s) \\ &= \sum_{t=1}^h (t^s - (t-1)^s) \sum_{\substack{0 \leq x < n \\ q_x \geq t}} x^r. \end{aligned}$$

Substitute $t = h - y$. By the preceding calculation,

$$q_x \geq h - y \iff n - u_y \leq x < n,$$

where

$$u_y = \left\lfloor \frac{my + c}{a} \right\rfloor.$$

Hence

$$F_{r,s} = \sum_{y=0}^{h-1} D_s(y) R_r(u_y).$$

Expanding the two polynomials,

$$D_s(y) = \sum_i d_i y^i, \quad R_r(u_y) = \sum_j e_j u_y^j,$$

gives exactly

$$\sum_{i,j} d_i e_j \sum_{y=0}^{h-1} y^i \left\lfloor \frac{my + c}{a} \right\rfloor^j.$$

The inner sum is

$$F_{i,j}(m, c; a, h).$$

□

The base case is simply

$$F_{r,0}(a, b; m, n) = P_r(n).$$

If $a = 0$ after reduction, then the floor is constant and the answer is immediate.

For the ordinary floor sum, $r = 0, s = 1$,

$$D_1(Y) = 1, \quad R_0(U) = U,$$

so [Theorem 57](#) reduces to the particularly simple identity

$$F_{0,1}(a, b; m, n) = F_{0,1}(m, c; a, h).$$

After the reciprocal step, reducing the new numerator m modulo the new denominator a performs exactly a Euclidean-algorithm step on (m, a) . Thus there are only

$$O(\log m)$$

levels.

For any fixed powers r, s , the running time is therefore $O(\log m)$. If all

$$F_{i,j}, \quad i + j \leq K,$$

are computed together, the straightforward implementation takes

$$O(K^4 \log m)$$

arithmetic operations. The transformations are polynomial convolutions, so this dependence on K can be reduced further with fast polynomial multiplication.

4.6.5 Multiplicative Function Prefix Sums

Let f, g be arithmetic functions and let

$$c = f * g$$

be their Dirichlet convolution:

$$c(n) = \sum_{d|n} f(n/d)g(d).$$

Write their prefix sums as

$$F(x) = \sum_{n \leq x} f(n), \quad G(x) = \sum_{n \leq x} g(n), \quad C(x) = \sum_{n \leq x} c(n).$$

Suppose $G(x)$ and $C(x)$ can be evaluated quickly and $g(1)$ is invertible. Then $F(x)$ can be recovered much faster than evaluating every value of f up to x .

Lemma 58 (Prefix Sum from Dirichlet Convolution). *For every x ,*

$$C(x) = \sum_{d=1}^x g(d)F\left(\left\lfloor \frac{x}{d} \right\rfloor\right).$$

Consequently,

$$F(x) = \frac{C(x) - \sum_{d=2}^x g(d)F\left(\left\lfloor \frac{x}{d} \right\rfloor\right)}{g(1)}.$$

Proof. Expand and swap the order of summation:

$$\begin{aligned} C(x) &= \sum_{n \leq x} \sum_{d|n} f(n/d)g(d) \\ &= \sum_{d \leq x} g(d) \sum_{k \leq x/d} f(k) \\ &= \sum_{d \leq x} g(d)F\left(\left\lfloor \frac{x}{d} \right\rfloor\right). \end{aligned}$$

Separating the $d = 1$ term gives the recurrence. □

The crucial optimization is that

$$\left\lfloor \frac{x}{d} \right\rfloor$$

takes only $O(\sqrt{x})$ distinct values. If

$$q = \left\lfloor \frac{x}{d} \right\rfloor,$$

then the largest index having the same quotient is

$$r = \left\lfloor \frac{x}{q} \right\rfloor.$$

Therefore the whole interval $d, \dots, r$ contributes

$$(G(r) - G(d-1))F(q).$$

Precompute $F(x)$ for

$$x \leq B$$

using the linear sieve, and memoize all larger recursive values.

Algorithm 4 Multiplicative Function Prefix Sum

Input: x , prefix functions G, C , table of $F(t)$ for $t \leq B$

Output: $F(x)$

```

1: function PREFIXF( $x$ )
2:   if  $x \leq B$  then
3:     return  $F_{\text{small}}[x]$ 
4:   if  $x$  is memoized then
5:     return memo[ $x$ ]
6:    $A \leftarrow C(x)$ 
7:    $l \leftarrow 2$ 
8:   while  $l \leq x$  do
9:      $q \leftarrow \lfloor x/l \rfloor$ 
10:     $r \leftarrow \lfloor x/q \rfloor$ 
11:     $A \leftarrow A - (G(r) - G(l-1)) \cdot \text{PREFIXF}(q)$ 
12:     $l \leftarrow r + 1$ 
13:    $A \leftarrow A/g(1)$ 
14:   memo[ $x$ ]  $\leftarrow A$ 
15: return  $A$ 
```

Theorem 59. *If $G(x)$ and $C(x)$ are evaluable in $O(1)$ arithmetic operations and $F(1), \dots, F(B)$ are precomputed, then Algorithm 4 computes $F(n)$ in*

$$O\left(B + \frac{n}{\sqrt{B}}\right)$$

arithmetic operations. Taking

$$B = n^{2/3}$$

gives

$$O(n^{2/3}).$$

Proof. All recursive arguments larger than B are of the form

$$\left\lfloor \frac{n}{k} \right\rfloor > B,$$

so there are at most

$$\frac{n}{B}$$

such memoized states.

For

$$x = \left\lfloor \frac{n}{k} \right\rfloor,$$

one call performs $O(\sqrt{x})$ quotient blocks. Hence the total work on large states is

$$\begin{aligned} \sum_{k \leq n/B} O\left(\sqrt{\frac{n}{k}}\right) &= O\left(\sqrt{n} \sum_{k \leq n/B} k^{-1/2}\right) \\ &= O\left(\sqrt{n} \sqrt{\frac{n}{B}}\right) \\ &= O\left(\frac{n}{\sqrt{B}}\right). \end{aligned}$$

Adding the $O(B)$ preprocessing gives the claim. □

Two standard examples are immediate.

Since

$$\varphi * 1 = \text{id},$$

we may take

$$g(n) = 1, \quad G(x) = x, \quad C(x) = \sum_{n \leq x} n = \frac{x(x+1)}{2}$$

to compute

$$\sum_{n \leq x} \varphi(n)$$

in $O(x^{2/3})$ time.

Similarly,

$$\mu * 1 = \varepsilon$$

gives

$$\sum_{n \leq x} \mu(n)$$

from

$$G(x) = x, \quad C(x) = 1.$$

4.6.6 LCM and GCD Convolution

Now take

$$P = \{1, \dots, N\}$$

ordered by divisibility:

$$a \leq b \iff a \mid b.$$

Then

$$a \vee b = \text{lcm}(a, b), \quad a \wedge b = \text{gcd}(a, b).$$

Hence

$$(f *_{\text{LCM}} g)(k) = \sum_{\text{lcm}(a,b)=k} f(a)g(b)$$

is join convolution and is diagonalized by the divisor zeta transform

$$(\zeta_{\text{div}} f)(k) = \sum_{d|k} f(d).$$

Likewise

$$(f *_{\text{GCD}} g)(k) = \sum_{\gcd(a,b)=k} f(a)g(b)$$

is meet convolution and is diagonalized by the multiple zeta transform

$$(\zeta_{\text{mult}} f)(k) = \sum_{\substack{j \leq N \\ k|j}} f(j).$$

Both transforms can be implemented by processing one prime (this can be viewed as a dimension) at a time.

For divisor zeta, for every prime p , process multiples x of p in increasing order:

$$F(x) \leftarrow F(x) + F(x/p).$$

Möbius inversion processes in the reverse direction with subtraction.

The total number of updates is

$$\sum_{p \leq N} \left\lfloor \frac{N}{p} \right\rfloor = O(N \log \log N).$$

For the multiple transform, process the same prime chains in the opposite direction:

$$F(x) \leftarrow F(x) + F(px).$$

Again inversion reverses the order and changes addition to subtraction.

Thus GCD and LCM convolution take

$$O(N \log \log N)$$

ring operations.

4.7 Concave Min-Plus Convolution

4.7.1 Description

Given an array a of length n (which is concave, meaning its forward differences are monotonically decreasing) and an arbitrary array b of length m , the concave min-plus convolution computes:

$$c[p] = \min_{0 \leq x < n, 0 \leq y < m, x+y=p} (a[x] + b[y])$$

We give an $O(m \log n + n)$ solution by chunking the array b and using a modified Li Chao Tree.

The algorithm processes b in chunks of size up to n . For a chunk starting at index s , we compute its contributions to a window of length $2n$ in the output array c using two passes:

- **Forward Pass:** Evaluates pairs where the relative index in a is greater than or equal to the relative index in b . It inserts functions $f_k(i) = a[i - k] + b[s + k]$ into a Li Chao tree and queries them for $i \geq k$.
- **Reverse Pass:** Symmetrically handles the remaining cross-terms where the relative index in a is strictly less than the relative index in b , sweeping backwards.

4.7.2 Crossings

Lemma 60 (Single Crossing Property). *For a concave array a , define the extended functions for $k \in [0, n)$:*

$$f_k(i) = \begin{cases} -\infty - k & \text{if } i < k \\ a[i - k] + b[s + k] & \text{if } i \geq k \end{cases}$$

For any $j < k$, the difference $D(i) = f_k(i) - f_j(i)$ transitions from negative to positive at most once.

Proof. We evaluate the difference $D(i)$ across three domains:

1. **For $i < j$:** Both are artificial. $D(i) = (-\infty - k) - (-\infty - j) = j - k < 0$.
2. **For $j \leq i < k$:** f_j is finite, but f_k is artificial. $D(i) = -\infty < 0$.
3. **For $i \geq k$:** Both are finite. $D(i) = a[i - k] - a[i - j] + C$. Because a is concave, its slope is decreasing. Since $i - k < i - j$, the difference $a[i - k] - a[i - j]$ strictly monotonically increases with i .

Because $D(i)$ is strictly negative for all $i < k$ and monotonically increases for all $i \geq k$, the functions f_j and f_k intersect at most once globally. $\square$

4.7.3 Boundary Conditions

While [Lemma 60](#) proves the functions cross at most once, their shapes are highly irregular due to the jump at $i = k$.

A standard Li Chao Tree assumes that if a function is better at the left endpoint L and worse at the midpoint M , the single continuous crossing must lie within $[L, M]$, and recursively routes the function leftward.

A function f_k might be artificially better at L (because it's about $-\infty$), worse at M , but then become the true minimum on the right side of the interval.

For a node spanning the discrete interval $[L, R)$, let z be the current resident function and x be the newly inserted function.

- If $x(L) \geq z(L)$ and $x(R - 1) \geq z(R - 1)$, then x is dominated at both strict boundaries. By [Lemma 60](#), it must be dominated everywhere inside the interval. We safely discard x .
- If $x(L) < z(L)$ and $x(R - 1) < z(R - 1)$, then x dominates z everywhere in the interval. We replace z with x and discard z .
- Otherwise, a crossing exists within the interval. We swap x and z if necessary to maintain the best function at the midpoint, and recursively push the loser to the appropriate child.

4.8 Fast Factorials and Deterministic Factorization

To compute the factorial we will obtain values of

$$F_v(x) = (x + 1)(x + 2) \cdots (x + v)$$

at the arithmetic progression

$$0, v, 2v, \dots$$

Using a product tree followed by multipoint evaluation gives two logarithmic factors. The special structure lets us do better: we never construct the coefficients of F_v at all. Instead, we recursively construct its values on the arithmetic progression.

The main tool is fast shifting of equally spaced polynomial evaluations.

4.8.1 Fast Factorial

Let p be prime and suppose

$$0 \leq n < p.$$

We compute

$$n! \pmod{p}$$

in

$$O(\sqrt{n} \log n)$$

field operations.

Choose a power of two

$$v = \Theta(\sqrt{n}), \quad v \geq \sqrt{n}.$$

For

$$d = 1, 2, 4, \dots, v,$$

define

$$F_d(x) = \prod_{j=1}^d (x + j).$$

Rather than storing the coefficients of F_d , store the $d + 1$ evaluations

$$S_d = (F_d(0), F_d(v), F_d(2v), \dots, F_d(dv)).$$

The base case is

$$S_1 = (1, v + 1).$$

The doubling identity is

$$F_{2d}(x) = F_d(x)F_d(x + d).$$

Suppose S_d is known. Using [Lemma 241](#), compute

$$A_i = F_d(iv + d), \quad 0 \leq i \leq d.$$

Then

$$F_{2d}(iv) = F_d(iv)F_d(iv + d) = S_d[i]A_i$$

gives the first $d + 1$ entries of S_{2d} .

For the remaining entries, compute by two more evaluation shifts

$$B_j = F_d((d + 1 + j)v),$$

and

$$C_j = F_d((d + 1 + j)v + d), \quad 0 \leq j \leq d.$$

Then

$$B_j C_j = F_{2d}((d + 1 + j)v).$$

Keeping

$$j = 0, \dots, d - 1$$

gives exactly the remaining d entries.

Thus

$$S_d \longrightarrow S_{2d}$$

uses three evaluation shifts and $O(d)$ pointwise multiplications.

Algorithm 5 Fast Rising Factorial Evaluations

Input: Power of two v

Output:

$$S_v[i] = F_v(iv) \quad (0 \leq i \leq v)$$

```

1:  $S \leftarrow (1, v + 1)$ 
2:  $d \leftarrow 1$ 
3: while  $d < v$  do
4:    $A \leftarrow \text{EVALSHIFT}(S, d)$ 
5:    $B \leftarrow \text{EVALSHIFT}(S, (d + 1)v)$ 
6:    $C \leftarrow \text{EVALSHIFT}(S, (d + 1)v + d)$ 
7:    $S' \leftarrow$  empty array
8:   for  $i = 0, \dots, d$  do
9:      $\lfloor$  append  $S[i]A[i]$  to  $S'$ 
10:  for  $j = 0, \dots, d - 1$  do
11:     $\lfloor$  append  $B[j]C[j]$  to  $S'$ 
12:   $S \leftarrow S'$ 
13:  $d \leftarrow 2d$ 
14: return  $S$ 

```

$$\triangleright A_i = F_d(iv + d)$$

Theorem 61 (Fast Factorial). *For prime p and $n < p = n^{O(1)}$, one can compute*

$$n! \pmod{p}$$

in

$$\boxed{O(\sqrt{n} \log n)}$$

time.

Proof. At a doubling step of size d , the work is

$$O(d \log d).$$

Therefore

$$T(v) = T(v/2) + v \log v = O(v \log v).$$

Since

$$v = \Theta(\sqrt{n}),$$

this is

$$O(\sqrt{n} \log n).$$

At the end we know

$$F_v(0), F_v(v), \dots, F_v(v^2).$$

Since

$$q = \left\lfloor \frac{n}{v} \right\rfloor \leq v,$$

the complete blocks give

$$(qv)! = \prod_{i=0}^{q-1} F_v(iv).$$

Finally multiply the fewer than v remaining terms

$$qv + 1, \dots, n.$$

□

4.8.2 Deterministic Integer Factorization

The same construction gives a deterministic $\tilde{O}(N^{1/4})$ -time factorization algorithm.

Let N be composite. If N is even, we immediately find the factor 2, so assume that N is odd.

Let v be the smallest power of two satisfying

$$v \geq N^{1/4}.$$

Thus

$$N^{1/4} \leq v < 2N^{1/4}.$$

Define

$$F_v(x) = \prod_{j=1}^v (x + j).$$

Then

$$F_v(iv) = \prod_{j=1}^v (iv + j)$$

is the product of the i -th block of v consecutive integers.

Using the fast evaluation construction from [Theorem 61](#), compute

$$\boxed{F_v(0), F_v(v), \dots, F_v((v-1)v) \pmod{N}}$$

in

$$O(v \log v)$$

arithmetic operations over $\mathbb{Z}/N\mathbb{Z}$.

The only additional issue is that $\mathbb{Z}/N\mathbb{Z}$ is not a field: the evaluation-shift algorithm may try to divide by a nonunit. For factorization, this causes no difficulty.

Batched Inversions Suppose one stage of the algorithm requires the inverses of

$$a_1, \dots, a_t \in \mathbb{Z}/N\mathbb{Z}.$$

Compute

$$A = \prod_{i=1}^t a_i \pmod{N}$$

and

$$g = \gcd(A, N).$$

If

$$g = 1,$$

then every a_i is invertible. One extended gcd computes A^{-1} , after which prefix and suffix products recover all

$$a_i^{-1}$$

using $O(t)$ modular multiplications.
If

$$g > 1,$$

split the product into two halves and compute the gcd of each half-product with N . Repeating this isolates a nonunit denominator. Its gcd with N reveals a factor of N .

Thus every required inversion either succeeds or produces a factor, while the batching adds only linear work to each evaluation-shift stage.

Theorem 62 (Strassen–Pollard Factorization). *A nontrivial factor of a composite integer N can be found deterministically in*

$$\boxed{O\left(N^{1/4}(\log N)^2 \log \log N\right)}$$

bit operations, assuming

$$M(\ell) = O(\ell \log \ell)$$

for multiplication of ℓ -bit integers.

Proof. Let p be the smallest prime divisor of N . Since N is composite,

$$p \leq \sqrt{N}.$$

By the choice of v ,

$$\sqrt{N} \leq v^2.$$

Hence p lies among the integers

$$1, 2, \dots, v^2.$$

Therefore there are

$$0 \leq i < v, \quad 1 \leq j \leq v$$

such that

$$p = iv + j.$$

For this i ,

$$F_v(iv) = \prod_{t=1}^v (iv + t)$$

contains p as a factor, and consequently

$$\gcd(F_v(iv), N) > 1.$$

After computing the v block products, calculate

$$g_i = \gcd(F_v(iv), N) \quad (0 \leq i < v).$$

At least one satisfies

$$g_i > 1.$$

If

$$1 < g_i < N,$$

then g_i is already a nontrivial factor.

The remaining possibility is

$$g_i = N.$$

In this case inspect the v integers in that block:

$$iv + 1, \dots, iv + v.$$

At least one is divisible by the smallest prime factor p . Since every one of these integers is at most

$$v^2 = O(\sqrt{N}) < N,$$

its gcd with N is a proper nontrivial factor.

It remains to bound the running time. Put

$$L = \lceil \log_2 N \rceil.$$

All arithmetic takes place modulo N , so every ring element has $O(L)$ bits.

The fast evaluation construction uses

$$O(v \log v)$$

ring operations. A modular multiplication costs

$$O(M(L))$$

bit operations, so the sampling phase costs

$$O(v \log v M(L)).$$

Since

$$v = \Theta(N^{1/4})$$

and

$$\log v = \Theta(\log N) = \Theta(L),$$

this is

$$O(N^{1/4} L M(L)).$$

The final $O(v)$ gcd computations cost

$$O(v M(L) \log L),$$

which is smaller than the sampling cost. The batched inversions likewise contribute only lower-order terms.

Therefore the total bit complexity is

$$O(N^{1/4} L M(L)).$$

Substituting

$$M(L) = O(L \log L)$$

gives

$$O\left(N^{1/4}(\log N)^2 \log \log N\right).$$

□

4.9 Miller–Rabin Primality Test

Let $n > 2$ be odd, and write

$$n - 1 = 2^s d, \quad d \text{ odd.}$$

The Miller–Rabin test studies the squaring sequence

$$a^d, \quad a^{2d}, \quad a^{4d}, \quad \dots, \quad a^{2^s d} = a^{n-1} \pmod{n}.$$

For a prime modulus, this sequence has a very restricted form.

Lemma 63. *Let p be an odd prime and write*

$$p - 1 = 2^s d$$

with d odd. For every $a \not\equiv 0 \pmod{p}$, either

$$a^d \equiv 1 \pmod{p},$$

or

$$a^{2^r d} \equiv -1 \pmod{p}$$

for some $0 \leq r < s$.

Proof. By Fermat's theorem,

$$a^{2^s d} = a^{p-1} \equiv 1 \pmod{p}.$$

If $a^d \equiv 1$, we are done.

Otherwise, let $j \geq 1$ be the smallest index such that

$$a^{2^j d} \equiv 1 \pmod{p}.$$

Then

$$x := a^{2^{j-1} d}$$

satisfies

$$x^2 \equiv 1 \pmod{p} \quad \text{but} \quad x \not\equiv 1 \pmod{p}.$$

Since

$$(x-1)(x+1) \equiv 0 \pmod{p}$$

and p is prime,

$$x \equiv -1 \pmod{p}.$$

□

For composite n , a base a satisfying the conclusion of [Lemma 63](#) is called a *strong liar*; otherwise it is a *strong witness* for the compositeness of n .

Algorithm 6 Miller–Rabin Primality Test

Input: Integer $n \geq 2$, number of rounds t

Output: Composite or Probably Prime

```

1: if  $n \in \{2, 3\}$  then
2:   return Probably Prime
3: if  $n < 2$  or  $n$  is even then
4:   return Composite
5: Write  $n - 1 = 2^s d$  with  $d$  odd
6: for  $k = 1, \dots, t$  do
7:   Choose  $a$  uniformly from  $\{2, \dots, n - 2\}$ 
8:   if  $\gcd(a, n) > 1$  then
9:     return Composite
10:   $x \leftarrow a^d \bmod n$ 
11:  if  $x = 1$  or  $x = n - 1$  then
12:    continue
13:   $pass \leftarrow \text{false}$ 
14:  for  $r = 1, \dots, s - 1$  do
15:     $x \leftarrow x^2 \bmod n$ 
16:    if  $x = n - 1$  then
17:       $pass \leftarrow \text{true}$ 
18:      break
19:    if  $x = 1$  then
20:      return Composite
21:  if not  $pass$  then
22:    return Composite
23: return Probably Prime

```

By [Lemma 63](#), a prime is always accepted. The usefulness of the test comes from the fact that very few bases can fool it.

Theorem 64 (Rabin–Monier). *Let n be an odd composite integer. At most one quarter of the nonzero residues modulo n are strong liars.*

Consequently, t independent rounds of the Miller–Rabin test accept a composite n with probability at most

$$\boxed{4^{-t}}.$$

The running time is

$$O(t M(\log n) \log n)$$

bit operations, where $M(\ell)$ is the cost of multiplying two ℓ -bit integers. Using

$$M(\ell) = O(\ell \log \ell),$$

this is

$$\boxed{O(t \log^2 n \log \log n)}.$$

Proof. Write

$$n - 1 = 2^s d, \quad d \text{ odd},$$

and factor

$$n = \prod_{i=1}^k p_i^{e_i}.$$

For each prime divisor p_i , write

$$p_i - 1 = 2^{s_i} u_i, \quad u_i \text{ odd},$$

and set

$$\nu = \min_i s_i.$$

We first place every strong liar inside a set that is easy to count. Suppose a is a strong liar. If

$$a^d \equiv 1 \pmod{n},$$

then certainly

$$a^{2^{\nu-1}d} \equiv 1 \pmod{n}.$$

Otherwise, for some $0 \leq r < s$,

$$a^{2^r d} \equiv -1 \pmod{n}.$$

Reducing modulo any p_i , the order of a modulo p_i has 2-adic valuation exactly $r + 1$. Since this order divides $p_i - 1$,

$$r + 1 \leq s_i$$

for every i , and hence

$$r + 1 \leq \nu.$$

Therefore, after squaring if necessary,

$$a^{2^{\nu-1}d} \equiv \pm 1 \pmod{n}.$$

Thus every strong liar belongs to

$$S = \left\{ a \in (\mathbb{Z}/n\mathbb{Z})^\times : a^{2^{\nu-1}d} \equiv \pm 1 \pmod{n} \right\}.$$

We now count S .

By the Chinese remainder theorem,

$$(\mathbb{Z}/n\mathbb{Z})^\times \cong \prod_{i=1}^k (\mathbb{Z}/p_i^{e_i}\mathbb{Z})^\times.$$

For odd p_i , the i -th group is cyclic of order

$$N_i = p_i^{e_i-1}(p_i - 1) = p_i^{e_i-1} 2^{s_i} u_i.$$

Set

$$m = 2^{\nu-1}d.$$

Since $p_i \nmid d$,

$$\gcd(m, N_i) = 2^{\nu-1} \gcd(d, u_i).$$

In a cyclic group of order N_i , the equation

$$x^m = 1$$

has exactly $\gcd(m, N_i)$ solutions.

The equation

$$x^m = -1$$

has the same number of solutions. Indeed,

$$\gcd(m, N_i) \mid \frac{N_i}{2},$$

because $s_i \geq \nu$, so -1 lies in the image of the m -th power map. Hence, writing

$$g_i = \gcd(d, u_i),$$

the CRT gives

$$|S| = 2 \prod_{i=1}^k 2^{\nu-1} g_i.$$

Since

$$\varphi(n) = \prod_{i=1}^k p_i^{e_i-1} 2^{s_i} u_i,$$

we obtain

$$\frac{\varphi(n)}{|S|} = \frac{1}{2} \prod_{i=1}^k p_i^{e_i-1} \left(2^{s_i-\nu+1} \frac{u_i}{g_i} \right).$$

Define

$$R_i = 2^{s_i-\nu+1} \frac{u_i}{g_i}.$$

Each R_i is an even integer, so in particular

$$R_i \geq 2.$$

It remains only to check that

$$\frac{\varphi(n)}{|S|} \geq 4.$$

If $k \geq 3$, then immediately

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} 2^k \geq 4.$$

Suppose $k = 2$. If n is not squarefree, some $e_i \geq 2$, and therefore

$$p_i^{e_i-1} \geq 3.$$

Hence

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} \cdot 3 \cdot 2 \cdot 2 > 4.$$

It remains to consider

$$n = pq, \quad p < q.$$

We always have $R_p \geq 2$.

If

$$s_q > \nu,$$

then

$$R_q \geq 4.$$

Otherwise $s_q = \nu$, so

$$q - 1 = 2^\nu u_q.$$

We claim that $u_q \nmid d$. Indeed, since $s \geq \nu$, if $u_q \mid d$ then

$$q - 1 = 2^\nu u_q \mid 2^s d = n - 1.$$

But

$$n - 1 = pq - 1 \equiv p - 1 \pmod{q - 1},$$

and

$$0 < p - 1 < q - 1,$$

so $q - 1 \nmid n - 1$, a contradiction.

Thus

$$\frac{u_q}{\gcd(u_q, d)}$$

is an odd integer greater than 1, hence at least 3. Therefore

$$R_q \geq 6.$$

So in either case,

$$\frac{\varphi(n)}{|S|} \geq \frac{1}{2} \cdot 2 \cdot 4 = 4.$$

Finally, suppose $k = 1$, so

$$n = p^e, \quad e \geq 2.$$

Since

$$p - 1 \mid p^e - 1 = n - 1,$$

the odd part u_1 divides d . Hence

$$R_1 = 2,$$

and therefore

$$\frac{\varphi(n)}{|S|} = p^{e-1}.$$

This is at least 4 except when

$$n = 3^2 = 9.$$

For $n = 9$, the only strong liars are

$$\pm 1,$$

so there are exactly

$$2 = \frac{9-1}{4}.$$

Thus in every case the number of strong liars is at most

$$\frac{n-1}{4}.$$

Therefore a random Miller–Rabin round accepts a composite number with probability at most $1/4$, and t independent rounds fail with probability at most

$$4^{-t}.$$

For the running time, let

$$\ell = \Theta(\log n).$$

Computing $a^d \bmod n$ by repeated squaring and then following the Miller–Rabin squaring chain uses

$$O(\log n)$$

modular multiplications. Each operates on $O(\log n)$ -bit integers, so one round takes

$$O(M(\log n) \log n)$$

bit operations. Therefore t rounds take

$$O(t M(\log n) \log n).$$

Using

$$M(\ell) = O(\ell \log \ell)$$

gives

$$O(t \log^2 n \log \log n).$$

□

5 Combinatorics

5.1 Entropy Method

Throughout this section, entropy is measured in bits.

5.1.1 Shearer's Lemma

Lemma 65 (Shearer's Lemma). *Let*

$$X = (X_1, \dots, X_n)$$

be a random vector, and let

$$\mathcal{A} = \{A_1, \dots, A_m\}$$

be a collection of subsets of $[n]$ such that every coordinate $i \in [n]$ belongs to at least k members of $\mathcal{A}$. Then

$$kH(X) \leq \sum_{j=1}^m H(X_{A_j}).$$

Proof. Order the coordinates as $1, \dots, n$. For every $A \subseteq [n]$, the chain rule gives

$$H(X_A) = \sum_{i \in A} H(X_i \mid X_j : j \in A, j < i).$$

Conditioning on more information can only decrease entropy, so

$$H(X_A) \geq \sum_{i \in A} H(X_i \mid X_1, \dots, X_{i-1}).$$

Summing over $A \in \mathcal{A}$,

$$\begin{aligned} \sum_{A \in \mathcal{A}} H(X_A) &\geq \sum_{A \in \mathcal{A}} \sum_{i \in A} H(X_i \mid X_1, \dots, X_{i-1}) \\ &\geq k \sum_{i=1}^n H(X_i \mid X_1, \dots, X_{i-1}) \\ &= kH(X). \end{aligned}$$

□

The interpretation is that if several overlapping projections together cover every coordinate k times, then together they contain at least k copies of the information in the whole object.

5.1.2 Entropy Bound for Binomial Sums

Define the binary entropy function

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

Theorem 66. *For*

$$0 \leq k \leq \frac{n}{2},$$

$$\sum_{j=0}^k \binom{n}{j} \leq 2^{nH(k/n)}.$$

In particular,

$$\binom{n}{k} \leq 2^{nH(k/n)}.$$

Proof. Let $X \subseteq [n]$ be chosen uniformly from all subsets of size at most k , and let

$$X_i = \mathbf{1}[i \in X].$$

Then

$$H(X_1, \dots, X_n) = \log_2 \sum_{j=0}^k \binom{n}{j}.$$

By subadditivity of entropy,

$$H(X_1, \dots, X_n) \leq \sum_{i=1}^n H(X_i).$$

By symmetry, every X_i has the same probability p of being 1. Moreover,

$$np = \mathbb{E}|X| \leq k,$$

so

$$p \leq \frac{k}{n} \leq \frac{1}{2}.$$

Since the binary entropy function is increasing on $[0, 1/2]$,

$$H(X_i) = H(p) \leq H(k/n).$$

Therefore

$$\log_2 \sum_{j=0}^k \binom{n}{j} \leq nH(k/n),$$

which gives the result. □

5.1.3 Irregular Moore Bound

For a directed edge

$$\vec{e} = (u, v),$$

a *non-returning walk* is a walk that never immediately traverses an edge and then its reverse.

Let $N_i(v)$ be the number of non-returning walks of length i starting at v , and let $N_i(\vec{e})$ be the number of non-returning walks of length $i + 1$ whose first edge is $\vec{e}$.

The entropy argument says that even though the degrees are irregular, the average number of such walks grows as though every vertex had the average degree.

Lemma 67 (Non-Returning Walk Growth). *Let G be a graph of minimum degree at least 2 and average degree*

$$\bar{d}.$$

Define the stationary vertex distribution

$$\pi(v) = \frac{d_v}{\bar{d}n}.$$

Then for $i \geq 1$,

$$\boxed{\mathbb{E}_{v \sim \pi} N_i(v) \geq \bar{d}(\bar{d} - 1)^{i-1},}$$

and if $\vec{e}$ is a uniformly random directed edge, then for $i \geq 0$,

$$\boxed{\mathbb{E}_{\vec{e}} N_i(\vec{e}) \geq (\bar{d} - 1)^i.}$$

Proof. We first prove the edge version.

Choose $\vec{e}_0$ uniformly among all directed edges, and then generate

$$\vec{e}_0, \vec{e}_1, \dots, \vec{e}_i$$

by choosing each next edge uniformly among all non-returning continuations.

Every $\vec{e}_j$ is again uniformly distributed among directed edges, so the head vertex of each $\vec{e}_j$ has distribution π .

Conditioned on $\vec{e}_0 = \vec{e}$, the random continuation

$$(\vec{e}_1, \dots, \vec{e}_i)$$

has support of size at most $N_i(\vec{e})$. Hence

$$H(\vec{e}_1, \dots, \vec{e}_i \mid \vec{e}_0 = \vec{e}) \leq \log_2 N_i(\vec{e}).$$

Averaging and using Jensen,

$$\begin{aligned} \log_2 \mathbb{E}_{\vec{e}} N_i(\vec{e}) &\geq \mathbb{E}_{\vec{e}} \log_2 N_i(\vec{e}) \\ &\geq H(\vec{e}_1, \dots, \vec{e}_i \mid \vec{e}_0). \end{aligned}$$

By the chain rule and the Markov property,

$$H(\vec{e}_1, \dots, \vec{e}_i \mid \vec{e}_0) = i \mathbb{E}_{v \sim \pi} \log_2(d_v - 1).$$

Now

$$\mathbb{E}_{v \sim \pi} \log_2(d_v - 1) = \frac{1}{d} \mathbb{E}_{v \text{ uniform}} [d_v \log_2(d_v - 1)].$$

The function

$$x \mapsto x \log(x - 1)$$

is convex for $x \geq 2$, so Jensen gives

$$\mathbb{E}_{v \sim \pi} \log_2(d_v - 1) \geq \log_2(\bar{d} - 1).$$

Thus

$$\mathbb{E}_{\vec{e}} N_i(\vec{e}) \geq (\bar{d} - 1)^i.$$

For the vertex version, choose $v \sim \pi$, then the first edge uniformly from the d_v choices, and continue non-returningly. Exactly the same argument gives

$$\log_2 \mathbb{E}_{v \sim \pi} N_i(v) \geq \mathbb{E}_{v \sim \pi} \log_2 d_v + (i - 1) \mathbb{E}_{v \sim \pi} \log_2(d_v - 1).$$

Since

$$x \mapsto x \log x$$

is convex,

$$\mathbb{E}_{v \sim \pi} \log_2 d_v = \frac{1}{d} \mathbb{E}[d_v \log_2 d_v] \geq \log_2 \bar{d}.$$

Combining the bounds gives

$$\mathbb{E}_{v \sim \pi} N_i(v) \geq \bar{d}(\bar{d} - 1)^{i-1}.$$

□

Theorem 68 (Alon–Hoory–Linial Moore Bound). *Let G be a simple graph with n vertices, minimum degree at least 2, average degree $\bar{d}$, and girth g .*

If

$$g = 2r + 1,$$

then

$$n \geq 1 + \bar{d} \sum_{i=0}^{r-1} (\bar{d} - 1)^i.$$

If

$$g = 2r,$$

then

$$n \geq 2 \sum_{i=0}^{r-1} (\bar{d} - 1)^i.$$

Proof. Suppose first that

$$g = 2r + 1.$$

Fix a vertex v . The endpoints of all non-returning walks of lengths

$$0, 1, \dots, r$$

starting at v are distinct: two such walks ending at the same vertex would produce a cycle of length at most $2r$.

Hence

$$n \geq 1 + \sum_{i=1}^r N_i(v).$$

Average over $v \sim \pi$ and apply [Lemma 67](#):

$$n \geq 1 + \sum_{i=1}^r \bar{d}(\bar{d} - 1)^{i-1}.$$

Now suppose

$$g = 2r.$$

Take an edge uv , and orient it both ways:

$$\vec{e} = (u, v), \quad \overleftarrow{e} = (v, u).$$

The endpoints of the non-returning walks of lengths up to r extending from the two sides of this edge are all distinct; otherwise they would form a cycle of length less than $2r$.

Therefore

$$n \geq \sum_{i=0}^{r-1} N_i(\vec{e}) + \sum_{i=0}^{r-1} N_i(\overleftarrow{e}).$$

Choose $\vec{e}$ uniformly among directed edges. Its reverse is also uniform, so taking expectations gives

$$n \geq 2 \sum_{i=0}^{r-1} (\bar{d} - 1)^i.$$

□

5.1.4 Permanent Bound

For a 0–1 matrix A , its permanent counts the perfect matchings in the corresponding bipartite graph.

Theorem 69 (Bregman's Theorem). *Let*

$$A \in \{0, 1\}^{n \times n}$$

have row sums

$$r_1, \dots, r_n.$$

Then

$$\text{per}(A) \leq \prod_{i=1}^n (r_i!)^{1/r_i}.$$

Proof. Let $\mathcal{M}$ be the set of perfect matchings represented by A , and choose a uniformly random

$$\sigma \in \mathcal{M}.$$

Then

$$H(\sigma) = \log_2 |\mathcal{M}| = \log_2 \text{per}(A).$$

Independently choose a uniformly random ordering of the rows. Reveal the values $\sigma(i)$ in this random order.

When row i is revealed, let R_i be the number of currently unused columns j such that

$$A_{ij} = 1.$$

Conditioned on everything already revealed, $\sigma(i)$ has at most R_i possible values. Thus

$$H(\sigma(i) \mid \text{earlier rows}) \leq \mathbb{E}[\log_2 R_i].$$

By the chain rule,

$$H(\sigma) \leq \sum_{i=1}^n \mathbb{E}[\log_2 R_i].$$

Now fix both i and the matching σ . For every allowed column

$$j \in N(i),$$

there is a unique row

$$\sigma^{-1}(j)$$

matched to j . These give exactly r_i rows, one of which is i itself.

A column $j \in N(i)$ is still available when row i is exposed exactly when the row $\sigma^{-1}(j)$ has not appeared earlier. Therefore R_i is the number of these r_i distinguished rows appearing at or after i .

Since the row order is uniformly random, the rank of i among these r_i rows is uniform. Consequently,

$$R_i \text{ is uniform on } \{1, 2, \dots, r_i\}.$$

Hence

$$\mathbb{E}[\log_2 R_i] = \frac{1}{r_i} \sum_{j=1}^{r_i} \log_2 j = \frac{1}{r_i} \log_2 (r_i!).$$

Therefore

$$\log_2 \text{per}(A) \leq \sum_{i=1}^n \frac{\log_2 (r_i!)}{r_i}.$$

Exponentiating gives

$$\text{per}(A) \leq \prod_{i=1}^n (r_i!)^{1/r_i}.$$

□

5.2 Additive Combinatorics

5.2.1 Kneser's Addition Theorem

For a finite set S in an abelian group G , its *stabilizer* is

$$H(S) = \{g \in G : S + g = S\}.$$

It is a subgroup of G .

Kneser's theorem says that periodicity is the only obstruction to the usual sumset lower bound. If $H = H(A + B)$, then after quotienting by H , the sumset has no remaining period, and one recovers the familiar “sum of the sizes minus one” bound.

Theorem 70 (Kneser's Addition Theorem). *Let A, B be finite nonempty subsets of an abelian group G , and let $H = H(A + B)$. Then*

$$|A + B| \geq |A + H| + |B + H| - |H|.$$

Since $A + H$, $B + H$, and $A + B$ are unions of H -cosets, this can equivalently be read in G/H as

$$\frac{|A + B|}{|H|} \geq \frac{|A + H|}{|H|} + \frac{|B + H|}{|H|} - 1.$$

Thus once the period has been factored out, Kneser's theorem looks like $|A + B| \geq |A| + |B| - 1$.

The proof has two ingredients. First, taking a union cannot simultaneously destroy too much periodicity and add too few new elements. Second, a “transfusion” operation moves elements between the two summands without increasing their sumset; at a minimal configuration, the impossibility of moving anything further forces one of the summands to be periodic.

Unions and Periodicity The following lemma formalizes the first idea.

Lemma 71 (Union of Periodic Sets). *If $S = S_1 \cup \dots \cup S_m$ for finite nonempty sets S_i , then*

$$|S| + |H(S)| \geq \min_i (|S_i| + |H(S_i)|).$$

The intuition is easiest for two sets. If S_1 has a large period H_1 and S_2 has a large period H_2 , but their union has a much smaller period, then the two periodic patterns must be incompatible. Inside a coset of $H_1 + H_2$, one pattern consists of whole H_1 -cosets and the other of whole H_2 -cosets. These look like rows and columns of a grid. If the two patterns do not line up to give a period of the union, enough rows and columns must stick out to compensate for the lost stabilizer.

Proof. It is enough to prove the case $S = S_1 \cup S_2$; the general statement then follows by induction.

Write $H_i = H(S_i)$ and $H = H(S)$. Quotient first by $H_1 \cap H_2$. Since this subgroup stabilizes S_1 , S_2 , and S , all relevant cardinalities are divided by the same factor. We may therefore assume

$$H_1 \cap H_2 = \{0\}.$$

Set $h_i = |H_i|$ and $K = H_1 + H_2$. Then $|K| = h_1 h_2$. Inside each K -coset, the H_1 -cosets and H_2 -cosets form a rectangular grid: every H_1 -coset meets every H_2 -coset in exactly one point.

Consider one K -coset. Suppose S_1 occupies r of its H_1 -cosets and S_2 occupies c of its H_2 -cosets. Here $0 \leq r \leq h_2$ and $0 \leq c \leq h_1$. Within this K -coset,

$$|S_2 \setminus S_1| = c(h_2 - r), \quad |S_1 \setminus S_2| = r(h_1 - c).$$

Suppose first that both patterns are partial: $0 < r < h_2$ and $0 < c < h_1$. Then

$$c(h_2 - r)r(h_1 - c) = (r(h_2 - r))(c(h_1 - c)) \geq (h_2 - 1)(h_1 - 1).$$

Consequently either

$$|S_2 \setminus S_1| \geq h_1 - 1 \quad \text{or} \quad |S_1 \setminus S_2| \geq h_2 - 1.$$

Since $|H| \geq 1$, in the first case

$$|S| - |S_1| \geq |H_1| - |H|,$$

and in the second

$$|S| - |S_2| \geq |H_2| - |H|.$$

Either inequality is exactly the desired conclusion.

We may therefore assume that no K -coset contains two partial patterns.

Next suppose some non-full K -coset meets only S_1 , while another meets only S_2 . The first contributes at least h_1 points to $S_1 \setminus S_2$, and the second at least h_2 points to $S_2 \setminus S_1$. If $h_1 \geq h_2$, then

$$|S| - |S_2| \geq h_1 \geq h_2 - |H|,$$

and if $h_2 \geq h_1$, the symmetric inequality works for S_1 .

The only remaining case is that all non-full K -cosets that meet S belong to the same pattern, say S_1 . Any K -coset met by both S_1 and S_2 must then be completely filled: otherwise both patterns would be partial there. Hence every intersection of S with a K -coset is a union of complete H_1 -cosets. Therefore $S + H_1 = S$, so $H_1 \leq H$.

In this case

$$|S| + |H| \geq |S_1| + |H_1|.$$

Thus in every case

$$|S| + |H(S)| \geq \min(|S_1| + |H(S_1)|, |S_2| + |H(S_2)|).$$

□

Transfusion We now prove the basic inequality

$$\boxed{|A + B| + |H(A + B)| \geq |A| + |B|}.$$

The full theorem will follow by applying this to $A + H$ and $B + H$.

Fix $b \in B$. Consider all pairs (A', B') such that $A \subseteq A'$, $b \in B'$,

$$A' + B' \subseteq A + B, \quad |A'| + |B'| = |A| + |B|.$$

The original pair (A, B) is admissible, so choose an admissible pair (A_b, B_b) for which $|B_b|$ is as small as possible. Let $S_b = A_b + B_b$.

The key point is that this minimality forces A_b to be periodic.

Take any $t \in A_b - b$, so $b + t \in A_b$, and define

$$B' = B_b \cap (A_b - t), \quad A' = A_b \cup (B_b + t).$$

Think of this as moving from B_b into A_b every element y for which $y + t \notin A_b$: removing y from B_b is paired with adding $y + t$ to A_b .

The number of removed elements is exactly the number of new elements added, so

$$|A'| + |B'| = |A_b| + |B_b|.$$

Also $b \in B'$, because $b + t \in A_b$.

Most importantly, the sumset does not grow. Certainly $A_b + B' \subseteq A_b + B_b$. If $z + t \in B_b + t$ and $y \in B'$, then $y + t \in A_b$, so

$$(z + t) + y = z + (y + t) \in B_b + A_b.$$

Hence $A' + B' \subseteq A_b + B_b \subseteq A + B$, and (A', B') is again admissible.

By the minimality of $|B_b|$, no element can actually have been removed. Thus $B' = B_b$, or equivalently

$$B_b + t \subseteq A_b \quad \text{for every } t \in A_b - b.$$

This is where periodicity appears. Take $c \in B_b - b$ and $a \in A_b$. Writing $t = a - b$, we have $b + c \in B_b$, and therefore

$$a + c = (b + c) + (a - b) \in A_b.$$

Thus $A_b + c \subseteq A_b$. Since A_b is finite, translation preserves cardinality, so in fact $A_b + c = A_b$. Therefore

$$B_b - b \subseteq H(A_b).$$

This is the point of the transfusion argument: if some difference $c \in B_b - b$ failed to preserve A_b , then translating by a suitable $t \in A_b - b$ would expose an element of B_b that could still be moved into A_b , contradicting minimality. Having no legal transfusion left is exactly what forces every difference $B_b - b$ to become a period of A_b .

Since translation by $-b$ is injective,

$$|H(A_b)| \geq |B_b|.$$

Moreover $H(A_b) \leq H(S_b)$, because every period of A_b also stabilizes $A_b + B_b$. Finally $b \in B_b$, so $A_b + b \subseteq S_b$ and hence $|S_b| \geq |A_b|$. Therefore

$$|S_b| + |H(S_b)| \geq |A_b| + |B_b| = |A| + |B|.$$

Now let b vary over B . Since $A \subseteq A_b$ and $b \in B_b$, we have $A + b \subseteq S_b \subseteq A + B$. Hence

$$A + B = \bigcup_{b \in B} S_b.$$

Applying [Lemma 71](#) gives

$$|A + B| + |H(A + B)| \geq \min_{b \in B} (|S_b| + |H(S_b)|) \geq |A| + |B|.$$

We have proved the basic inequality

$$|A + B| + |H(A + B)| \geq |A| + |B|.$$

Proof of [Theorem 70](#). Let $H = H(A + B)$. Since $A + B$ is H -periodic,

$$(A + H) + (B + H) = A + B + H = A + B.$$

Its stabilizer is still H . Applying the preceding inequality to $A + H$ and $B + H$ gives

$$|A + B| + |H| \geq |A + H| + |B + H|,$$

which is precisely

$$|A + B| \geq |A + H| + |B + H| - |H|.$$

□

5.3 Extremal Set Theory

5.3.1 Bollobás Two Families Theorem

Theorem 72 (Bollobás Two Families Theorem). *Let*

$$(A_1, B_1), \dots, (A_m, B_m)$$

be pairs of finite sets such that

$$A_i \cap B_i = \emptyset$$

for every i , while

$$A_i \cap B_j \neq \emptyset$$

whenever

$$i \neq j.$$

Then

$$\sum_{i=1}^m \left(\frac{|A_i| + |B_i|}{|A_i|} \right)^{-1} \leq 1.$$

Proof. Choose a uniformly random ordering of all elements appearing in the sets.

For each i , let E_i be the event that every element of A_i appears before every element of B_i .

Among the

$$|A_i| + |B_i|$$

elements of $A_i \cup B_i$, every relative order is equally likely. The event E_i occurs exactly when the first $|A_i|$ positions are occupied by A_i , so

$$\Pr[E_i] = \binom{|A_i| + |B_i|}{|A_i|}^{-1}.$$

We claim that the events E_i are pairwise disjoint. Indeed, suppose both E_i and E_j occurred for

$$i \neq j.$$

Choose

$$x \in A_i \cap B_j, \quad y \in A_j \cap B_i.$$

Since E_i occurs,

x appears before y .

Since E_j occurs,

y appears before x ,

a contradiction.

Therefore

$$\sum_i \Pr[E_i] \leq 1,$$

which is exactly the desired inequality. □

Corollary 73. *If*

$$|A_i| = a, \quad |B_i| = b$$

for every i , then

$$m \leq \binom{a+b}{a}.$$

5.3.2 Sunflower Lemma

A family $S_1, \dots, S_r$ is an r -*sunflower* if there is a set C , called the *core*, such that

$$S_i \cap S_j = C \quad (i \neq j).$$

Equivalently, after deleting the common core, the petals $S_1 \setminus C, \dots, S_r \setminus C$ are pairwise disjoint.

Let $\text{Sun}(k, r)$ be the least N such that every family of N distinct sets of size at most k contains an r -sunflower. The classical Erdős–Rado lemma gives

$$\text{Sun}(k, r) \leq (r-1)^k k! + 1.$$

A better bound replaces the factorial by only a logarithmic factor in the base:

$$\boxed{\text{Sun}(k, r) \leq (Cr \log(2k))^k}$$

for a universal constant C .

The proof passes through a stronger probabilistic notion.

Definition 74 (Robust Sunflower). Let $\mathcal{R}$ be a family of at least two finite sets, with common intersection $C = \bigcap_{S \in \mathcal{R}} S$. Let W_γ contain each element of the universe independently with probability γ .

We call $\mathcal{R}$ a (γ, ε) -*robust sunflower* if

$$\boxed{\Pr[\exists S \in \mathcal{R} : S \subseteq C \cup W_\gamma] \geq 1 - \varepsilon.}$$

After removing the common core, this says that a random set of density γ contains an entire petal with probability at least $1 - \varepsilon$.

The useful point is that a robust sunflower gives many alternative ways for a random set to succeed. No particular petal needs to be likely to appear; the family as a whole is rich enough that a sparse random sample almost always contains *some* petal. This will eventually let us put different petals into disjoint random parts and obtain an ordinary sunflower.

Theorem 75 (Robust Sunflower Lemma). *There is a universal constant C such that, for $0 < \gamma \leq 1$ and $0 < \varepsilon \leq 1/2$, every family of at least*

$$\boxed{\left(\frac{C \log(2k/\varepsilon)}{\gamma} \right)^k}$$

distinct sets of size at most k contains a (γ, ε) -robust sunflower.

Proof. Set

$$R = \frac{C \log(2k/\varepsilon)}{\gamma},$$

rounding R up if necessary, and let $\mathcal{F}$ be a family of R^k sets. We induct on k .

For $k = 1$, the claim is immediate. Apart from the trivial case in which $\emptyset \in \mathcal{F}$, the sets are distinct singletons, so

$$\Pr[W_\gamma \text{ contains none of them}] \leq (1 - \gamma)^R \leq e^{-\gamma R} \leq \varepsilon$$

for sufficiently large C .

Now suppose $k > 1$. First consider the case where many members of $\mathcal{F}$ share a nonempty set Z . If $0 < |Z| < k$ and Z is contained in at least $R^{k-|Z|}$ members, restrict to those members and delete Z from each. This gives a family of at least $R^{k-|Z|}$ sets of size at most $k - |Z|$, so the inductive hypothesis gives a robust sunflower. Adding Z back to every set simply adds Z to its core.

We may therefore assume the *spread condition*

$$|S \in \mathcal{F} : Z \subseteq S| \leq R^{k-|Z|} \quad \text{for every nonempty } Z. \quad (1)$$

In particular, $\bigcap_{S \in \mathcal{F}} S = \emptyset$.

We shall prove that in this case $\mathcal{F}$ itself is a (γ, ε) -robust sunflower.

Covering We gradually expose a random set T , with the goal that T eventually contain some member of $\mathcal{F}$.

Say that $S \in \mathcal{F}$ is *m-covered by T* if there is some $S' \in \mathcal{F}$ such that

$$S' \subseteq T \cup S \quad \text{and} \quad |S' \setminus T| \leq m.$$

Equivalently, T is already within m elements of completing some member S' of the family, and all of the still-missing elements of S' lie inside S . Thus S acts as a reservoir from which those remaining elements may be supplied.

Initially every $S \in \mathcal{F}$ is k -covered by $\emptyset$, by taking $S' = S$. At the other extreme, if some S is 0-covered by T , then its witness S' satisfies $S' \subseteq T$. Thus reaching covering parameter 0 means that T actually contains a member of the family.

The proof repeatedly adds random blocks to T . The key lemma says that one block typically cuts the covering parameter in half for almost every set.

Lemma 76 (Cover Halving). *Suppose N members of $\mathcal{F}$ are m -covered by T . Let B be a uniformly random set of t currently unused elements, where*

$$\frac{|U|}{t} \leq \frac{R}{1000}.$$

Then with probability at least $1/2$, all but at most

$$\frac{R^k}{4^m}$$

of these N sets are $\lfloor m/2 \rfloor$ -covered by $T \cup B$.

Proof. Fix T , and call an m -covered set S *bad* if it is not $\lfloor m/2 \rfloor$ -covered by $T \cup B$.

For a bad set S , consider the part of S still outside the exposed set:

$$Z_S = S \setminus (T \cup B).$$

Let

$$\mathcal{Z}(B) = \left\{ S \setminus (T \cup B) : \begin{array}{l} S \in \mathcal{F} \text{ is } m\text{-covered by } T, \\ S \text{ is not } \lfloor m/2 \rfloor\text{-covered by } T \cup B \end{array} \right\}.$$

We call $Z \in \mathcal{Z}(B)$ *minimal* if it contains no proper subset that also belongs to $\mathcal{Z}(B)$.

Every bad set contains some minimal residual Z . Moreover, every minimal residual has size greater than $m/2$. Indeed, if $Z = S \setminus (T \cup B)$ had $|Z| \leq \lfloor m/2 \rfloor$, then taking $S' = S$ would itself show that S is $\lfloor m/2 \rfloor$ -covered by $T \cup B$, a contradiction.

The strategy is therefore:

first bound the expected number of minimal residuals of each size b , and then use the spread condition to bound how many bad sets can contain each such residual.

Let n' be the number of unused elements, so B is uniformly distributed over the $\binom{n'}{t}$ subsets of size t . For $b > m/2$, let $X_b(B)$ denote the number of minimal residuals $Z \in \mathcal{Z}(B)$ with $|Z| = b$.

We claim that

$$\mathbb{E}[X_b(B)] \leq 2^m \frac{\binom{n'}{t+b}}{\binom{n'}{t}}. \quad (2)$$

To see this, count pairs (B, Z) for which B is a t -set and Z is a minimal residual of size b . Since $Z \subseteq U \setminus (T \cup B)$, the sets B and Z are disjoint. Thus

$$Y := B \cup Z$$

has exactly $t + b$ elements. There are $\binom{n'}{t+b}$ possibilities for Y .

We now show that once Y is fixed, there are at most 2^m possible minimal residuals Z .

To bound the number of possible residuals for a fixed union $Y = B \cup Z$, we compress Y to at most m relevant elements.

Among all m -covered sets $Q \in \mathcal{F}$ satisfying

$$Q \setminus T \subseteq Y,$$

choose one, denoted Q_Y , minimizing $|Q_Y \setminus T|$. Such a set exists whenever $Y = B \cup Z$ arises from a bad set.

Because Q_Y is m -covered, there is some $Q' \in \mathcal{F}$ with $Q' \subseteq T \cup Q_Y$ and $|Q' \setminus T| \leq m$. Since $Q' \setminus T \subseteq Q_Y \setminus T \subseteq Y$, the minimality of Q_Y gives

$$|Q_Y \setminus T| \leq |Q' \setminus T| \leq m.$$

Thus the set

$$W_Y := Q_Y \setminus T$$

has at most m elements.

Now consider a particular decomposition

$$Y = B \sqcup Z$$

coming from a minimal residual Z . By definition, there is some bad set $S \in \mathcal{F}$ such that

$$Z = S \setminus (T \cup B).$$

In particular, $Z \subseteq S$.

We claim that Q_Y is also bad for this same choice of B . Suppose not. Then Q_Y is $\lfloor m/2 \rfloor$ -covered by $T \cup B$, so there is some $R \in \mathcal{F}$ such that

$$R \subseteq (T \cup B) \cup Q_Y \quad \text{and} \quad |R \setminus (T \cup B)| \leq \lfloor m/2 \rfloor.$$

But $Q_Y \setminus T \subseteq Y = B \cup Z$. Hence

$$(T \cup B) \cup Q_Y \subseteq T \cup B \cup Z.$$

Since $Z \subseteq S$, we further have

$$T \cup B \cup Z \subseteq (T \cup B) \cup S.$$

Therefore

$$R \subseteq (T \cup B) \cup S$$

while still

$$|R \setminus (T \cup B)| \leq \lfloor m/2 \rfloor.$$

But this says exactly that the original set S is $\lfloor m/2 \rfloor$ -covered by $T \cup B$, contradicting the fact that S was bad.

Hence Q_Y is bad. Its residual

$$Q_Y \setminus (T \cup B)$$

therefore belongs to the same family of bad residuals. Moreover,

$$Q_Y \setminus (T \cup B) \subseteq Y \setminus B = Z.$$

Since Z was inclusion-minimal among bad residuals, equality must hold:

$$Z = Q_Y \setminus (T \cup B).$$

In particular,

$$Z \subseteq Q_Y \setminus T = W_Y.$$

Therefore, once Y is fixed, every possible minimal residual Z is a subset of the same set W_Y of size at most m . There are at most

$$2^{|W_Y|} \leq 2^m$$

such subsets.

Thus each $(t+b)$ -element set Y gives rise to at most 2^m pairs (B, Z) .

Therefore the total number of pairs (B, Z) is at most

$$2^m \binom{n'}{t+b}.$$

Since B is uniform among the $\binom{n'}{t}$ possible t -sets, linearity of expectation, or equivalently averaging over B , gives [Equation \(2\)](#).

It remains to estimate the binomial ratio. Cancelling factorials gives

$$\frac{\binom{n'}{t+b}}{\binom{n'}{t}} = \frac{(n'-t)(n'-t-1)\cdots(n'-t-b+1)}{(t+1)(t+2)\cdots(t+b)} = \prod_{j=1}^b \frac{n'-t-j+1}{t+j}.$$

Each numerator is at most n' , while each denominator is at least t . Hence every factor is at most n'/t , and therefore

$$\frac{\binom{n'}{t+b}}{\binom{n'}{t}} \leq \left(\frac{n'}{t}\right)^b \leq \left(\frac{|U|}{t}\right)^b \leq \left(\frac{R}{1000}\right)^b.$$

Substituting into [Equation \(2\)](#),

$$\boxed{\mathbb{E}[X_b(B)] \leq 2^m \left(\frac{R}{1000}\right)^b.}$$

We can now count bad sets. For a fixed choice of B , every bad set contains at least one minimal residual Z . If $|Z| = b$, the spread condition [Equation \(1\)](#) says that Z is contained in at most R^{k-b} members of $\mathcal{F}$. Thus, if $Y(B)$ denotes the number of bad sets,

$$Y(B) \leq \sum_{b > m/2} R^{k-b} X_b(B).$$

Taking expectations and using linearity of expectation,

$$\mathbb{E}[Y(B)] \leq \sum_{b > m/2} R^{k-b} \mathbb{E}[X_b(B)] \leq \sum_{b > m/2} R^{k-b} 2^m \left(\frac{R}{1000} \right)^b = R^k 2^m \sum_{b > m/2} 1000^{-b}.$$

Since the first exponent in the last sum is at least $m/2$,

$$\sum_{b > m/2} 1000^{-b} \leq \frac{1000^{-m/2}}{1 - 1/1000}.$$

Consequently

$$\mathbb{E}[Y(B)] \leq R^k \left(\frac{2}{\sqrt{1000}} \right)^m \frac{1000}{999} \leq \frac{R^k}{8^m}.$$

Finally, Markov's inequality gives

$$\Pr \left[Y(B) > \frac{R^k}{4^m} \right] \leq \frac{R^k / 8^m}{R^k / 4^m} = \frac{1}{2^m} \leq \frac{1}{2}.$$

Thus with probability at least $1/2$, at most $R^k / 4^m$ of the currently m -covered sets fail to become $\lfloor m/2 \rfloor$ -covered. $\square$

Repeated Halving We now expose a small subset of W_γ in several random blocks and repeatedly apply the cover-halving lemma.

Let

$$\ell = C_0 \log(2k/\varepsilon)$$

for a sufficiently large constant C_0 . Add dummy elements to the universe if necessary so that, writing $n = |U|$,

$$\Pr \left[|W_\gamma| < \frac{\gamma n}{2} \right] \leq \frac{\varepsilon}{2}.$$

Adding dummy elements does not affect whether W_γ contains a member of $\mathcal{F}$.

Condition on $|W_\gamma| \geq \gamma n/2$. Choose uniformly inside W_γ a set

$$B = B_1 \sqcup \cdots \sqcup B_\ell$$

whose blocks all have size

$$t = \left\lfloor \frac{\gamma n}{2\ell} \right\rfloor.$$

By symmetry, conditioned on the previously exposed blocks, each next block is a uniformly random t -subset of the unused elements.

Since $\ell = C_0 \log(2k/\varepsilon)$ and $R = C \log(2k/\varepsilon)/\gamma$, choosing C sufficiently large relative to C_0 ensures

$$\frac{n}{t} \leq \frac{R}{1000}.$$

Thus [Lemma 76](#) applies at every round.

Initially all R^k sets are k -covered by $B^0 = \emptyset$. Suppose that at some stage we have many sets that are m -covered. On a successful round, all but at most $R^k/4^m$ of them become $\lfloor m/2 \rfloor$ -covered. On an unsuccessful round we simply do nothing: enlarging the exposed set cannot destroy an existing covering relation, so all surviving sets remain m -covered. Thus only successful rounds matter, and every successful round approximately halves m .

Conditioned on everything that happened previously, each round is successful with probability at least $1/2$. Hence the number of successful rounds stochastically dominates $\text{Bin}(\ell, 1/2)$. For sufficiently large C_0 , a Chernoff bound gives

$$\Pr[\text{fewer than } 1 + \lceil \log_2 k \rceil \text{ successful rounds}] \leq \frac{\varepsilon}{2}.$$

After $1 + \lceil \log_2 k \rceil$ successful halvings, the covering parameter has reached 0.

It remains to check that some set survives all the successful rounds. When the current positive covering parameter is m , at most $R^k/4^m$ sets are discarded. The successive positive values of m are distinct, so the total number discarded is at most

$$R^k \sum_{j \geq 1} 4^{-j} = \frac{R^k}{3}.$$

Thus at least $2R^k/3 > 0$ sets survive until the covering parameter becomes 0. Consequently B , and hence W_γ , contains some $S \in \mathcal{F}$.

The probability that $|W_\gamma| < \gamma n/2$ is at most $\varepsilon/2$, and the probability of obtaining too few successful rounds is also at most $\varepsilon/2$. Therefore

$$\Pr[\exists S \in \mathcal{F} : S \subseteq W_\gamma] \geq 1 - \varepsilon.$$

Since $\bigcap_{S \in \mathcal{F}} S = \emptyset$, this says exactly that $\mathcal{F}$ is a (γ, ε) -robust sunflower. $\square$

Corollary 77 (Sunflower Lemma). *There is a universal constant C such that*

$$\boxed{\text{Sun}(k, r) \leq (Cr \log(2k))^k.}$$

Proof. By increasing C , it is enough to consider a uniform family. Indeed, among sets of size at most k , one of the $k+1$ possible sizes occurs in at least a $1/(k+1)$ fraction of the family, and this polynomial loss is absorbed by increasing the constant in the base.

Thus let $\mathcal{F}$ be a sufficiently large q -uniform family for some $q \leq k$. Apply [Theorem 75](#) with

$$\gamma = \frac{1}{2r}, \quad \varepsilon = \frac{1}{2}.$$

We obtain a robust sunflower $\mathcal{R} \subseteq \mathcal{F}$ with core $C = \bigcap_{S \in \mathcal{R}} S$.

Since the members of $\mathcal{R}$ are distinct and all have the same size, no member equals C . Thus every petal $S \setminus C$ is nonempty.

Now assign every element outside C , independently and uniformly, to one of $2r$ bins $B_1, \dots, B_{2r}$. For each fixed j , the marginal distribution of B_j is exactly $W_{1/(2r)}$. Robustness therefore gives

$$\Pr[\exists S \in \mathcal{R} : S \setminus C \subseteq B_j] \geq \frac{1}{2}.$$

By linearity of expectation, the expected number of bins containing a complete petal is at least $2r \cdot \frac{1}{2} = r$. Hence some partition has at least r such bins.

Choose one petal from each of r distinct good bins. Since the bins are disjoint, these petals are pairwise disjoint. Adding back the common core C gives r members of $\mathcal{R}$ forming an r -sunflower. $\square$

Lower Bound

Proposition 78. *For every $k, r \geq 2$,*

$$\boxed{\text{Sun}(k, r) > (r - 1)^k.}$$

Proof. Partition the universe into k disjoint blocks $X_1, \dots, X_k$, each of size $r - 1$, and let $\mathcal{F}$ contain every transversal

$$S = \{x_1, \dots, x_k\}, \quad x_i \in X_i.$$

Then $|\mathcal{F}| = (r - 1)^k$.

Suppose r distinct members of $\mathcal{F}$ formed a sunflower. Fix a block X_i . The r sets choose r elements from only $r - 1$ possibilities, so two of them choose the same element $x \in X_i$.

In a sunflower, any element lying in two members belongs to the core and therefore to every member. Thus all r sets choose the same element from X_i .

The same argument applies independently to every block X_i . Hence the r sets make the same choice in every block and are therefore identical, contradicting their distinctness. Thus $\mathcal{F}$ contains no r -sunflower. $\square$

5.4 Ramsey Theory

5.4.1 Hales–Jewett Theorem

Let

$$[m] = \{0, 1, \dots, m - 1\}.$$

A *combinatorial line* in $[m]^N$ is obtained from a word $\tau(*) \in ([m] \cup \{*\})^N$ containing at least one wildcard. For $a \in [m]$, let $\tau(a)$ be the word obtained by replacing every occurrence of $*$ by a . The corresponding line is

$$\{\tau(0), \dots, \tau(m - 1)\}.$$

Let $HJ(c, m)$ denote the least N such that every c -coloring of $[m]^N$ contains a monochromatic combinatorial line.

For the quantitative bound, define the hierarchy

$$h_1(n) = 2n, \quad h_{r+1}(n) = h_r^n(1),$$

where h_r^n denotes n -fold composition. Thus $h_2(n) = 2^n$, $h_3(n)$ is a tower of 2's of height n , and h_4 iterates the tower function.

Theorem 79 (Hales–Jewett). *For every $c, m \geq 1$,*

$$\boxed{HJ(c, m) \leq \frac{1}{cm} h_4(c + m + 2).}$$

In particular, $HJ(c, m)$ is finite and has a primitive-recursive upper bound.

We induct on the alphabet size and force the symbols 0 and 1 to become indistinguishable. We may then merge them into a single symbol and invoke Hales–Jewett for an alphabet of size $m - 1$.

Proof of the reduction. We induct on m . The case $m = 1$ is trivial.

Assume the theorem for an alphabet of size $m - 1$, and set

$$\ell = HJ(c, m - 1).$$

We shall construct ℓ blocks

$$\tau_1(*), \dots, \tau_\ell(*)$$

with the following property:

$$\chi(\tau_1(\nu_1) \cdots \tau_\ell(\nu_\ell)) \quad (3)$$

is unchanged if any collection of coordinates ν_i is switched between 0 and 1.

Once this is achieved, 0 and 1 can be regarded as the same letter, leaving an alphabet of size $m - 1$. The induction hypothesis will then finish the proof.

Constructing the Blocks We choose the block lengths large enough for a repeated pigeonhole argument. Set $L_0 = 0$, and for $i = 1, \dots, \ell$ define

$$n_i = c^{m^{L_{i-1} + \ell - i}}, \quad L_i = L_{i-1} + n_i.$$

We claim that

$$HJ(c, m) \leq L_\ell.$$

Let

$$\chi : [m]^{L_\ell} \rightarrow [c]$$

be arbitrary, and partition the coordinates into consecutive blocks of lengths $n_1, \dots, n_\ell$.

We construct the block roots backwards, starting with $\tau_\ell(*)$ and ending with $\tau_1(*)$. Suppose $\tau_{i+1}, \dots, \tau_\ell$ have already been chosen.

For $0 \leq t \leq n_i$, let

$$W_t = 0^t 1^{n_i - t}.$$

Think of W_t as placing a boundary between a string of 0's and a string of 1's. We shall find two positions of this boundary that are indistinguishable to the coloring.

For each t , consider the function

$$\Phi_t : [m]^{L_{i-1}} \times [m]^{\ell-i} \longrightarrow [c]$$

defined by

$$\Phi_t(z, \nu_{i+1}, \dots, \nu_\ell) = \chi(z W_t \tau_{i+1}(\nu_{i+1}) \cdots \tau_\ell(\nu_\ell)).$$

The arguments of Φ_t describe everything outside the i -th block that can still affect its color: z is an arbitrary assignment to all earlier coordinates, while $\nu_{i+1}, \dots, \nu_\ell$ specify the already constructed later blocks.

There are

$$m^{L_{i-1} + \ell - i}$$

such contexts. A function assigning one of c colors to every context can therefore take at most

$$c^{m^{L_{i-1} + \ell - i}} = n_i$$

different values.

But we have $n_i + 1$ words

$$W_0, W_1, \dots, W_{n_i}.$$

By the pigeonhole principle, two of them, say W_t and W_u with $t < u$, induce exactly the same function:

$$\Phi_t = \Phi_u.$$

Define

$$\tau_i(*) = 0^t *^{u-t} 1^{n_i-u}.$$

Then

$$\tau_i(0) = W_u, \quad \tau_i(1) = W_t.$$

Thus changing the parameter of the i -th block from 0 to 1, or back again, does not change the color, no matter how the earlier coordinates or later block parameters are chosen.

Proceeding backwards through all ℓ blocks proves [Equation \(3\)](#).

Merging 0 and 1 For $\nu = (\nu_1, \dots, \nu_\ell) \in [m]^\ell$, write

$$\tau(\nu) = \tau_1(\nu_1) \cdots \tau_\ell(\nu_\ell).$$

By construction, $\chi(\tau(\nu))$ is unchanged whenever any coordinates of ν are switched between 0 and 1. We may therefore identify these two symbols. Let

$$\overline{[m]} = \{\star, 2, \dots, m-1\},$$

where $\star$ represents either 0 or 1. Define

$$\overline{\chi}(\bar{\nu}) = \chi(\tau(\nu)),$$

where each occurrence of $\star$ in $\bar{\nu}$ may be represented by either 0 or 1 in ν .

The invariance just proved makes $\overline{\chi}$ well defined. It is a c -coloring of

$$\overline{[m]}^\ell,$$

whose alphabet has size $m-1$.

Since $\ell = HJ(c, m-1)$, this coloring contains a monochromatic combinatorial line. Let

$$\sigma(*) \in (\overline{[m]} \cup \{\star\})^\ell$$

be a root for such a line.

Lifting the Line We now turn $\sigma(*)$ into a root in the original space $[m]^{L_\ell}$, block by block.

For coordinate i ,

- if $\sigma_i = *$, use the entire root $\tau_i(*)$;
- if $\sigma_i = j \in \{2, \dots, m-1\}$, use the fixed block $\tau_i(j)$;
- if $\sigma_i = \star$, use the fixed block $\tau_i(0)$.

Call the resulting root $\rho(*)$.

For $a = 2, \dots, m-1$, the word $\rho(a)$ maps exactly to the a -point of the monochromatic line for $\overline{\chi}$. The word $\rho(0)$ maps to its $\star$ -point.

The word $\rho(1)$ differs from $\rho(0)$ only by changing some block parameters from 0 to 1. By [Equation \(3\)](#), this does not change its color. Therefore

$$\chi(\rho(0)) = \chi(\rho(1)) = \cdots = \chi(\rho(m-1)).$$

Thus $\rho(*)$ gives a monochromatic combinatorial line in $[m]^{L_\ell}$. □

Quantitative Bound The preceding argument gives an explicit recursive bound: if $B(c, 1) = 1$, and for $m \geq 2$ we set $\ell = B(c, m - 1)$ and recursively

$$L_0 = 0, \quad L_i = L_{i-1} + c^{m^{L_{i-1} + \ell - i}}, \quad B(c, m) = L_\ell,$$

then

$$HJ(c, m) \leq B(c, m).$$

5.5 Polynomial Method

5.5.1 Combinatorial Nullstellensatz

For

$$\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{N}^n,$$

write

$$x^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}.$$

We say that x^α is *maximal in the support* of f if

$$[x^\alpha]f \neq 0$$

and there is no different monomial x^β in the support of f with

$$\beta_i \geq \alpha_i \quad \text{for every } i.$$

We give an upgraded version of the Combinatorial Nullstellensatz is an exact formula for a coefficient.

Lemma 80 (Univariate Coefficient Formula). *Let*

$$A \subseteq \mathbb{F}, \quad |A| = d + 1,$$

and define

$$\phi_A(t) = \prod_{a \in A} (t - a).$$

Then for every polynomial g of degree at most d ,

$$\boxed{[t^d]g(t) = \sum_{a \in A} \frac{g(a)}{\phi'_A(a)}}.$$

In particular,

$$\sum_{a \in A} \frac{a^j}{\phi'_A(a)} = \begin{cases} 0, & j < d, \\ 1, & j = d. \end{cases}$$

Proof. Lagrange interpolation gives

$$g(t) = \sum_{a \in A} g(a) \frac{\phi_A(t)}{(t - a)\phi'_A(a)}.$$

Each polynomial

$$\frac{\phi_A(t)}{t - a}$$

is monic of degree d . Taking the coefficient of t^d gives the result. □

Theorem 81 (Coefficient Formula). *Let*

$$f \in \mathbb{F}[x_1, \dots, x_n],$$

and fix

$$d = (d_1, \dots, d_n).$$

For each i , choose

$$A_i \subseteq \mathbb{F}, \quad |A_i| = d_i + 1,$$

and define

$$\phi_i(t) = \prod_{a \in A_i} (t - a).$$

Suppose either

1. $x^d = x_1^{d_1} \cdots x_n^{d_n}$ is maximal in the support of f , or
- 2.

$$\deg f \leq d_1 + \cdots + d_n.$$

Then

$$[x^d]f = \sum_{a \in A_1 \times \cdots \times A_n} \frac{f(a)}{\prod_{i=1}^n \phi'_i(a_i)}.$$

In the second case the coefficient on the left is allowed to be zero.

Proof. By linearity, evaluate the right-hand side on a monomial

$$x^\alpha.$$

The sum factors:

$$\sum_a \frac{a^\alpha}{\prod_i \phi'_i(a_i)} = \prod_{i=1}^n \left(\sum_{a_i \in A_i} \frac{a_i^{\alpha_i}}{\phi'_i(a_i)} \right).$$

For $\alpha = d$, every factor equals 1 by [Lemma 80](#).

Now consider any other monomial x^α appearing in f . Under the first hypothesis, maximality of x^d implies that

$$\alpha_i < d_i$$

for some i .

Under the second hypothesis, if $\alpha_i \geq d_i$ for every i , then

$$|\alpha| \geq \sum_i d_i.$$

Since $|\alpha| \leq \deg f \leq \sum_i d_i$, equality would hold everywhere and $\alpha = d$. Thus again, for every $\alpha \neq d$, some coordinate satisfies

$$\alpha_i < d_i.$$

The corresponding factor is 0 by [Lemma 80](#). Hence every monomial except x^d is annihilated, while x^d contributes exactly its coefficient. $\square$

Theorem 82 (Combinatorial Nullstellensatz). *Suppose*

$$x_1^{d_1} \cdots x_n^{d_n}$$

is a maximal monomial in the support of

$$f \in \mathbb{F}[x_1, \dots, x_n].$$

If

$$|A_i| > d_i$$

for every i , then there exists

$$a \in A_1 \times \cdots \times A_n$$

such that

$$f(a) \neq 0.$$

Proof. Restrict each A_i to any $d_i + 1$ elements. By [Theorem 81](#),

$$0 \neq [x^d]f = \sum_a \frac{f(a)}{\prod_i \phi'_i(a_i)}.$$

Therefore not every evaluation $f(a)$ can be zero. □

The usual formulation, which assumes

$$\deg f = d_1 + \cdots + d_n \quad \text{and} \quad [x^d]f \neq 0,$$

is an immediate special case: the total-degree condition automatically makes x^d maximal.

Alon–Füredi Combinatorial Nullstellensatz guarantees one nonzero grid point. Alon–Füredi gives the sharp minimum number of them.

For positive integers $a_1, \dots, a_n$ define

$$\mu(a_1, \dots, a_n; d) = \min_{\substack{0 \leq d_i \leq a_i - 1 \\ \sum_i d_i \leq d}} \prod_{i=1}^n (a_i - d_i).$$

Theorem 83 (Alon–Füredi). *Let*

$$A = A_1 \times \cdots \times A_n, \quad |A_i| = a_i,$$

and suppose

$$f \in \mathbb{F}[x_1, \dots, x_n]$$

is not identically zero on A . If

$$\deg f \leq d,$$

then

$$|\{x \in A : f(x) \neq 0\}| \geq \mu(a_1, \dots, a_n; d).$$

Proof. Reduce f modulo

$$\prod_{a \in A_i} (x_i - a)$$

for every variable x_i . This preserves all values on A , does not increase total degree, and allows us to assume

$$\deg_{x_i} f < a_i.$$

We induct on n .

For $n = 1$, a nonzero polynomial of degree at most d has at most d roots, so it is nonzero on at least

$$a_1 - \min(d, a_1 - 1) = \mu(a_1; d)$$

points.

Now suppose $n > 1$. Let

$$S = \{c \in A_n : f(x_1, \dots, x_{n-1}, c) \equiv 0\}, \quad s = |S|.$$

Since f is nonzero somewhere on A ,

$$s < a_n.$$

Every $x_n - c$ for $c \in S$ divides f , so

$$f(x) = \left(\prod_{c \in S} (x_n - c) \right) g(x)$$

with

$$\deg g \leq d - s.$$

For every

$$c \in A_n \setminus S,$$

the slice

$$g_c(x_1, \dots, x_{n-1}) = g(x_1, \dots, x_{n-1}, c)$$

is nonzero. By induction, it is nonzero on at least

$$\mu(a_1, \dots, a_{n-1}; d - s)$$

points.

Therefore

$$|\{f \neq 0\}| \geq (a_n - s)\mu(a_1, \dots, a_{n-1}; d - s).$$

If

$$d_1, \dots, d_{n-1}$$

achieve the latter minimum, set

$$d_n = s.$$

Then

$$0 \leq d_n \leq a_n - 1, \quad \sum_i d_i \leq d,$$

so this choice is feasible in the definition of

$$\mu(a_1, \dots, a_n; d).$$

Thus

$$\mu(a_1, \dots, a_n; d) \leq (a_n - s)\mu(a_1, \dots, a_{n-1}; d - s),$$

which proves the theorem. □

The bound is sharp. For any feasible $d_1, \dots, d_n$, choose

$$S_i \subseteq A_i, \quad |S_i| = d_i,$$

and take

$$f(x) = \prod_{i=1}^n \prod_{a \in S_i} (x_i - a).$$

Then

$$\deg f = \sum_i d_i,$$

and f is nonzero on exactly

$$\prod_i (a_i - d_i)$$

grid points.

Ryser's Permanent Formula The coefficient formula can be significantly more useful than the ordinary version, as is shown in the following.

Theorem 84 (Ryser's Formula). *For*

$$A = (a_{ij}) \in \mathbb{F}^{n \times n},$$

$$\text{per}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_{i=1}^n \left(\sum_{j \in S} a_{ij} \right).$$

Consequently the permanent can be computed in

$$O(n2^n)$$

arithmetic operations.

Proof. Define

$$P(x_1, \dots, x_n) = \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j \right).$$

Choosing one variable from each factor shows

$$[x_1 x_2 \cdots x_n] P = \text{per}(A).$$

Apply [Theorem 81](#) with

$$A_j = \{0, 1\}, \quad d_j = 1.$$

For

$$\begin{aligned} \phi(t) &= t(t-1), \\ \phi'(0) &= -1, \quad \phi'(1) = 1. \end{aligned}$$

Thus

$$\text{per}(A) = \sum_{\varepsilon \in \{0,1\}^n} (-1)^{n-\sum_j \varepsilon_j} P(\varepsilon).$$

Writing

$$S = \{j : \varepsilon_j = 1\}$$

gives exactly

$$\text{per}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_i \sum_{j \in S} a_{ij}.$$

□

Chevalley–Warning The same coefficient identity can turn a degree bound into a congruence for the number of solutions of a polynomial system.

Theorem 85 (Chevalley–Warning). *Let*

$$f_1, \dots, f_m \in \mathbb{F}_q[x_1, \dots, x_n]$$

satisfy

$$\sum_{j=1}^m \deg f_j < n.$$

Then the number N of common zeros of

$$f_1, \dots, f_m$$

in $\mathbb{F}_q^n$ is divisible by the characteristic p of $\mathbb{F}_q$.

Proof. Define

$$P(x) = \prod_{j=1}^m (1 - f_j(x)^{q-1}).$$

For every

$$a \in \mathbb{F}_q^n,$$

Fermat's theorem gives

$$P(a) = \begin{cases} 1, & f_1(a) = \dots = f_m(a) = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Hence in $\mathbb{F}_q$,

$$\sum_{a \in \mathbb{F}_q^n} P(a) = N.$$

Also,

$$\deg P \leq (q-1) \sum_j \deg f_j < n(q-1).$$

Therefore

$$[x_1^{q-1} \dots x_n^{q-1}]P = 0.$$

Use the degree-bounded form of [Theorem 81](#) with

$$A_i = \mathbb{F}_q, \quad d_i = q-1.$$

Since

$$\phi_i(t) = \prod_{a \in \mathbb{F}_q} (t - a) = t^q - t,$$

we have

$$\phi'_i(a) = -1$$

for every $a \in \mathbb{F}_q$. Thus

$$0 = [x_1^{q-1} \dots x_n^{q-1}]P = (-1)^n \sum_{a \in \mathbb{F}_q^n} P(a) = (-1)^n N$$

inside $\mathbb{F}_q$.

Hence

$$N \equiv 0 \pmod{p}.$$

□

5.5.2 Multiplicity Schwartz–Zippel

Ordinary Schwartz–Zippel counts zeros. The multiplicity version counts a zero several times according to how many derivatives vanish there.

For a multi-index

$$\alpha \in \mathbb{N}^n,$$

define the Hasse derivative $D^\alpha f$ by

$$f(x+z) = \sum_{\alpha \in \mathbb{N}^n} D^\alpha f(x) z^\alpha.$$

Definition 86 (Multiplicity). For a nonzero polynomial f and a point a , define

$$\text{mult}(f, a) = \min\{|\alpha| : D^\alpha f(a) \neq 0\}.$$

Thus

$$\text{mult}(f, a) = 0$$

exactly when $f(a) \neq 0$.

We use the elementary fact

$$\text{mult}(D^\alpha f, a) \geq \text{mult}(f, a) - |\alpha|.$$

This follows immediately from the Taylor expansion: taking a Hasse derivative of order $|\alpha|$ can lower the smallest nonzero total degree by at most $|\alpha|$.

Theorem 87 (Multiplicity Schwartz–Zippel). *Let*

$$0 \neq f \in \mathbb{F}[x_1, \dots, x_n]$$

have total degree at most d , and let

$$S \subseteq \mathbb{F}$$

be finite. Then

$$\boxed{\sum_{a \in S^n} \text{mult}(f, a) \leq d|S|^{n-1}.$$

Consequently, for every $r \geq 1$,

$$\boxed{|\{a \in S^n : \text{mult}(f, a) \geq r\}| \leq \frac{d}{r}|S|^{n-1}.$$

Proof. We induct on n .

For $n = 1$, this is the usual fact that the sum of the multiplicities of the roots of a nonzero degree- d polynomial is at most d .

Now write

$$f(x_1, \dots, x_n) = \sum_{j=0}^t f_j(x_1, \dots, x_{n-1}) x_n^j,$$

where

$$f_t \neq 0.$$

Then

$$\deg f_t \leq d - t.$$

Fix

$$a' = (a_1, \dots, a_{n-1}) \in S^{n-1}$$

and put

$$r(a') = \text{mult}(f_t, a').$$

Choose a multi-index α of size $r(a')$ such that

$$D^\alpha f_t(a') \neq 0.$$

Then the univariate polynomial

$$g(T) = D^{(\alpha, 0)} f(a', T)$$

is nonzero and has degree exactly t .

For every $b \in S$,

$$\text{mult}(f, (a', b)) \leq r(a') + \text{mult}(g, b).$$

Summing over b and using the univariate multiplicity bound,

$$\sum_{b \in S} \text{mult}(f, (a', b)) \leq |S|r(a') + t.$$

Now sum over a' :

$$\begin{aligned} \sum_{a \in S^n} \text{mult}(f, a) &\leq |S| \sum_{a' \in S^{n-1}} \text{mult}(f_t, a') + t|S|^{n-1} \\ &\leq |S|(d-t)|S|^{n-2} + t|S|^{n-1} \\ &= d|S|^{n-1}. \end{aligned}$$

The second claim follows by Markov's inequality. □

5.5.3 Multilinear-Term Detection

Let $\mathbb{F}$ be a field of characteristic 2, and let $f \in \mathbb{F}[x_1, \dots, x_n]$ have total degree at most k .

For $r = 1, \dots, k$, introduce a vector of new variables

$$z_r = (z_{r,1}, \dots, z_{r,n}).$$

For $J \subseteq [k]$, write $z_J = \sum_{r \in J} z_r$.

Define

$$\boxed{\phi_k f(z_1, \dots, z_k) = \sum_{J \subseteq [k]} f(z_J).}$$

Thus $\phi_k f$ is a polynomial in the kn variables $z_{r,j}$.

The operator is most naturally understood as a mixed forward difference.

Forward-Difference Interpretation For a direction $h \in \mathbb{F}^n$, define

$$\Delta_h f(x) = f(x+h) - f(x).$$

Iterating gives

$$(\Delta_{z_1} \cdots \Delta_{z_k} f)(0) = \sum_{J \subseteq [k]} (-1)^{k-|J|} f(z_J).$$

Since $\text{char } \mathbb{F} = 2$, the signs disappear, so

$$\boxed{\phi_k f(z_1, \dots, z_k) = (\Delta_{z_1} \cdots \Delta_{z_k} f)(0).}$$

Thus ϕ_k is exactly the k -fold mixed finite difference of f .

There is an equivalent coefficient-extraction viewpoint that makes its effect on monomials immediate. Introduce auxiliary scalar variables $t_1, \dots, t_k$ and define

$$P_z(t_1, \dots, t_k) = f\left(\sum_{r=1}^k t_r z_r\right).$$

For $J \subseteq [k]$, let $1_J \in \{0, 1\}^k$ be its indicator vector. Then

$$P_z(1_J) = f\left(\sum_{r \in J} z_r\right) = f(z_J),$$

and consequently

$$\phi_k f(z_1, \dots, z_k) = \sum_{J \subseteq [k]} P_z(1_J).$$

Lemma 88 (Mixed Difference as Coefficient Extraction). *If $\deg f \leq k$, then*

$$\phi_k f(z_1, \dots, z_k) = [t_1 \cdots t_k] f\left(\sum_{r=1}^k t_r z_r\right).$$

Proof. Since $\deg f \leq k$, the polynomial $P_z(t)$ has total degree at most k .

Consider a monomial

$$c_\alpha(z) t_1^{\alpha_1} \cdots t_k^{\alpha_k}$$

of P_z . When we sum over $t = 1_J$, this monomial contributes $c_\alpha(z)$ exactly for those J containing every r with $\alpha_r > 0$. Hence its total contribution is

$$2^{k - |\text{supp}(\alpha)|} c_\alpha(z).$$

In characteristic 2, this is zero unless

$$|\text{supp}(\alpha)| = k.$$

So only monomials involving all $t_1, \dots, t_k$ survive. But P_z has total degree at most k , and a monomial involving all k variables already has degree at least k . Therefore the only possible surviving monomial is

$$t_1 t_2 \cdots t_k.$$

Its coefficient is exactly the claimed expression. □

Equivalently, the k -fold finite difference extracts the part of $f(t_1 z_1 + \cdots + t_k z_k)$ in which every direction z_r is used exactly once.

Lemma 89. *Let*

$$M = x_{i_1} \cdots x_{i_d}$$

be a monomial of degree $d \leq k$.

1. *If $d < k$, then $\phi_k M = 0$.*

2. If $d = k$, then

$$\boxed{\phi_k M = \text{perm } Z_M = \det Z_M,}$$

where

$$Z_M = \begin{pmatrix} z_{1,i_1} & z_{1,i_2} & \cdots & z_{1,i_k} \\ z_{2,i_1} & z_{2,i_2} & \cdots & z_{2,i_k} \\ \vdots & \vdots & & \vdots \\ z_{k,i_1} & z_{k,i_2} & \cdots & z_{k,i_k} \end{pmatrix}.$$

In particular, $\phi_k M = 0$ if M is nonmultilinear, while $\phi_k M \neq 0$ if M is multilinear.

Proof. By Lemma 88,

$$\phi_k M = [t_1 \cdots t_k] \prod_{\ell=1}^d \left(\sum_{r=1}^k t_r z_{r,i_\ell} \right).$$

If $d < k$, this product has degree d , so it cannot contain $t_1 \cdots t_k$. Hence $\phi_k M = 0$.

Now suppose $d = k$. To obtain $t_1 \cdots t_k$, the k factors must choose the k variables $t_1, \dots, t_k$ exactly once each. These choices are indexed by permutations $\sigma \in S_k$, so

$$\phi_k M = \sum_{\sigma \in S_k} \prod_{\ell=1}^k z_{\sigma(\ell), i_\ell} = \text{perm } Z_M.$$

In characteristic 2, $1 = -1$, so the signs in the determinant disappear:

$$\text{perm } Z_M = \det Z_M.$$

If M is nonmultilinear, then $i_a = i_b$ for some $a \neq b$. The corresponding columns of Z_M are identical, so $\det Z_M = 0$.

If M is multilinear, then $i_1, \dots, i_k$ are distinct. The determinant is nonzero: for example, the identity-permutation term

$$z_{1,i_1} z_{2,i_2} \cdots z_{k,i_k}$$

does not occur from any other permutation. □

This explains both ingredients in the construction. Finite differences kill degree below k , while characteristic 2 turns the permanent into a determinant. Repeated original variables then become repeated determinant columns and vanish.

Theorem 90 (Multilinear Detection Operator). *Let $f \in \mathbb{F}[x_1, \dots, x_n]$ have degree at most k , where $\text{char } \mathbb{F} = 2$. Then*

$$\boxed{\phi_k f \neq 0 \iff f \text{ contains a degree-}k \text{ multilinear monomial.}}$$

Proof. Write

$$f = \sum_{I \in \binom{[n]}{k}} c_I x_I + g,$$

where g contains no degree- k multilinear monomial.

By Lemma 89, every term of degree less than k and every nonmultilinear term of degree k is annihilated. Therefore

$$\phi_k f = \sum_{I \in \binom{[n]}{k}} c_I \det Z_I.$$

Different I 's cannot cancel. Indeed, every monomial in $\det Z_I$ contains exactly one variable from each row and has second indices precisely the elements of I . Thus if $I \neq I'$, the polynomials $\det Z_I$ and $\det Z_{I'}$ have disjoint monomial supports.

Hence $\phi_k f \equiv 0$ exactly when every $c_I = 0$. $\square$

Black-Box Detection The finite-difference formulation has an algorithmic advantage: evaluating $\phi_k f$ requires only evaluations of f . Geometrically, we evaluate f at the 2^k vertices

$$z_J = \sum_{r \in J} z_r \quad (J \subseteq [k])$$

of the k -dimensional parallelepiped generated by $z_1, \dots, z_k$.

Theorem 91 (Black-Box Multilinear-Term Detection). *Let*

$$f(x, y) \in \mathbb{F}[x_1, \dots, x_n, y_1, \dots, y_m],$$

where $\text{char } \mathbb{F} = 2$. Suppose f has degree at most k in the x -variables and degree at most d in the y -variables. There is a randomized one-sided-error algorithm deciding whether f contains a monomial that is multilinear of degree k in x .

One test uses exactly 2^k black-box evaluations of f . If every random coordinate is chosen independently and uniformly from a finite $S \subseteq \mathbb{F}$, then whenever such a monomial exists, the test succeeds with probability at least

$$\boxed{1 - \frac{d+k}{|S|}}.$$

Proof. Define the operator acting only on x by

$$\phi_k^x f(z_1, \dots, z_k, y) = \sum_{J \subseteq [k]} f(z_J, y).$$

Write the degree- k multilinear part in x as

$$f(x, y) = \sum_{I \in \binom{[n]}{k}} \lambda_I(y) x_I + g(x, y),$$

where g contains no degree- k multilinear monomial in x . Then

$$\phi_k^x f(z, y) = \sum_I \lambda_I(y) \det Z_I.$$

Since the determinant supports for distinct I 's are disjoint,

$$\phi_k^x f \neq 0$$

if and only if at least one coefficient polynomial $\lambda_I(y)$ is nonzero.

Each determinant has degree k in the z -variables, while each λ_I has degree at most d in y . Hence

$$\deg(\phi_k^x f) \leq d + k.$$

Schwartz–Zippel therefore gives

$$\Pr[\phi_k^x f(z_1, \dots, z_k, y) = 0] \leq \frac{d+k}{|S|}$$

whenever the polynomial is nonzero.

For fixed $z_1, \dots, z_k, y$, its value is

$$\phi_k^x f(z_1, \dots, z_k, y) = \sum_{J \subseteq [k]} f\left(\sum_{r \in J} z_r, y\right),$$

which requires exactly one black-box query for each of the 2^k subsets $J \subseteq [k]$.

If no desired monomial exists, then $\phi_k^x f \equiv 0$, so the algorithm never produces a false positive. $\square$

Algorithm 7 Black-Box Multilinear-Term Detection

Input: Black-box access to $f(x, y)$, target degree k , finite set S

Output: Whether f contains a degree- k multilinear monomial in x

```

1: Choose every coordinate of  $z_1, \dots, z_k$  and  $y$  independently and uniformly from  $S$ 
2:  $R \leftarrow 0$ 
3: for every  $J \subseteq [k]$  do
4:    $x^{(J)} \leftarrow \sum_{r \in J} z_r$ 
5:    $R \leftarrow R + f(x^{(J)}, y)$ 
6: if  $R \neq 0$  then
7:   return Yes
8: else
9:   return No
```

Repeating the test independently decreases the false-negative probability geometrically. If the base field is too small for a useful Schwartz–Zippel bound, one may instead evaluate over a sufficiently large finite extension field of characteristic 2.

5.6 Probabilistic Method

5.6.1 Isolation Lemma

Let

$$\mathcal{F} \subseteq 2^{[n]}$$

be any nonempty family of feasible sets. Assign independently chosen random integer weights

$$w_i \in [M]$$

to the ground elements, and define

$$w(S) = \sum_{i \in S} w_i.$$

Theorem 92 (Isolation Lemma). *With probability at least*

$$1 - \frac{n}{M},$$

there is a unique minimum-weight set in $\mathcal{F}$.

In particular, choosing

$$M = 2n$$

gives uniqueness with probability at least $1/2$.

Proof. For each $i \in [n]$, fix all weights except w_i .

Let

$$A_i = \min_{\substack{S \in \mathcal{F} \\ i \in S}} \sum_{j \in S \setminus \{i\}} w_j$$

and

$$B_i = \min_{\substack{S \in \mathcal{F} \\ i \notin S}} w(S),$$

with the convention that a minimum over an empty family is $+\infty$.

Call i *confused* if

$$A_i + w_i = B_i.$$

Once all the other weights have been fixed, there is at most one value of w_i that can satisfy this equality. Therefore

$$\Pr[i \text{ is confused}] \leq \frac{1}{M}.$$

Now suppose the minimum-weight feasible set is not unique. Let S, T be two distinct minimum-weight members of $\mathcal{F}$, and choose

$$i \in S \Delta T.$$

Assume without loss of generality that

$$i \in S \setminus T.$$

Then

$$A_i + w_i \leq w(S), \quad B_i \leq w(T).$$

But S and T already have globally minimum weight, so equality must hold:

$$A_i + w_i = B_i.$$

Thus some element is confused.

Hence

$$\Pr[\text{minimum is not unique}] \leq \Pr[\exists i \text{ confused}] \leq \sum_i \frac{1}{M} = \frac{n}{M}.$$

□

5.7 Concentration Inequalities

Most concentration inequalities come from the same idea:

$$\Pr[X \geq t] = \Pr[e^{\theta X} \geq e^{\theta t}] \leq e^{-\theta t} \mathbb{E} e^{\theta X},$$

followed by a bound on the moment generating function.

5.7.1 Hoeffding

Lemma 93 (Hoeffding's Lemma). *If $\mathbb{E}X = 0$ and $a \leq X \leq b$, then for every $\theta \in \mathbb{R}$,*

$$\boxed{\mathbb{E} e^{\theta X} \leq \exp\left(\frac{\theta^2(b-a)^2}{8}\right)}.$$

Proof. Let

$$\psi(\theta) = \log \mathbb{E} e^{\theta X}$$

be the cumulant generating function. Define the modified expectation

$$\mathbb{E}_\theta[g(X)] = \frac{\mathbb{E}[g(X)e^{\theta X}]}{\mathbb{E} e^{\theta X}}.$$

Differentiating gives

$$\psi'(\theta) = \mathbb{E}_\theta X$$

and

$$\boxed{\psi''(\theta) = \mathbb{E}_\theta X^2 - (\mathbb{E}_\theta X)^2 = \text{Var}_\theta(X).}$$

The modification changes the distribution of X , but not its support, so under the modified distribution we still have $a \leq X \leq b$. Any random variable $Y \in [a, b]$ satisfies

$$\text{Var}(Y) = \min_c \mathbb{E}(Y - c)^2 \leq \mathbb{E} \left(Y - \frac{a+b}{2} \right)^2 \leq \frac{(b-a)^2}{4}.$$

Therefore, for every θ ,

$$\psi''(\theta) \leq \frac{(b-a)^2}{4}.$$

Finally,

$$\psi(0) = 0, \quad \psi'(0) = \mathbb{E} X = 0.$$

By Taylor's theorem, for some ξ between 0 and θ ,

$$\psi(\theta) = \psi(0) + \theta\psi'(0) + \frac{\theta^2}{2}\psi''(\xi).$$

Therefore

$$\psi(\theta) = \frac{\theta^2}{2}\psi''(\xi) \leq \frac{\theta^2}{2} \cdot \frac{(b-a)^2}{4} = \frac{\theta^2(b-a)^2}{8}.$$

Exponentiating gives

$$\mathbb{E} e^{\theta X} \leq \exp \left(\frac{\theta^2(b-a)^2}{8} \right).$$

□

Theorem 94 (Hoeffding's Inequality). *Let $X_1, \dots, X_n$ be independent with*

$$a_i \leq X_i \leq b_i.$$

Then

$$\boxed{\Pr \left[\sum_i (X_i - \mathbb{E} X_i) \geq t \right] \leq \exp \left(-\frac{2t^2}{\sum_i (b_i - a_i)^2} \right).}$$

The same bound holds for the lower tail.

Proof. Apply [Lemma 93](#) independently:

$$\mathbb{E}e^{\theta \sum_i (X_i - \mathbb{E}X_i)} \leq \exp\left(\frac{\theta^2}{8} \sum_i (b_i - a_i)^2\right).$$

The exponential-moment bound gives

$$\Pr[\dots] \leq \exp\left(-\theta t + \frac{\theta^2}{8} \sum_i (b_i - a_i)^2\right).$$

Optimize over θ . □

5.7.2 Chernoff

Theorem 95 (Chernoff Bounds). *Let*

$$X = \sum_i X_i$$

where the X_i are independent Bernoulli variables, and let

$$\mu = \mathbb{E}X.$$

Then for $\delta \geq 0$,

$$\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1 + \delta}}\right)^\mu \leq \exp\left(-\frac{\delta^2}{2 + \delta}\mu\right).$$

For $0 \leq \delta \leq 1$,

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{-\delta^2 \mu / 2}.$$

Proof. For a Bernoulli variable of mean p ,

$$\mathbb{E}e^{\theta X_i} = 1 + p(e^\theta - 1) \leq \exp(p(e^\theta - 1)).$$

Hence

$$\mathbb{E}e^{\theta X} \leq \exp(\mu(e^\theta - 1)).$$

Therefore

$$\Pr[X \geq (1 + \delta)\mu] \leq \exp(-\theta(1 + \delta)\mu + \mu(e^\theta - 1)).$$

Set

$$e^\theta = 1 + \delta.$$

This gives the exact first bound. The simpler forms follow from elementary calculus.

The lower tail is identical with $\theta < 0$. □

5.7.3 Bernstein

Lemma 96 (Bernstein MGF Bound). *If*

$$\mathbb{E}X = 0, \quad |X| \leq R,$$

then for

$$0 \leq \theta < 3/R,$$

$$\log \mathbb{E}e^{\theta X} \leq \frac{\theta^2 \mathbb{E}X^2}{2(1 - \theta R/3)}.$$

Proof. For $k \geq 2$,

$$k! \geq 2 \cdot 3^{k-2},$$

and

$$|X|^k \leq R^{k-2} X^2.$$

Thus

$$\begin{aligned} e^{\theta X} &\leq 1 + \theta X + X^2 \sum_{k \geq 2} \frac{\theta^k R^{k-2}}{k!} \\ &\leq 1 + \theta X + \frac{\theta^2 X^2}{2(1 - \theta R/3)}. \end{aligned}$$

Take expectations, use $\mathbb{E}X = 0$, and use

$$1 + x \leq e^x.$$

□

Theorem 97 (Bernstein's Inequality). *Let $X_1, \dots, X_n$ be independent, mean zero, and satisfy*

$$|X_i| \leq R.$$

Set

$$\sigma^2 = \sum_i \mathbb{E}X_i^2.$$

Then

$$\Pr \left[\sum_i X_i \geq t \right] \leq \exp \left(-\frac{t^2}{2(\sigma^2 + Rt/3)} \right).$$

Proof. By [Lemma 96](#),

$$\Pr \left[\sum_i X_i \geq t \right] \leq \exp \left(-\theta t + \frac{\theta^2 \sigma^2}{2(1 - \theta R/3)} \right).$$

Take

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

□

5.8 Martingales

The exponential-moment method extends almost unchanged to martingales. The main difference is that moment-generating-function bounds are applied conditionally at each step.

Lemma 98 (Bounded Optional Stopping). *Let $(M_k)_{k=0}^n$ be a supermartingale and let $\tau \leq n$ be a stopping time. Then*

$$\mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

For a martingale, equality holds.

Proof. Write

$$M_\tau - M_0 = \sum_{i=1}^n \mathbf{1}_{\{\tau \geq i\}} (M_i - M_{i-1}).$$

Since

$$\{\tau \geq i\} \in \mathcal{F}_{i-1},$$

we have

$$\begin{aligned}\mathbb{E}[M_\tau - M_0] &= \sum_{i=1}^n \mathbb{E} \left[\mathbf{1}_{\{\tau \geq i\}} \mathbb{E}[M_i - M_{i-1} \mid \mathcal{F}_{i-1}] \right] \\ &\leq 0.\end{aligned}$$

For a martingale, every term is zero. □

A useful consequence is the martingale analogue of the maximal form of Markov's inequality.

Lemma 99 (Ville's Inequality). *Let $(M_k)_{k=0}^n$ be a nonnegative supermartingale. Then for every $a > 0$,*

$$\Pr \left[\max_{0 \leq k \leq n} M_k \geq a \right] \leq \frac{\mathbb{E}M_0}{a}.$$

Proof. Let τ be the first k such that $M_k \geq a$, taking $\tau = n$ if this never occurs. By bounded optional stopping,

$$\mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

On the event

$$\max_{k \leq n} M_k \geq a$$

we have $M_\tau \geq a$, while $M_\tau \geq 0$ always. Hence

$$a \Pr \left[\max_{k \leq n} M_k \geq a \right] \leq \mathbb{E}M_\tau \leq \mathbb{E}M_0.$$

□

5.8.1 Azuma and McDiarmid

Azuma's inequality is Hoeffding's inequality with independence replaced by the martingale property.

Theorem 100 (Maximal Azuma–Hoeffding). *Let*

$$Y_k = \sum_{i=1}^k X_i, \quad Y_0 = 0,$$

where

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$a_i \leq X_i \leq b_i.$$

Then

$$\Pr \left[\max_{0 \leq k \leq n} Y_k \geq t \right] \leq \exp \left(- \frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2} \right).$$

The same bound holds for

$$\Pr \left[\min_{0 \leq k \leq n} Y_k \leq -t \right].$$

Consequently,

$$\Pr \left[\max_{0 \leq k \leq n} |Y_k| \geq t \right] \leq 2 \exp \left(- \frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2} \right).$$

Proof. Set

$$c_i = b_i - a_i.$$

Conditionally on $\mathcal{F}_{i-1}$, Hoeffding's lemma gives

$$\mathbb{E}[e^{\theta X_i} \mid \mathcal{F}_{i-1}] \leq e^{\theta^2 c_i^2 / 8}.$$

Therefore

$$M_k = \exp \left(\theta Y_k - \frac{\theta^2}{8} \sum_{i=1}^k c_i^2 \right)$$

is a nonnegative supermartingale with $M_0 = 1$.

If $Y_k \geq t$ for some $k \leq n$, then

$$M_k \geq \exp \left(\theta t - \frac{\theta^2}{8} \sum_{i=1}^n c_i^2 \right).$$

Thus Ville's inequality gives

$$\Pr \left[\max_{k \leq n} Y_k \geq t \right] \leq \exp \left(-\theta t + \frac{\theta^2}{8} \sum_{i=1}^n c_i^2 \right).$$

Optimizing at

$$\theta = \frac{4t}{\sum_i c_i^2}$$

gives the claimed bound.

Apply the same argument to $-Y_k$ for the lower tail, and union bound the two tails for the final statement. $\square$

Theorem 101 (McDiarmid's Inequality). *Let $Z_1, \dots, Z_n$ be independent, and suppose*

$$|f(z) - f(z')| \leq c_i$$

whenever z, z' differ only in coordinate i . Then

$$\Pr[f(Z) - \mathbb{E}f(Z) \geq t] \leq \exp \left(-\frac{2t^2}{\sum_i c_i^2} \right).$$

Proof. Consider the Doob martingale

$$M_i = \mathbb{E}[f(Z) \mid Z_1, \dots, Z_i].$$

Fixing $Z_1, \dots, Z_{i-1}$, regard M_i as a function of Z_i . For any two possible values z_i, z'_i ,

$$|\mathbb{E}[f(Z) \mid Z_i = z_i, Z_1, \dots, Z_{i-1}] - \mathbb{E}[f(Z) \mid Z_i = z'_i, Z_1, \dots, Z_{i-1}]| \leq c_i,$$

because the two expectations can be coupled using the same values of $Z_{i+1}, \dots, Z_n$.

Thus, conditioned on $\mathcal{F}_{i-1}$, M_i ranges over an interval of length at most c_i . Since

$$M_{i-1} = \mathbb{E}[M_i \mid \mathcal{F}_{i-1}],$$

the martingale difference $M_i - M_{i-1}$ also lies in an interval of length at most c_i .

Now

$$M_n - M_0 = f(Z) - \mathbb{E}f(Z),$$

so [Theorem 100](#) gives the result. $\square$

5.8.2 Freedman's Inequality

Azuma only uses a worst-case bound on each martingale increment. When the increments usually have much smaller conditional variance, Freedman's inequality gives the martingale analogue of Bernstein's inequality.

Let

$$Y_k = \sum_{i=1}^k X_i, \quad V_k = \sum_{i=1}^k \mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}],$$

where

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0.$$

The quantity V_k is the accumulated conditional variance.

We first record the conditional version of the Bernstein MGF bound.

Lemma 102 (Conditional Bernstein MGF Bound). *Suppose*

$$\mathbb{E}[X \mid \mathcal{F}] = 0, \quad X \leq R$$

almost surely, where $R > 0$. Then for

$$0 \leq \theta < 3/R,$$

$$\boxed{\mathbb{E}[e^{\theta X} \mid \mathcal{F}] \leq \exp\left(\frac{\theta^2 \mathbb{E}[X^2 \mid \mathcal{F}]}{2(1 - \theta R/3)}\right)}.$$

Proof. For $\theta \geq 0$ and $x \leq R$,

$$e^{\theta x} \leq 1 + \theta x + \frac{e^{\theta R} - \theta R - 1}{R^2} x^2.$$

Taking conditional expectation and using

$$\mathbb{E}[X \mid \mathcal{F}] = 0$$

gives

$$\mathbb{E}[e^{\theta X} \mid \mathcal{F}] \leq 1 + \frac{e^{\theta R} - \theta R - 1}{R^2} \mathbb{E}[X^2 \mid \mathcal{F}].$$

Since

$$1 + x \leq e^x$$

and, for $\theta R < 3$,

$$e^{\theta R} - \theta R - 1 \leq \frac{(\theta R)^2}{2(1 - \theta R/3)},$$

the result follows. □

Theorem 103 (Freedman's Inequality). *Suppose*

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$X_i \leq R$$

almost surely. Then

$$\boxed{\Pr[\exists k \leq n : Y_k \geq t \text{ and } V_k \leq \sigma^2] \leq \exp\left(-\frac{t^2}{2(\sigma^2 + Rt/3)}\right)}.$$

Proof. Fix

$$0 \leq \theta < 3/R$$

and write

$$g(\theta) = \frac{\theta^2}{2(1 - \theta R/3)}.$$

By [Lemma 102](#),

$$\mathbb{E}[e^{\theta X_i} \mid \mathcal{F}_{i-1}] \leq \exp(g(\theta)\mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}]).$$

Hence

$$M_k = \exp(\theta Y_k - g(\theta)V_k)$$

is a nonnegative supermartingale with $M_0 = 1$.

If for some $k \leq n$,

$$Y_k \geq t, \quad V_k \leq \sigma^2,$$

then

$$M_k \geq \exp(\theta t - g(\theta)\sigma^2).$$

Ville's inequality therefore gives

$$\Pr[\exists k \leq n : Y_k \geq t, V_k \leq \sigma^2] \leq \exp(-\theta t + g(\theta)\sigma^2).$$

Take

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

When $\sigma^2 > 0$, this satisfies $\theta R < 3$, and

$$1 - \frac{\theta R}{3} = \frac{\sigma^2}{\sigma^2 + Rt/3}.$$

Therefore

$$\begin{aligned} -\theta t + g(\theta)\sigma^2 &= -\frac{t^2}{\sigma^2 + Rt/3} + \frac{t^2}{2(\sigma^2 + Rt/3)} \\ &= -\frac{t^2}{2(\sigma^2 + Rt/3)}. \end{aligned}$$

The case $\sigma^2 = 0$ follows by taking a limit. □

Thus Freedman has the same basic form as Bernstein:

$$\exp\left(-\Theta\left(\min\left\{\frac{t^2}{\sigma^2}, \frac{t}{R}\right\}\right)\right),$$

but it simultaneously controls every time k at which the accumulated conditional variance is at most σ^2 .

If also

$$X_i \geq -R,$$

then applying the theorem to $-Y_k$ gives the corresponding lower-tail bound.

5.9 Enumerative and Algebraic Combinatorics

5.9.1 Lindström–Gessel–Viennot Lemma

The LGV lemma is fundamentally a sign-reversing involution. The usual edge-weighted version is a special case of the following slightly more general statement.

Let G be a finite directed acyclic graph and let R be a commutative ring. Assign to every directed path P a weight

$$w(P) \in R.$$

We only require the following *tail-switching property*. Suppose two paths P, Q meet at a vertex v , and write

$$P = P^- P^+, \quad Q = Q^- Q^+$$

at v . Let

$$P' = P^- Q^+, \quad Q' = Q^- P^+.$$

Then require

$$\boxed{w(P)w(Q) = w(P')w(Q')}.$$

Ordinary multiplicative edge weights

$$w(P) = \prod_{e \in P} w(e)$$

automatically satisfy this condition.

Let

$$a_1, \dots, a_k, \quad b_1, \dots, b_k$$

be ordered vertices. Define the path matrix

$$M_{ij} = \sum_{P: a_i \rightarrow b_j} w(P).$$

For a permutation $\sigma \in S_k$, let $\mathcal{N}_\sigma$ be the set of k -tuples

$$(P_1, \dots, P_k)$$

of pairwise vertex-disjoint directed paths satisfying

$$P_i : a_i \rightarrow b_{\sigma(i)}.$$

Define

$$w(P_1, \dots, P_k) = \prod_i w(P_i).$$

Theorem 104 (Lindström–Gessel–Viennot). *With the notation above,*

$$\boxed{\det M = \sum_{\sigma \in S_k} \operatorname{sgn}(\sigma) \sum_{\mathbf{P} \in \mathcal{N}_\sigma} w(\mathbf{P}).}$$

Proof. Expand the determinant:

$$\det M = \sum_{\sigma \in S_k} \operatorname{sgn}(\sigma) \prod_{i=1}^k \left(\sum_{P_i: a_i \rightarrow b_{\sigma(i)}} w(P_i) \right).$$

Thus $\det M$ is the signed weighted sum over *all* path families

$$(P_1, \dots, P_k),$$

including intersecting ones.

We cancel the intersecting families by an involution.

Fix a topological ordering of the vertices of G . For an intersecting family, choose the earliest vertex v in this order that lies on at least two paths. Among the paths containing v , choose the first pair

$$P_i, P_j$$

according to some fixed ordering of the indices.

Swap their tails after v :

$$P_i = P_i^- P_i^+, \quad P_j = P_j^- P_j^+$$

becomes

$$P'_i = P_i^- P_j^+, \quad P'_j = P_j^- P_i^+.$$

Because G is acyclic, the switched objects are still directed paths. The endpoints $b_{\sigma(i)}$ and $b_{\sigma(j)}$ are exchanged, so the new permutation is

$$\sigma \circ (ij),$$

which has the opposite sign.

By the tail-switching assumption,

$$w(P_i)w(P_j) = w(P'_i)w(P'_j),$$

so the total weight is unchanged.

The chosen first intersection and pair are unchanged by performing the swap again, so this operation is an involution.

Hence every intersecting family cancels with another family of equal weight and opposite sign. The only uncanceled terms are the pairwise vertex-disjoint families, giving the stated formula. $\square$

A particularly common special case occurs when the sources and sinks are *compatible*: no vertex-disjoint path family can connect

$$a_i \rightarrow b_{\sigma(i)}$$

for any nonidentity permutation σ . Then

$$\det M = \sum_{\substack{P_i: a_i \rightarrow b_i \\ P_1, \dots, P_k \text{ pairwise disjoint}}} \prod_i w(P_i).$$

Thus a determinant directly enumerates nonintersecting path families.

5.9.2 Q-Analogs

Lemma 105 (q -Binomial Theorem). *For any positive integer n and indeterminates x and q ,*

$$\prod_{i=0}^{n-1} (1 + xq^i) = \sum_{k=0}^n q^{\binom{k}{2}} \binom{n}{k}_q x^k$$

Proof. We evaluate the coefficient of x^k on the left-hand side of the equation.

To form a term with exactly x^k in the expansion of $\prod_{i=0}^{n-1} (1 + xq^i)$, we must choose the " xq^i " term from exactly k of the n binomial factors, and the "1" from the remaining $n - k$ factors. Let the indices of the k chosen factors be $i_1, i_2, \dots, i_k$, ordered such that:

$$0 \leq i_1 < i_2 < \dots < i_k \leq n - 1$$

The product of these chosen terms is $x^k q^{i_1+i_2+\dots+i_k}$. Therefore, the total coefficient of x^k is:

$$[x^k] \prod_{i=0}^{n-1} (1 + xq^i) = \sum_{0 \leq i_1 < i_2 < \dots < i_k \leq n-1} q^{i_1+i_2+\dots+i_k}$$

We shift the sequence to remove this strict inequality by defining a new sequence $p_m = i_m - (m-1)$ for $m = 1, \dots, k$.

Our bounds are now:

$$0 \leq p_1 \leq p_2 \leq \dots \leq p_k \leq n-1 - (k-1) \implies 0 \leq p_1 \leq p_2 \leq \dots \leq p_k \leq n-k$$

This is exactly a partition that fits inside a $k \times (n-k)$ rectangle.

Let us rewrite the sum of the original indices i_m in terms of the partition parts p_m :

$$\sum_{m=1}^k i_m = \sum_{m=1}^k (p_m + (m-1)) = \sum_{m=1}^k p_m + \sum_{m=1}^k (m-1)$$

The first sum is the total size of the partition, denoted $|\lambda|$. The second sum is $\binom{k}{2}$.

Substituting this back into our coefficient sum:

$$[x^k] \prod_{i=0}^{n-1} (1 + xq^i) = \sum_{\lambda \subseteq k \times (n-k)} q^{|\lambda| + \binom{k}{2}} = q^{\binom{k}{2}} \sum_{\lambda \subseteq k \times (n-k)} q^{|\lambda|}$$

Thus, the sum simplifies to:

$$[x^k] \prod_{i=0}^{n-1} (1 + xq^i) = q^{\binom{k}{2}} \binom{n}{k}_q$$

□

Lemma 106 (Durfee Square Identity). *For indeterminates x and q ,*

$$\prod_{i=1}^{\infty} \frac{1}{1 - xq^i} = \sum_{k=0}^{\infty} \frac{x^k q^{k^2}}{(1-q)(1-q^2) \dots (1-q^k) \cdot (1-xq)(1-xq^2) \dots (1-xq^k)}$$

Proof. Consider the left-hand side. Letting the number of parts be $\ell(\lambda)$:

$$\prod_{i=1}^{\infty} \frac{1}{1 - xq^i} = \sum_{\lambda} x^{\ell(\lambda)} q^{|\lambda|}$$

Consider partitions with a Durfee square of size k , removing it splits the partition into three parts:

1. A $k \times k$ block of cells, its weight is $x^k q^{k^2}$.
2. A partition to the right of the Durfee square. It can have at most k rows ($\ell(\mu) \leq k$) and does not create any new parts. Thus the contribution is:

$$\frac{1}{(1-q)(1-q^2) \dots (1-q^k)}$$

3. A partition residing below the Durfee square. Its largest part can be at most k and every row adds a new part to λ . The contribution is thus:

$$\frac{1}{(1-xq)(1-xq^2) \dots (1-xq^k)}$$

We have that $|\lambda| = k^2 + |\mu| + |\nu|$ and $\ell(\lambda) = k + \ell(\nu)$, thus we have the result. □

5.9.3 Jacobi Triple Product

Lemma 107.

$$[y^0] \prod_{j=1}^{\infty} (1 + x^{2j-1}y^2)(1 + x^{2j-1}y^{-2}) = \sum_{m=0}^{\infty} \frac{x^{2m^2}}{\prod_{i=1}^m (1 - x^{2i})^2}$$

Proof. We must select m factors of y^2 and m factors of y^{-2} for some $m \geq 0$. Since these selections are independent, the constant term is $\sum_{m=0}^{\infty} G_m(x)^2$, where $G_m(x)$ is the generating function for the sum of m distinct positive odd integers.

Any sequence of m strictly increasing odd integers can be decomposed into $(1, 3, 5, \dots, 2m-1)$, which sums to m^2 , plus an increasing sequence of m non-negative even integers $(2p_1, 2p_2, \dots, 2p_m)$.

This sequence corresponds to a partition into at most m even parts, generated by $\prod_{i=1}^m (1 - x^{2i})^{-1}$. Thus:

$$G_m(x) = x^{m^2} \prod_{i=1}^m \frac{1}{1 - x^{2i}}$$

□

Theorem 108 (Jacobi Triple Product). *For complex numbers x and y with $|x| < 1$ and $y \neq 0$:*

$$\prod_{m=1}^{\infty} (1 - x^{2m})(1 + x^{2m-1}y^2)(1 + x^{2m-1}y^{-2}) = \sum_{n=-\infty}^{\infty} x^{n^2} y^{2n}$$

Proof. Let $f_x(y)$ be defined as the infinite product on the left-hand side:

$$f_x(y) = \prod_{m=1}^{\infty} (1 - x^{2m})(1 + x^{2m-1}y^2)(1 + x^{2m-1}y^{-2})$$

Substituting xy for y we have:

$$f_x(xy) = \prod_{m=1}^{\infty} (1 - x^{2m})(1 + x^{2m+1}y^2)(1 + x^{2m-3}y^{-2})$$

Comparing this to the original $f_x(y)$, the product containing y^2 has lost its $m = 1$ term, $(1 + xy^2)$, while the product containing y^{-2} has gained an $m = 0$ term, $(1 + x^{-1}y^{-2})$. Thus:

$$\begin{aligned} f_x(xy) &= \frac{1 + x^{-1}y^{-2}}{1 + xy^2} f_x(y) \\ &= \frac{x^{-1}y^{-2}(xy^2 + 1)}{1 + xy^2} f_x(y) \\ &= x^{-1}y^{-2} f_x(y) \end{aligned}$$

Because $f_x(y)$ is meromorphic for $|y| > 0$, it admits a Laurent series expansion in terms of y :

$$f_x(y) = \sum_{n=-\infty}^{\infty} c_n(x) y^{2n}$$

Substituting this series into our functional equation $xf_x(xy) = y^{-2}f_x(y)$, we obtain:

$$\sum_{n=-\infty}^{\infty} c_n(x) x^{2n+1} y^{2n} = y^{-2} \sum_{n=-\infty}^{\infty} c_n(x) y^{2n}$$

By shifting the index $n \rightarrow n + 1$ on the right-hand side, we have:

$$\sum_{n=-\infty}^{\infty} c_n(x) x^{2n+1} y^{2n} = \sum_{n=-\infty}^{\infty} c_{n+1}(x) y^{2n}$$

Equating the coefficients of y^{2n} yields the recurrence relation:

$$c_{n+1}(x) = c_n(x) x^{2n+1}$$

This gives:

$$c_n(x) = c_0(x) x^{1+3+5+\dots+(2n-1)} = c_0(x) x^{n^2}$$

Thus:

$$f_x(y) = c_0(x) \sum_{n=-\infty}^{\infty} x^{n^2} y^{2n}$$

We must show that $c_0(x) = 1$. This is the y^0 coefficient of the original infinite product. From [Lemma 107](#) and [Lemma 106](#) this is:

$$c_0(x) = \left(\prod_{m=1}^{\infty} (1 - x^{2m}) \right) \times \left(\prod_{m=1}^{\infty} \frac{1}{1 - x^{2m}} \right) = 1$$

□

5.9.4 MacMahon's Master Theorem

Theorem 109. *Let $A = (a_{ij})$ be an $n \times n$ matrix, and let $X = \text{diag}(x_1, \dots, x_n)$ be a diagonal matrix of formal variables. For any tuple $k = (k_1, \dots, k_n) \in \mathbb{N}^n$, let $x^k = x_1^{k_1} \dots x_n^{k_n}$. Then*

$$[x^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j \right)^{k_i} = [x^k] \frac{1}{\det(I - AX)}.$$

Proof. Let $R = \mathbb{C}[a_{11}, \dots, a_{nn}]$ be the polynomial ring representing the generic matrix A . We work in the ring of formal power series $R[[x_1, \dots, x_n]]$.

Let V be a free module of rank n over R with basis $v_1, \dots, v_n$. Let $M = AX$, meaning $M_{ij} = a_{ij} x_j$. We define a linear operator $T : V \rightarrow V$ by its action on the standard basis:

$$T(v_i) = \sum_{j=1}^n M_{ij} v_j.$$

The operator T extends naturally to the symmetric algebra $\text{Sym}(V) \cong R[v_1, \dots, v_n]$. The subspace $S^m(V)$ of homogeneous polynomials of degree m has a basis of monomials $v^k = v_1^{k_1} \dots v_n^{k_n}$ where $|k| = \sum_{i=1}^n k_i = m$. The action of T on this basis is:

$$T(v^k) = \prod_{i=1}^n T(v_i)^{k_i} = \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j v_j \right)^{k_i}.$$

The trace of T acting on $S^m(V)$ is the sum of its diagonal matrix entries in this standard basis:

$$\text{Tr}(S^m(T)) = \sum_{|k|=m} [v^k] T(v^k) = \sum_{|k|=m} [v^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j v_j \right)^{k_i}.$$

In the polynomial expansion of $\prod_{i=1}^n (\sum_{j=1}^n a_{ij} x_j v_j)^{k_i}$, the variables x_j and v_j are strictly coupled; every term chosen from the inner sum contributes exactly one x_j and one v_j . Therefore, the formal coefficient of v^k is exactly x^k multiplied by the scalar coefficient we would obtain if we evaluated at $v_j = 1$. Thus:

$$[v^k]T(v^k) = x^k \cdot [x^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j \right)^{k_i}.$$

Summing this trace over all degrees m yields a formal power series:

$$\sum_{m=0}^{\infty} \text{Tr}(S^m(T)) = \sum_{k \in \mathbb{N}^n} x^k \left([x^k] \prod_{i=1}^n \left(\sum_{j=1}^n a_{ij} x_j \right)^{k_i} \right).$$

We must now prove that this sum evaluates to $\det(I - AX)^{-1}$. Let W be an $n \times n$ matrix of independent formal variables w_{ij} . We will establish the generic trace identity $\sum_{m=0}^{\infty} \text{Tr}(S^m(W)) = \det(I - W)^{-1}$ in $\mathbb{C}[[w_{11}, \dots, w_{nn}]]$.

By comparing the homogeneous components of degree m , we define a sequence of polynomials:

$$F_m(W) = \text{Tr}(S^m(W)) - [\text{degree } m \text{ of } \det(I - W)^{-1}].$$

We must prove $F_m(W)$ is identically zero in $\mathbb{C}[[w_{11}, \dots, w_{nn}]]$.

Let $\Delta(W)$ be the discriminant of the characteristic polynomial of W with respect to λ . Because there exist matrices with distinct eigenvalues, Δ is a non-zero polynomial.

Let $C \in \mathbb{C}^{n \times n}$ be any specific complex matrix such that $\Delta(C) \neq 0$. This non-vanishing discriminant guarantees that C has n distinct roots, meaning C is diagonalizable. Let its eigenvalues be $\lambda_1, \dots, \lambda_n$, with a basis of eigenvectors $u_1, \dots, u_n$.

Because C acts diagonally on V , the induced basis of monomials $u^k = u_1^{k_1} \dots u_n^{k_n}$ is an eigenbasis for $S^m(C)$, with corresponding eigenvalues $\lambda_1^{k_1} \dots \lambda_n^{k_n}$. Computing the trace in this eigenbasis yields:

$$\sum_{m=0}^{\infty} \text{Tr}(S^m(C)) = \sum_{m=0}^{\infty} \sum_{|k|=m} \lambda_1^{k_1} \dots \lambda_n^{k_n} = \prod_{i=1}^n \left(\sum_{k_i=0}^{\infty} \lambda_i^{k_i} \right).$$

By formally evaluating the independent geometric series, we obtain:

$$\prod_{i=1}^n \frac{1}{1 - \lambda_i} = \frac{1}{\det(I - C)}.$$

Thus, $F_m(C) = 0$ for all strictly diagonalizable matrices C . We have established that the polynomial F_m evaluates to zero wherever the non-zero polynomial Δ is non-zero, thus $F_m \Delta = 0$, so $F_m = 0$ as $\Delta \neq 0$ and we're over $\mathbb{C}$.

By substituting the variables $w_{ij} = a_{ij} x_j$ (which corresponds to $W = AX$) back into the identity for W , we obtain:

$$\sum_{m=0}^{\infty} \text{Tr}(S^m(T)) = \frac{1}{\det(I - AX)}.$$

□

5.9.5 Hook-Length Formula

Let

$$\lambda = (\lambda_1 \geq \lambda_2 \geq \dots) \vdash n$$

be a Young diagram. For a cell $u = (i, j)$, its hook consists of u , the cells to its right, and the cells below it; let $h(u)$ denote its size.

Let f^λ be the number of standard Young tableaux of shape λ .

Theorem 110 (Hook-Length Formula).

$$f^\lambda = \frac{n!}{\prod_{u \in \lambda} h(u)}.$$

Why should hooks appear? At each cell, the row and column conditions combine into one requirement: its entry must be the smallest in its hook. A filling is standard exactly when this holds at every cell.

Among the $n!$ uniformly random fillings, a fixed cell u is smallest in its hook with probability $1/h(u)$. Multiplying these local probabilities suggests

$$f^\lambda = n! \prod_{u \in \lambda} \frac{1}{h(u)}.$$

For a single row, the hook lengths are $n, n-1, \dots, 1$, and this simply recovers the familiar $1/n!$ probability of being sorted. In a general diagram, the hooks overlap and their minimum-entry conditions are *not independent*. The remarkable content of the formula is that the probability of satisfying all of them nevertheless equals the product of their individual probabilities.

Let r be the number of rows of the original diagram. Throughout the branching argument, we regard every smaller diagram as also having r rows by appending zero rows if necessary. This is only bookkeeping: it lets us use the same coordinates even when the last nonempty row disappears.

Define the shifted row lengths

$$\ell_i = \lambda_i + r - i.$$

Because $\lambda_i \geq \lambda_{i+1}$,

$$\ell_i - \ell_{i+1} = \lambda_i - \lambda_{i+1} + 1 > 0,$$

so

$$\ell_1 > \ell_2 > \dots > \ell_r.$$

The point of this shift is that it turns the hook lengths in each row into almost all the integers from 1 up to ℓ_i .

Lemma 111 (Hook Product Identity).

$$\prod_{u \in \lambda} h(u) = \frac{\prod_{i=1}^r \ell_i!}{\prod_{1 \leq i < j \leq r} (\ell_i - \ell_j)}.$$

Proof. Fix a row i . Read its cells from right to left.

If there were no lower rows, the hook lengths would simply be

$$1, 2, \dots, \lambda_i.$$

Each lower row changes this sequence once. When we pass the last column occupied by row $j > i$, the next cell gains not only one new cell to its right but also the cell in row j below it. Thus the hook length jumps by 2 instead of 1, skipping one integer.

There are $r - i$ lower rows, so the λ_i hook lengths in row i are obtained from the $\ell_i = \lambda_i + r - i$ integers

$$1, 2, \dots, \ell_i$$

by deleting exactly $r - i$ values.

The value skipped when crossing the end of row j is

$$\lambda_i - \lambda_j + j - i = \ell_i - \ell_j.$$

Hence

$$\{h(i, 1), \dots, h(i, \lambda_i)\} = \{1, \dots, \ell_i\} \setminus \{\ell_i - \ell_j : j > i\},$$

and therefore

$$\prod_{q=1}^{\lambda_i} h(i, q) = \frac{\ell_i!}{\prod_{j>i} (\ell_i - \ell_j)}.$$

Multiplying over i proves the identity. □

Define

$$F(\lambda) = n! \frac{\prod_{i<j} (\ell_i - \ell_j)}{\prod_i \ell_i!}.$$

By the preceding lemma,

$$F(\lambda) = \frac{n!}{\prod_{u \in \lambda} h(u)}.$$

The remainder of the proof will show that F satisfies the same recurrence as f^λ . The benefit of the padded coordinates now becomes apparent: removing a corner from row i just changes some value,

$$\ell_i \mapsto \ell_i - 1,$$

even when that removal makes an entire row disappear.

The number of standard tableaux satisfies the branching recurrence

$$f^\lambda = \sum_{\mu \triangleleft \lambda} f^\mu,$$

because the largest entry n must lie in a removable corner. We will show that F satisfies the same recurrence.

Lemma 112. *For distinct numbers $\ell_1, \dots, \ell_r$,*

$$\boxed{\sum_{i=1}^r \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = \sum_{i=1}^r \ell_i - \binom{r}{2}}.$$

Proof. Let

$$P(x) = \prod_{j=1}^r (x - \ell_j)$$

and define

$$Q(x) = x(P(x-1) - P(x)) + rP(x).$$

Since both $P(x-1)$ and $P(x)$ have leading term x^r , their difference has degree at most $r-1$. The x^r -term in $x(P(x-1) - P(x))$ is $-rx^r$, which is cancelled by $rP(x)$, so

$$\deg Q \leq r-1.$$

At $x = \ell_i$, we have $P(\ell_i) = 0$, and hence

$$Q(\ell_i) = \ell_i P(\ell_i - 1) = -\ell_i \prod_{j \neq i} (\ell_i - \ell_j - 1).$$

Since Q has degree at most $r-1$, Lagrange interpolation says that its leading coefficient is

$$[x^{r-1}]Q = \sum_{i=1}^r \frac{Q(\ell_i)}{P'(\ell_i)}.$$

But

$$P'(\ell_i) = \prod_{j \neq i} (\ell_i - \ell_j),$$

so

$$[x^{r-1}]Q = - \sum_{i=1}^r \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j}.$$

It remains to compute the same coefficient directly. Write

$$s_1 = \sum_i \ell_i.$$

Then

$$P(x) = x^r - s_1 x^{r-1} + \dots.$$

Expanding $P(x-1)$ to the first two terms,

$$P(x-1) = x^r - (r + s_1)x^{r-1} + \left(\binom{r}{2} + (r-1)s_1 + \dots \right) x^{r-2} + \dots.$$

Hence

$$[x^{r-1}](x(P(x-1) - P(x))) = \binom{r}{2} + (r-1)s_1,$$

while

$$[x^{r-1}]rP(x) = -rs_1.$$

Therefore

$$[x^{r-1}]Q = \binom{r}{2} - s_1.$$

Comparing the two expressions for $[x^{r-1}]Q$ gives

$$\sum_{i=1}^r \ell_i \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = s_1 - \binom{r}{2},$$

as claimed. □

Now suppose a removable corner is deleted from row i . Then n becomes $n - 1$, and among the shifted row lengths only

$$\ell_i \mapsto \ell_i - 1$$

changes. Therefore

$$\frac{F(\lambda - \square_i)}{F(\lambda)} = \frac{\ell_i}{n} \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j}.$$

This formula also automatically detects rows from which no corner may be removed. For $i < r$, row i is not removable exactly when $\lambda_i = \lambda_{i+1}$. In that case

$$\ell_i - \ell_{i+1} = \lambda_i - \lambda_{i+1} + 1 = 1,$$

so the factor corresponding to $j = i + 1$ is

$$\frac{\ell_i - \ell_{i+1} - 1}{\ell_i - \ell_{i+1}} = 0.$$

For the last padded row, if $\lambda_r = 0$, then $\ell_r = 0$, so the prefactor ℓ_r vanishes. Thus we may sum the ratio over all $i = 1, \dots, r$; non-removable rows simply contribute 0.

Finally,

$$n = \sum_{i=1}^r \lambda_i = \sum_{i=1}^r (\ell_i - r + i) = \sum_{i=1}^r \ell_i - \binom{r}{2}.$$

Hence [Lemma 112](#) gives

$$\sum_{\mu \triangleleft \lambda} F(\mu) = F(\lambda) \sum_{i=1}^r \frac{\ell_i}{n} \prod_{j \neq i} \frac{\ell_i - \ell_j - 1}{\ell_i - \ell_j} = F(\lambda), \frac{1}{n} \left(\sum_i \ell_i - \binom{r}{2} \right) = F(\lambda).$$

Thus F satisfies exactly the same branching recurrence as f^λ .

For the empty diagram, $f^\emptyset = 1$. Also

$$F(\emptyset) = \frac{\prod_{i < j} (j - i)}{\prod_{i=1}^r (r - i)!} = 1,$$

since

$$\prod_{i < j} (j - i) = \prod_{j=1}^r (j - 1)!.$$

Therefore, by induction on the number of cells,

$$f^\lambda = F(\lambda).$$

Using [Lemma 111](#),

$$f^\lambda = \frac{n!}{\prod_{u \in \lambda} h(u)}.$$

This proves the hook-length formula.

5.9.6 Robinson–Schensted Correspondence

The Robinson–Schensted correspondence gives a bijection

$$S_n \longleftrightarrow \bigsqcup_{\lambda \vdash n} \text{SYT}(\lambda) \times \text{SYT}(\lambda).$$

Its basic operation is row insertion.

Row Insertion To insert x into a standard tableau P :

1. In the first row, find the leftmost entry $y > x$.
2. Replace y by x and insert y into the next row.
3. If there is no entry larger than x , append x to the row and stop.

Lemma 113. *Row insertion preserves the tableau property and adds exactly one corner cell.*

Proof. Replacing the first entry larger than x preserves increasing order within the row.

Suppose an entry y is bumped from column c into the next row. Because the tableau was column-increasing, the entry directly below y , if it exists, is larger than y . Thus the first entry larger than y in the next row occurs in some column $c' \leq c$.

Hence the bumping path moves weakly to the left as it descends. The entry directly above the newly inserted y is therefore smaller than y , so column order is also preserved.

Eventually an entry is appended at the end of some row. The bumping-path property ensures that this new cell is an outer corner. $\square$

The Correspondence Given

$$\pi = \pi_1 \pi_2 \cdots \pi_n \in S_n,$$

start with empty tableaux P, Q .

For $t = 1, \dots, n$:

- row-insert π_t into P ;
- exactly one new cell is created;
- put t into that cell of Q .

The insertion tableau P and recording tableau Q always have the same shape.

Since later cells receive larger labels and new cells are always outer corners, Q is also standard.

Reverse Insertion Suppose a corner cell of P contains x . Delete that cell. Moving upward one row at a time, replace the rightmost entry $y < x$ by x and set

$$x \leftarrow y.$$

When the first row has been processed, output the final value of x .

Lemma 114. *Reverse insertion is the inverse of row insertion.*

Proof. Suppose forward insertion placed x into a row and bumped y . By definition, y was the leftmost entry larger than x . Equivalently, after the replacement, x is the rightmost entry smaller than y . Thus the reverse rule exactly undoes this step. Apply this successively up the bumping path. $\square$

Theorem 115 (Robinson–Schensted). *The map*

$$\pi \longmapsto (P, Q)$$

is a bijection between permutations of $[n]$ and pairs of standard Young tableaux of the same shape.

Proof. Given (P, Q) , find the cell of Q containing n . It is a corner. Remove it and reverse-insert from the corresponding corner of P . The output is π_n .

Repeat with

$$n-1, n-2, \dots, 1.$$

By the preceding lemma this exactly reverses the forward algorithm. □

One immediate consequence is

$$n! = \sum_{\lambda \vdash n} (f^\lambda)^2.$$

Longest Increasing Subsequences

Theorem 116 (Schensted). *If the insertion tableau of π has shape λ , then*

$$\lambda_1 = \text{LIS}(\pi), \quad \lambda'_1 = \text{LDS}(\pi).$$

Proof. Consider the first row after processing a prefix of the permutation:

$$r_1 < r_2 < \dots < r_t.$$

We claim that

$$r_j$$

is the smallest possible final value of an increasing subsequence of length j in the processed prefix.

This is true initially. When a new value x is inserted, it replaces the first $r_j > x$, or is appended if none exists. This is exactly the standard update of the minimum possible tail of an increasing subsequence of each length.

Thus the first row has an entry in position j exactly when an increasing subsequence of length j exists. Therefore

$$\lambda_1 = \text{LIS}(\pi).$$

For decreasing subsequences, reverse the order on the alphabet. Row insertion in the reversed order is column insertion in the original order, so it transposes the shape. An increasing subsequence in the reversed order is exactly a decreasing subsequence in the original order. Hence

$$\lambda'_1 = \text{LDS}(\pi).$$

□

6 Convexity and Duality

6.1 Sion's Minimax Theorem

A function g on a convex set is *quasiconvex* if

$$g((1-t)x + ty) \leq \max\{g(x), g(y)\} \quad (t \in [0, 1]),$$

equivalently, if every sublevel set $\{x : g(x) \leq a\}$ is convex.

It is *quasiconcave* if

$$g((1-t)x + ty) \geq \min\{g(x), g(y)\},$$

equivalently, if every superlevel set $\{x : g(x) \geq a\}$ is convex.

Sion's theorem says that quasiconvexity in the minimizing variable and quasiconcavity in the maximizing variable are enough for minimax equality.

The proof is easiest to understand at a fixed threshold a . For a choice y , call x *good for y* if $f(x, y) \leq a$. Quasiconvexity in x says that the good points for each fixed y form a convex set.

The key fact is that if two choices y_0, y_1 have no common good point, then some convex combination of them is bad for *every* x . Quasiconcavity in y is exactly what makes this possible.

Lemma 117 (Two-Point Reduction). *Let X be compact and convex, let $y_0, y_1 \in Y$, and set*

$$y_t = (1-t)y_0 + ty_1.$$

Suppose that $f(\cdot, y)$ is quasiconvex on X for every y , that $f(x, \cdot)$ is quasiconcave on Y for every x , and that f is continuous.

Set

$$M = \min_{x \in X} \max\{f(x, y_0), f(x, y_1)\}.$$

Then for some $t \in [0, 1]$,

$$\boxed{\min_{x \in X} f(x, y_t) \geq M.}$$

Proof. Define

$$U_i = \{x \in X : f(x, y_i) < M\}, \quad i \in \{0, 1\},$$

and

$$D_t = \{x \in X : f(x, y_t) < M\}.$$

By the definition of M ,

$$U_0 \cap U_1 = \emptyset.$$

Indeed, if some x belonged to both, then

$$\max\{f(x, y_0), f(x, y_1)\} < M,$$

contradicting the definition of M .

Now quasiconcavity in y gives

$$f(x, y_t) \geq \min\{f(x, y_0), f(x, y_1)\}.$$

Hence if $f(x, y_t) < M$, then at least one of $f(x, y_0), f(x, y_1)$ is less than M . Therefore

$$D_t \subseteq U_0 \cup U_1.$$

On the other hand, D_t is convex, since it is a strict sublevel set of the quasiconvex function $f(\cdot, y_t)$. In particular, D_t is connected. Since U_0 and U_1 are disjoint open subsets of X , every nonempty D_t must therefore lie entirely in one of them:

$$D_t \subseteq U_0 \quad \text{or} \quad D_t \subseteq U_1.$$

If some D_t is empty, then $f(x, y_t) \geq M$ for every x , and we are done. Suppose therefore that D_t is nonempty for every $t \in [0, 1]$.

Define

$$I = \{t : D_t \subseteq U_0\}, \quad J = \{t : D_t \subseteq U_1\}.$$

Then I and J are disjoint and cover $[0, 1]$. Moreover, $D_0 = U_0$ and $D_1 = U_1$, so $0 \in I$ and $1 \in J$.

We claim that I is closed. Let $t_k \in I$ with $t_k \rightarrow t$. Choose $x \in D_{t_k}$, which is possible by assumption. Since $f(x, y_{t_k}) < M$, continuity gives

$$f(x, y_{t_k}) < M$$

for all sufficiently large k . Hence

$$x \in D_{t_k} \subseteq U_0,$$

so D_t meets U_0 . But D_t lies entirely in either U_0 or U_1 , and therefore $D_t \subseteq U_0$. Thus $t \in I$.

The same argument shows that J is closed. This writes the connected interval $[0, 1]$ as a disjoint union of two nonempty closed sets, a contradiction.

Hence $D_t = \emptyset$ for some t , and consequently

$$\min_x f(x, y_t) \geq M.$$

□

The lemma says that two incompatible choices of y can be replaced by a single convex combination that is at least as bad for the minimizer as the better of the two choices was pointwise.

Lemma 118 (Finite Reduction). *Let $y_1, \dots, y_m \in Y$ and $a \in \mathbb{R}$. If*

$$\bigcap_{i=1}^m \{x \in X : f(x, y_i) \leq a\} = \emptyset,$$

then there is

$$\bar{y} \in \text{conv}\{y_1, \dots, y_m\}$$

such that

$$\boxed{\min_{x \in X} f(x, \bar{y}) > a.}$$

Proof. We induct on m .

For $m = 1$, the hypothesis says $f(x, y_1) > a$ for every $x \in X$. Compactness and continuity give

$$\min_x f(x, y_1) > a.$$

Now suppose $m > 1$, and let

$$X' = \{x \in X : f(x, y_m) \leq a\}.$$

This is compact and convex: it is a closed sublevel set of the continuous quasiconvex function $f(\cdot, y_m)$.

If $X' = \emptyset$, then $\bar{y} = y_m$ works.

Otherwise, within X' the first $m - 1$ sublevel sets have empty intersection. By the inductive hypothesis, there is

$$y' \in \text{conv}\{y_1, \dots, y_{m-1}\}$$

such that

$$\min_{x \in X'} f(x, y') > a.$$

Thus for every $x \in X$,

$$\max\{f(x, y'), f(x, y_m)\} > a.$$

Indeed, if $x \in X'$, then $f(x, y') > a$; while if $x \notin X'$, then $f(x, y_m) > a$.

Since X is compact,

$$M := \min_{x \in X} \max\{f(x, y'), f(x, y_m)\} > a.$$

Applying [Lemma 117](#) to y' and y_m , there is some point $\bar{y}$ on the segment joining them such that

$$\min_x f(x, \bar{y}) \geq M > a.$$

Finally,

$$\bar{y} \in \text{conv}\{y_1, \dots, y_m\}.$$

□

Theorem 119 (Sion's Minimax Theorem). *Let X be a nonempty compact convex set and let Y be a nonempty convex set, both contained in finite-dimensional real vector spaces. Let*

$$f : X \times Y \rightarrow \mathbb{R}$$

be continuous.

Suppose that for every $y \in Y$, the function $f(\cdot, y)$ is quasiconvex on X , and for every $x \in X$, the function $f(x, \cdot)$ is quasiconcave on Y . Then

$$\min_{x \in X} \sup_{y \in Y} f(x, y) = \sup_{y \in Y} \min_{x \in X} f(x, y).$$

If Y is also compact, the supremum is attained, and hence

$$\min_{x \in X} \max_{y \in Y} f(x, y) = \max_{y \in Y} \min_{x \in X} f(x, y).$$

Proof. Set

$$\alpha = \sup_{y \in Y} \min_{x \in X} f(x, y).$$

Weak minimax always gives

$$\alpha \leq \min_x \sup_y f(x, y).$$

We prove the reverse inequality.

For every $y \in Y$, define

$$C_y = \{x \in X : f(x, y) \leq \alpha\}.$$

Each C_y is closed and convex.

We claim that the family $\{C_y : y \in Y\}$ has the finite intersection property. Suppose instead that for some $y_1, \dots, y_m \in Y$,

$$C_{y_1} \cap \dots \cap C_{y_m} = \emptyset.$$

By [Lemma 118](#), there would be some

$$\bar{y} \in \text{conv}\{y_1, \dots, y_m\} \subseteq Y$$

such that

$$\min_x f(x, \bar{y}) > \alpha.$$

This contradicts the definition

$$\alpha = \sup_y \min_x f(x, y).$$

Thus every finite subfamily has nonempty intersection. Since X is compact,

$$\bigcap_{y \in Y} C_y \neq \emptyset.$$

Choose x^* in this intersection. Then

$$f(x^*, y) \leq \alpha \quad \text{for every } y \in Y,$$

so

$$\sup_y f(x^*, y) \leq \alpha.$$

Consequently,

$$\min_x \sup_y f(x, y) \leq \alpha.$$

Together with weak minimax,

$$\sup_y \min_x f(x, y) \leq \min_x \sup_y f(x, y) \leq \alpha = \sup_y \min_x f(x, y),$$

and therefore equality holds. □

6.2 Linear Programming Duality

Sion's theorem makes LP duality into a direct calculation.

Consider the standard-form primal LP

minimize $c^T x$ subject to $Ax = b,$ $x \geq 0.$

Its dual is

maximize $b^T y$ subject to $A^T y \leq c.$
--

The variable y is unrestricted.

Theorem 120 (LP Strong Duality). *If the primal LP is feasible and has a finite optimum, then the dual is feasible, attains its optimum, and*

$$\min_{\substack{Ax=b \\ x \geq 0}} c^T x = \max_{A^T y \leq c} b^T y.$$

Proof. Let x^* be an optimal primal solution, and choose M so large that

$$0 \leq x_i^* < M \quad \text{for every } i.$$

Set

$$X = [0, M]^n.$$

Define the Lagrangian

$$L(x, y) = c^T x + y^T (b - Ax).$$

For fixed x ,

$$\sup_{y \in \mathbb{R}^m} L(x, y) = \begin{cases} c^T x, & Ax = b, \\ +\infty, & Ax \neq b. \end{cases}$$

Therefore

$$\min_{x \in X} \sup_y L(x, y) = c^T x^*.$$

The set X is compact and convex, and L is linear in each variable, so Sion's theorem gives

$$c^T x^* = \sup_y \min_{x \in X} L(x, y).$$

For fixed y , write

$$r = c - A^T y.$$

Then

$$L(x, y) = b^T y + r^T x,$$

and hence

$$q(y) := \min_{x \in X} L(x, y) = b^T y + M \sum_{i=1}^n \min(0, r_i).$$

This is a concave polyhedral function of y . Since its supremum is finite, it attains that supremum; let y^* be a maximizer. Thus

$$q(y^*) = c^T x^*.$$

Because x^* is primal feasible,

$$L(x^*, y^*) = c^T x^*.$$

But

$$q(y^*) = \min_{x \in X} L(x, y^*) = c^T x^*,$$

so x^* minimizes $L(\cdot, y^*)$ over the box X .

Now every coordinate satisfies

$$x_i^* < M.$$

If some

$$r_i = c_i - (A^T y^*)_i$$

were negative, increasing x_i^* slightly would remain inside X and would strictly decrease $L(x, y^*)$, contradicting minimality. Therefore

$$r_i \geq 0 \quad \text{for every } i,$$

or equivalently,

$$A^T y^* \leq c.$$

Thus y^* is dual feasible.

Since $r \geq 0$,

$$q(y^*) = \min_{0 \leq x \leq M} (b^T y^* + r^T x) = b^T y^*.$$

Consequently,

$$b^T y^* = q(y^*) = c^T x^*,$$

which proves strong duality. □

Weak duality is even simpler: if x is primal feasible and y is dual feasible, then

$$b^T y = x^T A^T y \leq x^T c = c^T x.$$

The signs in the other standard forms can be recovered without memorization.

Free primal variables. If x_j is unrestricted, write

$$x_j = x_j^+ - x_j^-, \quad x_j^+, x_j^- \geq 0.$$

The two corresponding dual inequalities are

$$(A^T y)_j \leq c_j \quad \text{and} \quad -(A^T y)_j \leq -c_j,$$

so together they give

$$(A^T y)_j = c_j.$$

Primal inequality constraints. For a minimization problem,

$$Ax \geq b$$

can be written

$$Ax - s = b, \quad s \geq 0.$$

The dual constraint corresponding to s is

$$-y \leq 0,$$

hence

$$y \geq 0.$$

Similarly,

$$Ax \leq b$$

gives

$$y \leq 0.$$

Thus the standard correspondences for a minimization primal are

Primal	Dual
$x_j \geq 0$	$(A^T y)_j \leq c_j$
x_j free	$(A^T y)_j = c_j$
$(Ax)_i = b_i$	y_i free
$(Ax)_i \geq b_i$	$y_i \geq 0$
$(Ax)_i \leq b_i$	$y_i \leq 0$

Negating the objective gives the familiar maximization form:

maximize $c^T x$ subject to $Ax \leq b,$ $x \geq 0$	$\Longleftrightarrow$	minimize $b^T y$ subject to $A^T y \geq c,$ $y \geq 0.$
---	-----------------------	---

6.3 Majorization

Theorem 121 (Hardy-Littlewood-Pólya). *Let $x, y \in \mathbb{R}^n$. If $x \prec_w y$, then there exists a doubly stochastic matrix M such that $x \leq My$. If $x \prec y$ then $x = My$.*

Proof. Let $\mathcal{M}$ be the compact, convex set of $n \times n$ doubly stochastic matrices, and let $\mathcal{V} = [0, 1]^n$, which is also compact and convex. Consider the bilinear function $f(M, v) = v^T(My - x)$.

For any fixed $v \in \mathcal{V}$, let $P \in \mathcal{M}$ be a permutation matrix that orders the components of y comonotonically with v . Because $x \prec y$, an application of summation by parts yields $v^T x \leq v^T P y$. Consequently, we can always choose $M = P$ to see that $\max_{M \in \mathcal{M}} f(M, v) \geq f(P, v) \geq 0$.

By Sion's Minimax Theorem:

$$\max_{M \in \mathcal{M}} \min_{v \in \mathcal{V}} v^T(My - x) = \min_{v \in \mathcal{V}} \max_{M \in \mathcal{M}} v^T(My - x) \geq 0$$

This guarantees the existence of some $M_0 \in \mathcal{M}$ such that $v^T(M_0 y - x) \geq 0$ for all possible vectors $v \in \mathcal{V}$.

Applying this inequality with the standard basis vectors $v = e_i \in \mathcal{V}$ implies that $M_0 y \geq x$ component-wise. However, because M_0 is doubly stochastic, the sum of the components of $M_0 y$ exactly equals the sum of the components of y . By the definition of majorization, $\sum y_i = \sum x_i$ if $x \prec y$, thus $M_0 y = x$. $\square$

6.3.1 Muirhead's Inequality

For

$$\alpha = (\alpha_1, \dots, \alpha_n) \in \mathbb{R}_{\geq 0}^n$$

and

$$x = (x_1, \dots, x_n) \in \mathbb{R}_{\geq 0}^n,$$

define the symmetric monomial mean

$$[\alpha](x) = \frac{1}{n!} \sum_{\pi \in S_n} x_1^{\alpha_{\pi(1)}} \cdots x_n^{\alpha_{\pi(n)}}.$$

We average over all permutations, including repetitions when some α_i 's are equal.

Muirhead's inequality says that making the exponent vector more unequal increases its symmetric monomial mean.

Theorem 122 (Muirhead's Inequality). *Let*

$$\alpha, \beta \in \mathbb{R}_{\geq 0}^n$$

satisfy

$$\beta \prec \alpha.$$

Then for every

$$x_1, \dots, x_n \geq 0,$$

$$[\beta](x) \leq [\alpha](x).$$

Proof. We first assume

$$x_i > 0 \quad (i = 1, \dots, n);$$

the general case follows by continuity.

By Hardy–Littlewood–Pólya,

$$\beta = M\alpha$$

for some doubly stochastic matrix M .

By the Birkhoff–von Neumann theorem, every doubly stochastic matrix is a convex combination of permutation matrices, so

$$M = \sum_{k=1}^N \theta_k P_k, \quad \theta_k \geq 0, \quad \sum_k \theta_k = 1.$$

Therefore

$$\beta = \sum_k \theta_k P_k \alpha.$$

Fix a permutation $Q \in S_n$. Then

$$Q\beta = \sum_k \theta_k Q P_k \alpha.$$

Writing

$$x^\gamma = \prod_{i=1}^n x_i^{\gamma_i},$$

we obtain

$$\begin{aligned} x^{Q\beta} &= x^{\sum_k \theta_k Q P_k \alpha} \\ &= \prod_k (x^{Q P_k \alpha})^{\theta_k}. \end{aligned}$$

Weighted AM–GM gives

$$x^{Q\beta} \leq \sum_k \theta_k x^{QP_k\alpha}.$$

Now average this inequality over all $Q \in S_n$. The left-hand side gives

$$\frac{1}{n!} \sum_Q x^{Q\beta} = [\beta](x).$$

For each fixed P_k , as Q ranges over S_n , so does QP_k . Hence

$$\frac{1}{n!} \sum_Q x^{QP_k\alpha} = [\alpha](x).$$

Therefore

$$\begin{aligned} [\beta](x) &\leq \sum_k \theta_k \left(\frac{1}{n!} \sum_Q x^{QP_k\alpha} \right) \\ &= \sum_k \theta_k [\alpha](x) \\ &= [\alpha](x). \end{aligned}$$

This proves Muirhead for positive x_i , and continuity gives the result when some $x_i = 0$. □

6.4 Radon's Theorem

Theorem 123 (Radon's Theorem). *Any $d + 2$ points*

$$x_1, \dots, x_{d+2} \in \mathbb{R}^d$$

can be partitioned into two nonempty sets whose convex hulls intersect.

Proof. The lifted vectors

$$(x_1, 1), \dots, (x_{d+2}, 1) \in \mathbb{R}^{d+1}$$

are linearly dependent. Thus there are coefficients λ_i , not all zero, such that

$$\sum_i \lambda_i x_i = 0, \quad \sum_i \lambda_i = 0.$$

Let

$$P = \{i : \lambda_i > 0\}, \quad N = \{i : \lambda_i < 0\}.$$

Both are nonempty. Set

$$L = \sum_{i \in P} \lambda_i = - \sum_{i \in N} \lambda_i > 0.$$

Then

$$\sum_{i \in P} \frac{\lambda_i}{L} x_i = \sum_{i \in N} \frac{-\lambda_i}{L} x_i.$$

Each side is a convex combination, so this common point lies in both convex hulls. □

6.5 Helly's Theorem

Theorem 124 (Helly's Theorem). *Let*

$$C_1, \dots, C_m \subseteq \mathbb{R}^d$$

be convex sets, with $m \geq d + 1$. If every $d + 1$ of them have nonempty intersection, then

$$\bigcap_{i=1}^m C_i \neq \emptyset.$$

Proof. We induct on m . The case $m = d + 1$ is exactly the hypothesis.

Suppose $m > d + 1$. By induction, deleting any one of the sets leaves a family with nonempty intersection. For

$$i = 1, \dots, d + 2,$$

choose

$$x_i \in \bigcap_{j \neq i} C_j.$$

Apply Radon's theorem to

$$x_1, \dots, x_{d+2}.$$

There is a partition

$$P \sqcup N = [d + 2]$$

and a point

$$x \in \text{conv}\{x_i : i \in P\} \cap \text{conv}\{x_i : i \in N\}.$$

We claim $x \in C_j$ for every j .

If $j \leq d + 2$, one of P, N does not contain j . Every x_i in that part belongs to C_j , so by convexity its convex hull, and hence x , lies in C_j .

If $j > d + 2$, then every x_i lies in C_j , so again $x \in C_j$.

Thus

$$x \in \bigcap_{j=1}^m C_j.$$

□

6.6 Carathéodory's Theorem

Theorem 125 (Carathéodory's Theorem). *If*

$$x \in \text{conv}(S) \subseteq \mathbb{R}^d,$$

then x lies in the convex hull of at most $d + 1$ points of S .

Proof. Choose a representation

$$x = \sum_{i=1}^m \alpha_i x_i, \quad \alpha_i > 0, \quad \sum_i \alpha_i = 1,$$

using as few points as possible.

Suppose

$$m > d + 1.$$

Then the lifted vectors

$$(x_i, 1) \in \mathbb{R}^{d+1}$$

are linearly dependent, so there are λ_i , not all zero, with

$$\sum_i \lambda_i x_i = 0, \quad \sum_i \lambda_i = 0.$$

Replace

$$\alpha_i$$

by

$$\alpha_i - t\lambda_i.$$

This does not change either

$$\sum_i \alpha_i \quad \text{or} \quad \sum_i \alpha_i x_i.$$

Choose

$$t = \min_{\lambda_i > 0} \frac{\alpha_i}{\lambda_i}.$$

Then all new coefficients remain nonnegative and at least one becomes zero. This gives a convex representation of x using fewer points, a contradiction.

Hence

$$m \leq d + 1.$$

□

6.7 Colorful Carathéodory

Theorem 126 (Colorful Carathéodory). *Let*

$$C_1, \dots, C_{d+1} \subseteq \mathbb{R}^d$$

satisfy

$$0 \in \text{conv}(C_i) \quad \text{for every } i.$$

Then one can choose

$$c_i \in C_i$$

such that

$$0 \in \text{conv}\{c_1, \dots, c_{d+1}\}.$$

Proof. By [Theorem 125](#), we may first replace each C_i by a finite subset whose convex hull still contains 0.

Among all colorful choices

$$c_i \in C_i,$$

choose one whose convex hull

$$K = \text{conv}\{c_1, \dots, c_{d+1}\}$$

has minimum distance from 0. Let $z \in K$ be the closest point.

Suppose

$$z \neq 0.$$

The closest-point property implies

$$z^T x \geq \|z\|^2 \quad \text{for every } x \in K.$$

Indeed, if some $x \in K$ had

$$z^T x < \|z\|^2,$$

then moving slightly from z toward x would decrease the distance to 0.

The minimal face of K containing z therefore lies in the hyperplane

$$H = \{x : z^T x = \|z\|^2\},$$

which has dimension $d - 1$. Hence z is a convex combination of at most d of the selected points. Thus some color, say C_j , is absent from this representation.

Since

$$0 \in \text{conv}(C_j),$$

not every $y \in C_j$ can satisfy $z^T y > 0$. Choose

$$y \in C_j$$

with

$$z^T y \leq 0.$$

Replace the selected point of color j by y . The new colorful convex hull contains both z and y , and for small $\varepsilon > 0$,

$$\|(1 - \varepsilon)z + \varepsilon y\|^2 = \|z\|^2 + 2\varepsilon z^T(y - z) + O(\varepsilon^2) < \|z\|^2.$$

This contradicts the minimal choice of K .

Therefore

$$z = 0.$$

□

6.8 Tverberg's Theorem

Tverberg's theorem generalizes Radon's theorem from two parts to an arbitrary number of parts.

Theorem 127 (Tverberg's Theorem). *Let $r \geq 2$. Any*

$$(r - 1)(d + 1) + 1$$

points in $\mathbb{R}^d$ can be partitioned into r nonempty sets

$$S_1, \dots, S_r$$

such that

$$\bigcap_{j=1}^r \text{conv}(S_j) \neq \emptyset.$$

Proof. Set

$$D = (r - 1)(d + 1).$$

Let

$$u_1, \dots, u_r \in \mathbb{R}^{r-1}$$

be the vertices of a simplex centered at the origin. Thus

$$u_1 + \dots + u_r = 0,$$

and this is, up to scaling, their only linear dependence.

Let the given points be

$$x_1, \dots, x_{D+1} \in \mathbb{R}^d,$$

and lift them to

$$\bar{x}_i = (x_i, 1) \in \mathbb{R}^{d+1}.$$

For every i , define a color class in

$$\mathbb{R}^{r-1} \otimes \mathbb{R}^{d+1} \cong \mathbb{R}^D$$

by

$$C_i = \{u_1 \otimes \bar{x}_i, \dots, u_r \otimes \bar{x}_i\}.$$

Because

$$u_1 + \dots + u_r = 0,$$

we have

$$0 \in \text{conv}(C_i)$$

for every i .

There are exactly

$$D + 1$$

color classes in $\mathbb{R}^D$. By colorful Carathéodory, for every i we may choose an index

$$j(i) \in [r]$$

such that

$$0 \in \text{conv}\{u_{j(i)} \otimes \bar{x}_i : i = 1, \dots, D + 1\}.$$

Thus there are coefficients

$$\lambda_i \geq 0, \quad \sum_i \lambda_i = 1,$$

such that

$$\sum_i \lambda_i u_{j(i)} \otimes \bar{x}_i = 0.$$

Define

$$S_\ell = \{i : j(i) = \ell\}$$

and

$$z_j = \sum_{i \in S_j} \lambda_i \bar{x}_i.$$

Then

$$\sum_{j=1}^r u_j \otimes z_j = 0.$$

Choose

$$u_1, \dots, u_{r-1}$$

as a basis and write

$$u_r = -\sum_{j=1}^{r-1} u_j.$$

Then

$$0 = \sum_{j=1}^{r-1} u_j \otimes (z_j - z_r).$$

Linear independence of $u_1, \dots, u_{r-1}$ implies

$$z_1 = \dots = z_r =: z.$$

The last coordinate of z_j is

$$\alpha_j = \sum_{i \in S_j} \lambda_i.$$

Since all the z_j are equal,

$$\alpha_1 = \cdots = \alpha_r.$$

But

$$\sum_{j=1}^r \alpha_j = \sum_i \lambda_i = 1,$$

so

$$\alpha_j = \frac{1}{r}.$$

In particular, every S_j is nonempty.

Let y be r times the first d coordinates of z . For every j ,

$$y = \sum_{i \in S_j} \frac{\lambda_i}{\alpha_j} x_i,$$

and the coefficients are nonnegative and sum to 1. Therefore

$$y \in \text{conv}\{x_i : i \in S_j\}$$

for every j .

Hence

$$y \in \bigcap_{j=1}^r \text{conv}(S_j).$$

□

For more than $(r-1)(d+1)+1$ points, apply the theorem to any subset of the required size and distribute the remaining points arbitrarily among the parts.

6.9 Matroid Intersection

Let

$$M_1 = (E, \mathcal{I}_1), \quad M_2 = (E, \mathcal{I}_2)$$

be two matroids on the same ground set, with rank functions r_1, r_2 . We want a largest set

$$I \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

The basic example is bipartite matching: take one partition matroid requiring at most one edge incident to each left vertex, and another requiring at most one edge incident to each right vertex.

Theorem 128 (Matroid Intersection). *For any two matroids on the same ground set,*

$$\boxed{\max_{I \in \mathcal{I}_1 \cap \mathcal{I}_2} |I| = \min_{U \subseteq E} (r_1(U) + r_2(E \setminus U)).}$$

The easy direction is immediate. For every common independent set I and every $U \subseteq E$,

$$|I| = |I \cap U| + |I \setminus U| \leq r_1(U) + r_2(E \setminus U).$$

We prove the reverse inequality algorithmically.

Exchange Graph Fix a common independent set

$$I \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

Its exchange graph D_I is the directed bipartite graph with sides I and $E \setminus I$.

For $y \in I$ and $x \notin I$, add

$$y \longrightarrow x$$

if

$$I - y + x \in \mathcal{I}_1,$$

and add

$$x \longrightarrow y$$

if

$$I - y + x \in \mathcal{I}_2.$$

Also define

$$X_1 = \{x \notin I : I + x \in \mathcal{I}_1\},$$

and

$$X_2 = \{x \notin I : I + x \in \mathcal{I}_2\}.$$

Thus an augmenting path has the form

$$x_0, y_1, x_1, \dots, y_k, x_k,$$

where

$$x_0 \in X_1, \quad x_k \in X_2.$$

If P is such a path, define

$$I' = I \Delta V(P) = I - \{y_1, \dots, y_k\} + \{x_0, \dots, x_k\}.$$

Then

$$|I'| = |I| + 1.$$

We use a shortest augmenting path, so it has no directed shortcut.

Lemma 129 (Unique Exchange Matching). *Let $M = (E, \mathcal{I})$ be a matroid, let $I \in \mathcal{I}$, and let J satisfy*

$$|J| = |I|.$$

Construct the bipartite exchange graph between $I \setminus J$ and $J \setminus I$, joining $y \in I \setminus J$ to $x \in J \setminus I$ whenever

$$I - y + x \in \mathcal{I}.$$

If this graph has a unique perfect matching, then

$$J \in \mathcal{I}.$$

Proof. Let the unique perfect matching be

$$(y_1, x_1), \dots, (y_k, x_k).$$

Orient the matching edges from x_i to y_i and all other exchange edges from y_i to x_j . There can be no directed cycle, since such a cycle would be an alternating cycle and would give a second perfect matching.

Thus we may order the pairs so that

$$y_i x_j$$

is never an exchange edge when $i < j$.

Suppose for contradiction that J is dependent, and let $C \subseteq J$ be a circuit. Choose the smallest i such that

$$x_i \in C.$$

For every other $x_j \in C \setminus I$, we have $j > i$, so

$$I - y_i + x_j$$

is dependent. Hence

$$x_j \in \text{cl}(I - y_i).$$

Every element of $C \cap I$ also lies in $I - y_i$. Therefore

$$C - x_i \subseteq \text{cl}(I - y_i).$$

Since C is a circuit,

$$x_i \in \text{cl}(C - x_i) \subseteq \text{cl}(I - y_i).$$

But (y_i, x_i) is an exchange edge, so

$$I - y_i + x_i$$

is independent, a contradiction. □

Lemma 130 (Augmenting Path). *If P is a shortest directed path from X_1 to X_2 in D_I , then*

$$I \Delta V(P) \in \mathcal{I}_1 \cap \mathcal{I}_2.$$

Proof. Write

$$P = x_0, y_1, x_1, \dots, y_k, x_k.$$

We prove independence in M_1 . Add a new element t which is a coloop, so that

$$I + t$$

is independent.

Between

$$(I \cap V(P)) \cup \{t\}$$

and

$$V(P) \setminus I,$$

the M_1 exchange graph has the perfect matching

$$tx_0, \quad y_1x_1, \quad \dots, \quad y_kx_k.$$

Because P is shortest, it has no shortcut, so this perfect matching is unique. By [Lemma 129](#),

$$I \Delta V(P)$$

is independent in M_1 .

For M_2 , use instead the perfect matching

$$y_1x_0, \quad y_2x_1, \quad \dots, \quad y_kx_{k-1}, \quad tx_k.$$

The same argument proves independence in M_2 . □

Algorithm 8 Matroid Intersection

Input: Independence oracles for M_1, M_2

Output: A maximum common independent set I

```
1:  $I \leftarrow \emptyset$ 
2: while true do
3:   Construct the exchange digraph  $D_I$ 
4:   Construct source set  $X_1$  and sink set  $X_2$ 
5:   Find a shortest directed path  $P$  from  $X_1$  to  $X_2$ 
6:   if no such path exists then
7:     return  $I$ 
8:    $I \leftarrow I \triangle V(P)$ 
```

It remains to prove that failure to find an augmenting path certifies optimality.

Proof of Theorem 128. The upper bound was already proved.

Run Algorithm 8. Every augmentation increases $|I|$ by one and preserves common independence by Lemma 130.

Suppose eventually there is no path from X_1 to X_2 . Let

$$U = \{e \in E : e \text{ can reach } X_2 \text{ in } D_I\}.$$

Then

$$X_2 \subseteq U, \quad X_1 \cap U = \emptyset.$$

We claim

$$r_1(U) = |I \cap U|.$$

If not, some

$$x \in U \setminus I$$

could be added to $I \cap U$ while preserving independence in M_1 . Since $x \notin X_1$, the set $I + x$ is dependent. Let C be its fundamental circuit.

Because

$$(I \cap U) + x$$

is independent, C must contain some

$$y \in I \setminus U.$$

Removing any element of the fundamental circuit gives an independent set, so

$$I - y + x \in \mathcal{I}_1.$$

Thus

$$y \longrightarrow x$$

is an arc. But $x \in U$ can reach X_2 , hence so can y , contradicting $y \notin U$.

Thus

$$r_1(U) = |I \cap U|.$$

Similarly,

$$r_2(E \setminus U) = |I \setminus U|.$$

Indeed, otherwise there is

$$x \in (E \setminus U) \setminus I$$

such that

$$(I \setminus U) + x$$

is independent in M_2 . Since $X_2 \subseteq U$, we have $x \notin X_2$, so $I + x$ has a fundamental circuit containing some

$$y \in I \cap U.$$

Then

$$I - y + x \in \mathcal{I}_2,$$

so

$$x \longrightarrow y$$

is an arc. Since y can reach X_2 , so can x , contradicting $x \notin U$.

Therefore

$$r_1(U) + r_2(E \setminus U) = |I \cap U| + |I \setminus U| = |I|.$$

This matches the general upper bound, so I is maximum and proves the min-max theorem. $\square$

With independence oracles, the direct implementation uses $O(|E|^2)$ oracle queries per augmentation and at most $|E|$ augmentations, hence $O(|E|^3)$ oracle queries.

7 Complex Analysis

7.1 Cauchy's Theorem

The key fact of complex integration is that a holomorphic function has zero integral around a closed contour. The essential case is a triangle.

Theorem 131 (Cauchy–Goursat for Triangles). *Let T be a triangle, and suppose that f is holomorphic on an open neighborhood of T . Then*

$$\oint_{\partial T} f(z) dz = 0.$$

Proof. Write

$$I(T) = \oint_{\partial T} f(z) dz.$$

Divide T into four similar triangles by joining the midpoints of its sides. The oriented boundary integrals of the four smaller triangles sum to $I(T)$, since every interior edge appears twice with opposite orientations. Hence one of the four triangles, call it T_1 , satisfies

$$|I(T_1)| \geq \frac{1}{4} |I(T)|.$$

Repeating this construction gives nested triangles

$$T \supset T_1 \supset T_2 \supset \cdots$$

such that

$$|I(T_n)| \geq 4^{-n} |I(T)|, \quad \text{diam}(T_n) = 2^{-n} \text{diam}(T).$$

The triangles shrink to a single point z_0 .

Since f is differentiable at z_0 ,

$$f(z) = f(z_0) + f'(z_0)(z - z_0) + r(z),$$

where

$$r(z) = o(|z - z_0|).$$

Thus, for every $\varepsilon > 0$, for all sufficiently large n and every $z \in T_n$,

$$|r(z)| \leq \varepsilon |z - z_0| \leq \varepsilon \operatorname{diam}(T_n).$$

The constant and linear terms have zero integral around a closed contour:

$$\oint_{\partial T_n} 1 \, dz = 0, \quad \oint_{\partial T_n} (z - z_0) \, dz = 0.$$

Therefore

$$I(T_n) = \oint_{\partial T_n} r(z) \, dz.$$

Hence

$$\begin{aligned} |I(T_n)| &\leq \max_{z \in T_n} |r(z)| \operatorname{perimeter}(T_n) \\ &\leq \varepsilon \operatorname{diam}(T_n) \operatorname{perimeter}(T_n) \\ &= C\varepsilon 4^{-n}, \end{aligned}$$

where

$$C = \operatorname{diam}(T) \operatorname{perimeter}(T).$$

Combining this with

$$4^{-n} |I(T)| \leq |I(T_n)|$$

gives

$$|I(T)| \leq C\varepsilon.$$

Since $\varepsilon > 0$ is arbitrary, $I(T) = 0$. □

The proof has one main idea: repeatedly zoom in while retaining a constant fraction of the integral. At the limiting point, holomorphicity says that f is linear up to an error smaller than the scale of the triangle; the linear part integrates to zero and the error is too small to support a non-zero integral.

Corollary 132 (Cauchy's Theorem for Polygonal Curves). *Let γ be a positively oriented simple closed polygonal curve, and suppose that f is holomorphic on an open neighborhood of γ and its interior. Then*

$$\oint_{\gamma} f(z) \, dz = 0.$$

Proof. Triangulate the polygon bounded by γ . By [Theorem 131](#), the integral around every triangle is zero. Summing these integrals, every interior edge appears twice with opposite orientations and cancels, leaving exactly

$$\oint_{\gamma} f(z) \, dz = 0.$$

□

The polygonal statement immediately gives the usual form of Cauchy's theorem.

Theorem 133 (Cauchy's Theorem). *Let $U \subseteq \mathbb{C}$ be simply connected, let $f : U \rightarrow \mathbb{C}$ be holomorphic, and let γ be a closed piecewise- C^1 curve in U . Then*

$$\oint_{\gamma} f(z) \, dz = 0.$$

Proof. First suppose that γ is a simple closed curve. Since U is simply connected, the interior of γ is contained in U .

Approximate γ by simple closed polygonal curves γ_m such that both the curves and their derivatives converge to γ on each C^1 piece. For sufficiently large m , γ_m and its interior lie inside U , so by [Corollary 132](#),

$$\oint_{\gamma_m} f(z) dz = 0.$$

Writing the curves on the same parameter interval,

$$\begin{aligned} \left| \int_{\gamma} f(z) dz - \int_{\gamma_m} f(z) dz \right| &= \left| \int (f(\gamma(t))\gamma'(t) - f(\gamma_m(t))\gamma'_m(t)) dt \right| \\ &\leq \int |f(\gamma) - f(\gamma_m)| |\gamma'| dt + \int |f(\gamma_m)| |\gamma' - \gamma'_m| dt. \end{aligned}$$

Since f is uniformly continuous and bounded on a compact neighborhood of the curves, both terms tend to zero. Hence

$$\oint_{\gamma} f(z) dz = 0.$$

A general closed piecewise- C^1 curve can be approximated in the same way by a closed polygonal curve. Splitting the polygon at its finitely many self-intersections decomposes it into finitely many simple closed polygonal curves, each of which has zero integral. Passing to the limit again proves the result. $\square$

Corollary 134 (Cauchy's Theorem with Holes). *Let Ω be a bounded region whose boundary consists of finitely many piecewise- C^1 curves, oriented so that Ω always lies to the left. If f is holomorphic on a neighborhood of $\overline{\Omega}$, then*

$$\oint_{\partial\Omega} f(z) dz = 0.$$

Equivalently, if γ is the counterclockwise outer boundary and $C_1, \dots, C_m$ are the counterclockwise boundaries of the holes, then

$$\oint_{\gamma} f(z) dz = \sum_{j=1}^m \oint_{C_j} f(z) dz.$$

Proof. Join every inner boundary to the outer boundary by non-intersecting cuts. The resulting region is simply connected, so Cauchy's theorem applies. Each cut appears twice in the resulting boundary, once in each direction, and the two integrals cancel. What remains is the displayed boundary integral. $\square$

7.2 Cauchy's Integral Formula

Cauchy's integral formula can be viewed retrospectively as the one-pole case of the residue theorem. We prove it first, however, since it follows directly from Cauchy's theorem and does not require Laurent series.

Theorem 135 (Cauchy's Integral Formula). *Let γ be a positively oriented simple closed piecewise- C^1 curve, let z lie in its interior, and suppose that f is holomorphic on a neighborhood of γ and its interior. Then*

$$f(z) = \frac{1}{2\pi i} \oint_{\gamma} \frac{f(w)}{w - z} dw.$$

Proof. Choose a sufficiently small circle

$$C_{\varepsilon} = \{w : |w - z| = \varepsilon\}$$

inside γ . The function

$$w \mapsto \frac{f(w)}{w - z}$$

is holomorphic on the region between γ and C_ε . By [Corollary 134](#),

$$\oint_{\gamma} \frac{f(w)}{w-z} dw = \oint_{C_\varepsilon} \frac{f(w)}{w-z} dw.$$

Parameterize the small circle by

$$w = z + \varepsilon e^{it}, \quad 0 \leq t \leq 2\pi.$$

Then

$$\begin{aligned} \oint_{C_\varepsilon} \frac{f(w)}{w-z} dw &= \int_0^{2\pi} \frac{f(z + \varepsilon e^{it})}{\varepsilon e^{it}} i\varepsilon e^{it} dt \\ &= i \int_0^{2\pi} f(z + \varepsilon e^{it}) dt. \end{aligned}$$

As $\varepsilon \rightarrow 0$, continuity of f gives

$$i \int_0^{2\pi} f(z + \varepsilon e^{it}) dt \longrightarrow 2\pi i f(z).$$

The left-hand side is independent of ε , so it must already equal $2\pi i f(z)$. $\square$

Corollary 136 (Cauchy's Differentiation Formula). *Under the hypotheses of [Theorem 135](#), for every integer $k \geq 0$,*

$$f^{(k)}(z) = \frac{k!}{2\pi i} \oint_{\gamma} \frac{f(w)}{(w-z)^{k+1}} dw.$$

Proof. Differentiate the integral formula k times with respect to z . Since the contour stays a positive distance from z , differentiation under the integral is valid, and

$$\frac{d^k}{dz^k} \frac{1}{w-z} = \frac{k!}{(w-z)^{k+1}}.$$

$\square$

7.3 Laurent Series

Cauchy's integral formula also explains why holomorphic functions are represented by power series, and why isolated singularities are represented by Laurent series.

Theorem 137 (Laurent Expansion). *Suppose that f is holomorphic on the annulus*

$$r < |z - a| < R.$$

Then there are unique coefficients $(c_n)_{n \in \mathbb{Z}}$ such that

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n$$

throughout the annulus, with convergence uniform on compact subannuli.

For every $r < \rho < R$,

$$\boxed{c_n = \frac{1}{2\pi i} \oint_{|w-a|=\rho} \frac{f(w)}{(w-a)^{n+1}} dw} \quad (n \in \mathbb{Z}).$$

Proof. Fix z with

$$r < |z - a| < R,$$

and choose radii

$$r < \rho_- < |z - a| < \rho_+ < R.$$

Applying the same argument as Cauchy's integral formula to the annulus between the two circles gives

$$f(z) = \frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{w-z} dw - \frac{1}{2\pi i} \oint_{|w-a|=\rho_-} \frac{f(w)}{w-z} dw.$$

For the outer circle, $|z - a| < |w - a|$, so

$$\begin{aligned} \frac{1}{w-z} &= \frac{1}{(w-a) - (z-a)} \\ &= \frac{1}{w-a} \frac{1}{1 - \frac{z-a}{w-a}} \\ &= \sum_{n=0}^{\infty} \frac{(z-a)^n}{(w-a)^{n+1}}. \end{aligned}$$

The geometric series converges uniformly on the circle, so we may integrate term-by-term:

$$\frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{w-z} dw = \sum_{n=0}^{\infty} \left(\frac{1}{2\pi i} \oint_{|w-a|=\rho_+} \frac{f(w)}{(w-a)^{n+1}} dw \right) (z-a)^n.$$

For the inner circle, $|w - a| < |z - a|$, so

$$\begin{aligned} \frac{1}{w-z} &= -\frac{1}{z-a} \frac{1}{1 - \frac{w-a}{z-a}} \\ &= -\sum_{m=0}^{\infty} \frac{(w-a)^m}{(z-a)^{m+1}}. \end{aligned}$$

Again the convergence is uniform, hence

$$-\frac{1}{2\pi i} \oint_{|w-a|=\rho_-} \frac{f(w)}{w-z} dw = \sum_{m=0}^{\infty} \left(\frac{1}{2\pi i} \oint_{|w-a|=\rho_-} f(w)(w-a)^m dw \right) (z-a)^{-m-1}.$$

Together these give a Laurent expansion

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z-a)^n.$$

For any fixed integer n , the function

$$w \mapsto \frac{f(w)}{(w-a)^{n+1}}$$

is holomorphic throughout the annulus. By [Corollary 134](#), its integral around two circles centered at a inside the annulus is the same. Thus both the positive and negative coefficients can be written uniformly as

$$c_n = \frac{1}{2\pi i} \oint_{|w-a|=\rho} \frac{f(w)}{(w-a)^{n+1}} dw$$

for any $r < \rho < R$.

The geometric-series argument is uniform whenever z ranges over a compact subannulus, which gives locally uniform convergence.

Finally, the coefficient formula determines every c_n , so the expansion is unique. $\square$

Corollary 138 (Taylor Expansion). *If f is holomorphic on $|z - a| < R$, then*

$$f(z) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (z - a)^n.$$

Proof. Apply [Theorem 137](#) with $r = 0$. For $n < 0$, the coefficient integrand

$$\frac{f(w)}{(w - a)^{n+1}}$$

has a non-negative power of $(w - a)$ and is holomorphic inside the circle, so Cauchy's theorem gives $c_n = 0$.

For $n \geq 0$, [Corollary 136](#) gives

$$c_n = \frac{f^{(n)}(a)}{n!}.$$

□

7.4 Residue Theorem

Let a be an isolated singularity of f . By [Theorem 137](#), in a punctured neighborhood of a ,

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z - a)^n.$$

The coefficient

$$\text{Res}(f, a) := c_{-1}$$

is called the *residue* of f at a .

The significance of this coefficient follows from the elementary identity

$$\oint_{|z-a|=\rho} (z - a)^n dz = \begin{cases} 2\pi i, & n = -1, \\ 0, & n \neq -1. \end{cases}$$

Indeed, uniform convergence of the Laurent series on the circle gives

$$\boxed{\oint_{|z-a|=\rho} f(z) dz = 2\pi i \text{Res}(f, a).}$$

Theorem 139 (Residue Theorem). *Let γ be a positively oriented simple closed piecewise- C^1 curve, and suppose that f is holomorphic on a neighborhood of γ and its interior except at finitely many isolated singularities*

$$a_1, \dots, a_m$$

inside γ . Then

$$\oint_{\gamma} f(z) dz = 2\pi i \sum_{j=1}^m \text{Res}(f, a_j).$$

Proof. Choose disjoint small counterclockwise circles C_j around the singularities. On the region inside γ with the interiors of these circles removed, f is holomorphic. Therefore [Corollary 134](#) gives

$$\oint_{\gamma} f(z) dz = \sum_{j=1}^m \oint_{C_j} f(z) dz.$$

By the Laurent expansion around a_j ,

$$\oint_{C_j} f(z) dz = 2\pi i \text{Res}(f, a_j).$$

Summing gives the result.

□

Corollary 140 (Residue at a Pole). *If a is a pole of order m of f , then*

$$\operatorname{Res}(f, a) = \frac{1}{(m-1)!} \frac{d^{m-1}}{dz^{m-1}} ((z-a)^m f(z)) \Big|_{z=a}.$$

In particular, at a simple pole,

$$\operatorname{Res}(f, a) = \lim_{z \rightarrow a} (z-a)f(z).$$

Proof. Write

$$f(z) = \frac{g(z)}{(z-a)^m},$$

where g is holomorphic at a . The residue is the coefficient of $(z-a)^{-1}$, hence it is the coefficient of $(z-a)^{m-1}$ in the Taylor series of g . Therefore

$$\operatorname{Res}(f, a) = \frac{g^{(m-1)}(a)}{(m-1)!}.$$

□

7.5 Cotangent Partial Fractions

The residue theorem gives a partial fraction expansion of the cotangent.

Theorem 141 (Cotangent Partial Fraction Expansion). *For every $z \notin \mathbb{Z}$,*

$$\pi \cot(\pi z) = \frac{1}{z} + 2z \sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Equivalently,

$$\pi \cot(\pi z) = \frac{1}{z} + \sum_{n=1}^{\infty} \left(\frac{1}{z-n} + \frac{1}{z+n} \right).$$

Proof. Fix $z \notin \mathbb{Z}$ and consider, as a function of w ,

$$f(w) = \frac{\pi \cot(\pi w)}{w^2 - z^2}.$$

Let C_N be the positively oriented square with vertices

$$\left(N + \frac{1}{2}\right)(\pm 1 \pm i).$$

For sufficiently large N , the points $\pm z$ lie inside C_N .

The poles of f inside C_N are the integers

$$-N, -N+1, \dots, N$$

and the two points $\pm z$.

At an integer n , the function $\pi \cot(\pi w)$ has residue 1, so

$$\operatorname{Res}(f, n) = \frac{1}{n^2 - z^2}.$$

At $w = z$,

$$\operatorname{Res}(f, z) = \frac{\pi \cot(\pi z)}{2z}.$$

Since $\cot$ is odd,

$$\operatorname{Res}(f, -z) = \frac{\pi \cot(-\pi z)}{-2z} = \frac{\pi \cot(\pi z)}{2z}.$$

Thus the residues at $\pm z$ sum to

$$\frac{\pi \cot(\pi z)}{z}.$$

It remains to show that the contour integral tends to zero. Writing $w = x + iy$,

$$|\cot(\pi w)|^2 = \frac{\cos^2(\pi x) + \sinh^2(\pi y)}{\sin^2(\pi x) + \sinh^2(\pi y)}.$$

On the vertical sides of C_N , $x = N + \frac{1}{2}$, so

$$|\cot(\pi w)| \leq 1.$$

On the horizontal sides, $|y| = N + \frac{1}{2}$, so $\cot(\pi w)$ is also uniformly bounded independently of N .

Furthermore,

$$|w^2 - z^2| = \Omega(N^2)$$

uniformly on C_N , while the length of C_N is $O(N)$. Hence

$$\oint_{C_N} f(w) dw = O(N^{-1}) \rightarrow 0.$$

By the residue theorem,

$$0 = \frac{\pi \cot(\pi z)}{z} + \sum_{n \in \mathbb{Z}} \frac{1}{n^2 - z^2}.$$

Therefore

$$\frac{\pi \cot(\pi z)}{z} = - \sum_{n \in \mathbb{Z}} \frac{1}{z^2 - n^2}.$$

Separating the $n = 0$ term and pairing n with $-n$ gives

$$\frac{\pi \cot(\pi z)}{z} = \frac{1}{z^2} + 2 \sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Multiplying by z proves

$$\pi \cot(\pi z) = \frac{1}{z} + 2z \sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

Finally,

$$\frac{2z}{z^2 - n^2} = \frac{1}{z - n} + \frac{1}{z + n},$$

which gives the equivalent form. □

The pairing of n and $-n$ is important: the paired terms are $O(n^{-2})$, so the series converges on compact subsets of $\mathbb{C} \setminus \mathbb{Z}$.

7.6 Euler's Product for Sine

The sine product follows almost immediately by integrating the cotangent partial fraction expansion.

Theorem 142 (Euler's Product for Sine). *For every $z \in \mathbb{C}$,*

$$\frac{\sin(\pi z)}{\pi z} = \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right).$$

Equivalently,

$$\sin(\pi z) = \pi z \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2}\right).$$

Proof. From [Theorem 141](#),

$$\pi \cot(\pi z) - \frac{1}{z} = 2z \sum_{n=1}^{\infty} \frac{1}{z^2 - n^2}.$$

The left-hand side is

$$\frac{d}{dz} \log \left(\frac{\sin(\pi z)}{\pi z} \right).$$

For every n ,

$$\frac{d}{dz} \log \left(1 - \frac{z^2}{n^2} \right) = \frac{2z}{z^2 - n^2}.$$

Hence

$$\frac{d}{dz} \log \left(\frac{\sin(\pi z)}{\pi z} \right) = \sum_{n=1}^{\infty} \frac{d}{dz} \log \left(1 - \frac{z^2}{n^2} \right).$$

The series on the right converges uniformly on compact subsets avoiding the integers, since its terms are $O(n^{-2})$. Thus we may integrate term-by-term.

Near $z = 0$, choose the logarithms so that they vanish at 0. Integrating from 0 to z gives

$$\log \left(\frac{\sin(\pi z)}{\pi z} \right) = \sum_{n=1}^{\infty} \log \left(1 - \frac{z^2}{n^2} \right),$$

because

$$\lim_{z \rightarrow 0} \frac{\sin(\pi z)}{\pi z} = 1.$$

Exponentiating,

$$\frac{\sin(\pi z)}{\pi z} = \prod_{n=1}^{\infty} \left(1 - \frac{z^2}{n^2} \right)$$

in a neighborhood of 0.

The infinite product converges uniformly on compact sets, since

$$\sum_{n=1}^{\infty} \frac{|z|^2}{n^2}$$

converges uniformly when z is restricted to a compact set. Thus the product defines an entire function. Since both sides are entire and agree on a neighborhood of 0, the identity theorem extends the equality to all $z \in \mathbb{C}$. $\square$

8 Data Structures

8.1 RMQ

We want to find the index of the minimum element within an arbitrary subarray $A[l \dots r]$. We can achieve $O(1)$ query time with $O(n)$ preprocessing time and space.

1. Blocks We partition the array into blocks of size $B = \Theta(\log n)$. For each index $i \in [0, n-1]$, we maintain a bitmask $mask[i]$ of length up to B . This mask represents a strictly increasing subsequence of values ending at i , extending leftward up to the block boundary or at most B elements.

Specifically, the k -th bit (0-indexed) of $mask[i]$ is set to 1 if and only if $A[i-k]$ is the minimum element in the suffix $A[i-k \dots i]$. As we iterate i from 0 to $n-1$, we compute $mask[i]$ iteratively:

1. Shift the previous mask left by 1: $mask_{\text{curr}} \leftarrow mask_{i-1} \ll 1$.
2. While the current element $A[i]$ is strictly less than the element corresponding to the lowest set bit in $mask_{\text{curr}}$, unset that bit.
3. Set the 0-th bit to 1, representing the element $A[i]$ itself.

2. Sparse Table We construct a Sparse Table over the $\lfloor \frac{n}{B} \rfloor$ blocks. Instead of storing all elements, the 0-th level of the Sparse Table only stores the overall minimum of each block. The minimum of the j -th block (which ends at index $i = j \cdot B + B - 1$) is found using its mask: $mask[i]$ contains suffix minimums, so its highest set bit is the minimum of the entire block. Because there are only n/B blocks, the Sparse Table takes $O((n/B) \log(n/B)) = O(n)$ time and space.

3. Query Resolution To answer a query for the range $[l, r]$:

- **Case 1: The range spans at most B elements.** The range is small enough that the entire answer is encoded within $mask[r]$. We truncate the mask to only consider the last $r-l+1$ elements by doing a bitwise AND with $(1 \ll (r-l+1)) - 1$. The index of the highest set bit in this truncated mask corresponds to the minimum in $[l, r]$.
- **Case 2: The range spans multiple blocks.** We decompose the range into three parts:
 1. A partial block on the left: from l to the end of its block.
 2. A set of completely covered intermediate blocks.
 3. A partial block on the right: from the start of its block to r .

The minimum of the left partial block is found using the highest set bit of the mask at the end of that block. The minimum of the right partial block is found using the highest set bit of $mask[r]$. The minimum of the completely covered intermediate blocks is queried in $O(1)$ time from the Sparse Table. We simply take the minimum of these three candidates.

Algorithm 9 RMQ

```
1: function INIT( $A[0 \dots n-1]$ )
2:    $B \leftarrow \Theta(\log n)$ 
3:    $mask \leftarrow$  array of size  $n$ 
4:    $curr\_mask \leftarrow 0$ 
5:   for  $i \leftarrow 0$  to  $n-1$  do
6:      $curr\_mask \leftarrow curr\_mask \ll 1$ 
7:     while  $curr\_mask \neq 0$  and  $A[i] < A[i - \text{trailing\_zeros}(curr\_mask)]$  do
8:        $curr\_mask \leftarrow curr\_mask \& (curr\_mask - 1)$   $\triangleright$  Pops the lowest set bit
9:        $curr\_mask \leftarrow curr\_mask \mid 1$ 
10:     $mask[i] \leftarrow curr\_mask$   $\triangleright$  Build Sparse Table over Blocks
11:    $num\_blocks \leftarrow \lfloor n/B \rfloor$ 
12:    $ST \leftarrow$  2D array of size  $\log(num\_blocks) \times num\_blocks$ 
13:   for  $j \leftarrow 0$  to  $num\_blocks-1$  do
14:      $end\_idx \leftarrow j \cdot B + B - 1$ 
15:      $ST[0][j] \leftarrow end\_idx - \lfloor \log_2(mask[end\_idx]) \rfloor$ 
16:     for  $k \leftarrow 1$  to  $\lfloor \log_2(num\_blocks) \rfloor$  do
17:       for  $j \leftarrow 0$  to  $num\_blocks - (1 \ll k)$  do
18:          $ST[k][j] \leftarrow \min(ST[k-1][j], ST[k-1][j + (1 \ll (k-1))])$ 
19:   function QUERY( $l, r$ )
20:     if  $r - l + 1 \leq B$  then
21:        $trunc\_mask \leftarrow mask[r] \& ((1 \ll (r - l + 1)) - 1)$ 
22:       return  $r - \lfloor \log_2(trunc\_mask) \rfloor$   $\triangleright$  Left partial block
23:    $left\_end \leftarrow (l/B) \cdot B + B - 1$ 
24:    $ans \leftarrow left\_end - \lfloor \log_2(mask[left\_end]) \rfloor$   $\triangleright$  Right partial block
25:    $ans \leftarrow \min(ans, r - \lfloor \log_2(mask[r]) \rfloor)$   $\triangleright$  Intermediate full blocks
26:    $x \leftarrow l/B + 1$ 
27:    $y \leftarrow r/B - 1$ 
28:   if  $x \leq y$  then
29:      $k \leftarrow \lfloor \log_2(y - x + 1) \rfloor$ 
30:      $mid\_ans \leftarrow \min(ST[k][x], ST[k][y - (1 \ll k) + 1])$ 
31:      $ans \leftarrow \min(ans, mid\_ans)$ 
32:   return  $ans$ 
```

8.2 DSU

8.2.1 Rank Facts

Use union by rank. Initially every vertex has rank 0. When roots of equal rank r are linked, the new root gets rank $r + 1$; otherwise the smaller-rank root is attached below the larger-rank root.

Hence ranks strictly increase along parent pointers.

Furthermore,

$$\#\{v : r(v) = r\} \leq \frac{n}{2^r},$$

because obtaining rank r requires having combined at least 2^r elements. In particular

$$\sum_v r(v) \leq \sum_{r \geq 0} \frac{nr}{2^r} = O(n).$$

8.2.2 Ackermann Levels

Define

$$A_0(x) = x + 1, \quad A_{k+1}(x) = A_k^{(x+1)}(1),$$

where $A_k^{(t)}$ denotes t -fold iteration.

Let

$$\alpha = \min\{k : A_k(1) > \log_2 n\}.$$

This differs from the usual definitions of $\alpha(n)$ by at most an additive constant. For a nonroot vertex x of positive rank define

$$\text{level}(x) = \max\{k : A_k(r(x)) \leq r(p(x))\}.$$

Since

$$r(p(x)) \leq \log n < A_\alpha(1) \leq A_\alpha(r(x)),$$

we always have

$$0 \leq \text{level}(x) < \alpha.$$

Once $k = \text{level}(x)$ is fixed, define

$$\text{index}(x) = \max\{i : A_k^{(i)}(r(x)) \leq r(p(x))\}.$$

Maximality of k implies

$$1 \leq \text{index}(x) \leq r(x).$$

Think of x as watching its parent's rank climb through the thresholds

$$A_k(r(x)), \quad A_k^{(2)}(r(x)), \quad A_k^{(3)}(r(x)), \dots$$

The index says how many thresholds have already been crossed; after all thresholds at level k are crossed, the level itself increases.

8.2.3 Potential

For a nonroot x of positive rank let

$$\phi(x) = (\alpha - \text{level}(x))r(x) - \text{index}(x).$$

For a root set

$$\phi(x) = \alpha r(x),$$

and put $\phi(x) = 0$ when $r(x) = 0$.

Because

$$\text{level}(x) \leq \alpha - 1, \quad \text{index}(x) \leq r(x),$$

we have

$$0 \leq \phi(x) \leq \alpha r(x).$$

Therefore the total amount of potential that can ever be introduced is

$$O\left(\alpha \sum_x r(x)\right) = O(n\alpha).$$

More importantly, changing a parent to an ancestor can only increase the parent's rank. Therefore either the level or the index increases, and the potential never increases under path compression.

8.2.4 Why a Find Costs $O(\alpha)$ Amortized

Consider a find path

$$x_0, x_1, \dots, x_t,$$

where x_t is the root.

Suppose x_i currently has level k and index j :

$$A_k^{(j)}(r(x_i)) \leq r(x_{i+1}) < A_k^{(j+1)}(r(x_i)).$$

Suppose there is a later vertex x_q , $q > i$, having the same level k . Then

$$r(x_q) \geq r(x_{i+1}) \geq A_k^{(j)}(r(x_i)),$$

and hence

$$r(x_t) \geq A_k(r(x_q)) \geq A_k^{(j+1)}(r(x_i)).$$

After path compression x_i becomes a child of x_t . Its parent has therefore crossed the next threshold. Thus either its index increases or its level increases. In either case

$$\phi(x_i)$$

drops by at least 1.

Consequently, for each level k , only the *last* vertex on the find path having that level might fail to decrease its potential. There are only

$$\alpha$$

levels. There is also at most one rank-0 vertex on the path, since ranks strictly increase. Therefore, if the path has length t ,

$$t \leq O(\alpha) + \text{potential decrease}.$$

Summing over all finds gives

$$O(m\alpha(n))$$

total time.

8.3 DSU with Potential

The DSU with Potential, maintains disjoint sets while tracking relative differences (potentials) between elements in the same set. The potentials must belong to a group G .

For any node x , the array $\mathbf{p}[x]$ maintains the potential difference from x to its current parent in the DSU tree, which we denote as $\Delta(x, \text{parent}(x))$.

1. Path Compression When querying the root of a node x , the algorithm recursively compresses the path. Suppose the recursive call finds the root R and the potential from the parent to the root, $\Delta(\text{parent}(x), R)$.

To update x to point directly to R , we compose the potential from x to its parent with the potential from its parent to R :

$$\Delta(x, R) = \Delta(x, \text{parent}(x)) \circ \Delta(\text{parent}(x), R)$$

2. Querying Potential To find the potential $\Delta(a, b)$ from node a to node b , we first verify they belong to the same set. Let R be their common root. The **find** operation gives us $v_a = \Delta(a, R)$ and $v_b = \Delta(b, R)$.

The path from a to b can be constructed by traversing from a up to R , and then down from R to b . Thus, the combined potential is:

$$\Delta(a, b) = \Delta(a, R) \circ \Delta(R, b) = v_a \circ v_b^{-1}$$

3. Edge Insertion When merging the sets containing a and b by adding a directed edge from a to b with a given potential P , we require that $\Delta(a, b) = P$.

Let R_a and R_b be the respective roots of a and b . Suppose we link R_a as a child of R_b . We need to compute the required weight $W = \Delta(R_a, R_b)$ for this new edge.

We know the full path $a \rightarrow R_a \rightarrow R_b \rightarrow b$ must equal the target potential P :

$$\Delta(a, R_a) \circ \Delta(R_a, R_b) \circ \Delta(R_b, b) = P$$

Substituting our known variables $v_a = \Delta(a, R_a)$, $W = \Delta(R_a, R_b)$, and $v_b^{-1} = \Delta(R_b, b)$:

$$v_a \circ W \circ v_b^{-1} = P$$

Thus:

$$W = v_a^{-1} \circ P \circ v_b$$

If the union-by-size heuristic determined that R_b should become a child of R_a , the weight of the reversed edge is the inverse W^{-1} .

8.4 Persistent Queue

A fully persistent queue allows us to push and pop elements from any historical state of the queue. By modeling the sequence of pushes as a tree, we can implement it by using [Algorithm 13](#).

1. State Each state in the persistent queue is represented as a node in a tree. A node u stores:

- v : The value inserted at this state.
- p : The parent node (the state of the queue before this element was pushed).
- depth : The depth of the node in the tree (the total number of pushes to reach this state).
- jmp : The jump pointer used for ancestor queries.
- fr : A pointer to the node representing the current front of the queue.

2. Push Operation When pushing a value onto a previous state p , we append a new child node u . The front pointer fr is inherited from p

We compute the new jump pointer $\text{jmp}[u]$ in $O(1)$ time using the combination rule from [Algorithm 13](#). We check:

$$\text{depth}[p] - \text{depth}[\text{jmp}[p]] = \text{depth}[\text{jmp}[p]] - \text{depth}[\text{jmp}[\text{jmp}[p]]]$$

If they are equal, we merge them: $\text{jmp}[u] \leftarrow \text{jmp}[\text{jmp}[p]]$. Otherwise, we reset the jump length: $\text{jmp}[u] \leftarrow p$.

3. Pop Operation To pop an element from state p , we must advance the front pointer fr to the next element in the queue. Because the queue's elements form a path from fr down to the current tail p , the next element to become the front is the child of fr that lies on the path to p .

This is a Level Ancestor query: we must find the ancestor of p whose depth is exactly $\text{depth}[fr] + 1$. We can find this in $O(\log n)$ time and then create a new state that copies p , but updates its fr pointer to this new ancestor.

4. Front Operation Querying the front of the queue simply requires reading the value stored at the node pointed to by fr .

Algorithm 10 Persistent Queue

```

1: function PUSH( $v, p$ )
2:   if  $p = \text{NULL}$  then
3:     return QueueNode( $v$ , parent: NULL, fr: nxt, jmp: nxt, depth: 0)
4:    $fr \leftarrow \text{nodes}[p].fr$ 
5:    $p_j \leftarrow \text{nodes}[p].jmp$ 
6:    $p_{jj} \leftarrow \text{nodes}[p_j].jmp$ 
7:   if  $\text{depth}[p] - \text{depth}[p_j] = \text{depth}[p_j] - \text{depth}[p_{jj}]$  then
8:      $jmp \leftarrow p_{jj}$ 
9:   else
10:     $jmp \leftarrow p$ 
11:     $depth \leftarrow \text{nodes}[p].depth + 1$ 
12:   return QueueNode( $v, p, fr, jmp, depth$ )
13: function LEVELANCESTOR( $p, target\_depth$ )
14:   while  $\text{depth}[p] > target\_depth$  do
15:     if  $\text{depth}[\text{jmp}[p]] \geq target\_depth$  then
16:        $p \leftarrow \text{jmp}[p]$ 
17:     else
18:        $p \leftarrow \text{parent}[p]$ 
19:   return  $p$ 
20: function POP( $p$ )
21:    $new\_node \leftarrow \text{nodes}[p].clone()$ 
22:    $target\_depth \leftarrow \text{nodes}[\text{nodes}[p].fr].depth + 1$ 
23:    $new\_node.fr \leftarrow \text{LEVELANCESTOR}(p, target\_depth)$ 
24:   return  $new\_node$ 

```

8.5 Online Suffix Max

To answer suffix maximum queries $\max(A[i \dots n - 1])$ in a setting where elements are appended, we can combine a monotonic stack with a DSU.

1. Monotonic Stack When a new element x is appended at index p , it invalidates any previous elements smaller than it. We maintain a stack of indices corresponding to a strictly decreasing subsequence of the array. As x is pushed, we pop all indices j from the stack where $A[j] \leq x$.

2. DSU Any index whose suffix maximum was previously $A[j]$ now shares a suffix maximum of x . For every popped index j , we union its set into p 's set. Finally, we push p onto the stack.

3. Queries To find the maximum in the suffix starting at index i , we simply query the DSU for the root of i .

8.6 Mo's Algorithm

Suppose we have a static array of length n and

$$q$$

offline range queries

$$[L_i, R_i).$$

Assume we can maintain the answer while moving either endpoint by one position, with $O(1)$ work per move.

Mo's algorithm orders the queries so consecutive intervals differ only a small amount.

Choose a block size B . Sort queries by

$$\left(\left\lfloor \frac{L}{B} \right\rfloor, R \right),$$

except that the direction of R alternates between blocks:

$$\begin{cases} R \text{ increasing,} & \lfloor L/B \rfloor \text{ even,} \\ R \text{ decreasing,} & \lfloor L/B \rfloor \text{ odd.} \end{cases}$$

Algorithm 11 Mo's Algorithm

Input: Array of length n , range queries $[L_i, R_i)$

1: choose block size B

2: sort queries by

$$\left(\lfloor L/B \rfloor, (-1)^{\lfloor L/B \rfloor} R \right)$$

with the second coordinate interpreted as alternating direction

3: $L \leftarrow 0, R \leftarrow 0$

4: **for** query $[a, b)$ in this order **do**

5: **while** $L > a$ **do**

6: $L \leftarrow L - 1$; $\text{ADD}(L)$

7: **while** $R < b$ **do**

8: $\text{ADD}(R)$; $R \leftarrow R + 1$

9: **while** $L < a$ **do**

10: $\text{REMOVE}(L)$; $L \leftarrow L + 1$

11: **while** $R > b$ **do**

12: $R \leftarrow R - 1$; $\text{REMOVE}(R)$

13: record current answer

Theorem 143 (Mo Movement Bound). *With block size B , the total number of endpoint moves is*

$$O\left(qB + \frac{n^2}{B} + n\right).$$

Choosing

$$B \asymp \frac{n}{\sqrt{q}}$$

gives

$$O(n\sqrt{q} + q)$$

total work when each endpoint update costs $O(1)$.

Proof. First consider the left endpoint.

Inside one block,

L

always lies in an interval of width B . Thus between consecutive queries in that block it moves by at most B . Over q queries,

$$\text{total movement of } L = O(qB + n).$$

Now consider R . Within one L -block, queries are sorted monotonically by R . Thus R traverses the array at most once inside that block.

There are at most

$$\left\lceil \frac{n}{B} \right\rceil$$

blocks. Therefore

$$\text{total movement of } R = O\left(n \frac{n}{B} + n\right) = O\left(\frac{n^2}{B} + n\right).$$

The odd/even reversal means that when one block finishes near one end of the array, the next begins from that same end rather than forcing an unnecessary reset.

Combining the two bounds,

$$O\left(qB + \frac{n^2}{B} + n\right).$$

Balance the two main terms:

$$qB = \frac{n^2}{B},$$

so

$$B = \frac{n}{\sqrt{q}}.$$

Then

$$qB = \frac{n^2}{B} = n\sqrt{q}.$$

□

8.7 A Fast Succinct Hash Table

We first assume access to the random hash functions used below; the next subsection explains how to replace fully random functions by a small explicit random data structure.

We work on a word RAM with word size

$$w = \Theta(\log n),$$

and consider a fixed polynomial-length sequence of operations independent of the random choices of the data structure. We maintain a capacity parameter

$$N = \Theta(n)$$

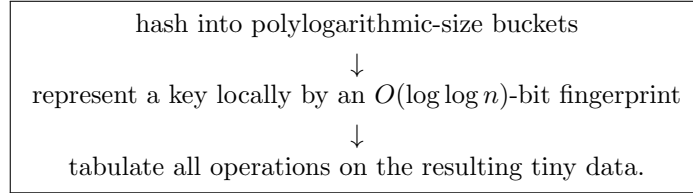
throughout an epoch.

The goal is an exact dynamic dictionary using

$$nw + O(n \log \log n)$$

bits, with constant-time queries and, with high probability, constant time per update after deamortization.

The construction has three layers:



The only updates that can occasionally be expensive are local fingerprint rebuilds. A final buffering step deamortizes these.

8.7.1 Small Buckets

Put

$$L = \lceil (\log N)^3 \rceil$$

and hash the keys uniformly into

$$B = \left\lceil \frac{N}{L} \right\rceil$$

buckets.

We maintain $n \leq N$. For any fixed bucket, its load X is binomial with mean

$$\mu = \frac{n}{B} \leq L.$$

Thus it exceeds $4L$ with superpolynomially small probability by a Chernoff bound.

At each time in a fixed update sequence, the current set of keys is fixed independently of the random top-level hash function, so the same argument applies. A union bound over $N^{O(1)}$ times still gives

$$N^{O(1)} e^{-\Omega(\log^3 N)} = N^{-\omega(1)}.$$

Henceforth condition on this event. Every bucket contains

$$O(\log^3 N)$$

keys throughout the epoch.

8.7.2 Storing the Full Keys Succinctly

The actual w -bit keys must be stored somewhere, but we do not want a word-sized pointer or other $\Theta(\log n)$ -bit overhead for every key.

Within each bucket, divide the keys into blocks of

$$s = \lceil (\log N)^2 \rceil$$

keys. All blocks except possibly the last are full.

A bucket of size $b \leq 4L$ therefore uses at most

$$\left\lceil \frac{b}{s} \right\rceil = O(\log N)$$

blocks. Give every bucket a directory containing the addresses of these blocks.

This representation has very little wasted space. Across all buckets, the number of allocated key slots is at most

$$\sum_{i=1}^B s \left\lceil \frac{|B_i|}{s} \right\rceil \leq \sum_i |B_i| + Bs = n + O\left(\frac{N}{\log N}\right).$$

Since $w = \Theta(\log N)$, the full keys therefore occupy

$$\boxed{nw + O(N)}$$

bits.

Each bucket directory has $O(\log N)$ block addresses, each using $O(\log N)$ bits. Hence all directories together use

$$O\left(\frac{N}{\log^3 N} \log^2 N\right) = O\left(\frac{N}{\log N}\right)$$

bits.

A position within one bucket consists of

$$\text{block number} \in [O(\log N)]$$

and

$$\text{offset} \in [(\log N)^2],$$

so it requires only

$$\boxed{O(\log \log N)}$$

bits.

Insertion appends to the last block, allocating a new block if necessary. Deletion replaces the deleted key by the last key in its bucket and frees the last block if it becomes empty. Thus the key storage itself supports every local update in $O(1)$ time.

The only remaining task is to locate the local position of a queried key.

8.7.3 Local Fingerprints

For each bucket choose a random fingerprint function

$$f : U \rightarrow [M], \quad M = L^3 = \Theta(\log^9 N).$$

A fingerprint therefore occupies

$$\log M = O(\log \log N)$$

bits.

We maintain the invariant that the fingerprints of the keys currently in a bucket are distinct.

For a bucket containing $b \leq 4L$ keys, a fresh random fingerprint function fails this condition with probability at most

$$\binom{b}{2} \frac{1}{M} \leq \frac{8L^2}{L^3} = \frac{8}{L}.$$

Thus a fresh function is collision-free with probability at least

$$1 - \frac{8}{L}.$$

Whenever a collision occurs, resample the local fingerprint function until all keys in that bucket have distinct fingerprints and rebuild its local metadata.

For a query key x , we compute its bucket and $f(x)$. If the local dictionary contains this fingerprint, it returns a candidate position. We then compare the full w -bit key stored at that position with x .

8.7.4 A Constant-Time Dictionary Inside a Bucket

Each bucket stores the map

$$f(x) \mapsto \text{position}(x).$$

Both sides use

$$O(\log \log N)$$

bits.

Store this map in a B-tree of fanout

$$F = \lfloor \sqrt{\log N} \rfloor.$$

A node contains $O(F)$ fingerprints, local positions, and local child indices, and therefore has

$$O(F \log \log N) = O(\sqrt{\log N} \log \log N) = o(\log N)$$

bits.

Consequently there are only

$$2^{o(\log N)} = N^{o(1)}$$

possible node states. The input to any elementary node operation also has only $O(\log \log N)$ bits, so all possible node operations—searching within a node, inserting or deleting an entry, splitting, merging, and choosing a child—can be stored in one global

$$N^{o(1)}$$

lookup table.

A bucket contains at most $4L = O(\log^3 N)$ entries. Hence the B-tree depth is

$$O(\log_F(\log^3 N)) = O(1),$$

since

$$\log_F(\log^3 N) = \frac{3 \log \log N + O(1)}{\frac{1}{2} \log \log N + O(1)} = 6 + o(1).$$

Thus lookup, insertion, deletion, splitting, and merging in the local dictionary all take

$$\boxed{O(1)}$$

time.

There is one dictionary entry per stored key, each using $O(\log \log N)$ bits. B-tree node overhead and bucket roots are smaller. Hence all local dictionaries use

$$\boxed{O(n \log \log N)}$$

bits.

8.7.5 Cost of Fingerprint Rebuilding

An ordinary update costs $O(1)$. The only expensive event is an insertion whose fingerprint collides with an existing fingerprint, forcing a rebuild of its $O(L)$ -key bucket.

We first quantify how often this happens.

Suppose the current bucket contains $b \leq 4L$ keys and its current random function is collision-free on those keys. For a new key whose fingerprint has not previously been exposed under this function, its value is uniform and independent of the b current values. Hence

$$\Pr[\text{collision}] = \frac{b}{M} \leq \frac{4L}{L^3} = \frac{4}{L^2}.$$

The same bound follows for a fixed oblivious update sequence by deferred decisions even when keys are deleted and later reinserted: keys that coexisted with x are already known not to collide with it, while fingerprints first exposed while x was absent are uniform subject only to avoiding the $O(L)$ fingerprints then present. This changes the denominator from M to $M - O(L)$, and hence, for sufficiently large N ,

$$\Pr[\text{collision} \mid \text{entire past}] \leq \frac{5}{L^2}.$$

We will use

$$p := \frac{5}{L^2}.$$

If a collision occurs, a fresh fingerprint function on the whole bucket fails to be injective with probability at most

$$\frac{8}{L}.$$

Thus the number G of trials needed to obtain a good function satisfies

$$\Pr[G > r] \leq \left(\frac{8}{L}\right)^r.$$

Taking

$$r = \lceil \log N \rceil,$$

$$\Pr[G > r] \leq \left(\frac{8}{\log^3 N}\right)^{\log N} = \exp(-\Omega(\log N \log \log N)) = N^{-\Omega(\log \log N)}.$$

A union bound over any polynomial number of rebuilds preserves a $N^{-\omega(1)}$ failure probability.

Conditioned on $G \leq \log N$, a rebuild costs

$$O(L \log N) = O(\log^4 N)$$

work.

8.7.6 Deamortizing Rebuilds

An individual rebuild still costs polylogarithmic time. We now turn this into constant time per external operation.

Set

$$q = \lfloor N^{1/4} \rfloor.$$

Maintain two deterministic $\sqrt{N}$ -ary tries, each holding at most q pending updates.

The *growing buffer* receives new updates. The *shrinking buffer* contains the previous batch, which is being applied to the hash table. Queries first inspect the growing buffer, then the shrinking buffer, and then the main table, so pending updates are immediately visible.

Every external operation performs a fixed constant amount of work on the shrinking batch. When the growing buffer reaches q updates, we swap the two buffers.

It remains only to prove that a batch of q updates requires $O(q)$ total work with high probability.

Let

$$I_j = \mathbf{1}[\text{the } j\text{-th update causes a fingerprint collision}], \quad C = \sum_{j=1}^q I_j.$$

Let $\mathcal{F}_{j-1}$ contain the entire history before update j . From the preceding calculation,

$$\Pr[I_j = 1 \mid \mathcal{F}_{j-1}] \leq p = \frac{5}{L^2}.$$

Although the I_j 's need not be independent, the usual Chernoff proof still applies. Indeed, for every $\lambda \geq 0$,

$$\begin{aligned} \mathbb{E}[e^{\lambda I_j} \mid \mathcal{F}_{j-1}] &= 1 + (e^\lambda - 1) \Pr[I_j = 1 \mid \mathcal{F}_{j-1}] \\ &\leq 1 + p(e^\lambda - 1). \end{aligned}$$

Therefore, iterating conditional expectation,

$$\mathbb{E}e^{\lambda C} \leq (1 + p(e^\lambda - 1))^q.$$

This is exactly the MGF bound for a $\text{Bin}(q, p)$ random variable. Hence all the standard binomial Chernoff upper-tail bounds remain valid.

Put

$$\mu = qp = \frac{5q}{L^2} = \frac{5q}{\log^6 N}.$$

Taking the standard $\delta = 1$ Chernoff bound,

$$\Pr[C \geq 2\mu] \leq \left(\frac{e}{4}\right)^\mu \leq e^{-\mu/3}.$$

Since $q = N^{1/4}$,

$$e^{-\mu/3} = \exp\left(-\Omega\left(\frac{N^{1/4}}{\log^6 N}\right)\right) = N^{-\omega(1)}.$$

Thus, with probability $1 - N^{-\omega(1)}$,

$$\boxed{C \leq \frac{10q}{\log^6 N}}.$$

We already showed that, with probability $1 - N^{-\omega(1)}$, every fingerprint rebuild requires at most $\log N$ trials. Since one trial rebuilds an $O(L) = O(\log^3 N)$ -key bucket, every collision costs at most

$$O(L \log N) = O(\log^4 N)$$

work. Therefore all rebuilds in the batch cost

$$O\left(\frac{q}{\log^6 N} \cdot \log^4 N\right) = O\left(\frac{q}{\log^2 N}\right) = O(q).$$

Together with the $O(q)$ ordinary update work, the entire batch costs $O(q)$ with probability $1 - N^{-\omega(1)}$.

Choosing the constant amount of draining work per external operation larger than this hidden constant ensures that the shrinking buffer is empty before the growing buffer fills.

A union bound over any polynomial number of batches still gives high probability.

8.7.7 The Update Buffers Are Succinct

A $\sqrt{N}$ -ary trie reads

$$\frac{1}{2} \log N$$

bits of the key per level. Since $w = \Theta(\log N)$, its depth is $O(1)$.

A deliberately crude representation already suffices. There are $O(q)$ nodes and each node may contain a direct-access array of $\sqrt{N}$ child pointers, each of $O(\log N)$ bits. Thus one buffer uses at most

$$O(q\sqrt{N} \log N) = O(N^{3/4} \log N) = o(N)$$

bits.

The two buffers therefore do not affect the $nw + O(n \log \log n)$ space bound.

8.7.8 Dynamic Resizing

Finally, N must track the current number of keys.

Use the usual constant-factor hysteresis so that

$$N = \Theta(n).$$

When a resize is triggered, initialize a new table with capacity $N' = \Theta(n)$ and migrate a constant number of old keys per external operation.

Crucially, when a key is inserted into the new table, immediately delete its full representation from the old table. Thus a key is never stored in both tables.

If the old and new tables currently contain n_1, n_2 keys, then

$$n_1 + n_2 = n,$$

and their full-key payload occupies

$$n_1 w + n_2 w = nw$$

bits, apart from the $O(n)$ block slack already accounted for.

Both tables together contain only $O(n)$ keys, so their combined metadata uses

$$O(n \log \log n)$$

bits.

Queries inspect both tables during migration. Since resizing is triggered only after a constant-factor change in n , there are $\Theta(n)$ external operations available before another resize can be necessary, enough to finish the migration at constant additional work per operation.

8.7.9 Summary

We obtain, throughout the execution,

$$\boxed{nw + O(n \log \log n)}$$

bits of space.

Queries take worst-case $O(1)$ time. Updates take $O(1)$ external work, and with probability $1 - n^{-\omega(1)}$ every background batch finishes before its buffer is needed again.

8.8 Simulating Fully Random Hashing

A truly random function

$$h : U \rightarrow V$$

cannot be stored explicitly when U is large. For the hash-table applications above, however, we only need a function to look fully random on small fixed subsets of U .

The following construction gives exactly that.

8.8.1 Private Random Cells

Fix

$$S \subseteq U, \quad |S| \leq n^{0.8}.$$

Choose a sufficiently large constant c . Independently for $j = 1, \dots, c$, choose a universal hash function

$$g_j : U \rightarrow [n^{0.9}]$$

and an array

$$A_j[1, \dots, n^{0.9}]$$

of independent uniform elements of the additive group V .

Define

$$h(x) = \sum_{j=1}^c A_j[g_j(x)].$$

Lemma 144 (Full Randomness on a Fixed Small Set). *For every constant K , c can be chosen as a constant depending only on K such that, for every fixed $S \subseteq U$ with $|S| \leq n^{0.8}$,*

$$\Pr[h|_S \text{ is a fully random function}] \geq 1 - n^{-K}.$$

Proof. For $x \in S$, call the cell $A_j[g_j(x)]$ *private* if no other $y \in S$ accesses it.

For fixed x, j , universality and a union bound give

$$\begin{aligned} \Pr[A_j[g_j(x)] \text{ is not private}] &\leq \sum_{y \in S \setminus \{x\}} \Pr[g_j(x) = g_j(y)] \\ &\leq \frac{|S| - 1}{n^{0.9}} \\ &\leq n^{-0.1}. \end{aligned}$$

The functions $g_1, \dots, g_c$ are independent, so

$$\Pr[x \text{ has no private cell}] \leq n^{-0.1c}.$$

Union-bounding over the at most $n^{0.8}$ elements of S ,

$$\Pr[\exists x \in S \text{ with no private cell}] \leq n^{0.8-0.1c}.$$

Choosing

$$c \geq 10K + 8$$

makes this at most n^{-K} .

Condition now on the g_j 's and on every $x \in S$ having a private cell. Choose one private cell

$$A_{j_x}[g_{j_x}(x)]$$

for each x .

These selected cells are all distinct. Moreover, the selected cell for x is not used in the hash value of any other key in S .

Condition on every array entry except these selected cells. Then, for every $x \in S$,

$$h(x) = A_{j_x}[g_{j_x}(x)] + c_x,$$

where c_x is now fixed.

The selected array entries are mutually independent and uniform in V . Adding fixed constants preserves uniformity, so

$$(h(x))_{x \in S}$$

is a vector of mutually independent uniform elements of V . Thus $h|_S$ is exactly a fully random function. $\square$

The construction uses

$$\tilde{O}(n^{0.9})$$

random bits when U, V have polynomial size, and evaluates h using a constant number of hash evaluations and table lookups.

8.8.2 Splitting a Large Set into Small Fixed Sets

The lemma only applies to fixed sets of size at most $n^{0.8}$. To use it for n stored keys, first split those keys into much smaller buckets.

Choose an independent function

$$h_1 : U \rightarrow [B], \quad B = \lceil \sqrt{n} \rceil,$$

using the preceding construction.

We first prove that every bucket has

$$(1 + o(1))\sqrt{n}$$

keys.

For the analysis only, partition the n stored keys into

$$G = \lceil n^{0.2} \rceil$$

fixed groups

$$S_1, \dots, S_G, \quad |S_j| \leq n^{0.8}.$$

Choose the constant c in [Lemma 144](#) large enough that the failure probability for each group is at most n^{-10} . A union bound gives

$$\Pr[h_1 \text{ is fully random on every } S_j] \geq 1 - Gn^{-10} = 1 - n^{-\Omega(1)}.$$

Condition on this event.

For destination bucket i and group S_j , let

$$X_{i,j} = |\{x \in S_j : h_1(x) = i\}|.$$

Then

$$X_{i,j} \sim \text{Bin}\left(|S_j|, \frac{1}{B}\right),$$

with mean

$$\mu_{i,j} = \frac{|S_j|}{B} \leq n^{0.3}.$$

Set

$$a = n^{0.15} \log n.$$

The Bernstein inequality gives

$$\Pr[|X_{i,j} - \mu_{i,j}| \geq a] \leq 2 \exp\left(-\frac{a^2}{2(\mu_{i,j} + a/3)}\right).$$

For sufficiently large n ,

$$a \leq n^{0.3},$$

so

$$2(\mu_{i,j} + a/3) \leq 3n^{0.3}.$$

Hence

$$\begin{aligned}\Pr[|X_{i,j} - \mu_{i,j}| \geq a] &\leq 2 \exp\left(-\frac{n^{0.3} \log^2 n}{3n^{0.3}}\right) \\ &= 2e^{-\log^2 n/3}.\end{aligned}$$

There are at most

$$BG \leq 2n^{0.7}$$

pairs (i, j) , so a union bound gives

$$\Pr[\exists i, j : |X_{i,j} - \mu_{i,j}| > a] \leq 4n^{0.7} e^{-\log^2 n/3} = n^{-\omega(1)}.$$

Therefore, simultaneously for every bucket i ,

$$\begin{aligned}|B_i| &= \sum_j X_{i,j} \\ &= \sum_j \mu_{i,j} + O(Ga) \\ &= \frac{n}{B} + O(n^{0.35} \log n) \\ &= \sqrt{n} + o(\sqrt{n}).\end{aligned}$$

Thus

$$\boxed{\max_i |B_i| = (1 + o(1))\sqrt{n}}$$

with high probability.

8.8.3 Randomize After Splitting

Now choose a second, independent function h_2 from the private-cell construction.

Condition on h_1 and on the event that every top-level bucket has $(1 + o(1))\sqrt{n}$ keys. The sets

$$B_1, \dots, B_B$$

are now fixed, and for sufficiently large n ,

$$|B_i| < n^{0.8}.$$

By [Lemma 144](#), for any desired constant K , we may choose the constant number of tables in h_2 so that

$$\Pr[h_2|_{B_i} \text{ is not fully random}] \leq n^{-K}$$

for each fixed i .

There are only $O(\sqrt{n})$ buckets. Hence

$$\Pr[\exists i : h_2|_{B_i} \text{ is not fully random}] \leq O(\sqrt{n}) n^{-K}.$$

Taking K sufficiently large gives high probability that

$$\boxed{h_2 \text{ is fully random on every bucket simultaneously.}}$$

9 Trees

9.1 LCA

1. Pre-Order Reduction Let $\text{dfs}[0 \dots n-1]$ be the sequence of nodes visited in a standard pre-order DFS traversal. Let $\text{pos}[u]$ be the pre-order index of node u , such that $\text{dfs}[\text{pos}[u]] = u$. Let $p[u]$ be the parent of node u , and let $\text{depth}[u]$ be its distance from the root.

We construct our $O(n)$ RMQ from [Algorithm 9](#) over an implicit array Z of size n , defined by the depth of the parent of the node at each pre-order step:

$$Z[i] = \text{depth}[p[\text{dfs}[i]]]$$

2. Queries To find $\text{LCA}(a, b)$, we first ensure that a is visited before b by swapping them if $\text{pos}[a] > \text{pos}[b]$. If $a = b$, the LCA is simply a . Otherwise, we query the RMQ for the index m that minimizes Z in the range $[\text{pos}[a] + 1, \text{pos}[b]]$. The LCA is exactly the parent of the node at that minimum index: $p[\text{dfs}[m]]$.

3. Correctness There are two cases:

- **Case 1 (a is an ancestor of b):** The pre-order interval $[\text{pos}[a] + 1, \text{pos}[b]]$ contains only proper descendants of a . The node in this interval with the shallowest parent is the direct child of a that lies on the path to b . The parent of this child is a .
- **Case 2 (a and b are in separate branches):** Let $L = \text{LCA}(a, b)$. After the DFS visits a and finishes its subtree, it backtracks to L and eventually descends into the branch containing b . Let v be the direct child of L that roots the subtree containing b . The node v is visited after a and before b , so $\text{pos}[v] \in [\text{pos}[a] + 1, \text{pos}[b]]$. Its parent is L . Because every other node visited in this interval is a descendant of L , their parents have a depth of at least $\text{depth}[L]$. Thus, the RMQ identifies v , and returns its parent $p[v] = L$.

Algorithm 12 LCA

```

1: function INITLCA( $n, p, \text{dfs}, \text{pos}, \text{depth}$ )
    $\triangleright$  Define a closure to compare the parent depths of two pre-order indices
2:    $\text{cmp} \leftarrow \text{Closure}(i, j)$  returning  $\text{depth}[p[\text{dfs}[i]]] < \text{depth}[p[\text{dfs}[j]]]$ 
    $\triangleright$  Initialize the RMQ data structure using this closure
3:    $\text{rmq} \leftarrow \text{RMQ\_Init}(n, \text{cmp})$ 
4:   return  $\text{LCA\_Struct}\{p, \text{dfs}, \text{pos}, \text{rmq}\}$ 
5: function QUERYLCA( $a, b$ )
6:   if  $a = b$  then
7:     return  $a$ 
    $\triangleright$  Order by entry time
8:    $l \leftarrow \min(\text{pos}[a], \text{pos}[b])$ 
9:    $r \leftarrow \max(\text{pos}[a], \text{pos}[b])$ 
10:   $m \leftarrow \text{rmq.query}(l + 1, r)$ 
11:  return  $p[\text{dfs}[m]]$ 

```

9.2 $O(n)$ Binary Lifting

Binary Lifting usually requires $O(n \log n)$ time and space to precompute an ancestor table, but using a modified method we can do it in $O(n)$ preprocessing and space.

1. Preprocessing We define $\text{jmp}[v]$ as a pointer to an ancestor of v . The idea is to grow the jump distances in powers of two as we ascend.

During a pre-order traversal, consider a node v with parent p . We look at the jump from the parent $p \rightarrow \text{jmp}[p]$, and the subsequent jump $\text{jmp}[p] \rightarrow \text{jmp}[\text{jmp}[p]]$. If these two jumps cover the exact same change in depth, we combine them. The new jump from v covers the edge (v, p) , the first jump of length L , and the second jump of length L . Thus, the new jump pointer points to $\text{jmp}[\text{jmp}[p]]$ and covers a distance of $1 + 2L$.

If the two preceding jumps do not match in length, we set $\text{jmp}[v] = p$, resetting the jump length to 1.

2. Querying The algorithm has two phases:

1. **Equate Depths:** If u and v are at different depths, we identify the deeper node and use our `depth_jmp` function to lift it exactly to the depth of the shallower node, this works basically the same as the LCA function. If the nodes are now identical, we have found the LCA.
2. **Simultaneous Ascent:** If the nodes are distinct but at the same depth, we ascend them simultaneously. We evaluate the jump pointers $\text{jmp}[u]$ and $\text{jmp}[v]$:
 - If $\text{jmp}[u] \neq \text{jmp}[v]$, the jump lands strictly below the LCA. We can safely take this maximal jump: $u \leftarrow \text{jmp}[u]$ and $v \leftarrow \text{jmp}[v]$.
 - If $\text{jmp}[u] = \text{jmp}[v]$, the jump has either hit the LCA exactly or overshoot it into a higher common ancestor. Because we cannot distinguish between these two cases, we take a single step up to the parents: $u \leftarrow p[u]$ and $v \leftarrow p[v]$.

We repeat this until $u = v$, at which point u is the LCA.

3. Complexity Let $L(u) = \text{depth}[u] - \text{depth}[\text{jmp}[u]]$. By construction, $L(u)$ is always of the form $2^k - 1$ for some integer $k \geq 1$. We call k the rank of the jump. A jump of rank k implicitly spans a tree of $2^k - 1$ edges.

Lemma 145 (Monotonicity of Jump Sequences). *For any node u , let $J(u) = [L_1, L_2, \dots, L_m]$ be the sequence of jump lengths encountered by repeatedly following the **jmp** pointers from u to the root. The sequence $J(u)$ is monotonically non-decreasing.*

Proof. The sequence of jumps from a node depends entirely on its depth d . Let J_d denote the sequence of jump lengths from a node at depth d .

By induction, all jump lengths are drawn from the set of skew-binary weights: $\mathcal{W} = \{2^k - 1 \mid k \geq 1\}$.

We will prove by induction on depth d that:

$$J_d \text{ satisfies } L_1 \leq L_2 < L_3 < \dots < L_m.$$

Base Case ($d = 1$):

Trivial.

Inductive Step:

Assume J_d satisfies the invariant. We consider J_{d+1} :

- **Case A:** J_d does *not* start with identical elements.

By the invariant, we have $L_1 < L_2 < L_3 < \dots$

The algorithm prepends a jump of 1, making $J_{d+1} = [1, L_1, L_2, \dots]$.

Because all elements are ≥ 1 , we have the invariant.

- **Case B:** J_d starts with two identical elements.

By the invariant, we have: $J_d = [L, L, L_3, L_4, \dots]$ with $L < L_3 < L_4 \dots$

The algorithm merges the first two jumps, replacing them with $2L+1$, making $J_{d+1} = [2L+1, L_3, L_4, \dots]$.

Because $L \in \mathcal{W}$, the next available weight in $\mathcal{W}$ is $2L+1$. Since $L_3 \in \mathcal{W}$ and $L_3 > L$, we have $L_3 \geq 2L+1$. Therefore, the new sequence satisfies the invariant again.

Thus J_d is always monotonically non-decreasing for any depth d . $\square$

By the lemma, $J(u)$ can contain at most two jumps of any length $2^k - 1$. Thus the entire sequence $J(u)$ consists of at most $2\lceil \log_2 n \rceil$ jumps.

During a query, we attempt to follow $J(u)$ to cover a target distance. Every operation falls into one of two categories:

1. Successful Jumps:

When we take $\text{jmp}[u]$, we consume one of the elements of $J(u)$, so there are at most $O(\log n)$ of these operations.

2. Penalty Steps:

If taking the jump L would overshoot our target we cannot consume it. Instead, we take a penalty step to the parent p . Because $L > 1$ was built by merging a step of 1 and two jumps of size $(L - 1)/2$, stepping to the parent "unpacks" L . The new sequence from the parent, $J(p)$, replaces L with two smaller jumps: $[(L - 1)/2, (L - 1)/2, \dots]$. We only unpack a jump of size L when it is greater than the remaining distance. Because the remaining distance strictly decreases, we will never need to unpack a jump of size L more than once across the query. Thus, the total number of penalty steps is bounded by the number of unique jump sizes, which is $\leq \log_2 n$.

Algorithm 13 Binary Lifting

```

1: function BUILDJUMPTABLE( $n, par, dfs, depth$ )
2:    $jmp \leftarrow$  array of size  $n$ 
3:   for each  $v$  in  $dfs$  do
4:      $p \leftarrow par[v]$ 
5:     if  $v = p$  then
6:        $jmp[v] \leftarrow v$ 
7:     else
8:        $p_j \leftarrow jmp[p]$ 
9:        $p_{jj} \leftarrow jmp[p_j]$ 
10:      if  $depth[p] - depth[p_j] = depth[p_j] - depth[p_{jj}]$  then
11:         $jmp[v] \leftarrow p_{jj}$ 
12:      else
13:         $jmp[v] \leftarrow p$ 
14:   return  $jmp$ 
15: function QUERYLCA( $u, v, par, jmp, depth$ )
16:   if  $depth[u] > depth[v]$  then
17:     Swap  $u$  and  $v$ 
18:    $v \leftarrow \text{DEPTHJUMP}(v, depth[u], par, jmp, depth)$ 
19:   while  $u \neq v$  do
20:     if  $jmp[u] = jmp[v]$  then
21:        $u \leftarrow par[u]$ 
22:        $v \leftarrow par[v]$ 
23:     else
24:        $u \leftarrow jmp[u]$ 
25:        $v \leftarrow jmp[v]$ 
26:   return  $u$ 

```

$\triangleright$ We would overshoot the LCA.

9.3 Virtual Tree

Let T be a rooted tree. Suppose a query specifies a set

$$S = \{v_1, \dots, v_k\}$$

of important vertices.

A *virtual tree* is the smallest compressed tree containing all vertices of S while preserving their ancestor relationships and distances in T .

Preprocess:

- DFS entry times $\text{tin}(v)$;
- depths or root distances;
- an LCA data structure.

Construction Sort the query vertices by DFS order:

$$v_1, \dots, v_k.$$

Add

$$\text{LCA}(v_i, v_{i+1}) \quad (1 \leq i < k).$$

Sort and deduplicate the resulting set U by DFS order.

Now scan U with a stack. Before inserting v , pop until the stack top is an ancestor of v . Connect v to that stack top and push v .

Algorithm 14 Virtual Tree

Input: Query vertices S

Output: Virtual tree on S and necessary LCAs

```

1: sort  $S$  by tin
2:  $U \leftarrow S$ 
3: for  $i = 1, \dots, |S| - 1$  do
4:    $\lfloor$  add  $\text{LCA}(S_i, S_{i+1})$  to  $U$ 
5: sort and deduplicate  $U$  by tin
6: stack  $\leftarrow$  empty
7: for  $v \in U$  in DFS order do
8:   while stack nonempty and top is not an ancestor of  $v$  do
9:      $\lfloor$  pop stack
10:  if stack nonempty then
11:     $\lfloor$  add virtual edge (top,  $v$ )
12:     $\lfloor$  assign its weight
13:     $\lfloor$   $\text{dist}_T(\text{top}, v)$ 
13:  $\lfloor$  push  $v$ 
```

Lemma 146. *The set U contains every branching vertex required by the minimal subtree spanning S .*

Proof. Any branching vertex x of the spanning subtree is the LCA of two marked vertices lying in two different child subtrees of x .

Take the last marked vertex in DFS order from one such child subtree and the first marked vertex from the next nonempty child subtree. They are consecutive among the marked vertices lying across that boundary, and their LCA is exactly x .

Thus every necessary branching LCA occurs among LCAs of consecutive marked vertices. $\square$

Lemma 147. *After sorting U , the parent of a vertex v in the virtual tree is the deepest earlier vertex of U that is an ancestor of v .*

Proof. DFS order makes the descendants of every vertex contiguous. The stack contains exactly the chain of currently open ancestors. Popping until the top contains v therefore leaves precisely its deepest ancestor in U . $\square$

There are only

$$k - 1$$

added LCAs, so

$$|U| \leq 2k - 1.$$

Therefore construction takes

$$O(k \log k)$$

time with $O(1)$ LCA queries, or

$$O(k \log k + k \log n)$$

with binary-lifting LCA.

9.4 XOR-Linked Traversal

XOR-Linked Traversal is a method to traverse a tree bottom-up iteratively with better constant factor.

We store:

- $d[u]$: The degree of node u .
- $p[u]$: The XOR sum of the indices of all neighbors of node u . Initially, $p[u] = \bigoplus_{v \in \text{neighbors}(u)} v$.

If a node u is a leaf, its degree is 1. Therefore, $p[u]$ is simply the index of its only neighbor, v . We can process the edge (u, v) , and then delete the edge from the graph using $p[v] \leftarrow p[v] \oplus u$ and decrementing its degree.

If decrementing v 's degree makes it a new leaf, we set $u \leftarrow v$ and continue.

Algorithm 15 XOR-Linked Traversal

Input: Number of nodes N , Initial XOR-sum array p , Initial degree array d

Output: Updated arrays p and d , processing nodes bottom-up

```

1: for  $i \leftarrow 0$  to  $N - 1$  do
2:    $u \leftarrow i$ 
3:   while  $d[u] = 1$  do
4:      $d[u] \leftarrow 0$ 
5:      $v \leftarrow p[u]$ 
6:     Process( $u, v$ )
7:      $p[v] \leftarrow p[v] \oplus u$ 
8:      $d[v] \leftarrow d[v] - 1$ 
9:    $u \leftarrow v$ 
10: return  $p, d$ 

```

9.5 DFS

The algorithm constructs a DFS order, alongside subtree sizes and depths, with a better constant factor.

Algorithm 16 DFS

Input: Number of nodes N , Parent array p , Degree array d

Output: Arrays $p, ss, pos, dfs, depth$

```

1: Initialize  $ss \leftarrow [1, \dots, 1]$  of size  $N$ 
2: Initialize  $dfs \leftarrow [0, \dots, 0]$  of size  $N$ 
3: Initialize  $depth \leftarrow [0, \dots, 0]$  of size  $N$ 
4:  $idx \leftarrow N - 1$ 

5: for all edge  $(u, v)$  processed via XOR-Linked Traversal do
6:    $ss[v] \leftarrow ss[v] + ss[u]$ 
7:    $dfs[idx] \leftarrow u$ 
8:    $idx \leftarrow idx - 1$ 

9:  $pos \leftarrow ss$ 
10: for  $i \leftarrow 1$  to  $N - 1$  do
11:    $u \leftarrow dfs[i]$ 
12:    $v \leftarrow p[u]$ 
13:    $depth[u] \leftarrow depth[v] + 1$ 
14:    $temp \leftarrow pos[v]$ 
15:    $pos[v] \leftarrow pos[v] - pos[u]$ 
16:    $pos[u] \leftarrow temp$ 

17: for  $i \leftarrow 0$  to  $N - 1$  do
18:    $pos[i] \leftarrow pos[i] - 1$ 
19:    $dfs[pos[i]] \leftarrow i$ 
20: return  $p, ss, pos, dfs, depth$ 

```

1. $ss[v] \leftarrow ss[v] + ss[u]$ calculates the subtree size for every node.
 2. We iterate top-down using the order in dfs . We initialize a pos array with the subtree sizes ss .
 - Imagine node v owns a contiguous block of indices ending at $pos[v]$.
 - It gives its child u a chunk of this block equal to u 's subtree size. u 's block now ends where v 's block ended.
 - v 's remaining block end pointer is shifted back by u 's size ($pos[v] - pos[u]$).
 - As this repeats for all children of v , the children partition the space reserved for v 's descendants, and v 's pointer ends up pointing to the beginning of its block.
 3. $pos[i]$ points to the 1-based index where node i should appear. Subtracting 1 makes it a 0-indexed DFS array, and placing i at $dfs[pos[i]]$ computes dfs .
-

9.6 Diameter

With the bottom-up sequence from XOR-Linked traversal, we compute the diameter of the tree using DP.

Algorithm 17 Tree Diameter

Input: Number of nodes N , Parent array p , Degree array d

Output: Integer representing the tree diameter

1: Initialize $max_depth \leftarrow [0, \dots, 0]$ of size N

2: Initialize $diameter \leftarrow [0, \dots, 0]$ of size N

3: **for all** edge (u, v) processed via XOR-Linked traversal **do**

$\triangleright$ Path through v connecting u 's deepest leaf and previously processed children's deepest leaf

4: $path_through_v \leftarrow max_depth[v] + max_depth[u] + 1$

5: $diameter[v] \leftarrow \max(\{diameter[v], diameter[u], path_through_v\})$

6: $max_depth[v] \leftarrow \max(max_depth[v], max_depth[u] + 1)$

7: **return** $\max_{i \in [0, N-1]} diameter[i]$

The algorithm maintains two variables for each vertex v :

- $max_depth[v]$: The length of the longest path from v down to a leaf in its subtree already processed.
- $diameter[v]$: The maximum diameter completely contained within the subtree rooted at v already processed.

Because the traversal processes children before their parents, when we evaluate the edge between child u and parent v , $max_depth[u]$ and $diameter[u]$ are already computed. We update v 's state by checking if joining u 's deepest path to v 's currently known deepest path creates a new maximum diameter for v 's subtree. Finally, we update v 's maximum depth to include u .

9.7 Shallowest Decomposition Tree

The Shallowest Decomposition Tree possesses the same properties as a Centroid Decomposition but can be constructed in $O(n)$ time with optimal depth.

Definition 148 (Valid Tree Labeling). A valid tree labeling is an assignment of integer labels $\ell : V \rightarrow \mathbb{N}$ such that for any two distinct vertices u, v sharing the same label ($\ell(u) = \ell(v)$), the simple path between them contains at least one vertex w with a strictly greater label ($\ell(w) > \ell(u)$).

The depth of the resulting decomposition tree is exactly the maximum label assigned.

1. The DP We compute the optimal labeling in $O(n)$ time using a bottom-up DP. For each node u , we maintain a bitmask $\mathcal{F}(u)$ representing the set of forbidden labels. The k -th bit of $\mathcal{F}(u)$ is 1 if assigning label k to u would violate the labeling condition with its descendants.

The optimal label $\ell(u)$ is the smallest non-forbidden integer, corresponding to the index of the lowest unset bit in $\mathcal{F}(u)$.

When a child v passes its constraints up to its parent u , the newly assigned label $\ell(v)$ acts as a “shield.” Any forbidden labels strictly smaller than $\ell(v)$ are shielded and are no longer forbidden for u . However, $\ell(v)$ itself becomes forbidden for u , and any constraints strictly greater than $\ell(v)$ pass through unaffected.

It is easy to see that the propagation is exactly:

$$\mathcal{P}(v) = \mathcal{F}(v) + 1$$

When u aggregates the propagated masks $\mathcal{P}(v)$ from all its children, it must resolve collisions:

1. **Union:** If a label is forbidden by any child, it is forbidden for u . Let $\mathcal{O}_{\text{once}} = \bigvee \mathcal{P}(v)$.
2. **Collision:** If a label k is forbidden by two or more distinct children, u must act as a shield for them by taking a label strictly greater than k . Let $\mathcal{O}_{\text{twice}}$ be the mask of bits set by at least two children.

If $\mathcal{O}_{\text{twice}} > 0$, let $k = \lfloor \log_2(\mathcal{O}_{\text{twice}}) \rfloor$ be its highest set bit. To force $\ell(u) > k$, we saturate all bits up to k by applying a mask $M_k = (1 \ll (k + 1)) - 1$. The final forbidden state for u is exactly:

$$\mathcal{F}(u) = \mathcal{O}_{\text{once}} \vee M_k$$

From this, we evaluate $\ell(u) = \text{trailing_zeros}(\mathcal{F}(u) + 1)$.

2. Decomposition Chains Let $\text{anc}(x)$ denote the lowest ancestor of x in the original rooted tree such that $\ell(\text{anc}(x)) > \ell(x)$. In the decomposition tree, u becomes the direct parent of all vertices x where $\text{anc}(x) = u$.

Two vertices x and y sharing the same $\text{anc}(x) = u$ must reside in the subtree of the same child v of u . Furthermore, they must have distinct labels (if they shared a label, a higher label would exist between them, violating the premise that u is their lowest higher-labeled ancestor).

Thus, the set of vertices in v 's subtree that attach to u form a strict total order: $x_1 > x_2 > \dots > x_k$. When u is removed during decomposition, x_1 becomes the highest-labeled root of the newly isolated component, x_2 becomes its child, and so on. This vertical parent-child line in the decomposition tree is called a Decomposition Chain.

The labels present in the chain from child v to parent u are exactly the labels exposed in v 's subtree that are strictly less than $\ell(u)$. Because $\mathcal{P}(v)$ encodes all exposed labels, we can extract this chain mask:

$$\text{ChainMask}(v \rightarrow u) = \mathcal{P}(v) \wedge ((1 \ll \ell(u)) - 1)$$

2. Global Chain Consider the set of vertices $S \subseteq V$ that have no higher-labeled ancestor in the original rooted tree. These vertices represent the sequence of maximum labels in the decomposition process.

By the same proof as before, the labels in S are distinct, let them be $x_1 > x_2 > \dots > x_k$.

In the decomposition tree, the vertex x_1 (which possesses the maximum label in the entire tree) naturally becomes the root, r_{decomp} . When x_1 is removed during decomposition, x_2 becomes the highest-labeled root of its newly isolated component, thus a child of x_1 . Thus the vertices in S form a chain.

3. Vertex Retrieval Knowing the labels in a chain is insufficient; we must retrieve the vertices in $O(1)$ time.

We maintain a global array of stacks, allocating one stack per possible label. During the DFS, immediately after computing $\ell(u)$, we push u onto $\text{stacks}[\ell(u)]$.

Because multiple subtrees share this global stack, we must manage the traversal order.

- **Forward:** In the DFS, we iterate through the children of u in forward order $(v_1, v_2, \dots, v_m)$. Their vertices are recursively evaluated and pushed onto the stacks in that order. The vertices belonging to the final child v_m reside at the top of the stacks.
- **Reversed:** To extract the chains to form edges, we iterate through the children of u in reverse $(v_m, \dots, v_1)$. Because we process the last child v_m first, the vertices defined by its ChainMask are guaranteed to be exactly at the top of their respective stacks.

For a given ChainMask, we iterate through its set bits in decreasing order (extracting the highest bit $L = \lfloor \log_2(\text{mask}) \rfloor$ first). This ensures that higher-labeled vertices become ancestors of lower-labeled vertices in the decomposition tree. We pop the vertex w from $\text{stacks}[L]$, add the directed edge $u \rightarrow w$, and update $u \leftarrow w$ to continue forming the chain downwards.

Algorithm 18 Shallowest Decomposition Tree

```
1: function EXTRACTCHAIN(mask, u)
2:   while mask  $\neq$  0 do
3:      $L \leftarrow \lfloor \log_2(\text{mask}) \rfloor$ 
4:      $\text{mask} \leftarrow \text{mask} \oplus (1 \ll L)$ 
5:     if  $\text{stacks}[L] \neq \emptyset$  then
6:        $v \leftarrow \text{stacks}[L].\text{pop}()$ 
7:       Add edge  $u \rightarrow v$  to decomposition tree
8:        $u \leftarrow v$ 
9: function DFS(u, p)
10:   $\text{once} \leftarrow 0$ ;  $\text{twice} \leftarrow 0$ 
11:  for all  $v \in \text{adj}[u]$  do
12:    if  $v \neq p$  then
13:      DFS(v, u)
14:       $\text{prop} \leftarrow \text{forbidden}[v] + 1$ 
15:       $\text{twice} \leftarrow \text{twice} \vee (\text{once} \wedge \text{prop})$ 
16:     $\text{once} \leftarrow \text{once} \vee \text{prop}$ 
17:  if  $\text{twice} = 0$  then
18:     $M \leftarrow 0$ 
19:  else
20:     $k \leftarrow \lfloor \log_2(\text{twice}) \rfloor$ 
21:     $M \leftarrow (1 \ll (k + 1)) - 1$ 
22:     $\text{forbidden}[u] \leftarrow \text{once} \vee M$ 
23:     $\text{label}_u \leftarrow \text{trailing\_zeros}(\text{forbidden}[u] + 1)$ 
24:     $\text{stacks}[\text{label}_u].\text{push}(u)$ 
25:    for all  $v \in \text{reversed}(\text{adj}[u])$  do
26:      if  $v \neq p$  then
27:         $\text{extract\_mask} \leftarrow (\text{forbidden}[v] + 1) \wedge ((1 \ll \text{label}_u) - 1)$ 
28:        EXTRACTCHAIN(extract_mask, u)
29: function SHALLOWESTDECOMP(adj, root)
30:  Initialize forbidden array to 0 for all nodes
31:  Initialize stacks as an array of empty stacks
32:  DFS(root, null)
33:   $\text{max\_label} \leftarrow \lfloor \log_2(\text{forbidden}[\text{root}] + 1) \rfloor$ 
34:   $\text{decomp\_root} \leftarrow \text{stacks}[\text{max\_label}].\text{pop}()$ 
35:   $\text{final\_mask} \leftarrow (\text{forbidden}[\text{root}] + 1) \wedge ((1 \ll \text{max\_label}) - 1)$ 
36:  EXTRACTCHAIN(final_mask, decomp_root)
37:  return decomp_root
```

9.8 Static Top Tree

A static top tree is best viewed as a tree expressing a rooted tree DP as a balanced expression.

The input tree itself never changes. We choose heavy children once, break the tree into heavy paths, and build a binary expression whose leaves are the original vertices. A point update changes one leaf and therefore requires recomputing only the $O(\log n)$ expression nodes above it.

The useful feature is that the expression distinguishes two kinds of partial trees:

Path	and	Point.
------	-----	--------

A **Path** object represents a connected piece whose distinguished direction follows one heavy path.

A **Point** object represents one or more completed child pieces hanging from the same vertex.

Only four operations are needed:

$\text{Vertex}_u : \text{Point} \longrightarrow \text{Path},$
 $\text{AddEdge} : \text{Path} \longrightarrow \text{Point},$
 $\text{Rake} : \text{Point} \times \text{Point} \longrightarrow \text{Point},$
 $\text{Compress} : \text{Path} \times \text{Path} \longrightarrow \text{Path}.$

We also require an identity point

0_P

for Rake.

9.8.1 The Four Operations

Suppose the original tree is rooted.

For a vertex u , all light-child subtrees are first converted to **Point** objects and combined. Inserting u turns the resulting point object into a path object:

$\text{Vertex}_u : \text{Point} \rightarrow \text{Path}.$

A completed child path becomes something hanging from its parent after we cross the parent-child edge:

$\text{AddEdge} : \text{Path} \rightarrow \text{Point}.$

Several light-child contributions hanging from the same parent are combined by

Rake.

Finally, consecutive pieces of one heavy path are concatenated by

Compress.

Thus the recursion around a vertex u has the form

light child paths
 $\downarrow$ AddEdge
 point contributions
 $\downarrow$ Rake
 all light children of u
 $\downarrow$ Vertex $_u$
 one heavy-path piece.

Consecutive heavy-path pieces are then joined with **Compress**.

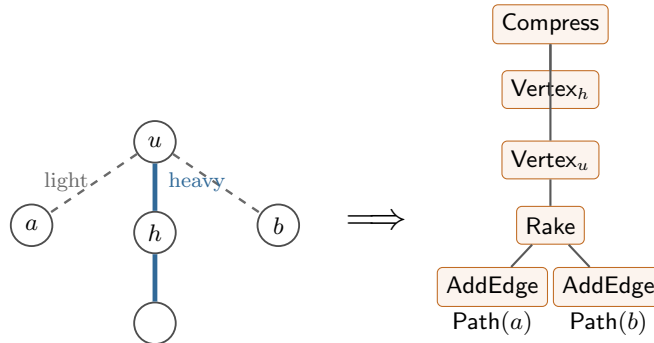


Figure 1: A vertex's light-child subtrees are converted to points and raked together; the vertex turns them back into a path piece. Heavy-path pieces are then compressed.

9.8.2 Heavy Paths

Let

$$\text{sz}(u)$$

be the number of vertices in the rooted subtree of u .

For every nonleaf u , choose as its heavy child

$$h(u) \in \arg \max_{v \text{ child of } u} \text{sz}(v).$$

All other child edges are light.

We use the standard fact:

Lemma 149 (Light-Edge Halving). *If v is a light child of u , then*

$$\text{sz}(v) \leq \frac{\text{sz}(u) - 1}{2}.$$

Consequently every root–leaf path contains at most

$$\lceil \log_2 n \rceil$$

light edges.

Proof. Since v is not the heavy child, there is another child $h(u)$ satisfying

$$\text{sz}(h(u)) \geq \text{sz}(v).$$

Hence

$$\text{sz}(u) \geq 1 + \text{sz}(v) + \text{sz}(h(u)) \geq 1 + 2\text{sz}(v).$$

Whenever a root–leaf path crosses a light edge, the subtree size therefore drops by at least a factor 2. Starting from at most n , this can happen at most $\lceil \log_2 n \rceil$ times. $\square$

9.8.3 The Expression Tree

For every original vertex u , define

$$\lambda(u) = 1 + \sum_{\substack{v \text{ child of } u \\ v \neq h(u)}} \text{sz}(v).$$

This is the number of original vertices represented by the local Vertex_u piece: u itself together with all of its light-child subtrees.

We construct the expression recursively.

For every light child v of u , recursively construct the heavy-path expression beginning at v , then wrap it in an AddEdge node.

Merge all such point objects with Rake , or use $\mathbf{0p}$ if there are none. Applying Vertex_u gives one path object of weight $\lambda(u)$.

Now follow the heavy chain

$$u, h(u), h^2(u), \dots$$

and construct one such path object for every vertex on it. Merge them, in top-to-bottom order, with Compress .

Thus if

$$u = u_0, u_1, \dots, u_k$$

is a maximal heavy path,

$$\text{BuildPath}(u) = \text{Compress}(\text{Vertex}_{u_0}(\dots), \dots, \text{Vertex}_{u_k}(\dots)).$$

The root expression is

$$\text{BuildPath}(r).$$

Lemma 150 (Correctness of the Construction). *For every heavy-path head u ,*

$$\text{BuildPath}(u)$$

represents exactly the rooted subtree T_u .

In particular, the root of the expression tree represents the entire input tree.

Proof. We induct on $\text{sz}(u)$.

Let

$$u = u_0, u_1, \dots, u_k$$

be the maximal heavy path beginning at u .

For every u_i , each light child v has strictly smaller subtree. By induction,

$$\text{BuildPath}(v)$$

represents exactly T_v . After **AddEdge**, it is a point contribution attached to u_i . Raking these contributions therefore represents exactly all light-child subtrees of u_i .

Applying **Vertex** _{u_i} adds u_i , so the resulting path object contains

$$u_i \quad + \quad \text{all light-child subtrees of } u_i.$$

The only missing descendants of u_i are those continuing through its heavy child u_{i+1} . Compressing the path pieces in order therefore adds precisely these heavy continuations.

Every vertex of T_u lies either on the heavy path or in the subtree of a unique light child of a heavy-path vertex. Hence the resulting object contains exactly T_u . $\square$

9.8.4 Weight-Balanced Merging

A naive left-associated merge of a heavy path could have linear depth. We therefore balance every **Compress** and **Rake** expression according to the number of original vertices represented by its children.

Suppose we must merge ordered objects

$$C_1, \dots, C_q$$

of positive weights

$$s_1, \dots, s_q.$$

Let

$$S = \sum_i s_i.$$

Scan from left to right, maintaining

$$z = S - 2 \sum_{j < i} s_j.$$

Put C_i on the left while

$$z > s_i;$$

once this fails, put C_i and all later objects on the right. Recursively merge the two sides.

Equivalently, C_i goes left precisely when the midpoint of its weighted interval lies to the left of $S/2$.

Lemma 151 (Weighted-Merge Depth). *Consider an item of weight s inside a weighted merge of total weight S .*

Its depth in the resulting binary merge tree is

$$\boxed{O\left(1 + \log \frac{S}{s}\right)}.$$

Proof. At a recursive call of total weight W , let C be the weight of the child containing our item.
If

$$C \leq \frac{2W}{3},$$

then after one level the containing weight has shrunk by a factor at least $3/2$.

Suppose instead that

$$C > \frac{2W}{3}.$$

Assume first that the heavy child is the right side. Let $P = W - C$ be the weight of the left side, and let x be the first item placed on the right, with weight s_x .

The condition for x to be placed on the right is

$$W - 2P \leq s_x.$$

Since $P = W - C$,

$$s_x \geq 2C - W.$$

Because $C > 2W/3$,

$$s_x > \frac{C}{2}.$$

When the right side is recursively split, x is its first item and has more than half its total weight. Hence x becomes a singleton child, and all remaining items lie in a child of weight

$$C - s_x \leq C - (2C - W) = W - C < \frac{W}{3}.$$

Thus after at most one additional level, our item is either isolated or lies in a subproblem of weight less than $W/3$.

The case in which the left side has weight $> 2W/3$ is symmetric: the last item on the left has weight $> C/2$, becomes isolated at the next split, and all other items lie in a subproblem of weight $< W/3$.

Therefore, unless the item has already become a leaf, every two levels reduce the containing total weight by at least a constant factor.

Starting from S , this can happen only

$$O\left(\log \frac{S}{s}\right)$$

times before the containing weight is $O(s)$, at which point the item is isolated after $O(1)$ further levels. $\square$

Theorem 152 (Static Top-Tree Height). *The expression tree constructed above has*

$$\boxed{O(\log n)}$$

height and $O(n)$ nodes.

Proof. First consider the node count.

There is exactly one **Vertex** node for each original vertex. Every light tree edge contributes one **AddEdge** node.

A binary merge of q items contributes $q - 1$ internal nodes. Summed over all rake lists, this is $O(n)$, because every raked item corresponds to a distinct light child.

Likewise, summed over all heavy paths, the number of **Compress** nodes is

$$n - \#\{\text{heavy paths}\} = O(n).$$

Hence the expression contains $O(n)$ nodes.

Now fix an original vertex x , and follow the expression-parent chain from its **Vertex** node to the root.

Every time this chain passes from a child heavy path into its parent's local piece, it crosses a light represented-tree edge. By [Lemma 149](#), this happens only $O(\log n)$ times.

Between two such light transitions, the chain passes through one weighted rake tree and one weighted compress tree. By [Lemma 151](#), the number of merge nodes contributed is bounded by a logarithm of the ratio between the enclosing represented size and the size of the child piece.

These size ratios telescope as we move upward through nested represented pieces. Their total logarithm is at most

$$\log \frac{n}{1} = O(\log n).$$

The nonmerge constructors contribute only $O(1)$ nodes per light transition. Thus the whole expression-parent chain has length $O(\log n)$. $\square$

Theorem 153 (Construction Time). *The recursive construction above takes*

$$\boxed{O(n \log n)}$$

time and $O(n)$ space.

Once the expression structure has been built, evaluating all DP values bottom-up takes $O(n)$ time.

Proof. Computing subtree sizes and heavy children takes $O(n)$ time.

The expression itself has $O(n)$ nodes. The only superlinear work comes from the simple weighted-merge routine, which scans the current list at every recursive level.

By [Lemma 151](#), every item occurs in $O(\log n)$ such scans. Thus all merge construction costs sum to $O(n \log n)$.

A bottom-up evaluation performs $O(1)$ work at each expression node, hence takes $O(n)$. $\square$

9.8.5 The DP Interface

After combining the two vertex cases, a static-top-tree DP needs

$$\boxed{\begin{array}{l} \text{vertex} : (\text{Point}, u) \mapsto \text{Path}, \\ \text{compress} : (\text{Path}, \text{Path}) \mapsto \text{Path}, \\ \text{rake} : (\text{Point}, \text{Point}) \mapsto \text{Point}, \\ \text{addEdge} : \text{Path} \mapsto \text{Point}, \end{array}}$$

together with $\mathbf{0_P}$.

At a vertex node u , if there is no rake child we evaluate

$$\text{vertex}(\mathbf{0_P}, u).$$

Otherwise we evaluate

$$\text{vertex}(\text{point}(\text{left}(u)), u).$$

9.8.6 Point Updates

Store the value of every expression node.

If the value of an original vertex u changes, start at its **Vertex** node and recompute every expression ancestor.

Theorem 154 (Static Top-Tree Update). *A vertex update takes*

$$\boxed{O(\log n)}$$

time.

Proof. Only ancestors of the corresponding expression leaf can depend on the updated value.

By [Theorem 152](#), there are $O(\log n)$ such ancestors, and every constructor is recomputed in $O(1)$ time. $\square$

9.8.7 Rooted-Subtree Queries

The representation also gives subtree queries.

The **Vertex** node for u already contains

$$u + \text{every light-child subtree of } u.$$

The only descendants missing from T_u are therefore the vertices below u on its heavy path, together with their light subtrees.

Those are exactly the later pieces of the same **Compress** expression.

Let

$$x = u, \quad R = \text{path}[u].$$

While the expression parent $p(x)$ is a **Compress** node:

$$\begin{cases} R \leftarrow \text{compress}(R, \text{path}[r(p(x))]), & x = l(p(x)), \\ R \text{ unchanged}, & x = r(p(x)). \end{cases}$$

Then set

$$x \leftarrow p(x)$$

and continue.

Stop at the first non-**Compress** parent.

Theorem 155 (Static Rooted-Subtree Query). *The preceding procedure returns the **Path** value of exactly the rooted subtree T_u , in*

$$\boxed{O(\log n)}$$

time.

Proof. The compress expression for a heavy path preserves the top-to-bottom order.

At any compress node, the left child represents an earlier, higher segment of the heavy path, and the right child represents a later, lower segment.

While climbing from the leaf corresponding to u :

If the current object is the left child, then the right sibling lies strictly below it on the same heavy path and therefore belongs to T_u . It must be appended.

If the current object is the right child, then the left sibling lies strictly above u and must be excluded.

Consequently the procedure accumulates exactly the suffix of the heavy path beginning at u . Each vertex piece on that suffix already contains all of its light-child subtrees.

Thus the accumulated represented vertices are exactly T_u .

When we reach a noncompress parent, we have reached the head of this heavy path. Crossing the next **AddEdge** would move to its represented parent, which lies outside T_u , so the query must stop there.

The number of expression ancestors visited is $O(\log n)$. □

9.8.8 Path Queries

The weight-balanced compress and rake trees also give

$$\boxed{O(\log n)}$$

path queries, rather than $O(\log^2 n)$.

Recall that for a vertex u ,

$$\lambda(u) = 1 + \sum_{\substack{v \text{ child of } u \\ v \neq h(u)}} \text{sz}(v).$$

Equivalently, if $h(u)$ exists,

$$\lambda(u) = \text{sz}(u) - \text{sz}(h(u)).$$

These $\lambda(u)$'s are exactly the weights used to balance the **Compress** tree of each heavy path.

We first record how to query a prefix of one heavy path.

Lemma 156 (Heavy-Path Prefix). *Let H be a maximal heavy path with head h , and let $x \in H$. The aggregate on the path segment*

$$P(h, x)$$

can be obtained in

$$O\left(1 + \log \frac{\text{sz}(h)}{\lambda(x)}\right)$$

time.

Proof. The leaves of the compress expression for H occur in top-to-bottom order along the heavy path.

Start at the leaf corresponding to x , with the aggregate of x itself, and climb toward the root of this compress tree.

Whenever the current node is a right child, its left sibling lies entirely before it on the heavy path, so prepend the aggregate of that left sibling. Whenever the current node is a left child, the right sibling lies below x and is ignored.

When the root is reached, the accumulated value is exactly the prefix

$$P(h, x).$$

The work is proportional to the depth of the leaf x . The total weight of the compress tree is

$$\sum_{u \in H} \lambda(u) = \text{sz}(h),$$

because the differences telescope along the heavy path.

By [Lemma 151](#), the leaf of weight $\lambda(x)$ has depth

$$O\left(1 + \log \frac{\text{sz}(h)}{\lambda(x)}\right).$$

□

For noncommutative aggregates, store both orientations of every **Path** value. Then the same procedure returns the segment in either direction.

Why the costs telescope. Consider walking upward from a vertex x_0 toward the root by the usual heavy-light decomposition.

Let

$$h_i$$

be the head of the heavy path containing x_i , and after processing the segment

$$P(h_i, x_i)$$

set

$$x_{i+1} = \text{parent}(h_i).$$

Thus the edge

$$x_{i+1}h_i$$

is light.

The cost of the i -th heavy-path segment is

$$O\left(1 + \log \frac{\text{sz}(h_i)}{\lambda(x_i)}\right).$$

Now observe the reason for having balanced the compress tree using the weights λ . For $i \geq 1$, the previous heavy-path head h_{i-1} is a light child of x_i . Therefore its entire subtree is included in the local weight of x_i :

$$\boxed{\lambda(x_i) \geq \text{sz}(h_{i-1}).}$$

Consequently,

$$\begin{aligned} \sum_{i=0}^k \log \frac{\text{sz}(h_i)}{\lambda(x_i)} &\leq \log \frac{\text{sz}(h_0)}{\lambda(x_0)} + \sum_{i=1}^k \log \frac{\text{sz}(h_i)}{\text{sz}(h_{i-1})} \\ &= \log \frac{\text{sz}(h_k)}{\lambda(x_0)} \\ &\leq \log n. \end{aligned}$$

There are also only $O(\log n)$ segments, because moving from h_i to x_{i+1} crosses a light edge, and every root–leaf path has only $O(\log n)$ light edges.

Hence all heavy-path prefixes on one upward walk cost

$$\boxed{O(\log n)}$$

in total.

Theorem 157 (Static Path Query). *An associative aggregate on an arbitrary represented-tree path*

$$P(u, v)$$

can be queried in

$$\boxed{O(\log n)}$$

time.

Proof. Use the ordinary heavy-light path decomposition.

While u and v lie on different heavy paths, process the endpoint whose heavy-path head is deeper. If h is the head of the heavy path containing that endpoint x , query

$$P(h, x)$$

using [Lemma 156](#), append or prepend it with the appropriate orientation, and replace

$$x \leftarrow \text{parent}(h).$$

The segments processed from the u -side form one upward heavy-light walk, and those processed from the v -side form another. By the telescoping calculation above, each side costs

$$O(\log n)$$

in total.

Eventually u and v lie on the same heavy path. The remaining segment between them is one interval in a single weight-balanced compress tree.

An interval in a binary expression tree can be obtained by climbing from its two endpoint leaves toward their least common ancestor, accumulating the siblings lying inside the interval. Its cost is at most the sum of the two endpoint depths, hence

$$O(\log n)$$

by [Lemma 151](#).

Combining the two upward portions and the final heavy-path interval gives

$$O(\log n)$$

total time. □

Theorem 158 (Static Top-Tree Queries). *After preprocessing, and after any point updates, the static top tree supports*

$\text{rooted-subtree query} : O(\log n),$ $\text{arbitrary path query} : O(\log n).$
--

Path aggregates may be noncommutative by storing both orientations of every Path value.

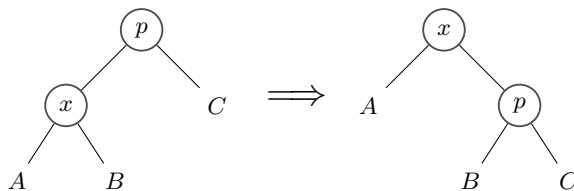
9.9 Splay Tree

A splay tree is a binary tree with no explicit balancing information. Whenever a node is accessed, rotations move it to the root.

Individual operations may take linear time, but the amortized cost of a splay is logarithmic.

9.9.1 Rotation

Suppose x is the left child of p .



The inorder order

$$A, x, B, p, C$$

is unchanged.

The other rotation is symmetric.

9.9.2 Splaying

To splay x , repeatedly inspect its parent p and, when it exists, its grandparent g .

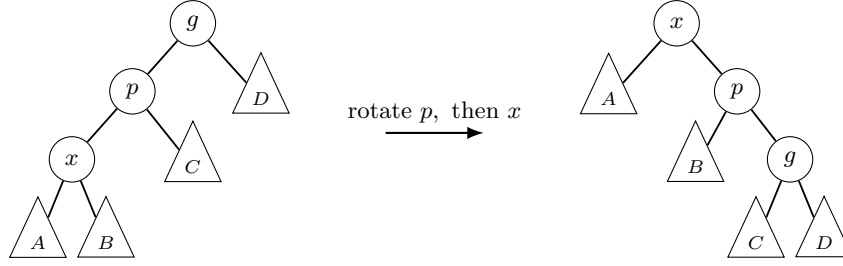
A final *zig* rotates x once.

In a *zig-zig*, x and p have the same orientation; rotate p , then x .

In a *zig-zag*, they have opposite orientations; rotate x twice.

Before rotating, all lazy tags on the auxiliary root- x path must be pushed from top to bottom.

Zig-Zig The only non-obvious part of splaying is the zig-zig step. In the left-left case it performs



The point of zig-zig is that splaying should improve more than just the accessed vertex. If we blindly rotate x upward twice, x reaches the top, but both vertices above it are pushed down:

$$g, p, x \longrightarrow x, g, p.$$

In particular, p , which was immediately next to the accessed vertex, actually becomes deeper.

Zig-zig instead reverses the whole three-vertex piece:

$$g, p, x \longrightarrow x, p, g.$$

Thus x gains two levels while p stays close to it; only the more distant ancestor g is pushed far downward. Vertices that were close to x remain relatively close to the top as well. This is the improvement that makes the amortized bound possible.

9.9.3 Access Lemma

Give every node v an arbitrary positive analysis weight

$$w(v) > 0.$$

For a splay subtree T_v , define

$$s(v) = \sum_{u \in T_v} w(u), \quad r(v) = \log_2 s(v),$$

and use the potential

$$\Phi = \sum_v r(v).$$

We need one elementary inequality.

Lemma 159. *If $a, b > 0$ and*

$$a + b \leq c,$$

then

$$\boxed{\log_2 a + \log_2 b \leq 2 \log_2 c - 2.}$$

Proof. By AM-GM,

$$ab \leq \left(\frac{a+b}{2} \right)^2 \leq \frac{c^2}{4}.$$

Take base-2 logarithms. □

Theorem 160 (Splay Access Lemma). *Suppose x is splayed in a tree of total analysis weight W . Its amortized number of rotations is at most*

$$\boxed{1 + 3 \log_2 \frac{W}{w(x)}}.$$

Proof. We prove that every double rotation has amortized cost at most

$$3(r'(x) - r(x)),$$

while the final zig has amortized cost at most

$$1 + 3(r'(x) - r(x)).$$

For a zig, only x and its parent p change rank, and

$$r'(x) = r(p).$$

Thus

$$\begin{aligned} 1 + \Delta\Phi &= 1 + r'(x) + r'(p) - r(x) - r(p) \\ &= 1 + r'(p) - r(x) \\ &\leq 1 + r'(x) - r(x). \end{aligned}$$

Now consider zig-zig:

$$x \subset p \subset g.$$

Again

$$r'(x) = r(g).$$

The old subtree of x and the new subtree of g are disjoint and both lie inside the new subtree of x . Hence

$$s(x) + s'(g) \leq s'(x).$$

By [Lemma 159](#),

$$r(x) + r'(g) \leq 2r'(x) - 2.$$

Also,

$$r'(p) \leq r'(x), \quad r(p) \geq r(x).$$

Therefore

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\leq 2 + r'(x) + r'(g) - 2r(x) \\ &\leq 3(r'(x) - r(x)). \end{aligned}$$

For zig-zag, the new subtrees rooted at p and g are disjoint inside the new subtree rooted at x , so

$$r'(p) + r'(g) \leq 2r'(x) - 2.$$

Thus

$$\begin{aligned} 2 + \Delta\Phi &= 2 + r'(p) + r'(g) - r(x) - r(p) \\ &\leq 2r'(x) - r(x) - r(p) \\ &\leq 2(r'(x) - r(x)) \\ &\leq 3(r'(x) - r(x)). \end{aligned}$$

Summing over the whole splay telescopes:

$$\sum (r'(x) - r(x)) = r_{\text{final}}(x) - r_{\text{initial}}(x).$$

Finally,

$$r_{\text{final}}(x) \leq \log_2 W, \quad r_{\text{initial}}(x) \geq \log_2 w(x).$$

□

With unit weights,

$$W = n,$$

so a splay costs

$$O(\log n)$$

amortized.

9.9.4 Augmentation

Suppose values lie in a monoid

$$(\mathcal{M}, \circ, e).$$

Store

$$\text{agg}(v) = \text{agg}(L_v) \circ \text{val}(v) \circ \text{agg}(R_v).$$

A rotation changes only constantly many child pointers, so the aggregates are restored in $O(1)$ time.

For sequence reversal, maintain a lazy reversal bit. If the monoid is noncommutative, store both

$$\text{fwd}(v) \quad \text{and} \quad \text{rev}(v).$$

Reversal swaps the children and these two aggregates.

9.10 Link-Cut Tree

A link-cut tree is a data structure for maintaining a forest while supporting operations on arbitrary tree paths in amortized

$$O(\log n)$$

time.

The main difficulty is that a tree path can be long. Therefore, we decompose it into paths, store each path in a splay tree, and change this path decomposition whenever a new path is accessed.

It is useful to distinguish three objects:

1. the *represented tree*, which is the actual tree we care about;
2. the *auxiliary trees*, which are splay trees representing disjoint paths of the represented tree;
3. the *tree of auxiliary trees*, obtained by contracting each such path to one object and keeping the represented edges between different paths.

9.10.1 Preferred Paths

At every vertex, at most one edge to a child is designated *preferred*. These choices are not determined by subtree size and are not permanent; they change during accesses.

Since every vertex has at most one preferred child, the preferred edges form a collection of vertex-disjoint downward paths.

Each such preferred path is stored in one *auxiliary splay tree*.

All remaining represented-tree edges are called *virtual*. They join one preferred path to another. Thus if we contract every preferred path, the virtual edges form a tree whose nodes are the auxiliary trees.

For example, consider the following represented tree.

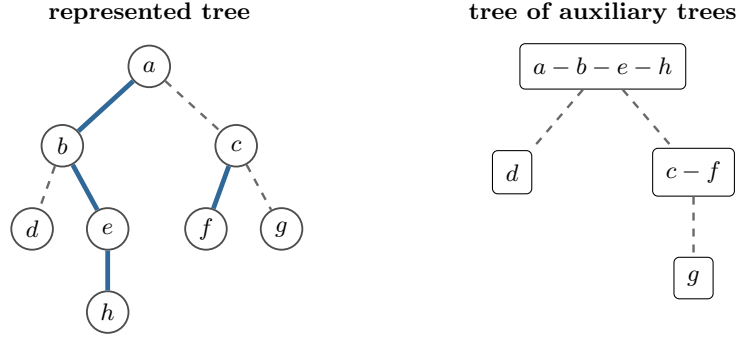


Figure 2: On the left, preferred edges partition the represented tree into the paths $a - b - e - h$, $c - f$, d , and g . Each path is stored in one auxiliary splay tree. Contracting those paths gives the tree of auxiliary trees on the right.

This second view is particularly useful for understanding **Access**: an access starts in one auxiliary tree and repeatedly moves through a virtual edge to the auxiliary tree above it.

9.10.2 How a Preferred Path Is Stored

Suppose a preferred path is

$$v_1, v_2, \dots, v_k$$

from top to bottom in the represented tree.

Its auxiliary tree is a splay tree whose *inorder order* is

$$v_1, v_2, \dots, v_k.$$

The shape of the splay tree has nothing to do with the shape of the represented tree.

For example, the represented path

$$a - b - e - h$$

could currently be stored as follows.

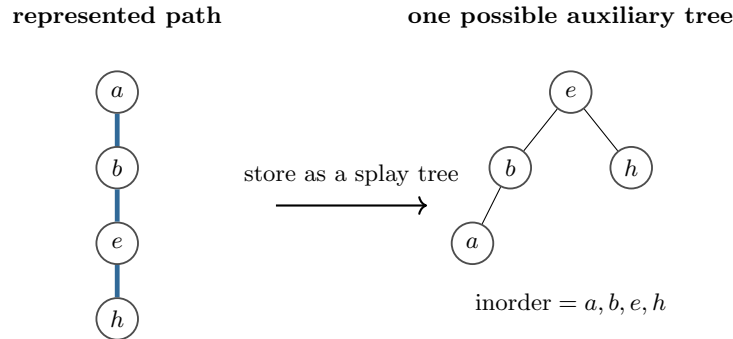


Figure 3: An auxiliary tree does not imitate the represented-tree parent pointers. Its only structural requirement is that inorder order agrees with top-to-bottom order along the preferred path.

Splaying may completely rearrange an auxiliary tree, but it preserves inorder order, so it continues to represent the same tree path.

It also means that ordinary binary-tree augmentation works immediately. If every splay node stores an aggregate of its subtree, then the root of an auxiliary tree stores the aggregate of the entire preferred path.

9.10.3 Connecting the Auxiliary Trees

The auxiliary trees must still remember how the preferred paths are joined inside the represented tree.

Suppose a preferred path begins at x , and the represented parent of x is p . Then the edge

$$px$$

is virtual and connects this auxiliary tree to the auxiliary tree containing p .

This connection is called the *path-parent* relation.

Conceptually, we could store two kinds of pointers:

- ordinary splay-tree parent pointers inside an auxiliary tree;
- a path-parent pointer from the whole auxiliary tree to the auxiliary tree above it.

A link-cut tree can encode both with a single field

$$\text{parent}(x).$$

If x is not the root of its auxiliary tree, this is its ordinary splay parent.

If x is the root of its auxiliary tree, then its parent pointer instead stores the path-parent connection of the whole auxiliary tree.

Thus x is an auxiliary root precisely when

$$\text{parent}(x) = \perp$$

or

$$x \notin \{\text{left}(\text{parent}(x)), \text{right}(\text{parent}(x))\}.$$

Notice that if x is an auxiliary root, $\text{parent}(x)$ need not be the represented parent of x itself. The pointer represents the virtual edge immediately above the top of the path stored by the entire auxiliary tree.

This is what allows us to climb from one auxiliary tree to the next.

9.10.4 Access

The fundamental operation is

$$\text{Access}(v).$$

Its purpose is simple:

make the entire represented root- v path one preferred path.

Equivalently, after an access, the root- v path is stored in one auxiliary splay tree.

Before the operation, that path may pass through many auxiliary trees. In the tree-of-auxiliary-trees picture, $\text{Access}(v)$ walks upward through these auxiliary trees and splices them together one at a time.

The key local operation is easiest to understand after splaying a vertex x .

Since inorder order is top-to-bottom represented-tree order,

- the left subtree of x consists of the part of its current preferred path *above* x ;
- the right subtree of x consists of the part *below* x .

Suppose we have already exposed the portion from x downward toward v , stored in an auxiliary tree called *last*.

We then replace

$$\text{right}(x)$$

by *last*.

This does exactly what we want:

- the old preferred continuation below x stops being preferred and becomes a separate auxiliary tree;
- the path from x toward v becomes preferred;
- hence the exposed suffix toward v is merged with the preferred path containing x .

We then follow the path-parent pointer to the auxiliary tree immediately above and repeat.

Algorithm 19 Link-Cut Tree Access

```

1: function ACCESS( $v$ )
2:    $last \leftarrow \perp$ 
3:    $x \leftarrow v$ 
4:   while  $x \neq \perp$  do
5:     SPLAY( $x$ )
6:      $right(x) \leftarrow last$ 
7:     PULL( $x$ )
8:      $last \leftarrow x$ 
9:      $x \leftarrow parent(x)$ 
10:  SPLAY( $v$ )

```

Here and below, the assignment

$$right(x) \leftarrow last$$

includes the corresponding parent-pointer bookkeeping.

Lemma 161 (Correctness of Access). *After ACCESS(v), the represented root- v path is one preferred path, represented by the auxiliary splay tree rooted at v .*

Proof. Consider an iteration after x has been splayed.

The right subtree of x is exactly the portion of the current preferred path strictly below x . Replacing it by $last$ therefore discards the old preferred continuation and makes the already exposed route from x toward v preferred.

Consequently, after the splice, the auxiliary tree rooted at x represents a preferred path ending at v .

Its path-parent pointer leads to the represented edge immediately above the top of this preferred path. The next loop iteration enters the auxiliary tree above this edge and splices that path onto the front.

Inductively, $last$ always represents an increasingly long suffix of the represented root- v path. When there is no path-parent left, that suffix begins at the represented root and hence is the entire path.

Finally, splaying v makes v the root of the resulting auxiliary tree. □

9.10.5 Why Access Is Fast

One access can pass through many auxiliary trees, so it can perform many splays. Thus, a single access need not take $O(\log n)$ worst-case time.

There are two pieces to the analysis:

1. if an access encounters k auxiliary trees, all of its splaying costs only

$$O(k + \log n)$$

amortized time;

2. over a sequence of accesses, the number of auxiliary-tree boundaries crossed is only $O(\log n)$ amortized per access.

The first part comes from the splay-tree potential. The second is a heavy-light accounting argument.

Importantly, *heavy and light edges are used only for the analysis*. The link-cut tree does not choose its preferred edges according to heavy-light decomposition.

9.10.6 The Analysis Tree

For the splay analysis, expand the tree of auxiliary trees back to the vertex level.

Keep every splay left/right edge. Whenever an auxiliary tree has a path-parent pointer, also keep that connection.

Ignoring which edges are splay edges and which are path-parent edges gives one ordinary rooted tree on the same vertex set. Call this the *analysis tree*.

For a vertex x , let

$$\sigma(x) = \#\{\text{descendants of } x \text{ in the analysis tree}\},$$

including x , and set

$$\rho(x) = \log_2 \sigma(x).$$

Use the potential

$$\Phi = \sum_x \rho(x).$$

The useful feature of this viewpoint is that the splice performed by **Access** does not change the underlying analysis tree.

When the old right subtree of x stops being a splay child, its root becomes the root of a separate auxiliary tree whose path-parent is x . Thus it was a child of x in the analysis tree before the splice and remains a child afterward.

Likewise, *last* was connected to x through a path-parent relation before the splice and becomes its right splay child afterward.

Thus a splice merely changes how an analysis-tree edge is represented: as a splay edge or as a path-parent edge.

Only splay rotations change the analysis tree and therefore the potential.

Lemma 162 (Total Splay Cost During Access). *Suppose an access performs k loop iterations. The total amortized rotation cost of its splays is*

$$O(k + \log n).$$

Proof. Consider one splay of x .

All auxiliary trees hanging virtually from vertices of the current preferred path may be regarded as fixed positive weights attached to those vertices. Then the weighted size of a splay subtree is exactly the corresponding analysis-tree subtree size σ .

Hence the weighted splay access lemma gives

$$\text{amortizedCost}(\text{Splay}(x)) \leq 1 + 3(\rho_{\text{after}}(x) - \rho_{\text{before}}(x)).$$

Let

$$x_1, x_2, \dots, x_k$$

be the successive loop vertices, starting near v and moving toward the represented root.

After processing x_i , the exposed structure rooted at x_i lies below the next loop vertex x_{i+1} in the analysis tree. Therefore

$$\sigma_{\text{after}}(x_i) \leq \sigma_{\text{before}}(x_{i+1}),$$

so

$$\rho_{\text{after}}(x_i) \leq \rho_{\text{before}}(x_{i+1}).$$

The rank increases therefore telescope:

$$\begin{aligned} \sum_{i=1}^k (\rho_{\text{after}}(x_i) - \rho_{\text{before}}(x_i)) &\leq \rho_{\text{after}}(x_k) - \rho_{\text{before}}(x_1) \\ &\leq \log_2 n. \end{aligned}$$

Thus the total amortized splay cost is

$$O(k + \log n).$$

The final splay of v contributes another $O(\log n)$, which is absorbed into the same bound. $\square$

9.10.7 How Many Auxiliary Trees Does Access Cross?

It remains to account for k , the number of preferred-path pieces encountered.

For this argument only, fix the represented tree and classify a child edge uv as *heavy* when

$$\text{sz}(v) > \frac{1}{2} \text{sz}(u),$$

and *light* otherwise.

Each vertex has at most one heavy child, and every represented root–leaf path contains at most

$$\log_2 n$$

light edges.

Now consider what an access does to preferred edges.

Every virtual edge on the newly exposed root– v path becomes preferred. Among these, only

$$O(\log n)$$

can be light.

What about heavy edges?

A heavy preferred edge ux can become nonpreferred only if some other child y of u becomes preferred. Since u has at most one heavy child, the new preferred edge uy must then be light.

Thus every event that destroys a heavy preferred edge can be charged to the creation of a light preferred edge.

Furthermore, after a heavy edge has become preferred once, making it preferred again requires it to have become nonpreferred in between.

Hence, over q accesses,

$$\#\{\text{heavy preferred creations}\} \leq n + \#\{\text{light preferred creations}\}.$$

Since each access creates only $O(\log n)$ light preferred edges,

$$\#\{\text{light preferred creations}\} = O(q \log n).$$

Therefore the total number of preferred-path boundaries crossed or changed is

$$O(n + q \log n).$$

Combining this with [Lemma 162](#), the total time for q accesses is

$$O(n + q \log n).$$

Thus access takes

$$\boxed{O(\log n)}$$

amortized time.

Theorem 163 (Link-Cut Access).

$$\boxed{\text{Access}(v) \text{ takes } O(\log n) \text{ amortized time.}}$$

9.10.8 Rerooting

Once we can expose a root–vertex path, rerooting is immediate.

After

$\text{Access}(v)$,

the represented root– v path is exactly one auxiliary tree, in top-to-bottom inorder order.

To make v the represented root, simply reverse this path.

We therefore define

$\text{MakeRoot}(v) : \quad \text{Access}(v); \quad \text{Reverse}(v).$

The reversal is stored lazily in the splay tree.

Reversing the order of the exposed path reverses every represented parent relation along it, so v becomes the represented root.

9.10.9 Dynamic-Forest Operations

Once Access and MakeRoot are available, the usual dynamic-tree operations become short.

Finding the root. Perform

$\text{Access}(v).$

The entire represented root– v path is now one auxiliary tree. Its leftmost vertex is therefore the represented root.

Walk left while pushing lazy reversals, and splay the resulting vertex.

Link. Suppose u and v lie in different represented trees.

First perform

$\text{MakeRoot}(u).$

Now u has no represented parent. Make its auxiliary tree a virtual child of v , thereby adding the represented edge

$uv.$

Cut. Suppose uv is a represented edge.

Perform

$\text{MakeRoot}(u), \quad \text{Access}(v).$

The auxiliary tree rooted at v now represents exactly

$P(u, v).$

Since u and v are adjacent in the represented tree, this path contains exactly their one edge. Detaching that exposed connection removes uv from the represented forest.

9.10.10 Path Queries and Updates

The most useful consequence of access is that an arbitrary represented path can be turned into one auxiliary tree.

To expose

$P(u, v),$

perform

$\text{MakeRoot}(u), \quad \text{Access}(v).$

After `MakeRoot(u)`, the path $P(u, v)$ is a root- v path. The access therefore makes exactly this path one preferred path.

Hence the entire auxiliary tree rooted at v consists precisely of $P(u, v)$.

If every splay node stores an aggregate of its subtree, then

$$\text{PathQuery}(u, v) = \text{agg}(v).$$

Similarly, any update that can be represented by a compatible lazy tag can be applied to the root v , thereby updating the entire path at once.

Theorem 164 (Link-Cut Tree). *A link-cut tree supports*

`makeRoot`, `findRoot`, `link`, `cut`,

together with path aggregates and compatible lazy path updates, in

$$O(\log n)$$

amortized time per operation and $O(n)$ space.

Proof. Every operation consists of a constant number of calls to `Access`, together with $O(1)$ additional splays, lazy reversals, and pointer changes.

By [Theorem 163](#), each access costs $O(\log n)$ amortized time.

Splay-tree aggregates and lazy information are maintained by `Pull` and `Push` in $O(1)$ time per rotation, so augmenting the auxiliary trees does not change the asymptotic bound. $\square$

9.11 Top Tree

The cleanest way to arrive at a top tree is to start from the one thing a link-cut tree does not maintain well.

A link-cut tree knows everything about the current preferred path.

It knows almost nothing about the collection of virtual children hanging from that path.

Suppose u has virtual child subtrees

$$T_1, T_2, \dots, T_k.$$

An ordinary LCT may maintain their total aggregate explicitly, but if

$$k = \Theta(n),$$

there is no balanced structure representing those children individually.

This becomes a problem as soon as we want subtree queries, subtree lazy updates, or more complicated DP information.

The top-tree idea is:

keep the LCT preferred-path splay tree, and also put every collection of virtual children in a splay tree.

The preferred-path splay is the *compress tree*.

The virtual-child splay is the *rake tree*.

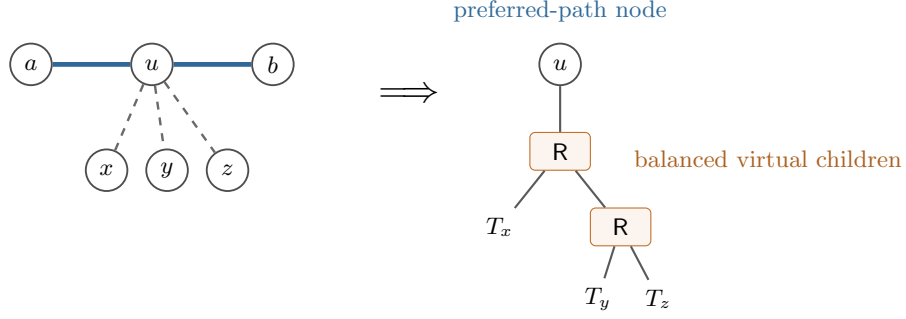


Figure 4: A top tree adds a balanced rake structure for the virtual children that an ordinary link-cut tree leaves implicit.

9.11.1 Real and Fake Nodes

A real represented-tree vertex stores its ordinary two splay children

$$\text{ch}[0], \quad \text{ch}[1].$$

These are the preferred-path or compress children.

We also give it two rake slots

$$\text{ch}[2], \quad \text{ch}[3].$$

A represented vertex may have arbitrarily many virtual children, so the rake slots contain a binary tree of *fake nodes*.

Fake nodes contain no represented-tree vertex value. Their sole purpose is to combine two virtual-child aggregates.

Whenever a virtual child is inserted or deleted, ordinary splay rotations within the rake tree keep this collection self-adjusting.

Since every fake node is an internal node of a binary tree whose leaves are represented virtual pieces, the total number of fake nodes is $O(n)$.

9.11.2 The Same Path/Point Interface Reappears

Recall the static-top-tree interface.

A preferred path is a

Path

object.

A collection of virtual children attached at one vertex is a

Point

object.

The four operations are again

$\text{Vertex}_u : \text{Point} \rightarrow \text{Path},$ $\text{AddEdge} : \text{Path} \rightarrow \text{Point},$ $\text{Rake} : \text{Point}^2 \rightarrow \text{Point},$ $\text{Compress} : \text{Path}^2 \rightarrow \text{Path}.$

The difference is only that the binary expression is now maintained dynamically by splay rotations.

9.11.3 Access with Virtual-Child Trees

Consider one iteration of ordinary

$\text{Access}(v)$

at a vertex x .

After splaying x , let

$old = \text{right}(x)$.

This is the old preferred continuation below x .

In an ordinary LCT we simply execute

$\text{right}(x) \leftarrow last$.

With virtual-child structures, we must also maintain where those represented subtrees live.

The old preferred continuation becomes virtual, so if $old \neq \perp$, convert it to a point cluster with

$\text{AddEdge}(old)$

and insert that point into x 's rake tree.

Conversely, $last$ was previously a virtual child contribution of x . Remove its point representation from the rake tree and install it as the new right preferred child.

Thus one splice is

if $old \neq \perp$: $\text{InsertVirtual}(x, old)$,
 if $last \neq \perp$: $\text{EraseVirtual}(x, last)$,
 $\text{right}(x) \leftarrow last$.

9.11.4 Aggregates

For a real node u , its auxiliary subtree consists of a piece of a preferred path together with arbitrary represented subtrees hanging off that path. Thus its represented material has a natural partition into two parts:

preferred-path spine $\sqcup$ material hanging off the spine.

We maintain one aggregate for each part.

For a real node u , let

$\text{path}(u)$ = aggregate of the represented vertices on its preferred-path spine,
 $\text{sub}(u)$ = aggregate of everything hanging off that spine,
 $\text{all}(u)$ = aggregate of the entire represented cluster.

Compress children versus rake children. Recall that a real node u has two different kinds of auxiliary children.

The children

$\text{ch}_0(u), \text{ch}_1(u)$

are *compress children*. They continue the same preferred-path spine as u .

The children

$\text{ch}_2(u), \text{ch}_3(u)$

are *rake children*. They represent entire clusters attached virtually to u , and hence lie completely off the preferred path containing u .

The path aggregate. Only the compress children lie on the same preferred path as u . Therefore

$$\text{path}(u) = \text{path}(\text{ch}_0 u) \oplus \text{val}(u) \oplus \text{path}(\text{ch}_1 u).$$

Inorder order in the compress tree is represented-tree path order.

The rake children do not appear here because none of their represented vertices lies on this preferred path.

The off-path aggregate. Now consider everything in the cluster that is *not* on the spine.

There are two sources.

First, each compress child may itself have represented material hanging from its own part of the spine.

We therefore take

$$\text{sub}(\text{ch}_0 u) \quad \text{and} \quad \text{sub}(\text{ch}_1 u).$$

Second, each rake child lies entirely off the current spine. Consequently we want the *whole* cluster of each rake child, not merely its own off-path part. Hence

$$\text{all}(\text{ch}_2 u) \quad \text{and} \quad \text{all}(\text{ch}_3 u).$$

Putting these together,

$$\begin{aligned} \text{sub}(u) &= \text{sub}(\text{ch}_0 u) \oplus \text{sub}(\text{ch}_1 u) \\ &\quad \oplus \text{all}(\text{ch}_2 u) \oplus \text{all}(\text{ch}_3 u). \end{aligned}$$

The whole cluster. Since the path and off-path pieces are disjoint and together form the entire cluster,

$$\text{all}(u) = \text{path}(u) \oplus \text{sub}(u).$$

Missing children contribute the identity element.

Fake rake nodes. A fake rake node does not correspond to a represented-tree vertex and does not lie on a preferred path. Its only purpose is to balance a collection of virtual child clusters.

Thus it simply combines the represented material of its two children. If x is a fake rake node with children a, b , then

$$\text{all}(x) = \text{all}(a) \oplus \text{all}(b).$$

Ordered versus unordered aggregates. Along a preferred path there is a canonical order, so **path** may represent a noncommutative aggregate. As in an ordinary link-cut tree, in that case we store both

$$\text{path}^{\rightarrow} \quad \text{and} \quad \text{path}^{\leftarrow}$$

and rerooting can reverse the path.

Rake children, on the other hand, have no meaningful order. Any aggregate that combines independent virtual subtrees must therefore be commutative.

9.11.5 Two Kinds of Lazy Tags

The same path/off-path distinction must also be remembered by lazy updates.

Suppose an auxiliary cluster represents

spine $\sqcup$ material hanging from the spine.

An update might be intended for only the first part or only the second. We therefore maintain two tags:

plazy and slazy.

Here

plazy = pending update to the preferred-path spine,

while

slazy = pending update to the material off that spine.

Pushing a path tag. A path tag affects u and the parts of its compress children lying on the same spine. It does not affect any rake child.

Thus when a path tag is pushed through a real node, it is propagated as a path tag only to

$$\text{ch}_0(u), \quad \text{ch}_1(u).$$

The rake children are untouched.

This is exactly what we would expect: a path update follows the preferred path and nowhere else.

Pushing an off-path tag. An off-path tag is slightly more subtle.

Consider first a compress child c . The path of c is part of the parent's path as well. Therefore it is *not* part of the parent's off-path region. Only the material hanging from that child should receive the update.

Hence an off-path tag pushed into a compress child remains

slazy.

Now consider a rake child c .

The entire represented cluster of c hangs off the parent's path. Therefore *every represented vertex inside c* must receive the update.

But internally c again divides its material into

$$\text{path}(c) \quad \text{and} \quad \text{sub}(c).$$

Both pieces belong to the parent's off-path region.

Consequently, when an off-path update crosses a rake edge, it becomes an update to the whole child cluster, plazy and slazy.

9.11.6 Push Before Changing the Decomposition

There is one important consequence of this interpretation.

Lazy tags describe a set of *represented vertices*, not merely a set of auxiliary nodes. We must therefore never change which represented material is classified as compress or rake while an unpushed tag above it still depends on the old classification.

In particular, before removing a virtual child y , push all tags along the rake-root- y path.

After the removal, recompute aggregates on the way back up and suppress any fake rake node left with only one child.

9.11.7 Path Operations

Path operations are exactly as in an ordinary link-cut tree.

To expose

$$P(u, v),$$

perform

$$\boxed{\text{MakeRoot}(u), \quad \text{Access}(v).}$$

Afterward every edge of $P(u, v)$ is preferred, so the path is precisely the preferred-path spine of the auxiliary cluster rooted at v .

Hence

$$\boxed{\text{PathQuery}(u, v) = \text{path}(v).}$$

Likewise, a lazy update to $P(u, v)$ is recorded as a path update:

$$\boxed{\text{plazy}(v).}$$

9.11.8 Subtree Operations

Now fix a represented root r , and suppose we want the rooted subtree T_u .

Perform

$$\boxed{\text{MakeRoot}(r), \quad \text{Access}(u).}$$

Afterward the preferred path ending at u is exactly

$$P(r, u).$$

Since u is the bottom endpoint of this path, it has no preferred child below it.

Therefore every proper descendant of u lies in a virtual child cluster attached at u .

Meanwhile, the compress material on the other side of u is precisely the ancestor path

$$r \rightsquigarrow u,$$

which must not be included in T_u .

Thus the rooted subtree has the decomposition

$$\boxed{T_u = \{u\} \sqcup \{\text{all virtual-child clusters attached at } u\}.$$

If the two rake children of u are $\text{ch}_2 u$ and $\text{ch}_3 u$, then

$$\boxed{\text{SubtreeQuery}(r, u) = \text{val}(u) \oplus \text{all}(\text{ch}_2 u) \oplus \text{all}(\text{ch}_3 u).}$$

For a lazy update to the whole rooted subtree, apply the update to

$$\text{val}(u)$$

and to the complete clusters represented by the rake children.

9.11.9 Link, Cut, and Reroot

The dynamic-forest operations retain the same interpretation as in the underlying link-cut tree.

To link two represented trees by an edge uv , reroot the first tree at u , expose v , and attach the complete represented tree rooted at u as a new virtual child cluster of v .

To cut an existing represented edge uv , perform

$$\text{MakeRoot}(u), \quad \text{Access}(v).$$

The edge uv is now exposed in the compress structure and can be detached exactly as in the ordinary link-cut tree.

Rerooting remains

$\text{Access}(u)$ followed by a lazy reversal of the exposed path.

9.11.10 Amortized Analysis

Regard every real and fake auxiliary node as belonging to one forest for analysis. For a node x , let

$$\sigma(x) = \text{represented mass contained below } x,$$

and define its rank by

$$\rho(x) = \log_2 \sigma(x).$$

Use the usual splay potential

$\Phi = \sum_x \rho(x).$

A rotation inside a compress tree and a rotation inside a rake tree are both ordinary splay tree rotations. The weighted splay access lemma therefore gives, for either kind of splay,

$$\text{amortizedCost}(\text{Splay}(x)) \leq 1 + 3(\rho_{\text{after}}(x) - \rho_{\text{before}}(x)).$$

Now consider the successive clusters reached during an expose,

$$x_1, x_2, \dots, x_k.$$

After x_i has been processed, its represented cluster is contained in the larger cluster reached at the next step. Consequently

$$\sigma_{\text{after}}(x_i) \leq \sigma_{\text{before}}(x_{i+1}),$$

and hence

$$\rho_{\text{after}}(x_i) \leq \rho_{\text{before}}(x_{i+1}).$$

The rank increases telescope exactly as in the ordinary link-cut-tree analysis:

$$\sum_i (\rho_{\text{after}}(x_i) - \rho_{\text{before}}(x_i)) = O(\log n).$$

The important point is that moving a represented cluster between the compress and rake structures does not alter the represented material. The same vertices are still present; only the auxiliary decomposition used to store them has changed.

The additive +1 terms in the splay access bound are charged to the same preferred-path changes as in the ordinary link-cut tree. Inserting or removing a cluster from a rake tree happens precisely because a represented edge has changed between preferred and virtual status, so these are the same.

It follows that one expose costs

$$O(\log n)$$

amortized time.

Aggregate recomputation and lazy propagation take $O(1)$ work per visited real or fake auxiliary node and are therefore absorbed into the same bound.

Theorem 165 (Self-Adjusting Top Tree). *The augmented link-cut structure above uses*

$$O(n)$$

space and supports

link, cut, reroot,

together with compatible path and rooted-subtree queries and lazy updates, in

$$O(\log n)$$

amortized time.

Proof. There are n real nodes. Every fake node is an internal node of a binary rake tree, so the total number of fake nodes is also $O(n)$.

By the preceding potential argument, every expose costs $O(\log n)$ amortized time.

Each supported operation performs only a constant number of exposes, together with a constant number of extra cluster insertions, deletions, reversals, or lazy-tag operations.

Therefore every operation takes $O(\log n)$ amortized time and the data structure uses $O(n)$ space. $\square$

9.12 Lazy Anti-Monopoly Tree

The Lazy AM-Tree maintains an incremental MSF in $O(\log n)$ amortized time per insertion. It maintains a Transformed MST (T-MST), a rooted tree that minimizes depth while preserving the results of Path-Max queries.

TW-Transformations and Promotion A fundamental property of T-MSTs is the *TW-Transformation*: given two adjacent edges (x, y, w_1) and (y, z, w_2) with $w_1 \geq w_2$, replacing the edge (x, y, w_1) with (x, z, w_1) preserves all Path-Max queries in the tree.

The Promote(x) operation leverages this to move a node x up the tree while preserving Path-Max queries. Let $y = \text{parent}[x]$ and $z = \text{parent}[y]$, with edge weights $w_1 = \text{weight}(x \rightarrow y)$ and $w_2 = \text{weight}(y \rightarrow z)$.

- **Shortcut** ($w_1 > w_2$): x is directly promoted to become z 's child with weight w_1 . Node y becomes a sibling of x .
- **Rotate** ($w_1 < w_2$ or y is root): y is pushed down to become a child of x with weight w_1 . If y is not the root, x is attached to z with weight w_2 .

Stitch-Based Insertion Instead of finding the heaviest edge on the cycle, the Stitch-based algorithm moves the new edge (u, v, w) to its correct position using TW-transformations.

If $\text{weight}(u \rightarrow \text{parent}[u]) < w$, the edge (u, v, w) is TW-transformed to $(\text{parent}[u], v, w)$ and recursively moved upwards. Once both $\text{weight}[u \rightarrow \text{parent}[u]] > w$ and $\text{weight}[v \rightarrow \text{parent}[v]] > w$, we attach the smaller subtree to the larger one (e.g., u becomes a child of v with weight w). This displaces u 's old parent edge (u, u', w') . Because $w' > w$, we apply another TW-transformation, moving the displaced edge to (u', v, w') , and recursively route it.

If the original edge closed a cycle, the relocation forces the two endpoints to eventually meet at their LCA ($u = v$). At this point, we discard the edge.

Algorithm 20 Lazy AM-Tree

```
1: function PROMOTE( $x$ )
2:    $y \leftarrow \text{parent}[x]$ 
3:    $z \leftarrow \text{parent}[y]$ 
4:    $w_1 \leftarrow \text{weight}(x \rightarrow y); w_2 \leftarrow \text{weight}(y \rightarrow z)$ 
5:   if  $z \neq \text{null}$  and  $w_1 > w_2$  then
6:      $\text{size}[y] \leftarrow \text{size}[y] - \text{size}[x]$ 
7:      $\text{parent}[x] \leftarrow z$ 
8:      $\text{weight}(x \rightarrow \text{parent}) \leftarrow w_1$ 
9:   else
10:     $\text{size}[y] \leftarrow \text{size}[y] - \text{size}[x]$ 
11:     $\text{size}[x] \leftarrow \text{size}[x] + \text{size}[y]$ 
12:     $\text{parent}[x] \leftarrow z$ 
13:     $\text{weight}(x \rightarrow \text{parent}) \leftarrow w_2$ 
14:     $\text{parent}[y] \leftarrow x$ 
15:     $\text{weight}(y \rightarrow \text{parent}) \leftarrow w_1$ 
16: function UPWARDCALIBRATE( $x$ )
17:   while  $\text{parent}[x] \neq \text{null}$  do
18:      $y \leftarrow \text{parent}[x]$ 
19:     if  $\text{size}[x] \leq \frac{2}{3} \text{size}[y]$  then
20:        $x \leftarrow y$ 
21:     else
22:       PROMOTE( $x$ )
23: function LINKBYSTITCH( $u, v, w$ )
24:   if  $u = v$  or  $u = \text{null}$  or  $v = \text{null}$  then
25:     return
26:   if  $\text{parent}[u] \neq \text{null}$  and  $w > \text{weight}(u \rightarrow \text{parent})$  then
27:     LINKBYSTITCH( $\text{parent}[u], v, w$ )
28:   else if  $\text{parent}[v] \neq \text{null}$  and  $w > \text{weight}(v \rightarrow \text{parent})$  then
29:     LINKBYSTITCH( $u, \text{parent}[v], w$ )
30:   else
31:     if  $\text{size}[u] > \text{size}[v]$  then
32:       Swap  $u \leftrightarrow v$ 
33:        $u_{\text{old\_parent}} \leftarrow \text{parent}[u]$ 
34:        $w_{\text{old}} \leftarrow \text{weight}(u \rightarrow \text{parent})$ 
35:        $\text{parent}[u] \leftarrow v$ 
36:        $\text{weight}(u \rightarrow \text{parent}) \leftarrow w$ 
37:        $\text{size}[v] \leftarrow \text{size}[v] + \text{size}[u]$ 
38:     LINKBYSTITCH( $u_{\text{old\_parent}}, v, w_{\text{old}}$ )
39: function LAZYINSERT( $u, v, w$ )
40:   UPWARDCALIBRATE( $u$ )
41:   UPWARDCALIBRATE( $v$ )
42:   LINKBYSTITCH( $u, v, w$ )
```

▷ Shortcut: x bypasses y to become its sibling

▷ Rotate: y is pushed down as a child of x

▷ Route the displaced heavier edge

Correctness and Complexity

Theorem 166. *The Lazy AM-Tree maintains a valid T-MST and supports insertions in $O(\log n)$ amortized time.*

Proof. Correctness: Every operation in LinkByStitch is a valid TW-transformation. By continually replacing edges with equivalent higher-reaching edges, the sequence preserves Path-Max queries. If u and v were previously connected, the endpoints of the heaviest cycle-edge move to their LCA, where $u = v$ triggers the base case that drops the redundant edge. Therefore, the tree remains valid.

Complexity: We use the potential function $\Phi = \sum_{i=1}^n \log(\text{size}[i])$.

1. **Calibrate:** The Promote function is only called when x is a heavy child ($\text{size}[x] > \frac{2}{3}\text{size}[y]$). Under this condition, rotating x upwards decreases y 's size by at least a factor of $1/3$, meaning Φ decreases by an amount strictly greater than the $O(1)$ actual cost of the rotation. Therefore, Promote has an amortized cost of zero. Because the loop only advances upward when the AM-rule holds, size increases by a factor of $3/2$ per advancement, limiting iterations to $O(\log n)$.
2. **Link:** The LinkByStitch recursion traverses upward, performing $O(d(u) + d(v))$ actual operations. Structural size changes only occur when the smaller subtree is attached to the larger one. Because the smaller subtree's size at most doubles, the potential Φ increases by at most 1 per topological attachment. Across the $O(d(u) + d(v))$ affected nodes, the total potential increase is bounded by $O(\log n)$.

Thus the total amortized cost per insertion is $O(\log n)$. □

10 Graph Algorithms

10.1 Incremental SCCs

Suppose directed edges

$$e_1, e_2, \dots, e_m$$

are inserted in this order, and let G_t denote the graph containing the first t edges.

Strongly connected components only merge as t increases. Thus for every pair u, v , there is a threshold

$$\tau(u, v) = \min\{t : u, v \text{ lie in the same SCC of } G_t\},$$

with $\tau(u, v) = m + 1$ if they never become strongly connected.

In particular, if $e_t = (u_t, v_t)$, then e_t strictly changes the SCC partition exactly when

$\tau(u_t, v_t) = t.$

Indeed, if the endpoints were already strongly connected before e_t arrived, then adding e_t cannot change any reachability relation: whenever a path uses the new edge $u_t \rightarrow v_t$, we may replace it by the old path from u_t to v_t inside their SCC. Conversely, if the endpoints first become strongly connected at time t , the SCC partition has clearly changed.

Thus it is enough to compute these m threshold values simultaneously.

Divide and Conquer on Time The basic idea is binary search, with one important optimization.

Suppose we are considering a time interval $[l, r]$, and let $h = \lfloor (l + r)/2 \rfloor$. Compute the SCCs of the graph at time h .

For a query pair u, v ,

- if u, v are already in the same SCC at time h , then $\tau(u, v) \leq h$, so the query goes left;
- otherwise $\tau(u, v) > h$, so the query goes right.

If we did this independently at every recursive node, we would repeatedly process the same old edges. The crucial observation is that we can simplify the graph separately for the two recursive calls.

For the left half, an edge joining two different SCCs at time h is irrelevant. Such an edge cannot participate in any directed cycle at any earlier time: if it did, its endpoints would already belong to the same SCC at time h .

For the right half, every SCC present at time h will remain strongly connected forever. We may therefore contract each such SCC to a single vertex.

Thus every relevant edge is sent to only one child.

Algorithm 21 Offline Incremental SCCs

Input: Vertices V , chronological edges $E = \langle e_1, \dots, e_m \rangle$

Output: The edges whose insertion strictly changes the SCC partition

```

1:  $Q \leftarrow \langle (i, u_i, v_i) : e_i = (u_i, v_i) \rangle$ 
2:  $\tau_i \leftarrow m + 1$  for every  $i$ 
3:  $A \leftarrow \langle (i, u_i, v_i) : e_i = (u_i, v_i) \rangle$ 
4:  $\text{SOLVE}(0, m + 1, V, A, Q, \tau)$ 
5:  $M \leftarrow \langle \rangle$ 
6: for  $i = 1, \dots, m$  do
7:   if  $\tau_i = i$  then
8:      $\text{append } e_i \text{ to } M$ 
9: return  $M$ 
10: function  $\text{SOLVE}(l, r, V', A, Q, \tau)$ 
11:   if  $l = r$  then
12:     for  $(i, u, v) \in Q$  do
13:        $\tau_i \leftarrow l$ 
14:   return
15:    $h \leftarrow \lfloor (l + r) / 2 \rfloor$ 
                                      $\triangleright$  SCCs at the midpoint using edges already present by time  $h$ 
16:    $C \leftarrow \text{SCC}(V', \langle (t, u, v) : (t, u, v) \in A, t \leq h \rangle)$ 
17:    $A_L, A_R, Q_L, Q_R \leftarrow \langle \rangle$ 
18:   for  $(t, u, v) \in A$  do
19:     if  $t \leq h$  and  $C[u] = C[v]$  then
20:        $\text{append } (t, u, v) \text{ to } A_L$ 
21:     else if  $C[u] \neq C[v]$  then
22:        $\text{append } (t, C[u], C[v]) \text{ to } A_R$ 
                                      $\triangleright$  If  $t > h$  and  $C[u] = C[v]$ , this future edge is already internal and can be discarded
23:   for  $(i, u, v) \in Q$  do
24:     if  $C[u] = C[v]$  then
25:        $\text{append } (i, u, v) \text{ to } Q_L$ 
26:     else
27:        $\text{append } (i, C[u], C[v]) \text{ to } Q_R$ 
28:    $V_R \leftarrow \{C[v] : v \in V'\}$ 
29:    $\text{SOLVE}(l, h, V', A_L, Q_L, \tau)$ 
30:    $\text{SOLVE}(h + 1, r, V_R, A_R, Q_R, \tau)$ 

```

Why the Edge Reduction is Correct The query routing is immediate from monotonicity of SCCs. The only issue is showing that the reduced edge sets still contain all information needed by the two recursive calls.

Let C be the SCC partition at time h .

For the left call, consider an edge $e = (u, v)$ present by time h with $C[u] \neq C[v]$. This edge cannot affect the SCC partition at any time $t \leq h$. Indeed, if it belonged to a directed cycle at such a time, then u and v

would be mutually reachable at time t , and hence also at time h , contradicting $C[u] \neq C[v]$. More generally, the edges between different C -components form the condensation DAG, and deleting edges of this DAG does not alter the SCCs themselves.

Hence the left recursion needs only the edges already present by time h whose endpoints lie in the same C -component.

For the right call, every C -component is already strongly connected at time h , so it remains strongly connected at every later time. Contracting it is therefore exact. Edges already internal to a C -component can be discarded. Every other edge is represented simply by the edge between the corresponding contracted components.

Notice also that if an edge arrives only after time h but its endpoints already satisfy $C[u] = C[v]$, then it can be discarded permanently: when it eventually arrives, its endpoints are already strongly connected, so it can never change the SCC partition.

These are precisely the rules used by [Algorithm 21](#).

Theorem 167. [Algorithm 21](#) returns exactly the edges whose insertion strictly changes the strongly connected component partition, in

$$O(m \log m)$$

time, apart from the initial $O(|V|)$ cost of reading isolated vertices.

Proof. We prove recursively that every query (i, u, v) passed to a call $\text{SOLVE}(l, r, \dots)$ satisfies

$$\tau(u, v) \in [l, r],$$

where the vertices are interpreted after undoing the contractions made on the path from the root.

At a midpoint h , the preceding discussion shows that the SCC partition computed from the reduced graph is exactly the SCC partition of G_h , modulo the contractions already made.

If $C[u] = C[v]$, then the two query vertices are strongly connected by time h , so their threshold lies in $[l, h]$. Otherwise their threshold is strictly larger than h , so it lies in $[h + 1, r]$. Thus every query is sent to the child containing its threshold.

The edge-reduction argument above proves that the reduced graph passed to each child has exactly the same SCC partitions, throughout that child's time interval, as the corresponding original graphs. Hence the invariant continues to hold recursively.

At a leaf $l = r$, the only possible threshold is l , so assigning $\tau_i = l$ is correct.

Finally, as observed at the beginning, $e_i = (u_i, v_i)$ changes the SCC partition exactly when $\tau(u_i, v_i) = i$. Therefore the returned list M is correct.

For the running time, implement each recursive call only on vertices appearing as endpoints of its active edges or queries. Computing SCCs and partitioning the lists then takes linear time in the size of that call.

Every active edge is passed to at most one child, and every query is passed to exactly one child. Hence the total size of all calls at any fixed recursion depth is $O(m)$. The recursion has $O(\log m)$ levels, giving total time

$$O(m \log m).$$

□

10.2 All-Pairs Connectivity

Let

$$G = (V, E)$$

be a directed graph with

$$m = |E|.$$

For vertices s, t , let

$$\lambda(s, t)$$

be the maximum number of pairwise edge-disjoint directed s – t paths.

There is a direct algebraic representation of all these values at once.

10.2.1 A Walk Generating Matrix

Index rows and columns by directed edges.

For every composable pair

$$e, f \in E, \quad \text{head}(e) = \text{tail}(f),$$

introduce a variable

$$x_{ef}.$$

Define

$$X \in \mathbb{F}[x]^{E \times E}$$

by

$$X[e, f] = \begin{cases} x_{ef}, & \text{head}(e) = \text{tail}(f), \\ 0, & \text{otherwise.} \end{cases}$$

Work first over the formal power-series ring. Then

$$\Gamma = (I - X)^{-1} = I + X + X^2 + \cdots$$

If

$$W = (e_1, \dots, e_\ell)$$

is a directed walk, define

$$\xi(W) = \prod_{j=1}^{\ell-1} x_{e_j, e_{j+1}}.$$

Lemma 168 (Walk Enumeration). *For edges e, f ,*

$$\Gamma[e, f] = \sum_{W: e \rightsquigarrow f} \xi(W),$$

where the sum ranges over directed walks beginning with e and ending with f .

Proof. The entry

$$X^j[e, f]$$

is exactly the sum of the monomials of walks consisting of $j + 1$ edges, starting at e and ending at f . Sum over $j \geq 0$. $\square$

10.2.2 Determinants Enumerate Edge-Disjoint Walks

Let

$$S = \{e_1, \dots, e_r\}, \quad T = \{f_1, \dots, f_r\}$$

be equally sized edge sets.

Expand

$$\det \Gamma[S, T].$$

Each term chooses

- a permutation $\sigma \in S_r$;
- for every i , a walk from e_i to $f_{\sigma(i)}$.

Thus it is a signed generating function for families of walks.

We now work in characteristic 2.

Lemma 169 (Walk Cancellation). *Over characteristic 2,*

$$\det \Gamma[S, T]$$

is the generating function of pairwise edge-disjoint walk families connecting S to T . In particular,

$$\boxed{\det \Gamma[S, T] \neq 0}$$

if and only if there exist r pairwise edge-disjoint directed paths joining the starting edges in S to the ending edges in T in some pairing.

Proof. Consider a family in which two walks share an edge. Fix an ordering of walks and positions, and choose canonically the first shared edge

$$h$$

and the first pair of walks containing it.

Write the two walks as

$$P = P^- h P^+, \quad Q = Q^- h Q^+.$$

Swap the suffixes:

$$P' = P^- h Q^+, \quad Q' = Q^- h P^+.$$

The endpoint permutation is composed with a transposition. More importantly, the multiset of transition variables

$$x_{ef}$$

is unchanged, so the two families have exactly the same monomial.

The operation is an involution. In characteristic 2, the two identical contributions cancel.

Thus only pairwise edge-disjoint families remain.

For an edge-disjoint family, its transition monomial together with its starting edges reconstructs every walk, so distinct surviving families have distinct monomials and cannot cancel.

Finally, any edge-disjoint walk can have directed cycles deleted without creating an edge intersection, giving edge-disjoint paths. $\square$

10.2.3 Connectivity is Matrix Rank

Let

$$E^{\text{out}}(s) = \{e : \text{tail}(e) = s\}$$

and

$$E^{\text{in}}(t) = \{e : \text{head}(e) = t\}.$$

Theorem 170 (Connectivity via Rank). *Over the field of rational functions,*

$$\boxed{\lambda(s, t) = \text{rank } \Gamma[E^{\text{out}}(s), E^{\text{in}}(t)]}.$$

Proof. A matrix has rank at least r exactly when it has a nonzero $r \times r$ minor.

By Lemma 169, such a minor is nonzero exactly when there are r pairwise edge-disjoint directed s - t paths.

The largest possible r is therefore

$$\lambda(s, t).$$

$\square$

Thus all-pairs connectivity is encoded by one matrix:

$$\boxed{(I - X)^{-1}}.$$

10.2.4 Random Evaluation

We cannot manipulate the rational functions explicitly. Instead choose a sufficiently large finite field

$$\mathbb{F}_{2^q}, \quad q = \Theta(\log n),$$

and substitute independent uniformly random field elements for all variables x_{ef} .

Let

$$A$$

be the resulting numerical matrix.

By Schwartz–Zippel, with high probability,

- $I - A$ is invertible;
- every minor relevant to any pair (s, t) is zero after evaluation exactly when it was identically zero.

Hence simultaneously for every s, t ,

$$\lambda(s, t) = \text{rank} \left((I - A)^{-1} [E^{\text{out}}(s), E^{\text{in}}(t)] \right)$$

with high probability.

Algorithm 22 All-Pairs Directed Edge Connectivity

- 1: Choose random transition weights $A[e, f] \in \mathbb{F}_{2^q}$ whenever $\text{head}(e) = \text{tail}(f)$
 - 2: $M \leftarrow (I - A)^{-1}$
 - 3: **for** $s, t \in V$ **do**
 - 4: $\lambda(s, t) \leftarrow \text{rank } M[E^{\text{out}}(s), E^{\text{in}}(t)]$
 - 5: **return** all $\lambda(s, t)$
-

Theorem 171. *Algorithm 22 computes every directed edge-connectivity with high probability in*

$$\tilde{O}(m^\omega)$$

time.

Proof. The matrix inversion costs

$$O(m^\omega)$$

field operations.

The rank of an $a \times b$ matrix can be computed in

$$O(ab^{\omega-1})$$

operations after exchanging a, b if necessary. Summing these costs over all pairs (s, t) and using

$$\sum_s \deg^+(s) = \sum_t \deg^-(t) = m$$

is bounded by

$$O(m^\omega).$$

The field operations cost only polylogarithmic time. □

10.2.5 Bounded All-Pairs Connectivity

Suppose now that we only want

$$\lambda_k(s, t) = \min\{k, \lambda(s, t)\}$$

for every pair s, t .

The algorithm above uses an $m \times m$ inverse. For small k , this contains much more information than necessary. We can compress the transition matrix to rank $O(kn)$ while preserving all minors of size at most k .

There are two steps:

1. modify the graph so every queried source has outdegree k and every queried target has indegree k ;
2. factor the edge-transition matrix through only k copies of each vertex.

Degree Reduction Construct a graph G' as follows. For every original vertex v , create two additional vertices v_{in} and v_{out} .

Replace every original edge

$$(u, v)$$

by

$$(u_{\text{out}}, v_{\text{in}}).$$

Also add k parallel copies of

$$v \rightarrow v_{\text{out}} \quad \text{and} \quad v_{\text{in}} \rightarrow v.$$

Thus every original vertex has exactly k outgoing edges and exactly k incoming edges in G' .

The new graph has

$$n' = 3n, \quad m' = m + 2kn.$$

Lemma 172 (Degree Reduction). *For original vertices s, t ,*

$$\lambda_{G'}(s, t) = \min\{k, \lambda_G(s, t)\}.$$

Proof. Any directed s - t path in G' projects to a directed s - t path in G by retaining only the original vertices. Furthermore, edge-disjoint paths in G' project to edge-disjoint paths in G , because every transformed original edge $u_{\text{out}} \rightarrow v_{\text{in}}$ corresponds to a unique original edge $u \rightarrow v$.

Hence

$$\lambda_{G'}(s, t) \leq \lambda_G(s, t).$$

Also s has only k outgoing edges in G' , so

$$\lambda_{G'}(s, t) \leq k.$$

For the reverse inequality, let

$$r = \min\{k, \lambda_G(s, t)\},$$

and choose r edge-disjoint s - t paths $P_1, \dots, P_r$ in G .

Lift P_i to G' by replacing every original edge $u \rightarrow v$ by

$$u \rightarrow u_{\text{out}} \rightarrow v_{\text{in}} \rightarrow v,$$

using the i -th parallel copy of each gadget edge $u \rightarrow u_{\text{out}}$ and $v_{\text{in}} \rightarrow v$.

The transformed original edges remain disjoint because the P_i were edge-disjoint, and the gadget edges are disjoint because the paths use different copy indices i . Thus the lifted paths are edge-disjoint in G' , proving

$$\lambda_{G'}(s, t) \geq r.$$

□

A Low-Rank Transition Matrix We now apply the walk-generating construction to G' .

Instead of giving every composable pair of edges e, f an independent variable x_{ef} , factor this transition through one of k copies of their common vertex.

For every edge e , vertex v , and $j \in [k]$, introduce variables $y_{e,j}$ and $z_{j,e}$. Define

$$Y \in \mathbb{F}[y]^{E' \times (V' \times [k])}$$

by

$$Y[e, (v, j)] = \begin{cases} y_{e,j}, & \text{head}(e) = v, \\ 0, & \text{otherwise,} \end{cases}$$

and define

$$Z \in \mathbb{F}[z]^{(V' \times [k]) \times E'}$$

by

$$Z[(v, j), f] = \begin{cases} z_{j,f}, & \text{tail}(f) = v, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$(YZ)[e, f] = \begin{cases} \sum_{j=1}^k y_{e,j} z_{j,f}, & \text{head}(e) = \text{tail}(f), \\ 0, & \text{otherwise.} \end{cases}$$

Thus YZ is obtained from the old transition matrix X by the substitution

$$x_{ef} \mapsto \sum_{j=1}^k y_{e,j} z_{j,f}.$$

The advantage is that

$$Y : \mathbb{F}^{kn'} \rightarrow \mathbb{F}^{m'}, \quad Z : \mathbb{F}^{m'} \rightarrow \mathbb{F}^{kn'},$$

so

$$\text{rank}(YZ) \leq kn' = O(kn).$$

Lemma 173 (Low-Rank Path Detection). *Let $S, T \subseteq E'$ have*

$$|S| = |T| = r \leq k.$$

Then

$$\boxed{\det(I - YZ)^{-1}[S, T] \neq 0}$$

if and only if there are r pairwise edge-disjoint directed paths joining S to T .

Proof. Start with the exact generating function

$$\det(I - X)^{-1}[S, T].$$

By [Lemma 169](#), this is nonzero exactly when there are r edge-disjoint paths from S to T . If no such paths exist, this formal power series is identically zero. Substituting

$$x_{ef} = \sum_{j=1}^k y_{e,j} z_{j,f}$$

cannot turn the zero series into a nonzero one. Since this substitution sends X to YZ , we obtain

$$\det(I - YZ)^{-1}[S, T] \equiv 0.$$

Conversely, suppose

$$P_1, \dots, P_r$$

are edge-disjoint paths from S to T . Since $r \leq k$, assign a different label i to each path P_i . If

$$P_i = (e_{i,1}, e_{i,2}, \dots, e_{i,\ell_i}),$$

then in every transition $e_{i,a} \rightarrow e_{i,a+1}$, choose the $j = i$ term from

$$\sum_{j=1}^k y_{e_{i,a},j} z_{j,e_{i,a+1}}.$$

This produces the monomial

$$\mu = \prod_{i=1}^r \prod_{a=1}^{\ell_i-1} y_{e_{i,a},i} z_{i,e_{i,a+1}}.$$

Because the paths are edge-disjoint and use distinct labels, this monomial records the edges belonging to each individual path. The starting and ending edges are fixed by S and T , so the corresponding path family is recovered uniquely. Hence μ cannot be cancelled by another surviving family.

Therefore its coefficient is nonzero, and

$$\det(I - YZ)^{-1}[S, T] \neq 0.$$

□

Reducing the Matrix Inversion Now use the inverse identity

$$(I - YZ)^{-1} = I + Y(I - ZY)^{-1}Z$$

from [Theorem 21](#).

The left side is an $m' \times m'$ matrix, but the only matrix we actually need to invert on the right is

$$I - ZY,$$

whose dimension is

$$kn' = 3kn.$$

This is the source of the speedup.

Random Evaluation Choose a sufficiently large field $\mathbb{F}_{2^q}$, with $q = \Theta(\log n)$, and independently evaluate all y - and z -variables at uniformly random field elements.

Let the resulting numerical matrices be

$$B \in \mathbb{F}^{m' \times kn'}, \quad C \in \mathbb{F}^{kn' \times m'}.$$

Let E^{out} be the union of the k edges leaving each *original* vertex, and let E^{in} be the union of the k edges entering each original vertex. Define

$$\tilde{B} = B[E^{\text{out}}, \cdot], \quad \tilde{C} = C[\cdot, E^{\text{in}}],$$

and compute

$$M = \tilde{B}(I - CB)^{-1}\tilde{C}.$$

For original vertices s, t , the sets $E^{\text{out}}(s)$ and $E^{\text{in}}(t)$ are disjoint, so the identity term in

$$(I - BC)^{-1} = I + B(I - CB)^{-1}C$$

vanishes on the corresponding submatrix. Hence

$$(I - BC)^{-1}[E^{\text{out}}(s), E^{\text{in}}(t)] = M[E^{\text{out}}(s), E^{\text{in}}(t)].$$

By [Lemma 173](#) and Schwartz–Zippel, with high probability every minor of size at most k needed below has the same zero/nonzero status after the random evaluation as it had formally. Therefore, simultaneously for every original s, t ,

$$\text{rank } M[E^{\text{out}}(s), E^{\text{in}}(t)] = \min\{k, \lambda_G(s, t)\}.$$

Algorithm 23 k -Bounded All-Pairs Directed Edge Connectivity

- 1: Construct the degree-reduced graph G'
 - 2: Randomly evaluate the low-rank matrices Y, Z to B, C
 - 3: $M \leftarrow \tilde{B}(I - CB)^{-1}\tilde{C}$
 - 4: **for** $s, t \in V$ **do**
 - 5: $\lambda_k(s, t) \leftarrow \text{rank } M[E^{\text{out}}(s), E^{\text{in}}(t)]$
 - 6: **return** all $\lambda_k(s, t)$
-

Theorem 174 (Bounded All-Pairs Connectivity). *For a simple directed graph on n vertices and any $1 \leq k < n$, all values*

$$\min\{k, \lambda(s, t)\}$$

can be computed with high probability in

$$\boxed{\tilde{O}((kn)^\omega)}$$

time.

Proof. Correctness follows from Lemmas 172 and 173 and the random-evaluation argument above.

It remains to bound the running time.

The matrix CB has dimension

$$kn' \times kn' = 3kn \times 3kn.$$

Its entries have the form

$$(CB)[(u, i), (v, j)] = \sum_{\substack{e \in E' \\ \text{tail}(e)=u \\ \text{head}(e)=v}} c_{i,e} b_{e,j}.$$

Because of the special structure of G' , this matrix can be assembled in $O((kn)^2)$ field operations: between non-original gadget vertices there is at most one transformed original edge, while the only parallel classes have exactly k edges.

We can therefore compute

$$(I - CB)^{-1}$$

in

$$O((kn)^\omega)$$

field operations. Multiplying on the left by $\tilde{B}$ and on the right by $\tilde{C}$ has the same asymptotic cost. Finally, each queried block

$$M[E^{\text{out}}(s), E^{\text{in}}(t)]$$

is only $k \times k$. Computing its rank costs $O(k^\omega)$ field operations, so all n^2 ranks cost

$$O(k^\omega n^2) \leq O((kn)^\omega).$$

All field elements have $O(\log n)$ bits, giving the stated $\tilde{O}((kn)^\omega)$ bit complexity. $\square$

10.3 Flows

10.3.1 Highest Label Push-Relabel

We analyze the highest-level selection rule for the preflow-push algorithm.

Let $K = \sqrt{m}$. For any node v , define $d'(v) = |\{w : d(w) \leq d(v)\}|$. Thus $d'(v)$ is the number of nodes with a distance label no greater than v 's label. We define the potential function Φ as:

$$\Phi = \sum_{v \text{ active}} \frac{d'(v)}{K}$$

Initially, $\Phi \leq n^2/K$, and $\Phi \geq 0$ throughout the execution. Let us examine how the operations affect Φ :

- **Relabeling:** A relabeling of a node v increases Φ by at most n/K , because $d'(v)$ may increase to at most n , while $d'(w)$ for all other nodes $w \neq v$ does not increase.
- **Saturating push:** This operation creates at most one new active node, increasing Φ by at most n/K .
- **Nonsaturating push:** A nonsaturating push from v to w deactivates v and activates w (if it is not already active). Since the push only happens on eligible edges where $d(w) < d(v)$, we know $d'(w) \leq d'(v)$, meaning Φ does not increase.

To bound the number of nonsaturating pushes, we divide the execution into phases. A phase consists of all pushes between two consecutive changes of the maximum active node label, $d^* = \max\{d(v) : v \text{ is active}\}$. We classify a phase as *cheap* if it contains at most K nonsaturating pushes, and *expensive* otherwise.

Claim 175. *The number of phases is at most $4n^2$, and the number of nonsaturating pushes in cheap phases is at most $4n^2K$.*

Proof. Initially, $d^* = 0$, and any increase in d^* is strictly due to a relabeling. The number of relabelings is bounded by $2n^2$. Thus, d^* increases at most $2n^2$ times and consequently decreases at most $2n^2$ times, limiting the total number of changes (and thus phases) to $4n^2$. The bound on cheap pushes follows immediately since each cheap phase has at most K pushes. $\square$

Claim 176. *An expensive phase containing $Q > K$ nonsaturating pushes decreases Φ by at least Q .*

Proof. During an expensive phase, d^* remains constant, meaning all Q nonsaturating pushes originate from nodes at level d^* . The phase ends either when all nodes at level d^* are deactivated, or a node is relabeled to $d^* + 1$. Therefore, level d^* must contain $Q > K$ nodes (active or inactive) at all times during the phase. For any nonsaturating push along an edge (u, v) , because there are at least K nodes at level $d(u)$, each push decreases Φ by at least one. $\square$

The total increase of Φ is bounded by the sum of increases from relabelings and saturating pushes. There are at most $2n^2$ relabelings and $2nm$ saturating pushes, so the total increase is at most $(2n^2 + 2nm)(n/K)$. Because $\Phi \geq 0$, the total decrease is bounded by the total increase plus the initial Φ value (n^2/K) .

This bounds the number of nonsaturating pushes in expensive phases by $(2n^3 + 2n^2m + n^2)/K \leq (3n^3 + 2n^2m)/K$. Adding the cheap pushes, the total number of nonsaturating pushes is at most $(3n^3 + 2n^2m)/K + 4n^2K$. By setting $K = \sqrt{m}$ and noting $n = O(m)$, this evaluates to $O(n^2\sqrt{m})$.

Theorem 177. *The running time of the preflow-push algorithm using the highest-level selection rule is $O(n^2\sqrt{m})$.*

Proof. Each push and relabel operation can be implemented in $O(1)$ time. The highest-level selection rule can be implemented using a bucket data structure where each bucket i holds all nodes w with $d(w) = i$. Maintaining this structure incurs a total running time overhead of $O(nm)$. Therefore, the algorithm's running time is dominated by the $O(n^2\sqrt{m})$ nonsaturating pushes, resulting in an overall time complexity of $O(n^2\sqrt{m})$. $\square$

10.3.2 Flow Rounding

Flow rounding converts a fractional circulation into an integral one without worsening the overall cost. We do this by eliminating fractional cycles.

Availability and Cycle Cancelling To maintain edge capacity constraints while pushing flow around cycles, the algorithm relies on the concept of availability. The availability of a directed edge (u, v) is the amount of flow that can be pushed before the flow reaches the next integer bound.

Link-Cut Trees To efficiently detect cycles and push flow, the algorithm maintains a forest of the current fractional edges using a Link-Cut Tree. It supports querying the minimum availability along a path, calculating path costs, and pushing flow along paths in $O(\log n)$ time.

Algorithm 24 Rounding Fractional Circulations

Input: Graph $G = (V, E)$, Original fractional circulation f

Output: Integral circulation f' with equal or lesser cost

```

1: function ROUNDFLOW( $G, f$ )
2:    $f' \leftarrow f$ 
3:   Initialize empty Link-Cut Tree forest  $G' = (V, E' = \emptyset)$ 
4:   for each edge  $(u, v) \in E$  with fractional  $f'(u, v)$  do
5:     if FINDROOT( $u$ )  $\neq$  FINDROOT( $v$ ) then
6:       LINK( $u, v$ )  $\triangleright$  Edge does not form a cycle
7:     else
8:        $\triangleright$  A fractional cycle is formed by  $(u, v)$  and the  $v \rightarrow u$  tree path
9:        $P \leftarrow$  Path between  $v$  and  $u$  in  $G'$ 
10:      Determine cycle direction (forward or reverse) that yields  $\leq 0$  cost
11:       $\triangleright$  Push flow until at least one edge becomes integral
12:       $\delta \leftarrow \min(\text{FINDMIN}(u, v), \text{AVAILABLE}(v, u))$ 
13:      PUSHFLOW( $u, v, \delta$ )  $\triangleright$  Update flow along cycle
14:       $\triangleright$  Remove all newly integral edges from the forest
15:      while ( $e_{\text{int}} \leftarrow \text{FINDINTEGRALEDGE}(u, v)$ )  $\neq$  null do
16:        CUT( $e_{\text{int}}$ )
17:        if  $f'(u, v)$  is still fractional then
18:          LINK( $u, v$ )
19:   return  $f'$ 

```

Correctness

Theorem 178. *The algorithm correctly produces an integral circulation of no worse cost in $O(m \log n)$ time.*

Proof. By evaluating the cost of the path in the Link-Cut tree, we can always push flow around a newly formed cycle in a direction where the total cost is non-positive, ensuring the overall circulation cost never increases. At termination, every fractional edge has been evaluated. The remaining edges in the Link-Cut tree explicitly form a forest, meaning the final circulation f' must be integral. $\square$

Lemma 179. *Let f be an integral circulation on a digraph D and d any positive integer. Then f can be written as the sum $g_1 + g_2 + \dots + g_d$ of integral circulations such that for each index j and each edge e :*

$$\lfloor f(e)/d \rfloor \leq g_j(e) \leq \lceil f(e)/d \rceil$$

Proof Sketch. We proceed by induction on d . Round $\frac{f}{d}$ to some integral circulation g , then consider $f - g = g_2 + \dots + g_d$ inductively, then the bounds we get for $g_2, \dots, g_d$ imply also that they satisfy the needed bounds. $\square$

10.4 Arboricity

The k -disjoint forest partition algorithm determines whether the edges of a weighted multigraph G can be partitioned into k edge-disjoint forests. By binary searching over k , this algorithm can compute the exact arboricity of a graph. It operates in $O(n(n + m)^2 \log(kn))$ time using a series of max-flow computations.

Nash-Williams Theorem The Nash-Williams Theorem (refer to Diestel for a simple proof of the stronger Packing-Covering Theorem and the implication) states that a multigraph can be partitioned into k edge-disjoint forests if and only if for every non-empty subset of vertices $U \subseteq V$, the sum of the weights of the edges induced by U satisfies:

$$c(E(U)) \leq k(|U| - 1)$$

Network Construction For a chosen "root" vertex $r \in V$, we construct a bipartite flow network to test all subsets U containing r . The network consists of a source s , a sink t , a layer of vertex-nodes representing V , and a layer of edge-nodes representing E .

1. **Source to Vertices:** A directed edge from s to every vertex $v \in V \setminus \{r\}$ with capacity k .
2. **Vertices to Edges:** For every edge $e = (u, v)$ in the original graph, we add directed edges from node u to node e , and from node v to node e , both with capacity ∞ .
3. **Edges to Sink:** A directed edge from every edge-node e to t with capacity $c(e)$ (the weight of the edge).

Consider an arbitrary finite-capacity $s - t$ cut, partitioned into sets S and T , let $U = V \cap T$.

Because the directed edges from vertices to edge-nodes have infinite capacity, placing an edge-node in T while one of its endpoints remains in S would cut infinite capacity. Therefore, the only edge-nodes in T are in $E(U)$ and to minimize the cut capacity for a given U , we let exactly $E(U)$ be on the sink side.

The cut edges come from:

1. Edges from s to the vertices in $U \setminus \{r\}$.
2. Edges from $E \setminus E(U)$ to the sink t .

And the total capacity is:

$$\text{Cut Capacity} = k|U \setminus \{r\}| + c(E) - c(E(U))$$

For the maximum flow to satisfy the total edge weight $c(E)$, every cut must have a capacity of at least $c(E)$. Thus:

$$k|U \setminus \{r\}| + c(E) - c(E(U)) \geq c(E) \implies c(E(U)) \leq k|U \setminus \{r\}| = k(|U| - 1)$$

Algorithm 25 k -Disjoint Forest Partition

Input: Graph V , edges E with weights c_e , integer k

Output: True if E can be partitioned into $\leq k$ forests, False otherwise

```
1: function ISKFORESTPARTITIONABLE( $V, E, k$ )
2:    $n \leftarrow |V|$ ;  $m \leftarrow |E|$ 
3:    $C_{total} \leftarrow \sum_{e \in E} c_e$ 
4:   if  $C_{total} > k(n-1)$  then
5:     return False
6:   for  $r \leftarrow 0$  to  $n-1$  do
7:     Initialize flow network  $G_{flow}$  with source  $s$ , sink  $t$ 
8:     for  $v \in V$  do
9:       if  $v \neq r$  then
10:        Add edge  $s \rightarrow v$  with capacity  $k$ 
11:     for each edge  $e = (u, v) \in E$  do
12:       Add edge  $u \rightarrow e$  with capacity  $\infty$ 
13:       Add edge  $v \rightarrow e$  with capacity  $\infty$ 
14:       Add edge  $e \rightarrow t$  with capacity  $c_e$ 
15:      $flow \leftarrow \text{DINITZSCALING}(G_{flow}, s, t)$ 
16:     if  $flow < C_{total}$  then
17:       return False
18: return True
```

Complexity

Theorem 180. *The algorithm correctly verifies the k -forest partition property in $O(n(n+m)^2 \log(kn))$ time.*

Proof. We use Dinitz's algorithm with capacity scaling, which finds a maximum flow in $O(VE \log U)$ time. For our network, the number of vertices is $V = n + m + 2$ and the number of edges is $E = n + 3m$. The maximum capacity connected to the source is $U \leq kn$. Thus we get a runtime of $O((n+m)^2 \log(kn))$. Because we execute this flow algorithm n times (once for each root r), the overall time complexity is $O(n(n+m)^2 \log(kn))$. $\square$

10.5 Directed Minimum Spanning Tree

Let

$$G = (V, E)$$

be a directed graph with edge weights $w : E \rightarrow \mathbb{R}$, and fix a root $r \in V$.

An r -arborescence is a set of $n-1$ edges such that every $v \neq r$ has exactly one incoming edge and every vertex is reachable from r .

We want a minimum-weight r -arborescence.

10.5.1 The Two Reductions

We first isolate the two transformations on which the algorithm is based.

Lemma 181 (Incoming-Edge Reduction). *Let X be a non-root vertex of the current contracted graph, and let*

$$\delta = \min_{e \in \delta^-(X)} w(e).$$

Subtract δ from the weight of every edge entering X .

Then the minimum arborescence weight decreases by exactly δ .

Proof. Every rooted arborescence contains exactly one edge entering X . Thus every feasible solution loses exactly δ in weight. $\square$

After the reduction, at least one incoming edge to X has weight zero.

Choose one such zero-weight edge for every non-root vertex. If the chosen edges are acyclic, they form an optimal arborescence.

Otherwise they contain a directed cycle.

Lemma 182 (Zero-Cycle Contraction). *Suppose*

$$C = X_1 X_2 \cdots X_k X_1$$

is a directed cycle of chosen zero-weight edges. Contract C to one supervertex.

The minimum residual arborescence weight is unchanged.

Proof. Consider any arborescence of the original graph. At least one edge enters C from outside.

Suppose an entering edge enters X_i . Delete all other edges entering vertices of C and instead include every zero-weight cycle edge except the cycle edge entering X_i . This preserves connectivity and indegree one at every vertex of C and does not increase the weight.

Hence some optimum contains exactly one external edge entering C .

Such solutions correspond exactly to arborescences after contracting C : if the contracted solution enters the supervertex through an edge whose original head is X_i , expand C and add every zero cycle edge except the one entering X_i . The weights are identical. $\square$

10.5.2 Meldable Incoming-Edge Heaps

For each current contracted component X , maintain a heap

$$H_X$$

containing the original edges whose heads lie in X .

An edge

$$e = (u, v)$$

stored in H_X may later become internal because its tail is also contracted into X . We do not eagerly delete such edges. Instead, whenever an internal edge reaches the minimum of the heap, we discard it.

Thus we need the following heap operations:

operation	meaning
$\text{Min}(H)$	minimum reduced-cost edge
$\text{Pop}(H)$	delete the minimum
$\text{AddAll}(H, -\delta)$	subtract δ from every key
$\text{Meld}(H_1, H_2)$	merge two incoming-edge heaps.

A skew heap with one lazy additive tag per heap node supports these operations in amortized $O(\log m)$ time, with AddAll taking $O(1)$ time.

When the minimum incoming edge of X has reduced weight δ , we perform

$$\text{AddAll}(H_X, -\delta).$$

When a cycle is contracted,

$$H_C = \text{Meld}(H_{X_1}, \dots, H_{X_k}).$$

Edges that have become internal are discarded lazily from the top.

10.5.3 Detecting Cycles with DSU

Maintain the already chosen zero edges as an undirected forest in a disjoint set union structure D_{cyc} .

The important invariant is that before a current component X chooses its incoming edge, X is the unique vertex of its chosen-edge component with no chosen incoming edge. Thus the chosen edges in that component form an arborescence directed away from X .

Suppose the chosen minimum edge is

$$e = (Y, X).$$

If X and Y lie in different components of D_{cyc} , adding e cannot create a cycle, so we simply union them.

If they already lie in the same component, the unique chosen-edge path from X to Y is directed

$$X \rightsquigarrow Y.$$

Adding

$$Y \rightarrow X$$

therefore creates exactly one directed cycle.

Hence an ordinary DSU detects the cycles.

10.5.4 Maintaining Contractions

A second DSU,

$$D_{\text{ctr}},$$

represents the currently contracted vertices.

For every original vertex v ,

$$D_{\text{ctr}}.\text{find}(v)$$

is its present supervertex.

When a chosen edge

$$e = (u, v)$$

is inspected, its current tail is therefore

$$D_{\text{ctr}}.\text{find}(u).$$

When a cycle is found, union all its vertices in D_{ctr} and meld their heaps.

We make D_{ctr} rollback-capable. Before contracting a cycle, record the current rollback time. This lets us expand all contractions in reverse order once the optimum weight has been found.

10.5.5 Fast Directed MST Algorithm

For a current component X , let

$$e_X$$

denote the zero reduced-cost incoming edge currently chosen for X .

The algorithm processes a component as follows.

Algorithm 26 Tarjan's Directed MST Algorithm

Input: Directed graph $G = (V, E)$ and root r

Output: Minimum r -arborescence, or None

```
1: put every edge  $(u, v)$  into the meldable heap  $H_v$ 
2: initialize  $D_{\text{cyc}}$ 
3: initialize rollback DSU  $D_{\text{ctr}}$ 
4:  $\text{answer} \leftarrow 0$ 
5:  $\mathcal{C} \leftarrow ()$   $\triangleright$  contraction records
6: for each original vertex  $s \neq r$  do
7:    $X \leftarrow s$ 
8:   loop
9:     while  $H_X \neq \emptyset$  and  $\text{tail}(\text{Min}(H_X))$  is currently inside  $X$  do
10:       $\text{Pop}(H_X)$ 
11:      if  $H_X = \emptyset$  then
12:         $\text{return None}$ 
13:       $e_X \leftarrow \text{Min}(H_X)$ 
14:       $\delta \leftarrow w_{\text{red}}(e_X)$ 
15:       $\text{answer} \leftarrow \text{answer} + \delta$ 
16:       $\text{AddAll}(H_X, -\delta)$ 
17:       $Y \leftarrow D_{\text{ctr}}.\text{find}(\text{tail}(e_X))$ 
18:      if  $D_{\text{cyc}}.\text{union}(X, Y)$  succeeds then
19:         $\text{break}$ 
20:      record rollback time  $\tau$  of  $D_{\text{ctr}}$ 
21:      find the directed chosen-edge cycle containing  $X$ 
22:      contract all its components into a new component  $Z$ 
23:      meld all their incoming-edge heaps into  $H_Z$ 
24:      append the contraction information and  $\tau$  to  $\mathcal{C}$ 
25:    $X \leftarrow Z$ 
26: expand the contractions in reverse order
27: return  $\text{answer}$  and the resulting parent edges
```

10.5.6 Correctness

Theorem 183. *Algorithm 26 returns a minimum-weight rooted arborescence whenever one exists.*

Proof. Maintain the invariant

$$\text{OPT}(G_{\text{original}}) = \text{answer} + \text{OPT}(G_{\text{current}}).$$

Initially this is immediate.

Suppose we process a current non-root component X , whose cheapest external incoming edge has reduced weight δ . Subtracting δ from every edge entering X decreases every feasible arborescence by exactly δ by [Lemma 181](#). Adding δ to answer therefore preserves the invariant.

The chosen edge now has reduced weight zero.

If no cycle is created, no contraction is required.

If a zero-weight directed cycle is created, [Lemma 182](#) shows that contracting it leaves the residual optimum unchanged. Thus the invariant again survives.

Eventually all current non-root components have chosen incoming edges and no chosen cycle remains. Those edges form an arborescence of residual cost zero. Therefore

$$\text{OPT}(G_{\text{current}}) = 0,$$

and

$$answer = \text{OPT}(G_{\text{original}}).$$

It remains to recover the actual edges. Expand contractions in reverse order. Suppose a contracted cycle is entered by the already reconstructed edge f , whose original head lies in cycle component X_i . Restore every selected zero cycle edge except the one entering X_i , and use f instead. By the proof of [Lemma 182](#), this preserves an arborescence and its weight.

Repeating through all contractions recovers an optimum arborescence of the original graph. $\square$

10.5.7 Rollback Reconstruction

When contracting a cycle, save its DSU snapshot τ .

During reverse processing, the cycle is still one supervertex. Its currently selected incoming edge tells us which original member of the cycle receives the external edge.

Roll back to τ . The cycle members are now distinct again. If the external edge enters X_i , restore the old cycle edge for every other member and replace precisely the old edge entering X_i .

Thus every contraction requires only its rollback snapshot and the chosen edges already stored by the algorithm; the contracted graph never needs to be explicitly rebuilt.

10.5.8 Running Time

Theorem 184 (Tarjan). *Using lazy skew heaps and disjoint-set union, [Algorithm 26](#) runs in*

$$\boxed{O(m \log n)}$$

time and $O(m + n)$ space.

Proof. Each original edge is inserted into exactly one heap.

Once an edge becomes internal to a contracted component, it remains internal forever. Thus every edge is popped at most once.

A contraction permanently reduces the number of current components, so there are fewer than n contractions and only $O(n)$ successful contraction-DSU unions.

Every heap is formed only by melding heaps whose components are contracted. Hence there are $O(m + n)$ heap operations in total.

Lazy subtraction costs $O(1)$. Meld and pop in a skew heap cost $O(\log m) = O(\log n)$ amortized on the relevant sparse representation, and the DSU work is also at most this bound.

Therefore the total running time is

$$O(m \log n).$$

The reverse rollback pass performs only $O(n)$ unions/rollbacks and edge changes. $\square$

After reconstruction, if $e_v = (u, v)$ is the selected incoming edge of $v \neq r$, one may return simply

$$\text{parent}[v] = u, \quad \text{parent}[r] = r.$$

10.6 Matching and Coloring

10.6.1 Hall's Marriage Theorem

Theorem 185 (Hall). *Let $G = (L \sqcup R, E)$ be bipartite. There is a matching saturating L if and only if*

$$\boxed{|N(S)| \geq |S| \quad \text{for every } S \subseteq L.}$$

Proof. Necessity is immediate: if a matching saturates S , its matched neighbors are $|S|$ distinct elements of $N(S)$.

For sufficiency, let M be a maximum matching. Suppose some $x_0 \in L$ is unmatched.

Let $S \subseteq L$ and $T \subseteq R$ be the vertices reachable from x_0 by alternating paths, starting with an edge outside M .

No vertex of T is unmatched, since otherwise the alternating path to it would be an augmenting path, contradicting maximality of M .

Moreover,

$$\boxed{T = N(S)}.$$

Indeed, every reached right vertex is clearly a neighbor of S . Conversely, from any $x \in S$, every nonmatching edge can extend the alternating path to its other endpoint; the unique matching edge incident to x , if any, already lies on the alternating path.

Finally, M gives a bijection

$$T \longleftrightarrow S \setminus \{x_0\}.$$

Every $y \in T$ is matched, and its matched partner is reachable and hence lies in $S \setminus \{x_0\}$; conversely every $x \in S \setminus \{x_0\}$ was reached through its matching edge.

Therefore

$$|N(S)| = |T| = |S| - 1,$$

contradicting Hall's condition. Hence M saturates L . □

10.6.2 Gale–Ryser Theorem

Hall's theorem generalizes naturally from choosing one neighbor for every left vertex to prescribing all degrees on both sides.

Theorem 186 (Bipartite Degree Realization). *Let $G = (L \sqcup R, E)$ be bipartite, and let*

$$a : L \rightarrow \mathbb{Z}_{\geq 0}, \quad b : R \rightarrow \mathbb{Z}_{\geq 0}$$

satisfy

$$\sum_{u \in L} a_u = \sum_{v \in R} b_v = D.$$

There is a subgraph $H \subseteq G$ with

$$\deg_H(u) = a_u \quad (u \in L), \quad \deg_H(v) = b_v \quad (v \in R)$$

if and only if for every $X \subseteq L$ and $Y \subseteq R$,

$$\boxed{a(X) \leq b(Y) + |E(X, R \setminus Y)|}.$$

Proof. Construct the network

$$s \longrightarrow L \longrightarrow R \longrightarrow t$$

with capacities

$$c(s, u) = a_u, \quad c(u, v) = 1 \quad (uv \in E), \quad c(v, t) = b_v.$$

An integral flow of value D chooses exactly a_u edges incident to every $u \in L$ and exactly b_v edges incident to every $v \in R$. Thus such a flow is exactly the desired subgraph.

By max-flow min-cut, it remains to characterize when every cut has capacity at least D .

Any cut is determined by sets $X \subseteq L$ and $Y \subseteq R$ placed on the source side. Its capacity is

$$a(L \setminus X) + |E(X, R \setminus Y)| + b(Y).$$

Since $a(L) = D$, this is at least D exactly when

$$a(X) \leq b(Y) + |E(X, R \setminus Y)|.$$

Thus the stated inequalities are precisely the min-cut conditions. Integrality of max flow completes the proof. $\square$

10.6.3 Bipartite Regular Matching

Given a d -regular bipartite graph $G = (P, Q, E)$ with $|P| = |Q| = n$, a perfect matching can be found in $O(n \log n)$ expected time.

The algorithm iteratively augments a partial matching M by discovering augmenting paths via random walks. If M currently leaves k nodes unmatched in both P and Q , we implicitly construct a directed matching graph H :

1. Orient all edges of G from P to Q .
2. Contract each matched edge pair $(u, v) \in M$ into a single supernode.
3. Add a source vertex s connected by d parallel edges to each unmatched node in P , directed outward.
4. Add a sink vertex t connected by d parallel edges from each unmatched node in Q , directed inward.

Any $s \rightarrow t$ path in H corresponds directly to a valid augmenting path in G with respect to M . The algorithm simply performs a standard random walk starting from s , choosing an outgoing edge uniformly at random at each step, until it hits t .

Lemma 187 (Random Walk Hitting Time). *Given a partial matching leaving k vertices unmatched, the expected number of steps before a random walk started at s reaches t is at most $2 + \frac{n}{k}$.*

Proof. Construct a modified graph H^* by identifying the source s and sink t into a single unified node s^* . In H^* , the out-degree of every vertex equals its in-degree, making it an Eulerian digraph.

The out-degree of the unified node s^* is kd , while the out-degree of every other node is $d - 1$. The expected time for a random walk in H^* starting at s^* to return to s^* is exactly the inverse of its stationary probability π_{s^*} . Because H^* is balanced, this stationary probability is simply the ratio of the degree of s^* to the sum of all degrees in H^* . Evaluating this yields an expected return time of:

$$\frac{1}{\pi_{s^*}} = \frac{n(d-1) + 2kd + kd}{kd} \leq 2 + \frac{n}{k}$$

Running a walk from s^* until it returns to s^* in H^* is identical to running a walk from s until it hits t in the original matching graph H . $\square$

To construct a perfect matching, the algorithm starts with an empty matching ($k = n$) and repeatedly runs untruncated random walks to augment the matching n times until $k = 0$. By the lemma, the total expected number of steps taken across all random walks is bounded by:

$$\sum_{k=1}^n \left(2 + \frac{n}{k}\right) = 2n + nH_n = O(n \log n)$$

The same runtime with high probability can easily be achieved by terminating the walks if they take too long and repeating. We then use Markov's inequality and a Chernoff bound.

10.6.4 Bipartite Edge Coloring

Let

$$G = (L \sqcup R, E)$$

be a bipartite multigraph with maximum degree

$$\Delta.$$

König's line-coloring theorem states that

$$\chi'(G) = \Delta.$$

We give an algorithmic proof which constructs a proper Δ -edge-coloring in

$$O(m \log m)$$

time.

The algorithm has three ingredients:

1. reduce to a regular bipartite multigraph of only $O(m)$ edges;
2. split any even-regular multigraph into two equal-degree regular subgraphs using Euler tours;
3. use the splitting operation to find a perfect matching, and then recursively color the graph.

Regularization The first reduction is useful because a proper edge-coloring of a graph obtained by identifying vertices on the same side remains proper when those vertices are separated again.

Lemma 188 (Regularization). *Let G be a nonempty bipartite multigraph with m edges and maximum degree Δ . In $O(m)$ time we can construct a Δ -regular bipartite multigraph G' with*

$$O(m)$$

edges such that any proper Δ -edge-coloring of G' restricts to a proper Δ -edge-coloring of G .

Proof. Consider one side of the bipartition. Greedily combine original vertices into groups, keeping the sum of degrees in each group at most Δ . When the next vertex would make the current group exceed Δ , start a new group.

Let the resulting group loads be

$$d_1, \dots, d_q \leq \Delta.$$

Whenever group $i + 1$ was started,

$$d_i + d_{i+1} > \Delta,$$

because the first vertex placed in group $i + 1$ already failed to fit in group i . Hence, pairing consecutive groups,

$$q = O\left(\frac{m}{\Delta} + 1\right).$$

Do this independently on both sides.

Identify all vertices belonging to one group. Every resulting vertex still has degree at most Δ . A coloring of the identified graph is also valid before identification, because identifying vertices only introduces additional constraints.

Add isolated dummy vertices to the smaller side until both sides contain the same number N of vertices. Since the degree sum on each side is m , both sides have the same total deficit

$$N\Delta - m.$$

Repeatedly join a deficient left vertex to a deficient right vertex until all degrees equal Δ .

The final graph has

$$N\Delta = O(m + \Delta) = O(m)$$

edges, since $\Delta \leq m$. Deleting the dummy edges and undoing the identifications gives a coloring of the original graph. $\square$

Euler Splitting The fundamental operation is especially simple.

Lemma 189 (Euler Split). *Let H be a $2d$ -regular multigraph. Then its edges can be partitioned into two d -regular spanning subgraphs*

$$H_0, H_1$$

in linear time.

Proof. Every connected component of H is Eulerian. Take an Euler tour of each component and alternately color its edges red and blue.

At every vertex, every entrance in the Euler tour is paired with the following exit, and the two edges receive opposite colors. Thus exactly half of the $2d$ incident edges are red and half are blue. $\square$

The same operation works efficiently when an edge e is stored with a multiplicity $m(e)$ rather than explicitly storing all copies.

Lemma 190 (Euler Split with Multiplicities). *Suppose a $2d$ -regular multigraph is represented by multiplicities*

$$m(e)$$

on a set F of distinct edges. It can be split into two d -regular spanning multigraphs in

$$O(|F|)$$

time.

Proof. For every edge e , place

$$\left\lfloor \frac{m(e)}{2} \right\rfloor$$

copies in each side.

One copy remains precisely when $m(e)$ is odd. At every vertex, the number of remaining copies is even, because the original degree is even and we removed an even number of incident copies. Hence the graph consisting of the remaining copies is Eulerian.

Take Euler tours of this residual graph and alternate its edges between the two sides. Each side therefore receives exactly half of every vertex's residual degree, and consequently has degree d . Only the distinct edges need to be inspected. $\square$

Lemma 191 (Perfect Matching in a Regular Bipartite Multigraph). *Let G be a k -regular bipartite multigraph with N vertices on each side and*

$$m = Nk$$

edges. A perfect matching can be found in

$$O(m \log m)$$

time.

Proof. Let

$$Q = 2^t$$

be the smallest power of two satisfying

$$Q \geq m.$$

Write

$$Q = \alpha k + \beta, \quad 0 \leq \beta < k.$$

Choose an arbitrary bijection between the two vertex classes, and let M_0 be the corresponding perfect matching in the complete bipartite graph. The edges of M_0 need not belong to G .

Construct a multigraph H by

- replacing every edge of G by α parallel copies;
- adding β copies of every edge of M_0 .

Call the latter copies *bad*. Every vertex now has degree

$$\alpha k + \beta = Q,$$

so H is Q -regular.

The initial number of bad edges is

$$N\beta < Nk = m \leq Q.$$

Since Q is a power of two, repeatedly apply [Lemma 190](#). At each split, retain the half containing fewer bad edges. Thus after j splits, the retained graph is

$$Q/2^j$$

regular and contains fewer than

$$Q/2^j$$

bad edges.

After t splits it is 1-regular and contains fewer than one bad edge. Hence it contains no bad edge at all.

A 1-regular spanning bipartite graph is a perfect matching, and all of its edges are copies of edges of G . Discarding multiplicities therefore gives a perfect matching of G .

At every level there are only

$$O(m)$$

distinct edges to process, and

$$t = O(\log m).$$

□

It is tempting to take Q to be merely the smallest power of two with $Q \geq k$, pad G to a Q -regular graph, and then repeatedly halve. The problem is that the padding introduces bad edges at *every* vertex.

Indeed, writing $Q = \alpha k + \beta$, the construction adds β bad matching edges at each of the N vertices on one side, for a total of $N\beta$ bad edges. At each Euler split we can keep the half containing fewer bad edges, so after j splits we only know that the number of bad edges has fallen by a factor 2^j . When the degree has finally fallen from Q to 1, this gives only

$$\# \text{bad edges} \leq \frac{N\beta}{Q}.$$

If Q is only $O(k)$, this can still be $\Theta(N)$.

Coloring a Power-of-Two Regular Graph

Lemma 192. *If G is 2^t -regular, then it can be properly edge-colored with exactly 2^t colors in*

$$O(m \log 2^t)$$

time.

Proof. Repeatedly Euler-split every current regular graph into two equal-degree subgraphs. After t levels, every leaf is 1-regular, hence a matching. Give each leaf a different color.

At each level the subgraphs partition the original edge set, so the total work per level is $O(m)$. □

The General Recursion We can now color a k -regular graph using exactly k colors.

Suppose first that k is even, and put

$$d = k/2.$$

Euler-split

$$G = G_1 \sqcup G_2,$$

where both graphs are d -regular.

Recursively color G_1 with colors

$$1, \dots, d.$$

Each color class is a perfect matching.

Let

$$q$$

be the smallest power of two satisfying

$$q \geq d.$$

Since $q < 2d = k$ unless $d = q$, we need exactly

$$q - d$$

additional perfect matchings to make G_2 q -regular.

Take any $q - d$ color classes from G_1 and temporarily move them into G_2 . The resulting graph

$$H$$

is q -regular, so [Lemma 192](#) colors it with q colors.

The part of G_1 that was not moved contains

$$d - (q - d) = 2d - q = k - q$$

of its old color classes. Give these $k - q$ matchings colors disjoint from those used on H .

Altogether we use exactly

$$q + (k - q) = k$$

colors.

If k is odd, first find a perfect matching using [Lemma 191](#), give it one color, remove it, and apply the even-degree procedure to the remaining $(k - 1)$ -regular graph.

Algorithm 27 Edge Coloring a Regular Bipartite Multigraph

```

1: function COLORREGULAR( $G, k$ )
2:   if  $k = 0$  then
3:     return
4:   if  $k = 1$  then
5:     give all edges of  $G$  one new color
6:     return
7:   if  $k$  is odd then
8:      $M \leftarrow \text{PERFECTMATCHING}(G)$ 
9:     give  $M$  one new color
10:     $G \leftarrow G - M$ 
11:     $k \leftarrow k - 1$ 
12:   $d \leftarrow k/2$ 
13:   $(G_1, G_2) \leftarrow \text{EULERSPLIT}(G)$ 
14:  COLORREGULAR( $G_1, d$ )
15:   $q \leftarrow$  least power of two with  $q \geq d$ 
16:  move any  $q - d$  color classes of  $G_1$  into  $G_2$ 
17:   $H \leftarrow$  the resulting  $q$ -regular graph
18:  color  $H$  with  $q$  fresh colors by repeated Euler splitting
19:  retain the remaining  $k - q$  old color classes of  $G_1$ 

```

Theorem 193 (König’s Line-Coloring Theorem, Algorithmic Form). *Every bipartite multigraph of maximum degree Δ has a proper Δ -edge-coloring. Such a coloring can be found in*

$$O(m \log m)$$

time.

Proof. Apply [Lemma 188](#), and let $M = O(m)$ be the number of edges after regularization.

Correctness follows from the construction above.

For the running time, consider a recursive call on a k -regular graph with M' edges. Everything outside the recursive call costs

$$O(M' \log M) :$$

Euler splitting is linear, power-of-two coloring costs $O(M' \log k)$, and when k is odd the perfect matching costs $O(M' \log M)$.

The recursive call is made on a graph with half as many edges. Thus

$$T(M') \leq T(M'/2) + O(M' \log M),$$

and therefore

$$T(M) = O(M \log M) = O(m \log m).$$

Finally discard dummy edges and undo the identified vertices. □

10.6.5 Chromatic Number

Let

$$G = (V, E), \quad |V| = n.$$

For

$$S \subseteq V,$$

let

$$I(S) = |\{T \subseteq S : T \text{ is independent}\}|.$$

The key inclusion–exclusion identity is extremely simple.

Theorem 194 (Coloring by Independent-Set Counts). *For $k \geq 1$, define*

$$C_k = \sum_{S \subseteq V} (-1)^{n-|S|} I(S)^k.$$

Then

$$G \text{ is } k\text{-colorable} \iff C_k > 0.$$

Proof. The quantity

$$I(S)^k$$

counts ordered tuples

$$(A_1, \dots, A_k)$$

of independent subsets of S .

Consider one fixed tuple of independent subsets of V . Its contribution to C_k is

$$\sum_{\substack{S \subseteq V \\ A_1 \cup \dots \cup A_k \subseteq S}} (-1)^{n-|S|}.$$

This equals

$$\begin{cases} 1, & A_1 \cup \dots \cup A_k = V, \\ 0, & \text{otherwise,} \end{cases}$$

by inclusion–exclusion.

Hence C_k counts ordered k -tuples of independent sets whose union is all of V .

Such a tuple exists exactly when G is k -colorable: given a coloring, take its color classes; conversely, from a covering tuple, assign each vertex to any independent set containing it. $\square$

Define an array

$$e(S)$$

initially by

$$e(\{u, v\}) = 1 \quad \text{for every edge } \{u, v\} \in E,$$

and 0 elsewhere.

Apply the subset zeta transform:

$$E(S) = \sum_{T \subseteq S} e(T).$$

Then

$$E(S)$$

is exactly the number of graph edges contained in S .

Thus

$$a(S) = \begin{cases} 1, & E(S) = 0, \\ 0, & E(S) > 0 \end{cases}$$

is the indicator that S is independent.

Apply the subset zeta transform a second time:

$$I(S) = \sum_{T \subseteq S} a(T).$$

This computes every independent-subset count in

$$O(n2^n)$$

arithmetic operations.

Algorithm 28 Exact Chromatic Number

```

1: initialize  $e(S) \leftarrow 0$  for all  $S \subseteq V$ 
2: for each edge  $\{u, v\} \in E$  do
3:    $e(\{u, v\}) \leftarrow 1$ 
4:  $E \leftarrow \zeta_{\downarrow} e$ 
5: for  $S \subseteq V$  do
6:    $a(S) \leftarrow \mathbf{1}[E(S) = 0]$ 
7:  $I \leftarrow \zeta_{\downarrow} a$ 
8:  $P(S) \leftarrow 1$  for all  $S$ 
9: for  $k = 1, \dots, n$  do
10:  for  $S \subseteq V$  do
11:     $P(S) \leftarrow P(S)I(S)$ 
12:   $C_k \leftarrow \sum_{S \subseteq V} (-1)^{n-|S|} P(S)$ 
13:  if  $C_k \neq 0$  then
14:    return  $k$ 

```

Theorem 195. *The chromatic number of an n -vertex graph can be computed in*

$$O(n2^n)$$

arithmetic operations and

$$O(2^n)$$

space.

10.7 Node-Weighted Triangle

Let $G = (A \sqcup B \sqcup C, E)$ be tripartite, with a weight

$$w(v)$$

on every vertex. The *Node-Weighted Triangle* problem asks whether there is a triangle

$$a \in A, \quad b \in B, \quad c \in C$$

such that

$$w(a) + w(b) + w(c) = 0.$$

There is no loss in assuming the graph is tripartite. Given an arbitrary graph, make three copies of its vertex set and put an edge between two copies exactly when the corresponding original vertices are adjacent. Triangles are preserved up to a constant multiplicity.

The main point is that unequal multiplicities of the vertex weights do not make the problem harder than ordinary triangle detection.

10.7.1 One Fixed Weight

The basic case is immediate.

Lemma 196 (One Fixed Weight). *Suppose every vertex of A has the same weight u . Then Node-Weighted Triangle can be solved in*

$$O(\text{MM}(|A|, |B|, |C|))$$

time, up to linear input-processing time.

Proof. For every edge $bc \in E(B, C)$, keep it only if

$$u + w(b) + w(c) = 0.$$

Then a triangle

$$a, b, c$$

in the remaining graph satisfies

$$w(a) + w(b) + w(c) = u + w(b) + w(c) = 0.$$

Conversely every zero-weight triangle survives.

Thus the problem has become ordinary tripartite triangle detection, which is one rectangular matrix multiplication. $\square$

So the whole problem is to partition the vertices into instances in which one side has a fixed weight, without paying too much for the partition.

10.7.2 The Square Case

Assume

$$|A|, |B|, |C| \leq n.$$

For a weight u , write

$$f_A(u) = |\{a \in A : w(a) = u\}|,$$

and define f_B similarly.

We first process triangles for which the A -weight is at least as frequent as the B -weight.

Fix an A -weight u , and put

$$k = f_A(u).$$

Only consider B -weights v satisfying

$$f_B(v) \leq k.$$

Greedy partition these weights into groups P such that

$$\sum_{v \in P} f_B(v) \leq 2k,$$

and every group except possibly the last has total frequency at least k . This is possible because every individual weight has frequency at most k .

For a group P , define

$$\begin{aligned} A_u &= \{a \in A : w(a) = u\}, \\ B_P &= \{b \in B : w(b) \in P\}, \end{aligned}$$

and

$$C_P = \{c \in C : -u - w(c) \in P\}.$$

Every zero-weight triangle whose A -vertex has weight u and whose B -weight lies in P is contained in

$$A_u \times B_P \times C_P.$$

Moreover,

$$|A_u| = k, \quad |B_P| \leq 2k.$$

Split C_P into blocks of at most $2k$ vertices and apply [Lemma 196](#) to every block.

Theorem 197 (Node-Weighted Triangle). *Node-Weighted Triangle on a tripartite graph with $O(n)$ vertices in each part can be solved in*

$$\boxed{O(n^\omega)}$$

time, the same time as ordinary triangle detection.

Proof. Fix an A -weight u of frequency

$$k = f_A(u).$$

Because every completed B -group contains at least k vertices, there are

$$O(n/k)$$

groups.

Also, every $c \in C$ belongs to at most one set C_P , because its required complementary B -weight

$$-u - w(c)$$

belongs to at most one group. Hence

$$\sum_P |C_P| \leq n.$$

After splitting the C_P 's into $O(k)$ -sized blocks, the total number of blocks is therefore

$$O(n/k).$$

Each block has dimensions

$$O(k) \times O(k) \times O(k),$$

so it costs

$$O(k^\omega)$$

to process. Thus the total cost for the fixed weight u is

$$\boxed{O(n k^{\omega-1})}.$$

Now sum over all distinct weights u in A . Since

$$\sum_u f_A(u) = |A| \leq n$$

and $\omega \geq 2$,

$$k^{\omega-1} \leq n^{\omega-2} k.$$

Therefore

$$\begin{aligned} \sum_u n f_A(u)^{\omega-1} &\leq n^1 n^{\omega-2} \sum_u f_A(u) \\ &= n^\omega. \end{aligned}$$

This detects every zero-weight triangle satisfying

$$f_A(w(a)) \geq f_B(w(b)).$$

Run the same construction with A and B interchanged. Every triangle satisfies one of

$$f_A(w(a)) \geq f_B(w(b)), \quad f_B(w(b)) > f_A(w(a)),$$

so every zero-weight triangle is detected.

The second orientation has the same running time. □

The balancing principle is simple. A weight occurring k times gives a matrix dimension of order k , but there can be only $O(n/k)$ relevant blocks. Since matrix multiplication grows at least quadratically with the dimension, these effects balance.

10.7.3 Rectangular Version

Suppose the three parts have sizes

$$|A| \leq n^a, \quad |B| \leq n^b, \quad |C| \leq n^c,$$

and let

$$\omega = \omega(a, b, c).$$

For an A -weight u , write

$$f_A(u) = |\{x \in A : w(x) = u\}|,$$

and similarly define $f_B(v)$.

Since A and B have different sizes, absolute frequencies should not be compared directly. Define their normalized frequencies by

$$\rho_A(u) = f_A(u)^{1/a}, \quad \rho_B(v) = f_B(v)^{1/b}.$$

Thus

$$f_A(u) = \rho_A(u)^a, \quad f_B(v) = \rho_B(v)^b.$$

The point is that a normalized frequency ρ naturally gives matrix dimensions

$$\rho^a, \quad \rho^b, \quad \rho^c,$$

whose multiplication cost is

$$O(\rho^\omega).$$

Theorem 198 (Rectangular Node-Weighted Triangle). *Node-Weighted Triangle on a tripartite graph with part sizes*

$$n^a, \quad n^b, \quad n^c$$

can be solved in

$$O(n^{\omega(a,b,c)})$$

time.

Proof. We first find every zero-weight triangle satisfying

$$\rho_A(w(x)) \geq \rho_B(w(y)).$$

Fix an A -weight u , and set

$$k = f_A(u), \quad \rho = k^{1/a}.$$

Thus

$$k = \rho^a.$$

Consider exactly those B -weights v for which

$$f_B(v) \leq \rho^b.$$

Greedily partition these weights into groups P such that

$$\sum_{v \in P} f_B(v) \leq 2\rho^b,$$

and every completed group has total frequency at least ρ^b .

For each group P , define

$$A_u = \{x \in A : w(x) = u\},$$

$$B_P = \{y \in B : w(y) \in P\},$$

and

$$C_P = \{z \in C : -u - w(z) \in P\}.$$

Then

$$|A_u| = \rho^a, \quad |B_P| = O(\rho^b).$$

Split every C_P into blocks of size at most

$$\rho^c.$$

On each resulting instance, all vertices of A_u have the same weight, so [Lemma 196](#) reduces it to ordinary triangle detection with dimensions

$$O(\rho^a) \times O(\rho^b) \times O(\rho^c).$$

By the definition of $\omega(a, b, c)$, each such instance costs

$$O(\rho^{\omega(a, b, c)}).$$

We now count the number of instances for this fixed u .

Since every completed B -group contains at least ρ^b vertices, there are

$$O\left(\frac{n^b}{\rho^b}\right)$$

groups.

Moreover, for fixed u , the sets C_P are disjoint: a vertex z determines the unique complementary weight

$$-u - w(z).$$

Hence

$$\sum_P |C_P| \leq n^c.$$

After splitting into ρ^c -sized blocks, the total number of blocks is therefore

$$O\left(\frac{n^b}{\rho^b} + \frac{n^c}{\rho^c}\right).$$

Thus the total cost associated with u is

$$O\left(n^b \rho^{\omega(a, b, c) - b + \varepsilon} + n^c \rho^{\omega(a, b, c) - c + \varepsilon}\right).$$

Now sum this directly over all A -weights u .

Since

$$\rho^a = f_A(u)$$

and

$$\sum_u \rho^a = \sum_u f_A(u) = |A| \leq n^a,$$

we can charge both terms to this sum.

First, the trivial input-size lower bound

$$\omega(a, b, c) \geq a + b$$

gives

$$\omega(a, b, c) - a - b \geq 0.$$

Since $\rho \leq n$,

$$\rho^{\omega(a,b,c)-b} = \rho^a \rho^{\omega(a,b,c)-a-b} \leq \rho^a n^{\omega(a,b,c)-a-b}.$$

Therefore

$$\begin{aligned} \sum_u n^b \rho^{\omega(a,b,c)-b} &\leq n^b n^{\omega(a,b,c)-a-b} \sum_u \rho^a \\ &\leq n^b n^{\omega(a,b,c)-a-b} n^a \\ &= n^{\omega(a,b,c)}. \end{aligned}$$

Likewise,

$$\omega(a, b, c) \geq a + c,$$

so

$$\begin{aligned} \sum_u n^c \rho^{\omega(a,b,c)-c} &\leq n^c n^{\omega(a,b,c)-a-c} \sum_u \rho^a \\ &\leq n^{\omega(a,b,c)}. \end{aligned}$$

Hence this orientation costs

$$O(n^{\omega(a,b,c)}).$$

It finds every zero-weight triangle x, y, z satisfying

$$\rho_A(w(x)) \geq \rho_B(w(y)),$$

because when $u = w(x)$, the weight $w(y)$ is among the allowed B -weights, and the group containing it also contains z in its corresponding set C_P .

Run the same construction with A and B interchanged. This finds all remaining triangles. Rectangular matrix multiplication has the elementary symmetry

$$\omega(b, a, c) = \omega(a, b, c),$$

so the second run has the same complexity. □

10.7.4 Minimum Node-Weighted Triangle

Suppose now that the vertex weights are integers satisfying

$$|w(v)| \leq W,$$

and we want a triangle of minimum total weight rather than merely a zero-weight triangle.

Only a logarithmic number of NWT calls are needed.

Lemma 199 (Testing a Target). *Suppose $G = (A \sqcup B \sqcup C, E)$ is tripartite. For any integer t , we can test whether G contains a triangle of total weight exactly t using one call to NWT.*

Proof. Replace the weights on A by

$$w'(a) = w(a) - t$$

and leave the weights on $B \cup C$ unchanged.

Then

$$w'(a) + w(b) + w(c) = 0$$

if and only if

$$w(a) + w(b) + w(c) = t.$$

□

Theorem 200 (Minimum Node-Weighted Triangle from NWT). *Suppose NWT on a given family of n -vertex instances can be solved in time $T(n)$. Then Minimum Node-Weighted Triangle with integer weights*

$$|w(v)| \leq W$$

can be solved in

$$\boxed{O(T(n) \log(W + 2))}$$

time.

Proof. We recursively reduce the magnitude of the weights.

Replace every weight $w(v)$ by

$$\widehat{w}(v) = \left\lfloor \frac{w(v)}{2} \right\rfloor.$$

Recursively compute the minimum triangle weight

$$s$$

under the rounded weights.

For every integer x ,

$$x = 2 \left\lfloor \frac{x}{2} \right\rfloor + \varepsilon_x, \quad \varepsilon_x \in \{0, 1\}.$$

Therefore for every triangle T ,

$$\boxed{w(T) = 2\widehat{w}(T) + \varepsilon_T, \quad \varepsilon_T \in \{0, 1, 2, 3\}.$$

Let M be the minimum triangle weight under the original weights.

Since

$$\widehat{w}(T) \geq s$$

for every triangle,

$$w(T) \geq 2s,$$

and hence

$$M \geq 2s.$$

On the other hand, let T_0 be a triangle with

$$\widehat{w}(T_0) = s.$$

Then

$$w(T_0) \leq 2s + 3,$$

so

$$M \leq 2s + 3.$$

Thus

$$\boxed{M \in \{2s, 2s + 1, 2s + 2, 2s + 3\}.$$

Using [Lemma 199](#), make at most four NWT calls to determine which of these four values is attained, and return the smallest one.

The rounded weights satisfy

$$|\widehat{w}(v)| \leq \left\lceil \frac{W}{2} \right\rceil.$$

Thus, if $T_{\min}(W)$ denotes the running time,

$$T_{\min}(W) \leq T_{\min}(\lceil W/2 \rceil) + 4T(n) + O(n).$$

After $O(\log(W + 2))$ levels the weights are bounded by a constant, at which point the minimum can be found with constantly many target queries.

Hence

$$T_{\min}(W) = O(T(n) \log(W + 2)).$$

□

10.8 An Algebraic Criterion for Bipartite Perfect Matching

Let

$$G = (L \sqcup R, E), \quad L = R = [n],$$

with bipartite adjacency matrix M .

Hall's theorem says that G has no perfect matching exactly when some set

$$I \subseteq L$$

has fewer neighbors than vertices:

$$|N(I)| < |I|.$$

We would like to turn this directly into a homogeneous linear system.

The intended picture is simple. Each active left vertex should contribute some number of variables, while each neighboring right vertex should impose some number of linear constraints. Then a Hall-deficient set I should leave more variables than constraints and therefore give a nonzero solution.

The difficulty is the converse. Even if

$$|N(I)| \geq |I|,$$

the constraints arising from different right vertices could conceivably be highly dependent. We need a way to force every active left vertex and every neighboring right vertex to consume some genuinely independent resource.

We use high-multiplicity zeros for this purpose.

Each left vertex i receives its own point α_i , and its polynomial will be forced to have a very high-order zero there. These are the *private signatures* of the left vertices.

Each right vertex j receives a fresh point β_j , and the linear combination corresponding to j will be forced to have a high-order zero there.

The Wronskian gives the crucial fact: a low-dimensional space of low-degree polynomials has only a limited total supply of such high-multiplicity zeros.

10.8.1 The Wronskian as a Multiplicity Budget

Let W be an s -dimensional space of polynomials of degree at most d over a field of characteristic greater than d . For $\gamma \in \mathbb{F}$, define

$$N_W(\gamma) = \max_{0 \neq f \in W} \text{mult}_\gamma(f).$$

Thus $N_W(\gamma)$ is the largest order of vanishing at γ that can be achieved by a nonzero polynomial in W .

Lemma 201 (Wronskian Multiplicity Budget).

$$\sum_{\gamma \in \mathbb{F}} \max\{N_W(\gamma) - s + 1, 0\} \leq sd.$$

Proof. Choose a basis

$$f_1, \dots, f_s$$

of W , and form the Wronskian

$$\mathcal{W}(X) = \det \begin{pmatrix} f_1 & \cdots & f_s \\ f'_1 & \cdots & f'_s \\ \vdots & & \vdots \\ f_1^{(s-1)} & \cdots & f_s^{(s-1)} \end{pmatrix}.$$

We first note that

$$\mathcal{W} \neq 0.$$

Indeed, by row-reducing the coefficient vectors of the basis polynomials, we may choose a basis with distinct degrees

$$d_1 < d_2 < \cdots < d_s.$$

If the leading coefficient of f_j is c_j , then the highest-degree term of the Wronskian is

$$\left(\prod_{j=1}^s c_j \right) \det((d_j)_{i-1})_{i,j=1}^s X^{\sum_j d_j - \binom{s}{2}},$$

where

$$(d)_r = d(d-1) \cdots (d-r+1).$$

The determinant is the Vandermonde

$$\prod_{1 \leq i < j \leq s} (d_j - d_i),$$

which is nonzero because the d_i 's are distinct and all are smaller than the characteristic.

Now fix γ , and let

$$t = N_W(\gamma).$$

Choose $f \in W$ with

$$\text{mult}_\gamma(f) = t,$$

and change the basis so that f is its first element. Changing the basis multiplies the Wronskian by a nonzero scalar, so its zeros and their multiplicities do not change.

For

$$0 \leq j < s,$$

the derivative $f^{(j)}$ vanishes at γ to order at least

$$\max\{t - j, 0\}.$$

Hence every entry of the first Wronskian column is divisible by

$$(X - \gamma)^{\max\{t-s+1, 0\}}.$$

Every term in the determinant uses exactly one entry from that column, so

$$\text{mult}_\gamma(\mathcal{W}) \geq \max\{t - s + 1, 0\}.$$

Therefore

$$\text{mult}_\gamma(\mathcal{W}) \geq \max\{N_W(\gamma) - s + 1, 0\}.$$

Summing over all γ ,

$$\sum_{\gamma} \max\{N_W(\gamma) - s + 1, 0\} \leq \deg \mathcal{W}.$$

Finally,

$$\deg \mathcal{W} \leq \sum_{j=1}^s \deg f_j \leq sd.$$

□

10.8.2 Designing the Polynomial System

Choose a field of characteristic larger than all degrees appearing below, and choose

$$\alpha_1, \dots, \alpha_n, \beta_1, \dots, \beta_n$$

to be $2n$ distinct field elements.

Set

$$\ell = 2n^2, \quad a = 2n^2 - 2n + 1, \quad k = 2n^3.$$

The roles of these parameters are different:

- $a + 1$ is the number of degrees of freedom attached to each left vertex;
- ℓ is the number of vanishing constraints imposed by each right vertex;
- k is made much larger than both, solely to ensure that the private left-vertex signatures are linearly independent.

The first two parameters are chosen so that two opposite inequalities hold:

$$(a + 1)s > \ell(s - 1) \quad (1 \leq s \leq n),$$

while

$$\ell > a + 2n - 2.$$

Indeed,

$$(a + 1)s - \ell(s - 1) = 2n^2 - (2n - 2)s > 0,$$

and

$$a + 2n - 2 = 2n^2 - 1 < \ell.$$

The first inequality will make every Hall deficiency produce a nonzero solution. The second will make Hall expansion incompatible with the Wronskian budget.

Variables For each left vertex i , introduce a polynomial

$$A_i(X), \quad \deg A_i \leq a,$$

and set

$$\boxed{P_i(X) = (X - \alpha_i)^k A_i(X).}$$

Thus a nonzero P_i has a private k -fold zero at α_i .

There are

$$a + 1$$

coefficients in A_i , so altogether we have

$$n(a + 1)$$

scalar variables.

Constraints For each right vertex j , define

$$Q_j(X) = \sum_{i=1}^n M_{ij} P_i(X).$$

Require

$$(X - \beta_j)^\ell \mid Q_j(X).$$

Since the characteristic exceeds all relevant degrees, this is equivalent to

$$Q_j(\beta_j) = Q'_j(\beta_j) = \cdots = Q_j^{(\ell-1)}(\beta_j) = 0.$$

Thus each right vertex contributes ℓ homogeneous linear equations in the coefficients of the A_i 's.

The entire construction is therefore one explicit homogeneous linear system with

$$n(a+1) = O(n^3)$$

variables and

$$n\ell = O(n^3)$$

equations.

Theorem 202 (Polynomial Hall Criterion). *With the construction above,*

$$G \text{ has no perfect matching} \iff \text{the polynomial system has a nonzero solution.}$$

Equivalently,

$$G \text{ has a perfect matching} \iff P_1 = \cdots = P_n = 0$$

is the only solution.

The two directions mirror the two sides of Hall's theorem.

10.8.3 Hall Deficiency Leaves Too Much Freedom

Lemma 203 (Deficiency Gives a Nonzero Solution). *If there is a set*

$$I \subseteq L$$

with

$$|N(I)| < |I|,$$

then the polynomial system has a nonzero solution.

Proof. Let

$$J = N(I), \quad s = |I|.$$

Set

$$P_i = 0 \quad (i \notin I).$$

If

$$j \notin J,$$

then $M_{ij} = 0$ for every $i \in I$, so

$$Q_j = 0.$$

Thus all constraints belonging to right vertices outside J are automatically satisfied.

The remaining variables are the coefficients of

$$A_i, \quad i \in I.$$

There are

$$(a+1)s$$

such variables.

Each $j \in J$ contributes at most ℓ homogeneous linear equations, so there are at most

$$\ell|J| \leq \ell(s-1)$$

constraints.

But

$$(a+1)s - \ell(s-1) = 2n^2 - (2n-2)s > 0.$$

Hence there are strictly more variables than homogeneous equations, so the restricted system has a nonzero solution. $\square$

This direction is exactly Hall deficiency expressed as dimension counting:

$$\boxed{|I| > |N(I)| \implies \text{more left-vertex freedom than right-vertex constraints.}}$$

Nothing about the Wronskian is needed here.

10.8.4 Private Zeros Force the Active Polynomials to be Independent

The converse needs more work. A nonzero solution may activate several left vertices, and we need to know that these contribute distinct directions in polynomial space.

That is exactly what the huge private zeros at the α_i 's guarantee.

Lemma 204 (Private-Zero Independence). *Choose distinct indices*

$$i_1, \dots, i_t, \quad t \leq n,$$

and nonzero polynomials

$$P_{i_r} = (X - \alpha_{i_r})^k A_{i_r}, \quad \deg A_{i_r} \leq a.$$

Then

$$P_{i_1}, \dots, P_{i_t}$$

are linearly independent.

Proof. Suppose instead that their span

$$W$$

has dimension

$$r < t.$$

Every polynomial in W has degree at most

$$k+a.$$

For each of the t private points α_{i_j} , the space W contains the corresponding nonzero polynomial P_{i_j} , which has a k -fold zero there. Hence

$$N_W(\alpha_{i_j}) \geq k.$$

Applying [Lemma 201](#),

$$t(k-r+1) \leq r(k+a).$$

We show that this is impossible.

For fixed t , the difference

$$t(k - r + 1) - r(k + a)$$

decreases with r , so the hardest case is

$$r = t - 1.$$

It is therefore enough to prove

$$t(k - t + 2) > (t - 1)(k + a),$$

or equivalently

$$k + t > (t - 1)(t + a).$$

Since $t \leq n$,

$$(t - 1)(t + a) \leq (n - 1)(n + a).$$

Using

$$a = 2n^2 - 2n + 1$$

and

$$k = 2n^3,$$

we obtain

$$(n - 1)(n + a) = 2n^3 - 3n^2 + 2n - 1 < 2n^3 \leq k + t.$$

This is a contradiction. □

The point of the large value of k is now clear. If t active left polynomials tried to live in a space of dimension $r < t$, that r -dimensional space would have to support t different k -fold private zeros. The Wronskian budget cannot afford them.

10.8.5 Hall Expansion Exhausts the Remaining Multiplicity Budget

We can now prove the converse.

Lemma 205 (Hall Expansion Forces the Zero Solution). *Suppose*

$$|N(I)| \geq |I| \quad \text{for every } I \subseteq L.$$

Then the polynomial system has only the zero solution.

Proof. Suppose for contradiction that there is a nonzero solution, and let

$$I = \{i : P_i \neq 0\}, \quad s = |I|.$$

Set

$$W = \text{span}\{P_i : i \in I\}.$$

By [Lemma 204](#),

$$\boxed{\dim W = s.}$$

Every active left vertex contributes its private zero:

$$N_W(\alpha_i) \geq k \quad (i \in I).$$

Now let

$$J = N(I).$$

Hall's condition gives

$$|J| \geq s.$$

For every $j \in J$,

$$Q_j = \sum_{i \in I} M_{ij} P_i.$$

Because $j \in N(I)$, at least one coefficient M_{ij} in this combination is nonzero. Since the P_i 's are linearly independent,

$$Q_j \neq 0.$$

The defining constraints give

$$(X - \beta_j)^\ell \mid Q_j,$$

so

$$N_W(\beta_j) \geq \ell.$$

Thus the same s -dimensional space W contains

s distinct points α_i with multiplicity at least k ,

and at least

s distinct points β_j with multiplicity at least ℓ .

All these points are distinct. Applying [Lemma 201](#) and keeping only s of the β_j 's gives

$$s(k - s + 1) + s(\ell - s + 1) \leq s(k + a),$$

because every polynomial in W has degree at most $k + a$.

Divide by s and cancel k :

$$\ell \leq a + 2s - 2.$$

Since

$$s \leq n,$$

this implies

$$\ell \leq a + 2n - 2.$$

But our parameters satisfy

$$a + 2n - 2 = 2n^2 - 1 < 2n^2 = \ell,$$

a contradiction. □

Combining [Lemmas 203](#) and [205](#) proves [Theorem 202](#).

10.8.6 The Resulting Algorithm

The criterion above is already an algorithm: all of the divisibility conditions are homogeneous linear equations in the coefficients of the polynomials A_i . We now write this linear system explicitly.

It is convenient to use Hasse derivatives. Define

$$D_t(X^r) = \binom{r}{t} X^{r-t}.$$

They satisfy the product rule

$$D_t(fg) = \sum_{t_1+t_2=t} D_{t_1}(f)D_{t_2}(g).$$

Since the characteristic is larger than all relevant degrees,

$$(X - \beta)^\ell \mid f(X)$$

if and only if

$$D_t f(\beta) = 0 \quad (0 \leq t < \ell).$$

Write

$$A_i(X) = \sum_{r=0}^a A_{i,r} X^r.$$

The variables of our linear system are therefore

$$A_{i,r}, \quad i \in [n], \quad 0 \leq r \leq a.$$

Recall that

$$Q_j(X) = \sum_{i=1}^n M_{ij} A_i(X) (X - \alpha_i)^k.$$

The constraint

$$(X - \beta_j)^\ell \mid Q_j(X)$$

is equivalent to

$$D_t Q_j(\beta_j) = 0 \quad (0 \leq t < \ell).$$

Expanding by the Hasse product rule,

$$\begin{aligned} D_t Q_j(\beta_j) &= \sum_i M_{ij} D_t (A_i(X) (X - \alpha_i)^k) (\beta_j) \\ &= \sum_i M_{ij} \sum_{t_1+t_2=t} D_{t_1} A_i(\beta_j) D_{t_2} (X - \alpha_i)^k (\beta_j). \end{aligned}$$

Now

$$D_{t_1} A_i(\beta_j) = \sum_{r=0}^a A_{i,r} \binom{r}{t_1} \beta_j^{r-t_1},$$

while

$$D_{t_2} (X - \alpha_i)^k (\beta_j) = \binom{k}{t_2} (\beta_j - \alpha_i)^{k-t_2}.$$

Hence

$$D_t Q_j(\beta_j) = \sum_{i=1}^n \sum_{r=0}^a A_{i,r} M_{ij} \Gamma_{i,r,j,t},$$

where

$$\Gamma_{i,r,j,t} = \sum_{t_1+t_2=t} \binom{r}{t_1} \binom{k}{t_2} \beta_j^{r-t_1} (\beta_j - \alpha_i)^{k-t_2}.$$

Thus define the matrix

$$F \in \mathbb{F}^{n(a+1) \times n\ell}$$

whose rows are indexed by

$$(i, r), \quad i \in [n], \quad 0 \leq r \leq a,$$

whose columns are indexed by

$$(j, t), \quad j \in [n], \quad 0 \leq t < \ell,$$

and whose entries are

$$F_{(i,r),(j,t)} = M_{ij} \Gamma_{i,r,j,t}.$$

If

$$\mathbf{A} = (A_{i,r})_{i,r}$$

is the row vector containing all coefficients of the A_i 's, then all of the divisibility constraints are exactly

$$\boxed{\mathbf{A}F = 0.}$$

There are

$$n(a+1) = O(n^3)$$

rows and

$$n\ell = O(n^3)$$

columns. Therefore F has a nonzero left kernel exactly when

$$\text{rank}(F) < n(a+1).$$

By [Theorem 202](#), we conclude that

$$\boxed{G \text{ has a perfect matching} \iff \text{rank}(F) = n(a+1).}$$

Thus the algorithm is simply:

1. choose distinct

$$\alpha_1, \dots, \alpha_n, \beta_1, \dots, \beta_n$$

over a field of sufficiently large characteristic;

2. construct the matrix F above from the adjacency matrix M ;
3. compute $\text{rank}(F)$;
4. return that G has a perfect matching exactly when F has full row rank.

Constructing the Matrix The constants $\Gamma_{i,r,j,t}$ can of course be computed directly. There is also a convenient way to compute all of them simultaneously.

For fixed i, j , observe that

$$\binom{a}{r} \Gamma_{i,r,j,t}$$

is exactly the coefficient of $Y^r Z^t$ in

$$\boxed{(1 + Y(Z + \beta_j))^a (Z + (\beta_j - \alpha_i))^k.}$$

Indeed, the coefficient of Y^r in the first factor is

$$\binom{a}{r} (Z + \beta_j)^r,$$

and then the coefficient of Z^t in

$$(Z + \beta_j)^r (Z + (\beta_j - \alpha_i))^k$$

is

$$\sum_{t_1+t_2=t} \binom{r}{t_1} \binom{k}{t_2} \beta_j^{r-t_1} (\beta_j - \alpha_i)^{k-t_2}.$$

Only coefficients with

$$t < \ell$$

are ever needed. Hence for each i, j we may perform the computation modulo Z^ℓ :

$$(1 + Y(Z + \beta_j))^a (Z + (\beta_j - \alpha_i))^k \bmod Z^\ell.$$

Reading off its coefficients gives every $\Gamma_{i,r,j,t}$ for this pair i, j .

Since

$$a, \ell = O(n^2),$$

the relevant bivariate polynomial has only $O(n^4)$ coefficients. Using repeated squaring and fast polynomial multiplication, all of these coefficients for one pair i, j can be computed in

$$\tilde{O}(n^4)$$

field operations, and doing this for all n^2 pairs takes

$$\tilde{O}(n^6)$$

operations.

The graph itself enters only through the final multiplication

$$F_{(i,r),(j,t)} = M_{ij}\Gamma_{i,r,j,t};$$

the $\Gamma_{i,r,j,t}$'s depend only on i, r, j, t and the fixed field points.

10.9 Shortest Paths

10.9.1 Eppstein's Algorithm

Eppstein's algorithm elegantly computes the k shortest paths from a source s to a destination t in a directed graph in $O((m+n)\log n + k\log k)$ time. Rather than searching for full paths from scratch, the algorithm characterizes every possible $s \rightarrow t$ path as the base shortest path plus a sequence of deviations, or sidetracks.

First, we compute the shortest path from every vertex u to t using a reverse Dijkstra's search. This yields the shortest path distance $d[u]$ for all vertices and implicitly defines a shortest path tree T rooted at t .

For any directed edge $e = (u, v)$ with weight w , its sidetrack cost represents the distance penalty incurred by taking e instead of following the shortest path tree T from u . It is defined as:

$$\delta(e) = w + d[v] - d[u] \geq 0$$

Any path P from s to t can be uniquely identified by the sequence of non-tree edges (sidetracks) it traverses: $e_1, e_2, \dots, e_p$. The total length of P is exactly the base distance plus the sum of its sidetrack penalties:

$$\text{length}(P) = d[s] + \sum_{i=1}^p \delta(e_i)$$

State Space and Persistent Heaps To efficiently find the k lightest sidetrack sequences, we construct a min-heap $H[u]$ for every vertex u . This heap contains all possible sidetrack edges originating from u and all ancestors of u in T , ordered by $\delta(e)$.

Because $H[u]$ shares almost all its elements with the heap of its parent in T , we use a Persistent Skew Heap. This allows us to implicitly build the heaps for all vertices in $O(m \log n)$ time. Once the heaps are built, enumerating the k -shortest paths reduces to a Dijkstra-like search over the structure of these heaps, transitioning either by swapping to heavier sidetracks within the same heap or appending new sidetracks from a destination vertex.

Algorithm 29 Eppstein's k -Shortest Paths

Input: Graph $G = (V, E)$, source s , destination t , integer k

Output: Lengths of the k shortest $s \rightarrow t$ paths

```
1: function EPPSTEIN( $G, s, t, k$ )
2:    $d, T \leftarrow \text{REVERSEDJIKSTRA}(G, t)$ 
3:   if  $d[s] = \infty$  then
4:     return  $\emptyset$ 
5:   Order  $V$  by  $d[u]$  ascending
6:   for  $u \in V$  do
7:      $root \leftarrow$  heap root of  $u$ 's parent in  $T$  (or null if none)
8:     for each outgoing edge  $e = (u, v) \in E \setminus T$  do
9:        $\delta \leftarrow \text{weight}(e) + d[v] - d[u]$ 
10:       $root \leftarrow \text{PERSISTENTINSERT}(root, \delta, v)$ 
11:     $H[u] \leftarrow root$ 
12:   $Q \leftarrow$  empty min-priority queue
13:   $Q.\text{push}(\text{length} = d[s], \text{node} = H[s])$ 
14:   $R \leftarrow$  empty list
15:  while  $Q$  is not empty and  $|R| < k$  do
16:     $(L, \text{node}) \leftarrow Q.\text{pop}()$ 
17:    Append  $L$  to  $R$ 
18:    if  $\text{node.left} \neq \text{null}$  then
19:       $Q.\text{push}(L - \text{node}.\delta + \text{node.left}.\delta, \text{node.left})$ 
20:    if  $\text{node.right} \neq \text{null}$  then
21:       $Q.\text{push}(L - \text{node}.\delta + \text{node.right}.\delta, \text{node.right})$ 
22:     $\text{next\_heap} \leftarrow H[\text{node.dest}]$ 
23:    if  $\text{next\_heap} \neq \text{null}$  then
24:       $Q.\text{push}(L + \text{next\_heap}.\delta, \text{next\_heap})$ 
25:  return  $R$ 
```

Complexity Analysis

Theorem 206. Eppstein's algorithm correctly enumerates the k shortest paths in $O((m + n) \log n + k \log k)$ time.

Proof. The base distances and shortest path tree are found in $O((m + n) \log n)$ time. Constructing the persistent heaps requires inserting at most m non-tree edges. Each persistent insertion takes $O(\log n)$ time, leading to $O(m \log n)$ time and space for the heap construction.

During the path extraction phase, every valid transition in the state space explores paths in non-decreasing order of total weight because the heaps are strictly min-ordered ($\delta(\text{child}) \geq \delta(\text{parent})$) and $\delta(e) \geq 0$. Since each state expands to at most 3 subsequent states, the priority queue size never exceeds $3k$. Performing k extractions from this queue takes $O(k \log k)$ time. Summing these phases yields the final bounds. $\square$

10.9.2 Bellman–Ford–Dijkstra

Dijkstra's algorithm efficiently computes shortest paths under nonnegative edges. Bellman–Ford handles negative edges but has a bad running time.

The following hybrid separates the two jobs: after every use of a negative edge, run Dijkstra to exhaustion on all nonnegative edges.

Let

$$E^+ = \{e : w(e) \geq 0\}, \quad E^- = \{e : w(e) < 0\}.$$

For $i \geq 1$, define

$$d_i(v) = \min\{w(P) : P \text{ is an } s\text{-}v \text{ path containing fewer than } i \text{ negative edges}\}.$$

Thus

$$d_1$$

is simply shortest-path distance in the nonnegative subgraph.

Also define

$$d'_i(v) = \min \left\{ d_i(v), \min_{(u,v) \in E^-} d_i(u) + w(u,v) \right\}.$$

The crucial recurrence is

$$d_{i+1}(v) = \min \left\{ d_i(v), \min_u (d'_i(u) + \text{dist}_{G^+}(u, v)) \right\},$$

where it is enough to consider vertices u whose label actually decreased in the negative-edge relaxation.

Indeed, take a shortest path using at most i negative edges and cut it after its last negative edge. Everything after that edge lies in G^+ .

Algorithm 30 Bellman–Ford–Dijkstra

Input: Directed weighted graph G , source s

Output: Shortest-path distances, assuming no reachable negative cycle

```

1:  $d[s] \leftarrow 0$ 
2:  $d[v] \leftarrow \infty$  for  $v \neq s$ 
3: insert  $s$  into priority queue  $Q$ 
4: repeat
5:    $A \leftarrow \emptyset$   $\triangleright$  Dijkstra through nonnegative edges
6:   while  $Q$  is nonempty do
7:     remove a minimum-label vertex  $u$  from  $Q$ 
8:     add  $u$  to  $A$ 
9:     for each  $(u, v) \in E^+$  do
10:      if  $d[u] + w(u, v) < d[v]$  then
11:         $d[v] \leftarrow d[u] + w(u, v)$ 
12:        decrease-key  $v$  in  $Q$   $\triangleright$  Allow one additional negative edge
13:   for  $u \in A$  do
14:     for each  $(u, v) \in E^-$  do
15:       if  $d[u] + w(u, v) < d[v]$  then
16:          $d[v] \leftarrow d[u] + w(u, v)$ 
17:         decrease-key  $v$  in  $Q$ 
18: until  $Q$  is empty
19: return  $d$ 

```

Theorem 207. After the i -th Dijkstra phase,

$$d[v] = d_i(v)$$

for every vertex v .

After the following negative-edge phase,

$$d[v] = d'_i(v).$$

Consequently, if the graph contains no reachable negative cycle, the algorithm terminates with the true shortest-path distances.

Proof. We induct on i .

For $i = 1$, the first phase is ordinary Dijkstra on G^+ , so it computes d_1 .

Suppose a negative phase has produced d'_i . The only newly improved vertices are inserted into Q . Running Dijkstra from these labels computes

$$\min_u (d'_i(u) + \text{dist}_{G^+}(u, v)),$$

while retaining any old smaller value $d_i(v)$. By the last-negative-edge decomposition, this is exactly

$$d_{i+1}(v).$$

Now suppose the Dijkstra phase has produced d_i . Relaxing every negative outgoing edge from a vertex whose value became newly relevant computes

$$d'_i.$$

A negative edge from a vertex whose d_i value did not change was already relaxed when that value was first obtained, so rescanning it is unnecessary.

Thus the invariant holds in every round.

If no reachable negative cycle exists, every shortest path may be chosen simple and therefore contains at most $n - 1$ negative edges. Hence eventually

$$d_i(v) = \text{dist}(s, v)$$

for all v , after which no negative edge improves a label and Q becomes empty. □

Let

$$\eta(v)$$

be the minimum number of negative edges among shortest s - v paths. Then v can become newly active in at most

$$\eta(v) + 1$$

Dijkstra phases.

With a binary heap, the running time is therefore

$$O\left(\sum_v (\deg(v) + \log n)(\eta(v) + 1)\right).$$

If every shortest path uses at most h negative edges or we only consider such paths, this is

$$O((m + n \log n)(h + 1)).$$

Thus the algorithm interpolates naturally between Dijkstra ($h = 0$) and Bellman–Ford ($h = \Theta(n)$).

11 Algebraic and Spectral Graph Theory

11.1 The Matrix–Tree Theorem

Lemma 208 (Spanning Tree Minors). *For*

$$F \subseteq E, \quad |F| = n - 1,$$

$$\det B_r[F, :] = \begin{cases} \pm 1, & F \text{ is a spanning tree,} \\ 0, & \text{otherwise.} \end{cases}$$

Proof. If F contains a cycle, the corresponding incidence rows are dependent. If F is disconnected, its incidence matrix has rank less than $n - 1$.

Thus only spanning trees can give a nonzero determinant. For a tree, repeatedly remove a leaf other than r . The corresponding column contains a single nonzero entry ± 1 , so expansion along that column reduces the determinant to a smaller tree. Induction gives ± 1 . $\square$

A more useful form of Matrix–Tree is the following asymmetric statement.

Theorem 209 (Generalized Matrix–Tree Identity). *Let*

$$W = \text{diag}(w_e)$$

and let

$$A \in \mathbb{F}^{E \times (n-1)}$$

be arbitrary. Then

$$\det(B_r^\top W A) = \sum_{T \text{ spanning tree}} \det(B_r[T, :]) \det(A[T, :]) \prod_{e \in T} w_e.$$

Proof. Apply Cauchy–Binet:

$$\begin{aligned} \det(B_r^\top W A) &= \sum_{\substack{F \subseteq E \\ |F|=n-1}} \det(B_r[F, :]^\top W_F) \det(A[F, :]) \\ &= \sum_F \det(B_r[F, :]) \det(A[F, :]) \prod_{e \in F} w_e. \end{aligned}$$

By [Lemma 208](#), every non-tree term vanishes. $\square$

Taking

$$A = B_r$$

gives the usual weighted Matrix–Tree theorem:

$$\det(B_r^\top W B_r) = \sum_{T \text{ spanning tree}} \prod_{e \in T} w_e.$$

The arbitrary second matrix is often useful because it can impose additional combinatorial conditions while the first incidence matrix enforces that the chosen edge set is a tree.

11.1.1 Counting Hamiltonian Paths

Let $G = (V, E)$ be an unweighted graph,

$$|V| = n,$$

and fix vertices s, t .

Give an edge

$$e = \{u, v\}$$

weight

$$w_e = x_u x_v.$$

By Matrix–Tree,

$$P_G(x) := \det(B_r^\top W(x) B_r) = \sum_{T \text{ tree}} \prod_{v \in V} x_v^{\deg_T(v)}.$$

Theorem 210 (Hamiltonian Paths from Matrix–Tree).

$$\# \text{HamPath}(G; s, t) = \frac{1}{2^n} \sum_{x \in \{-1, 1\}^V} x_s x_t P_G(x).$$

Consequently s – t Hamiltonian paths can be counted in

$$O^*(2^n)$$

time and polynomial space.

Proof. Consider one spanning tree T . Its contribution after multiplying by $x_s x_t$ is

$$x_s^{\deg_T(s)+1} x_t^{\deg_T(t)+1} \prod_{v \notin \{s, t\}} x_v^{\deg_T(v)}.$$

For an integer a ,

$$\sum_{x \in \{-1, 1\}} x^a = \begin{cases} 2, & a \text{ even,} \\ 0, & a \text{ odd.} \end{cases}$$

Thus T survives the outer sum exactly when

$$\deg_T(s), \deg_T(t)$$

are odd and every other degree is even.

All degrees are positive, and

$$\sum_v \deg_T(v) = 2n - 2.$$

The minimum possible sum under these parity conditions is already

$$1 + 1 + 2(n - 2) = 2n - 2.$$

Therefore equality must hold everywhere:

$$\deg_T(s) = \deg_T(t) = 1$$

and

$$\deg_T(v) = 2 \quad (v \neq s, t).$$

Such a tree is precisely an s – t Hamiltonian path.

Every surviving tree contributes 2^n . □

11.1.2 Counting k -Matchings in a Bipartite Graph

Let

$$G = (U \sqcup V, E), \quad |U| = n_1 \leq n_2 = |V|.$$

Add a new vertex s adjacent to every vertex of $U \cup V$, obtaining H .

Assign weights

$$w(uv) = x_u \quad (u \in U, v \in V),$$

$$w(su) = y_1 \quad (u \in U),$$

and

$$w(sv) = y_2 \quad (v \in V).$$

Let

$$P_H(X, y_1, y_2)$$

be the resulting spanning-tree polynomial.

Also define the elementary symmetric polynomial

$$e_k(X) = \sum_{\substack{S \subseteq U \\ |S|=k}} \prod_{u \in S} x_u.$$

Theorem 211 (k -Matching Count).

$$\#M_k(G) = \frac{1}{2^{n_1}} \sum_{x \in \{-1, 1\}^U} [y_1^{n_1} y_2^{n_2-k}] (e_k(X) P_H(X, y_1, y_2)).$$

Hence all size- k matchings can be counted in

$$O^*(2^{n_1}) \leq O^*(2^{n/2})$$

time and polynomial space.

Proof. The coefficient

$$[y_1^{n_1} y_2^{n_2-k}]$$

restricts attention to spanning trees containing

- all n_1 edges from s to U ;
- exactly $n_2 - k$ edges from s to V .

Such a tree therefore contains exactly k original edges of G .

Fix a term

$$x_S = \prod_{u \in S} x_u, \quad |S| = k,$$

from $e_k(X)$. For a spanning tree T , the exponent of x_u in

$$x_S P_H$$

is

$$d_T^G(u) + \mathbf{1}[u \in S],$$

where $d_T^G(u)$ counts original G -edges incident to u .

Averaging over

$$x_u \in \{-1, 1\}$$

kills the term unless all these exponents are even.

Since there are exactly k original G -edges,

$$\sum_{u \in U} d_T^G(u) = k.$$

Also

$$|S| = k.$$

The parity condition therefore forces

$$d_T^G(u) = \begin{cases} 1, & u \in S, \\ 0, & u \notin S. \end{cases}$$

The k original edges hence have distinct endpoints in U . They must also have distinct endpoints in V : if

$$u_1v, u_2v$$

were both chosen, then because the tree already contains

$$su_1, su_2,$$

the four vertices would contain the cycle

$$s - u_1 - v - u_2 - s.$$

Thus the k original edges form a matching.

Conversely, every k -matching extends uniquely to one such spanning tree by adding

- every edge su for $u \in U$;
- every edge sv for unmatched $v \in V$.

Taking S to be the saturated vertices of U , its parity term contributes exactly 2^{n_1} . □

11.2 Cheeger Inequality

Let

$$G = (V, E, w)$$

be an undirected graph with nonnegative edge weights. Define

$$d_v = \sum_u w_{uv}, \quad \text{vol}(S) = \sum_{v \in S} d_v,$$

and

$$w(S, \bar{S}) = \sum_{\substack{u \in S \\ v \notin S}} w_{uv}.$$

The conductance of S is

$$\phi(S) = \frac{w(S, \bar{S})}{\min\{\text{vol}(S), \text{vol}(\bar{S})\}},$$

and

$$\phi(G) = \min_{\emptyset \neq S \neq V} \phi(S).$$

Let

$$D = \text{diag}(d_v), \quad L = D - A, \quad \mathcal{L} = D^{-1/2} L D^{-1/2}$$

be the normalized Laplacian.

Equivalently, its second eigenvalue satisfies

$$\lambda_2 = \min_{\substack{f \neq 0 \\ \sum_v d_v f_v = 0}} \frac{\sum_{\{u,v\} \in E} w_{uv} (f_u - f_v)^2}{\sum_v d_v f_v^2}.$$

Theorem 212 (Weighted Cheeger Inequality). *For every connected weighted graph,*

$$\boxed{\frac{\lambda_2}{2} \leq \phi(G) \leq \sqrt{2\lambda_2}.}$$

The Easy Direction

Lemma 213.

$$\lambda_2 \leq 2\phi(G).$$

Proof. Take any set S with

$$s = \text{vol}(S) \leq \frac{1}{2} \text{vol}(V).$$

Let

$$T = \text{vol}(V)$$

and define

$$f_v = \begin{cases} 1 - s/T, & v \in S, \\ -s/T, & v \notin S. \end{cases}$$

Then

$$\sum_v d_v f_v = 0.$$

Only crossing edges contribute to the numerator, and the difference across such an edge is 1. Thus

$$f^T L f = w(S, \bar{S}).$$

Meanwhile,

$$\begin{aligned} f^T D f &= s(1 - s/T)^2 + (T - s)(s/T)^2 \\ &= \frac{s(T - s)}{T} \\ &\geq \frac{s}{2}. \end{aligned}$$

Therefore

$$\lambda_2 \leq \frac{w(S, \bar{S})}{s/2} = 2\phi(S).$$

Minimize over S . □

The Sweep Lemma

Lemma 214 (Sweep Cut). *Let $g : V \rightarrow \mathbb{R}_{\geq 0}$ be nonzero and suppose*

$$\text{vol}(\text{supp } g) \leq \frac{1}{2} \text{vol}(V).$$

Then some threshold set

$$S_t = \{v : g_v^2 \geq t\}$$

satisfies

$$\boxed{\phi(S_t) \leq \sqrt{2 \frac{\sum_{\{u,v\}} w_{uv}(g_u - g_v)^2}{\sum_v d_v g_v^2}}}.$$

Proof. We have that

$$\int_0^\infty \text{vol}(S_t) dt = \sum_v d_v g_v^2.$$

Similarly,

$$\int_0^\infty w(S_t, \bar{S}_t) dt = \sum_{\{u,v\}} w_{uv} |g_u^2 - g_v^2|.$$

Every S_t lies inside $\text{supp } g$, so

$$\text{vol}(S_t) \leq \frac{1}{2} \text{vol}(V).$$

Thus

$$w(S_t, \bar{S}_t) \geq \phi(G) \text{vol}(S_t).$$

Integrating,

$$\phi(G) \leq \frac{\sum_{\{u,v\}} w_{uv} |g_u^2 - g_v^2|}{\sum_v d_v g_v^2}.$$

Now

$$|g_u^2 - g_v^2| = |g_u - g_v|(g_u + g_v),$$

so Cauchy–Schwarz gives

$$\sum_{\{u,v\}} w_{uv} |g_u^2 - g_v^2| \leq \left(\sum_{\{u,v\}} w_{uv} (g_u - g_v)^2 \right)^{1/2} \left(\sum_{\{u,v\}} w_{uv} (g_u + g_v)^2 \right)^{1/2}.$$

Since

$$(g_u + g_v)^2 \leq 2g_u^2 + 2g_v^2,$$

the second factor is at most

$$\sqrt{2 \sum_v d_v g_v^2}.$$

The result follows. □

The Hard Direction

Proof that $\phi(G) \leq \sqrt{2\lambda_2}$. Let f be a λ_2 eigenvector of $D^{-1}L$, which is similar to $\mathcal{L}$:

$$Lf = \lambda_2 Df.$$

Define

$$f_+ = \max(f, 0), \quad f_- = \max(-f, 0).$$

Their supports are disjoint, so at least one has volume at most half of the total volume. Call that function g .

We claim

$$\frac{g^T Lg}{g^T Dg} \leq \lambda_2.$$

For example, if $g = f_+$, then

$$f^T Lf_+ = \lambda_2 f^T Df_+ = \lambda_2 f_+^T Df_+.$$

For each edge,

$$(f_u - f_v)(f_{+,u} - f_{+,v}) \geq (f_{+,u} - f_{+,v})^2,$$

so

$$f^T Lf_+ \geq f_+^T Lf_+.$$

Hence

$$f_+^T Lf_+ \leq \lambda_2 f_+^T Df_+.$$

The same proof applies to f_- .

Apply [Lemma 214](#) to g :

$$\phi(G) \leq \sqrt{2 \frac{g^T Lg}{g^T Dg}} \leq \sqrt{2\lambda_2}.$$

□

12 Electrical Flows

Let

$$G = (V, E, c)$$

be a connected undirected graph with positive edge conductances

$$c_e > 0, \quad r_e = \frac{1}{c_e}$$

the corresponding resistances.

Fix an arbitrary orientation of the edges. For an oriented edge $e = (u, v)$, write

$$b_e = \mathbf{1}_u - \mathbf{1}_v.$$

Let

$$B \in \mathbb{R}^{E \times V}$$

have rows $b_e^\top$, and let

$$C = \text{diag}(c_e).$$

Then the weighted Laplacian is

$$L = B^\top C B.$$

For a flow $\theta \in \mathbb{R}^E$,

$$B^\top \theta$$

is its divergence vector, and its energy is

$$\mathcal{E}(\theta) = \theta^\top C^{-1} \theta = \sum_{e \in E} r_e \theta_e^2.$$

12.1 Electrical Flows

More generally than s - t flows, let

$$b \in \mathbb{R}^V, \quad \mathbf{1}^\top b = 0.$$

A flow realizing demand b satisfies

$$B^\top \theta = b.$$

The familiar unit s - t case is

$$b = \mathbf{1}_s - \mathbf{1}_t.$$

The form of the solution to electrical flows can be seen by applying Lagrange multipliers to the optimization problem. Thereby we can see that there exists some $\phi \in \mathbb{R}^n$ such that $\theta = CB\phi$, then we know $L\phi = b$ so $\phi = L^+b$ is natural.

Theorem 215 (Thomson's Principle). *For every demand $b \perp \mathbf{1}$, the unique minimum-energy flow satisfying*

$$B^\top \theta = b$$

is

$$\theta^* = CBL^+b.$$

Its energy is

$$\mathcal{E}(\theta^*) = b^\top L^+b.$$

Proof. First,

$$B^\top \theta^\star = B^\top CBL^+b = LL^+b = b,$$

since $b \perp \mathbf{1} = \ker L$.

Now let θ be any other feasible flow and write

$$\theta = \theta^\star + z.$$

Then

$$B^\top z = 0.$$

The energy expands as

$$\mathcal{E}(\theta) = \mathcal{E}(\theta^\star) + 2(\theta^\star)^\top C^{-1}z + \mathcal{E}(z).$$

The cross term vanishes:

$$(\theta^\star)^\top C^{-1}z = (CBL^+b)^\top C^{-1}z = (L^+b)^\top B^\top z = 0.$$

Therefore

$$\mathcal{E}(\theta) = \mathcal{E}(\theta^\star) + \mathcal{E}(z) \geq \mathcal{E}(\theta^\star),$$

with equality only for $z = 0$.

Finally,

$$\begin{aligned} \mathcal{E}(\theta^\star) &= b^\top L^+ B^\top C C^{-1} C B L^+ b \\ &= b^\top L^+ L L^+ b \\ &= b^\top L^+ b. \end{aligned}$$

□

12.1.1 Potentials and Ohm's Law

A potential

$$\phi : V \rightarrow \mathbb{R}$$

induces the flow

$$\theta = CB\phi,$$

or edgewise

$$\theta_{uv} = c_{uv}(\phi_u - \phi_v).$$

Thus [Theorem 215](#) says that the electrical flow for demand b is induced by the potential

$$\phi = L^+b.$$

Potentials are unique up to adding a constant.

12.1.2 Effective Resistance

For vertices s, t , define

$$b_{st} = \mathbf{1}_s - \mathbf{1}_t.$$

Definition 216 (Effective Resistance).

$$R_{\text{eff}}(s, t) = b_{st}^\top L^+ b_{st}.$$

By Thomson's principle,

$$R_{\text{eff}}(s, t) = \min_{\theta: B^\top \theta = b_{st}} \mathcal{E}(\theta).$$

If

$$\phi = L^+ b_{st}$$

is the unit-current potential, then

$$R_{\text{eff}}(s, t) = \phi_s - \phi_t = \mathcal{E}(\theta^*).$$

Indeed,

$$\phi_s - \phi_t = b_{st}^\top L^+ b_{st}.$$

12.1.3 Maximum Principle

Lemma 217 (Maximum Principle). *Let ϕ be harmonic on*

$$V \setminus S,$$

meaning

$$(L\phi)_v = 0 \quad (v \notin S).$$

Then every maximum and minimum of ϕ is attained on S .

Proof. At a harmonic vertex v ,

$$0 = \sum_{u \sim v} c_{uv}(\phi_v - \phi_u),$$

so

$$\phi_v = \frac{\sum_{u \sim v} c_{uv} \phi_u}{\sum_{u \sim v} c_{uv}}.$$

Thus ϕ_v is a weighted average of its neighbors.

If an interior vertex achieves the global maximum, all of its neighbors must have the same value. By connectedness this propagates until reaching the boundary. The minimum case is identical. $\square$

Because of the maximum principle and nonnegativity of effective resistances we have, for the unit s - t electrical potential,

$$\phi_s \geq \phi_v \geq \phi_t \quad \text{for every } v.$$

12.1.4 Dirichlet's Principle

Dirichlet's principle is most naturally stated as a boundary-value problem. Fix potentials on some set of boundary vertices; among all extensions to the rest of the graph, the minimum-energy extension is exactly the harmonic one.

Let $G = (V, E)$ be connected, with weighted Laplacian L , and define

$$\mathcal{E}(\phi) = \phi^\top L \phi = \sum_{uv \in E} w_{uv} (\phi_u - \phi_v)^2.$$

Theorem 218 (Dirichlet's Principle). *Let $\emptyset \neq S \subseteq V$, and prescribe boundary values $b \in \mathbb{R}^S$. Among all potentials satisfying*

$$\phi|_S = b,$$

there is a unique minimizer h of $\mathcal{E}(\phi)$. It is characterized by

$$(Lh)_v = 0 \quad (v \in V \setminus S),$$

that is, h is harmonic at every non-boundary vertex.

Proof. Let $U = V \setminus S$. The harmonicity equations are

$$L_{U,U}h_U = -L_{U,S}b.$$

The matrix $L_{U,U}$ is positive definite. Indeed, if $x \in \mathbb{R}^U$, extend it by 0 on S . Then

$$x^\top L_{U,U}x = \sum_{uv \in E} w_{uv}(x_u - x_v)^2.$$

If this is 0, the extended vector is constant on every edge. Since G is connected and it vanishes on the nonempty set S , it must vanish everywhere. Hence $x = 0$.

Thus there is a unique harmonic extension h .

Now let ϕ be any other extension of the same boundary and write

$$\phi = h + \psi, \quad \psi|_S = 0.$$

Then

$$\phi^\top L\phi = h^\top Lh + 2\psi^\top Lh + \psi^\top L\psi.$$

Since h is harmonic on U , the vector Lh is supported on S . Since ψ vanishes on S ,

$$\psi^\top Lh = 0.$$

Therefore

$$\boxed{\mathcal{E}(\phi) = \mathcal{E}(h) + \mathcal{E}(\psi) \geq \mathcal{E}(h).}$$

Equality forces $\mathcal{E}(\psi) = 0$, so ψ is constant; because it vanishes on S , it is identically zero. Thus h is the unique minimizer. $\square$

Boundary Current and Power For the harmonic extension h , let

$$j = Lh.$$

The coordinate j_v is the net current injected at v . Harmonicity says that $j_v = 0$ away from S , so current enters and leaves the network only through the boundary.

Moreover,

$$\boxed{\mathcal{E}(h) = h^\top Lh = h^\top j = \sum_{v \in S} b_v j_v.}$$

Thus the energy is exactly the power supplied through the boundary: potential times injected current, summed over the boundary vertices.

Define the effective conductance

$$C_{\text{eff}}(s, t) = \frac{1}{R_{\text{eff}}(s, t)}.$$

Corollary 219 (Two-Terminal Dirichlet Principle). *For distinct vertices s, t , let h be harmonic on $V \setminus \{s, t\}$ with*

$$h_s = 1, \quad h_t = 0.$$

Then

$$\boxed{C_{\text{eff}}(s, t) = \min_{\substack{\phi_s=1 \\ \phi_t=0}} \phi^\top L\phi.}$$

Proof. By [Theorem 218](#), h is the unique minimizer.

Let

$$I = (Lh)_s$$

be the total current leaving s . Since Lh vanishes away from s, t , and

$$\mathbf{1}^\top Lh = 0,$$

we have

$$(Lh)_t = -I.$$

Therefore

$$\begin{aligned} h^\top Lh &= h_s(Lh)_s + h_t(Lh)_t \\ &= 1 \cdot I + 0 \cdot (-I) \\ &= I. \end{aligned}$$

Equivalently, the energy is

$$(\text{voltage drop}) \cdot (\text{current}) = 1 \cdot I.$$

Now scale the potential by $1/I$. Currents scale by the same factor, so the new potential sends one unit of current from s to t . Its voltage drop is

$$\frac{1}{I}.$$

Now, by the definition of effective resistance,

$$R_{\text{eff}}(s, t) = \frac{1}{I}.$$

Hence

$$I = \frac{1}{R_{\text{eff}}(s, t)} = C_{\text{eff}}(s, t),$$

and therefore

$$h^\top Lh = C_{\text{eff}}(s, t).$$

□

	current	voltage drop	energy
unit current	1	R_{eff}	R_{eff}
unit voltage	C_{eff}	1	C_{eff}

12.1.5 Series and Parallel Laws

If two networks of resistances R_1, R_2 are connected in series, every unit flow must pass through both. Hence Thomson's principle immediately gives

$$\boxed{R_{\text{series}} = R_1 + R_2.}$$

If they are connected in parallel and a fraction α of the unit current uses the first branch, the minimum possible energy for this split is

$$\alpha^2 R_1 + (1 - \alpha)^2 R_2.$$

Minimizing gives

$$\alpha R_1 = (1 - \alpha) R_2$$

and hence

$$\boxed{R_{\text{parallel}} = \frac{R_1 R_2}{R_1 + R_2}.}$$

Equivalently,

$$\boxed{C_{\text{parallel}} = C_1 + C_2.}$$

More generally, parallel currents divide proportionally to conductance.

12.1.6 Rayleigh Monotonicity

Theorem 220 (Rayleigh Monotonicity). *Increasing edge resistances can only increase effective resistance. Equivalently, increasing conductances can only decrease effective resistance.*

Proof. Suppose

$$r'_e \geq r_e$$

for every edge. For every unit s - t flow θ ,

$$\sum_e r'_e \theta_e^2 \geq \sum_e r_e \theta_e^2.$$

Taking the minimum over all unit flows and applying Thomson's principle gives

$$R'_{\text{eff}}(s, t) \geq R_{\text{eff}}(s, t).$$

□

Thus:

- deleting edges increases resistance;
- adding edges decreases resistance;
- contracting or shorting vertices decreases resistance.

12.1.7 Effective Resistance is a Metric

Theorem 221. *On the vertices of a connected graph,*

$$R_{\text{eff}}$$

is a metric.

Proof. Nonnegativity, symmetry, and

$$R_{\text{eff}}(s, t) = 0 \iff s = t$$

are immediate from

$$R_{\text{eff}}(s, t) = b_{st}^\top L^+ b_{st}.$$

For the triangle inequality, let

$$p = L^+(\mathbf{1}_s - \mathbf{1}_t).$$

By the maximum principle,

$$p_u \geq p_t.$$

Now

$$R_{\text{eff}}(s, t) + R_{\text{eff}}(t, u) - R_{\text{eff}}(s, u) = 2(p_u - p_t) \geq 0.$$

Hence

$$R_{\text{eff}}(s, u) \leq R_{\text{eff}}(s, t) + R_{\text{eff}}(t, u).$$

□

12.2 Random Walks and Electrical Networks

Define the weighted random walk by

$$P(u, v) = \frac{c_{uv}}{c_u}, \quad c_u = \sum_v c_{uv}.$$

Its stationary distribution is

$$\pi(u) = \frac{c_u}{\text{vol}(G)}, \quad \text{vol}(G) = \sum_u c_u.$$

This walk is reversible:

$$\boxed{c_u P(u, v) = c_{uv} = c_v P(v, u).}$$

12.2.1 Voltages are Hitting Probabilities

Let

$$T_A = \inf\{j \geq 0 : X_j \in A\}.$$

Theorem 222. *Let*

$$q(u) = \Pr_u[T_s < T_t].$$

Then q is exactly the unit-voltage electrical potential:

$$q(s) = 1, \quad q(t) = 0,$$

and

$$Lq = 0 \quad \text{on } V \setminus \{s, t\}.$$

Proof. Conditioning on the first random-walk step gives, for

$$u \notin \{s, t\},$$

$$q(u) = \sum_v P(u, v) q(v).$$

Multiplying by c_u gives

$$\sum_v c_{uv} (q(u) - q(v)) = 0,$$

which is precisely

$$(Lq)_u = 0.$$

The boundary values are clearly 1 and 0. Uniqueness follows from the maximum principle. $\square$

If

$$p = L^+(\mathbf{1}_s - \mathbf{1}_t)$$

is instead the unit-current potential, then

$$\boxed{q(u) = \frac{p_u - p_t}{p_s - p_t} = \frac{p_u - p_t}{R_{\text{eff}}(s, t)}.$$

12.2.2 Currents are Expected Net Traversals

Run a random walk from s until its first visit to t . For $v \neq t$, let

$$G(v) = \mathbb{E}_s[\# \text{ visits to } v \text{ before } T_t],$$

and set

$$G(t) = 0.$$

Define

$$g(v) = \frac{G(v)}{c_v}.$$

Also let $N_{u \rightarrow v}$ be the number of times we walk on the edge $u \rightarrow v$ in that direction.

Theorem 223.

$$\boxed{Lg = \mathbf{1}_s - \mathbf{1}_t.}$$

Consequently,

$$g = L^+(\mathbf{1}_s - \mathbf{1}_t) + \alpha \mathbf{1}$$

for some constant α , and the electrical current on (u, v) is

$$\boxed{c_{uv}(g(u) - g(v)) = \mathbb{E}_s[N_{u \rightarrow v} - N_{v \rightarrow u}].}$$

Proof. For $v \neq t$, every visit to v is either the initial visit when $v = s$ or results from a transition into v :

$$G(v) = \mathbf{1}[v = s] + \sum_u G(u)P(u, v).$$

We have

$$G(u)P(u, v) = c_{uv}g(u).$$

Hence

$$c_v g(v) - \sum_u c_{uv} g(u) = \mathbf{1}[v = s],$$

so

$$(Lg)_v = \mathbf{1}[v = s] \quad (v \neq t).$$

Because the entries of Lg sum to zero, its t -coordinate is -1 .

Also

$$\mathbb{E}N_{u \rightarrow v} = G(u)P(u, v) = c_{uv}g(u),$$

so subtraction gives the final identity. □

12.2.3 Hitting a Vertex Through a Given Edge

Corollary 224. Suppose $\{s, t\} \in E$. A walk starting at s first enters t through the edge $s \rightarrow t$ with probability

$$\boxed{c_{st}R_{\text{eff}}(s, t).}$$

Proof. Before T_t , the edge $s \rightarrow t$ can be traversed at most once, and traversing it ends the walk. Thus its expected number of traversals is exactly the desired probability.

By [Theorem 223](#),

$$\mathbb{E}N_{s \rightarrow t} = c_{st}(g(s) - g(t)) = c_{st}R_{\text{eff}}(s, t).$$

□

In particular,

$$c_{st}R_{\text{eff}}(s, t) \leq 1.$$

12.2.4 Hitting Times

Let

$$H_{u,t} = \mathbb{E}_u[T_t].$$

For fixed t , define

$$h_t(u) = H_{u,t}.$$

For $u \neq t$,

$$h_t(u) = 1 + \sum_v P(u,v)h_t(v).$$

Multiplying by c_u gives

$$(Lh_t)_u = c_u.$$

Let

$$\mathbf{c} = (c_v)_{v \in V}, \quad \text{vol}(G) = \sum_v c_v,$$

and define

$$b_t = \mathbf{c} - \text{vol}(G)\mathbf{1}_t.$$

Since both Lh_t and b_t have coordinate sum zero,

$$\boxed{Lh_t = b_t.}$$

Theorem 225 (Hitting-Time Potential). *Let*

$$p_t = L^+ b_t.$$

Then

$$\boxed{H_{u,t} = p_t(u) - p_t(t).}$$

Proof. Both h_t and p_t solve

$$Lx = b_t.$$

Their difference therefore lies in $\ker L = \text{span}\{\mathbf{1}\}$. Since

$$h_t(t) = 0,$$

subtracting the value at t gives the formula. □

12.2.5 Commute Time

Define

$$C_{u,v} = H_{u,v} + H_{v,u}.$$

Theorem 226 (Commutate-Time Identity).

$$\boxed{C_{u,v} = \text{vol}(G)R_{\text{eff}}(u,v).}$$

Proof. Using the preceding theorem,

$$p_v - p_u = L^+(b_v - b_u) = \text{vol}(G)L^+(\mathbf{1}_u - \mathbf{1}_v).$$

Therefore

$$\begin{aligned} H_{u,v} + H_{v,u} &= (\mathbf{1}_u - \mathbf{1}_v)^\top (p_v - p_u) \\ &= \text{vol}(G)(\mathbf{1}_u - \mathbf{1}_v)^\top L^+(\mathbf{1}_u - \mathbf{1}_v). \end{aligned}$$

□

12.2.6 Tetali's Hitting-Time Formula

Theorem 227 (Tetali's Formula). *For all s, t ,*

$$H_{s,t} = \frac{1}{2} \sum_{x \in V} c_x (R_{\text{eff}}(s, t) + R_{\text{eff}}(t, x) - R_{\text{eff}}(s, x)).$$

Proof. Write

$$R(a, b) = L_{aa}^+ + L_{bb}^+ - 2L_{ab}^+.$$

Then

$$\begin{aligned} & \frac{1}{2} \sum_x c_x (R(s, t) + R(t, x) - R(s, x)) \\ &= \sum_x c_x (L_{sx}^+ - L_{tx}^+) - \text{vol}(G)(L_{st}^+ - L_{tt}^+) \\ &= (\mathbf{1}_s - \mathbf{1}_t)^\top L^+ (\mathbf{c} - \text{vol}(G)\mathbf{1}_t) \\ &= p_t(s) - p_t(t) \\ &= H_{s,t}. \end{aligned}$$

□

12.2.7 Cover-Time Bound

Let

$$\text{cov}_u(G)$$

be the expected time for an unweighted random walk starting at u to visit every vertex.

Theorem 228. *For every connected unweighted graph with n vertices and m edges,*

$$\text{cov}_u(G) \leq 2m(n-1).$$

Proof. Fix any spanning tree T . Take an Euler tour of T , which traverses every tree edge once in each direction.

Starting from u , wait until the random walk visits the successive vertices of this tour. Once this process finishes, every vertex has been visited. Its expected duration is at most

$$\sum_{\{x,y\} \in T} (H_{x,y} + H_{y,x}) = \sum_{\{x,y\} \in T} C_{x,y}.$$

For a tree edge $\{x, y\}$, Rayleigh monotonicity gives

$$R_{\text{eff}}(x, y) \leq 1.$$

Since

$$\text{vol}(G) = 2m,$$

the commute-time identity yields

$$C_{x,y} \leq 2m.$$

There are $n-1$ tree edges.

□

12.3 Kron Reduction

Electrical networks remain electrical networks after eliminating internal vertices.

Partition the vertices into a boundary S and interior I , and write

$$L = \begin{pmatrix} L_{SS} & L_{SI} \\ L_{IS} & L_{II} \end{pmatrix}.$$

Theorem 229 (Kron Reduction). *The network induced on the boundary after eliminating the interior has Laplacian*

$$L_{\text{Kron}} = L_{SS} - L_{SI}L_{II}^{-1}L_{IS}.$$

It preserves all effective resistances between vertices of S .

Proof. Fix boundary potentials x_S . The energy is

$$\begin{pmatrix} x_S \\ x_I \end{pmatrix}^\top L \begin{pmatrix} x_S \\ x_I \end{pmatrix}.$$

The minimizing interior potential satisfies

$$L_{IS}x_S + L_{II}x_I = 0,$$

so

$$x_I = -L_{II}^{-1}L_{IS}x_S.$$

Substituting gives

$$x_S^\top (L_{SS} - L_{SI}L_{II}^{-1}L_{IS}) x_S.$$

Dirichlet's principle therefore shows that all boundary effective resistances are unchanged. $\square$

There is also a random-walk interpretation.

Let

$$P = D^{-1}A$$

be the transition matrix. Partition it in the same way.

Theorem 230 (Trace Chain). *Observe the random walk only at visits to S . Its transition matrix is*

$$Q = P_{SS} + P_{SI}(I - P_{II})^{-1}P_{IS}.$$

Moreover,

$$L_{\text{Kron}} = D_S(I - Q).$$

Proof. The term

$$P_{SI}P_{II}^kP_{IS}$$

is the probability of leaving S , taking exactly k internal steps, and returning to S . Summing over $k \geq 0$ gives the formula for Q .

Also,

$$L_{II} = D_I(I - P_{II}),$$

$$L_{SI} = -D_S P_{SI}, \quad L_{IS} = -D_I P_{IS},$$

and

$$L_{SS} = D_S(I - P_{SS}).$$

Substitution into the Schur complement gives

$$L_{\text{Kron}} = D_S(I - P_{SS} - P_{SI}(I - P_{II})^{-1}P_{IS}) = D_S(I - Q).$$

$\square$

Thus for distinct boundary vertices a, b , the reduced conductance is

$$\tilde{c}_{ab} = c_a Q(a, b).$$

As a special case, eliminating the center of a star whose incident conductances are

$$c_1, \dots, c_k$$

creates between leaves i, j the conductance

$$\tilde{c}_{ij} = \frac{c_i c_j}{\sum_{\ell} c_{\ell}}.$$

12.4 Infinite Graphs, Recurrence, and Transience

Now let G be a connected locally finite infinite graph.

Choose finite connected sets

$$K_1 \subseteq K_2 \subseteq \dots, \quad \bigcup_n K_n = V,$$

containing a fixed vertex o .

Wire every vertex outside K_n into one sink ∂_n , and let

$$R_n = R_{\text{eff}}(o, \partial_n).$$

Because the n -th wired network is obtained from the $(n+1)$ -st by additional contractions, Rayleigh monotonicity gives

$$R_n \leq R_{n+1}.$$

Define

$$R_{\text{eff}}(o \leftrightarrow \infty) = \lim_{n \rightarrow \infty} R_n \in [0, \infty].$$

12.4.1 Infinite Thomson Principle

A unit flow from o to infinity satisfies

$$\text{div } \theta(o) = 1$$

and

$$\text{div } \theta(v) = 0 \quad (v \neq o).$$

Equivalently, one unit of net current crosses every finite cut separating o from infinity.

Theorem 231 (Infinite Thomson Principle).

$$R_{\text{eff}}(o \leftrightarrow \infty) = \inf_{\theta: \text{unit flow to infinity}} \mathcal{E}(\theta).$$

In particular,

$$R_{\text{eff}}(o \leftrightarrow \infty) < \infty$$

if and only if there exists a finite-energy unit flow to infinity.

Proof. Let θ be any unit flow to infinity. Restrict it to the edges inside K_n and crossing its boundary. After wiring the exterior, this is a feasible unit flow from o to ∂_n . Therefore finite Thomson gives

$$R_n \leq \mathcal{E}(\theta).$$

Taking $n \rightarrow \infty$ gives

$$R_\infty \leq \mathcal{E}(\theta).$$

Conversely, suppose

$$R_\infty < \infty.$$

Let θ_n be the electrical unit flow from o to ∂_n , extended by zero outside K_n . Then

$$\mathcal{E}(\theta_n) = R_n \leq R_\infty.$$

For every fixed edge e ,

$$r_e \theta_n(e)^2 \leq R_\infty,$$

so the sequence $\theta_n(e)$ is bounded.

Since E is countable, enumerate its edges as

$$E = \{e_1, e_2, \dots\}.$$

For each fixed edge e , the sequence $(\theta_n(e))_n$ is bounded. By Bolzano–Weierstrass, first pass to a subsequence on which $\theta_n(e_1)$ converges; then to a further subsequence on which $\theta_n(e_2)$ also converges; and so on. Taking the diagonal subsequence of these nested subsequences gives indices n_j such that, simultaneously for every fixed edge e ,

$$\theta_{n_j}(e) \rightarrow \theta(e).$$

Local finiteness lets us pass the limit through every divergence sum, so θ is a unit flow to infinity.

Finally Fatou's lemma gives

$$\mathcal{E}(\theta) \leq \liminf_j \mathcal{E}(\theta_{n_j}) = R_\infty.$$

Together with the opposite inequality, equality holds. □

12.4.2 Escape Probability

Let

$$\tau_o^+ = \inf\{j \geq 1 : X_j = o\}.$$

Theorem 232.

$$C_{\text{eff}}(o \leftrightarrow \infty) = c_o \Pr[\tau_o^+ = \infty].$$

Consequently,

$$G \text{ is transient} \iff R_{\text{eff}}(o \leftrightarrow \infty) < \infty.$$

Proof. In the finite wired network, impose voltage 1 at o and 0 at ∂_n . The voltage at v is

$$h(v) = \Pr_v[T_o < T_{\partial_n}].$$

Thus the total current leaving o is

$$\begin{aligned} C_{\text{eff}}(o, \partial_n) &= \sum_v c_{ov}(1 - h(v)) \\ &= c_o \sum_v P(o, v) \Pr_v[T_{\partial_n} < T_o] \\ &= c_o \Pr_o[T_{\partial_n} < \tau_o^+]. \end{aligned}$$

Letting $n \rightarrow \infty$,

$$C_{\text{eff}}(o \leftrightarrow \infty) = c_o \Pr[\tau_o^+ = \infty].$$

The right-hand side is positive exactly when the walk is transient. □

Corollary 233 (Green Function). *Let*

$$G(o, o) = \sum_{j \geq 0} \Pr_o[X_j = o]$$

be the expected number of visits to o . Then in a transient network,

$$\boxed{G(o, o) = c_o R_{\text{eff}}(o \leftrightarrow \infty)}.$$

Proof. After every visit to o , the walk escapes forever with probability

$$p = \Pr_o[\tau_o^+ = \infty].$$

Thus the number of visits is geometric with mean $1/p$. Apply [Theorem 232](#). □

12.4.3 Nash–Williams Recurrence Criterion

Theorem 234 (Nash–Williams). *Let*

$$\Pi_1, \Pi_2, \dots$$

be pairwise edge-disjoint cutsets separating o from infinity. Then

$$\boxed{R_{\text{eff}}(o \leftrightarrow \infty) \geq \sum_i \frac{1}{\sum_{e \in \Pi_i} c_e}}.$$

Proof. Let θ be any unit flow to infinity. The net current crossing each Π_i is 1. Hence by weighted Cauchy–Schwarz,

$$1 = \left(\sum_{e \in \Pi_i} \theta_e \right)^2 \leq \left(\sum_{e \in \Pi_i} c_e \right) \left(\sum_{e \in \Pi_i} \frac{\theta_e^2}{c_e} \right).$$

Therefore

$$\sum_{e \in \Pi_i} \frac{\theta_e^2}{c_e} \geq \frac{1}{\sum_{e \in \Pi_i} c_e}.$$

The cutsets are edge-disjoint, so summing gives the claimed lower bound on $\mathcal{E}(\theta)$. Now minimize using infinite Thomson. □

For the unit grid $\mathbb{Z}^2$, take square shells. The r -th shell has

$$O(r)$$

edges, so

$$R_{\text{eff}}(0 \leftrightarrow \infty) \geq \sum_{r \geq 1} \Omega(1/r) = \infty.$$

Thus the two-dimensional random walk is recurrent.

12.4.4 Trees

For a finite binary tree of depth n with all leaves wired together, let R_n be the resistance from the root to the boundary.

Then

$$R_0 = 0$$

and the two branches each have resistance

$$1 + R_n.$$

They occur in parallel, so

$$R_{n+1} = \frac{1 + R_n}{2}.$$

Hence

$$R_n = 1 - 2^{-n}$$

and

$$R_{\text{eff}}(o \leftrightarrow \infty) = 1.$$

The minimizing flow sends

$$2^{-j}$$

through every edge at level j , and its energy is

$$\sum_{j \geq 1} 2^j (2^{-j})^2 = 1.$$

12.5 Random Spanning Trees

Fix a root vertex r . Let

$$B_r \in \mathbb{R}^{E \times (V \setminus \{r\})}$$

be the incidence matrix with the column of r deleted.

12.5.1 Weighted Random Spanning Trees

The weighted spanning-tree distribution is

$$\Pr[T] = \frac{\prod_{e \in T} c_e}{Z},$$

where

$$Z = \sum_T \prod_{e \in T} c_e = \det(B_r^\top C B_r).$$

Define the transfer-current matrix

$$K = C^{1/2} B L^+ B^\top C^{1/2}.$$

Thus

$$K_{ef} = \sqrt{c_e c_f} b_e^\top L^+ b_f.$$

For one edge,

$$K_{ee} = c_e R_{\text{eff}}(e).$$

Theorem 235 (Burton–Pemantle). *For every set of edges*

$$F \subseteq E,$$

$$\Pr[F \subseteq T] = \det K[F, F].$$

Proof. Write

$$F = \{e_1, \dots, e_k\}.$$

Introduce variables

$$z_1, \dots, z_k$$

and multiply the conductance of e_i by

$$1 + z_i.$$

Let

$$U = (\sqrt{c_{e_1}} b_{e_1,r} \quad \cdots \quad \sqrt{c_{e_k}} b_{e_k,r}), \quad Z_F = \text{diag}(z_1, \dots, z_k),$$

where $b_{e,r}$ denotes the incidence vector with the root coordinate deleted.

The reduced Laplacian becomes

$$L_r(z) = L_r + U Z_F U^\top.$$

Matrix–Tree gives

$$\frac{\det L_r(z)}{\det L_r} = \mathbb{E}_T \left[\prod_{e_i \in T} (1 + z_i) \right].$$

Therefore the coefficient of

$$z_1 z_2 \cdots z_k$$

is exactly

$$\Pr[F \subseteq T].$$

The matrix determinant lemma gives

$$\begin{aligned} \frac{\det L_r(z)}{\det L_r} &= \det(I + Z_F U^\top L_r^{-1} U) \\ &= \det(I + Z_F K[F, F]). \end{aligned}$$

The coefficient of

$$\prod_{i=1}^k z_i$$

in this determinant is the principal minor

$$\det K[F, F].$$

Equating coefficients proves the theorem. □

Corollary 236 (Edge Marginal).

$$\boxed{\Pr[e \in T] = c_e R_{\text{eff}}(e).}$$

For an unweighted graph this is simply

$$\Pr[e \in T] = R_{\text{eff}}(e).$$

Corollary 237 (Foster’s Identity).

$$\boxed{\sum_{e \in E} c_e R_{\text{eff}}(e) = n - 1.}$$

Proof. The matrix

$$K = C^{1/2} B L^+ B^\top C^{1/2}$$

is an orthogonal projection:

$$\begin{aligned} K^2 &= C^{1/2} B L^+ (B^\top C B) L^+ B^\top C^{1/2} \\ &= C^{1/2} B L^+ L L^+ B^\top C^{1/2} \\ &= K. \end{aligned}$$

Its rank is $n - 1$, so

$$\text{tr } K = n - 1.$$

But

$$\text{tr } K = \sum_e K_{ee} = \sum_e c_e R_{\text{eff}}(e).$$

□

Corollary 238 (Pairwise Negative Correlation). *For distinct edges e, f ,*

$$\Pr[e, f \in T] \leq \Pr[e \in T] \Pr[f \in T].$$

Proof. Burton–Pemantle gives

$$\Pr[e, f \in T] = \det \begin{pmatrix} K_{ee} & K_{ef} \\ K_{fe} & K_{ff} \end{pmatrix} = K_{ee}K_{ff} - K_{ef}^2.$$

□

12.6 Localization of Electrical Flows

Recall the symmetric transfer-current matrix

$$\Pi = C^{1/2} B L^+ B^\top C^{1/2}.$$

It is an orthogonal projection, so

$$\|\Pi\|_{2 \rightarrow 2} = 1.$$

This bound uses cancellation between positive and negative entries. We want to understand what remains if all signs are discarded.

Write

$$\bar{\Pi} = |\Pi|$$

for entrywise absolute value.

Why localization is interesting. For an unweighted graph, the e -th column of

$$\Pi = B L^+ B^\top$$

is exactly the electrical flow obtained by sending one unit of current between the endpoints of e . Thus

$$\|\Pi \mathbf{1}_e\|_1 = \sum_{f \in E} |\Pi_{f,e}|$$

measures how much total edge usage this electrical flow has.

This is a natural measure of how far the flow spreads. Indeed, an electrical flow has no circulation: current always moves from higher to lower potential. It can therefore be decomposed into s – t paths carrying nonnegative amounts of flow. Since the total flow is 1, its ℓ_1 -norm is the average number of edges used by a unit of current under such a path decomposition. A flow concentrated on short paths has small ℓ_1 -norm, while a flow spread through long routes has large ℓ_1 -norm.

There is no reason for every individual electrical flow to be local. For some edges, the unit current may spread through a large part of the graph. The localization theorem says that this cannot happen for too many source edges at once:

$$\frac{1}{m} \sum_{e \in E} \|\Pi \mathbf{1}_e\|_1 = O(\log n).$$

Equivalently, although exceptional electrical flows may be highly delocalized, the average electrical flow generated by an edge has only $O(\log n)$ total edge usage.

The matrix Π itself does not reveal this, since

$$\|\Pi\|_{2 \rightarrow 2} = 1$$

relies on cancellations between positive and negative entries. To measure unsigned spreading we must instead understand

$$|\Pi|.$$

Theorem 239 (Localization of Electrical Flows). *For every connected weighted graph on n vertices,*

$$\|\bar{\Pi}\|_{2 \rightarrow 2} \leq 2 \log n.$$

We prove the stronger statement

$$w^\top \bar{\Pi} w \leq 2H(\mu_w) \|w\|_2^2,$$

where

$$\mu_w(x) = \frac{\sum_{e \ni x} w_e^2}{2\|w\|_2^2}.$$

Since $H(\mu_w) \leq \log n$, the theorem follows immediately.

12.6.1 Replacing L^+ by a Continuous-Time Walk

Fix $w \geq 0$. We want to bound

$$w^\top \bar{\Pi} w = \sum_{e,f} w_e w_f \sqrt{c_e c_f} |b_e^\top L^+ b_f|.$$

The difficult object here is L^+ . A useful scalar identity is

$$\frac{1}{\lambda} = \int_0^\infty e^{-t\lambda} dt \quad (\lambda > 0).$$

Thus instead of treating an inverse eigenvalue $1/\lambda$ directly, we can regard it as the total contribution over time of $e^{-t\lambda}$.

This suggests representing L^+ by a continuous-time random walk.

For the moment let μ be any strictly positive probability distribution on V , and write

$$M = \text{diag}(\mu).$$

Consider the continuous-time walk whose generator is

$$-M^{-1}L.$$

Its transition matrix at time t is

$$P_t = e^{-tM^{-1}L}.$$

Equivalently, from x the walk jumps to y at rate

$$\frac{c_{xy}}{\mu(x)}.$$

The distribution μ is stationary, and the chain is reversible.

It is convenient to use the symmetric kernel

$$H_t = P_t M^{-1}.$$

By reversibility, H_t is symmetric.

There is also a probabilistic meaning to its columns. If

$$h_t^{(v)} = H_t \mathbf{1}_v,$$

then

$$\Pr_v[X_t = x] = \mu(x) h_t^{(v)}(x).$$

Thus $h_t^{(v)}$ is simply the distribution of the walk started at v , written as a density relative to its stationary distribution μ . In particular,

$$\mu^\top h_t^{(v)} = 1.$$

At $t = 0$,

$$h_0^{(v)}(x) = \frac{\mathbf{1}[x = v]}{\mu(v)},$$

while as $t \rightarrow \infty$,

$$h_t^{(v)}(x) \rightarrow 1.$$

The reason for introducing this walk is the following identity.

Lemma 240 (Integrated Walk Kernel).

$$BL^+B^\top = \int_0^\infty BH_tB^\top dt.$$

Proof. Set

$$S = M^{-1/2}LM^{-1/2}.$$

Then

$$P_t = M^{-1/2}e^{-tS}M^{1/2}, \quad H_t = M^{-1/2}e^{-tS}M^{-1/2}.$$

Diagonalize

$$S = \sum_{i=1}^n \lambda_i \psi_i \psi_i^\top, \quad 0 = \lambda_1 < \lambda_2 \leq \dots \leq \lambda_n.$$

The zero eigenvector is proportional to $M^{1/2}\mathbf{1}$, so

$$BM^{-1/2}\psi_1 = 0.$$

Therefore

$$BH_tB^\top = \sum_{i=2}^n e^{-t\lambda_i} BM^{-1/2}\psi_i \psi_i^\top M^{-1/2}B^\top.$$

Integrating and using

$$\int_0^\infty e^{-t\lambda_i} dt = \frac{1}{\lambda_i}$$

gives

$$\int_0^\infty BH_tB^\top dt = BM^{-1/2}S^+M^{-1/2}B^\top.$$

Let

$$\Gamma = M^{-1/2}S^+M^{-1/2}.$$

For every edge incidence vector b_e ,

$$M^{-1/2}b_e \perp M^{1/2}\mathbf{1} = \ker S,$$

so

$$L\Gamma b_e = M^{1/2}SS^+M^{-1/2}b_e = b_e.$$

Also

$$LL^+b_e = b_e.$$

Hence

$$\Gamma b_e - L^+b_e \in \ker L = \text{span}\{\mathbf{1}\}.$$

Applying B kills this difference, proving the identity. $\square$

So our original quantity satisfies

$$w^\top \bar{\Pi} w \leq \int_0^\infty w^\top C^{1/2} |BH_t B^\top| C^{1/2} w \, dt.$$

We have replaced the inverse L^+ by something more concrete: transition probabilities of a continuous-time walk, integrated over time.

12.6.2 Splitting the Walk at its Midpoint

After replacing L^+ by the time integral of H_t , we are left with matrices of the form

$$BH_t B^\top.$$

The difficulty is that an entry of this matrix involves two edges at once. The semigroup property of the walk can give us more symmetry in the expression by duplicating H_t up to a factor of M . This will allow us to express everything in terms of edge differences of a single function.

Recall that

$$H_t = P_t M^{-1}, \quad M = \text{diag}(\mu).$$

Since $P_{s+t} = P_s P_t$,

$$H_{s+t} = P_s P_t M^{-1} = P_s M^{-1} M P_t M^{-1} = H_s M H_t.$$

In particular,

$$\boxed{H_{2t} = H_t M H_t.}$$

Now expand the diagonal matrix M as

$$M = \sum_{v \in V} \mu(v) \mathbf{1}_v \mathbf{1}_v^\top.$$

Substituting this into the preceding identity gives

$$\begin{aligned} H_{2t} &= H_t \left(\sum_v \mu(v) \mathbf{1}_v \mathbf{1}_v^\top \right) H_t \\ &= \sum_v \mu(v) (H_t \mathbf{1}_v) (\mathbf{1}_v^\top H_t). \end{aligned}$$

Since H_t is symmetric we have $\mathbf{1}_v^\top H_t = (H_t \mathbf{1}_v)^\top$. Writing

$$h_t^{(v)} = H_t \mathbf{1}_v,$$

we obtain

$$\boxed{H_{2t} = \sum_{v \in V} \mu(v) h_t^{(v)} (h_t^{(v)})^\top.}$$

Applying the incidence matrix on both sides gives

$$\begin{aligned} BH_{2t}B^\top &= B \left(\sum_v \mu(v) h_t^{(v)} (h_t^{(v)})^\top \right) B^\top \\ &= \sum_v \mu(v) (Bh_t^{(v)})(Bh_t^{(v)})^\top. \end{aligned}$$

Hence

$$BH_{2t}B^\top = \sum_v \mu(v) (Bh_t^{(v)})(Bh_t^{(v)})^\top.$$

This is exactly the form we wanted. The vector

$$Bh_t^{(v)}$$

has one coordinate for each oriented edge $e = (x, y)$, namely

$$(Bh_t^{(v)})_e = h_t^{(v)}(x) - h_t^{(v)}(y).$$

So it records the edge-by-edge differences of the probability density of a walk started at v .

The original matrix $BH_{2t}B^\top$ contained correlations between every pair of edges. The midpoint identity rewrites it as an average of rank-one matrices

$$(Bh)(Bh)^\top.$$

After taking entrywise absolute values,

$$|BH_{2t}B^\top| \leq \sum_v \mu(v) |Bh_t^{(v)}| |Bh_t^{(v)}|^\top.$$

Therefore, for $w \geq 0$,

$$\begin{aligned} w^\top C^{1/2} |BH_{2t}B^\top| C^{1/2} w \\ \leq \sum_v \mu(v) \left(w^\top C^{1/2} |Bh_t^{(v)}| \right)^2. \end{aligned}$$

Thus it is enough to control, for one starting vertex v ,

$$w^\top C^{1/2} |Bh_t^{(v)}| = \sum_{e=xy} w_e \sqrt{c_e} |h_t^{(v)}(x) - h_t^{(v)}(y)|.$$

This is a much more concrete object: it is simply a weighted sum of how different the time- t walk probabilities are across adjacent vertices. The rest of the proof is devoted to bounding the integral over time of the square of this quantity.

12.6.3 Turning Edge Differences into a Time Derivative

After splitting the walk at its midpoint, it remains to control, for a positive vertex function h ,

$$Q(h) := \left(\sum_{e=xy} w_e \sqrt{c_e} |h_x - h_y| \right)^2.$$

Eventually $h = h_t^{(v)}$ will evolve according to

$$\frac{d}{dt} h_t = -M^{-1} L h_t.$$

Since we will integrate $Q(h_t)$ over all time, ideally, one factor in the bound should be a time derivative, so that its integral telescopes.

Start with weighted Cauchy–Schwarz. For any positive edge scales s_{xy} ,

$$\begin{aligned} Q(h) &= \left(\sum_{e=xy} w_e \sqrt{s_{xy}} \cdot \sqrt{c_e} \frac{|h_x - h_y|}{\sqrt{s_{xy}}} \right)^2 \\ &\leq \left(\sum_{e=xy} w_e^2 s_{xy} \right) \left(\sum_{e=xy} c_e \frac{(h_x - h_y)^2}{s_{xy}} \right). \end{aligned}$$

We now choose s_{xy} .

Consider a real-valued functional

$$F_\varphi(h) = \sum_x \mu(x) \varphi(h_x).$$

The walk traces out a vector-valued path $t \mapsto h_t$; composing with F_φ gives an ordinary real-valued function

$$t \mapsto F_\varphi(h_t).$$

Let

$$F_\varphi(h) = \sum_x \mu(x) \varphi(h_x), \quad M = \text{diag}(\mu).$$

Here

$$\varphi'(h)$$

denotes the vector obtained by applying φ' coordinatewise:

$$\varphi'(h) = (\varphi'(h_x))_{x \in V}.$$

Along the continuous-time walk,

$$\frac{d}{dt} h_t = -M^{-1} L h_t.$$

Applying the ordinary multivariable chain rule,

$$\begin{aligned} \frac{d}{dt} F_\varphi(h_t) &= \sum_x \mu(x) \varphi'(h_t(x)) \frac{d}{dt} h_t(x) \\ &= \varphi'(h_t)^\top M \frac{d}{dt} h_t \\ &= -\varphi'(h_t)^\top L h_t. \end{aligned}$$

Therefore

$$\boxed{-\frac{d}{dt} F_\varphi(h_t) = \varphi'(h_t)^\top L h_t.}$$

Now use

$$L = B^\top C B.$$

Then

$$\begin{aligned} \varphi'(h)^\top L h &= \varphi'(h)^\top B^\top C B h \\ &= (B \varphi'(h))^\top C (B h). \end{aligned}$$

For an oriented edge $e = (x, y)$,

$$(B h)_e = h_x - h_y$$

and

$$(B \varphi'(h))_e = \varphi'(h_x) - \varphi'(h_y).$$

Since C is diagonal with $C_{ee} = c_e$,

$$(B\varphi'(h))^\top C(Bh) = \sum_{e=(x,y)} c_e (\varphi'(h_x) - \varphi'(h_y))(h_x - h_y).$$

Hence

$$-\frac{d}{dt}F_\varphi(h_t) = \sum_{e=xy} c_e (h_x - h_y)(\varphi'(h_x) - \varphi'(h_y)).$$

Notice that the orientation of e does not matter: reversing x and y changes the sign of both factors, so their product is unchanged.

Thus the second Cauchy–Schwarz factor becomes exactly $-\frac{d}{dt}F_\varphi(h_t)$ if we set

$$s_{xy} = \frac{h_x - h_y}{\varphi'(h_x) - \varphi'(h_y)}.$$

For convex φ , this quantity is positive.

There are still many possible choices of φ . To choose one, look at the first Cauchy–Schwarz factor

$$\sum_{e=xy} w_e^2 s(h_x, h_y), \quad s(a, b) = \frac{a - b}{\varphi'(a) - \varphi'(b)}.$$

Ultimately we want to control this using the conserved mass

$$\sum_x \mu(x) h_x = 1.$$

Since conserved mass is linear in h , it is natural to require the edge scale s to scale linearly as well:

$$s(ta, tb) = t s(a, b) \quad (t > 0).$$

In other words, multiplying all densities by t should multiply the corresponding edge scale by t .

Substituting the definition of s , for $a \neq b$,

$$\frac{t(a - b)}{\varphi'(ta) - \varphi'(tb)} = t \frac{a - b}{\varphi'(a) - \varphi'(b)}.$$

Hence

$$\varphi'(ta) - \varphi'(tb) = \varphi'(a) - \varphi'(b).$$

For each fixed t , this says that

$$\varphi'(tx) - \varphi'(x)$$

is independent of x . Differentiating in x ,

$$t\varphi''(tx) = \varphi''(x).$$

Therefore

$$(tx)\varphi''(tx) = x\varphi''(x),$$

so $x\varphi''(x)$ is constant. Thus

$$\varphi''(x) = \frac{c}{x}.$$

Integrating twice,

$$\varphi(x) = cx \log x + \text{affine terms}.$$

Affine terms disappear from $\varphi'(a) - \varphi'(b)$, and an overall positive constant only rescales the two Cauchy–Schwarz factors inversely. We may therefore take

$$\varphi(x) = x \log x.$$

Then

$$\varphi'(x) = 1 + \log x,$$

and the corresponding edge scale is

$$s(a, b) = \frac{a - b}{\log a - \log b} = \Lambda(a, b),$$

the logarithmic mean.

The logarithmic mean has exactly the upper bound we need:

$$\Lambda(a, b) \leq \frac{a + b}{2}.$$

Indeed,

$$\Lambda(a, b) = \int_0^1 a^t b^{1-t} dt,$$

while weighted AM–GM gives

$$a^t b^{1-t} \leq ta + (1 - t)b.$$

Integrating over $t \in [0, 1]$ yields the claim.

We can now carry out Cauchy–Schwarz with

$$s_{xy} = \Lambda(h_x, h_y).$$

The second factor becomes

$$\begin{aligned} \sum_{e=xy} c_e \frac{(h_x - h_y)^2}{\Lambda(h_x, h_y)} &= \sum_{e=xy} c_e (h_x - h_y) (\log h_x - \log h_y) \\ &= h^\top L \log h. \end{aligned}$$

Define

$$I(h) = h^\top L \log h.$$

Then

$$Q(h) \leq \left(\sum_{e=xy} w_e^2 \Lambda(h_x, h_y) \right) I(h).$$

Using

$$\Lambda(h_x, h_y) \leq \frac{h_x + h_y}{2},$$

we obtain

$$\begin{aligned} Q(h) &\leq \frac{1}{2} \left(\sum_{e=xy} w_e^2 (h_x + h_y) \right) I(h) \\ &= \frac{1}{2} \left(\sum_x h_x \sum_{e \ni x} w_e^2 \right) I(h). \end{aligned}$$

The calculation has now produced both objects that will finish the proof: the vertex weights

$$\sum_{e \ni x} w_e^2$$

and the quantity

$$I(h) = h^\top L \log h.$$

12.6.4 Choosing the Stationary Distribution

Recall that the representation of L^+ by the continuous-time walk was valid for any positive stationary distribution μ . We now choose that distribution to simplify the factor that just appeared.

Let

$$d_w(x) = \sum_{e \ni x} w_e^2.$$

Since every edge contributes once at each endpoint,

$$\sum_x d_w(x) = 2 \sum_e w_e^2 = 2\|w\|_2^2.$$

Therefore

$$\mu_w(x) = \frac{d_w(x)}{2\|w\|_2^2}$$

is a probability distribution.

Now let the continuous-time walk use this μ_w as its stationary distribution. Every density $h_t^{(v)}$ arising from the walk satisfies

$$\sum_x \mu_w(x) h_t^{(v)}(x) = 1.$$

Consequently,

$$\begin{aligned} \frac{1}{2} \sum_x h_x d_w(x) &= \frac{1}{2} \sum_x h_x (2\|w\|_2^2 \mu_w(x)) \\ &= \|w\|_2^2. \end{aligned}$$

Thus the preceding inequality simplifies to

$$Q(h) \leq \|w\|_2^2 I(h)$$

for every probability density h relative to μ_w .

In particular,

$$\left(w^\top C^{1/2} |B h_t^{(v)}| \right)^2 \leq \|w\|_2^2 I(h_t^{(v)}).$$

12.6.5 Integrating the Dissipation

It remains to understand

$$\int_0^\infty I(h_t) dt.$$

But $I(h)$ was constructed precisely to be a time derivative.

For a density h relative to $\mu = \mu_w$, define the KL-divergence-like value

$$F(h) = \sum_x \mu(x) h_x \log h_x.$$

Along the walk,

$$\frac{d}{dt}h_t = -M^{-1}Lh_t, \quad M = \text{diag}(\mu).$$

Since

$$\nabla F(h) = M(\mathbf{1} + \log h),$$

the chain rule gives

$$\begin{aligned} \frac{d}{dt}F(h_t) &= \nabla F(h_t)^\top \frac{d}{dt}h_t \\ &= -(\mathbf{1} + \log h_t)^\top Lh_t \\ &= -(\log h_t)^\top Lh_t \\ &= -I(h_t), \end{aligned}$$

where we used

$$\mathbf{1}^\top L = 0.$$

Hence

$$I(h_t) = -\frac{d}{dt}F(h_t).$$

Integrating in time now telescopes:

$$\int_0^\infty I(h_t) dt = F(h_0) - F(h_\infty).$$

For the walk started at v ,

$$h_t^{(v)} = H_t \mathbf{1}_v.$$

At time 0,

$$h_0^{(v)}(x) = \frac{\mathbf{1}[x = v]}{\mu(v)},$$

while as $t \rightarrow \infty$, the walk converges to stationarity, so

$$h_t^{(v)} \rightarrow \mathbf{1}.$$

Therefore

$$F(h_\infty^{(v)}) = 0,$$

while, with the convention $0 \log 0 = 0$,

$$F(h_0^{(v)}) = \mu(v) \frac{1}{\mu(v)} \log \frac{1}{\mu(v)} = \log \frac{1}{\mu(v)}.$$

Thus

$$\int_0^\infty I(h_t^{(v)}) dt = \log \frac{1}{\mu(v)}.$$

Strictly speaking, $h_0^{(v)}$ has zero coordinates, whereas the derivative calculation used $\log h_t$. For every $t > 0$, connectedness makes $h_t^{(v)}$ strictly positive. Applying the identity on $[\varepsilon, T]$ and then taking $\varepsilon \downarrow 0$ and $T \rightarrow \infty$ gives the displayed formula.

Thus the logarithm that arose locally from the logarithmic mean has now turned into

$$\log \frac{1}{\mu(v)}$$

after integration over the walk.

12.6.6 Completing the Localization Bound

We now return to

$$\Pi = C^{1/2}BL^+B^\top C^{1/2}.$$

Assume first that $w_e > 0$ for every edge, so that $\mu_w(x) > 0$ for every vertex.

The integrated-walk representation gives

$$\Pi = \int_0^\infty C^{1/2}BH_tB^\top C^{1/2} dt.$$

Taking entrywise absolute values,

$$|\Pi| \leq \int_0^\infty C^{1/2}|BH_tB^\top|C^{1/2} dt.$$

Therefore, after writing $t = 2s$,

$$w^\top |\Pi| w \leq 2 \int_0^\infty w^\top C^{1/2}|BH_{2s}B^\top|C^{1/2}w ds.$$

From the midpoint decomposition,

$$H_{2s} = \sum_v \mu(v) h_s^{(v)} (h_s^{(v)})^\top,$$

and hence

$$|BH_{2s}B^\top| \leq \sum_v \mu(v) |Bh_s^{(v)}| |Bh_s^{(v)}|^\top.$$

Thus

$$\begin{aligned} w^\top |\Pi| w &\leq 2 \sum_v \mu(v) \int_0^\infty \left(w^\top C^{1/2} |Bh_s^{(v)}| \right)^2 ds \\ &\leq 2 \|w\|_2^2 \sum_v \mu(v) \int_0^\infty I(h_s^{(v)}) ds \\ &= 2 \|w\|_2^2 \sum_v \mu(v) \log \frac{1}{\mu(v)}. \end{aligned}$$

Writing

$$H(\mu) = \sum_v \mu(v) \log \frac{1}{\mu(v)},$$

we have proved

$$\boxed{w^\top |\Pi| w \leq 2H(\mu_w) \|w\|_2^2.}$$

For an arbitrary signed vector w ,

$$|w^\top |\Pi| w| \leq |w|^\top |\Pi| |w|.$$

If some coordinates of w vanish, apply the preceding argument to

$$|w| + \varepsilon \mathbf{1}$$

and let $\varepsilon \downarrow 0$. With $0 \log 0 = 0$, the entropy is continuous in this limit. Hence for every w ,

$$\boxed{|w^\top |\Pi| w| \leq 2H(\mu_w) \|w\|_2^2.}$$

Finally, every probability distribution on n points satisfies

$$H(\mu_w) \leq \log n.$$

Since $|\Pi|$ is symmetric,

$$\boxed{\| |\Pi| \|_{2 \rightarrow 2} \leq 2 \log n.}$$

13 Polynomial Algorithms

13.1 Formal Power Series Operations

13.1.1 Inverse

To compute the inverse $F = A^{-1}$, we want to find the root of the equation $\Phi(F) = F^{-1} - A = 0$.

Taking the derivative with respect to F gives $\Phi'(F) = -F^{-2}$. Plugging this into the Newton iteration formula $F_{k+1} = F_k - \frac{\Phi(F_k)}{\Phi'(F_k)}$ yields:

$$F_{k+1} = F_k - \frac{F_k^{-1} - A}{-F_k^{-2}} = F_k + F_k(1 - AF_k)$$

$$F_{k+1} = F_k(2 - AF_k) \pmod{x^{2^{k+1}}}$$

Because F_k is already the inverse modulo x^k , we know that $AF_k \equiv 1 \pmod{x^k}$. We can express this product as $AF_k = 1 + x^k E$, where E represents the upper coefficients of the product.

Substituting this into our Newton iteration:

$$F_{k+1} = F_k - F_k(AF_k - 1) = F_k - F_k(x^k E)$$

$$F_{k+1} = F_k - x^k(F_k E \pmod{x^k})$$

This confirms that the lower k coefficients of F_{k+1} are exactly F_k . To compute the upper k coefficients, we only need to extract E and multiply it by F_k .

To avoid the cost of negating the result array at every step, the algorithm maintains the negative inverse $R_k = -F_k$. Evaluating the product yields $AR_k = -1 - x^k E$. By shifting the coefficients right by k , we isolate $-E$. Multiplying this by R_k yields $(-E)(-F_k) = EF_k$.

Because $F_{k+1} = F_k - x^k(EF_k)$, we have $-F_{k+1} = -F_k + x^k(EF_k)$, meaning R_{k+1} is formed by writing EF_k into the upper half of R_k .

13.1.2 Logarithm

To compute $F = \ln(A)$, we take the formal derivative:

$$F' = \frac{A'}{A}$$

Then we just formally integrate:

$$F = \int \frac{A'}{A} dx$$

13.1.3 Square Root

To compute the square root $F = \sqrt{A}$, we want to solve $\Phi(F) = F^2 - A = 0$.

The derivative is $\Phi'(F) = 2F$. The standard Newton iteration becomes:

$$F_{k+1} = F_k - \frac{F_k^2 - A}{2F_k} \pmod{x^{2^{k+1}}}$$

We simultaneously maintain the inverse $G_k \equiv F_k^{-1} \pmod{x^{2^k}}$.

1. Updating the Square Root We can rewrite the square root iteration using G_k :

$$F_{k+1} = F_k - \frac{1}{2}G_k(F_k^2 - A) \pmod{x^{2^{k+1}}}$$

Because $F_k^2 \equiv A \pmod{x^{2^k}}$ by definition of the previous step, the lower 2^k coefficients of $(F_k^2 - A)$ are guaranteed to be zero. Let Δ represent the non-zero upper coefficients of this error term. We can compute F_{k+1} strictly by evaluating:

$$F_{k+1} = F_k - x^{2^k} \left(\frac{1}{2}G_k\Delta \pmod{x^{2^k}} \right)$$

2. Updating the Inverse Once F_{k+1} is computed, we must update our maintained inverse so that $G_{k+1} \equiv F_{k+1}^{-1} \pmod{x^{2^{k+1}}}$. We apply one step of the standard Newton iteration for polynomial inversion, substituting our newly found F_{k+1} :

$$G_{k+1} = G_k(2 - F_{k+1}G_k) \pmod{x^{2^{k+1}}}$$

$$G_{k+1} = G_k - G_k(F_{k+1}G_k - 1) \pmod{x^{2^{k+1}}}$$

Just like the square root update, the lower 2^k coefficients of $(F_{k+1}G_k - 1)$ are guaranteed to vanish. We extract the upper coefficients and multiply them by G_k to find the upper half of G_{k+1} .

13.1.4 Inverse Square Root

To compute the inverse square root $F = 1/\sqrt{A}$, we want to solve $\Phi(F) = F^{-2} - A = 0$.

The derivative is $\Phi'(F) = -2F^{-3}$. The iteration becomes:

$$F_{k+1} = F_k - \frac{F_k^{-2} - A}{-2F_k^{-3}} = F_k + \frac{F_k}{2}(1 - AF_k^2)$$

$$F_{k+1} = \frac{F_k}{2}(3 - AF_k^2) \pmod{x^{2^{k+1}}}$$

13.1.5 Exponential

The newton iteration for exponential $F = \exp(A)$ is $F_{2k} \equiv F_k(1 + A_{2k} - \ln F_k) \pmod{x^{2^k}}$.

We define the differential operator $\theta = x \frac{d}{dx}$. Because $F_k \equiv \exp(A_k) \pmod{x^k}$, we have $\theta F_k \equiv (\theta A_k) \exp(A_k) \equiv (\theta A_k) F_k \pmod{x^{k-1}}$. Let the residual be V :

$$V = (\theta A_k) F_k - \theta F_k$$

We have that $V \equiv 0 \pmod{x^{k-1}}$. Furthermore, θF_k has degree at most $k-1$. Therefore:

$$V_{\geq k} = ((\theta A_k) F_k)_{\geq k}$$

We will use $\ln F_k = \int F_k' F_k^{-1}$. Let $G_{2k} := F_k^{-1} \pmod{x^{2^k}}$. Rearranging the residual definition yields $\theta F_k = (\theta A_k) F_k - V$. Multiplying both sides by G_{2k} gives:

$$(\theta F_k) G_{2k} = (\theta A_k) F_k G_{2k} - V G_{2k}$$

Since $F_k G_{2k} \equiv 1 \pmod{x^{2^k}}$, we have:

$$(\theta F_k) G_{2k} \equiv \theta A_k - V G_{2k} \pmod{x^{2^{k-1}}}$$

Consider $VG_{2k} \pmod{x^{2k}}$. V has no terms below degree $k - 1$. If we multiply V by any term in G_{2k} of degree k or higher, the resulting degree will be at least $2k - 1$, thus we only need the first k terms of the inverse, which is G_k , which we already computed.

$$VG_{2k} \equiv VG_k \pmod{x^{2k-1}}$$

Substituting this back yields:

$$(\theta F_k)G_{2k} \equiv \theta A_k - VG_k \pmod{x^{2k-1}}$$

Applying the inverse differential operator θ^{-1} (equivalent to $F \mapsto \int \frac{F}{x}$) to both sides gives:

$$\ln F_k \equiv A_k - \theta^{-1}(VG_k) \pmod{x^{2k}}$$

Substituting this into the Newton iteration yields the update rule:

$$F_{2k} \equiv F_k + F_k (A_{2k} - A_k + \theta^{-1}(VG_k)) \pmod{x^{2k}}$$

Algorithm 31 FPS Exponential

Input: Series $A(x)$, target precision N

Output: Series $F(x) \equiv \exp(A(x)) \pmod{x^N}$

```

1: function EXPOPTIMIZED( $A, N$ )
2:    $F \leftarrow 1; G \leftarrow 1$   $\triangleright G$  maintains  $F^{-1}$ 
3:    $k \leftarrow 1$ 
4:   while  $k < N$  do
5:      $V \leftarrow ((\theta A_k)F)_{\geq k} \pmod{x^{2k}}$ 
6:      $W \leftarrow VG \pmod{x^{2k}}$ 
7:      $C \leftarrow A_{2k} - A_k + \theta^{-1}(W)$ 
8:      $F \leftarrow F + F \cdot C \pmod{x^{2k}}$ 
9:      $G \leftarrow G(2 - FG) \pmod{x^{2k}}$ 
10:     $k \leftarrow 2k$ 
11: return  $F$ 

```

13.1.6 Power

1. Logarithm and Exponential We have the identity:

$$G = F^k = \exp(k \ln F)$$

To use this identity formally, the constant term of F must be 1 so that $\ln F$ is well-defined. If $F(x) = c_0 + c_1x + \dots$ where $c_0 \neq 1$, we can factor out the constant term:

$$F^k = c_0^k \exp\left(k \ln\left(\frac{F}{c_0}\right)\right)$$

Note: If the lowest non-zero term of F is c_mx^m , we factor out x^{mk} , apply the algorithm to the shifted series, and then shift the result back by mk .

2. Recurrence We can derive a recurrence by differentiating $G = F^k$.

Taking the formal derivative of both sides gives:

$$G' = kF^{k-1}F'$$

Multiplying both sides by F we get:

$$FG' = kGF'$$

Let $F(x) = \sum c_n x^n$ and $G(x) = \sum q_n x^n$. We can extract the coefficient of x^{i-1} on both sides of the equation. On the left side (FG'):

$$[x^{i-1}](FG') = \sum_{j=0}^i c_j (i-j) q_{i-j} = c_0 i q_i + \sum_{j=1}^i c_j (i-j) q_{i-j}$$

On the right side (kGF'):

$$[x^{i-1}](kGF') = k \sum_{j=0}^i q_{i-j} (j c_j) = k \sum_{j=1}^i j c_j q_{i-j}$$

Equating the two sides and isolating the q_i term yields:

$$c_0 i q_i = \sum_{j=1}^i k j c_j q_{i-j} - \sum_{j=1}^i (i-j) c_j q_{i-j}$$

$$q_i = \frac{1}{i c_0} \sum_{j=1}^i (k j - (i-j)) c_j q_{i-j}$$

We start with the base case $q_0 = c_0^k$. If $\deg F = d$, the inner summation only runs up to $\min(i, d)$, so the time complexity is $O(Nd)$.

13.2 Polynomial Arithmetic

13.2.1 Division

Given $A(x)$ and $B(x)$, we want to find the unique quotient $Q(x)$ and remainder $R(x)$ such that:

$$A(x) = Q(x)B(x) + R(x)$$

where $\deg(R) < \deg(B) = m$.

We substitute $x \leftarrow 1/x$ into the division identity and multiply both sides by x^n :

$$x^n A(1/x) = x^{n-m} Q(1/x) \cdot x^m B(1/x) + x^{n-m+1} \cdot x^{m-1} R(1/x)$$

Using the reversal definition, this perfectly rewrites to:

$$A_{\text{rev}}(x) = Q_{\text{rev}}(x) B_{\text{rev}}(x) + x^{d+1} R_{\text{rev}}(x)$$

Taking the equation modulo x^{d+1} :

$$A_{\text{rev}}(x) \equiv Q_{\text{rev}}(x) B_{\text{rev}}(x) \pmod{x^{d+1}}$$

Since $B(x)$ has degree m , its leading coefficient is non-zero. Therefore, the constant term of $B_{\text{rev}}(x)$ is non-zero, guaranteeing that $B_{\text{rev}}(x)$ is invertible. Thus:

$$Q_{\text{rev}}(x) \equiv A_{\text{rev}}(x) B_{\text{rev}}^{-1}(x) \pmod{x^{d+1}}$$

To extract the quotient, we invert $B_{\text{rev}}(x)$ modulo x^{d+1} in $O(n \log n)$. We then multiply it by $A_{\text{rev}}(x)$, truncate to degree d , and reverse the output.

We then compute the remainder as $R(x) = A(x) - Q(x)B(x)$.

Algorithm 32 Division

```
1: function DIVMOD( $A(x), B(x)$ )
2:    $n \leftarrow \deg(A)$ 
3:    $m \leftarrow \deg(B)$ 
4:    $d \leftarrow n - m$ 
5:   if  $d < 0$  then
6:     return  $(0, A(x))$ 
7:   if  $\min(d, m) \leq \text{THRESHOLD}$  then
8:     return NAIVEDIVMOD( $A(x), B(x)$ )
9:    $A_{\text{rev}}(x) \leftarrow \text{Reverse}(A(x)) \pmod{x^{d+1}}$ 
10:   $B_{\text{rev}}(x) \leftarrow \text{Reverse}(B(x)) \pmod{x^{d+1}}$ 
11:   $B_{\text{inv}}(x) \leftarrow \text{PolyInv}(B_{\text{rev}}(x), d + 1)$ 
12:   $Q_{\text{rev}}(x) \leftarrow (A_{\text{rev}}(x) \cdot B_{\text{inv}}(x)) \pmod{x^{d+1}}$ 
13:   $Q(x) \leftarrow \text{Reverse}(Q_{\text{rev}}(x), d)$ 
14:   $R(x) \leftarrow A(x) - (Q(x) \cdot B(x))$ 
15:  return  $(Q(x), R(x))$ 
```

13.2.2 Shift

The Taylor shift operation transforms a polynomial $P(x)$ into $P(x + c)$ for some scalar c in $O(n \log n)$ time.

Derivation Let $P(x) = \sum_{i=0}^n p_i x^i$. We want to compute the coefficients of $P(x + c)$:

$$P(x + c) = \sum_{i=0}^n p_i (x + c)^i$$

We expand $(x + c)^i$:

$$P(x + c) = \sum_{i=0}^n p_i \sum_{j=0}^i \binom{i}{j} c^{i-j} x^j$$

Expand the binomial coefficient $\binom{i}{j} = \frac{i!}{j!(i-j)!}$ and swap the order of summation to group by the powers of x :

$$P(x + c) = \sum_{j=0}^n \frac{x^j}{j!} \sum_{i=j}^n (p_i i!) \frac{c^{i-j}}{(i-j)!}$$

Notice that the inner summation evaluates the coefficient of x^j in the shifted polynomial. Let q_j denote this inner sum:

$$q_j = \sum_{i=j}^n (p_i i!) \frac{c^{i-j}}{(i-j)!}$$

This is a cross-correlation (or transposed convolution) because the indices in the product subtract to j . To evaluate this using NTT, we convert it into a convolution by reversing one of the sequences.

Let A be the sequence defined by $A_i = p_i i!$, and let A^{rev} be its reversal, so $A_k^{\text{rev}} = A_{n-k}$.

Let B be the sequence defined by $B_k = \frac{c^k}{k!}$.

Substitute $i = n - k$. The limits $i = j \dots n$ become $k = 0 \dots n - j$. The indices in our sum become:

$$q_j = \sum_{k=0}^{n-j} A_{n-(j+k)} B_k = \sum_{k=0}^{n-j} A_{n-j-k}^{\text{rev}} B_k$$

Thus q_j is the coefficient of x^{n-j} in the product polynomial $C(x) = A^{\text{rev}}(x)B(x)$.

Once $C(x)$ is computed, we simply read the coefficients backwards and divide by $j!$ to construct the final shifted polynomial.

Algorithm 33 Shift

Input: Polynomial $P(x)$ of degree n , scalar c

Output: Polynomial $P(x + c)$

```

1: function TAYLORSHIFT( $P, c$ )
2:    $A^{\text{rev}} \leftarrow$  array of size  $n + 1$ 
3:   for  $i \leftarrow 0$  to  $n$  do
4:      $A^{\text{rev}}[n - i] \leftarrow P[i] \times i!$ 
5:    $B \leftarrow$  array of size  $n + 1$ 
6:    $c_{\text{pow}} \leftarrow 1$ 
7:   for  $i \leftarrow 0$  to  $n$  do
8:      $B[i] \leftarrow \frac{c_{\text{pow}}}{i!}$ 
9:      $c_{\text{pow}} \leftarrow c_{\text{pow}} \times c$ 
10:   $C(x) \leftarrow A^{\text{rev}}(x) \times B(x)$ 
11:   $R \leftarrow$  array of size  $n + 1$ 
12:  for  $j \leftarrow 0$  to  $n$  do
13:     $R[j] \leftarrow \frac{C[n-j]}{j!}$ 
14:  return  $R(x)$ 

```

13.2.3 Evaluation Shift

Let

$$f \in \mathbb{F}[x], \quad \deg f \leq d,$$

and suppose we know

$$f(0), f(1), \dots, f(d).$$

Given any $s \in \mathbb{F}$, we want

$$f(s), f(s+1), \dots, f(s+d).$$

Lemma 241 (Polynomial Evaluation Shift). *Given*

$$f(0), \dots, f(d),$$

the values

$$f(s), \dots, f(s+d)$$

can be computed in

$$\boxed{O(d \log d)},$$

time.

Proof. Let

$$P(x) = \prod_{j=0}^d (x - j).$$

Lagrange interpolation gives

$$f(x) = P(x) \sum_{i=0}^d \frac{f(i)}{P'(i)(x - i)}.$$

Now

$$P'(i) = (-1)^{d-i} i! (d-i)!,$$

so if

$$a_i = \frac{(-1)^{d-i} f(i)}{i! (d-i)!},$$

then

$$f(s+k) = P(s+k) \sum_{i=0}^d \frac{a_i}{s+k-i}.$$

For all $k = 0, \dots, d$, the sums

$$\sum_{i=0}^d \frac{a_i}{s+k-i}$$

form a cross-correlation of the two sequences

$$(a_i)_{i=0}^d \quad \text{and} \quad \left(\frac{1}{s-d+j} \right)_{j=0}^{2d},$$

and hence are obtained by one convolution.

Finally,

$$P(s) = \prod_{j=0}^d (s-j),$$

and the remaining values are obtained in linear time from

$$\boxed{P(s+k+1) = P(s+k) \frac{s+k+1}{s+k-d}}.$$

□

If some target point coincides with one of

$$0, \dots, d,$$

that value is already known and can simply be separated out.

More generally, if the known evaluations are

$$f(a), f(a+h), \dots, f(a+dh),$$

with $h \neq 0$, define

$$g(x) = f(a+hx).$$

A shift by δ in the original coordinates becomes a shift by

$$\frac{\delta}{h}$$

for g .

13.2.4 GCD

HGCD: Given polynomials $A, B \in \mathbb{K}[x]$ with $\deg(A) > \deg(B) \geq 0$, the Half-GCD algorithm returns a regular matrix R such that:

$$\begin{pmatrix} A' \\ B' \end{pmatrix} = R \begin{pmatrix} A \\ B \end{pmatrix}$$

where the degrees of the output straddle half the degree of A . Specifically:

$$\deg(A') \geq \lceil \deg(A)/2 \rceil > \deg(B')$$

The HGCD Algorithm The idea is that some prefix of the quotient sequence is determined only by the high degree coefficients. The algorithm considers a threshold $m = \lceil \deg(A)/2 \rceil$. It calls itself on the upper halves of the polynomials shifted right by m . After applying the returned matrix R_1 , the resulting polynomials A' and B' will straddle $\approx 3m/2$. Then we perform one Euclidean division step to decrease the degree below m , then make a second recursive call.

Algorithm 34 Half-GCD

```

1: function HALFGCD( $A, B$ )
2:    $n \leftarrow \deg(A)$ 
3:    $m \leftarrow \lceil n/2 \rceil$ 
4:   if  $\deg(B) < m$  then
5:     return  $E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ 
6:    $A_0 \leftarrow A \operatorname{div} x^m$ 
7:    $B_0 \leftarrow B \operatorname{div} x^m$ 
8:    $R_1 \leftarrow \text{HALFGCD}(A_0, B_0)$ 
9:    $\begin{pmatrix} A' \\ B' \end{pmatrix} \leftarrow R_1 \begin{pmatrix} A \\ B \end{pmatrix}$ 
10:  if  $\deg(B') < m$  then
11:    return  $R_1$ 
12:   $Q \leftarrow A' \operatorname{div} B'$ 
13:   $R \leftarrow A' \bmod B'$ 
14:   $C \leftarrow B'$ 
15:   $D \leftarrow R$ 
16:   $k \leftarrow 2m - \deg(C)$ 
17:   $C_0 \leftarrow C \operatorname{div} x^k$ 
18:   $D_0 \leftarrow D \operatorname{div} x^k$ 
19:   $R_2 \leftarrow \text{HALFGCD}(C_0, D_0)$ 
20:  return  $R_2 \cdot \begin{pmatrix} 0 & 1 \\ 1 & -Q \end{pmatrix} \cdot R_1$ 

```

Correctness

Definition 242 (Regular Matrix). A matrix associated with a quotient Q is defined as $E_Q = \begin{pmatrix} Q & 1 \\ 1 & 0 \end{pmatrix}$.

A matrix W is called regular if it is the product of elementary matrices associated a Euclidean remainder sequence: $W = E_{Q_1} E_{Q_2} \cdots E_{Q_k}$, where $\deg(Q_i) \geq 1$ for all i .

Lemma 243 (Ordering Property of Regular Matrices). *Let $W = \begin{pmatrix} P & Q \\ S & T \end{pmatrix}$ be a regular matrix constructed from quotients $Q_1, \dots, Q_k$ with $\deg(Q_i) \geq 1$. The degrees of the entries satisfy:*

$$\deg(P) = \sum_{i=1}^k \deg(Q_i)$$

$$\deg(P) \geq \max(\deg(Q), \deg(S)) \geq \min(\deg(Q), \deg(S)) \geq \deg(T)$$

Furthermore, $\deg(P) > \deg(Q)$ and $\deg(P) > \deg(S)$.

Proof. We proceed by induction on k . Base case $k = 1$: $W = \begin{pmatrix} Q_1 & 1 \\ 1 & 0 \end{pmatrix}$. Here $\deg(P) = \deg(Q_1) \geq 1$, $\deg(Q) = \deg(S) = 0$, and $\deg(T) = -\infty$.

Inductive step: Assume $W_k = \begin{pmatrix} P_k & Q_k \\ S_k & T_k \end{pmatrix}$ satisfies the property. For $W_{k+1} = W_k \begin{pmatrix} Q_{k+1} & 1 \\ 1 & 0 \end{pmatrix}$, we have:

$$W_{k+1} = \begin{pmatrix} P_k Q_{k+1} + Q_k & P_k \\ S_k Q_{k+1} + T_k & S_k \end{pmatrix}$$

Because $\deg(P_k) > \deg(Q_k)$ by the inductive hypothesis, the degree of the new top-left entry is exactly $\deg(P_k) + \deg(Q_{k+1})$. Since $\deg(Q_{k+1}) \geq 1$, this strictly dominates the degree of P_k , preserving the property. $\square$

We prove that applying a transformation matrix computed from truncated top-coefficients correctly reduces the full polynomials.

Lemma 244 (Parallel Reduction Criteria). *Let $A, B \in \mathbb{K}[x]$ be polynomials, and let $m \geq 1$ be a truncation threshold. Define $A_0 = A \operatorname{div} x^m$ and $B_0 = B \operatorname{div} x^m$.*

Let R_1 be the matrix returned by $\text{HGCD}(A_0, B_0)$ such that $\begin{pmatrix} A'_0 \\ B'_0 \end{pmatrix} = R_1 \begin{pmatrix} A_0 \\ B_0 \end{pmatrix}$. If $\deg(A'_0) \geq \lceil \deg(A_0)/2 \rceil > \deg(B'_0)$, then applying R_1 to the full polynomials (A, B) yields (A', B') satisfying:

$$\deg(A') = \deg(A'_0) + m \quad \text{and} \quad \deg(A') > \deg(B')$$

Proof. We can decompose the original polynomials as $A = A_0 x^m + A_1$ and $B = B_0 x^m + B_1$, where the residues strictly satisfy $\deg(A_1) < m$ and $\deg(B_1) < m$.

Let $R_1^{-1} = W = \begin{pmatrix} P & Q \\ S & T \end{pmatrix}$. We have $A_0 = PA'_0 + QB'_0$. Since $\deg(A'_0) > \deg(B'_0)$ and $\deg(P) > \deg(Q)$ (by [Definition 242](#)), the degree of the product PA'_0 dominates QB'_0 . Therefore:

$$\deg(A_0) = \deg(P) + \deg(A'_0) \implies \deg(P) = \deg(A_0) - \deg(A'_0)$$

Now we apply the forward reduction matrix R_1 to the full polynomials. Since $\det(W) = \delta = \pm 1$, we have $R_1 = \delta \begin{pmatrix} T & -Q \\ -S & P \end{pmatrix}$. By linearity:

$$\begin{pmatrix} A' \\ B' \end{pmatrix} = R_1 \begin{pmatrix} A_0 x^m + A_1 \\ B_0 x^m + B_1 \end{pmatrix} = \begin{pmatrix} A'_0 x^m \\ B'_0 x^m \end{pmatrix} + \delta \begin{pmatrix} TA_1 - QB_1 \\ -SA_1 + PB_1 \end{pmatrix}$$

Let A'_1 and B'_1 be the error terms. We upper-bound their degrees:

$$\begin{aligned} \deg(A'_1) &\leq \max(\deg(T) + \deg(A_1), \deg(Q) + \deg(B_1)) \\ &\leq \max(\deg(T), \deg(Q)) + m - 1 \\ &\leq \deg(P) + m - 1 \quad (\text{by } \text{Definition 242}) \end{aligned}$$

Substitute $\deg(P) = \deg(A_0) - \deg(A'_0)$ into the bound:

$$\deg(A'_1) \leq \deg(A_0) - \deg(A'_0) + m - 1$$

For A' to be determined by the leading term $A'_0 x^m$, we require $\deg(A'_0 x^m) > \deg(A'_1)$. We evaluate this:

$$\deg(A'_0) + m > \deg(A_0) - \deg(A'_0) + m - 1$$

$$2 \deg(A'_0) \geq \deg(A_0) \implies \deg(A'_0) \geq \lceil \deg(A_0)/2 \rceil$$

Which we assume, so $\deg(A') = \deg(A'_0) + m$.

By the same logic, we upper-bound the error B'_1 :

$$\begin{aligned}\deg(B'_1) &\leq \max(\deg(S) + \deg(A_1), \deg(P) + \deg(B_1)) \\ &\leq \deg(P) + m - 1 = \deg(A_0) - \deg(A'_0) + m - 1\end{aligned}$$

Since $B' = B'_0 x^m + B'_1$, its degree is bounded by:

$$\deg(B') \leq \max(\deg(B'_0) + m, \deg(A_0) - \deg(A'_0) + m - 1)$$

We compare this against $\deg(A') = \deg(A'_0) + m$. Since $\deg(B'_0) \leq \deg(A'_0) - 1$, the first term inside the max is strictly less than $\deg(A')$. Since $2\deg(A'_0) \geq \deg(A_0)$, we have $\deg(A'_0) \geq \deg(A_0) - \deg(A'_0)$. Therefore, the second term $\deg(A_0) - \deg(A'_0) + m - 1 \leq \deg(A'_0) + m - 1$, which is also strictly less than $\deg(A')$. Thus, $\deg(A') > \deg(B')$. $\square$

Lemma 245 (HGCD Straddling). *Let $A, B \in \mathbb{K}[x]$ be polynomials with $\deg(A) > \deg(B) \geq 0$. The HGCD(A, B) algorithm returns a matrix R mapping $(A, B) \rightarrow (C', D')$ such that the final degrees straddle $m = \lceil \deg(A)/2 \rceil$:*

$$\deg(C') \geq m > \deg(D')$$

Proof. We induct on the degree of A .

Let $m = \lceil \deg(A)/2 \rceil$. The first recursive call $\text{HGCD}(A_0, B_0)$ returns R_1 . By the inductive hypothesis, R_1 correctly straddles $\lceil \deg(A_0)/2 \rceil$, satisfying the premise of [Lemma 244](#). Applying R_1 yields A', B' with $\deg(A') \geq m > \deg(B'_0)$.

Case 1: If $\deg(B') < m$, the algorithm exits at Step 2. Since $\deg(A') \geq m$, the straddling condition $\deg(A') \geq m > \deg(B')$ is satisfied.

Case 2: If $\deg(B') \geq m$, the algorithm proceeds to Step 3. It computes $C = B'$ and $D = A' \bmod B'$. We have $\deg(C) > \deg(D)$ and because we did not exit, we know $\deg(C) \geq m$. To find the second threshold, we define $k = 2m - \deg(C)$. We must verify that $k \geq 0$ and that $\deg(C) \geq k$.

From [Lemma 244](#), $\deg(C) = \deg(B') \leq \max(\deg(B'_0) + m, \deg(A_0) - \deg(A'_0) + m - 1)$. Using $\deg(A'_0) \geq \lceil \deg(A_0)/2 \rceil$, this evaluates to $\deg(C) < \frac{3}{2}m$. Since $\deg(C) < 2m$, we have $k > 0$. Furthermore, because $\deg(C) \geq m$, it follows that $2\deg(C) \geq 2m$, which implies $\deg(C) \geq 2m - \deg(C) = k$.

In Step 4, we make the second recursive call $\text{HGCD}(C_0, D_0)$ where $C_0 = C \operatorname{div} x^k$ and $D_0 = D \operatorname{div} x^k$. By definition, $\deg(C_0) = \deg(C) - k$. Substitute k :

$$\deg(C_0) = \deg(C) - (2m - \deg(C)) = 2\deg(C) - 2m$$

The inductive hypothesis on this second call guarantees that the returned matrix R_2 maps $(C_0, D_0) \rightarrow (C'_0, D'_0)$ such that:

$$\deg(C'_0) \geq \lceil \deg(C_0)/2 \rceil = \deg(C) - m \quad \text{and} \quad \deg(C'_0) > \deg(D'_0)$$

This satisfies the premises of [Lemma 244](#) for (C, D) with threshold k . Applying R_2 to the full (C, D) yields the final polynomials (C', D') .

We now calculate the bounds of the final degrees. By [Lemma 244](#):

$$\deg(C') = \deg(C'_0) + k \geq (\deg(C) - m) + (2m - \deg(C)) = m$$

For the remainder D' , the lemma bounds its degree by:

$$\deg(D') \leq \max(\deg(D'_0) + k, \deg(C_0) - \deg(C'_0) + k - 1)$$

We bound both terms. Since the inductive hypothesis forces $\deg(D'_0) \leq \lceil \deg(C_0)/2 \rceil - 1 = \deg(C) - m - 1$, we have:

$$\deg(D'_0) + k \leq (\deg(C) - m - 1) + (2m - \deg(C)) = m - 1 < m$$

We use the lower bound $\deg(C'_0) \geq \deg(C) - m$, which implies $-\deg(C'_0) \leq m - \deg(C)$:

$$\begin{aligned} \deg(C_0) - \deg(C'_0) + k - 1 &\leq (2\deg(C) - 2m) + (m - \deg(C)) + (2m - \deg(C)) - 1 \\ &= m - 1 < m \end{aligned}$$

We conclude $\deg(D') < m$. Therefore, the final degrees satisfy $\deg(C') \geq m > \deg(D')$. $\square$

Full GCD Algorithm To compute the complete Extended GCD of A and B , we interleave HGCD with a Euclidean division step.

Algorithm 35 GCD

```

1: function GCD( $A, B$ )
2:    $M_{\text{full}} \leftarrow \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ 
3:   while  $B \neq 0$  do
4:      $Q \leftarrow A \text{ div } B$ 
5:      $R \leftarrow A \bmod B$ 
6:      $M_{\text{step}} \leftarrow \begin{pmatrix} 0 & 1 \\ 1 & -Q \end{pmatrix}$ 
7:      $A \leftarrow B; \quad B \leftarrow R$ 
8:      $M_{\text{full}} \leftarrow M_{\text{step}} \cdot M_{\text{full}}$ 
9:      $R_{\text{half}} \leftarrow \text{HALFGCD}(A, B)$ 
10:     $\begin{pmatrix} A \\ B \end{pmatrix} \leftarrow R_{\text{half}} \begin{pmatrix} A \\ B \end{pmatrix}$ 
11:     $M_{\text{full}} \leftarrow R_{\text{half}} \cdot M_{\text{full}}$ 
12:  return  $A, M_{\text{full}}$ 

```

13.2.5 Modular Inversion

Given a polynomial $A(x)$ and a modulus polynomial $M(x)$, the modular inversion problem asks to find a polynomial $A^{-1}(x)$ such that:

$$A(x)A^{-1}(x) \equiv 1 \pmod{M(x)}$$

This can be computed in $O(n \log^2 n)$ time using Half-GCD.

1. Bézout's Identity By running Extended GCD on $M(x)$ and $A(x)$, we obtain the greatest common divisor $g(x) = \gcd(M(x), A(x))$ alongside the transition matrix T that satisfies Bézout's identity:

$$\begin{pmatrix} T_{11}(x) & T_{12}(x) \\ T_{21}(x) & T_{22}(x) \end{pmatrix} \begin{pmatrix} M(x) \\ A(x) \end{pmatrix} = \begin{pmatrix} g(x) \\ 0 \end{pmatrix}$$

Expanding the top row of the matrix multiplication yields:

$$T_{11}(x)M(x) + T_{12}(x)A(x) = g(x)$$

Taking this equation modulo $M(x)$, the first term vanishes, leaving:

$$T_{12}(x)A(x) \equiv g(x) \pmod{M(x)}$$

2. Normalization An inverse exists if and only if $\deg(g(x)) = 0$ so it's a unit in $\mathbb{K}[x]$, we then divide both sides by $g(x)$ and obtain the inverse.

$$(T_{12}(x) \cdot g_0^{-1}) A(x) \equiv 1 \pmod{M(x)}$$

Algorithm 36 Modular Inversion

```

1: function INVERSEMOD( $A(x), M(x)$ )
2:   if  $A(x) = 0$  then
3:     return None
                                     ▷ Compute Extended GCD and Transformation Matrix  $T$ 
4:    $(g(x), \dots, T) \leftarrow \text{FULLGCD}(M(x), A(x))$ 
                                     ▷ Check for coprimality
5:   if  $\deg(g) \neq 0$  then
6:     return None
                                     ▷ No inverse exists
                                     ▷  $T_{12}$  is the coefficient corresponding to  $A(x)$ 
7:    $U(x) \leftarrow T_{12}(x)$ 
8:    $g_0 \leftarrow g[0]$ 
9:   return  $U(x) \cdot g_0^{-1} \pmod{M(x)}$ 

```

13.2.6 Root Finding

The Berlekamp-Rabin algorithm isolates roots of $f(x)$ over $\mathbb{F}_p$ by first extracting distinct linear factors via $f_{\text{linear}}(x) = \gcd(f(x), x^p - x)$. It then divides the roots among two parts using random shifts.

1. Pairwise Separation For any two distinct roots α, β , a random shift $r \in \mathbb{F}_p$ separates them if the test polynomial $(x + r)^{(p-1)/2} \pmod{p}$ yields different values for them. This occurs when:

$$-1 = \left(\frac{\alpha + r}{p} \right) \left(\frac{\beta + r}{p} \right) = \left(\frac{1 + (\beta - \alpha)(\alpha + r)^{-1}}{p} \right)$$

Letting $x = \alpha + r$ and $c = \beta - \alpha \neq 0$, the inner term $1 + cx^{-1}$ maps to all $\frac{p-1}{2}$ non-residues in $\mathbb{F}_p$ as x varies. Thus, a random shift separates any specific pair of roots with probability at least $\frac{p-1}{2p}$. Consequently, the probability two roots remain together is bounded by $\rho \leq \frac{p+1}{2p} \leq \frac{2}{3}$ (for $p \geq 3$).

2. Recursion Depth Let $D_k(\alpha)$ be the degree of the polynomial containing α at recursion depth k . Let $D_k(\alpha)$ be the degree of the polynomial containing α at recursion depth k . At the root of the recursion tree, we have $D_0(\alpha) = n$.

Consider a single step from depth k to $k + 1$. Suppose that at depth k , the polynomial containing α has degree d .

We define an indicator random variable for each $\beta \in S$ where S is the set of other $d - 1$ roots in the polynomial:

$$I_\beta = \begin{cases} 1 & \text{if } \beta \text{ remains in the same factor as } \alpha \\ 0 & \text{if } \beta \text{ is separated from } \alpha \end{cases}$$

The degree of the polynomial containing α at the next depth is:

$$D_{k+1}(\alpha) = 1 + \sum_{\beta \in S} I_\beta$$

From the above analysis, we know that the probability any specific β fails to separate from α is bounded by $\rho \leq \frac{p+1}{2p} \leq \frac{2}{3}$, and $\mathbb{E}[I_\beta] \leq \rho$.

The expected degree at depth $k + 1$ is:

$$\mathbb{E}[D_{k+1}(\alpha) \mid D_k(\alpha) = d] = 1 + \sum_{\beta \in S} \mathbb{E}[I_\beta] \leq 1 + (d-1)\rho$$

Thus:

$$\mathbb{E}[D_{k+1}(\alpha) - 1 \mid D_k(\alpha) = d] \leq \rho(d-1)$$

$$\mathbb{E}[D_{k+1}(\alpha) - 1] \leq \rho \cdot \mathbb{E}[D_k(\alpha) - 1]$$

$$\mathbb{E}[D_k(\alpha) - 1] \leq \rho^k(n-1)$$

By Markov's Inequality, the probability α is not yet isolated is $\leq \rho^k n$. Union Bounding we have:

$$P(L > k) \leq n^2 \rho^k$$

For $k = C \log n$ with a sufficiently large constant C , this probability is $< 1/n$.

3. Complexity

1. **Powering:** Computing $x^p \pmod{f(x)}$ or $(x+r)^{\frac{p-1}{2}} \pmod{f(x)}$ requires $O(n \log n \log p)$ time.
2. **Recursion:** At any depth k , splitting a polynomial of degree d takes $O(d \log d \log p)$ time. Thus the work across all disjoint polynomials at depth k is bounded by $O(n \log n \log p)$.

Summing the $O(n \log n \log p)$ work across the $O(\log n)$ levels yields an overall bound of $O(n \log^2 n \log p)$.

Algorithm 37 Root Finding

```
1: function FINDROOTS( $f(x), p$ )
2:   if  $\deg(f) < 1$  then
3:     return  $\emptyset$ 
4:    $A(x) \leftarrow x^p \pmod{f(x)}$ 
5:    $A(x) \leftarrow A(x) - x$ 
6:    $f_{\text{linear}}(x) \leftarrow \gcd(f(x), A(x))$ 
7:    $R \leftarrow$  empty list
8:   FINDROOTSRECURSIVE( $f_{\text{linear}}(x), p, R$ )
9:   Sort  $R$  and
10:  return  $R$ 
11: function FINDROOTSRECURSIVE( $f(x), p, R$ )
12:   $d \leftarrow \deg(f)$ 
13:  if  $d = 0$  then
14:    return
15:  if  $d = 1$  then
16:     $r \leftarrow -f[0] \cdot f[1]^{-1} \pmod{p}$ 
17:    Add  $r$  to  $R$ 
18:    return
19:  while true do
20:    Pick a random  $r \in [0, p-1]$ 
21:     $q(x) \leftarrow x + r$ 
22:     $A(x) \leftarrow q(x)^{(p-1)/2} \pmod{f(x)}$ 
23:    if  $A(x)$  is a constant polynomial then
24:      continue
25:     $A(x) \leftarrow A(x) - 1$ 
26:     $g_1(x) \leftarrow \gcd(f(x), A(x))$ 
27:    if  $0 < \deg(g_1) < \deg(f)$  then
28:       $g_2(x) \leftarrow f(x)/g_1(x)$ 
29:      FINDROOTSRECURSIVE( $g_1(x), p, R$ )
30:      FINDROOTSRECURSIVE( $g_2(x), p, R$ )
31:    break
```

▷ If $A(x) \equiv 1$ or $A(x) \equiv 0$, we made no progress

13.2.7 Minimum Linear Recurrence

We can find the minimum linear recurrence of a sequence in $O(n \log^2 n)$ time.

Given a sequence $s_0, s_1, \dots, s_{n-1}$, we define its generating polynomial:

$$S(x) = \sum_{i=0}^{n-1} s_i x^i$$

The sequence satisfies a linear recurrence of order d if there exist coefficients $c_0, c_1, \dots, c_d$ (with $c_0 = 1$) such that for all $k \geq d$, $\sum_{j=0}^d c_j s_{k-j} = 0$.

If we define the characteristic polynomial of this recurrence as $C(x) = \sum_{j=0}^d c_j x^j$, the recurrence relation says that the higher-order terms of the product $S(x)C(x)$ are 0. Specifically, the product is some error polynomial $P(x)$ of degree $< d$. Thus:

$$S(x)C(x) \equiv P(x) \pmod{x^n}$$

where $\deg(C) = d$, $C(0) = 1$, and $\deg(P) < d$. Our goal is to find $C(x)$ while minimizing d .

Reversal To avoid dealing with the requirement $C(0) = 1$, we can reverse the sequence and ensure that it's monic.

Suppose $C(x)$ defines a linear recurrence of order d for $S(x)$. This means $\deg(C) = d$ and we can write:

$$S(x)C(x) = P(x) + x^n R(x)$$

where $\deg(P) < d$. Substituting x with x^{-1} and multiplying both sides by x^{n-1+d} yields:

$$x^{n-1}S(x^{-1}) \cdot x^d C(x^{-1}) = x^{n-1+d}P(x^{-1}) + x^{d-1}R(x^{-1})$$

We define the reversed polynomials as follows:

$$\begin{aligned}\tilde{S}(x) &= x^{n-1}S(x^{-1}) \\ \tilde{C}(x) &= x^d C(x^{-1}) \\ \tilde{P}(x) &= x^{d-1}P(x^{-1}) \\ \tilde{R}(x) &= x^{n-1+d}P(x^{-1})\end{aligned}$$

Because $\deg(S) < n$ and $\deg(C) = d$, the highest degree of $S(x)C(x)$ is strictly less than $n + d$, meaning $\deg(R) \leq d - 1$. Thus, $R(x^{-1})$ contains only non-positive powers of x down to $x^{-(d-1)}$, ensuring $\tilde{P}(x)$ is a polynomial with $\deg(\tilde{P}) \leq d - 1$.

Conversely, because $\deg(P) \leq d - 1$, the smallest power of x in $P(x^{-1})$ is $-(d - 1)$. The lowest power of x in $\tilde{R}(x)$ is therefore $n - 1 + d - (d - 1) = n$. This guarantees that $\tilde{R}(x) \equiv 0 \pmod{x^n}$.

This gives us the congruence:

$$\tilde{S}(x)\tilde{C}(x) = \tilde{P}(x) + \tilde{R}(x) \equiv \tilde{P}(x) \pmod{x^n}$$

Therefore, if $\deg(P) < \deg(C)$, it holds that $\deg(\tilde{P}) < \deg(\tilde{C})$. To find the minimum recurrence for $S(x)$, we compute the minimum recurrence for its reverse $\tilde{S}(x)$ and then reverse the resulting polynomial.

Degree Invariant We consider applying the Extended Euclidean Algorithm to $r_{-1} = x^n$ and $r_0 = \tilde{S}(x)$. This generates a sequence of quotients A_k , remainders r_k , and denominators Q_k defined by:

$$\begin{aligned}r_k &= r_{k-2} - A_k r_{k-1} \quad \text{where} \quad A_k = r_{k-2} \operatorname{div} r_{k-1} \\ Q_k &= A_k Q_{k-1} + Q_{k-2} \quad \text{with} \quad Q_{-1} = 0, Q_0 = 1\end{aligned}$$

Lemma 246 (Convergent Congruence). *For all $k \geq 0$, the Euclidean convergent denominator $Q_k(x)$ satisfies:*

$$Q_k(x)\tilde{S}(x) \equiv (-1)^k r_k(x) \pmod{x^n}$$

Proof. We proceed by induction on k . Recall the initial conditions: $r_{-1} = x^n$, $r_0 = \tilde{S}(x)$, $Q_{-1} = 0$, and $Q_0 = 1$.

Base Cases ($k = 0, 1$):

For $k = 0$, $Q_0\tilde{S}(x) = 1 \cdot \tilde{S}(x) = r_0(x) = (-1)^0 r_0(x)$, which holds exactly.

For $k = 1$, the first division step gives $r_1 = r_{-1} - A_1 r_0$. Taking this modulo x^n (since $r_{-1} = x^n \equiv 0$) yields $r_1 \equiv -A_1 \tilde{S}(x) \pmod{x^n}$. Since $Q_1 = A_1 Q_0 + Q_{-1} = A_1$, we have:

$$Q_1\tilde{S}(x) = A_1\tilde{S}(x) \equiv -r_1(x) = (-1)^1 r_1(x) \pmod{x^n}$$

Inductive Step:

Assume the congruence holds for $k - 2$ and $k - 1$. Multiply the recurrence $Q_k = A_k Q_{k-1} + Q_{k-2}$ by $\tilde{S}(x)$:

$$\begin{aligned}Q_k\tilde{S}(x) &= A_k(Q_{k-1}\tilde{S}(x)) + (Q_{k-2}\tilde{S}(x)) \\ &\equiv A_k((-1)^{k-1}r_{k-1}(x)) + ((-1)^{k-2}r_{k-2}(x)) \pmod{x^n}\end{aligned}$$

Factor out $(-1)^k$, noting that $(-1)^{k-1} = -(-1)^k$ and $(-1)^{k-2} = (-1)^k$:

$$Q_k \tilde{S}(x) \equiv (-1)^k \left(-A_k r_{k-1}(x) + r_{k-2}(x) \right) \pmod{x^n}$$

By the remainder recurrence $r_k = r_{k-2} - A_k r_{k-1}$, the term inside the parentheses is exactly $r_k(x)$. Thus:

$$Q_k \tilde{S}(x) \equiv (-1)^k r_k(x) \pmod{x^n}$$

□

Lemma 247 (Degree Invariant). *For all $k \geq 0$, the sum of the degree of the k -th convergent and the $(k-1)$ -th remainder is exactly n :*

$$\deg(Q_k) + \deg(r_{k-1}) = n$$

Proof. We proceed by induction on k . Base case ($k = 1$): Since $A_1 = r_{-1} \operatorname{div} r_0$, we have $\deg(A_1) = \deg(r_{-1}) - \deg(r_0) = n - \deg(r_0)$. By definition, $Q_1 = A_1 Q_0 + Q_{-1} = A_1(1) + 0 = A_1$. Thus, $\deg(Q_1) = n - \deg(r_0)$, so $\deg(Q_1) + \deg(r_0) = n$.

Inductive step: Assume $\deg(Q_{k-1}) + \deg(r_{k-2}) = n$. By the division algorithm, $A_k = r_{k-2} \operatorname{div} r_{k-1}$, so $\deg(A_k) = \deg(r_{k-2}) - \deg(r_{k-1})$. For the convergent, $\deg(Q_k) = \deg(A_k Q_{k-1} + Q_{k-2})$. Since $\deg(A_k) \geq 1$, the degree strictly grows, meaning $\deg(Q_k) = \deg(A_k) + \deg(Q_{k-1})$. Substituting $\deg(A_k)$:

$$\deg(Q_k) = (\deg(r_{k-2}) - \deg(r_{k-1})) + \deg(Q_{k-1})$$

Rearranging terms yields:

$$\deg(Q_k) + \deg(r_{k-1}) = \deg(Q_{k-1}) + \deg(r_{k-2}) = n$$

□

Because $\deg(r_k) < \deg(r_{k-1})$, this invariant implies that $\deg(Q_k) + \deg(r_k) < n$ for all k .

Minimality A linear recurrence requires: $\deg(r_k) < \deg(Q_k)$. Because $\deg(r_k)$ strictly decreases and $\deg(Q_k)$ strictly increases, there is a unique crossover point.

Lemma 248 (Minimal Recurrence Uniqueness). *Let k be the first step in the Euclidean sequence where $\deg(r_k) < \deg(Q_k)$. Then $Q_k(x)$ is the unique minimal-degree characteristic polynomial satisfying the linear recurrence.*

Proof. Because k is the first step where the degrees cross over, the preceding step must not have crossed over. Therefore, the $(k-1)$ -th remainder and convergent satisfy:

$$\deg(r_{k-1}) \geq \deg(Q_{k-1})$$

Assume for contradiction there exists another valid pair (Q^*, r^*) satisfying $Q^* \tilde{S} \equiv r^* \pmod{x^n}$ with $\deg(r^*) < \deg(Q^*)$ and strictly smaller degree: $\deg(Q^*) < \deg(Q_k)$.

We cross-multiply this valid pair with the $(k-1)$ -th Euclidean congruence $Q_{k-1} \tilde{S} \equiv \pm r_{k-1} \pmod{x^n}$:

$$\begin{aligned} Q^*(\pm r_{k-1}) &\equiv Q^*(Q_{k-1} \tilde{S}) \pmod{x^n} \\ Q_{k-1} r^* &\equiv Q_{k-1}(Q^* \tilde{S}) \pmod{x^n} \end{aligned}$$

Subtracting these yields $\pm Q^* r_{k-1} - Q_{k-1} r^* \equiv 0 \pmod{x^n}$.

Now, we bound the maximum degree of this difference. By the degree invariant, we know $\deg(r_{k-1}) = n - \deg(Q_k)$.

$$1. \deg(Q^* r_{k-1}) = \deg(Q^*) + \deg(r_{k-1}) = \deg(Q^*) + n - \deg(Q_k).$$

Because we assumed $\deg(Q^*) < \deg(Q_k)$, this evaluates to strictly less than n .

$$2. \deg(Q_{k-1}r^*) = \deg(Q_{k-1}) + \deg(r^*).$$

Since $\deg(Q_{k-1}) \leq \deg(r_{k-1}) = n - \deg(Q_k)$, and $\deg(r^*) < \deg(Q^*) \leq \deg(Q_k) - 1$, the sum is strictly bounded by $(n - \deg(Q_k)) + (\deg(Q_k) - 1) = n - 1 < n$.

Because both cross-terms have a degree strictly less than n , their difference must also have a degree strictly less than n . However, the difference is congruent to 0 (mod x^n), thus it's equal to 0.

Thus, we obtain:

$$Q^*r_{k-1} = \pm Q_{k-1}r^*$$

Taking the degree of both sides gives:

$$\deg(Q^*) + \deg(r_{k-1}) = \deg(Q_{k-1}) + \deg(r^*)$$

Rearranging the terms to group by step yields:

$$\deg(r_{k-1}) - \deg(Q_{k-1}) = \deg(r^*) - \deg(Q^*)$$

Since k is the first crossover step, we know the left-hand side is non-negative (≥ 0), thus so is the right-hand side, meaning $\deg(r^*) \geq \deg(Q^*)$, which is a contradiction. $\square$

We can find this step k without expanding the polynomials by using only the degrees of the quotients A_i . Substitute $\deg(Q_k) = n - \deg(r_{k-1})$ into the minimality condition $\deg(r_k) < \deg(Q_k)$:

$$\deg(r_k) < n - \deg(r_{k-1}) \implies \deg(r_{k-1}) + \deg(r_k) < n$$

We define a tracking variable $\delta_k = \deg(r_{k-1}) + \deg(r_k) - n$. The optimal crossover point occurs at the first k where $\delta_k < 0$.

To compute δ_k iteratively, we start with $\delta_0 = \deg(r_{-1}) + \deg(r_0) - n = \deg(r_0)$. At each step, we update the variable:

$$\begin{aligned} \delta_k - \delta_{k-1} &= (\deg(r_{k-1}) + \deg(r_k) - n) - (\deg(r_{k-2}) + \deg(r_{k-1}) - n) \\ &= \deg(r_k) - \deg(r_{k-2}) \end{aligned}$$

Since $r_{k-2} = A_k r_{k-1} + r_k$ and $r_{k-1} = A_{k+1} r_k + r_{k+1}$, we have $\deg(r_{k-2}) - \deg(r_{k-1}) = \deg(A_k)$ and $\deg(r_{k-1}) - \deg(r_k) = \deg(A_{k+1})$. Summing these gives $\deg(r_{k-2}) - \deg(r_k) = \deg(A_k) + \deg(A_{k+1})$. Therefore, the update rule is:

$$\delta_k = \delta_{k-1} - \deg(A_k) - \deg(A_{k+1})$$

Recovery To recover the characteristic polynomial, we need to compute the convergent matrix:

$$\mathcal{M}_{0,k} = \prod_{i=0}^{k-1} \begin{pmatrix} A_i & 1 \\ 1 & 0 \end{pmatrix}$$

The top-left entry of this product is the convergent denominator $Q_k(x)$. We use divide and conquer, splitting the sequence based on the sum of degrees in the left and right halves.

Algorithm 38 Minimum Linear Recurrence

```
1: function EVALUATECONVERGENT( $l, r, A_{\text{quots}}$ )
2:   if  $r - l \leq 1$  then
3:     return  $\begin{pmatrix} A_{\text{quots}}[l] & 1 \\ 1 & 0 \end{pmatrix}$ 
4:    $S \leftarrow \sum_{i=l}^{r-1} \deg(A_{\text{quots}}[i])$ 
5:    $k \leftarrow l + 1$ 
6:    $C \leftarrow \deg(A_{\text{quots}}[l])$ 
7:   while  $k < r - 1$  and  $2 \cdot C \leq S$  do
8:      $C \leftarrow C + \deg(A_{\text{quots}}[k])$ 
9:      $k \leftarrow k + 1$ 
10:   $M_{\text{left}} \leftarrow \text{EVALUATECONVERGENT}(l, k, A_{\text{quots}})$ 
11:   $M_{\text{right}} \leftarrow \text{EVALUATECONVERGENT}(k, r, A_{\text{quots}})$ 
12:  return  $M_{\text{left}} \cdot M_{\text{right}}$ 
13: function MINRECURRENCE( $S(x), n$ )
14:    $r_{-1}(x) \leftarrow x^n$ 
15:    $r_0(x) \leftarrow \text{Reverse}(S(x) \pmod{x^n})$ 
16:   if  $r_0(x) = 0$  then
17:     return 1
18:    $(g, \dots, A_{\text{quots}}, \dots) \leftarrow \text{FULLGCD}(r_{-1}, r_0)$ 
19:   Append 0 to  $A_{\text{quots}}$ 
20:    $\text{pref} \leftarrow 0$ 
21:    $\delta \leftarrow n - \deg(A_{\text{quots}}[0])$ 
22:   while  $\delta \geq 0$  do
23:      $\delta \leftarrow \delta - (\deg(A_{\text{quots}}[\text{pref}]) + \deg(A_{\text{quots}}[\text{pref} + 1]))$ 
24:      $\text{pref} \leftarrow \text{pref} + 1$ 
25:    $M \leftarrow \text{EVALUATECONVERGENT}(0, \text{pref}, A_{\text{quots}})$ 
26:    $Q(x) \leftarrow M_{0,0}$ 
27:    $Q_{\text{rev}}(x) \leftarrow \text{Reverse}(Q(x))$ 
28:    $c \leftarrow Q_{\text{rev}}[0]$ 
29:   return  $Q_{\text{rev}}(x) \cdot c^{-1}$ 
```

13.3 Rational Functions and Transposition

13.3.1 Bostan-Mori

The Bostan-Mori algorithm computes the N -th coefficient of a rational function $\frac{U(x)}{V(x)}$ in $O(d \log d \log N)$ time, where d is the degree of the denominator.

1. The Bostan-Mori Algorithm To compute $[x^N] \frac{U(x)}{V(x)}$, we use the fact that $V(x)V(-x)$ is an even polynomial, meaning it can be written as $E(x^2)$. We multiply the numerator and denominator by $V(-x)$:

$$\frac{U(x)}{V(x)} = \frac{U(x)V(-x)}{V(x)V(-x)} = \frac{U(x)V(-x)}{E(x^2)}$$

Since the denominator now only contains even powers of x , the parity of the non-zero terms in the expansion is determined by the numerator. If N is even, we only care about the even terms of the numerator. If N is odd, we only care about the odd terms. We can therefore discard half of the terms:

$$[x^N] \frac{U(x)}{V(x)} = \begin{cases} [x^{N/2}] \frac{(U(x)V(-x))_{\text{even}}}{E(x)} & \text{if } N \text{ is even} \\ [x^{(N-1)/2}] \frac{(U(x)V(-x))_{\text{odd}}}{E(x)} & \text{if } N \text{ is odd} \end{cases}$$

By iterating this process, N is halved at each step until $N = 0$, requiring $O(\log N)$ polynomial multiplications of size $O(d)$ as we double and then halve the degree.

Computing $U(x)U(-x)$ We can compute $V(x)$ such that $V(x^2) = U(x)U(-x)$ faster assuming we have access to the NTT of $U(x)$.

In the frequency domain, the array represents the evaluations of $U(x)$ at the $2N$ -th roots of unity, $\omega_0, \omega_1, \dots, \omega_{2N-1}$. Because $\omega_{k+N} = -\omega_k$, evaluating the polynomial at these antipodal points yields $U(\omega_k)$ and $U(-\omega_k)$.

Let $A = U(\omega_k)$ and $B = U(-\omega_k)$. The product of these two evaluations is:

$$A \times B = U(\omega_k)U(-\omega_k) = V(\omega_k^2)$$

ω_k^2 is an N -th root of unity. This means computing the products yields the evaluation of $V(y)$ at the N -th roots of unity.

Bit Reversal:

In bit-reversed order the antipodal roots of unity are next to each other: specifically, the values $U(\omega_k)$ and $U(-\omega_k)$ are stored as pairs $[a, b]$ in the array.

Therefore, we can iterate through the array in chunks of 2 and multiply the adjacent elements.

Algorithm 39 Bostan-Mori

Input: Polynomials $U(x), V(x)$ of degree d , integer N

Output: $[x^N] \frac{U(x)}{V(x)}$

```

1: function BOSTANMORI( $U, V, N$ )
2:   while  $N > 0$  do
3:      $U_{next}(x) \leftarrow U(x)V(-x)$ 
4:      $V_{next}(x) \leftarrow V(x)V(-x)$ 
5:     if  $N \pmod{2} = 0$  then
6:        $U(x) \leftarrow (U_{next})_{even}$   $\triangleright$  Take even terms:  $x^{2k} \rightarrow x^k$ 
7:     else
8:        $U(x) \leftarrow (U_{next})_{odd}$   $\triangleright$  Take odd terms:  $x^{2k+1} \rightarrow x^k$ 
9:        $V(x) \leftarrow (V_{next})_{even}$ 
10:     $N \leftarrow \lfloor N/2 \rfloor$ 
11: return  $U[0]/V[0]$ 

```

13.3.2 Transposition Principle

The transposition principle (as used in algorithms on formal power series) states that if a linear map $f : \mathbb{K}^n \rightarrow \mathbb{K}^m$ can be computed efficiently, then so can its transposed linear map $f^T : \mathbb{K}^m \rightarrow \mathbb{K}^n$.

Let V and W be vector spaces over $\mathbb{K}$, and let $\mathcal{L} : V \rightarrow W$ be a linear operator, we consider a bilinear form $\langle \cdot, \cdot \rangle : V \times V \rightarrow \mathbb{K}$. The transposed operator $\mathcal{L}^T : W \rightarrow V$ is defined as the operator that satisfies the identity:

$$\langle \mathcal{L}(u), v \rangle_W = \langle u, \mathcal{L}^T(v) \rangle_V$$

By taking an algorithm and breaking it down into a composition of linear operators $\mathcal{L} = \mathcal{L}_k \circ \dots \circ \mathcal{L}_1$, we can find the adjoint of the algorithm by composing the adjoints of the steps in reverse order: $\mathcal{L}^T = \mathcal{L}_1^T \circ \dots \circ \mathcal{L}_k^T$.

We primarily use two pairings:

The Dot-Product: We can define our pairing over polynomials of degree $< k$ as the dot-product.

$$\langle A, B \rangle_k = \sum_{i=0}^{k-1} a_i b_i$$

The Middle Product: We can also use the middle product pairing. For $U, A \in \mathbb{K}[x]_{<d}$:

$$\langle U, A \rangle = [x^{d-1}](U(x)A(x)) = \sum_{i=0}^{d-1} u_i a_{d-1-i}$$

13.3.3 Transposed Multiplication

Dot Product Pairing

Lemma 249. Let $\mathcal{M}_P(Q) = (P(x)Q(x)) \bmod x^k$ denote multiplication by a fixed polynomial $P(x)$. Its transposed operator $\mathcal{M}_P^T$, known as the Middle Product or Transposed Multiplication (denoted $\circ$), is given by:

$$\mathcal{M}_P^T(A) = A \circ P = \sum_{j=0}^{k-1} x^j \sum_{\ell=0}^{k-1-j} a_{j+\ell} p_\ell$$

Proof. We seek the adjoint such that $\langle \mathcal{M}_P(Q), A \rangle_k = \langle Q, \mathcal{M}_P^T(A) \rangle_k$. Expanding the left side:

$$\langle P \cdot Q \bmod x^k, A \rangle_k = \sum_{i=0}^{k-1} a_i \sum_{j=0}^i q_j p_{i-j}$$

We swap the order of summation to isolate the coefficients of Q :

$$\sum_{j=0}^{k-1} q_j \sum_{i=j}^{k-1} a_i p_{i-j}$$

Let $\ell = i - j$. The inner sum becomes $\sum_{\ell=0}^{k-1-j} a_{j+\ell} p_\ell$. This represents the j -th coefficient of the required transposed state, matching the definition of the middle product $A \circ P$. $\square$

Middle Product Pairing

Lemma 250. Let the pairing over $\mathbb{K}[x]_{<k}$ be defined by: $\langle A, B \rangle_k = [x^{k-1}](A(x)B(x))$. The adjoint of polynomial multiplication by a fixed polynomial $P(x)$ corresponds to multiplication by $P(x)$ followed by a degree right-shift.

Specifically, if the forward operator maps an input of degree $< n$ to an output space of degree $< N$, the transposed operator applied to a state $Z(x)$ is given by:

$$\mathcal{M}_P^T(Z(x)) = \left\lfloor \frac{P(x)Z(x)}{x^{N-n}} \right\rfloor \pmod{x^n}$$

Proof. Let $\mathcal{M}_P : \mathbb{K}[x]_{<n} \rightarrow \mathbb{K}[x]_{<N}$ be the forward operator defined by $U(x) \mapsto U(x)P(x)$. We seek the transposed state $A(x) = \mathcal{M}_P^T(Z(x)) \in \mathbb{K}[x]_{<n}$ given an incoming transposed state $Z(x) \in \mathbb{K}[x]_{<N}$. By the definition of the adjoint, the following must hold for any polynomial $U(x) \in \mathbb{K}[x]_{<n}$:

$$\langle \mathcal{M}_P(U), Z \rangle_N = \langle U, A \rangle_n$$

Expanding both sides using the definition of our pairing yields:

$$[x^{N-1}](U(x)P(x)Z(x)) = [x^{n-1}](U(x)A(x))$$

We factor x^{N-n} out of the left-hand expression:

$$[x^{n-1}] \left(U(x) \frac{P(x)Z(x)}{x^{N-n}} \right) = [x^{n-1}](U(x)A(x))$$

Since $U(x)$ is arbitrary, the coefficients of the factor must be identical on both sides for degrees 0 through $n - 1$.

Any fractional power or power $\geq n$ will result in a negative or overly large degree when multiplied by $U(x)$, so they can be discarded using floor and modulo.

Therefore, the transposed state $A(x)$ is the range of valid degrees:

$$A(x) = \left\lfloor \frac{P(x)Z(x)}{x^{N-n}} \right\rfloor \pmod{x^n}$$

□

13.3.4 Transposed NTT and INTT

If an implementation does not do bit reversal then the transforms are self-adjoint.

Let n be a power of 2, and $\text{rev}(k)$ as the bit-reversal of k up to the $\log_2(n)$ -th digit. Consider bit-reversed FFT and bit-reversed IFFT:

- **Bit-reversed FFT:** Input a and output b defined by:

$$b_i = \sum_j \omega^{-\text{rev}(i) \cdot j} a_j$$

- **Bit-reversed IFFT:** Input b and output a defined by:

$$a_i = \frac{1}{n} \sum_j \omega^{i \cdot \text{rev}(j)} b_j$$

Their transposes are:

- **Transposed Bit-reversed FFT:** Input b and output a defined by:

$$a_i = \sum_j \omega^{-i \cdot \text{rev}(j)} b_j$$

- **Transposed Bit-reversed IFFT:** Input a and output b defined by:

$$b_i = \frac{1}{n} \sum_j \omega^{\text{rev}(i) \cdot j} a_j$$

“Bit reversed FFT” and “transposed bit reversed IFFT”, and “bit reversed IFFT” and “transposed bit reversed FFT”, are essentially the same.

The operation of reversing the second to last elements sends the k -th element to the $(n - k)$ -th position, and $\omega^{n-k} = \omega^{-k}$, so the exponent is negated. Therefore:

- The **transposed bit-reversed FFT** is: bit-reversed IFFT, then reversing all but the first element, then scaling it by n .
- The **transposed bit-reversed IFFT** is: reversing all but the first element, then bit-reversed FFT, then scaling it by $1/n$.

Transposed Multiplication Using these exact transposed operators, we can compute transposed multiplication:

$$\text{Pad}_n(Z) \xrightarrow{\text{intt.t}} \cdot \odot \text{NTT}(V) \xrightarrow{\text{ntt.t}} \text{Truncate}_{m-k+1}$$

The difference between doing this and computing the middle product by reversing is that we compute less coefficients of the output and thus it's faster if less are needed.

13.3.5 Transposed Bostan-Mori

The Bostan-Mori algorithm evaluates the linear functional $U(x) \mapsto [x^N] \frac{U(x)}{V(x)}$ for $U \in \mathbb{K}[x]_{<d}$. The transposed algorithm computes a polynomial $A(x) \in \mathbb{K}[x]_{<d}$ under a chosen inner product.

We use the middle product pairing. The function **TransposedProj** computes the polynomial $A(x)$ such that for any $U(x)$ of degree $< d$:

$$[x^{d-1}]U(x)A(x) = [x^N] \frac{U(x)}{V(x)}$$

1. Recovering the Remainder To justify the correctness of the algorithm, we prove that computing this transposed state $A(x)$ is equivalent to computing $x^N \pmod{P(x)}$.

Theorem 251. *Let $P(x) = V^R(x)$. If $R(x) = x^N \pmod{P(x)}$, then $A(x) \equiv \frac{R^R(x)}{V(x)} \pmod{x^d}$.*

Proof. By polynomial division, $x^N = Q(x)P(x) + R(x)$. Substituting $x \leftarrow 1/x$ yields:

$$x^{-N} = Q(1/x)P(1/x) + R(1/x)$$

Notice that $V(x) = x^d P(1/x) \implies P(1/x)/V(x) = x^{-d}$. We divide our equation by $V(x)$ and multiply by $U(x)$:

$$x^{-N} \frac{U(x)}{V(x)} = x^{-d} Q(1/x) U(x) + \frac{R(1/x) U(x)}{V(x)}$$

We extract the $[x^0]$ coefficient from both sides. On the left, $[x^0] x^{-N} \frac{U(x)}{V(x)} = [x^N] \frac{U(x)}{V(x)}$.

On the right side, the first term vanishes: $Q(1/x)$ has maximum degree 0, and $U(x)$ has maximum degree $d-1$, so $x^{-d} Q(1/x) U(x)$ has strictly negative powers of x , yielding an $[x^0]$ coefficient of 0. We are left with:

$$[x^N] \frac{U(x)}{V(x)} = [x^0] \frac{R(1/x) U(x)}{V(x)}$$

Since $\deg(R) < d$, its reversal is $R^R(x) = x^{d-1} R(1/x)$. Substituting $R(1/x) = x^{-(d-1)} R^R(x)$ gives:

$$[x^0] x^{-(d-1)} \frac{R^R(x) U(x)}{V(x)} = [x^{d-1}] U(x) \frac{R^R(x)}{V(x)}$$

By equating this with our pairing definition $[x^{d-1}]U(x)A(x)$, it follows immediately that:

$$A(x) \equiv \frac{R^R(x)}{V(x)} \pmod{x^d} \implies R^R(x) \equiv A(x)V(x) \pmod{x^d}$$

□

2. Transposing the Recurrence We now derive the recurrence for the state polynomial $A(x)$ by transposing the two linear operators of the forward step: polynomial multiplication by $V(-x)$, and the extraction of the Even/Odd terms.

Lemma 252. *Let $\mathcal{E}$ and $\mathcal{O}$ denote the operators that extract the even and odd terms of a polynomial, respectively. Their adjoints under the middle product pairing correspond to upsampling (interleaving zeroes), with a parity shift that swaps them: $\mathcal{E}^T(A) = xA(x^2)$ and $\mathcal{O}^T(A) = A(x^2)$.*

Proof. Suppose N is even, meaning the forward step applies $U_{next} = \mathcal{E}(W)$. We seek a polynomial $Z(x)$ such that $\langle \mathcal{E}(W), A \rangle_{d-1} = \langle W, Z \rangle_{2d-1}$.

Expanding the left side yields:

$$\langle \mathcal{E}(W), A \rangle_{d-1} = [x^{d-1}] \mathcal{E}(W(x)) A(x) = \sum_{i=0}^{d-1} w_{2i} a_{d-1-i}$$

Consider $Z(x) = xA(x^2)$. Because its non-zero terms are strictly at odd powers, its middle product with $W(x)$ at degree $2d - 1$ filters out the odd coefficients of W :

$$[x^{2d-1}]W(x)Z(x) = \sum_k w_k [x^{2d-1-k}]xA(x^2)$$

For the Z coefficient to be non-zero, the power $2d - 1 - k$ must be odd, which strictly forces k to be even. Let $k = 2i$. The corresponding coefficient is $[x^{2d-2-2i}]A(x^2) = a_{d-1-i}$. Thus, the summation matches the left side, proving that $\mathcal{E}^T(A) = xA(x^2)$.

By identical logic, if N is odd, the forward step uses $U_{next} = \mathcal{O}(W)$. The required adjoint $Z(x)$ does not need the $+1$ degree shift to align the terms, yielding $\mathcal{O}^T(A) = A(x^2)$. $\square$

3. Base Case ($N = 0$) When $N = 0$, the forward algorithm evaluates to $\frac{U(0)}{V(0)}$. We need $[x^{d-1}]U(x)A(x) = \frac{U(0)}{V(0)}$. This holds if and only if $A(x) = \frac{1}{V(0)}x^{d-1}$.

Algorithm 40 Transposed Bostan-Mori

```

1: function TRANSPOSEDPROJ( $N, Q(x), d$ )
2:   if  $N = 0$  then
3:      $R(x) \leftarrow \frac{1}{Q[0]}x^{d-1}$ 
4:   return  $R(x)$ 
5:    $Q_{next}(x) \leftarrow (Q(x)Q(-x))_{\text{even}}$ 
6:    $p(x) \leftarrow \text{TRANSPOSEDPROJ}(\lfloor N/2 \rfloor, Q_{next}(x), d)$ 
7:   if  $N \pmod{2} = 0$  then
8:      $p_{ext}(x) \leftarrow x \cdot p(x^2)$ 
9:   else
10:     $p_{ext}(x) \leftarrow p(x^2)$ 
11:     $p_{new}(x) \leftarrow p_{ext}(x) \cdot Q(-x)$ 
12:     $p_{ret}(x) \leftarrow (p_{new}(x) \gg d) \pmod{x^d}$   $\triangleright$  Keep terms  $x^d \dots x^{2d-1}$ 
13:  return  $p_{ret}(x)$ 
14: function XIMOD( $P(x), N$ )
15:    $d \leftarrow \deg(P)$ 
16:   if  $N < d$  then
17:     return  $x^N$ 
18:    $Q(x) \leftarrow \text{Reverse}(P(x))$ 
19:    $A(x) \leftarrow \text{TRANSPOSEDPROJ}(N, Q, d)$ 
20:    $R^R(x) \leftarrow (A(x) \cdot Q(x)) \pmod{x^d}$ 
21:  return  $\text{Reverse}(R^R(x))$ 

```

13.4 Composition and Inversion

13.4.1 Power Projection

The power projection problem asks us to compute the sequence of inner products $V_k = \langle W, P^k \rangle$ for $k = 0 \dots n - 1$. Under the middle product pairing, this is equivalent to extracting the $(n - 1)$ -th coefficient:

$$V_k = [x^{n-1}](W^R(x)P(x)^k)$$

We can construct the generating function for the sequence V :

$$V(y) = \sum_{k=0}^{n-1} V_k y^k = \sum_{k=0}^{n-1} [x^{n-1}]W^R(x)P(x)^k y^k = [x^{n-1}]\frac{W^R(x)}{1 - yP(x)}$$

This formulation reduces the problem to extracting the x^{n-1} coefficient of a bivariate rational function $\frac{A(x,y)}{B(x,y)}$, where initially $A(x,y) = W^R(x)$ and $B(x,y) = 1 - yP(x)$. We can compute this in $O(n \log^2 n)$ time by executing the Bostan-Mori algorithm on the variable x , treating the coefficients as polynomials in y .

1. Handling $P(0) \neq 0$ If $P(0) = c \neq 0$, then $[x^{n-1}] \sum_{k=0}^{\infty} y^k P(x)^k$ is not necessarily equal to $[x^{n-1}] \sum_{k=0}^{n-1} y^k P(x)^k$. To circumvent this, we project onto $\tilde{P}(x) = P(x) - c$, which safely has $\tilde{P}(0) = 0$. Let $\tilde{V}_k$ be this shifted projection sequence. We recover V_k using the binomial expansion:

$$V_k = \langle W, (c + \tilde{P})^k \rangle = \sum_{j=0}^k \binom{k}{j} c^{k-j} \tilde{V}_j$$

Dividing by $k!$ yields $\frac{V_k}{k!} = \sum_{j=0}^k \frac{c^{k-j}}{(k-j)!} \frac{\tilde{V}_j}{j!}$, which is a convolution of $\exp(cx)$ and the Borel transform of $\tilde{V}$. This shift takes $O(n \log n)$ time.

2. Bostan-Mori on Bivariate Polynomials Assume $P(0) = 0$. We pad our x -degree bound to the nearest power of two, $m = 2^L \geq n$. Because m is a power of 2, the index we are extracting ($m-1$) has a binary representation consisting entirely of 1s ($\underbrace{11 \dots 1}_L$). In the Bostan-Mori algorithm, we thus extract the

odd part of the numerator at every step.

Let $A(x,y) = \sum_{i=0}^{m-1} p_i(y)x^i$ and $B(x,y) = \sum_{i=0}^{m-1} q_i(y)x^i$. Initially, $p_i(y) = w_{m-1-i}$ and $q_i(y) = -P_i y$ (with $q_0(y) = 1$). At each step, we compute:

$$\begin{aligned} A_{new}(x,y) &= \mathcal{O}_x(A(x,y)B(-x,y)) \\ B_{new}(x,y) &= \mathcal{E}_x(B(x,y)B(-x,y)) \end{aligned}$$

Because the coefficients are polynomials in y , we can compute the NTT on each row and column to compute the convolution, and we can avoid duplicating the polynomials and apply various constant factor optimizations.

Complexity At each step, we perform $O(m)$ NTTs of size k over y , which takes $O(m \cdot k \log k)$ time. We then perform $O(k)$ NTTs of size m over x , taking $O(k \cdot m \log m)$ time. Because the invariant $m \cdot k = O(n)$ is preserved at every step, the time spent per step simplifies to:

$$O(n \log k + n \log m) = O(n \log(k \cdot m)) = O(n \log n)$$

Since m halves at each step, there are $\log_2(n)$ steps. Thus, the overall complexity is $O(n \log^2 n)$.

Algorithm 41 Power Projection

```
1: function POWERPROJ( $W(x), P(x), n$ )
2:   if  $n = 0$  then
3:     return 0
4:    $c \leftarrow P[0]$ 
5:   if  $c \neq 0$  then
6:      $\tilde{P}(x) \leftarrow P(x) - c$ 
7:      $\tilde{V}(y) \leftarrow \text{POWERPROJ}(W(x), \tilde{P}(x), n)$ 
8:     for  $j \leftarrow 0$  to  $n - 1$  do
9:        $\tilde{V}_{\text{borel}}[j] \leftarrow \tilde{V}[j] \cdot (j!)^{-1} \pmod{M}$ 
10:     $E(y) \leftarrow \sum_{i=0}^{n-1} c^i \cdot (i!)^{-1} y^i$ 
11:     $V_{\text{borel}}(y) \leftarrow (\tilde{V}_{\text{borel}}(y) \cdot E(y)) \pmod{y^n}$ 
12:    for  $k \leftarrow 0$  to  $n - 1$  do
13:       $V[k] \leftarrow V_{\text{borel}}[k] \cdot (k!) \pmod{M}$ 
14:    return  $V(y)$ 
15:   $m \leftarrow \text{next\_power\_of\_2}(n)$ 
16:   $k \leftarrow 1$ 
17:   $A(x, y) \leftarrow \text{Pad\_to\_Degree}(\text{Reverse}(W(x)), m - 1)$ 
18:   $B(x, y) \leftarrow 1 - yP(x)$ 
19:  while  $m > 1$  do
20:     $B_{\text{neg}}(x, y) \leftarrow B(-x, y)$ 
21:     $A_{\text{next}}(x, y) \leftarrow \mathcal{O}_x(A(x, y) \cdot B_{\text{neg}}(x, y))$ 
22:     $B_{\text{next}}(x, y) \leftarrow \mathcal{E}_x(B(x, y) \cdot B_{\text{neg}}(x, y))$ 
23:     $A(x, y) \leftarrow A_{\text{next}}(x, y)$ 
24:     $B(x, y) \leftarrow B_{\text{next}}(x, y)$ 
25:     $m \leftarrow m/2$ 
26:     $k \leftarrow 2k$ 
27:  return  $A(0, y) \pmod{y^n}$ 
```

13.4.2 Composition

The problem of computing $C(x) = F(P(x)) \pmod{x^n}$ can be solved in $O(n \log^2 n)$ by transposing the algorithm for Power Projection.

Let $\mathbf{M}_P$ be an $m \times m$ transformation matrix where the k -th column represents the coefficients of $P(x)^k \pmod{x^m}$.

- **Composition:** Evaluating $F(P(x))$ equates to taking a linear combination of the powers of $P(x)$ scaled by the coefficients of F . This is $\mathbf{c} = \mathbf{M}_P \mathbf{f}$.
- **Power Projection:** Transposing this map asks us to evaluate $\mathbf{v} = \mathbf{M}_P^T \mathbf{w}$. This corresponds to taking the dot product of a vector $\mathbf{w}$ with every column of $\mathbf{M}_P$, yielding the sequence $V_k = [x^{m-1}]W^R(x)P(x)^k$.

1. Handling $P(0) \neq 0$ If $P(0) = c \neq 0$, we must shift the polynomial to $\tilde{P}(x) = P(x) - c$ so that $\tilde{P}(0) = 0$.

We simultaneously shift the polynomial $F(x)$ using a Taylor shift to compute $F_{\text{shift}}(x) = F(x + c)$. The original composition is recovered by evaluating $F_{\text{shift}}(\tilde{P}(x))$.

2. Transposed Odd Extraction Because we pad the polynomials to a power of two n , our target coefficient is exactly $[x^{n-1}]$. Since $n - 1$ in binary is composed entirely of 1s, the forward Power Projection must extract the odd terms at every recursive layer.

Let $\mathcal{O}_x$ denote the extraction of odd terms: $\mathcal{O}_x(U(x)) = \frac{U(x) - U(-x)}{2x}$. In the forward pass, a state $A(x)$ is multiplied by the denominator $Q_s(-x)$ and projected via:

$$A_{next}(x^2) = \mathcal{O}_x(A(x)Q_s(-x)) = \frac{A(x)Q_s(-x) - A(-x)Q_s(x)}{2x}$$

We execute the adjoint directly in the frequency domain. Evaluating this odd extraction at a root of unity ω combines the two frequencies ω and $-\omega$:

$$A_{next}(\omega^2) = \frac{1}{2\omega} \left(A(\omega)Q_s(-\omega) - A(-\omega)Q_s(\omega) \right)$$

This is a linear map from the tuple $(A(\omega), A(-\omega))$ to $A_{next}(\omega^2)$. The adjoint operator is:

$$\begin{aligned} A^*(\omega) &= P(\omega^2) \cdot \frac{1}{2\omega} Q_s(-\omega) \\ A^*(-\omega) &= P(\omega^2) \cdot \frac{-1}{2\omega} Q_s(\omega) \end{aligned}$$

In a bit-reversed NTT array, the frequencies ω and $-\omega$ sit at adjacent indices $2i$ and $2i + 1$. Therefore, we get $P(\omega^2)$ from index i , compute $c = P[i] \cdot \frac{1}{2}\omega^{-1}$, and write:

$$\begin{aligned} A^*[2i] &= c \cdot Q_s[2i + 1] \\ A^*[2i + 1] &= -c \cdot Q_s[2i] \end{aligned}$$

Algorithm 42 Composition

```
1: function COMPOSE( $F(x), P(x), m$ )
2:   if  $m = 0$  then
3:     return 0
4:    $c \leftarrow P[0]$ 
5:   if  $c \neq 0$  then
6:      $\tilde{P}(x) \leftarrow P(x) - c$ 
7:      $F_{\text{shift}}(x) \leftarrow \text{TAYLORSHIFT}(F(x), c)$ 
8:     return COMPOSE( $F_{\text{shift}}(x), \tilde{P}(x), m$ )
9:    $n \leftarrow \text{next\_power\_of\_2}(m)$ 
10:  Pad  $F(x)$  and  $P(x)$  with zeroes to degree  $n - 1$ 
11:   $V \leftarrow \text{BitReversedInverseRoots}(2n)$ 
12:   $Q(x) \leftarrow 1 - yP(x)$ 
13:   $A(x) \leftarrow \text{TRANPOSEDPOWERPROJ}(n, 1, Q(x), F(x), V)$ 
14:  return Truncate $_{m-1}(\text{Reverse}(A(x)))$ 
15: function TRANPOSEDPOWERPROJ( $n, k, Q(x), F(x), V$ )
16:   if  $n = 1$  then
17:      $A \leftarrow$  array of size  $2k$  initialized to 0
18:      $F_{\text{rev}} \leftarrow \text{Reverse}(F)$ 
19:     for  $i \leftarrow 0$  to  $k - 1$  do
20:        $A[2i] \leftarrow F_{\text{rev}}[i]$ 
21:     return  $A$ 
22:    $Q_{\text{freq}} \leftarrow \text{NTT}_x(Q)$ 
23:    $Q_{\text{next}} \leftarrow \text{INTT}_x(\mathcal{E}_x(Q_{\text{freq}} \cdot Q_{\text{freq}}(\text{negated})))$ 
24:    $P \leftarrow \text{TRANPOSEDPOWERPROJ}(n/2, 2k, Q_{\text{next}}, F, V)$ 
25:    $P_{\text{freq}} \leftarrow \text{INTT}_x^T(P)$ 
26:    $A_{\text{freq}} \leftarrow$  array of size  $2nk$ 
27:   for  $j \leftarrow 0$  to  $k - 1$  do
28:     for  $i \leftarrow 0$  to  $\frac{n}{2} - 1$  do
29:        $idx \leftarrow j \cdot n + i$ 
30:        $c \leftarrow P_{\text{freq}}[idx] \cdot \frac{1}{2} \cdot V[i]$ 
31:        $A_{\text{freq}}[2 \cdot idx] \leftarrow c \cdot Q_{\text{freq}}[2 \cdot idx + 1]$ 
32:        $A_{\text{freq}}[2 \cdot idx + 1] \leftarrow -c \cdot Q_{\text{freq}}[2 \cdot idx]$ 
33:    $A_{\text{final}} \leftarrow \text{NTT}_x^T(A_{\text{freq}})$ 
34:   return  $A_{\text{final}}$ 
```

$\triangleright V[i]$ represents the ω^{-1} factor

13.4.3 Lagrange Inversion

Lemma 253 (Lagrange Inversion). *For formal power series f , Φ , and H , assume $f(x) = x\Phi(f(x))$. Then:*

$$[x^n]H(f(x)) = \frac{1}{n}[x^{n-1}]H'(x)\Phi(x)^n$$

Proof. By the linearity of the coefficient extraction operator, we can assume without loss of generality that $H(x) = x^k$. We then need to show:

$$[x^n]f(x)^k = \frac{1}{n}[x^{n-1}]kx^{k-1}\Phi(x)^n = \frac{k}{n}[x^{n-k}]\Phi(x)^n$$

We proceed by induction on n .

Base Cases: The cases $n = 0$ and $k = 0$ are trivial. If $k > n$, the claim is trivially true: since $f(0) = 0$, the lowest non-zero term of $f(x)^k$ has degree k , making $[x^n]f(x)^k = 0$. Since $n - k < 0$, the right-hand side is also exactly 0. If $k = n$, then $n[x^n]A^k(x) = n[x^0]\Phi^n(A(x)) = n(c_0)^n = k[z^{n-k}]\Phi^n(x)$.

Inductive Step: Assume the theorem holds for indices strictly less than n . Using the relation $f(x) = x\Phi(f(x))$, we pull out the x^k :

$$\begin{aligned}[x^n]f(x)^k &= [x^n]x^k\Phi(f(x))^k \\ &= [x^{n-k}]\Phi(f(x))^k\end{aligned}$$

We now apply the induction hypothesis to evaluate the $(n-k)$ -th coefficient of the composite function $\Phi(f(x))^k$. Treating $\Phi(x)^k$ as our new $H(x)$, we have:

$$\begin{aligned}[x^{n-k}]\Phi(f(x))^k &= \frac{1}{n-k}[x^{n-k-1}](\Phi(x)^k)' \Phi(x)^{n-k} \\ &= \frac{1}{n-k}[x^{n-k-1}](k\Phi'(x)\Phi(x)^{k-1})\Phi(x)^{n-k} \\ &= \frac{k}{n-k}[x^{n-k-1}]\Phi'(x)\Phi(x)^{n-1}\end{aligned}$$

We rewrite $\Phi'(x)\Phi(x)^{n-1}$ using the chain rule in reverse:

$$\begin{aligned}\frac{k}{n-k}[x^{n-k-1}]\Phi'(x)\Phi(x)^{n-1} &= \frac{k}{n-k}[x^{n-k-1}]\frac{1}{n}(\Phi(x)^n)' \\ &= \frac{1}{n}\frac{k}{n-k}[x^{n-k-1}](\Phi(x)^n)'\end{aligned}$$

Finally, using the identity $[x^{m-1}]P'(x) = m[x^m]P(x)$ with $m = n-k$, we evaluate:

$$\begin{aligned}\frac{1}{n}\frac{k}{n-k}[x^{n-k-1}](\Phi(x)^n)' &= \frac{1}{n}\frac{k}{n-k}(n-k)[x^{n-k}]\Phi(x)^n \\ &= \frac{k}{n}[x^{n-k}]\Phi(x)^n\end{aligned}$$

□

Lemma 254 (Lagrange Inversion for Compositional Inverse). *Let $F(x) = \sum_{r \geq 1} f_r x^r$ be a formal power series where f_1 is invertible, and let $F^{(-1)}(x)$ denote its compositional inverse such that $F(F^{(-1)}(x)) = x$. For any formal power series $H(x)$ and any integer $n \geq 1$:*

$$[x^n]H(F^{(-1)}(x)) = \frac{1}{n}[x^{n-1}]H'(x) \left(\frac{x}{F(x)} \right)^n$$

Proof. To apply [Lemma 253](#), we define $\Phi(x) := \frac{x}{F(x)}$. Because $f_1 \neq 0$, $\Phi(x)$ is a well-defined formal power series with a non-zero constant term $\Phi(0) = 1/f_1$.

Let $A(x) := F^{(-1)}(x)$. By the definition of the compositional inverse, we know that $F(A(x)) = x$.

We evaluate our function Φ at $A(x)$:

$$\Phi(A(x)) = \frac{A(x)}{F(A(x))} = \frac{A(x)}{x}$$

Multiplying both sides by x , we recover the necessary functional equation:

$$A(x) = x\Phi(A(x))$$

This satisfies the hypothesis of [Lemma 253](#) with $f(x) = A(x)$. We can therefore apply it to recover the coefficients of the composite function $H(A(x))$:

$$[x^n]H(A(x)) = \frac{1}{n}[x^{n-1}]H'(x)\Phi(x)^n$$

Substituting $A(x) = F^{\langle -1 \rangle}(x)$ and our definition of $\Phi(x) = \frac{x}{F(x)}$ back into the relation yields:

$$[x^n]H(F^{\langle -1 \rangle}(x)) = \frac{1}{n}[x^{n-1}]H'(x) \left(\frac{x}{F(x)} \right)^n$$

□

13.4.4 Compositional Inverse

Let $F(x)$ be a formal power series such that $F(0) = 0$ and $F'(0) \neq 0$. The compositional inverse of $F(x)$ is the unique formal power series $G(x)$ satisfying $F(G(x)) = x$.

1. Reduction to $F'(0) = 1$ We can assume without loss of generality that $F'(0) = 1$. If $F'(0) = c \neq 1$, we can scale the series by defining $\tilde{F}(x) = c^{-1}F(x)$. Once we find its compositional inverse $\tilde{G}(x)$ such that $\tilde{F}(\tilde{G}(x)) = x$, we have $c^{-1}F(\tilde{G}(x)) = x$, meaning $F(\tilde{G}(x)) = cx$. By substituting $x \mapsto c^{-1}x$, we obtain $F(\tilde{G}(c^{-1}x)) = x$, which gives $G(x) = \tilde{G}(c^{-1}x)$.

2. Lagrange Inversion Assume $F(x) = x + f_2x^2 + \dots$, and let $G(x)$ be its compositional inverse. By [Lemma 254](#), we can relate the powers of $F(x)$ to the coefficients of $G(x)$. Specifically, for any k :

$$[x^n]F(x)^k = \frac{k}{n}[x^{n-k}] \left(\frac{x}{G(x)} \right)^n$$

Let us define an auxiliary series $H(x) = \frac{x}{G(x)}$. Our goal is to compute the polynomial $H(x)^n \pmod{x^n}$. Let its coefficients be $H(x)^n = \sum_{j=0}^{n-1} u_j x^j$. From the inversion formula, setting $j = n - k$ (so $k = n - j$), we can isolate u_j :

$$u_j = [x^j]H(x)^n = \frac{n}{n-j}[x^n]F(x)^{n-j}$$

3. Power Projection If we define the target vector $W(x) = x^n$, computing the power projection yields the sequence $v_k = [x^n]F(x)^k$ for $k = 0 \dots n$ in $O(n \log^2 n)$ time.

Once we have the sequence v_k , we can construct $H(x)^n$ in linear time by scaling and reversing the sequence:

$$u_j = \frac{n}{n-j} v_{n-j}$$

Finally, recovering $G(x)$ requires taking the inverse power of $H(x)^n$:

$$H(x)^{-1} = (H(x)^n)^{-1/n}$$

Since $H(x) = x/G(x)$, we recover $G(x) = x \cdot H(x)^{-1}$.

Algorithm 43 Compositional Inverse

```
1: function COMPINVERSE( $F(x), n$ )
2:   if  $n = 0$  then
3:     return 0
4:    $c \leftarrow F[1]$ 
5:    $F_{\text{monic}}(x) \leftarrow c^{-1}F(x)$ 
6:    $W(x) \leftarrow x^n$ 
7:    $V \leftarrow \text{POWERPROJECTION}(F_{\text{monic}}(x), W(x), n)$ 
8:    $A \leftarrow$  empty array of size  $n$ 
9:   for  $k \leftarrow 0$  to  $n - 1$  do
10:     $A[k] \leftarrow n \cdot V[k + 1] \cdot (k + 1)^{-1}$ 
11:     $H_n(x) \leftarrow \text{Reverse}(A)$ 
12:     $P \leftarrow -n^{-1} \pmod{M}$ 
13:     $H_{\text{inv}}(x) \leftarrow \text{PolyPow}(H_n(x), P, n)$ 
14:     $\tilde{G}(x) \leftarrow x \cdot H_{\text{inv}}(x)$ 
15:    for  $k \leftarrow 1$  to  $n$  do
16:       $G[k] \leftarrow \tilde{G}[k] \cdot c^{-k}$ 
17:  return  $G(x)$ 
```

$\triangleright V[k] = [x^n]F_{\text{monic}}(x)^k$ for $k \in [0, n]$

$\triangleright H_n[j] = A[n - 1 - j]$

13.5 Evaluation and Interpolation

13.5.1 Multipoint Evaluation

Multipoint evaluation computes the values of a polynomial $F(x) \in \mathbb{K}[x]_{<n}$ at n distinct points $\alpha_1, \dots, \alpha_n$. This linear map corresponds to multiplying the Vandermonde matrix $V(\alpha)$ by the coefficient vector of $F(x)$. We construct an algorithm for the transposed problem, and then transpose it.

The transposed problem multiplies the transposed Vandermonde matrix $V(\alpha)^T$ by a vector $c = (c_1, \dots, c_n)^T$. The j -th entry of the output is $\sum_{i=1}^n c_i \alpha_i^j$, which represents the first n coefficients of the rational function power series:

$$R(x) = \sum_{i=1}^n \frac{c_i}{1 - \alpha_i x} \pmod{x^n}$$

1. The Algorithm for the Transposed Problem To compute $R(x)$ efficiently, we use a bottom-up divide-and-conquer approach on a subproduct tree. For any node u representing a subset of points S_u , we maintain a numerator $N_u(x)$ and a denominator $D_u(x) = \prod_{i \in S_u} (1 - \alpha_i x)$.

At the leaves, $N_{\{i\}}(x) = c_i$ and $D_{\{i\}}(x) = 1 - \alpha_i x$. For an internal node u with children L and R , we combine them as:

$$N_u(x) = N_L(x)D_R(x) + N_R(x)D_L(x), \quad D_u(x) = D_L(x)D_R(x)$$

At the root, the final output is obtained by computing $N_{\text{root}}(x) \cdot D_{\text{root}}(x)^{-1} \pmod{x^n}$.

Since $D_u(x)$ depends only on the fixed evaluation points α_i , it is treated as a constant in the linear map.

2. Transposing the Recurrence We use the standard dot-product. In the forward transposed problem, the root step is $S(x) = N_{\text{root}}(x) \cdot D_{\text{root}}(x)^{-1} \pmod{x^n}$. By applying the lemma, if the input to the multipoint evaluation is $F(x)$ (which acts as the dual to $S(x)$), the transposed state at the root is:

$$T_{\text{root}}(x) = F(x) \circ D_{\text{root}}^{-1}(x)$$

During the upward combination phase, a node u computes $N_u = N_L D_R + N_R D_L$. Because the inner product is linear, transposing this addition distributes the middle product to the children. The transposed

states passed down from u to L and R are:

$$T_L(x) = T_u(x) \circ D_R(x) \quad \text{and} \quad T_R(x) = T_u(x) \circ D_L(x)$$

At the base case (the leaves), the forward step initializes $N_{\{i\}} = c_i$. The transposed operator simply extracts the degree-0 coefficient of the leaf state. Thus, the evaluated value $F(\alpha_i)$ is identically $[x^0]T_{\{i\}}(x)$.

Algorithm 44 Multipoint Evaluation

```

1: function FASTMULTIPOINTEVAL( $F(x), \alpha_1, \dots, \alpha_n$ )
2:    $D \leftarrow \text{PRECOMPUTESUBPRODUCTTREE}(\alpha_1, \dots, \alpha_n)$ 
3:    $I_{root}(x) \leftarrow D_{root}(x)^{-1} \pmod{x^n}$ 
4:    $T_{root}(x) \leftarrow \text{TRANPOSEDMUL}(F(x), I_{root}(x))$ 
5:    $Y \leftarrow \text{array of size } n$ 
6:   EVALTOPDOWN( $root, T_{root}(x), D, Y$ )
7: return  $Y$ 
8: function EVALTOPDOWN( $u, T_u(x), D, Y$ )
9:   if  $u$  is a leaf for point  $\alpha_i$  then
10:     $Y[i] \leftarrow T_u[0]$ 
11:   return
12:    $T_L(x) \leftarrow \text{TRANPOSEDMUL}(T_u(x), D_{R.child}(x))$ 
13:    $T_R(x) \leftarrow \text{TRANPOSEDMUL}(T_u(x), D_{L.child}(x))$ 
14:   EVALTOPDOWN( $L.child, T_L(x), D, Y$ )
15:   EVALTOPDOWN( $R.child, T_R(x), D, Y$ )

```

13.5.2 Chirp-Z

When the evaluation points form a geometric progression $\alpha_j = z^j$ for $j = 0, \dots, k-1$, we can compute the multipoint evaluation in $O(n \log n)$.

To evaluate $F(x) = \sum_{i=0}^{d-1} p_i x^i$ at z^j , we substitute $x = z^j$:

$$F(z^j) = \sum_{i=0}^{d-1} p_i z^{ij}$$

We apply the identity $ij = \binom{i+j}{2} - \binom{i}{2} - \binom{j}{2}$:

$$F(z^j) = \sum_{i=0}^{d-1} p_i z^{\binom{i+j}{2}} z^{-\binom{i}{2}} z^{-\binom{j}{2}} = z^{-\binom{j}{2}} \sum_{i=0}^{d-1} \left(p_i z^{-\binom{i}{2}} \right) z^{\binom{i+j}{2}}$$

Notice that the inner summation is the Middle Product of $F_{scaled}(x)$ with a sequence $B(x)$:

$$F_{scaled}(x) = \sum_{i=0}^{d-1} \left(p_i z^{-\binom{i}{2}} \right) x^i, \quad B(x) = \sum_{m=0}^{k+d-1} z^{\binom{m}{2}} x^m$$

The evaluation over all k points is given by $Y = (F_{scaled}(x) \circ B(x)) \pmod{x^k}$, followed by a scaling by $z^{-\binom{j}{2}}$.

Algorithm 45 Chirp-Z

```
1: function SCALEQBINOMIAL( $F(x), a$ )
    $\triangleright$  Multiplies the  $i$ -th coefficient by  $a^{\binom{i}{2}}$  in  $O(n)$ 
2:    $d \leftarrow \deg(F)$ 
3:    $cur \leftarrow 1; total \leftarrow 1$ 
4:   for  $i \leftarrow 0$  to  $d$  do
5:      $F[i] \leftarrow (F[i] \cdot total) \pmod{M}$ 
6:      $total \leftarrow (total \cdot cur) \pmod{M}$ 
7:      $cur \leftarrow (cur \cdot a) \pmod{M}$ 
8:   return  $F(x)$ 
9: function CHIRPZ( $F(x), z, k$ )
10:   $d \leftarrow \deg(F)$ 
11:   $z_{inv} \leftarrow z^{-1} \pmod{M}$ 
12:   $B(x) \leftarrow$  Polynomial of  $(k + d)$  ones
13:   $B(x) \leftarrow \text{SCALEQBINOMIAL}(B(x), z)$ 
14:   $F_{scaled}(x) \leftarrow \text{SCALEQBINOMIAL}(F(x), z_{inv})$ 
15:   $Y(x) \leftarrow \text{TRANPOSEDMUL}(B(x), F_{scaled}(x)) \pmod{x^k}$ 
16:   $Y(x) \leftarrow \text{SCALEQBINOMIAL}(Y(x), z_{inv})$ 
17:  return  $Y(x)$ 
```

13.5.3 Interpolation

Polynomial interpolation finds the unique polynomial $F(x) \in \mathbb{K}[x]_{<n}$ such that $F(\alpha_i) = y_i$ for n distinct points $\alpha_1, \dots, \alpha_n$. By the Lagrange interpolation formula:

$$F(x) = \sum_{i=1}^n y_i \prod_{j \neq i} \frac{x - \alpha_j}{\alpha_i - \alpha_j}$$

Let $D(x) = \prod_{i=1}^n (x - \alpha_i)$. We have $D'(\alpha_i) = \prod_{j \neq i} (\alpha_i - \alpha_j)$. Thus, the formula is:

$$F(x) = \sum_{i=1}^n \frac{y_i}{D'(\alpha_i)} \frac{D(x)}{x - \alpha_i}$$

Thus we need to solve two problems:

1. Compute the values $v_i = D'(\alpha_i)$ for all $i = 1, \dots, n$.
2. Compute the coefficients $c_i = y_i/v_i$, and evaluate $\sum_{i=1}^n c_i \prod_{j \neq i} (x - \alpha_j)$.

1. The Subproduct Tree Both tasks rely on a subproduct tree over the points α_i . For any node u corresponding to a subset of points S_u , we consider $D_u(x) = \prod_{i \in S_u} (x - \alpha_i)$ and $D_u^R(x) = \prod_{i \in S_u} (1 - \alpha_i x)$.

- **Downward Pass:** To evaluate $D'(x)$ at all α_i , we initialize the root state:

$$T_{root}(x) = D'_{root}(x) \circ (D_{root}^R(x))^{-1} \pmod{x^n}$$

At any internal node u , we compute:

$$T_L(x) = T_u(x) \circ D_R^R(x) \quad \text{and} \quad T_R(x) = T_u(x) \circ D_L^R(x)$$

- **Leaves:** When the recursion reaches a leaf representing α_i , we have: $[x^0]T_{\{i\}}(x) = D'(\alpha_i)$. We compute the $c_i = y_i/D'(\alpha_i)$, and return $N_{\{i\}}(x) = c_i$.

- **Combine:** We combine $N_L(x)$ and $N_R(x)$ to form N_u :

$$N_u(x) = N_L(x)D_R(x) + N_R(x)D_L(x)$$

At the root, $N_{root}(x) = F(x)$.

Algorithm 46 Interpolation

```

1: function FASTINTERPOLATION( $\alpha_1, \dots, \alpha_n, y_1, \dots, y_n$ )
2:    $D, D^R \leftarrow \text{PRECOMPUTESUBPRODUCTTREE}(\alpha_1, \dots, \alpha_n)$ 
3:    $I_{root}(x) \leftarrow (D^R[\text{root}](x))^{-1} \pmod{x^n}$ 
4:    $Q_{root}(x) \leftarrow \frac{d}{dx} D[\text{root}](x)$ 
5:    $T_{root}(x) \leftarrow \text{TRANPOSEDMUL}(Q_{root}(x), I_{root}(x))$ 
6:   return INTERPREC( $root, T_{root}(x), D, D^R, y$ )
7: function INTERPREC( $u, T_u(x), D, D^R, y$ )
8:   if  $u$  is a leaf for point  $\alpha_i$  then
9:      $v_i \leftarrow T_u[0]$ 
10:    return  $(y_i \cdot v_i^{-1}) \pmod{M}$ 
11:    $T_L(x) \leftarrow \text{TRANPOSEDMUL}(T_u(x), D^R[R.child](x))$ 
12:    $T_R(x) \leftarrow \text{TRANPOSEDMUL}(T_u(x), D^R[L.child](x))$ 
13:    $N_L(x) \leftarrow \text{INTERPREC}(L.child, T_L(x), D, D^R, y)$ 
14:    $N_R(x) \leftarrow \text{INTERPREC}(R.child, T_R(x), D, D^R, y)$ 
15:    $N_u(x) \leftarrow N_L(x) \cdot D[R.child](x) + N_R(x) \cdot D[L.child](x)$ 
16:   return  $N_u(x)$ 

```

13.5.4 Inverse Chirp-Z

The Inverse Chirp-Z algorithm reconstructs a polynomial $F(x) \in \mathbb{K}[x]_{<k}$ from its evaluations $y_i = F(z^i)$ at a geometric progression for $i = 0, \dots, k-1$ in $O(k \log k)$.

The Lagrange interpolation formula gives:

$$F(x) = \sum_{i=0}^{k-1} \frac{y_i}{D'(z^i)} \prod_{j \neq i} (x - z^j)$$

where $D(x) = \prod_{j=0}^{k-1} (x - z^j)$. We consider the evaluation $D'(z^i) = \prod_{j \neq i} (z^i - z^j)$.

For $j < i$, we factor out z^i and substitute $m = i - j$:

$$\prod_{j=0}^{i-1} (z^i - z^j) = \prod_{j=0}^{i-1} z^i (1 - z^{j-i}) = z^{i^2} \prod_{m=1}^i (1 - z^{-m})$$

For $j > i$, we again factor out z^i and substitute $m = j - i$:

$$\prod_{j=i+1}^{k-1} (z^i - z^j) = \prod_{j=i+1}^{k-1} z^i (1 - z^{j-i}) = z^{i(k-1-i)} \prod_{m=1}^{k-1-i} (1 - z^m)$$

We multiply these together ($i^2 + ik - i - i^2 = i(k-1)$), yielding:

$$D'(z^i) = z^{i(k-1)} \left(\prod_{m=1}^i (1 - z^{-m}) \right) \left(\prod_{m=1}^{k-1-i} (1 - z^m) \right)$$

Thus, the weight $w_i = 1/D'(z^i)$ is a product of $z^{-i(k-1)}$, a prefix product of $(1 - z^{-m})^{-1}$, and a suffix product of $(1 - z^m)^{-1}$. We define $y'_i = y_i w_i$.

Consider the reversal of the interpolating polynomial, $F^R(x) = x^{k-1}F(x^{-1})$. Substituting x^{-1} yields:

$$F^R(x) = \sum_{i=0}^{k-1} y'_i x^{k-1} \prod_{j \neq i} (x^{-1} - z^j) = \sum_{i=0}^{k-1} y'_i \prod_{j \neq i} (1 - z^j x)$$

Notice that this reversed polynomial is exactly the numerator of a partial fraction decomposition. Defining $Q(x) = \prod_{j=0}^{k-1} (1 - z^j x)$, we obtain:

$$\frac{F^R(x)}{Q(x)} = \sum_{i=0}^{k-1} \frac{y'_i}{1 - z^i x}$$

We expand the rational function:

$$\sum_{i=0}^{k-1} \frac{y'_i}{1 - z^i x} = \sum_{i=0}^{k-1} y'_i \sum_{m=0}^{\infty} (z^i x)^m = \sum_{m=0}^{\infty} \left(\sum_{i=0}^{k-1} y'_i z^{im} \right) x^m$$

Let $S_m = \sum_{i=0}^{k-1} y'_i z^{im}$. S_m is the evaluation of $Y(x) = \sum_{i=0}^{k-1} y'_i x^i$ at the point z^m . Thus, the first k terms, $S(x) = \sum_{m=0}^{k-1} S_m x^m$, can be computed by applying a Chirp-Z transform to $Y(x)$.

Because $\deg(F^R) < k$, we can then compute:

$$F^R(x) \equiv S(x)Q(x) \pmod{x^k}$$

We thus need to compute $Q(x) = \prod_{j=0}^{k-1} (1 - z^j x)$. We apply [Lemma 105](#), which provides:

$$Q(x) = \prod_{j=0}^{k-1} (1 - z^j x) = \sum_{i=0}^k (-1)^i z^{\binom{i}{2}} \binom{k}{i}_z x^i$$

where $\binom{k}{i}_z$ denotes the q -binomial coefficient. This can be computed in $O(k)$.

Algorithm 47 Inverse Chirp-Z

```
1: function PREFPRODINVERSES( $a, k$ )
2:    $P \leftarrow$  array of size  $k$  with  $P[0] = 1$ 
3:    $a_j \leftarrow a$ 
4:   for  $m \leftarrow 1$  to  $k - 1$  do
5:      $P[m] \leftarrow P[m - 1] \cdot (1 - a_j)^{-1} \pmod{M}$ 
6:      $a_j \leftarrow (a_j \cdot a) \pmod{M}$ 
7:   return  $P$ 
8: function INVERSECHIRPZ( $y_0 \dots y_{k-1}, z, k$ )
9:    $P_{pos} \leftarrow$  PREFPRODINVERSES( $z, k$ )
10:   $P_{neg} \leftarrow$  PREFPRODINVERSES( $z^{-1}, k$ )
11:   $z_k \leftarrow z^{-(k-1)} \pmod{M}$ 
12:   $z_{ki} \leftarrow 1$ 
13:   $Y(x) \leftarrow 0$ 
14:  for  $i \leftarrow 0$  to  $k - 1$  do
15:     $w_i \leftarrow z_{ki} \cdot P_{neg}[i] \cdot P_{pos}[(k - 1) - i] \pmod{M}$ 
16:     $Y[i] \leftarrow y_i \cdot w_i \pmod{M}$ 
17:     $z_{ki} \leftarrow z_{ki} \cdot z_k \pmod{M}$ 
18:   $S(x) \leftarrow$  CHIRPZ( $Y(x), z, k$ )
19:   $Q(x) \leftarrow \sum_{i=0}^{k-1} \left( (-1)^i z^{\binom{i}{2}} \binom{k}{i}_z \right) x^i$ 
20:   $F^R(x) \leftarrow (S(x) \cdot Q(x)) \pmod{x^k}$ 
21:  return Reverse( $F^R(x), k$ )
```

$\triangleright$ Returns array P where $P[m] = \prod_{j=1}^m (1 - a^j)^{-1}$

13.5.5 Hermite Interpolation

Hermite interpolation reconstructs a polynomial from several coefficients of its Taylor expansion at each of several points. We use Hasse derivatives, since these work without any restriction on the characteristic.

Definition 255 (Hasse Derivatives). For $f \in R[x]$, define $D^{(j)}f(a)$ by

$$f(a + z) = \sum_{j \geq 0} D^{(j)}f(a)z^j.$$

Thus

$$D^{(0)}f(a) = f(a),$$

and whenever $j!$ is invertible,

$$D^{(j)}f(a) = \frac{f^{(j)}(a)}{j!}.$$

The basic observation is that specifying the first several Hasse derivatives at a is exactly the same as specifying the polynomial modulo a power of $x - a$.

Lemma 256 (Determination by Remainders). For every $f \in R[x]$,

$$f(x) \equiv \sum_{j=0}^{s-1} D^{(j)}f(a)(x - a)^j \pmod{(x - a)^s}.$$

Proof. Substitute

$$z = x - a$$

into the defining identity

$$f(a+z) = \sum_{j \geq 0} D^{(j)} f(a) z^j.$$

Modulo $(x-a)^s$, all terms with $j \geq s$ vanish. □

Hence the conditions

$$D^{(j)} f(a) = y_j, \quad 0 \leq j < s,$$

are equivalent to the single congruence

$$f(x) \equiv \sum_{j=0}^{s-1} y_j (x-a)^j \pmod{(x-a)^s}.$$

Hermite interpolation is therefore polynomial Chinese remaindering.

Lemma 257 (Coprimalty of the Moduli). *Let $a, b \in R$, and suppose $a-b$ is a unit. Then for every $r, s \geq 1$,*

$$(x-a)^r \quad \text{and} \quad (x-b)^s$$

are coprime in $R[x]$.

Proof. The ideals

$$I = (x-a), \quad J = (x-b)$$

are coprime because

$$(x-a) - (x-b) = b-a$$

is a unit. Thus there are $u \in I$ and $v \in J$ such that

$$u + v = 1.$$

Now expand

$$1 = (u+v)^{r+s-1}.$$

Every term contains either at least r copies of u , and hence belongs to I^r , or at least s copies of v , and hence belongs to J^s . Therefore

$$I^r + J^s = R[x].$$

□

Theorem 258 (Hermite Interpolation). *Let*

$$a_1, \dots, a_k \in R$$

satisfy

$$a_i - a_j \in R^\times \quad (i \neq j),$$

and let

$$s_1, \dots, s_k \geq 1.$$

Set

$$n = \sum_{i=1}^k s_i.$$

For arbitrary values

$$y_{i,j} \in R, \quad 0 \leq j < s_i,$$

there is a unique polynomial $f \in R[x]$ of degree less than n satisfying

$$D^{(j)} f(a_i) = y_{i,j}$$

for every i, j .

Proof. For each i , define

$$P_i(x) = (x - a_i)^{s_i}$$

and the prescribed local Taylor polynomial

$$T_i(x) = \sum_{j=0}^{s_i-1} y_{i,j}(x - a_i)^j.$$

By [Lemma 256](#), the desired conditions are exactly

$$f \equiv T_i \pmod{P_i} \quad (i = 1, \dots, k).$$

By [Lemma 257](#), the P_i 's are pairwise coprime. The Chinese remainder theorem therefore gives a unique residue class modulo

$$P = \prod_{i=1}^k P_i.$$

Since

$$\deg P = \sum_i s_i = n,$$

this residue class has a unique representative of degree less than n . □

CRT Formula The preceding proof gives an explicit formula. Put

$$P = \prod_i P_i, \quad Q_i = \frac{P}{P_i}.$$

Since Q_i is invertible modulo P_i , let

$$U_i \equiv Q_i^{-1} \pmod{P_i}.$$

Then

$$Q_i U_i \equiv \begin{cases} 1 & \pmod{P_i}, \\ 0 & \pmod{P_j}, \quad j \neq i. \end{cases}$$

Hence

$$f \equiv \sum_{i=1}^k T_i U_i Q_i \pmod{P}.$$

Theorem 259 (Fast Hermite Interpolation). *Given*

$$D^{(j)} f(a_i), \quad 0 \leq j < s_i,$$

with total multiplicity

$$n = \sum_i s_i,$$

the polynomial f of degree less than n can be reconstructed in

$$O(n \log^2 n)$$

ring operations.

Proof. We implement the CRT formula by a product tree.

1. Build the product tree. At the leaves put

$$P_i = (x - a_i)^{s_i}.$$

Build a balanced binary tree in which every internal node ν stores the product

$$P_\nu = \prod_{i \text{ below } \nu} P_i.$$

The root stores

$$P = \prod_i P_i.$$

At any fixed level, the degrees of all node polynomials sum to n . Thus all polynomial products at that level cost

$$O(n \log n).$$

There are $O(\log n)$ levels, so the entire product tree costs

$$O(n \log^2 n).$$

2. Compute every cofactor modulo its own modulus. We need

$$Q_i = \frac{P}{P_i},$$

but only modulo P_i .

This can be done by one downward pass through the same tree.

For a node ν , let

$$Q_\nu = \frac{P}{P_\nu}.$$

We store only the residue

$$q_\nu = Q_\nu \bmod P_\nu.$$

At the root,

$$q_{\text{root}} = 1.$$

Suppose ν has left and right children λ, ρ . Since

$$Q_\lambda = Q_\nu P_\rho, \quad Q_\rho = Q_\nu P_\lambda,$$

we may compute

$$\boxed{q_\lambda = q_\nu P_\rho \bmod P_\lambda, \quad q_\rho = q_\nu P_\lambda \bmod P_\rho.}$$

Here q_ν may first be reduced modulo the relevant child modulus.

Inductively,

$$q_\nu \equiv \frac{P}{P_\nu} \pmod{P_\nu}.$$

At a leaf i , therefore,

$$\boxed{q_i \equiv Q_i \pmod{P_i}.}$$

Polynomial multiplication and reduction at a node of total degree d_ν cost

$$O(d_\nu \log d_\nu).$$

Again the node degrees sum to n at every level, so the whole downward pass costs

$$O(n \log^2 n).$$

3. Invert the local cofactors. For each i ,

$$Q_i(a_i) = \prod_{j \neq i} (a_i - a_j)^{s_j}$$

is a unit. Hence Q_i , and therefore q_i , is invertible modulo

$$P_i = (x - a_i)^{s_i}.$$

Compute

$$U_i = q_i^{-1} \bmod P_i.$$

Power-series inversion after the shift $x \mapsto x + a_i$, computes this in

$$O(s_i \log s_i)$$

operations.

Summing over all i , all inverses cost

$$O(n \log n).$$

Now set

$$C_i = T_i U_i \bmod P_i.$$

Then

$$C_i Q_i \equiv T_i \pmod{P_i}.$$

4. Assemble the CRT sum. We must compute

$$\sum_i C_i Q_i = \sum_i C_i \frac{P}{P_i}$$

without constructing all the Q_i 's separately.

For every node ν , define

$$F_\nu = \sum_{i \text{ below } \nu} C_i \frac{P_\nu}{P_i}.$$

At a leaf i ,

$$F_i = C_i.$$

If ν has children λ, ρ , then

$$\boxed{F_\nu = F_\lambda P_\rho + F_\rho P_\lambda.}$$

Thus all F_ν 's can be computed in one upward pass through the product tree.

At the root,

$$F_{\text{root}} = \sum_i C_i \frac{P}{P_i} = \sum_i C_i Q_i.$$

Moreover,

$$\deg F_{\text{root}} < n.$$

For every i ,

$$F_{\text{root}} \equiv C_i Q_i \equiv T_i \pmod{P_i}.$$

Hence, by uniqueness in [Theorem 258](#),

$$F_{\text{root}} = f.$$

The upward pass has the same complexity as the product tree:

$$O(n \log^2 n).$$

Combining the four stages gives

$$\boxed{O(n \log^2 n)}$$

ring operations. □

13.5.6 Multivariate Evaluation

Let

$$f(x_1, \dots, x_m)$$

have individual degree less than d , so its dense coefficient representation has size

$$d^m.$$

Given points

$$a_1, \dots, a_N \in \mathbb{F}^m,$$

we want to compute

$$f(a_1), \dots, f(a_N).$$

There are two useful algorithms.

When the number of variables is small, multimodular reduction gives an almost linear algorithm. Its natural grid has size roughly

$$(dm)^m,$$

which is harmless for fixed m .

For arbitrary m , this m^m factor is too expensive. The second algorithm instead uses prime powers p^m . This removes the factor m from the size of the auxiliary primes, at the cost of working over rings with nilpotents. Hasse derivatives make this possible.

Product-Grid Evaluation Both algorithms eventually reduce to evaluation on a Cartesian product.

Lemma 260 (Product-Grid Evaluation). *Let*

$$A_1, \dots, A_m \subseteq R, \quad q_i = |A_i|, \quad Q = \prod_{i=1}^m q_i.$$

An m -variate polynomial of individual degree less than d can be evaluated at every point of

$$A_1 \times \dots \times A_m$$

in

$$\boxed{O\left(m(d^m + Q) \log^2(d + \max_i q_i)\right)}$$

ring operations.

Proof. Reorder the variables so that

$$q_1 \leq q_2 \leq \dots \leq q_m.$$

After evaluating the first i variables, the coefficient/evaluation tensor has size

$$D_i = q_1 \cdots q_i d^{m-i},$$

where

$$D_0 = d^m, \quad D_m = Q.$$

To process x_i , fix the already evaluated coordinates and all remaining exponents. This leaves

$$q_1 \cdots q_{i-1} d^{m-i}$$

univariate polynomials of degree less than d , each evaluated at the q_i points of A_i .

Fast univariate multipoint evaluation therefore costs

$$O((D_{i-1} + D_i) \log^2(d + q_i))$$

at stage i .

Finally,

$$\frac{D_i}{D_{i-1}} = \frac{q_i}{d}.$$

Since the q_i 's are nondecreasing, the sequence D_i first decreases and then increases. Hence

$$D_i \leq \max\{D_0, D_m\} \leq d^m + Q$$

for every i . Summing over the m stages proves the result. $\square$

Few Variables The simpler algorithm is due to Kedlaya and Umans. Its main operation is ordinary multimodular reduction.

Suppose first that

$$f \in (\mathbb{Z}/r\mathbb{Z})[x_1, \dots, x_m].$$

Lift its coefficients and the evaluation points to integers in

$$\{0, \dots, r-1\}.$$

Let F denote the lifted polynomial.

Since F has at most d^m monomials and total degree at most $m(d-1)$,

$$0 \leq F(a_i) < d^m r^{dm}.$$

Thus it suffices to know $F(a_i)$ modulo distinct primes whose product exceeds

$$d^m r^{dm}.$$

The required primes have size around

$$O(md \log r).$$

We reduce the polynomial and all evaluation points modulo these primes and recurse.

After the modulus has become small enough, we simply evaluate on the full grid

$$\mathbb{F}_p^m.$$

The terminal characteristic size is

$$p^m \approx (dm)^m = d^m m^m.$$

Thus ordinary multimodular reduction is already excellent when m is small, but is expensive for large m . The rest of this subsection is devoted to removing this cost.

Multivariate Hasse Derivatives For

$$\alpha = (\alpha_1, \dots, \alpha_m) \in \mathbb{N}^m,$$

define the Hasse derivative

$$\partial^\alpha f$$

by

$$f(x+z) = \sum_{\alpha \in \mathbb{N}^m} (\partial^\alpha f)(x) z^\alpha.$$

Equivalently, if

$$f(x) = \sum_{\beta} c_{\beta} x^{\beta},$$

then

$$\partial^{\alpha} f = \sum_{\beta \geq \alpha} \binom{\beta}{\alpha} c_{\beta} x^{\beta - \alpha}.$$

We will only need derivatives satisfying

$$|\alpha| < s, \quad s \leq m.$$

Their number is

$$\binom{m+s-1}{s-1} \leq \binom{2m}{m} < 4^m.$$

Thus every recursive step needs fewer than 4^m Hasse derivatives.

Lemma 261 (Nilpotent Taylor Expansion). *Let R be a commutative ring and let $I \subseteq R$ satisfy*

$$I^s = 0.$$

Suppose

$$a, b \in R^m$$

and every coordinate of $a - b$ lies in I . Then

$$f(a) = \sum_{|\alpha| < s} (\partial^{\alpha} f)(b)(a - b)^{\alpha}.$$

Proof. The multivariate Hasse-Taylor identity gives

$$f(a) = f(b + (a - b)) = \sum_{\alpha} (\partial^{\alpha} f)(b)(a - b)^{\alpha}.$$

If

$$|\alpha| \geq s,$$

then

$$(a - b)^{\alpha} \in I^{|\alpha|} \subseteq I^s = 0.$$

□

For

$$R = \mathbb{Z}/r^s\mathbb{Z}, \quad I = rR,$$

we have $I^s = 0$. Hence two points that agree modulo r differ by a nilpotent quantity, and their Taylor expansion truncates after total degree $s - 1$.

Why Prime Powers Remove the m^m Loss Suppose

$$f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1, \dots, x_m], \quad s \leq m.$$

For each evaluation point a_i , let

$$\bar{a}_i \in \{0, \dots, r - 1\}^m$$

be its coordinatewise residue modulo r .

Nilpotent Taylor gives

$$f(a_i) = \sum_{|\alpha| < s} (\partial^\alpha f)(\bar{a}_i)(a_i - \bar{a}_i)^\alpha.$$

Thus it suffices to evaluate fewer than 4^m derivative polynomials at the integer points $\bar{a}_i$.

Lift one such derivative to an integer polynomial F . It has at most d^m monomials, its coefficients are less than r^s , and every coordinate of $\bar{a}_i$ is less than r . Hence

$$0 \leq F(\bar{a}_i) < d^m r^s r^{m(d-1)} \leq d^m r^{dm} = (dr^d)^m.$$

With ordinary CRT we would need

$$\prod_j p_j > (dr^d)^m,$$

which is exactly where the unwanted factor m in the prime size comes from.

Instead choose distinct primes satisfying only

$$\prod_j p_j > dr^d$$

and work modulo p_j^m . Then

$$\prod_j p_j^m > (dr^d)^m > F(\bar{a}_i),$$

so the residues modulo the p_j^m 's still determine the integer value uniquely.

By the small-prime-product bound, all these primes can be chosen with

$$p_j = O(d(1 + \log r)).$$

Lemma 262 (One Prime-Power Reduction). *Let*

$$f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1, \dots, x_m], \quad s \leq m.$$

Evaluation of f at N arbitrary points reduces to fewer than

$$4^m$$

families of evaluation problems over rings

$$\mathbb{Z}/p^m\mathbb{Z}, \quad p = O(d(1 + \log r)),$$

followed by CRT and the truncated Taylor formula.

Proof. For every $|\alpha| < s$, lift

$$\partial^\alpha f$$

to an integer polynomial F_α .

Choose primes p_j with

$$\prod_j p_j > dr^d.$$

For each pair (α, j) , recursively compute

$$F_\alpha(\bar{a}_i) \pmod{p_j^m}$$

for every i .

Since

$$0 \leq F_\alpha(\bar{a}_i) < (dr^d)^m < \prod_j p_j^m,$$

CRT reconstructs $F_\alpha(\bar{a}_i)$ exactly as an integer. Reducing modulo r^s gives

$$(\partial^\alpha f)(\bar{a}_i).$$

The nilpotent Taylor formula then recovers $f(a_i)$. □

How Fast the Modulus Shrinks After one reduction,

$$r \mapsto O(d(1 + \log r)).$$

Iterating this observation gives the recursion estimate we need.

Lemma 263 (Modulus Shrinkage). *After $t \geq 1$ prime-power reductions, every base modulus is at most*

$$O(d(1 + \log d + \log^{ot} r)).$$

Moreover, up to logarithmic factors, the total number of branches created by the small-prime choices is at most

$$d^t(1 + \log r).$$

Proof. Write $r_0 = r$. From [Lemma 262](#),

$$r_{i+1} = O(d(1 + \log r_i)).$$

The claimed modulus bound follows by induction. Indeed,

$$r_1 = O(d(1 + \log r)),$$

and if

$$r_i = O(d(1 + \log d + \log^{oi} r)),$$

then

$$\log r_i = O(1 + \log d + \log^{o(i+1)} r),$$

which gives the next case.

At the first level there are

$$O(d(1 + \log r))$$

possible primes. At every later level the number of primes is

$$O(d(1 + \log r_i)) = d \cdot \text{polylog}(d, \log r).$$

Multiplying over the t levels gives

$$\tilde{O}_t(d^t(1 + \log r)).$$

□

Theorem 264 (Evaluation over Prime-Power Rings). *Fix $t \geq 0$. Let*

$$f \in (\mathbb{Z}/r^s\mathbb{Z})[x_1, \dots, x_m], \quad s \leq m.$$

Then f can be evaluated at N arbitrary points using

$$\tilde{O}_t\left(2^{O(tm)} md^t(1 + \log r) \left([d(1 + \log d + \log^{ot} r)]^m + N\right)\right)$$

ring operations.

Proof. Every recursive level introduces fewer than 4^m Hasse derivatives. After t levels this contributes

$$4^{mt}.$$

At a leaf we must evaluate fewer than another 4^m derivatives on the complete residue grid. Thus all derivative multiplicities contribute

$$4^{m(t+1)} = 2^{O(tm)}.$$

By [Lemma 263](#), the number of prime branches contributes

$$\tilde{O}_t(d^t(1 + \log r)),$$

and every terminal modulus is

$$O(d(1 + \log d + \log^{ot} r)).$$

Hence the terminal product grid has the displayed size.

The derivative construction, reductions, CRTs, and final Taylor sums have the same or smaller asymptotic cost. $\square$

Extension Fields Let

$$\mathbb{F}_q = \mathbb{F}_p[z]/(E(z)), \quad e = \deg E.$$

We reduce evaluation over $\mathbb{F}_q$ to the prime-power rings above.

Lift field elements to degree- $< e$ polynomials in $\mathbb{Z}[z]$ with coefficients in

$$\{0, \dots, p-1\}.$$

Lift f to

$$F \in \mathbb{Z}[z][x_1, \dots, x_m]$$

and the evaluation points to

$$\tilde{a}_i(z) \in \mathbb{Z}[z]^m.$$

Set

$$G_i(z) = F(\tilde{a}_i(z)).$$

Then

$$f(a_i) = G_i(z) \pmod{p, E(z)}.$$

We have

$$\deg G_i < edm.$$

Also,

$$\|G_i\|_1 < d^m(ep)^{m(d-1)+1} < (d(ep)^d)^m.$$

Lemma 265 (Extension-Field Reduction). *There is an integer r' satisfying*

$$\boxed{d(ep)^d \leq r' < 2ed^2(ep)^d}$$

such that evaluation over $\mathbb{F}_q$ reduces to fewer than

$$\boxed{e4^m}$$

instances of multivariate evaluation over

$$\mathbb{Z}/(r')^m\mathbb{Z},$$

each on edN points.

The remaining work is

$$\tilde{O}(e4^mmd^m + e^2d4^m m^2N + Nedm).$$

Proof. By Bertrand's postulate, choose a prime

$$ed \leq P < 2ed$$

and let r' be the smallest power of P satisfying

$$r' \geq d(ep)^d.$$

Then

$$r' < P d(ep)^d < 2ed^2(ep)^d.$$

Work over

$$S = \mathbb{Z}/(r')^m \mathbb{Z}.$$

Take the ed points

$$j = 0, \dots, ed - 1.$$

For distinct j, j' ,

$$0 < |j - j'| < ed \leq P,$$

so $j - j'$ is a unit in S .

Write

$$F(x, z) = \sum_{u=0}^{e-1} f_u(x) z^u.$$

For each i, j , put

$$b_{i,j} = \tilde{a}_i(j) \in S^m.$$

Every coordinate of

$$\tilde{a}_i(z) - b_{i,j}$$

is divisible by $z - j$.

In

$$S[z]/(z - j)^m,$$

the ideal generated by $z - j$ has m -th power zero. Therefore [Lemma 261](#) gives

$$f_u(\tilde{a}_i(z)) \equiv \sum_{|\alpha| < m} (\partial^\alpha f_u)(b_{i,j}) (\tilde{a}_i(z) - b_{i,j})^\alpha \pmod{(z - j)^m}.$$

Hence

$$G_i(z) \equiv \sum_{u=0}^{e-1} z^u \sum_{|\alpha| < m} (\partial^\alpha f_u)(b_{i,j}) (\tilde{a}_i(z) - b_{i,j})^\alpha \pmod{(z - j)^m}.$$

There are e choices of u and fewer than 4^m relevant multi-indices α . Thus fewer than $e4^m$ multivariate evaluation instances suffice, each on the edN points $b_{i,j}$.

After these evaluations we know, for every i, j ,

$$G_i(z) \pmod{(z - j)^m}.$$

The moduli

$$(z - j)^m, \quad 0 \leq j < ed,$$

are pairwise coprime and have total degree

$$edm > \deg G_i.$$

By Hermite interpolation, they determine G_i uniquely modulo $(r')^m$.

Since

$$(r')^m > \|G_i\|_1,$$

lifting the reconstructed coefficients to

$$\{0, \dots, (r')^m - 1\}$$

recovers the integer polynomial G_i exactly. Reducing it modulo

$$p \quad \text{and} \quad E(z)$$

gives $f(a_i)$.

Fast Hermite interpolation costs only

$$O(edm \log^2(edm))$$

per output polynomial; the displayed additional-work bound follows. $\square$

Theorem 266 (Elementary Multivariate Evaluation). *Suppose*

$$q \leq \exp^{\circ h}(d)$$

for some fixed $h \geq 1$, and write

$$\mathbb{F}_q = \mathbb{F}_p[z]/(E(z)), \quad e = \deg E.$$

Then an m -variate polynomial of individual degree less than d can be evaluated at N arbitrary points deterministically in

$$\boxed{\tilde{O}_h \left(2^{O(hm)} emd^{h+2} (1 + \log q) ((d \log d)^m + eN) \right)}$$

bit operations.

Proof. Use

$$t = h + 1$$

levels of prime-power reduction inside [Lemma 265](#).

The auxiliary modulus satisfies

$$r' < 2ed^2(ep)^d,$$

so

$$\log r' = O(d(1 + \log q))$$

up to logarithmic factors in d and e .

Since

$$q \leq \exp^{\circ h}(d),$$

we have

$$\log^{\circ h} q \leq d.$$

After $h + 1$ logarithms,

$$\log^{\circ(h+1)} r' = O(\log d).$$

Therefore [Theorem 264](#) has terminal grid size

$$O((d \log d)^m).$$

The $t = h + 1$ recursion levels contribute

$$d^{h+1}(1 + \log r') = \tilde{O}_h(d^{h+2}(1 + \log q)),$$

and the derivative factors contribute

$$2^{O(hm)}.$$

Finally, the extension-field reduction contributes the outer factor $e4^m$, which is absorbed into $e2^{O(hm)}$, and replaces N by edN . This gives the stated bound. $\square$

14 Set Power Series

14.1 Subset Convolution

Given two functions $f, g : 2^V \rightarrow \mathbb{F}$ over a set V of size n , their subset convolution $h = f * g$ is defined as:

$$h(S) = \sum_{A \subseteq S} f(A)g(S \setminus A)$$

Computing this naively requires $O(3^n)$ time. We can optimize this to $O(n^2 2^n)$ using the Zeta and Möbius transforms.

Define the ranked representations $\hat{f}$ and $\hat{g}$:

$$\hat{f}_k(S) = \begin{cases} f(S) & \text{if } |S| = k \\ 0 & \text{otherwise} \end{cases}$$

The Zeta transform computes: $\mathcal{Z}(f)(S) = \sum_{A \subseteq S} f(A)$. We have that point-wise multiplication after the Zeta transform computes:

$$\mathcal{Z}^{-1}(\mathcal{Z}(u) \cdot \mathcal{Z}(v))(S) = \sum_{A \cup B = S} u(A)v(B)$$

By performing a standard linear convolution over the rank dimension and a point-wise multiplication in the Zeta domain, we compute a new ranked function $\hat{h}$:

$$\hat{h}_i(S) = \mathcal{Z}^{-1} \left(\sum_{j=0}^i \mathcal{Z}(\hat{f}_j) \cdot \mathcal{Z}(\hat{g}_{i-j}) \right) (S) = \sum_{A \cup B = S} \hat{f}_j(A) \hat{g}_{i-j}(B)$$

To extract the final answer for a set S , we evaluate the rank slice corresponding to its cardinality: $h(S) = \hat{h}_{|S|}(S)$.

By substituting $i = |S|$, the non-zero terms mandate that $|A| = j$ and $|B| = i - j$, which means $|A| + |B| = i = |S|$ so $A \cap B = \emptyset$.

Complexity

- **Time:** The Forward and Inverse Zeta Transforms process $n + 1$ arrays of size 2^n , taking $O(n \cdot n 2^n)$ time. The ranked point-wise multiplication takes $O(n^2)$ for each of the 2^n sets. Total Time: $O(n^2 2^n)$.
- **Space:** Allocating the ranked arrays requires $(n + 1) \times 2^n$ memory. Total Space: $O(n 2^n)$.

Algorithm 48 Subset Convolution

```
1: function ZETATransform( $A, n$ )
2:   for  $i \leftarrow 0$  to  $n - 1$  do
3:     for  $S \leftarrow 0$  to  $2^n - 1$  do
4:       if  $(S \& (1 \ll i)) \neq 0$  then
5:          $A[S] \leftarrow A[S] + A[S \oplus (1 \ll i)]$ 
6: function MOBIUSTransform( $A, n$ )
7:   for  $i \leftarrow 0$  to  $n - 1$  do
8:     for  $S \leftarrow 0$  to  $2^n - 1$  do
9:       if  $(S \& (1 \ll i)) \neq 0$  then
10:         $A[S] \leftarrow A[S] - A[S \oplus (1 \ll i)]$ 
11: function SUBSETCONVOLUTION( $f, g, n$ )
12:   Initialize  $\hat{f}, \hat{g}, \hat{h}$  as arrays of size  $(n + 1) \times 2^n$  filled with 0
13:   for  $S \leftarrow 0$  to  $2^n - 1$  do
14:      $\hat{f}[|S|][S] \leftarrow f[S]$ 
15:      $\hat{g}[|S|][S] \leftarrow g[S]$ 
16:   for  $k \leftarrow 0$  to  $n$  do
17:     ZETATransform( $\hat{f}[k], n$ )
18:     ZETATransform( $\hat{g}[k], n$ )
19:   for  $i \leftarrow 0$  to  $n$  do
20:     for  $j \leftarrow 0$  to  $i$  do
21:       for  $S \leftarrow 0$  to  $2^n - 1$  do
22:          $\hat{h}[i][S] \leftarrow \hat{h}[i][S] + \hat{f}[j][S] \cdot \hat{g}[i - j][S]$ 
23:   for  $k \leftarrow 0$  to  $n$  do
24:     MOBIUSTransform( $\hat{h}[k], n$ )
25:   Initialize  $h$  as an array of size  $2^n$ 
26:   for  $S \leftarrow 0$  to  $2^n - 1$  do
27:      $h[S] \leftarrow \hat{h}[|S|][S]$ 
28:   return  $h$ 
```

14.2 Transposed Subset Convolution

Given two functions $f, g : 2^V \rightarrow \mathbb{F}$ over a set V of size n , the Transposed Subset Convolution is equivalent to Superset Convolution, and computes h defined as:

$$h(S) = \sum_{Y \supseteq S} f(Y)g(Y \setminus S)$$

We reverse f to get $\tilde{f}$, then compute the subset convolution with g , then reverse the output.

To see why this yields the superset convolution, we expand the definition of $h(S)$ from the three steps above by substituting $K = U \setminus S$:

$$h(S) = \tilde{h}(U \setminus S) = \sum_{X \subseteq U \setminus S} f(U \setminus X)g((U \setminus S) \setminus X)$$

Letting $Y = U \setminus X$, the summation condition $X \subseteq U \setminus S$ becomes $S \subseteq Y$. The sum becomes:

$$h(S) = \sum_{S \subseteq Y} f(Y)g(Y \setminus S)$$

14.3 Inversion and Quotient

Given a subset power series $g : 2^V \rightarrow \mathbb{F}$ where $g(\emptyset) \neq 0$, its inverse $h = g^{-1}$ is the unique series satisfying $h * g = \varepsilon$, where $\varepsilon(\emptyset) = 1$ and $\varepsilon(S) = 0$ for all $S \neq \emptyset$. More generally, the quotient $h = f/g$ is the series

satisfying $h * g = f$.

We can compute the quotient layer-by-layer (by rank) in $O(n^2 2^n)$ time by a DP directly in the Ranked Zeta Transform domain.

Expanding the definition of the subset convolution evaluated at a specific set X , we isolate the contribution of $h(X)$:

$$f(X) = \sum_{A \subseteq X} h(A)g(X \setminus A) = h(X)g(\emptyset) + \sum_{A \subsetneq X} h(A)g(X \setminus A)$$

Let $E(X) = \sum_{A \subsetneq X} h(A)g(X \setminus A)$. If we process the sets in increasing order of cardinality $l = |X|$, all required values of $h(A)$ for $|A| < l$ will have already been computed.

This allows us to inductively construct the ranked components $\hat{h}_l$ of the quotient. For a fixed rank l , the transformed accumulation of the smaller ranks is:

$$\mathcal{Z}(\hat{E}_l) = \sum_{i=1}^l \mathcal{Z}(\hat{h}_{l-i}) \cdot \mathcal{Z}(\hat{g}_i)$$

We recover E in the subset domain by applying the inverse Zeta (Möbius) transform. Then, for every set X with $|X| = l$, we extract the value of $h(X)$:

$$h(X) = (f(X) - E(X)) \cdot g(\emptyset)^{-1}$$

Finally, we apply the forward Zeta transform to $\hat{h}_l$ so that its transformed values $\mathcal{Z}(\hat{h}_l)$ are available for computing the subsequent ranks $l+1, \dots, n$.

Algorithm The computation proceeds iteratively over the rank dimension:

1. **Initialization:** Compute the inverse scalar $g(\emptyset)^{-1}$. Initialize $(n+1) \times 2^n$ arrays $\hat{g}$ and $\hat{h}$. Fill $\hat{g}$ by placing $g(S)$ into $\hat{g}_{|S|}[S]$ and apply the Fast Zeta Transform to every rank slice of $\hat{g}$.
2. **Layered DP:** For each rank l from 0 to n :
 - (a) **Accumulate:** For each element S , compute $\hat{E}_l[S] = \sum_{i=1}^l \hat{h}_{l-i}[S] \cdot \hat{g}_i[S]$.
 - (b) **Möbius Inversion:** Apply the Fast Möbius Transform to $\hat{E}_l$.
 - (c) **Extract:** For every set X where $|X| = l$, compute $\hat{h}_l[X] = (f(X) - \hat{E}_l[X]) \cdot g(\emptyset)^{-1}$.
 - (d) **Zeta Transform:** Apply the Fast Zeta Transform to $\hat{h}_l$.

For SPS inversion, this identical algorithm is used by setting $f(\emptyset) = 1$ and $f(X) = 0$ for all $|X| > 0$.

Algorithm 49 SPS Quotient

```
1: function SPS_QUOTIENT( $f, g, n$ )
2:    $g_{\text{inv}} \leftarrow g[0]^{-1}$ 
3:   Initialize  $\hat{g}, \hat{h}$  as arrays of size  $(n+1) \times 2^n$  filled with 0
4:   for  $S \leftarrow 0$  to  $2^n - 1$  do
5:      $\hat{g}[S][S] \leftarrow g[S]$ 
6:   for  $k \leftarrow 0$  to  $n$  do
7:     FASTZETA_TRANSFORM( $\hat{g}[k], n$ )
8:   for  $l \leftarrow 0$  to  $n$  do
9:     for  $i \leftarrow 1$  to  $l$  do
10:      for  $S \leftarrow 0$  to  $2^n - 1$  do
11:         $\hat{h}[l][S] \leftarrow \hat{h}[l][S] + \hat{h}[l-i][S] \cdot \hat{g}[i][S]$ 
12:      FASTMOBIUS_TRANSFORM( $\hat{h}[l], n$ )
13:      for  $X \leftarrow 0$  to  $2^n - 1$  do
14:        if  $|X| == l$  then
15:           $\hat{h}[l][X] \leftarrow (f[X] - \hat{h}[l][X]) \cdot g_{\text{inv}}$ 
16:        else
17:           $\hat{h}[l][X] \leftarrow 0$   $\triangleright$  Clear side-effects from the Möbius transform
18:      FASTZETA_TRANSFORM( $\hat{h}[l], n$ )
19:   Initialize  $h$  as an array of size  $2^n$ 
20:   for  $S \leftarrow 0$  to  $2^n - 1$  do
21:      $h[S] \leftarrow \hat{h}[S][S]$ 
22:   return  $h$ 
```

14.4 Exponential and Logarithm

Given a subset power series $f : 2^V \rightarrow \mathbb{F}$, we can compute its exponential $h = \exp(f)$ and logarithm $h = \log(f)$ in $O(n^2 2^n)$ time. For the exponential to exist, we require $f(\emptyset) = 0$. For the logarithm to exist, we require $f(\emptyset) = 1$.

The coefficient of the partial derivative $\frac{\partial}{\partial x_i}$ on a subset S (where $i \notin S$) yields the coefficient of $S \cup \{i\}$ in the original series.

By the chain rule, the exponential $h = \exp(f)$ satisfies the differential equation:

$$\frac{\partial h}{\partial x_i} = h * \frac{\partial f}{\partial x_i}$$

We build the series h iteratively, adding elements from 0 to $n-1$. Let $h_{<i}$ denote the series restricted to subsets drawn from $\{0, \dots, i-1\}$. When we introduce the i -th element, we want to compute the slice of h for subsets where i is the maximum element. We denote this slice as $h_{\text{slice}(i)}$.

By restricting the equation above to the domain of the first i elements, we have:

$$h_{\text{slice}(i)} = h_{<i} * f_{\text{slice}(i)}$$

This is a subset convolution of size 2^i . Since $f(\emptyset) = 0$, the i -th half of the output is determined by the i -th half of f and the already-computed lower half of h .

Similarly, for the logarithm $h = \log(f)$, the chain rule dictates:

$$\frac{\partial h}{\partial x_i} = \frac{\partial f}{\partial x_i} * f^{-1} \implies \frac{\partial f}{\partial x_i} = \frac{\partial h}{\partial x_i} * f$$

Restricting this to the new element i , we obtain:

$$f_{\text{slice}(i)} = h_{\text{slice}(i)} * f_{<i}$$

This means the new slice of the logarithm can be directly computed using the SPS quotient:

$$h_{\text{slice}(i)} = f_{\text{slice}(i)} / f_{<i}$$

Algorithm The algorithm iteratively computes the exact slices of size 2^i for $i = 0$ to $n - 1$.

- **Exp:** We initialize $h[\emptyset] = 1$. At step i , we extract the previously computed half $h[0 \dots 2^i - 1]$ and multiply it by $f[2^i \dots 2^{i+1} - 1]$ using a Subset Convolution of size 2^i . We write the output into $h[2^i \dots 2^{i+1} - 1]$.
- **Log:** We initialize $h[\emptyset] = 0$. At step i , we compute the Subset Quotient of $f[2^i \dots 2^{i+1} - 1]$ divided by $f[0 \dots 2^i - 1]$ and write the output into $h[2^i \dots 2^{i+1} - 1]$.

Algorithm 50 SPS Exponential and Logarithm

```

1: function SPS_EXP( $f, n$ )
2:   if  $f[0] \neq 0$  then
3:     return Error
4:   Initialize  $h$  as an array of size  $2^n$  filled with 0
5:    $h[0] \leftarrow 1$ 
6:   for  $i \leftarrow 0$  to  $n - 1$  do
7:      $h[2^i \dots 2^{i+1} - 1] \leftarrow h[0 \dots 2^i - 1] * f[2^i \dots 2^{i+1} - 1]$ 
8:   return  $h$ 
9: function SPS_LOG( $f, n$ )
10:  if  $f[0] == 0$  then
11:    return Error
12:  Initialize  $h$  as an array of size  $2^n$  filled with 0
13:  for  $i \leftarrow 0$  to  $n - 1$  do
14:     $h[2^i \dots 2^{i+1} - 1] \leftarrow f[2^i \dots 2^{i+1} - 1] * f[0 \dots 2^i - 1]^{-1}$ 
15:  return  $h$ 

```

Complexity For both operations, at the i -th iteration, we perform a subset convolution (or quotient) on arrays of length 2^i , which requires exactly $O(i^2 2^i)$ time. The total complexity is the sum:

$$T(n) = \sum_{i=0}^{n-1} O(i^2 2^i) = O(n^2 2^n)$$

14.5 Composition

Given a formal power series $F(x) = \sum_{k=0}^{m-1} f_k x^k$ and a subset power series $G : 2^V \rightarrow \mathbb{F}$ over n variables, their composition $H(S) = F(G(S))$ can be computed in $O(n^2 2^n)$ time.

1. Handling $G(\emptyset) \neq 0$ Subset composition requires $G(\emptyset) = 0$ to converge. We first apply the Inverse Borel transform to F , which maps $f_i \rightarrow i! f_i = F^{(i)}(0)$.

If the constant term is non-zero (let $a = G(\emptyset)$), we must shift the power series. We have $F^{(i)}(a) = \sum_{j \geq i} F^{(j)}(0) \frac{a^{j-i}}{(j-i)!}$. This shift is computed as a semi-correlation of the transformed coefficients with $\exp(ax)$. Once shifted, we set $G(\emptyset) = 0$.

2. Taylor Expansion We evaluate $F(G)$ by introducing the elements of the set one by one. Let $G_{<k}$ denote the subset power series restricted to the first k elements, and let $G_{\text{slice}(k)}$ be the slice of G where the k -th element is the strict maximum (with x_k factored out). We can iteratively decompose G :

$$G_{\leq k} = G_{<k} + x_k G_{\text{slice}(k)}$$

We use the Taylor expansion of F evaluated at $A + B$, where $A = G_{<k}$ and $B = x_k G_{\text{slice}(k)}$:

$$F(A + B) = F(A) + B * F'(A) + \frac{B * B}{2!} * F''(A) + \dots$$

In subset convolution, elements must be disjoint to yield a non-zero product and every term in B contains the element x_k , thus we only need to consider the first derivative:

$$F(G_{\leq k}) = F(G_{<k}) + x_k G_{\text{slice}(k)} * F'(G_{<k})$$

Notice that computing $F(G_{\leq k})$ requires $F'(G_{<k})$. Consequently, computing $F'(G_{<k})$ will require $F''(G_{<k-1})$, etc. The transition for evaluating the i -th derivative is:

$$F^{(i)}(G_{\leq k}) = F^{(i)}(G_{<k}) + x_k G_{\text{slice}(k)} * F^{(i+1)}(G_{<k})$$

3. DP Optimization We maintain a single working array C of length 2^n . We iterate the derivative order i downwards from n to 0. For a fixed i , our goal is to transform the state of C from representing $F^{(i+1)}(G)$ into $F^{(i)}(G)$.

We iterate the introduced variable index j downwards. At the start of step j , the lower prefix $C[0 \dots 2^j - 1]$ holds the evaluation of $F^{(i+1)}(G_{<j})$. We compute its subset convolution with $G_{\text{slice}(j)}$ and write the result into the upper half $C[2^j \dots 2^{j+1} - 1]$.

Because we iterate j downwards, we compute the upper halves first, preventing the overwriting of the smaller prefixes. We then set the base case (the empty set evaluated at the i -th derivative) by assigning $C[\emptyset] = f_i$. This is correct as our coefficients were mapped to $F^{(i)}(a)$.

Algorithm 51 SPS Composition

```

1: function SPS_COMPOSE( $F, G, n, m$ )
2:    $F \leftarrow \text{INVERSEBOREL}(F)$ 
3:    $a \leftarrow G[0]$ 
4:   if  $a \neq 0$  then
5:      $F \leftarrow \text{SEMICORR}(F, \exp(ax))$ 
6:      $G[0] \leftarrow 0$ 
7:   Initialize  $C$  as an array of size  $2^n$  filled with 0
8:   for  $i \leftarrow n$  to 0 step -1 do
9:     for  $j \leftarrow n - i - 1$  to 0 step -1 do
10:       $X \leftarrow C[0 \dots 2^j - 1]$ 
11:       $G_{\text{slice}(j)} \leftarrow G[2^j \dots 2^{j+1} - 1]$ 
12:       $C[2^j \dots 2^{j+1} - 1] \leftarrow X * G_{\text{slice}(j)}$ 
13:      if  $i < m$  then
14:         $C[0] \leftarrow F[i]$ 
15:      else
16:         $C[0] \leftarrow 0$ 
17:   return  $C$ 

```

Complexity The Inverse Borel Transform takes $O(m)$ time, and the Taylor shift takes $O(m \log m)$ time. The outer loop executes $O(n)$ times, and the inner loop performs a subset convolution on slices of size 2^j , requiring $O(j^2 2^j)$ operations. Summing this over i and j gives $O(n^2 2^n)$. Thus, the complexity is $O(n^2 2^n + m \log m)$.

14.6 Power Projection

Given a subset power series $G : 2^V \rightarrow \mathbb{F}$ over n variables and a weight function $W : 2^V \rightarrow \mathbb{F}$, the Power Projection problem asks us to compute the sequence of scalars c_k for $k = 0 \dots m-1$, defined as the inner product of W with the convolution powers of G :

$$c_k = \langle W, G^k \rangle = \sum_{S \subseteq V} W(S) [x^S] G^k$$

This is the transpose of the subset composition algorithm. Let $L_G(F) = F(G)$ be the operator defined by subset composition. Its adjoint L_G^T maps the vector W to the polynomial $C(x) = \sum c_k x^k$ such that $\langle W, L_G(F) \rangle = \langle L_G^T(W), F \rangle$.

Assume $G(\emptyset) = 0$. In the forward composition DP, the state vector $C \in \mathbb{F}^{2^n}$ is updated iteratively by nesting loops: i downwards from n to 0, and j downwards from $n-i-1$ to 0, we reverse this in the transposed algorithm.

Let X denote the lower half of the state ($0 \dots 2^j - 1$) and Y denote the upper half ($2^j \dots 2^{j+1} - 1$). The forward transition at step j is the map:

$$Y \leftarrow G_{\text{slice}(j)} * X$$

Let X^* and Y^* be the corresponding halves in the transposed algorithm. The adjoint of reading from X to overwrite Y is to project the dual values from Y^* through the transposed operator and accumulate them into X^* , then to clear Y^* .

$$\begin{aligned} X^* &\leftarrow X^* + G_{\text{slice}(j)}^* Y^* \\ Y^* &\leftarrow 0 \end{aligned}$$

Finally, the forward DP puts the input coefficient f_i into $C[\emptyset]$ at the conclusion of the j iterations. The transposed operation extracts this value into the output $c_i = W[\emptyset]$ and clears $W[\emptyset] = 0$ at the start of the transposed iterations.

If the original series had a non-zero constant term $a = G(\emptyset)$, the forward composition shifted $F^{(i)}(0)$ to $F^{(i)}(a)$. The algorithm computes the semicorrelation with $\exp(ax)$, the transpose of which is multiplication with $\exp(ax)$.

In the forward algorithm, we applied the Inverse Borel transform, since this is diagonal, it's self-adjoint.

Algorithm 52 SPS Power Projection

```
1: function SPS_POWERPROJ( $G, W, n, m$ )
2:    $a \leftarrow G[0]$ 
3:    $G[0] \leftarrow 0$ 
4:   Initialize  $C$  as an array of size  $n + 1$  filled with 0
5:   Initialize  $S$  as an intermediate array of size  $2^n$ 
6:   for  $i \leftarrow 0$  to  $n$  do
7:      $C[i] \leftarrow W[0]$ 
8:      $W[0] \leftarrow 0$ 
9:     for  $j \leftarrow 0$  to  $n - i - 1$  do
10:       $X \leftarrow W[0 \dots 2^j - 1]$ 
11:       $Y \leftarrow W[2^j \dots 2^{j+1} - 1]$ 
12:       $G_{\text{slice}(j)} \leftarrow G[2^j \dots 2^{j+1} - 1]$ 
13:       $S \leftarrow Y *^T G_{\text{slice}(j)}$ 
14:       $X \leftarrow X + S$ 
15:     $Y \leftarrow 0$ 
16:   if  $a \neq 0$  then
17:      $C \leftarrow C \cdot \exp(ax) \pmod{x^m}$ 
18:    $C \leftarrow \text{INVERSEBOREL}(C)$ 
19: return  $C$ 
```

14.7 Power

Given a subset power series $f : 2^V \rightarrow \mathbb{F}$ over n variables, computing the k -th power $h = f^k$ can be done in $O(n^2 2^n)$ time.

Case 1: $f(\emptyset) \neq 0$ If the constant term $a_0 = f(\emptyset)$ is non-zero, we can compute the power using the exponential and logarithm.

$$f^k = a_0^k \exp\left(k \log\left(\frac{f}{a_0}\right)\right)$$

Case 2: $f(\emptyset) = 0$ If the constant term is zero, every non-zero term in the series corresponds to a subset of size ≥ 1 . Consequently, convolving the series with itself k times means every term in the result must have a subset size of at least k . Thus, if $k > n$, f^k is identically zero.

If $k \leq n$, we compute the power as a composition: $h(S) = P(f(S))$, where $P(x) = x^k$.

We can specialize our SPS Composition algorithm. We have:

$$P^{(i)}(0) = \begin{cases} k! & \text{if } i = k \\ 0 & \text{if } i < k \end{cases}$$

We C with $C[\emptyset] = k!$, which represents $P^{(k)}(f)$. We then run the DP downwards from $i = k - 1$ to 0. At the end of each derivative step, we set the base case $C[\emptyset] = P^{(i)}(0) = 0$.

Algorithm 53 SPS Power

```
1: function SPS_POWER( $f, k, n$ )
2:    $a_0 \leftarrow f[0]$ 
3:   if  $a_0 \neq 0$  then
4:      $a_0^{\text{inv}} \leftarrow (a_0)^{-1}$ 
5:      $f_{\text{norm}} \leftarrow f \cdot a_0^{\text{inv}}$ 
6:      $L \leftarrow \text{SPS\_LOG}(f_{\text{norm}}, n)$ 
7:      $L \leftarrow L \cdot k$ 
8:      $E \leftarrow \text{SPS\_EXP}(L, n)$ 
9:     return  $E \cdot (a_0^k)$ 
10:  else
11:    if  $k > n$  then
12:      return Array of 0s
13:    Initialize  $C$  as an array of size  $2^n$  filled with 0
14:     $C[0] \leftarrow k!$ 
15:    for  $i \leftarrow k - 1$  to 0 step  $-1$  do
16:      for  $j \leftarrow n - i - 1$  to 0 step  $-1$  do
17:         $X \leftarrow C[0 \dots 2^j - 1]$ 
18:         $f_{\text{slice}(j)} \leftarrow f[2^j \dots 2^{j+1} - 1]$ 
19:         $C[2^j \dots 2^{j+1} - 1] \leftarrow \text{SUBSETCONVOLUTION}(X, f_{\text{slice}(j)})$ 
20:       $C[0] \leftarrow 0$ 
21:  return  $C$ 
```

Complexity If $f(\emptyset) \neq 0$, we perform one subset logarithm and one subset exponential, taking $O(n^2 2^n)$ time. If $f(\emptyset) = 0$, the DP matches the complexity of composition, and thus is also $O(n^2 2^n)$ time.

15 Matrix Analysis

15.1 Sampling Matrices without Replacement

We can show that sampling without replacement is more concentrated than sampling with replacement, this works for both scalars and matrices.

Lemma 267 (Hoeffding's Coupling). *Let $\mathcal{C}$ be a finite set of Hermitian matrices. Let $X = (X_1, \dots, X_m)$ be a sequence of m matrices sampled uniformly with replacement from $\mathcal{C}$. Let $Y = (Y_1, \dots, Y_m)$ be a sequence sampled uniformly without replacement from $\mathcal{C}$.*

There exists a randomized surrogate function $Z(Y) = (Z_1, \dots, Z_m)$ such that:

1. *The unconditional distribution of $Z(Y)$ is identical to X .*
2. *The conditional expectation satisfies $\mathbb{E}_{Z|Y}[\sum_{i=1}^m Z_i(Y)] = \sum_{i=1}^m Y_i$.*

Proof. We construct the sequence $Z_1, \dots, Z_m$ conditionally from the drawn set Y via the following sequential recipe.

For the k -th step, let D_k be the subset of matrices from Y that have already been chosen in the sequence $Z_1, \dots, Z_{k-1}$. We draw Z_k as follows:

- With probability $\frac{|D_k|}{|\mathcal{C}|}$, we sample Z_k uniformly from the already-drawn elements D_k .
- With probability $1 - \frac{|D_k|}{|\mathcal{C}|}$, we sample Z_k uniformly from the remaining un-drawn elements $Y \setminus D_k$.

It is straightforward to verify that the joint probability of drawing any sequence $x \in \mathcal{C}^m$ is $|\mathcal{C}|^{-m}$.

For the second property, observe that the conditional expectation $\mathbb{E}_{Z|Y}[\sum_{i=1}^m Z_i]$ is a linear combination of the variables $Y_1, \dots, Y_m$ and invariant under a permutation of the indices of Y .

It thus must take the form $K \sum_{i=1}^m Y_i$ for some scalar constant K . To determine K , we take the expectation over Y :

$$\mathbb{E}_Y \left[\mathbb{E}_{Z|Y} \left[\sum_{i=1}^m Z_i \right] \right] = m\mathbb{E}[Y_1] \quad \text{and} \quad \mathbb{E}_Y \left[K \sum_{i=1}^m Y_i \right] = Km\mathbb{E}[Y_1]$$

So $K = 1$. □

15.2 Schur and Kronecker Product

Lemma 268 (Kronecker Product Preserves Positivity). *Let A and B be Hermitian matrices. If $A, B \succeq 0$, then $A \otimes B \succeq 0$. Furthermore, if $A, B \succ 0$, then $A \otimes B \succ 0$.*

Proof. Assume $A \succeq 0$ and $B \succeq 0$. Every PSD matrix has a unique PSD square root. We can factor the Kronecker product:

$$A \otimes B = (A^{1/2} A^{1/2}) \otimes (B^{1/2} B^{1/2}) = (A^{1/2} \otimes B^{1/2})(A^{1/2} \otimes B^{1/2})$$

Because the Kronecker product of two Hermitian matrices is Hermitian, $A^{1/2} \otimes B^{1/2}$ is Hermitian. Therefore, we have expressed $A \otimes B$ as the square of a Hermitian matrix, which implies $A \otimes B \succeq 0$.

Now assume $A \succ 0$ and $B \succ 0$. In this case, both A and B are invertible. The Kronecker product of invertible matrices is invertible, with its inverse given by $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$. □

The Schur product theorem has an especially transparent interpretation in terms of Gram matrices. A positive semidefinite matrix is a table of inner products; entrywise multiplication simply tensors the vectors producing those inner products.

Lemma 269 (Schur Product Theorem). *Let $A, B \in M_n(\mathbb{C})$ be Hermitian. If $A, B \succeq 0$, then*

$$A \circ B \succeq 0.$$

If $A, B \succ 0$, then $A \circ B \succ 0$.

Proof. Because $A \succeq 0$ and $B \succeq 0$, there are vectors $u_1, \dots, u_n$ and $v_1, \dots, v_n$ such that

$$A_{ij} = \langle u_i, u_j \rangle, \quad B_{ij} = \langle v_i, v_j \rangle.$$

Therefore

$$(A \circ B)_{ij} = \langle u_i, u_j \rangle \langle v_i, v_j \rangle = \langle u_i \otimes v_i, u_j \otimes v_j \rangle.$$

Thus $A \circ B$ is the Gram matrix of the vectors

$$u_1 \otimes v_1, \dots, u_n \otimes v_n,$$

and every Gram matrix is positive semidefinite.

Now suppose $A, B \succ 0$. Then the Gram vectors $u_1, \dots, u_n$ are linearly independent, and every v_i is nonzero. If

$$\sum_i c_i u_i \otimes v_i = 0,$$

choose, for each j , a linear functional φ_j on $\text{span}\{u_i\}$ satisfying $\varphi_j(u_i) = \delta_{ij}$. Applying $\varphi_j \otimes I$ gives $c_j v_j = 0$, hence $c_j = 0$. So the vectors $u_i \otimes v_i$ are linearly independent, and their Gram matrix $A \circ B$ is positive definite. □

15.3 Differentiating Matrix Functions

For scalar functions, one writes

$$\frac{d}{dt}f(x(t)) = f'(x(t))x'(t).$$

The analogous formula is generally false for matrices because X and X' need not commute. There are two simple replacements.

For algebraic expressions, differentiate by the ordinary product rule while preserving the order of the factors. For functions defined by spectral calculus, scalar derivatives are replaced by *divided differences*. Inside a trace, only the diagonal divided differences survive, and the usual scalar derivative rule reappears.

For a differentiable matrix-valued map F , write

$$DF(X)[H] = \left. \frac{d}{dt}F(X + tH) \right|_{t=0}.$$

Thus for a differentiable path $X(t)$,

$$\frac{d}{dt}F(X(t)) = DF(X(t))[X'(t)].$$

15.3.1 Product, Inverse, and Power Rules

The ordinary product rule remains valid because multiplication is bilinear; the only care is that the order of the factors cannot be changed.

Lemma 270 (Basic matrix differential rules). *Let $A(t), B(t)$ be differentiable matrix-valued functions.*

Then

$$\boxed{\frac{d}{dt}(AB) = A'B + AB'.$$

If $A(t)$ is invertible, then

$$\boxed{\frac{d}{dt}A^{-1} = -A^{-1}A'A^{-1}.$$

For every integer $k \geq 1$,

$$\boxed{\frac{d}{dt}A^k = \sum_{r=0}^{k-1} A^r A' A^{k-1-r}.$$

Proof. The product rule follows by inserting and subtracting $A(t+h)B(t)$:

$$\begin{aligned} A(t+h)B(t+h) - A(t)B(t) &= (A(t+h) - A(t))B(t) \\ &\quad + A(t+h)(B(t+h) - B(t)). \end{aligned}$$

Divide by h and let $h \rightarrow 0$.

For the inverse, differentiate

$$AA^{-1} = I.$$

The product rule gives

$$A'A^{-1} + A(A^{-1})' = 0.$$

Multiplying on the left by A^{-1} yields

$$(A^{-1})' = -A^{-1}A'A^{-1}.$$

Finally, repeatedly applying the product rule to $A^k = A \cdots A$ gives one term for each factor at which the derivative may land:

$$(A^k)' = A'A^{k-1} + AA'A^{k-2} + \cdots + A^{k-1}A'.$$

□

When A commutes with A' , all k terms are equal and this reduces to the formula

$$(A^k)' = kA^{k-1}A'.$$

15.3.2 Divided Differences

Let $f : I \rightarrow \mathbb{R}$ be differentiable. Its first divided difference is

$$f^{[1]}(a, b) = \begin{cases} \frac{f(a) - f(b)}{a - b}, & a \neq b, \\ f'(a), & a = b. \end{cases}$$

By the fundamental theorem of calculus,

$$f^{[1]}(a, b) = \int_0^1 f'((1-s)b + sa) ds.$$

Thus a divided difference is simply the average scalar derivative between the two eigenvalues.

For a Hermitian matrix

$$X = U \operatorname{diag}(\lambda_1, \dots, \lambda_n) U^*,$$

recall that spectral calculus defines

$$f(X) = U \operatorname{diag}(f(\lambda_1), \dots, f(\lambda_n)) U^*.$$

Theorem 271 (Derivative of a spectral matrix function). *Let X be Hermitian with spectrum in the interior of an interval I , and let $f \in C^1(I)$. For a Hermitian direction H , put*

$$\tilde{H} = U^* H U.$$

Then

$$U^* Df(X)[H]U = (f^{[1]}(\lambda_i, \lambda_j) \tilde{H}_{ij})_{i,j}.$$

Equivalently,

$$Df(X)[H] = U \left((f^{[1]}(\lambda_i, \lambda_j))_{i,j} \circ \tilde{H} \right) U^*.$$

Proof. First take $f(x) = x^k$. By [Lemma 270](#),

$$D(X^k)[H] = \sum_{r=0}^{k-1} X^r H X^{k-1-r}.$$

In an eigenbasis of X , its ij -entry is

$$\tilde{H}_{ij} \sum_{r=0}^{k-1} \lambda_i^r \lambda_j^{k-1-r}.$$

If $\lambda_i \neq \lambda_j$, the geometric-sum identity gives

$$\sum_{r=0}^{k-1} \lambda_i^r \lambda_j^{k-1-r} = \frac{\lambda_i^k - \lambda_j^k}{\lambda_i - \lambda_j}.$$

If $\lambda_i = \lambda_j$, the sum is $k\lambda_i^{k-1}$. Thus in either case the coefficient is $f^{[1]}(\lambda_i, \lambda_j)$. Linearity proves the formula for polynomials.

For a general C^1 function, let $\theta_1, \dots, \theta_r$ be the distinct eigenvalues of X . Choose a Hermite interpolation polynomial p satisfying

$$p(\theta_a) = f(\theta_a), \quad p'(\theta_a) = f'(\theta_a) \quad (a = 1, \dots, r).$$

Put $g = f - p$. Then

$$g(\theta_a) = g'(\theta_a) = 0,$$

so uniformly over the finitely many eigenvalues,

$$g(\theta_a + s) = o(|s|) \quad (s \rightarrow 0).$$

If E is Hermitian and small, every eigenvalue of $X + E$ lies within $\|E\|$ of an eigenvalue of X . Indeed,

$$-\|E\|I \preceq E \preceq \|E\|I$$

gives

$$X - \|E\|I \preceq X + E \preceq X + \|E\|I,$$

and hence the same bound on the ordered eigenvalues. Therefore

$$\|g(X + E)\| = \max_{\mu \in \sigma(X + E)} |g(\mu)| = o(\|E\|).$$

Since $g(X) = 0$,

$$f(X + E) - f(X) = p(X + E) - p(X) + o(\|E\|).$$

Thus f and p have the same Fréchet derivative at X .

Finally, p agrees with f at every eigenvalue of X , and their derivatives agree there as well, so

$$p^{[1]}(\lambda_i, \lambda_j) = f^{[1]}(\lambda_i, \lambda_j).$$

The polynomial formula therefore gives the stated result. $\square$

Thus we use the ordinary derivative on one eigenspace and divided difference between two eigenspaces. In particular, if X and H commute, they can be simultaneously diagonalized and

$$Df(X)[H] = f'(X)H.$$

15.3.3 Traces Recover the Scalar Derivative Rule

Although $Df(X)[H]$ is not generally $f'(X)H$, its trace behaves as though the matrices commuted.

Corollary 272 (Derivative of a trace spectral function). *Let*

$$F(X) = \text{Tr } f(X).$$

Then

$$\boxed{DF(X)[H] = \text{Tr}(f'(X)H).}$$

With the trace inner product, therefore,

$$\boxed{\nabla F(X) = f'(X).}$$

Proof. Work in an eigenbasis of X . Trace keeps only the diagonal entries of $Df(X)[H]$, and on the diagonal

$$f^{[1]}(\lambda_i, \lambda_i) = f'(\lambda_i).$$

Hence

$$\begin{aligned} DF(X)[H] &= \sum_i f'(\lambda_i) \tilde{H}_{ii} \\ &= \text{Tr}(f'(X)H). \end{aligned}$$

$\square$

There is also a second-order version.

Corollary 273 (Hessian of a trace spectral function). *Let $f \in C^2(I)$, and let*

$$F(X) = \text{Tr } f(X).$$

In an eigenbasis of $X = \text{diag}(\lambda_1, \dots, \lambda_n)$,

$$D^2 F(X)[H, H] = \sum_{i,j} (f')^{[1]}(\lambda_i, \lambda_j) |H_{ij}|^2.$$

Proof. By Corollary 272,

$$\nabla F(X) = f'(X).$$

Therefore

$$D^2 F(X)[H, H] = \text{Tr}(H D(f')(X)[H]).$$

Apply Theorem 271 to f' :

$$(D(f')(X)[H])_{ij} = (f')^{[1]}(\lambda_i, \lambda_j) H_{ij}.$$

Since H is Hermitian, $H_{ji} = \overline{H_{ij}}$, and hence

$$\begin{aligned} \text{Tr}(H D(f')(X)[H]) &= \sum_{i,j} H_{ji} (f')^{[1]}(\lambda_i, \lambda_j) H_{ij} \\ &= \sum_{i,j} (f')^{[1]}(\lambda_i, \lambda_j) |H_{ij}|^2. \end{aligned}$$

□

Examples. For $f(x) = x^{-1}$,

$$f^{[1]}(a, b) = -\frac{1}{ab},$$

including the limiting value $-1/a^2$ when $a = b$. Thus

$$D(X^{-1})[H] = -X^{-1} H X^{-1},$$

recovering the inverse rule.

For matrix entropy,

$$h(X) = \text{Tr}(X \log X),$$

take $f(x) = x \log x$. Since

$$f'(x) = 1 + \log x,$$

Corollary 272 gives immediately

$$\nabla h(X) = I + \log X.$$

The Hessian is

$$D^2 h(X)[H, H] = \sum_{i,j} (\log)^{[1]}(\lambda_i, \lambda_j) |H_{ij}|^2.$$

Likewise,

$$\log \det X = \text{Tr} \log X,$$

so

$$D(\log \det X)[H] = \text{Tr}(X^{-1} H).$$

Exponentiating gives Jacobi's formula

$$D(\det X)[H] = \det(X) \operatorname{Tr}(X^{-1}H).$$

Finally, when the *exponent* rather than the matrix varies, no noncommutativity issue arises. For $X \succ 0$,

$$X^s = U \operatorname{diag}(\lambda_i^s) U^*,$$

so differentiating the scalar eigenvalues gives

$$\frac{d}{ds} X^s = X^s \log X = (\log X) X^s.$$

In particular,

$$\left. \frac{d}{ds} X^s \right|_{s=0} = \log X, \quad \left. \frac{d}{ds} X^{1-s} \right|_{s=0} = -X \log X.$$

15.3.4 The Matrix Exponential

There is also another useful formula for the derivative of the matrix exponential.

Proposition 274 (Derivative of the matrix exponential). *If $X(t)$ is differentiable, then*

$$\frac{d}{dt} e^{X(t)} = \int_0^1 e^{\alpha X(t)} X'(t) e^{(1-\alpha)X(t)} d\alpha.$$

Equivalently,

$$De^X[H] = \int_0^1 e^{\alpha X} H e^{(1-\alpha)X} d\alpha.$$

Proof. We first prove the finite-difference identity

$$e^A - e^B = \int_0^1 e^{\alpha A} (A - B) e^{(1-\alpha)B} d\alpha.$$

Define

$$F(\alpha) = e^{\alpha A} e^{(1-\alpha)B}.$$

Since A commutes with $e^{\alpha A}$ and B commutes with $e^{(1-\alpha)B}$,

$$\begin{aligned} F'(\alpha) &= A e^{\alpha A} e^{(1-\alpha)B} - e^{\alpha A} B e^{(1-\alpha)B} \\ &= e^{\alpha A} (A - B) e^{(1-\alpha)B}. \end{aligned}$$

Thus the fundamental theorem of calculus gives

$$\begin{aligned} e^A - e^B &= F(1) - F(0) \\ &= \int_0^1 e^{\alpha A} (A - B) e^{(1-\alpha)B} d\alpha. \end{aligned}$$

Now take

$$A = X(t+h), \quad B = X(t).$$

Then

$$\frac{e^{X(t+h)} - e^{X(t)}}{h} = \int_0^1 e^{\alpha X(t+h)} \frac{X(t+h) - X(t)}{h} e^{(1-\alpha)X(t)} d\alpha.$$

As $h \rightarrow 0$,

$$X(t+h) \rightarrow X(t), \quad \frac{X(t+h) - X(t)}{h} \rightarrow X'(t),$$

so continuity of the matrix exponential lets us pass the limit through the integral. Hence

$$\frac{d}{dt}e^{X(t)} = \int_0^1 e^{\alpha X(t)} X'(t) e^{(1-\alpha)X(t)} d\alpha.$$

Taking $X(t) = X + tH$ gives the Fréchet derivative formula. □

If X commutes with H , the integrand is independent of α and we recover the scalar formula.

15.4 Integral Representations and Fractional Powers

15.4.1 The Logarithm

The logarithm has a simpler resolvent representation than the fractional powers, so it is best proved directly.

Lemma 275 (Resolvent representation of the logarithm). *For every $A \succ 0$,*

$$\log A = \int_0^\infty \left(\frac{1}{1+t} I - (A+tI)^{-1} \right) dt.$$

The integral converges in operator norm.

Proof. Start with the scalar identity. For $x > 0$ and $R > 0$,

$$\begin{aligned} \int_0^R \left(\frac{1}{1+t} - \frac{1}{x+t} \right) dt &= [\log(1+t) - \log(x+t)]_0^R \\ &= \log(1+R) - \log(x+R) + \log x \\ &= \log x + \log \frac{1+R}{x+R}. \end{aligned}$$

As $R \rightarrow \infty$,

$$\frac{1+R}{x+R} \rightarrow 1,$$

so

$$\log x = \int_0^\infty \left(\frac{1}{1+t} - \frac{1}{x+t} \right) dt.$$

Now diagonalize

$$A = U \operatorname{diag}(\lambda_1, \dots, \lambda_n) U^*.$$

Then

$$(A+tI)^{-1} = U \operatorname{diag} \left(\frac{1}{\lambda_1+t}, \dots, \frac{1}{\lambda_n+t} \right) U^*.$$

Applying the scalar identity to each eigenvalue gives

$$\begin{aligned} \int_0^\infty \left(\frac{1}{1+t} I - (A+tI)^{-1} \right) dt &= U \operatorname{diag}(\log \lambda_1, \dots, \log \lambda_n) U^* \\ &= \log A. \end{aligned}$$

Since there are finitely many positive eigenvalues and the scalar integrand is $O(t^{-2})$ as $t \rightarrow \infty$, the integral converges in operator norm. □

Differentiating this representation gives another useful form of the Fréchet derivative. By the inverse rule,

$$D((X + tI)^{-1})[H] = -(X + tI)^{-1}H(X + tI)^{-1}.$$

Therefore

$$D \log(X)[H] = \int_0^\infty (X + tI)^{-1}H(X + tI)^{-1} dt.$$

Differentiation under the integral is justified locally uniformly for $X \succ 0$, since the differentiated integrand is $O((1 + t)^{-2})$.

15.4.2 Fractional Powers

Many matrix inequalities for fractional powers come from one scalar formula. For $0 < \alpha < 1$, let

$$c_\alpha = \frac{\sin(\pi\alpha)}{\pi}.$$

For $0 < \alpha < 1$, set

$$I_\alpha = \int_0^\infty \frac{x^{\alpha-1}}{1+x} dx.$$

Consider

$$f(z) = \frac{z^{\alpha-1}}{1+z},$$

using the branch

$$0 < \arg z < 2\pi,$$

so the branch cut is the positive real axis. Integrate around a keyhole contour enclosing the slit plane.

On the upper side of the positive real axis,

$$z^{\alpha-1} = x^{\alpha-1},$$

so the contribution tends to I_α .

On the lower side, $\arg z = 2\pi$, and the contour runs from ∞ back to 0. Hence

$$z^{\alpha-1} = e^{2\pi i(\alpha-1)}x^{\alpha-1} = e^{2\pi i\alpha}x^{\alpha-1},$$

and its contribution tends to

$$-e^{2\pi i\alpha}I_\alpha.$$

The two circular arcs vanish: on the outer circle their contribution is

$$O(R^{\alpha-1}) \rightarrow 0,$$

while on the inner circle it is

$$O(\varepsilon^\alpha) \rightarrow 0.$$

The only pole inside the contour is $z = -1$. Since $\arg(-1) = \pi$,

$$\text{Res}_{z=-1} f(z) = (-1)^{\alpha-1} = e^{i\pi(\alpha-1)} = -e^{i\pi\alpha}.$$

Therefore the residue theorem gives

$$(1 - e^{2\pi i\alpha})I_\alpha = -2\pi i e^{i\pi\alpha}.$$

Using

$$1 - e^{2\pi i\alpha} = -2ie^{i\pi\alpha} \sin(\pi\alpha),$$

we obtain

$$I_\alpha = \frac{\pi}{\sin(\pi\alpha)}.$$

Consequently, after the substitution $t = xs$,

$$\int_0^\infty t^{\alpha-1} \frac{x}{t+x} dt = x^\alpha \int_0^\infty \frac{s^{\alpha-1}}{1+s} ds = x^\alpha \frac{\pi}{\sin(\pi\alpha)}.$$

Thus

$$x^\alpha = \frac{\sin(\pi\alpha)}{\pi} \int_0^\infty t^{\alpha-1} \frac{x}{t+x} dt.$$

Applying the scalar identity to every eigenvalue gives the matrix formula.

Lemma 276 (Formula for Fractional Powers). *For $A \succ 0$ and $0 < \alpha < 1$,*

$$A^\alpha = c_\alpha \int_0^\infty t^{\alpha-1} A(tI + A)^{-1} dt.$$

Equivalently,

$$A^\alpha = c_\alpha \int_0^\infty t^\alpha \left(\frac{1}{t} I - (tI + A)^{-1} \right) dt.$$

Two other useful forms follow immediately. If $-1 < p < 0$, then

$$A^p = \frac{\sin(-\pi p)}{\pi} \int_0^\infty t^p (tI + A)^{-1} dt.$$

If $1 < p < 2$, applying the preceding formula to A^{p-1} and multiplying by A gives

$$A^p = c_{p-1} \int_0^\infty t^{p-2} A^2 (tI + A)^{-1} dt.$$

Using

$$A^2(tI + A)^{-1} = A - tI + t^2(tI + A)^{-1}$$

recovers the more expanded resolvent formula when that form is convenient.

We also have the scalar identity

$$|x|^\alpha = c'_\alpha \int_0^\infty \frac{1 - \cos(tx)}{t^{\alpha+1}} dt, \quad 0 < \alpha < 2,$$

for a positive constant c'_α .

15.4.3 Operator Convexity of Inversion

The formulas are useful because inversion has a very simple matrix-convexity property.

Lemma 277 (Operator Convexity of Inversion). *For $A, B \succ 0$ and $0 \leq \lambda \leq 1$,*

$$(\lambda A + (1 - \lambda)B)^{-1} \preceq \lambda A^{-1} + (1 - \lambda)B^{-1}.$$

Proof. For every $A \succ 0$,

$$\begin{pmatrix} A^{-1} & I \\ I & A \end{pmatrix} = \begin{pmatrix} A^{-1/2} & \\ & A^{1/2} \end{pmatrix} \begin{pmatrix} A^{-1/2} & A^{1/2} \end{pmatrix} \succeq 0.$$

Average this matrix with its analogue for B :

$$\begin{pmatrix} \lambda A^{-1} + (1-\lambda)B^{-1} & I \\ I & \lambda A + (1-\lambda)B \end{pmatrix} \succeq 0.$$

Taking the Schur complement of the lower-right block gives exactly the claimed inequality. $\square$

We also repeatedly use that inversion reverses the Loewner order:

$$0 \prec A \preceq B \implies B^{-1} \preceq A^{-1}.$$

Indeed,

$$I \preceq A^{-1/2} B A^{-1/2}.$$

Inverting reverses the inequality,

$$A^{1/2} B^{-1} A^{1/2} \preceq I,$$

and conjugating by $A^{-1/2}$ gives $B^{-1} \preceq A^{-1}$.

15.4.4 Löwner–Heinz and Operator Concavity of Powers

The same formula now gives two fundamental facts about fractional powers essentially for free.

Theorem 278 (Löwner–Heinz). *If*

$$0 \preceq A \preceq B$$

and $0 \leq \alpha \leq 1$, *then*

$$\boxed{A^\alpha \preceq B^\alpha}.$$

Thus $x \mapsto x^\alpha$ *is operator monotone on the positive cone.*

Proof. It is enough to consider $0 < \alpha < 1$ and positive definite A, B ; the semidefinite case follows by replacing A, B with $A + \varepsilon I, B + \varepsilon I$ and taking $\varepsilon \downarrow 0$.

For every $t > 0$,

$$A \preceq B \implies (tI + A)^{-1} \succeq (tI + B)^{-1}.$$

Therefore

$$A(tI + A)^{-1} = I - t(tI + A)^{-1} \preceq I - t(tI + B)^{-1} = B(tI + B)^{-1}.$$

Integrating this inequality against the positive weight $c_\alpha t^{\alpha-1}$ proves $A^\alpha \preceq B^\alpha$. $\square$

Lemma 279 (Operator Concavity of Fractional Powers). *For* $0 \leq \alpha \leq 1$, *the map*

$$A \mapsto A^\alpha$$

is operator concave on the positive cone.

Proof. Again take $0 < \alpha < 1$. For fixed $t > 0$,

$$A(tI + A)^{-1} = I - t(tI + A)^{-1}.$$

By [Lemma 277](#), the map

$$A \mapsto (tI + A)^{-1}$$

is operator convex. Hence its negative, and therefore $A \mapsto A(tI + A)^{-1}$, is operator concave.

The representation [Lemma 276](#) is a positive weighted integral of these maps, so

$$A \mapsto A^\alpha$$

is operator concave. $\square$

15.5 Golden–Thompson Inequality

The proof below has one basic mechanism: repeated Cauchy–Schwarz makes a word in A and A^* more alternating. The completely alternating word is therefore the extremal one. The Lie–Trotter formula then turns this “word-straightening” statement into Golden–Thompson.

Lemma 280 (Word Straightening). *Let $A \in M_d(\mathbb{C})$, and let*

$$A_1, \dots, A_{2n} \in \{A, A^*\}.$$

Then

$$|\mathrm{Tr}(A_1 \cdots A_{2n})| \leq \mathrm{Tr}((A^*A)^n).$$

Proof. If $A = A^*$ the claim is immediate. Otherwise regard A and A^* as two distinct symbols.

Among all length- $2n$ words in A, A^* , choose one

$$P = A_1 \cdots A_{2n}$$

maximizing $|\mathrm{Tr} P|$, and among all such maximizers choose one with the largest number of transitions between A and A^* in cyclic order.

If every adjacent pair is different, the word alternates, so it is a cyclic shift of either

$$(A^*A)^n \quad \text{or} \quad (AA^*)^n.$$

These have the same trace, and we are done.

Otherwise two adjacent symbols agree. By cyclically rotating the word, we may arrange that the equal pair occurs across the middle. Write

$$P = QR, \quad Q = A_1 \cdots A_n, \quad R = A_{n+1} \cdots A_{2n},$$

with

$$A_n = A_{n+1}.$$

Frobenius Cauchy–Schwarz gives

$$|\mathrm{Tr}(QR)|^2 \leq \mathrm{Tr}(Q^*Q) \mathrm{Tr}(R^*R).$$

Both Q^*Q and R^*R are again length- $2n$ words in A, A^* . Since P was a maximizer, neither trace magnitude can exceed $|\mathrm{Tr} P|$. Hence equality must hold throughout, and both Q^*Q and R^*R are themselves maximizers.

It remains to compare transitions. Let t_Q, t_R be the numbers of transitions internal to Q, R . The chosen middle pair contributes no transition, so P has at most

$$t_Q + t_R + 1$$

cyclic transitions.

In Q^*Q , the transitions internal to Q occur once in each half, and the two junctions are automatically transitions because a letter meets its adjoint. Thus Q^*Q has exactly

$$2t_Q + 2$$

transitions, and similarly R^*R has $2t_R + 2$. Their average is

$$t_Q + t_R + 2,$$

strictly larger than the number of transitions of P . Therefore one of Q^*Q, R^*R is a maximizing word with more transitions than P , a contradiction.

So every maximizing word alternates, proving the claim. $\square$

Lemma 281 (Disentangling). *Let U, V be Hermitian and let $k \geq 1$. Then*

$$\boxed{\begin{aligned} \left| \operatorname{Tr}((UV)^{2^k}) \right| &\leq \operatorname{Tr}((U^2V^2)^{2^{k-1}}) \\ &\leq \operatorname{Tr}((U^4V^4)^{2^{k-2}}) \\ &\leq \cdots \leq \operatorname{Tr}(U^{2^k}V^{2^k}). \end{aligned}}$$

Proof. Apply [Lemma 280](#) to the matrix $A = UV$ and to the word consisting of 2^k copies of A . Since $A^* = VU$,

$$\left| \operatorname{Tr}((UV)^{2^k}) \right| \leq \operatorname{Tr}((VU^2V)^{2^{k-1}}).$$

Cyclicity of trace turns the right-hand side into

$$\operatorname{Tr}((U^2V^2)^{2^{k-1}}).$$

Now repeat with U^2, V^2 , then U^4, V^4 , and so on. □

Theorem 282 (Golden–Thompson). *For Hermitian A, B ,*

$$\boxed{\operatorname{Tr}(e^{A+B}) \leq \operatorname{Tr}(e^A e^B)}.$$

Proof. The Lie–Trotter product formula says

$$e^{A+B} = \lim_{N \rightarrow \infty} \left(e^{A/N} e^{B/N} \right)^N.$$

For completeness, this follows from

$$e^{A/N} e^{B/N} = I + \frac{A+B}{N} + O(N^{-2})$$

in operator norm. For sufficiently large N the matrix on the left lies near I , so its matrix logarithm is defined there and

$$\log(e^{A/N} e^{B/N}) = \frac{A+B}{N} + O(N^{-2}).$$

Thus

$$\left(e^{A/N} e^{B/N} \right)^N = \exp(A+B+O(N^{-1})) \longrightarrow e^{A+B}.$$

Take $N = 2^k$ and set

$$U = e^{A/2^k}, \quad V = e^{B/2^k}.$$

These are positive definite, hence Hermitian. By [Lemma 281](#),

$$\left| \operatorname{Tr} \left((e^{A/2^k} e^{B/2^k})^{2^k} \right) \right| \leq \operatorname{Tr}(e^A e^B).$$

Letting $k \rightarrow \infty$, the matrix inside the trace converges to e^{A+B} . Since $e^{A+B} \succ 0$, its trace is positive, and the limit gives

$$\operatorname{Tr}(e^{A+B}) \leq \operatorname{Tr}(e^A e^B).$$

□

15.6 Trace Rearrangement and Schatten Hölder Inequalities

The matrix analogue of the scalar rearrangement inequality is von Neumann's trace inequality. Once this is established, the two-factor Schatten Hölder inequality is literally scalar Hölder applied to the singular values. Exterior powers then give the corresponding product inequality and hence the general multi-factor trace form.

Lemma 283 (Trace Rearrangement for Hermitian Matrices). *Let X, Y be Hermitian with eigenvalues*

$$\lambda_1(X) \geq \cdots \geq \lambda_n(X), \quad \lambda_1(Y) \geq \cdots \geq \lambda_n(Y).$$

Then

$$\boxed{\operatorname{Tr}(XY) \leq \sum_{i=1}^n \lambda_i(X) \lambda_i(Y).}$$

Proof. Write

$$X = \sum_i \lambda_i(X) u_i u_i^*, \quad Y = \sum_j \lambda_j(Y) v_j v_j^*.$$

Then

$$\operatorname{Tr}(XY) = \sum_{i,j} \lambda_i(X) \lambda_j(Y) |\langle u_i, v_j \rangle|^2.$$

The matrix

$$M_{ij} = |\langle u_i, v_j \rangle|^2$$

is doubly stochastic, because both (u_i) and (v_j) are orthonormal bases.

By the Birkhoff–von Neumann theorem, M is a convex combination of permutation matrices. Therefore $\operatorname{Tr}(XY)$ is a convex combination of quantities

$$\sum_i \lambda_i(X) \lambda_{\pi(i)}(Y).$$

The scalar rearrangement inequality says that each is at most the aligned sum

$$\sum_i \lambda_i(X) \lambda_i(Y).$$

□

Theorem 284 (Von Neumann's Trace Inequality). *For arbitrary $A, B \in M_n(\mathbb{C})$,*

$$\boxed{|\operatorname{Tr}(A^* B)| \leq \sum_{i=1}^n \sigma_i(A) \sigma_i(B),}$$

where the singular values are sorted in decreasing order.

Proof. For a matrix A , form its Hermitian dilation

$$\mathcal{H}(A) = \begin{pmatrix} 0 & A \\ A^* & 0 \end{pmatrix}.$$

If

$$A = U \Sigma V^*$$

is a singular-value decomposition, then the eigenvalues of $\mathcal{H}(A)$ are

$$\sigma_1(A), \dots, \sigma_n(A), -\sigma_n(A), \dots, -\sigma_1(A).$$

Let

$$z = \operatorname{Tr}(A^*B).$$

If $z = 0$ there is nothing to prove. Otherwise multiply B by a complex phase so that z becomes positive real; this does not change the singular values of B . Then

$$\operatorname{Tr}(\mathcal{H}(A)\mathcal{H}(B)) = 2 \operatorname{Re} \operatorname{Tr}(A^*B) = 2|z|.$$

Apply [Lemma 283](#) to the two Hermitian dilations. Pairing their ordered eigenvalues gives

$$2|z| \leq 2 \sum_{i=1}^n \sigma_i(A) \sigma_i(B).$$

Divide by 2. □

Corollary 285 (Schatten Hölder, Trace Form). *If*

$$\frac{1}{p} + \frac{1}{q} = 1, \quad 1 \leq p, q \leq \infty,$$

then

$$|\operatorname{Tr}(A^*B)| \leq \|A\|_{S_p} \|B\|_{S_q}.$$

Proof. By von Neumann's inequality and scalar Hölder,

$$\begin{aligned} |\operatorname{Tr}(A^*B)| &\leq \sum_i \sigma_i(A) \sigma_i(B) \\ &\leq \left(\sum_i \sigma_i(A)^p \right)^{1/p} \left(\sum_i \sigma_i(B)^q \right)^{1/q} \\ &= \|A\|_{S_p} \|B\|_{S_q}. \end{aligned}$$

The endpoint cases have the usual interpretation. □

The same exterior-power idea gives the stronger product form. Recall that $\bigwedge^k A$ acts on $\bigwedge^k \mathbb{C}^n$, satisfies

$$\bigwedge^k (AB) = \left(\bigwedge^k A \right) \left(\bigwedge^k B \right),$$

and has operator norm

$$\left\| \bigwedge^k A \right\| = \prod_{i=1}^k \sigma_i(A).$$

The last identity follows immediately from an SVD: the singular values of $\bigwedge^k A$ are all k -fold products of singular values of A , and the largest such product uses the k largest singular values.

Lemma 286 (Singular Values of a Product). *For every k ,*

$$\prod_{i=1}^k \sigma_i(AB) \leq \prod_{i=1}^k \sigma_i(A) \sigma_i(B).$$

Equivalently,

$$\sigma(AB) \prec_{w, \log} \sigma(A) \sigma(B),$$

where the product on the right is coordinatewise.

Proof. By multiplicativity of exterior powers and submultiplicativity of the operator norm,

$$\begin{aligned}
\prod_{i=1}^k \sigma_i(AB) &= \left\| \bigwedge^k (AB) \right\| \\
&= \left\| \left(\bigwedge^k A \right) \left(\bigwedge^k B \right) \right\| \\
&\leq \left\| \bigwedge^k A \right\| \left\| \bigwedge^k B \right\| \\
&= \prod_{i=1}^k \sigma_i(A) \sigma_i(B).
\end{aligned}$$

□

Theorem 287 (Schatten Hölder, Product Form). *Let*

$$1 \leq p, q, r \leq \infty, \quad \frac{1}{r} = \frac{1}{p} + \frac{1}{q}.$$

Then

$$\|AB\|_{S_r} \leq \|A\|_{S_p} \|B\|_{S_q}.$$

Consequently, if $p_1, \dots, p_m \in [1, \infty]$ satisfy

$$\sum_{j=1}^m \frac{1}{p_j} = 1,$$

then

$$|\operatorname{Tr}(A_1 \cdots A_m)| \leq \prod_{j=1}^m \|A_j\|_{S_{p_j}}.$$

Proof. Assume first that A, B are invertible; the general case follows by regularization and continuity.

By [Lemma 286](#), the logarithms of the singular values of AB are weakly majorized by

$$\log \sigma_i(A) + \log \sigma_i(B).$$

Applying the increasing convex function

$$x \mapsto e^{rx}$$

gives

$$\sum_i \sigma_i(AB)^r \leq \sum_i (\sigma_i(A) \sigma_i(B))^r.$$

Scalar Hölder with conjugate exponents p/r and q/r now gives

$$\sum_i (\sigma_i(A) \sigma_i(B))^r \leq \left(\sum_i \sigma_i(A)^p \right)^{r/p} \left(\sum_i \sigma_i(B)^q \right)^{r/q}.$$

Taking the r -th root proves

$$\|AB\|_{S_r} \leq \|A\|_{S_p} \|B\|_{S_q}.$$

For the multi-factor form, define

$$\frac{1}{r_j} = \sum_{i=1}^j \frac{1}{p_i}.$$

Since the total reciprocal sum is 1, each intermediate $r_j \geq 1$. Repeatedly applying the product inequality gives

$$\|A_1 \cdots A_m\|_{S_1} \leq \prod_{j=1}^m \|A_j\|_{S_{p_j}}.$$

Finally,

$$|\operatorname{Tr} X| \leq \|X\|_{S_1}$$

by [Theorem 284](#) applied to I and X . □

15.7 Generalized Klein Inequality

Lemma 288 (Generalized Klein Inequality). *Let $f_i, g_i : I \rightarrow \mathbb{R}$ be functions on an interval I such that $\sum_i f_i(x)g_i(y) \geq 0$ for all $x, y \in I$. If A and H are Hermitian matrices with eigenvalues in I , then $\sum_i \operatorname{Tr}[f_i(A)g_i(H)] \geq 0$.*

Proof. Consider the eigenvalue decompositions $A = \sum_j \lambda_j u_j u_j^*$ and $H = \sum_k \mu_k v_k v_k^*$. We evaluate the sum of traces:

$$\sum_i \operatorname{Tr}[f_i(A)g_i(H)] = \sum_i \operatorname{Tr} \left[\left(\sum_j f_i(\lambda_j) u_j u_j^* \right) \left(\sum_k g_i(\mu_k) v_k v_k^* \right) \right]$$

Using the linearity and cyclic property of the trace, this simplifies to:

$$\sum_{j,k} \left[\sum_i f_i(\lambda_j) g_i(\mu_k) \right] \operatorname{Tr}(u_j u_j^* v_k v_k^*) = \sum_{j,k} \left[\sum_i f_i(\lambda_j) g_i(\mu_k) \right] |\langle u_j, v_k \rangle|^2$$

By assumption, the bracketed sum is non-negative for all eigenvalue pairs (λ_j, μ_k) , and the squared inner product $|\langle u_j, v_k \rangle|^2 \geq 0$. Thus, the entire sum is non-negative. □

15.8 Wielandt Minimax Formula

Theorem 289 (Wielandt Minimax Formula). *Let A be an $n \times n$ Hermitian matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$. For any sequence of indices $1 \leq i_1 < i_2 < \cdots < i_k \leq n$,*

$$\sum_{j=1}^k \lambda_{i_j}(A) = \max_{\substack{V_1 \subset \cdots \subset V_k \\ \dim(V_j) = i_j}} \min_{W \in X(V_1, \dots, V_k)} \operatorname{tr}_W(A)$$

where $X(V_1, \dots, V_k)$ denotes the set of k -dimensional subspaces W that admit a basis $v_1, \dots, v_k$ with $v_j \in V_j$.

Proof. The $\leq$ case: Let $V_j = \operatorname{span}(e_1, \dots, e_{i_j})$, where e_i are the eigenvectors of A . For any $W \in X(V_1, \dots, V_k)$, it remains to show that $\sum_{j=1}^k \lambda_{i_j}(A) \leq \operatorname{tr}_W(A)$.

To show this, we construct an orthonormal set of vectors $v_1, \dots, v_k$ such that $v_j \in V_j \cap W$. Since $\dim(V_1 \cap W) \geq 1$, we pick any unit vector $v_1 \in V_1 \cap W$. Next, since $\dim(V_2 \cap W) \geq 2$, we pick any unit vector $v_2 \in V_2 \cap W$ that is perpendicular to v_1 , and so on. Then we have

$$\operatorname{tr}_W(A) \geq \sum_{j=1}^k \langle v_j, A v_j \rangle \geq \sum_{j=1}^k \lambda_{i_j}(A),$$

where the last inequality follows from the Rayleigh quotient minimum property on the spaces V_j .

The $\geq$ case: For any sequence of subspaces $V_1 \subset \cdots \subset V_k$ with $\dim(V_j) = i_j$, we must find some $W \in X(V_1, \dots, V_k)$ such that $\sum_{j=1}^k \lambda_{i_j}(A) \geq \operatorname{tr}_W(A)$.

We prove this by induction on n . The $n = 1$ case is trivially the Courant-Fischer theorem. Assume now $n \geq 2$.

If $i_1 \geq 2$, then we can apply induction. Let $E = \text{span}(e_{i_1}, \dots, e_n)$. We construct a partial flag within E from the intersection of E with $V_1, \dots, V_k$. We begin by picking an $(i_k - (i_1 - 1))$ -dimensional subspace $W'_k \subset E \cap V_k$, which exists by counting dimensions (it has codimension $i_1 - 1$ within V_k). Then we go down by one space, picking an $(i_{k-1} - (i_1 - 1))$ -dimensional subspace $W'_{k-1} \subset W'_k \cap V_{k-1}$. This still exists. We repeat this to form a flag $W'_1 \subset \dots \subset W'_k$. Now, since $\dim(E) \leq n - 1$, we can apply the induction hypothesis: there exists some $W \in X(W'_1, \dots, W'_k)$ such that

$$\sum_{j=1}^k \lambda_{i_j - (i_1 - 1)}(A|_E) \geq \text{tr}_W(A).$$

Note that $\lambda_{i_j - (i_1 - 1)}(A|_E)$ is the $(i_j - (i_1 - 1))$ -th eigenvalue of A orthogonally projected down to E . By the Cauchy interlacing theorem, $\lambda_{i_j - (i_1 - 1)}(A|_E) \leq \lambda_{i_j}(A)$. Since $X(W'_1, \dots, W'_k) \subset X(V_1, \dots, V_k)$, we are done.

If $i_1 = 1$, we perform a similar construction. Let $E = \text{span}(e_2, \dots, e_n)$. If $V_k \subset E$, then we can induct immediately. Otherwise, we construct a partial flag sequence $W'_2 \subset \dots \subset W'_k$ inside E . By induction, there exists some $W' \in X(W'_2, \dots, W'_k) \subset X(V_2, \dots, V_k)$ such that

$$\sum_{j=2}^k \lambda_{i_j - 1}(A|_E) \geq \text{tr}_{W'}(A).$$

Thus, by Cauchy interlacing, $\sum_{j=2}^k \lambda_{i_j}(A) \geq \text{tr}_{W'}(A)$. It remains to find some vector v such that $W' \oplus \text{span}(v) \in X(V_1, \dots, V_k)$. If $V_1 \not\subset W'$, then any $v \in V_1 \setminus W'$ would work. Otherwise, if $V_2 \not\subset W'$, then any $v \in V_2 \setminus W'$ would work, and so on. If none of these work, it implies $V_k \subset E$, which is a contradiction. This completes the induction. $\square$

Corollary 290 (Weyl's Inequality). *Let A and B be $n \times n$ Hermitian matrices with eigenvalues sorted in descending order: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. For all $1 \leq i, j \leq n$ such that $i + j - 1 \leq n$, we have:*

$$\lambda_{i+j-1}(A+B) \leq \lambda_i(A) + \lambda_j(B)$$

Theorem 291 (Lidskii's Inequality). *Let A and B be $n \times n$ Hermitian matrices with their eigenvalues sorted in descending order, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. For any sequence of indices $1 \leq i_1 < \dots < i_k \leq n$, it holds that*

$$\begin{aligned} \sum_{j=1}^k \lambda_{i_j}(A+B) &\leq \sum_{j=1}^k \lambda_{i_j}(A) + \sum_{j=1}^k \lambda_j(B) \\ \sum_{j=1}^k \lambda_{i_j}(A+B) &\geq \sum_{j=1}^k \lambda_{i_j}(A) + \sum_{j=1}^k \lambda_{n-j+1}(B) \end{aligned}$$

Corollary 292 (Extremal Partial Trace). *Let A be a Hermitian matrix. We have:*

$$\begin{aligned} \sum_{j=1}^k \lambda_j(A) &= \sup_{\dim(V)=k} \text{tr}_V(A) \\ \sum_{j=1}^k \lambda_{n-j+1}(A) &= \inf_{\dim(V)=k} \text{tr}_V(A) \end{aligned}$$

Corollary 293 (Schur-Horn Inequality). *The sum of the largest k eigenvalues is a convex function, and the sum of the smallest k eigenvalues is a concave function. Furthermore, for any subset of indices $i_1 < \dots < i_k$,*

$$\sum_{j=1}^k \lambda_{n-j+1}(A) \leq \sum_{j=1}^k A_{i_j, i_j} \leq \sum_{j=1}^k \lambda_j(A).$$

Corollary 294 (Hadamard's Determinant Inequality). *For every positive definite matrix A ,*

$$\det A \leq \prod_{i=1}^n A_{ii}.$$

Proof. Write

$$A = V^*V$$

for an invertible matrix V , and let $v_1, \dots, v_n$ be the columns of V . Then

$$A_{ij} = \langle v_i, v_j \rangle, \quad A_{ii} = \|v_i\|_2^2.$$

Moreover,

$$\det A = |\det V|^2.$$

Geometrically, $|\det V|$ is the volume of the parallelepiped spanned by $v_1, \dots, v_n$. Gram–Schmidt writes this volume as the product of the successive perpendicular heights:

$$|\det V| = \prod_{i=1}^n \text{dist}(v_i, \text{span}(v_1, \dots, v_{i-1})) \leq \prod_{i=1}^n \|v_i\|_2.$$

Squaring gives

$$\det A \leq \prod_i \|v_i\|_2^2 = \prod_i A_{ii}.$$

□

15.9 The Operator Jensen Inequality

Ordinary Jensen says that applying a convex function after averaging is better than averaging after applying the function. The operator version allows the “weights” themselves to be matrices.

The clean way to understand the proof is in two steps. First embed the matrix convex combination as the compression of a larger block matrix. Then average that block matrix with a sign-flipped copy to erase its off-diagonal blocks.

Definition 295 (Matrix Convex Combination). Let A_1, A_2 be Hermitian and suppose

$$K_1^* K_1 + K_2^* K_2 = I.$$

Then

$$K_1^* A_1 K_1 + K_2^* A_2 K_2$$

is called a matrix convex combination of A_1, A_2 .

Theorem 296 (Operator Jensen). *Let f be operator convex on an interval I , let A_1, A_2 be Hermitian with spectra in I , and suppose*

$$K_1^* K_1 + K_2^* K_2 = I.$$

Then

$$f(K_1^* A_1 K_1 + K_2^* A_2 K_2) \preceq K_1^* f(A_1) K_1 + K_2^* f(A_2) K_2.$$

For operator-concave f , the inequality reverses.

Proof. Put

$$\mathcal{A} = \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix}, \quad V = \begin{pmatrix} K_1 \\ K_2 \end{pmatrix}.$$

The condition on K_1, K_2 says precisely that

$$V^*V = I,$$

so V is an isometry. Extend its columns to a unitary matrix

$$Q = (V \ W).$$

Then

$$M := Q^* \mathcal{A} Q = \begin{pmatrix} C & * \\ * & D \end{pmatrix},$$

where

$$C = V^* \mathcal{A} V = K_1^* A_1 K_1 + K_2^* A_2 K_2.$$

Now let

$$U = \begin{pmatrix} I & 0 \\ 0 & -I \end{pmatrix}.$$

Averaging M with $U^* M U$ deletes the off-diagonal blocks:

$$\frac{1}{2}(M + U^* M U) = \begin{pmatrix} C & 0 \\ 0 & D \end{pmatrix}.$$

Operator convexity therefore gives

$$\begin{aligned} \begin{pmatrix} f(C) & 0 \\ 0 & f(D) \end{pmatrix} &= f\left(\frac{1}{2}(M + U^* M U)\right) \\ &\preceq \frac{1}{2}(f(M) + U^* f(M) U). \end{aligned}$$

Because functional calculus commutes with unitary conjugation,

$$f(M) = Q^* f(\mathcal{A}) Q.$$

Taking the upper-left block gives

$$f(C) \preceq V^* f(\mathcal{A}) V = K_1^* f(A_1) K_1 + K_2^* f(A_2) K_2.$$

The concave version follows by applying the convex statement to $-f$. □

15.10 Lieb's Concavity Theorem

In one variable,

$$x^q y^{1-q}$$

is the weighted geometric mean of x and y . The operator perspective gives the canonical noncommutative version of this expression, and Operator Jensen gives its joint concavity.

Definition 297 (Operator Perspective). Let $f : (0, \infty) \rightarrow \mathbb{R}$ and let $A, B \succ 0$. Define

$$\boxed{\mathcal{P}_f(A, B) = B^{1/2} f(B^{-1/2} A B^{-1/2}) B^{1/2}.}$$

If A and B commute, this reduces to

$$\mathcal{P}_f(A, B) = B f(AB^{-1}).$$

In particular, for $f(t) = t^q$,

$$\mathcal{P}_q(A, B) = B^{1/2} (B^{-1/2} A B^{-1/2})^q B^{1/2},$$

the weighted matrix geometric mean.

Lemma 298 (Perspective Preserves Operator Concavity). *If f is operator concave, then*

$$(A, B) \mapsto \mathcal{P}_f(A, B)$$

is jointly operator concave on positive-definite pairs.

Proof. Take two positive-definite pairs

$$(A_1, B_1), \quad (A_2, B_2)$$

and a weight $\lambda \in [0, 1]$. Put

$$A = \lambda A_1 + (1 - \lambda)A_2, \quad B = \lambda B_1 + (1 - \lambda)B_2,$$

and define

$$K_1 = \sqrt{\lambda} B_1^{1/2} B^{-1/2}, \quad K_2 = \sqrt{1 - \lambda} B_2^{1/2} B^{-1/2}.$$

Then

$$K_1^* K_1 + K_2^* K_2 = B^{-1/2} (\lambda B_1 + (1 - \lambda)B_2) B^{-1/2} = I.$$

Set

$$X_i = B_i^{-1/2} A_i B_i^{-1/2}.$$

A direct calculation gives

$$K_1^* X_1 K_1 + K_2^* X_2 K_2 = B^{-1/2} A B^{-1/2}.$$

The concave form of Operator Jensen therefore gives

$$f(B^{-1/2} A B^{-1/2}) \succeq K_1^* f(X_1) K_1 + K_2^* f(X_2) K_2.$$

Conjugating by $B^{1/2}$ yields

$$\mathcal{P}_f(A, B) \succeq \lambda \mathcal{P}_f(A_1, B_1) + (1 - \lambda) \mathcal{P}_f(A_2, B_2).$$

□

Corollary 299 (Joint Concavity of Tensor Powers). *For $0 \leq q \leq 1$,*

$$(A, B) \mapsto A^q \otimes B^{1-q}$$

is jointly operator concave.

Proof. Take

$$f(t) = t^q,$$

which is operator concave by [Lemma 279](#). Apply [Lemma 298](#) to

$$X = A \otimes I, \quad Y = I \otimes B.$$

Since X and Y commute,

$$\begin{aligned} \mathcal{P}_q(X, Y) &= (I \otimes B^{1/2})(A \otimes B^{-1})^q (I \otimes B^{1/2}) \\ &= A^q \otimes B^{1-q}. \end{aligned}$$

Both X and Y depend linearly on A, B , so the claimed joint concavity follows. □

The same argument gives all exponents whose total is at most one.

Lemma 300 (Joint Concavity Below Total Exponent One). *If*

$$q, r \geq 0, \quad q + r \leq 1,$$

then

$$(A, B) \mapsto A^q \otimes B^r$$

is jointly operator concave.

Proof. The cases $q + r = 0$ and $q + r = 1$ are immediate from the preceding corollary. Suppose

$$s = q + r \in (0, 1).$$

Define

$$G(A, B) = A^{q/s} \otimes B^{r/s}.$$

The exponents in G sum to one, so G is jointly operator concave. Also

$$A^q \otimes B^r = G(A, B)^s.$$

Now $X \mapsto X^s$ is both operator monotone by [Theorem 278](#) and operator concave by [Lemma 279](#). Hence

$$\begin{aligned} G(\lambda z_1 + (1 - \lambda)z_2)^s &\succeq (\lambda G(z_1) + (1 - \lambda)G(z_2))^s \\ &\succeq \lambda G(z_1)^s + (1 - \lambda)G(z_2)^s. \end{aligned}$$

This is exactly the desired joint concavity. □

The last step in Lieb's theorem is easiest to understand by viewing a matrix as a vector.

For $K \in \mathbb{C}^{m \times n}$, define

$$\text{vec}(K) = \sum_{i=1}^m \sum_{j=1}^n K_{ij} e_i \otimes e_j.$$

Thus the row index becomes the first tensor coordinate and the column index the second. Consequently, left multiplication acts on the first factor and right multiplication on the second:

$$\text{vec}(XKY) = (X \otimes Y^T) \text{vec}(K).$$

This can easily be checked on $K = e_i e_j^*$ which implies it for all matrices. It is clear that $\text{vec}(A)^* \text{vec}(B) = \text{Tr}(A^* B)$, thus we can now deal in terms of inner products rather than traces.

Lemma 301 (Lieb's Concavity Theorem). *Let $K \in \mathbb{C}^{m \times n}$ be fixed. If*

$$q, r \geq 0, \quad q + r \leq 1,$$

then

$$F(A, B) = \text{Tr}(K^* A^q K B^r)$$

is jointly concave for $A \succ 0$ and $B \succ 0$.

Proof. Vectorization turns the trace into a fixed quadratic form. Using the preceding formulas, we obtain

$$F(A, B) = \text{vec}(K)^* ((B^T)^r \otimes A^q) \text{vec}(K).$$

The transpose map is linear and preserves positive definiteness. By [Lemma 300](#),

$$(A, B) \mapsto (B^T)^r \otimes A^q$$

is jointly operator concave. Applying the fixed positive linear functional

$$M \mapsto \text{vec}(K)^* M \text{vec}(K)$$

preserves concavity. □

15.10.1 Joint Convexity of Matrix Relative Entropy

The first important consequence is obtained by differentiating Lieb's concavity at exponent zero.

Lemma 302 (Joint Convexity of Matrix Relative Entropy). *For $A, B \succ 0$, define*

$$S(A\|B) = \text{Tr}(A \log A - A \log B - A + B).$$

Then $S(A\|B)$ is jointly convex in (A, B) .

Proof. For $t \in (0, 1)$ define

$$J_t(A, B) = \frac{1}{t} (\text{Tr}(B^t A^{1-t}) - \text{Tr} A).$$

By [Lemma 301](#), the first trace is jointly concave, while the second term is linear. Hence J_t is jointly concave.

As $t \downarrow 0$,

$$\left. \frac{d}{dt} B^t \right|_{t=0} = \log B, \quad \left. \frac{d}{dt} A^{1-t} \right|_{t=0} = -A \log A.$$

Therefore

$$\begin{aligned} \lim_{t \downarrow 0} J_t(A, B) &= \text{Tr}((\log B)A - A \log A) \\ &= \text{Tr}(A \log B - A \log A). \end{aligned}$$

A finite pointwise limit of concave functions is concave. Hence

$$(A, B) \mapsto \text{Tr}(A \log B - A \log A)$$

is jointly concave. Negating it and adding the linear term

$$\text{Tr}(B - A)$$

gives the claimed joint convexity. □

15.10.2 A Variational Formula for the Trace

The next identity is simply nonnegativity of relative entropy, rearranged as a variational principle.

Lemma 303 (Variational Formula for Trace). *For $M \succ 0$,*

$$\boxed{\text{Tr} M = \max_{T \succ 0} \text{Tr}(T \log M - T \log T + T).}$$

Equality is attained at $T = M$.

Proof. The scalar inequality

$$a \log a - a \log h - a + h \geq 0 \quad (a, h > 0)$$

is the tangent-line inequality for the convex function

$$x \mapsto x \log x - x.$$

By the generalized Klein inequality,

$$S(T\|M) = \text{Tr}(T \log T - T \log M - T + M) \geq 0.$$

Rearranging gives

$$\text{Tr} M \geq \text{Tr}(T \log M - T \log T + T).$$

Equality holds for $T = M$. □

15.10.3 Lieb's Trace-Exponential Theorem

Now the trace exponential becomes a partial maximization of a jointly concave function. This is the form of Lieb's theorem used throughout matrix concentration.

Theorem 304 (Lieb). *For fixed Hermitian H , the map*

$$A \longmapsto \operatorname{Tr} \exp(H + \log A)$$

is concave on the positive-definite cone.

Proof. Apply Lemma 303 to

$$M = e^{H + \log A}.$$

Then

$$\begin{aligned} \operatorname{Tr} e^{H + \log A} &= \max_{T \succ 0} \operatorname{Tr}(T(H + \log A) - T \log T + T) \\ &= \max_{T \succ 0} \{\operatorname{Tr}(TH) - S(T\|A) + \operatorname{Tr} A\}. \end{aligned}$$

By Lemma 302, $S(T\|A)$ is jointly convex. Therefore the quantity inside braces is jointly concave in (T, A) .

Partial maximization preserves concavity. Indeed, take A_1, A_2 and nearly optimal T_1, T_2 . At

$$A = \lambda A_1 + (1 - \lambda)A_2$$

the point

$$T = \lambda T_1 + (1 - \lambda)T_2$$

is admissible. Joint concavity then gives

$$F(A, T) \geq \lambda F(A_1, T_1) + (1 - \lambda)F(A_2, T_2).$$

Taking the approximations to the maxima arbitrarily close proves concavity. $\square$

Corollary 305 (Lieb's Trace Inequality). *For fixed Hermitian H and a random Hermitian matrix X for which the expectations exist,*

$$\mathbb{E} \operatorname{Tr} e^{H+X} \leq \operatorname{Tr} \exp(H + \log \mathbb{E} e^X).$$

Proof. Apply Jensen's inequality to the concave map from Theorem 304 with the random positive-definite matrix

$$A = e^X.$$

Then

$$\mathbb{E} \operatorname{Tr} e^{H + \log(e^X)} \leq \operatorname{Tr} e^{H + \log \mathbb{E} e^X}.$$

Since X is Hermitian,

$$\log(e^X) = X.$$

$\square$

15.11 Lieb–Thirring Inequalities

For powers of two, the word-straightening argument from Golden–Thompson gives the inequality almost for free. For arbitrary real exponents, the main ingredient is the Cordes inequality, itself a one-line consequence of Löwner–Heinz. Exterior powers then upgrade an operator-norm inequality to all eigenvalues at once.

Theorem 306 (Lieb–Thirring Inequality). *For $A, B \succ 0$ and every real $q \geq 1$,*

$$\operatorname{Tr}((BAB)^q) \leq \operatorname{Tr}(B^q A^q B^q).$$

The powers-of-two case. Suppose

$$q = 2^m.$$

Cyclicity gives

$$\mathrm{Tr}((BAB)^q) = \mathrm{Tr}((B^2A)^q).$$

Applying [Lemma 281](#) to

$$U = B^2, \quad V = A$$

yields

$$\mathrm{Tr}((B^2A)^q) \leq \mathrm{Tr}(B^{2q}A^q) = \mathrm{Tr}(B^qA^qB^q).$$

Lemma 307 (Cordes Inequality). *If $C, D \succ 0$ and $0 \leq s \leq 1$, then*

$$\boxed{\|C^s D^s\| \leq \|CD\|^s.}$$

Here $\|\cdot\|$ is the operator norm.

Proof. Let

$$c = \|CD\|.$$

Then

$$DC^2D = (CD)^*(CD) \preceq c^2I.$$

Conjugating by D^{-1} gives

$$C^2 \preceq c^2 D^{-2}.$$

Apply Löwner–Heinz, [Theorem 278](#), with exponent s :

$$C^{2s} \preceq c^{2s} D^{-2s}.$$

Conjugating by D^s ,

$$D^s C^{2s} D^s \preceq c^{2s} I.$$

Finally,

$$\|C^s D^s\|^2 = \|D^s C^{2s} D^s\|,$$

so

$$\|C^s D^s\| \leq c^s = \|CD\|^s.$$

□

Proof of [Theorem 306](#) for general q . Set

$$s = \frac{1}{q}.$$

Apply [Lemma 307](#) to

$$C = B^q, \quad D = A^{q/2}.$$

Since

$$C^s = B, \quad D^s = A^{1/2},$$

we obtain

$$\|BA^{1/2}\| \leq \|B^q A^{q/2}\|^{1/q}.$$

Raise to the power $2q$. Since

$$\|BA^{1/2}\|^2 = \|BAB\|$$

and

$$\|B^q A^{q/2}\|^2 = \|B^q A^q B^q\|,$$

we get

$$\|(BAB)^q\| \leq \|B^q A^q B^q\|.$$

This controls only the largest eigenvalue. Exterior powers apply the same inequality simultaneously to every product of the largest eigenvalues.

Apply the operator-norm inequality above to the positive matrices

$$\bigwedge^k A, \quad \bigwedge^k B.$$

Exterior powers commute with products and functional calculus, so

$$\left\| \bigwedge^k (BAB)^q \right\| \leq \left\| \bigwedge^k (B^q A^q B^q) \right\|.$$

Since the largest eigenvalue of $\bigwedge^k X$ is the product of the largest k eigenvalues of X , we obtain, for every k ,

$$\prod_{i=1}^k \lambda_i((BAB)^q) \leq \prod_{i=1}^k \lambda_i(B^q A^q B^q).$$

At $k = n$ there is equality, because

$$\begin{aligned} \det((BAB)^q) &= \det(B)^{2q} \det(A)^q \\ &= \det(B^q A^q B^q). \end{aligned}$$

Hence, after taking logarithms,

$$(\log \lambda_i((BAB)^q))_{i=1}^n$$

is majorized by

$$(\log \lambda_i(B^q A^q B^q))_{i=1}^n.$$

Applying Karamata to the convex function

$$x \mapsto e^x$$

gives

$$\sum_i \lambda_i((BAB)^q) \leq \sum_i \lambda_i(B^q A^q B^q),$$

which is precisely

$$\mathrm{Tr}((BAB)^q) \leq \mathrm{Tr}(B^q A^q B^q).$$

□

The following “extended” inequality looks more complicated, but its proof is much easier. Once A is diagonal, the entire theorem reduces to the scalar weighted AM–GM inequality.

Theorem 308 (Extended Lieb–Thirring Inequality). *For $A \succ 0$, Hermitian B , and $\alpha \in [0, 1]$,*

$$\mathrm{Tr}(BA^\alpha BA^{1-\alpha}) \leq \mathrm{Tr}(B^2 A).$$

Proof. Work in an eigenbasis of A :

$$A = \mathrm{diag}(\lambda_1, \dots, \lambda_n), \quad \lambda_i > 0.$$

Since B is Hermitian,

$$B_{ji} = \overline{B_{ij}}.$$

Therefore

$$\mathrm{Tr}(BA^\alpha BA^{1-\alpha}) = \sum_{i,j} |B_{ij}|^2 \lambda_i^{1-\alpha} \lambda_j^\alpha.$$

Symmetrizing the pair (i, j) ,

$$\mathrm{Tr}(BA^\alpha BA^{1-\alpha}) = \frac{1}{2} \sum_{i,j} |B_{ij}|^2 (\lambda_i^{1-\alpha} \lambda_j^\alpha + \lambda_j^{1-\alpha} \lambda_i^\alpha).$$

Weighted AM–GM gives

$$\lambda_i^{1-\alpha} \lambda_j^\alpha \leq (1-\alpha)\lambda_i + \alpha\lambda_j,$$

while swapping i, j gives

$$\lambda_j^{1-\alpha} \lambda_i^\alpha \leq (1-\alpha)\lambda_j + \alpha\lambda_i.$$

Adding,

$$\lambda_i^{1-\alpha} \lambda_j^\alpha + \lambda_j^{1-\alpha} \lambda_i^\alpha \leq \lambda_i + \lambda_j.$$

Hence

$$\begin{aligned} \mathrm{Tr}(BA^\alpha BA^{1-\alpha}) &\leq \frac{1}{2} \sum_{i,j} |B_{ij}|^2 (\lambda_i + \lambda_j) \\ &= \sum_{i,j} |B_{ij}|^2 \lambda_i \\ &= \mathrm{Tr}(B^2 A). \end{aligned}$$

□

15.12 Matrix Concentration

The scalar Chernoff/Bernstein method has two steps:

$$\Pr[S \geq t] \leq e^{-\theta t} \mathbb{E} e^{\theta S},$$

and, for independent scalars,

$$\mathbb{E} e^{\theta \sum_i X_i} = \prod_i \mathbb{E} e^{\theta X_i}.$$

For matrices, the first step survives after replacing the scalar exponential by

$$\mathrm{Tr} e^{(\cdot)}.$$

The second step fails because noncommuting exponentials do not factor. Lieb's theorem is exactly the replacement: it lets us move out one random matrix at a time while putting its matrix MGF inside a logarithm.

After this one substitution, Chernoff and Bernstein are similar to their scalar proofs.

Let all matrices below be $d \times d$ self-adjoint matrices.

Lemma 309 (Matrix Laplace Transform). *For independent self-adjoint random matrices $X_1, \dots, X_n$,*

$$\Pr \left[\lambda_{\max} \left(\sum_i X_i \right) \geq t \right] \leq \inf_{\theta > 0} e^{-\theta t} \mathrm{Tr} \exp \left(\sum_i \log \mathbb{E} e^{\theta X_i} \right).$$

Proof. Let

$$Y = \sum_i X_i.$$

Since

$$e^{\theta \lambda_{\max}(Y)} \leq \text{Tr } e^{\theta Y},$$

Markov's inequality gives

$$\Pr[\lambda_{\max}(Y) \geq t] \leq e^{-\theta t} \mathbb{E} \text{Tr } e^{\theta Y}.$$

It remains to control the trace MGF.

Condition on

$$X_1, \dots, X_{n-1}$$

and apply [Corollary 305](#) with

$$H = \theta \sum_{i < n} X_i, \quad X = \theta X_n.$$

Then

$$\mathbb{E}_{X_n} \text{Tr } e^{\theta \sum_{i=1}^n X_i} \leq \text{Tr} \exp \left(\theta \sum_{i < n} X_i + \log \mathbb{E} e^{\theta X_n} \right).$$

Now condition on

$$X_1, \dots, X_{n-2}$$

and apply the same inequality to X_{n-1} , treating the deterministic matrix

$$\log \mathbb{E} e^{\theta X_n}$$

as part of H . Continuing backwards gives

$$\mathbb{E} \text{Tr } e^{\theta \sum_i X_i} \leq \text{Tr} \exp \left(\sum_i \log \mathbb{E} e^{\theta X_i} \right).$$

Substitute this into the Markov bound and optimize over $\theta > 0$. □

Theorem 310 (Matrix Chernoff). *Let X_i be independent and*

$$0 \preceq X_i \preceq RI.$$

Set

$$\mu_{\max} = \lambda_{\max} \left(\sum_i \mathbb{E} X_i \right), \quad \mu_{\min} = \lambda_{\min} \left(\sum_i \mathbb{E} X_i \right).$$

Then

$$\Pr[\lambda_{\max}(\sum_i X_i) \geq (1 + \delta)\mu_{\max}] \leq d \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}} \right)^{\mu_{\max}/R}.$$

For $0 \leq \delta < 1$,

$$\Pr[\lambda_{\min}(\sum_i X_i) \leq (1 - \delta)\mu_{\min}] \leq d \left(\frac{e^{-\delta}}{(1 - \delta)^{1-\delta}} \right)^{\mu_{\min}/R}.$$

Proof. For $0 \preceq X \preceq RI$, convexity of $e^{\theta x}$ on $[0, R]$ gives by functional calculus

$$e^{\theta X} \preceq I + \frac{e^{\theta R} - 1}{R} X.$$

Hence

$$\log \mathbb{E} e^{\theta X} \preceq \frac{e^{\theta R} - 1}{R} \mathbb{E} X.$$

Insert this into [Lemma 309](#) and optimize exactly as in the scalar Chernoff proof. The lower tail uses $\theta < 0$. $\square$

Theorem 311 (Matrix Bernstein). *Let X_i be independent with*

$$\mathbb{E} X_i = 0, \quad \lambda_{\max}(X_i) \leq R.$$

Set

$$\sigma^2 = \left\| \sum_i \mathbb{E} X_i^2 \right\|.$$

Then

$$\Pr \left[\lambda_{\max} \left(\sum_i X_i \right) \geq t \right] \leq d \exp \left(-\frac{t^2}{2(\sigma^2 + Rt/3)} \right).$$

If

$$\|X_i\| \leq R,$$

then

$$\Pr \left[\left\| \sum_i X_i \right\| \geq t \right] \leq 2d \exp \left(-\frac{t^2}{2(\sigma^2 + Rt/3)} \right).$$

Proof. The scalar inequality used for Freedman holds under functional calculus:

$$e^{\theta X} \preceq I + \theta X + g(\theta) X^2.$$

Taking expectations,

$$\mathbb{E} e^{\theta X} \preceq I + g(\theta) \mathbb{E} X^2 \preceq \exp(g(\theta) \mathbb{E} X^2).$$

Hence

$$\log \mathbb{E} e^{\theta X} \preceq g(\theta) \mathbb{E} X^2.$$

Apply [Lemma 309](#):

$$\Pr[\cdots] \leq d \exp(-\theta t + g(\theta) \sigma^2),$$

and optimize as before.

Apply the same bound to $-\sum_i X_i$ for the lower tail. $\square$

Theorem 312 (Matrix Freedman). *Let X_k be a self-adjoint matrix martingale difference sequence:*

$$\mathbb{E}[X_k \mid \mathcal{F}_{k-1}] = 0, \quad \lambda_{\max}(X_k) \leq R.$$

Define

$$Y_k = \sum_{i=1}^k X_i$$

and

$$W_k = \sum_{i=1}^k \mathbb{E}[X_i^2 \mid \mathcal{F}_{i-1}].$$

Then

$$\Pr \left[\exists k : \lambda_{\max}(Y_k) \geq t \text{ and } \lambda_{\max}(W_k) \leq \sigma^2 \right] \leq d \exp \left(-\frac{t^2}{2(\sigma^2 + Rt/3)} \right).$$

Proof. Fix $\theta \geq 0$ and write

$$g(\theta) = \frac{e^{\theta R} - 1 - \theta R}{R^2}.$$

The scalar inequality

$$e^{\theta x} \leq 1 + \theta x + g(\theta)x^2 \quad (x \leq R)$$

gives, by functional calculus,

$$e^{\theta X_k} \preceq I + \theta X_k + g(\theta)X_k^2.$$

Taking conditional expectation and using $\mathbb{E}[X_k \mid \mathcal{F}_{k-1}] = 0$,

$$\mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}] \preceq I + g(\theta)V_k \preceq e^{g(\theta)V_k},$$

where

$$V_k = \mathbb{E}[X_k^2 \mid \mathcal{F}_{k-1}].$$

Since the matrix logarithm is operator monotone,

$$\boxed{\log \mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}] \preceq g(\theta)V_k.}$$

Now define the scalar process

$$Z_k = \text{tr} \exp(\theta Y_k - g(\theta)W_k).$$

We claim that Z_k is a positive supermartingale.

Indeed, conditioned on $\mathcal{F}_{k-1}$, the matrix V_k is fixed, and

$$W_k = W_{k-1} + V_k.$$

Lieb's trace inequality gives

$$\begin{aligned} \mathbb{E}[Z_k \mid \mathcal{F}_{k-1}] &= \mathbb{E} \text{tr} \exp(\theta Y_{k-1} - g(\theta)W_k + \theta X_k) \\ &\leq \text{tr} \exp(\theta Y_{k-1} - g(\theta)W_k + \log \mathbb{E}[e^{\theta X_k} \mid \mathcal{F}_{k-1}]) \\ &\leq \text{tr} \exp(\theta Y_{k-1} - g(\theta)W_k + g(\theta)V_k) \\ &= Z_{k-1}. \end{aligned}$$

Also

$$Z_0 = \text{tr} I = d.$$

Suppose that for some k ,

$$\lambda_{\max}(Y_k) \geq t, \quad \lambda_{\max}(W_k) \leq \sigma^2.$$

Then

$$\begin{aligned} \lambda_{\max}(\theta Y_k - g(\theta)W_k) &\geq \theta \lambda_{\max}(Y_k) - g(\theta) \lambda_{\max}(W_k) \\ &\geq \theta t - g(\theta)\sigma^2. \end{aligned}$$

Hence

$$Z_k \geq \exp(\theta t - g(\theta)\sigma^2).$$

Ville's inequality for the positive supermartingale Z_k therefore gives

$$\Pr [\exists k : \lambda_{\max}(Y_k) \geq t, \lambda_{\max}(W_k) \leq \sigma^2] \leq d \exp(-\theta t + g(\theta)\sigma^2).$$

Finally, for

$$0 \leq \theta < \frac{3}{R},$$

the scalar Bernstein estimate

$$g(\theta) \leq \frac{\theta^2}{2(1 - \theta R/3)}$$

holds. Choose

$$\theta = \frac{t}{\sigma^2 + Rt/3}.$$

Then

$$1 - \frac{\theta R}{3} = \frac{\sigma^2}{\sigma^2 + Rt/3},$$

and so

$$-\theta t + g(\theta)\sigma^2 \leq -\frac{t^2}{2(\sigma^2 + Rt/3)}.$$

This proves the result. □

Lemma 313 (Matrix Azuma MGF). *Let X be a random self-adjoint matrix with*

$$\mathbb{E}X = 0, \quad X^2 \preceq A^2,$$

where A is deterministic. Then for every self-adjoint H ,

$$\boxed{\mathbb{E} \operatorname{tr} e^{H+\theta X} \leq \operatorname{tr} e^{H+2\theta^2 A^2}.$$

The same statement holds conditionally when H, A are measurable.

Proof. The reason for symmetrization is that our hypothesis controls X^2 , rather than X itself. Introducing an independent random sign turns the exponential into an even function of X , which can be controlled by X^2 .

We first use the standard trace symmetrization inequality

$$\boxed{\mathbb{E} \operatorname{tr} e^{H+\theta X} \leq \mathbb{E}_{X,\varepsilon} \operatorname{tr} e^{H+2\theta\varepsilon X},$$

where ε is an independent Rademacher sign.

For completeness, let X' be an independent copy of X . Since $\mathbb{E}X' = 0$, convexity of the trace exponential gives

$$\mathbb{E}_X \operatorname{tr} e^{H+\theta X} \leq \mathbb{E}_{X,X'} \operatorname{tr} e^{H+\theta(X-X')}.$$

The matrix $X - X'$ is symmetric in distribution, so we may multiply it by an independent Rademacher sign ε . Split

$$H + \theta\varepsilon(X - X') = \left(\frac{H}{2} + \theta\varepsilon X\right) + \left(\frac{H}{2} - \theta\varepsilon X'\right).$$

Golden–Thompson and Cauchy–Schwarz for the trace give

$$\begin{aligned} \operatorname{tr} e^{H+\theta\varepsilon(X-X')} &\leq \operatorname{tr} \left(e^{H/2+\theta\varepsilon X} e^{H/2-\theta\varepsilon X'} \right) \\ &\leq \sqrt{\operatorname{tr} e^{H+2\theta\varepsilon X} \operatorname{tr} e^{H-2\theta\varepsilon X'}}. \end{aligned}$$

Taking expectation and applying scalar Cauchy–Schwarz, the two factors have the same distribution, so

$$\mathbb{E} \operatorname{tr} e^{H+\theta X} \leq \mathbb{E}_{X,\varepsilon} \operatorname{tr} e^{H+2\theta\varepsilon X}.$$

Now condition on X . Lieb’s inequality gives

$$\mathbb{E}_\varepsilon \operatorname{tr} e^{H+2\theta\varepsilon X} \leq \operatorname{tr} \exp \left(H + \log \mathbb{E}_\varepsilon e^{2\theta\varepsilon X} \right).$$

But

$$\mathbb{E}_\varepsilon e^{2\theta\varepsilon X} = \cosh(2\theta X).$$

The scalar inequality

$$\cosh z \leq e^{z^2/2}$$

gives by functional calculus

$$\cosh(2\theta X) \preceq e^{2\theta^2 X^2}.$$

Taking logarithms,

$$\log \cosh(2\theta X) \preceq 2\theta^2 X^2 \preceq 2\theta^2 A^2.$$

Therefore

$$\mathbb{E}_\varepsilon \operatorname{tr} e^{H+2\theta\varepsilon X} \leq \operatorname{tr} e^{H+2\theta^2 A^2}.$$

□

Theorem 314 (Matrix Azuma–Hoeffding). *Let X_k be self-adjoint martingale differences and suppose*

$$X_k^2 \preceq A_k^2$$

for deterministic self-adjoint matrices A_k . Set

$$\sigma^2 = \left\| \sum_k A_k^2 \right\|.$$

Then

$$\Pr \left[\lambda_{\max} \left(\sum_k X_k \right) \geq t \right] \leq d \exp \left(-\frac{t^2}{8\sigma^2} \right).$$

Consequently,

$$\Pr \left[\left\| \sum_k X_k \right\| \geq t \right] \leq 2d \exp \left(-\frac{t^2}{8\sigma^2} \right).$$

Proof. The matrix Laplace bound gives

$$\Pr \left[\lambda_{\max} \left(\sum_k X_k \right) \geq t \right] \leq e^{-\theta t} \mathbb{E} \operatorname{tr} e^{\theta \sum_k X_k}.$$

Apply [Lemma 313](#) conditionally to the last martingale difference, then the next-to-last, and so on. This gives

$$\mathbb{E} \operatorname{tr} e^{\theta \sum_k X_k} \leq \operatorname{tr} \exp \left(2\theta^2 \sum_k A_k^2 \right).$$

Since

$$\sum_k A_k^2 \preceq \sigma^2 I,$$

we obtain

$$\mathbb{E} \operatorname{tr} e^{\theta \sum_k X_k} \leq d e^{2\theta^2 \sigma^2}.$$

Hence

$$\Pr \left[\lambda_{\max} \left(\sum_k X_k \right) \geq t \right] \leq d e^{-\theta t + 2\theta^2 \sigma^2}.$$

The exponent is minimized at

$$\theta = \frac{t}{4\sigma^2},$$

giving

$$de^{-t^2/(8\sigma^2)}.$$

Apply the same result to $-X_k$ and take a union bound to obtain the spectral-norm estimate. $\square$

Theorem 315 (Matrix McDiarmid). *Let $Z_1, \dots, Z_n$ be independent and let*

$$H(Z_1, \dots, Z_n)$$

be self-adjoint. Suppose that whenever z, z' differ only in coordinate i ,

$$(H(z) - H(z'))^2 \preceq A_i^2$$

for a deterministic self-adjoint matrix A_i . Set

$$\sigma^2 = \left\| \sum_i A_i^2 \right\|.$$

Then

$$\Pr [\lambda_{\max}(H(Z) - \mathbb{E}H(Z)) \geq t] \leq de^{-t^2/(8\sigma^2)}.$$

Proof. Take the Doob martingale

$$M_i = \mathbb{E}[H(Z) \mid Z_1, \dots, Z_i], \quad X_i = M_i - M_{i-1}.$$

Then

$$\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = 0$$

and

$$\sum_i X_i = H(Z) - \mathbb{E}H(Z).$$

We only need to show

$$X_i^2 \preceq A_i^2.$$

Condition on $Z_1, \dots, Z_i$. Let Z'_i be an independent copy of Z_i , and use the same random future

$$Z_{i+1}, \dots, Z_n$$

on both sides. Then

$$X_i = \mathbb{E}[H(Z_1, \dots, Z_i, \dots, Z_n) - H(Z_1, \dots, Z'_i, \dots, Z_n) \mid Z_1, \dots, Z_i].$$

The map

$$X \mapsto X^2$$

is operator convex, so Jensen gives

$$\begin{aligned} X_i^2 &\preceq \mathbb{E} \left[(H(Z_1, \dots, Z_i, \dots, Z_n) - H(Z_1, \dots, Z'_i, \dots, Z_n))^2 \mid Z_1, \dots, Z_i \right] \\ &\preceq A_i^2. \end{aligned}$$

Thus the Doob differences satisfy the hypotheses of [Theorem 314](#), which proves the claim. $\square$

16 Convex Optimization

A gradient tells us how the objective changes to first order. By itself, it cannot tell us how far to move: the linear approximation keeps improving in the same direction even after the true objective has turned upward. For a differentiable convex function f , the discrepancy is its *Bregman divergence*

$$D_f(y, x) = f(y) - f(x) - \langle \nabla f(x), y - x \rangle.$$

The second argument is the point where we linearize; the first is where we evaluate the approximation. Convexity says exactly that $D_f(y, x) \geq 0$. The divergence need not be symmetric or satisfy the triangle inequality.

The first choice is therefore a tractable bound on that error. Minimizing a linear term plus a quadratic bound gives gradient descent or a linear solve. Replacing the quadratic by an entropy divergence gives a multiplicative update. The convergence proof answers a separate question: why does the progress certified by this local calculation lead toward a global optimum? Sometimes the local decrease is already a fixed fraction of the objective gap. Otherwise, the update itself often suggests a potential whose decrease can account for the errors over many iterations.

We call dependence polynomial in $1/\varepsilon$ *low accuracy*, and dependence logarithmic in $1/\varepsilon$ *high accuracy*. Unless a constraint set is specified, optimization is over $\mathbb{R}^n$. A minimizer is assumed to exist whenever a statement refers to x^* . Constrained problems use a convex set K .

We count arithmetic operations. Logarithms are natural unless a base is indicated. For $M \succeq 0$, write $\|v\|_M = (v^\top M v)^{1/2}$. Recall that a differentiable function is convex exactly when its gradient is monotone: $\langle \nabla f(y) - \nabla f(x), y - x \rangle \geq 0$ for all x, y . For twice differentiable functions this is equivalent to $\nabla^2 f(x) \succeq 0$.

16.1 Linear Systems, Richardson Iteration, and Preconditioning

Let $A \succ 0$ be symmetric. Solving $Ax = b$ is equivalent to minimizing $f(x) = \frac{1}{2}x^\top Ax - b^\top x$, since $\nabla f(x) = Ax - b$. For the quadratic case we have

$$f(x + \Delta) - f(x) = \langle g, \Delta \rangle + \frac{1}{2}\|\Delta\|_A^2, \quad g = Ax - b.$$

In particular, $D_f(x + \Delta, x) = \frac{1}{2}\|\Delta\|_A^2$. Expanding instead at the minimizer, where $Ax^* = b$, gives the useful formula

$$f(x) - f(x^*) = \frac{1}{2}(x - x^*)^\top A(x - x^*) = \frac{1}{2}\|x - x^*\|_A^2.$$

Suppose $\mu I \preceq A \preceq LI$, where $0 < \mu \leq L$, and put $\kappa = L/\mu$. Then

$$\langle g, \Delta \rangle + \frac{\mu}{2}\|\Delta\|_2^2 \leq f(x + \Delta) - f(x) \leq \langle g, \Delta \rangle + \frac{L}{2}\|\Delta\|_2^2.$$

The two bounds answer different questions. The upper bound limits the cost of a proposed move, so minimizing it gives a safe step. The lower bound limits how much improvement is possible at all, even at the unknown minimizer. Comparing their minima will relate the improvement we obtain to the improvement still available.

16.1.1 Richardson Iteration

Complete the square in the upper bound: $\langle g, \Delta \rangle + (L/2)\|\Delta\|_2^2 = (L/2)\|\Delta + g/L\|_2^2 - \|g\|_2^2/(2L)$. Thus its minimizer is $\Delta = -g/L$, giving Richardson iteration,

$$x_{t+1} = x_t - \frac{1}{L}(Ax_t - b).$$

There are two useful proofs of convergence: one explains the optimization principle, while the other exploits the linear recurrence.

The two methods. The chosen step gives

$$f(x - g/L) - f(x) \leq -\frac{\|g\|_2^2}{2L}.$$

On the other hand, evaluating the lower bound at $x^* - x$ and then minimizing it over all displacements gives

$$\begin{aligned} f(x^*) - f(x) &\geq \langle g, x^* - x \rangle + \frac{\mu}{2} \|x^* - x\|_2^2 \\ &\geq \min_{\Delta} \left\{ \langle g, \Delta \rangle + \frac{\mu}{2} \|\Delta\|_2^2 \right\} = -\frac{\|g\|_2^2}{2\mu}. \end{aligned}$$

Negating the lower-bound inequality gives $f(x) - f(x^*) \leq \|g\|_2^2/(2\mu)$, or equivalently $\|g\|_2^2 \geq 2\mu(f(x) - f(x^*))$. Substitute this in the descent bound:

$$\begin{aligned} f(x^+) - f(x^*) &\leq f(x) - f(x^*) - \frac{\|g\|_2^2}{2L} \\ &\leq f(x) - f(x^*) - \frac{\mu}{L} (f(x) - f(x^*)). \end{aligned}$$

The guaranteed decrease is therefore at least a μ/L fraction of the remaining gap,

$$\boxed{f(x_{t+1}) - f(x^*) \leq (1 - \mu/L)(f(x_t) - f(x^*)).}$$

This argument will apply unchanged whenever the same upper and lower bounds are available.

The exact recurrence. Let $e_t = x_t - x^*$ and $r_t = Ax_t - b = Ae_t$. Both satisfy

$$e_{t+1} = (I - A/L)e_t, \quad r_{t+1} = (I - A/L)r_t.$$

Each eigendirection with eigenvalue λ is multiplied by $1 - \lambda/L$. The largest curvature determines the safe step size, but the smallest curvature determines the slowest remaining component. Since all these factors lie in $[0, 1 - 1/\kappa]$, starting from $x_0 = 0$ gives

$$\|Ax_T - b\|_2 \leq (1 - 1/\kappa)^T \|b\|_2 \leq e^{-T/\kappa} \|b\|_2.$$

Consequently $O(\kappa \log(1/\varepsilon))$ matrix-vector products suffice for relative residual ε . The same recurrence also contracts $\|e_t\|_A$, and hence the quadratic objective gap, with the corresponding squared factor.

16.1.2 Preconditioning

Richardson is slow when the curvature forcing a short step is much larger than the curvature in the directions where most error remains. Changing only the scalar step size cannot resolve this conflict. A preconditioner changes which directions count as long: it replaces the Euclidean movement penalty by one adapted to A . Suppose an easier matrix $B \succ 0$ satisfies $\mu B \preceq A \preceq LB$. In coordinates $y = B^{1/2}x$, the objective becomes

$$f(B^{-1/2}y) = \frac{1}{2}y^\top Cy - \langle B^{-1/2}b, y \rangle, \quad C = B^{-1/2}AB^{-1/2}.$$

Congruencing $\mu B \preceq A \preceq LB$ by $B^{-1/2}$ gives $\mu I \preceq C \preceq LI$. Its gradient is $Cy - B^{-1/2}b = B^{-1/2}(Ax - b)$. A step in y subtracts this gradient divided by L ; multiplying that step by $B^{-1/2}$ to return to x gives

$$x_{t+1} = x_t - \frac{1}{L}B^{-1}(Ax_t - b).$$

Equivalently, this step minimizes $\langle g, \Delta \rangle + (L/2)\|\Delta\|_B^2$. We do not form $B^{1/2}$; the change of variables explains the method, while applying B^{-1} implements it.

Algorithm 54 Preconditioned Richardson iteration

Input: $\mu B \preceq A \preceq LB$ with $A, B \succ 0$; right-hand side b ; accuracy $\varepsilon \in (0, 1)$

Output: $\|Ax - b\|_{B^{-1}} \leq \varepsilon \|b\|_{B^{-1}}$

1: $x \leftarrow 0$, $T \leftarrow \lceil (L/\mu) \log(1/\varepsilon) \rceil$

2: **for** $t = 1, \dots, T$ **do**

3: $r \leftarrow Ax - b$

4: Solve $Bz = r$

5: $x \leftarrow x - z/L$

6: **return** x

The residual guarantee is now $\|Ax_T - b\|_{B^{-1}} \leq \varepsilon \|b\|_{B^{-1}}$. Taking $B = I$ recovers the original method. Taking $B = A$ gives one-step convergence, but makes each step the original problem. A useful preconditioner must improve the curvature ratio *and* be cheaper to invert; spectral closeness alone is not enough.

16.1.3 Chebyshev Acceleration: Optimize the Whole Iteration

Preconditioning changes the spectrum. Even with the spectrum fixed, Richardson leaves another opportunity: it applies the same linear factor at every step. We can instead choose the *whole sequence of steps* so that their combined effect is small on every eigendirection. The right object to design is the polynomial multiplying the initial error.

From an iteration to a polynomial. First take $B = I$, and write $e_0 = x_0 - x^*$. Richardson gives $e_T = (I - A/L)^T e_0$, so its error polynomial is $p_T(\lambda) = (1 - \lambda/L)^T$. More generally, suppose we can arrange $e_T = p(A)e_0$. In an orthonormal eigenbasis $Av_i = \lambda_i v_i$, write $e_0 = \sum_i a_i v_i$. Then

$$p(A)e_0 = \sum_i a_i p(\lambda_i) v_i, \quad \|p(A)e_0\|_A^2 = \sum_i \lambda_i a_i^2 p(\lambda_i)^2.$$

Consequently, a uniform bound $|p(\lambda)| \leq E$ on $[\mu, L]$ gives $\|e_T\|_A \leq E \|e_0\|_A$. The same argument applies to the residual, since $r_T = Ae_T = p(A)r_0$.

We cannot compute $p(A)e_0$ directly because e_0 contains the unknown solution. The normalization $p(0) = 1$ removes this obstacle. It makes $1 - p(\lambda)$ divisible by λ , so for some polynomial q , $p(\lambda) = 1 - \lambda q(\lambda)$. The computable point $x = x_0 - q(A)(Ax_0 - b)$ then satisfies

$$\begin{aligned} x - x^* &= e_0 - q(A)Ae_0 \\ &= (I - Aq(A))e_0 = p(A)e_0. \end{aligned}$$

A degree- T error polynomial requires only degree $T - 1$ for q . Thus T matrix-vector products suffice in principle. We will obtain a short recurrence that avoids forming either polynomial's coefficients.

Using only the spectral bounds, our problem is therefore

$$\min_{\deg p \leq T, p(0)=1} \max_{\lambda \in [\mu, L]} |p(\lambda)|.$$

The value at zero is fixed, but zero lies outside the spectral interval. We want a polynomial that is small throughout that interval and large relative to those values at zero. Chebyshev polynomials provide exactly this separation.

A polynomial bounded on one interval and growing outside it. Define $T_0(s) = 1$, $T_1(s) = s$, and

$$T_{t+1}(s) = 2sT_t(s) - T_{t-1}(s).$$

The identity $\cos((t+1)\theta) = 2\cos\theta\cos(t\theta) - \cos((t-1)\theta)$ obeys the same recurrence and initial values. Hence $\mathbb{T}_t(\cos\theta) = \cos(t\theta)$. Every $s \in [-1, 1]$ is a cosine, so $|\mathbb{T}_t(s)| \leq 1$ there. For $T \geq 1$, its values at $s_j = \cos(j\pi/T)$, $j = 0, \dots, T$, alternate between 1 and -1.

Outside this interval, let $s > 1$ and solve $\zeta + \zeta^{-1} = 2s$ for the root $\zeta = s + \sqrt{s^2 - 1} > 1$. The sequence $u_t = (\zeta^t + \zeta^{-t})/2$ has $u_0 = 1$, $u_1 = s$, and

$$\begin{aligned} 2su_t - u_{t-1} &= \frac{(\zeta + \zeta^{-1})(\zeta^t + \zeta^{-t}) - (\zeta^{t-1} + \zeta^{-(t-1)})}{2} \\ &= \frac{\zeta^{t+1} + \zeta^{-(t+1)}}{2} = u_{t+1}. \end{aligned}$$

Thus $\mathbb{T}_t(s) = (\zeta^t + \zeta^{-t})/2$. The same recurrence that produces bounded oscillation on $[-1, 1]$ produces exponential growth at any fixed $s > 1$.

Move the spectrum to the bounded interval. Assume $\mu < L$; the case $\mu = L$ is a one-step solve. Put $c = (L + \mu)/2$, $d = (L - \mu)/2$, and $s = c/d > 1$. The map $\lambda \mapsto (c - \lambda)/d$ sends μ to 1, L to -1, and zero to s . Therefore define

$$p_T(\lambda) = \frac{\mathbb{T}_T((c - \lambda)/d)}{\mathbb{T}_T(s)}. \quad (4)$$

The denominator makes $p_T(0) = 1$. On the spectrum, the numerator has absolute value at most one, and attains one at the mapped extrema. Hence

$$\max_{\lambda \in [\mu, L]} |p_T(\lambda)| = \frac{1}{\mathbb{T}_T(s)}.$$

The alternating extrema prove optimality of this polynomial. Suppose another polynomial, after the same change of variables, is q with $q(s) = 1$ and $\max_{[-1, 1]} |q| < 1/\mathbb{T}_T(s)$. Then $R = \mathbb{T}_T - \mathbb{T}_T(s)q$ has sign $(-1)^j$ at s_j . It has a root between every two consecutive extrema, giving T roots in $(-1, 1)$, and another at $s > 1$ because $R(s) = 0$. But R is a nonzero polynomial of degree at most T , a contradiction. Thus (4) is minimax for the stated interval problem.

Where the square root of the condition number comes from. Write $\kappa = L/\mu > 1$. The outside point and its exponential growth factor are

$$\begin{aligned} s &= \frac{\kappa + 1}{\kappa - 1}, & \sqrt{s^2 - 1} &= \frac{2\sqrt{\kappa}}{\kappa - 1}, \\ \zeta = s + \sqrt{s^2 - 1} &= \frac{\kappa + 1 + 2\sqrt{\kappa}}{\kappa - 1} = \frac{\sqrt{\kappa} + 1}{\sqrt{\kappa} - 1}. \end{aligned}$$

Set $\rho = \zeta^{-1} = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$. The uniform error and a convenient upper bound are

$$\frac{1}{\mathbb{T}_T(s)} = \frac{2\rho^T}{1 + \rho^{2T}} \leq 2\rho^T.$$

For $a = 1/\sqrt{\kappa} \in (0, 1)$,

$$-\log \rho = \log \frac{1+a}{1-a} = \int_0^a \frac{2}{1-u^2} du \geq 2a.$$

It follows that $2\rho^T \leq 2e^{-2T/\sqrt{\kappa}}$. Richardson needs $\Theta(\kappa)$ steps for a constant error reduction in the worst spectral direction; this polynomial needs only $O(\sqrt{\kappa})$. The improvement comes from choosing all the polynomial coefficients jointly.

Turn the polynomial recurrence into an update. The normalizing values $\mathsf{T}_t(s)$ grow exponentially, so maintain only their successive ratios $\theta_t = \mathsf{T}_t(s)/\mathsf{T}_{t+1}(s)$. Their recurrence is

$$\theta_0 = \frac{1}{s}, \quad \theta_t = \frac{\mathsf{T}_t(s)}{2s\mathsf{T}_t(s) - \mathsf{T}_{t-1}(s)} = \frac{1}{2s - \theta_{t-1}} \quad (t \geq 1).$$

The ratios stay in $(0, 1)$: $\theta_0 = 1/s < 1$, and if $0 < \theta_{t-1} < 1$, then $2s - \theta_{t-1} > 1$, so $0 < \theta_t < 1$. In particular, the recurrence never divides by zero.

Divide the Chebyshev recurrence at $(c - \lambda)/d$ by $\mathsf{T}_{t+1}(s)$. For $t \geq 1$ this gives

$$\begin{aligned} p_{t+1}(\lambda) &= 2\theta_t \frac{c - \lambda}{d} p_t(\lambda) - \theta_t \theta_{t-1} p_{t-1}(\lambda) \\ &= (1 + \beta_t - \alpha_t \lambda) p_t(\lambda) - \beta_t p_{t-1}(\lambda), \end{aligned}$$

where $\alpha_t = 2\theta_t/d$ and $\beta_t = \theta_t \theta_{t-1}$. For the second line, use $2(c/d)\theta_t - \theta_t \theta_{t-1} = \theta_t(2s - \theta_{t-1}) = 1$. This identity is essential: at $\lambda = 0$, the two recurrence coefficients sum to one, preserving the normalization $p_t(0) = 1$.

Apply the polynomial recurrence to e_0 . Since $Ae_t = Ax_t - b$,

$$\begin{aligned} e_{t+1} &= e_t - \alpha_t Ae_t + \beta_t(e_t - e_{t-1}), \\ x_{t+1} &= x_t - \alpha_t(Ax_t - b) + \beta_t(x_t - x_{t-1}). \end{aligned}$$

The unknown x^* cancels in the difference of consecutive errors. Only the current residual, the two most recent points, and one scalar ratio are needed.

Algorithm 55 Preconditioned Chebyshev iteration

Input: $A, B \succ 0$ and $0 < \mu \leq L$ with $\mu B \preceq A \preceq LB$; right-hand side b , initial point x_0 , and $T \geq 0$

```

1: if  $T = 0$  then
2:   return  $x_0$ 
3: Solve  $Bu_0 = Ax_0 - b$ 
4: if  $\mu = L$  then
5:   return  $x_0 - u_0/\mu$ 
6:  $c \leftarrow (L + \mu)/2$ ;  $d \leftarrow (L - \mu)/2$ ;  $s \leftarrow c/d$ 
7:  $x_1 \leftarrow x_0 - u_0/c$ ;  $\theta_{\text{old}} \leftarrow 1/s$ 
8: for  $t = 1, \dots, T - 1$  do
9:    $\theta \leftarrow 1/(2s - \theta_{\text{old}})$ 
10:   $\alpha \leftarrow 2\theta/d$ ;  $\beta \leftarrow \theta\theta_{\text{old}}$ 
11:  Solve  $Bu_t = Ax_t - b$ 
12:   $x_{t+1} \leftarrow x_t - \alpha u_t + \beta(x_t - x_{t-1})$ 
13:   $\theta_{\text{old}} \leftarrow \theta$ 
14: return  $x_T$ 
```

Preconditioning and the precise error guarantee. For general B , work in the transformed coordinates $\hat{e}_t = B^{1/2}(x_t - x^*)$ and $C = B^{-1/2}AB^{-1/2}$. The transformed residual is $\hat{r}_t = B^{-1/2}(Ax_t - b) = C\hat{e}_t$. Multiplying the update by $B^{1/2}$ gives exactly the recurrence just proved with matrix C , so $\hat{e}_T = p_T(C)\hat{e}_0$ and $\hat{r}_T = p_T(C)\hat{r}_0$. Also

$$\|\hat{r}_t\|_2 = \|Ax_t - b\|_{B^{-1}}, \quad \|\hat{e}_t\|_C = \|x_t - x^*\|_A.$$

The eigenvalues of C lie in $[\mu, L]$. Applying the spectral bound in these two norms proves

$$\begin{aligned} \|Ax_T - b\|_{B^{-1}} &\leq 2\rho^T \|Ax_0 - b\|_{B^{-1}}, \\ \|x_T - x^*\|_A &\leq 2\rho^T \|x_0 - x^*\|_A. \end{aligned}$$

Thus $T = \lceil (\sqrt{\kappa}/2) \log(2/\varepsilon) \rceil$ suffices for relative error ε in either norm. If $\mu = L$, then $A = \mu B$ and the separate first step is exact. For $T > 0$, the algorithm uses at most T products by A and T solves in B , and stores only the two most recent points together with its working vectors. No square root of B is formed; it is used only in the analysis.

16.2 Smoothness, Strong Convexity, and Euclidean Gradient Methods

The nonlinear analogue of $\mu I \preceq A \preceq LI$ is a two-sided bound on the Bregman divergence. Let $q(x) = \frac{1}{2}\|x\|_2^2$, so that $D_q(y, x) = \frac{1}{2}\|y - x\|_2^2$.

Definition 316 (Smoothness and strong convexity). For $L > 0$ and $\mu \geq 0$, a differentiable convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is *L-smooth* if $D_f(y, x) \leq LD_q(y, x)$, and *μ -strongly convex* if $D_f(y, x) \geq \mu D_q(y, x)$, for all x, y . Thus the two conditions together say

$$\frac{\mu}{2}\|y - x\|_2^2 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle \leq \frac{L}{2}\|y - x\|_2^2.$$

The upper bound gives the same step as before, $x^+ = x - \nabla f(x)/L$. If the lower bound also holds with $\mu > 0$, the Richardson bound comparison proves geometric convergence of the objective gap. Without strong convexity, that comparison is unavailable: we first see what replaces it.

16.2.1 Smooth Gradient Descent

Smoothness still guarantees a decrease of $\|\nabla f(x)\|_2^2/(2L)$, but there is no lower curvature bound with which to compare it. Convexity only gives $f(x) - f(x^*) \leq \langle \nabla f(x), x - x^* \rangle$. The right-hand side suggests the potential: it is the cross term in $\|x - \nabla f(x)/L - x^*\|_2^2$. The squared gradient term created by that expansion is related to the descent bound.

Algorithm 56 Smooth gradient descent

Input: Convex L -smooth f , initial point x_0 , iteration count $T \geq 1$

- 1: **for** $t = 0, \dots, T - 1$ **do**
 - 2: $x_{t+1} \leftarrow x_t - \nabla f(x_t)/L$
 - 3: **return** x_T
-

Theorem 317 (Smooth gradient descent). *For every minimizer x^* , Algorithm 56 satisfies*

$$f(x_T) - f(x^*) \leq \frac{L\|x_0 - x^*\|_2^2}{2T}.$$

Proof. Write $g = \nabla f(x)$ and $x^+ = x - g/L$. Convexity gives $f(x) - f(x^*) \leq \langle g, x - x^* \rangle$, while smoothness gives $f(x^+) - f(x) \leq -\|g\|_2^2/(2L)$. Adding them yields

$$f(x^+) - f(x^*) \leq \langle g, x - x^* \rangle - \frac{\|g\|_2^2}{2L} = \frac{L}{2}(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2).$$

Since

$$\begin{aligned} \|x^+ - x^*\|_2^2 &= \|x - x^*\|_2^2 - \frac{2}{L}\langle g, x - x^* \rangle + \frac{1}{L^2}\|g\|_2^2, \\ \frac{L}{2}(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2) &= \langle g, x - x^* \rangle - \frac{1}{2L}\|g\|_2^2. \end{aligned}$$

The negative squared-gradient term supplied by smoothness is exactly the term required by this potential difference. Sum over the T steps:

$$\sum_{t=0}^{T-1} (f(x_{t+1}) - f(x^*)) \leq \frac{L}{2}(\|x_0 - x^*\|_2^2 - \|x_T - x^*\|_2^2) \leq \frac{L}{2}\|x_0 - x^*\|_2^2.$$

The objective values are nonincreasing, so every term in the sum is at least $f(x_T) - f(x^*)$. Divide by T . $\square$

It is enough that its total decrease bounds the sum of the objective gaps. Monotonicity then turns a bound on their average into a bound on the last one.

16.2.2 Subgradient Descent and Averaging

For a nonsmooth convex function, a subgradient still separates the current point from better points, but it does not specify a safe step length. Even for $f(x) = |x|$, a step toward zero can cross it and increase the objective. We therefore keep the squared-distance calculation, but no longer expect monotonicity of the function values.

Suppose $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex and G -Lipschitz in ℓ_2 . Every $g \in \partial f(x)$ has $\|g\|_2 \leq G$: the subgradient inequality at $x + g$ gives $\|g\|_2^2 \leq f(x + g) - f(x) \leq G\|g\|_2$.

Algorithm 57 Subgradient descent

Input: Convex G -Lipschitz f , initial point x_0 , step size $\eta > 0$, iteration count $T \geq 1$

```

1:  $s \leftarrow 0$ 
2: for  $t = 0, \dots, T - 1$  do
3:    $s \leftarrow s + x_t$ 
4:   Choose  $g_t \in \partial f(x_t)$ 
5:    $x_{t+1} \leftarrow x_t - \eta g_t$ 
6: return  $s/T$ 
```

Theorem 318 (Subgradient descent). *Let $R = \|x_0 - x^*\|_2$. The average $\bar{x} = T^{-1} \sum_{t < T} x_t$ returned by Algorithm 57 satisfies*

$$f(\bar{x}) - f(x^*) \leq \frac{R^2}{2\eta T} + \frac{\eta G^2}{2}.$$

For $R, G > 0$, choosing $\eta = R/(G\sqrt{T})$ gives $f(\bar{x}) - f(x^*) \leq RG/\sqrt{T}$. If $R = 0$, return x_0 ; if $G = 0$, the function is constant.

Proof. The subgradient inequality and $\|g_t\|_2 \leq G$ give

$$\begin{aligned} \|x_{t+1} - x^*\|_2^2 &= \|x_t - x^*\|_2^2 - 2\eta \langle g_t, x_t - x^* \rangle + \eta^2 \|g_t\|_2^2 \\ &\leq \|x_t - x^*\|_2^2 - 2\eta (f(x_t) - f(x^*)) + \eta^2 G^2. \end{aligned}$$

Move the gap term to the left, divide by 2η , and sum:

$$\begin{aligned} \sum_{t=0}^{T-1} (f(x_t) - f(x^*)) &\leq \frac{1}{2\eta} \sum_{t=0}^{T-1} (\|x_t - x^*\|_2^2 - \|x_{t+1} - x^*\|_2^2) + \frac{T\eta G^2}{2} \\ &= \frac{\|x_0 - x^*\|_2^2 - \|x_T - x^*\|_2^2}{2\eta} + \frac{T\eta G^2}{2} \\ &\leq \frac{R^2}{2\eta} + \frac{T\eta G^2}{2}. \end{aligned}$$

Divide by T . Finally, convexity gives $f(\bar{x}) - f(x^*) \leq T^{-1} \sum_{t < T} (f(x_t) - f(x^*))$. For $\eta = R/(G\sqrt{T})$, each of the two terms in the average bound is $RG/(2\sqrt{T})$, giving the claimed total. $\square$

Unlike smooth descent, there is no objective-decrease estimate to cancel $\eta^2 \|g_t\|_2^2$. After summing, the initial squared distance contributes $R^2/(2\eta T)$ and the movement costs contribute at most $\eta G^2/2$. Equating the contributions gives $\eta = R/(G\sqrt{T})$ and the $T^{-1/2}$ rate. The proof controls an average of objective values; convexity makes the average iterate a way to realize that bound without assuming that the last iterate is best.

16.2.3 Equivalent Curvature Conditions

The definitions above are convenient for designing a step. Gradient and Hessian conditions are often more convenient for checking that the hypotheses hold. An identity we will use is

$$D_f(y, x) + D_f(x, y) = \langle \nabla f(y) - \nabla f(x), y - x \rangle.$$

Theorem 319 (Equivalent forms of Euclidean curvature). *Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable and convex, and put $q(x) = \frac{1}{2}\|x\|_2^2$. The following conditions are equivalent:*

$$\begin{aligned} f \text{ is } \mu\text{-strongly convex} &\iff f - \mu q \text{ is convex} \\ &\iff \langle \nabla f(y) - \nabla f(x), y - x \rangle \geq \mu\|y - x\|_2^2. \end{aligned}$$

Likewise, the following conditions are equivalent:

$$\begin{aligned} f \text{ is } L\text{-smooth} &\iff Lq - f \text{ is convex} \\ &\iff \langle \nabla f(y) - \nabla f(x), y - x \rangle \leq L\|y - x\|_2^2 \\ &\iff \|\nabla f(y) - \nabla f(x)\|_2 \leq L\|y - x\|_2. \end{aligned}$$

A convex L -smooth function also has a $1/L$ -co-coercive gradient:

$$\langle \nabla f(y) - \nabla f(x), y - x \rangle \geq \frac{1}{L}\|\nabla f(y) - \nabla f(x)\|_2^2.$$

When f is twice differentiable, the respective conditions are equivalent to $\nabla^2 f(x) \succeq \mu I$ and $\nabla^2 f(x) \preceq LI$ for every x .

Proof. Bregman divergence is linear in its defining function. Thus $D_{f-\mu q} = D_f - \mu D_q$ and $D_{Lq-f} = LD_q - D_f$, which prove the shifted-convexity characterizations. Apply gradient monotonicity to these two functions:

$$\begin{aligned} \langle \nabla(f - \mu q)(y) - \nabla(f - \mu q)(x), y - x \rangle &= \langle \nabla f(y) - \nabla f(x), y - x \rangle - \mu\|y - x\|_2^2, \\ \langle \nabla(Lq - f)(y) - \nabla(Lq - f)(x), y - x \rangle &= L\|y - x\|_2^2 - \langle \nabla f(y) - \nabla f(x), y - x \rangle. \end{aligned}$$

Nonnegativity is exactly the corresponding inner-product condition. The Hessian statements follow by differentiating the same shifted functions. The Lipschitz formulation of smoothness is also a consequence of convexity.

If the gradient is L -Lipschitz, put $\Delta = y - x$ and integrate:

$$\begin{aligned} D_f(y, x) &= \int_0^1 \langle \nabla f(x + t\Delta) - \nabla f(x), \Delta \rangle dt \\ &\leq \int_0^1 Lt\|\Delta\|_2^2 dt = \frac{L}{2}\|\Delta\|_2^2. \end{aligned}$$

For the converse, remove the irrelevant linear part at y : the convex function $\phi(z) = f(z) - \langle \nabla f(y), z \rangle$ is minimized at y , retains the same Bregman divergence, and has gradient $g = \nabla f(x) - \nabla f(y)$ at x . Its gradient step cannot improve by more than its gap above the minimum. This gives a bound on g in terms of $D_f(x, y) = \phi(x) - \phi(y)$. At $z = x - g/L$ we have

$$\begin{aligned} f(z) &\leq f(x) - \frac{1}{L}\langle \nabla f(x), g \rangle + \frac{1}{2L}\|g\|_2^2, \\ f(z) &\geq f(y) + \langle \nabla f(y), x - y \rangle - \frac{1}{L}\langle \nabla f(y), g \rangle. \end{aligned}$$

The lower bound for $f(z)$ cannot exceed the upper bound. Moving the terms at y to the left and the gradient terms to the right gives

$$\begin{aligned} f(x) - f(y) - \langle \nabla f(y), x - y \rangle &\geq \frac{1}{L} \langle \nabla f(x), g \rangle - \frac{1}{L} \langle \nabla f(y), g \rangle - \frac{1}{2L} \|g\|_2^2 \\ &= \frac{1}{L} \langle g, g \rangle - \frac{1}{2L} \|g\|_2^2 = \frac{1}{2L} \|g\|_2^2. \end{aligned}$$

The left side is $D_f(x, y)$. Interchanging x, y changes g to $-g$ and therefore gives the same lower bound for $D_f(y, x)$. Add them:

$$\frac{1}{L} \|g\|_2^2 \leq D_f(x, y) + D_f(y, x) = \langle g, x - y \rangle \leq \|g\|_2 \|x - y\|_2.$$

The first inequality is co-coercivity; the last is Cauchy–Schwarz. If $g \neq 0$, division by $\|g\|_2/L$ proves $\|g\|_2 \leq L\|x - y\|_2$. If $g = 0$, that conclusion is immediate. $\square$

16.3 Geometry Beyond Euclidean Space

Choosing a norm means choosing which displacements count as small. Once that choice is made, the steepest direction is the displacement that gives the largest linear improvement among unit steps. It need not be parallel to the Euclidean gradient.

16.3.1 Dual Norms and the Steepest Direction

For a norm $\|\cdot\|$, define its dual by $\|g\|_* = \max_{\|u\| \leq 1} \langle g, u \rangle$. Scaling gives $|\langle g, u \rangle| \leq \|g\|_* \|u\|$. Thus $\|g\|_*$ is exactly the largest change that the linear functional g can detect in a unit displacement. This is why displacements use the original norm and gradients use the dual norm.

The dual of ℓ_p is ℓ_q when $1/p + 1/q = 1$. For $1 < p < \infty$ and $g \neq 0$, a maximizing unit vector has coordinates $u_i = \text{sgn}(g_i) |g_i|^{q-1} / \|g\|_q^{q-1}$. Indeed, $p(q-1) = q$ implies $\sum_i |u_i|^p = \sum_i |g_i|^q / \|g\|_q^q = 1$, while $\langle g, u \rangle = \sum_i |g_i|^q / \|g\|_q^{q-1} = \|g\|_q$. Together with Hölder's inequality, this proves the duality claim. In ℓ_∞ we may take $u = \text{sgn}(g)$: every coordinate can move by one unit. In ℓ_1 we instead spend the entire unit displacement on a coordinate where $|g_i|$ is largest.

Definition 320 (Curvature in a norm). For differentiable convex $f : \mathbb{R}^n \rightarrow \mathbb{R}$, μ -strong convexity and L -smoothness in $\|\cdot\|$ mean, respectively,

$$D_f(y, x) \geq \frac{\mu}{2} \|y - x\|^2, \quad D_f(y, x) \leq \frac{L}{2} \|y - x\|^2.$$

Smoothness again suggests minimizing a linear term plus a squared movement cost. The direction and the length separate: first maximize the linear gain on the unit ball, then optimize the length of the step.

Lemma 321 (Steepest descent in a norm). For $L > 0$,

$$\min_{\Delta} \left\{ \langle g, \Delta \rangle + \frac{L}{2} \|\Delta\|^2 \right\} = -\frac{\|g\|_*^2}{2L}.$$

If $u \in \arg \max_{\|u\| \leq 1} \langle g, u \rangle$, then $\Delta = -(\|g\|_*/L)u$ is a minimizer.

Proof. For a step of length s , duality bounds the objective below by $-s\|g\|_* + Ls^2/2$. This scalar quadratic is minimized at $s = \|g\|_*/L$. The stated direction attains the duality bound. If $g = 0$, take $\Delta = 0$. $\square$

Algorithm 58 Steepest descent in a norm

Input: Convex L -smooth f in $\|\cdot\|$, point x , iteration count T

```
1: for  $t = 1, \dots, T$  do
2:    $g \leftarrow \nabla f(x)$ 
3:   if  $g = 0$  then
4:     return  $x$ 
5:   Choose  $u \in \arg \max_{\|u\| \leq 1} \langle g, u \rangle$ 
6:    $x \leftarrow x - (\|g\|_*/L)u$ 
7: return  $x$ 
```

16.3.2 Convergence from a Sublevel Radius

Each step decreases f by at least $\|\nabla f(x)\|_*^2/(2L)$. In a general norm, we cannot repeat the Euclidean squared-distance expansion: there may be no underlying inner product. Instead, use convexity to compare the objective gap directly with the gradient size.

Fix a minimizer x^* and assume the initial sublevel radius $R = \sup\{\|x - x^*\| : f(x) \leq f(x_0)\}$ is finite. The iterates stay in this sublevel set. If $R = 0$, the initial point is optimal; assume $R > 0$. Then

$$f(x) - f(x^*) \leq \langle \nabla f(x), x - x^* \rangle \leq R \|\nabla f(x)\|_*.$$

Writing $\delta_t = f(x_t) - f(x^*)$, we obtain

$$\delta_{t+1} \leq \delta_t - \frac{\delta_t^2}{2LR^2}.$$

The guaranteed decrease is quadratic in the remaining gap, rather than a fixed fraction of it. We should therefore expect a $1/T$ bound.

The reciprocal is the useful scalar potential: a decrease of order δ_t^2 becomes a constant increase in $1/\delta_t$. If any $\delta_t = 0$, the algorithm has reached a minimizer. Otherwise, $\nabla f(x^*) = 0$ and smoothness give $\delta_t = D_f(x_t, x^*) \leq (L/2)\|x_t - x^*\|^2 \leq LR^2/2$. Put $C = 2LR^2$, so $0 < \delta_t/C \leq 1/4$. As long as the next gap is positive,

$$\begin{aligned} \frac{1}{\delta_{t+1}} &\geq \frac{1}{\delta_t(1 - \delta_t/C)} \\ &\geq \frac{1}{\delta_t} \left(1 + \frac{\delta_t}{C}\right) = \frac{1}{\delta_t} + \frac{1}{C}, \end{aligned}$$

where $1/(1 - a) \geq 1 + a$ for $0 \leq a < 1$. Thus $1/\delta_T \geq 1/\delta_0 + T/C \geq T/C$, or $f(x_T) - f(x^*) \leq 2LR^2/T$. This argument uses a sublevel radius, not merely the initial distance to x^* , an important distinction.

16.3.3 Gradient and Hessian Characterizations

Lemma 322 (Gradient formulations in a norm). *For differentiable convex $f : \mathbb{R}^n \rightarrow \mathbb{R}$,*

$$\begin{aligned} f \text{ is } \mu\text{-strongly convex} &\iff \langle \nabla f(y) - \nabla f(x), y - x \rangle \geq \mu\|y - x\|^2, \\ f \text{ is } L\text{-smooth} &\iff \|\nabla f(y) - \nabla f(x)\|_* \leq L\|y - x\|. \end{aligned}$$

Proof. Adding $D_f(y, x) \geq (\mu/2)\|y - x\|^2$ and its reversed version proves the forward implication by the Bregman identity. Conversely, put $\Delta = y - x$. Applying the assumed gradient inequality to x and $x + t\Delta$ gives

$$\langle \nabla f(x + t\Delta) - \nabla f(x), t\Delta \rangle \geq \mu\|t\Delta\|^2 = \mu t^2 \|\Delta\|^2.$$

Divide by $t > 0$; one power of t remains on the right. Integration gives

$$D_f(y, x) = \int_0^1 \langle \nabla f(x + t\Delta) - \nabla f(x), \Delta \rangle dt \geq \int_0^1 \mu t \|\Delta\|^2 dt = \frac{\mu}{2} \|\Delta\|^2.$$

Gradient Lipschitzness implies smoothness by the same integral, using duality and the upper bound $Lt\|\Delta\|^2$.

For the reverse implication, let $g = \nabla f(x) - \nabla f(y)$ and choose a unit u with $\langle g, u \rangle = \|g\|_*$. The convex function $\phi(z) = f(z) - \langle \nabla f(y), z \rangle$ is minimized at y and has $\nabla \phi(x) = g$. Its smoothness bound at $z = x - (\|g\|_*/L)u$ gives

$$\phi(y) \leq \phi(z) \leq \phi(x) - \frac{\|g\|_*^2}{2L}.$$

Since $\phi(x) - \phi(y) = D_f(x, y) \leq L\|x - y\|^2/2$, we obtain $\|g\|_* \leq L\|x - y\|$. $\square$

For twice differentiable f , the respective conditions are equivalent to

$$z^\top \nabla^2 f(x) z \geq \mu \|z\|^2, \quad z^\top \nabla^2 f(x) z \leq L \|z\|^2.$$

16.3.4 The Squared ℓ_p Norm

For regression, we can separate the curvature of the loss from the action of the matrix. First bound the error of the loss in residual space; then substitute the residual displacement $A\Delta$. The squared ℓ_p norm is particularly useful because its smoothness in ℓ_p depends on p .

Lemma 323. *For $2 \leq p < \infty$, $f(x) = \|x\|_p^2$ is $2(p-1)$ -smooth in ℓ_p .*

Proof. For $p = 2$, the Hessian is $2I$ and the claimed constant is 2. Suppose $p > 2$ and $x \neq 0$. Write $S = \sum_i |x_i|^p$ and $v_i = |x_i|^{p-2}x_i$. The chain rule first gives $\nabla f(x) = (2/p)S^{2/p-1}(pv) = 2S^{2/p-1}v$. Differentiate the scalar factor and the vector v separately:

$$\begin{aligned} \frac{\partial}{\partial x_j}(2S^{2/p-1}) &= 2(2-p)S^{2/p-2}v_j, \\ \frac{\partial v_i}{\partial x_j} &= (p-1)|x_i|^{p-2}\mathbf{1}[i=j]. \end{aligned}$$

Since $S = \|x\|_p^p$, their contributions are

$$\nabla^2 f(x) = 2(2-p)\|x\|_p^{2-2p}vv^\top + 2(p-1)\|x\|_p^{2-p}\text{diag}(|x_i|^{p-2}).$$

The rank-one term is negative semidefinite, so dropping it can only help. The remaining sum has exactly the powers needed for Hölder with exponents $p/(p-2)$ and $p/2$. Thus,

$$\begin{aligned} z^\top \nabla^2 f(x) z &\leq 2(p-1)\|x\|_p^{2-p} \sum_i |x_i|^{p-2} z_i^2 \\ &\leq 2(p-1)\|x\|_p^{2-p} \left(\sum_i |x_i|^p \right)^{(p-2)/p} \left(\sum_i |z_i|^p \right)^{2/p} = 2(p-1)\|z\|_p^2. \end{aligned}$$

Integrating along a segment proves the smoothness inequality. If the segment passes through the origin, apply the bound on either side and use continuity of ∇f there; the Hessian at the origin is not needed. $\square$

Consequently, for $F(x) = \|Ax - b\|_p^2$,

$$D_F(x + \Delta, x) \leq (p-1)\|A\Delta\|_p^2.$$

Where the Hessian exists, this is also $z^\top \nabla^2 F(x) z \leq 2(p-1)\|Az\|_p^2$. Thus an operator bound $\|Az\|_p \leq M\|z\|$ yields a smoothness constant $2(p-1)M^2$ in the chosen norm on the variables.

16.4 Bregman Geometry and Relative Curvature

A fixed norm assigns the same cost to the same displacement everywhere. When curvature varies with position, a bound valid over the whole domain may therefore be governed by regions far from the current point. We would like the movement penalty to vary as well, while retaining useful properties of the algebra. A reference function h provides both: its Bregman divergence adapts to its curvature and satisfies a useful three-point identity.

For $h(x) = \frac{1}{2}x^\top Bx$, this is exactly preconditioning, $D_h(y, x) = \frac{1}{2}\|y - x\|_B^2$. The purpose of a nonquadratic h is to model the objective's changing curvature while leaving the resulting local optimization problem easy to solve.

16.4.1 Comparing Curvature to a Reference Function

Definition 324 (Relative smoothness and strong convexity). For differentiable convex f, h , f is L -smooth relative to h if $D_f(y, x) \leq LD_h(y, x)$, and μ -strongly convex relative to h if $D_f(y, x) \geq \mu D_h(y, x)$, for all points in their common domain. Equivalently,

$$Lh - f \text{ is convex,} \quad f - \mu h \text{ is convex,}$$

respectively, because $D_{Lh-f} = LD_h - D_f$ and $D_{f-\mu h} = D_f - \mu D_h$.

It is useful to look at the twice differentiable case. A Bregman divergence integrates the directional curvature encountered along the entire step, not just the Hessian at its starting point.

Lemma 325 (Integrated Hessian formula). If h is C^2 on a neighborhood of the segment from x to $x + \Delta$, then

$$D_h(x + \Delta, x) = \int_0^1 (1-t) \Delta^\top \nabla^2 h(x + t\Delta) \Delta dt.$$

Proof. For $\phi(t) = h(x + t\Delta)$, we have $\phi'(0) = \langle \nabla h(x), \Delta \rangle$ and $\phi''(s) = \Delta^\top \nabla^2 h(x + s\Delta) \Delta$. Therefore

$$\begin{aligned} D_h(x + \Delta, x) &= \phi(1) - \phi(0) - \phi'(0) \\ &= \int_0^1 (\phi'(t) - \phi'(0)) dt \\ &= \int_0^1 \int_0^t \phi''(s) ds dt = \int_0^1 (1-s) \phi''(s) ds. \end{aligned}$$

In the last equality, a fixed s occurs for $t \in [s, 1]$, an interval of length $1 - s$. □

Relative curvature is therefore equivalent, in the C^2 setting, to

$$\mu \nabla^2 h(x) \preceq \nabla^2 f(x) \preceq L \nabla^2 h(x).$$

The reference Hessian may change with x , but the comparison factors μ, L remain uniform. This is the analogue of having a well-conditioned system in preconditioned coordinates.

16.4.2 Proximal Steps and the Three-Point Identity

For a linear term g and base point $x \in K$, define

$$\text{Prox}_x^h(g) \in \arg \min_{z \in K} \{\langle g, z \rangle + D_h(z, x)\}.$$

Subtracting the tangent of h at x makes the penalty have zero value and zero gradient at the current point. It discourages movement without imposing an additional first-order term unrelated to g . In the unconstrained case, optimality gives

$$\nabla h(x^+) = \nabla h(x) - g.$$

Thus the step is always additive in the reference gradient. A quadratic h makes that gradient linear in x ; a logarithmic reference gradient will make the update multiplicative in x .

To analyze the step, we need to turn this change in ∇h into a change in a potential. The following identity does exactly that.

Lemma 326 (Three-point identity). *For differentiable convex h ,*

$$\langle \nabla h(y) - \nabla h(x), z - y \rangle = D_h(z, x) - D_h(z, y) - D_h(y, x).$$

Proof. Cancel the function values first, then combine the two terms with $\nabla h(x)$:

$$\begin{aligned} D_h(z, x) - D_h(z, y) - D_h(y, x) &= -\langle \nabla h(x), z - x \rangle + \langle \nabla h(y), z - y \rangle + \langle \nabla h(x), y - x \rangle \\ &= -\langle \nabla h(x), z - y \rangle + \langle \nabla h(y), z - y \rangle \\ &= \langle \nabla h(y) - \nabla h(x), z - y \rangle. \end{aligned}$$

□

A useful way to remember the three-point identity is to compare going directly from x to z with stopping at y and relinearizing there. The direct error is $D_h(z, x)$, while the two-leg errors are $D_h(y, x) + D_h(z, y)$. The only difference is that on the remaining leg $y \rightarrow z$, the linear prediction changes from slope $\nabla h(x)$ to slope $\nabla h(y)$, contributing

$$\langle \nabla h(y) - \nabla h(x), z - y \rangle.$$

Lemma 327 (Proximal three-point inequality). *If $x^+ = \text{Prox}_x^h(\eta g)$, then every $u \in K$ satisfies*

$$\eta \langle g, x^+ - u \rangle \leq D_h(u, x) - D_h(u, x^+) - D_h(x^+, x).$$

Proof. The derivative of the proximal objective in the feasible direction $u - x^+$ is nonnegative at its minimizer:

$$\langle \eta g + \nabla h(x^+) - \nabla h(x), u - x^+ \rangle \geq 0.$$

Move the linear-loss term to the other side. The three-point identity with $(x, y, z) = (x, x^+, u)$ gives

$$\begin{aligned} \eta \langle g, x^+ - u \rangle &\leq \langle \nabla h(x^+) - \nabla h(x), u - x^+ \rangle \\ &= D_h(u, x) - D_h(u, x^+) - D_h(x^+, x). \end{aligned}$$

□

This leaves $-D_h(x^+, x)$, a nonpositive movement term. In a convergence proof we should keep this term until we see which local error it can absorb.

16.4.3 Relative Gradient Descent

Relative smoothness supplies an upper bound $f(x) + \langle \nabla f(x), z - x \rangle + LD_h(z, x)$. Minimize it, just as for Richardson iteration.

Algorithm 59 Relative gradient descent

Input: Convex f that is L -smooth relative to h on K ; point x_0 ; iteration count T

- 1: **for** $t = 0, \dots, T - 1$ **do**
 - 2: $x_{t+1} \leftarrow \text{Prox}_{x_t}^h(\nabla f(x_t)/L)$
 - 3: **return** x_T
-

Theorem 328 (Relative gradient descent). *Suppose f is L -smooth and μ -strongly convex relative to h , with $L > 0$ and $0 \leq \mu \leq L$, and x^* minimizes f over K . One step satisfies*

$$f(x^+) - f(x^*) + LD_h(x^*, x^+) \leq (L - \mu)D_h(x^*, x).$$

In particular, $D_h(x^*, x^+) \leq (1 - \mu/L)D_h(x^*, x)$.

Proof. Write $g = \nabla f(x)$. The step is $x^+ = \text{Prox}_x^h(g/L)$, so multiply the proximal inequality with comparator x^* by L :

$$\langle g, x^+ - x^* \rangle \leq LD_h(x^*, x) - LD_h(x^*, x^+) - LD_h(x^+, x).$$

Next split $x^+ - x = (x^+ - x^*) + (x^* - x)$ in the upper smoothness bound:

$$\begin{aligned} f(x^+) &\leq f(x) + \langle g, x^* - x \rangle + \langle g, x^+ - x^* \rangle + LD_h(x^+, x) \\ &\leq f(x) + \langle g, x^* - x \rangle + LD_h(x^*, x) - LD_h(x^*, x^+). \end{aligned}$$

The two copies of $LD_h(x^+, x)$ cancel. Relative strong convexity supplies the remaining comparison, $f(x) + \langle g, x^* - x \rangle \leq f(x^*) - \mu D_h(x^*, x)$. Substitution and rearrangement give

$$f(x^+) - f(x^*) + LD_h(x^*, x^+) \leq (L - \mu)D_h(x^*, x).$$

Discard the nonnegative objective gap and divide by L to obtain the contraction. $\square$

Notice what contracts: it is $D_h(x^*, x_t)$, not necessarily the objective gap by that same factor at each step. The stronger displayed inequality controls the next objective gap as well. A useful reference function must make the curvature approximate that of f and make its proximal problem inexpensive.

16.5 Mirror Descent, Entropy, and Multiplicative Weights

Suppose the linear objective changes each round. We choose $x_t \in K$, then observe a loss vector g_t and incur $\langle g_t, x_t \rangle$. There is no single objective value to decrease. Instead, compare the cumulative loss with that of any fixed $u \in K$.

Mirror descent uses the same proximal step as relative gradient descent, $x_{t+1} = \text{Prox}_{x_t}^h(\eta g_t)$. The proximal inequality controls the loss at x_{t+1} , which has already seen g_t , whereas the actual loss was incurred at x_t . The analysis consists of controlling this one-step delay.

Algorithm 60 Mirror descent

Input: Reference function h , initial point $x_0 \in K$, step size $\eta > 0$, iteration count T

```

1: for  $t = 0, \dots, T - 1$  do
2:   Play  $x_t$  and receive the loss vector  $g_t$ 
3:    $x_{t+1} \leftarrow \text{Prox}_{x_t}^h(\eta g_t)$ 

```

Mirror descent can also be viewed as gradient descent in the dual coordinates defined by a reference function h . In the unconstrained case,

$$\nabla h(x_{t+1}) = \nabla h(x_t) - \eta g_t,$$

so

$$x_{t+1} = \nabla h^*(\nabla h(x_t) - \eta g_t).$$

For a convex constraint set K , the equivalent proximal form is

$$x_{t+1} = \arg \min_{x \in K} \{ \eta \langle g_t, x \rangle + D_h(x, x_t) \}.$$

Indeed, when $K = \mathbb{R}^n$, its first-order condition is exactly $\nabla h(x_{t+1}) = \nabla h(x_t) - \eta g_t$.

Theorem 329 (Mirror descent). *Suppose $D_h(y, x) \geq (\sigma/2)\|y - x\|^2$ for $\sigma > 0$. Then every comparator $u \in K$ satisfies*

$$\sum_{t < T} \langle g_t, x_t - u \rangle \leq \frac{D_h(u, x_0)}{\eta} + \frac{\eta}{2\sigma} \sum_{t < T} \|g_t\|_*^2.$$

Proof. Split the loss into $\langle g_t, x_{t+1} - u \rangle + \langle g_t, x_t - x_{t+1} \rangle$. The proximal inequality bounds the first term by

$$\frac{1}{\eta} (D_h(u, x_t) - D_h(u, x_{t+1}) - D_h(x_{t+1}, x_t)).$$

Duality, Young's inequality, and strong convexity bound the second by

$$\begin{aligned} \langle g_t, x_t - x_{t+1} \rangle &\leq \|g_t\|_* \|x_t - x_{t+1}\| \\ &\leq \frac{\eta}{2\sigma} \|g_t\|_*^2 + \frac{\sigma}{2\eta} \|x_t - x_{t+1}\|^2 \leq \frac{\eta}{2\sigma} \|g_t\|_*^2 + \frac{1}{\eta} D_h(x_{t+1}, x_t). \end{aligned}$$

Adding the two bounds cancels the movement divergence and gives the one-step inequality

$$\langle g_t, x_t - u \rangle \leq \frac{D_h(u, x_t) - D_h(u, x_{t+1})}{\eta} + \frac{\eta}{2\sigma} \|g_t\|_*^2.$$

After summing, every intermediate $D_h(u, x_t)$ occurs once with each sign. Hence the sum of losses is at most $[D_h(u, x_0) - D_h(u, x_T)]/\eta + (\eta/(2\sigma)) \sum_{t < T} \|g_t\|_*^2$. Discard $-D_h(u, x_T)/\eta \leq 0$. $\square$

We impose few constraints on The loss vectors. Strong convexity is imposed on the reference function: it makes a large response expensive and thereby limits the loss from having acted before seeing g_t . The two terms in the regret bound have the same roles as in subgradient descent: initial divergence and accumulated movement error.

16.5.1 Why Entropy Fits the Simplex

On $\Delta_n = \{p \geq 0 : \mathbf{1}^\top p = 1\}$, take negative entropy $h(p) = \sum_i p_i \log p_i$. For $p_i > 0$ and arbitrary $q \in \Delta_n$,

$$D_h(q, p) = \sum_i q_i \log \frac{q_i}{p_i}.$$

Indeed, expansion gives

$$\begin{aligned} D_h(q, p) &= \sum_i (q_i \log q_i - p_i \log p_i - (1 + \log p_i)(q_i - p_i)) \\ &= \sum_i q_i \log(q_i/p_i) - \sum_i q_i + \sum_i p_i = \sum_i q_i \log(q_i/p_i). \end{aligned}$$

The last two sums cancel because both points have total mass one. Terms with $q_i = 0$ are interpreted by continuity.

To choose a regularizer, inspect both terms of the regret bound. For coordinatewise bounded losses, we want the dual norm to be ℓ_∞ , so strong convexity should hold in ℓ_1 . We also want a small initial divergence to every comparator. Entropy meets both requirements: at the uniform point p_0 , $D_h(q, p_0) = \log n + \sum_i q_i \log q_i \leq \log n$, and in the interior,

$$z^\top \nabla^2 h(p) z = \sum_i \frac{z_i^2}{p_i} \geq \frac{(\sum_i |z_i|)^2}{\sum_i p_i} = \|z\|_1^2.$$

Integrating along the segment gives $D_h(q, p) \geq \frac{1}{2} \|q - p\|_1^2$, with boundary comparators obtained by continuity. The regret bound therefore depends on $\log n$ and $\|g_t\|_\infty^2$, rather than paying the dimension factor in $\|g_t\|_2^2$. This is the geometric reason to use entropy; the convenient closed-form update is the computational reason.

16.5.2 The Multiplicative Update

For an interior minimizer, introduce a multiplier λ for $\sum_i q_i = 1$. Differentiating $\eta\langle g, q \rangle + \sum_i q_i \log(q_i/p_i) + \lambda(\sum_i q_i - 1)$ gives $\eta g_i + \log q_i - \log p_i + 1 + \lambda = 0$. Thus $q_i = e^{-1-\lambda} p_i e^{-\eta g_i}$, and normalization determines the common factor:

$$p_i^+ = \frac{p_i e^{-\eta g_i}}{\sum_j p_j e^{-\eta g_j}}.$$

This formula is positive, has total mass one, and satisfies the first-order condition; convexity therefore proves it is the minimizer. The update is multiplicative because the reference gradient is logarithmic.

We will reuse the entropy proximal step below. Write $\text{EntropyStep}(p, v, \beta)$ for the minimizer of $\langle v, q \rangle + \beta D_h(q, p)$; all coordinates of p are positive and $\beta > 0$.

Algorithm 61 Entropy proximal step and multiplicative weights

```

1: function ENTROPYSTEP( $p, v, \beta$ )
2:    $s_i \leftarrow \log p_i - v_i/\beta$  for every  $i$ 
3:    $s \leftarrow s - (\max_i s_i) \mathbf{1}$ 
4:   return  $\exp(s)/(\mathbf{1}^\top \exp(s))$ 

5: procedure MULTIPLICATIVEWEIGHTS( $n, \eta, T$ )
6:    $p \leftarrow \mathbf{1}/n$ 
7:   for  $t = 0, \dots, T-1$  do
8:     Play  $p$  and receive  $g_t$ 
9:      $p \leftarrow \text{ENTROPYSTEP}(p, \eta g_t, 1)$ 
```

Subtracting the largest log-weight does not change the normalization and avoids unnecessarily large exponentials. In exact arithmetic, positive initial weights remain positive. With $p_0 = \mathbf{1}/n$, mirror descent gives regret at most $\log n/\eta + (\eta/2) \sum_{t < T} \|g_t\|_\infty^2$.

16.6 Matrix Multiplicative Weights from Entropy Geometry

Scalar multiplicative weights maintains a distribution over coordinates. Its matrix analogue maintains a density matrix

$$\mathcal{D}_n = \{X \in \mathbb{R}^{n \times n} : X = X^\top, X \succeq 0, \text{Tr } X = 1\}.$$

By the spectral theorem,

$$X = \sum_i p_i u_i u_i^\top$$

is a distribution over orthogonal directions. Against a symmetric loss matrix G ,

$$\langle G, X \rangle = \text{Tr}(GX) = \sum_i p_i u_i^\top G u_i.$$

Thus the matrix algorithm chooses both the weights and the directions on which those weights lie. Restricting all matrices to a common diagonal basis recovers ordinary multiplicative weights.

We use the trace inner product

$$\langle A, B \rangle = \text{Tr}(AB).$$

The corresponding dual norms are the trace norm and operator norm:

$$|\text{Tr}(GH)| \leq \|G\|_{\text{op}} \|H\|_{\text{tr}}.$$

Indeed, if

$$H = U \text{diag}(\lambda_i) U^\top,$$

then

$$\begin{aligned} |\operatorname{Tr}(GH)| &= \left| \sum_i \lambda_i (U^\top GU)_{ii} \right| \\ &\leq \|G\|_{\text{op}} \sum_i |\lambda_i| = \|G\|_{\text{op}} \|H\|_{\text{tr}}. \end{aligned}$$

Equality in the dual characterization is obtained by $G = \operatorname{sgn}(H)$.

16.6.1 Entropy Determines the Update

Take matrix negative entropy

$$h(X) = \operatorname{Tr}(X \log X),$$

where $0 \log 0 = 0$ on the boundary. For $X \succ 0$, we have

$$\boxed{\nabla h(X) = I + \log X.}$$

Hence, for $X \succ 0$ and $Y \in \mathcal{D}_n$,

$$\begin{aligned} D_h(Y, X) &= h(Y) - h(X) - \operatorname{Tr}((I + \log X)(Y - X)) \\ &= \operatorname{Tr}(Y \log Y) - \operatorname{Tr}(Y \log X), \end{aligned}$$

because

$$\operatorname{Tr}(Y - X) = 0.$$

Thus

$$\boxed{D_h(Y, X) = \operatorname{Tr}(Y(\log Y - \log X)).}$$

Mirror descent with loss matrix G takes the step

$$X^+ = \arg \min_{Y \in \mathcal{D}_n} \{\eta \operatorname{Tr}(GY) + D_h(Y, X)\}.$$

The minimizer has the explicit form

$$\boxed{X^+ = \frac{\exp(\log X - \eta G)}{\operatorname{Tr} \exp(\log X - \eta G)}. \quad (5)}$$

To verify this, put

$$Z = \operatorname{Tr} \exp(\log X - \eta G).$$

Then

$$\log X^+ = \log X - \eta G - (\log Z)I.$$

Since

$$\nabla_Y D_h(Y, X) = \nabla h(Y) - \nabla h(X) = \log Y - \log X,$$

the gradient of the proximal objective at X^+ is

$$\begin{aligned} \eta G + \log X^+ - \log X &= \eta G + \log X - \eta G - (\log Z)I - \log X \\ &= -(\log Z)I. \end{aligned}$$

Every feasible displacement $U - X^+$ has trace zero, so

$$\langle -(\log Z)I, U - X^+ \rangle = -(\log Z) \operatorname{Tr}(U - X^+) = 0.$$

The objective is convex, so X^+ is its global minimizer.

The crucial point is that the additive update happens in logarithmic coordinates. In general,

$$\exp(\log X - \eta G) \neq X e^{-\eta G},$$

because $\log X$ and G need not commute; indeed the product on the right need not even be symmetric.

Starting from

$$X_0 = \frac{I}{n},$$

write the update as

$$\log X_{t+1} = \log X_t - \eta G_t - c_t I,$$

where c_t is the logarithm of the normalizing constant. Iterating,

$$\log X_t = -\eta \sum_{s < t} G_s - c'_t I.$$

The scalar multiple of I disappears upon normalization, giving

$$X_t = \frac{\exp(-\eta S_t)}{\text{Tr} \exp(-\eta S_t)}, \quad S_t = \sum_{s < t} G_s.$$

Thus the algorithm only needs the cumulative loss matrix.

Algorithm 62 Matrix multiplicative weights

Input: Matrix dimension n , step size $\eta > 0$, number of rounds T

- 1: $S \leftarrow 0$
 - 2: **for** $t = 0, \dots, T-1$ **do**
 - 3: Compute $-\eta S = U \text{diag}(\lambda_1, \dots, \lambda_n) U^\top$
 - 4: $p_i \leftarrow e^{\lambda_i - \max_j \lambda_j}$ for every i
 - 5: $p \leftarrow p / (\sum_i p_i)$
 - 6: $X_t \leftarrow U \text{diag}(p) U^\top$
 - 7: Play X_t and receive the symmetric loss matrix G_t
 - 8: $S \leftarrow S + G_t$
-

16.6.2 Potential Analysis

We analyze via a proof like the standard MWU analysis; we could give a proof analogous to the preceding one, but it adds no generality and is more complicated.

We bound the same potential in two ways: its one-step change tracks the algorithm's loss, while its final value is at least the weight assigned to the best fixed direction. Golden-Thompson supplies the one-step comparison even when the loss matrices do not commute.

Theorem 330 (Matrix multiplicative weights). *Let $G_0, \dots, G_{T-1}$ be symmetric $n \times n$ loss matrices, and let $\eta > 0$ satisfy $\eta \|G_t\|_{\text{op}} \leq 1$ for every t . Then the iterates of [Algorithm 62](#) satisfy, for every $U \in \mathcal{D}_n$,*

$$\sum_{t < T} \text{Tr}(G_t(X_t - U)) \leq \frac{\log n}{\eta} + \eta \sum_{t < T} \text{Tr}(X_t G_t^2).$$

In particular, if $\|G_t\|_{\text{op}} \leq G$ for every t , then

$$\sum_{t < T} \text{Tr}(G_t(X_t - U)) \leq \frac{\log n}{\eta} + \eta T G^2.$$

For $n \geq 2$, $G > 0$, and $T \geq \log n$, choosing $\eta = \sqrt{\log n / (TG^2)}$ gives

$$\sum_{t < T} \text{Tr}(G_t(X_t - U)) \leq 2G\sqrt{T \log n}.$$

Proof. Put

$$S_t = \sum_{s < t} G_s, \quad W_t = e^{-\eta S_t}, \quad Z_t = \text{Tr } W_t, \quad X_t = \frac{W_t}{Z_t}.$$

The potential is Z_t , with $Z_0 = \text{Tr } I = n$.

First control its one-step change. For $|x| \leq 1$, the exponential series gives

$$e^{-x} \leq 1 - x + \sum_{k \geq 2} \frac{|x|^k}{k!} \leq 1 - x + x^2 \sum_{k \geq 2} \frac{1}{k!} \leq 1 - x + x^2.$$

Since every eigenvalue of ηG_t lies in $[-1, 1]$, applying this inequality in an eigenbasis of G_t gives $e^{-\eta G_t} \preceq I - \eta G_t + \eta^2 G_t^2$. Golden-Thompson therefore yields

$$\begin{aligned} Z_{t+1} &= \text{Tr } e^{-\eta(S_t + G_t)} \\ &\leq \text{Tr}(e^{-\eta S_t} e^{-\eta G_t}) \\ &\leq \text{Tr}(W_t(I - \eta G_t + \eta^2 G_t^2)) \\ &= Z_t(1 - \eta \text{Tr}(X_t G_t) + \eta^2 \text{Tr}(X_t G_t^2)). \end{aligned}$$

The second inequality uses $W_t \succeq 0$: taking the trace against a positive semidefinite matrix preserves the positive semidefinite order. Taking logarithms and using $\log(1+x) \leq x$ gives

$$\log Z_{t+1} - \log Z_t \leq -\eta \text{Tr}(X_t G_t) + \eta^2 \text{Tr}(X_t G_t^2).$$

Summing makes the intermediate log-potentials cancel:

$$\log Z_T \leq \log n - \eta \sum_{t < T} \text{Tr}(X_t G_t) + \eta^2 \sum_{t < T} \text{Tr}(X_t G_t^2).$$

For the opposite bound, the trace contains the exponential weight of every eigendirection of S_T , including one with minimum eigenvalue:

$$Z_T = \sum_{i=1}^n e^{-\eta \lambda_i(S_T)} \geq e^{-\eta \lambda_{\min}(S_T)}.$$

Thus $\log Z_T \geq -\eta \lambda_{\min}(S_T)$. Comparing the two bounds and dividing by η gives

$$\sum_{t < T} \text{Tr}(X_t G_t) - \lambda_{\min}(S_T) \leq \frac{\log n}{\eta} + \eta \sum_{t < T} \text{Tr}(X_t G_t^2).$$

Finally,

$$\lambda_{\min}(S_T) = \min_{U \in \mathcal{D}_n} \text{Tr}(US_T).$$

Indeed, $S_T \succeq \lambda_{\min}(S_T)I$ gives the lower bound for every density matrix U , and equality holds at $U = uu^\top$ for a unit eigenvector u of minimum eigenvalue. Since $\text{Tr}(US_T) = \sum_{t < T} \text{Tr}(UG_t)$, this proves the first assertion.

If $\|G_t\|_{\text{op}} \leq G$, then $G_t^2 \preceq G^2 I$, so

$$\text{Tr}(X_t G_t^2) \leq G^2 \text{Tr } X_t = G^2.$$

Substitution gives $\log n / \eta + \eta T G^2$. The stated step size satisfies $\eta G = \sqrt{\log n / T} \leq 1$ and makes both terms equal to $G\sqrt{T \log n}$. $\square$

16.7 Under Construction

I will finish this section eventually.